

Quantum Computing - Notes - v1.0.0

260236

September 2026

Preface

Every theory section in these notes has been taken from the sources:

- Course slides. [3]
- Bryan Eastin and Emanuel Knill. Restrictions on transversal encoded quantum gate sets. *Physical review letters*, 102(11):110502, 2009. [1]
- Albert Einstein, Boris Podolsky, and Nathan Rosen. Can quantum-mechanical description of physical reality be considered complete? *Physical review*, 47(10):777, 1935. [2]

About:

 [GitHub repository](#)



These notes are an unofficial resource and shouldn't replace the course material or any other book on quantum computing. It is not made for commercial purposes. I've made the following notes to help me improve my knowledge and maybe it can be helpful for everyone.

As I have highlighted, a student should choose the teacher's material or a book on the topic. These notes can only be a helpful material.

Contents

1	Introduction	8
1.1	Complex Numbers	8
1.2	Dirac's Notation	12
2	Single Qubit States	14
2.1	Building and Measuring Qubits (Intuition)	14
2.2	From Polarization to Qubits	18
2.3	Single Qubit Measurement	23
2.4	Superposition	27
2.5	Information Carried by a Single Qubit	34
2.6	State Space of a Single-Qubit System	36
3	Single Qubit Gates	42
3.1	Operations on Qubits	42
3.2	Quantum Logic Gates Overview	46
3.3	Main Single-Qubit Gates	49
3.3.1	Identity Gate (I)	49
3.3.2	Pauli Gates	50
3.3.2.1	Rotations Generated by Pauli Operators	50
3.3.2.2	Pauli-X (NOT) Gate	51
3.3.2.3	Pauli-Z (Phase Flip) Gate	56
3.3.2.4	Pauli-Y Gate	60
3.3.3	Phase Gate (S)	63
3.3.4	Hadamard Gate (H)	65
3.4	Properties	71
3.5	When Does a Gate Create Superposition?	73
3.6	Single-Qubit Quantum Circuits	76
3.7	Outer Product of Kets	78
3.8	Measurement	80
4	Multiple Qubit Gates	86
4.1	Multiple Qubit States	86
4.1.1	Tensor Product	88
4.1.2	Building a Two-Qubit System	92
4.1.3	Separable vs Entangled States	94
4.1.4	Why Entanglement Matters	95
4.1.5	Exercise: Normalization of Tensor Product States	96
4.2	Introduction to Multiple Qubit Gates	97
4.3	Parallel Gates	98
4.4	Hadamard Transform on Multiple Qubits	102
4.5	Main Multi-Qubit Gates	103
4.5.1	Controlled NOT (CNOT) Gate	103
4.5.2	Generic Controlled Gate (CU)	106
4.5.3	SWAP gate	109
4.5.4	Controlled-Controlled-NOT (CCNOT) Gate (Toffoli)	112
4.6	Quantum Circuit Representation	115
4.7	Entanglement	117
4.8	Observables and Measurements	126

4.8.1	Observables: What Are We Measuring?	126
4.8.2	Measurement Operators and Projectors	136
4.8.3	Expectation Value	142
4.8.4	Measuring $ +\rangle$ with the Pauli-Z Observable	144
4.8.5	Recovering Probabilities from Expectation Values	148
4.9	Heisenberg Representation of Quantum Circuits	151
4.10	Clifford Gates	154
4.10.1	Definition	154
4.10.2	Non-Clifford Gates	158
4.10.3	Stabilizer States	161
4.11	Universal set of quantum gates	164
4.12	Fault-Tolerant Quantum Gates	167
4.12.1	Logical vs Physical Qubits	168
4.12.2	Transversal Gates	169
4.12.3	Why Non-Clifford Gates Are Difficult	171
4.12.4	Physical vs Logical Gate Sets	173
4.13	Exercises	175
4.13.1	Single-Qubit States and Probabilities	175
4.13.2	Bloch sphere states	177
4.13.3	Tensor products and multi-qubit probabilities	179
4.13.4	Common factors, global phase and relative phase	181
4.13.5	Unitaries, Pauli Gates, and Eigenstates of Z, X, Y, H	187
4.13.6	Circuit notation and gate composition	191
4.14	Phase Kickback	197
4.14.1	Prerequisites: Eigenstates and Eigenvalues	197
4.14.2	Eigenvalues of Unitary Operators as Phase Factors	199
4.14.3	Controlled Unitary Operators	203
4.14.4	Definition and Derivation of Phase Kickback	206
4.14.5	Making the Kicked-Back Phase Observable	209
4.14.6	Repeated Controlled Operations	213
4.14.7	Phase Kickback with CNOT	216
4.14.8	Phase Kickback with Controlled Phase Gates	220
4.14.9	Phase Kickback with Controlled Hadamard	223
4.15	Deutsch's Algorithm	225
4.15.1	Problem Definition and the Four Possible Functions	225
4.15.2	Classical Solution and Query Complexity	228
4.15.3	Quantum Oracle U_f	230
4.15.4	Preparing the Input State	233
4.15.5	Oracle Action and Phase Kickback	238
4.15.6	Interference and Final Measurement	241
4.15.7	Quantum Advantage and Importance	246
4.16	Bernstein-Vazirani's Algorithm	248
4.16.1	Problem Definition and Classical Solution	248
4.16.2	Quantum Oracle Construction	251
4.16.3	Circuit and Algorithm Steps	256
4.16.4	Phase-Kickback Intuition	260
4.16.5	Complete Example for $w = 101$ (Exam Question)	266
4.16.6	General Mathematical Derivation	274
4.16.7	Complexity and Importance	278
4.17	Simon's Algorithm	281

4.17.1	Problem Definition and Classical Solution	281
4.17.2	Quantum Circuit and State Evolution	290
4.17.3	Why Simon's Algorithm Works	299
4.17.4	Recovering the Hidden Period	303
4.17.5	Examples	309
4.17.6	The Linear-Algebra Interpretation	316
4.17.7	Why Simon's Algorithm Is Important	321
4.17.8	Simon as a Fourier / Period-Finding Algorithm	325
5	Limits of Quantum Information	329
5.1	No-Cloning Principle	329
5.1.1	Why Cloning Matters	329
5.1.2	Formal Statement	333
5.2	No-Deleting Principle	338
5.3	No-Signaling Principle	340
5.4	EPR Paradox and Quantum Correlation	344
5.4.1	Definition of EPR Paradox	344
5.4.2	Entanglement and Quantum Correlation	346
5.4.3	Classical vs. Quantum Correlation	347
5.4.4	Entanglement vs. Quantum Correlation	351
5.5	Quantum Teleportation	352
5.5.1	Problem Definition and Required Resources	352
5.5.2	Shared Bell Pair and Initial Three-Qubit State	354
5.5.3	Alice's Operations and Mathematical Derivation	356
5.5.4	Measurement Outcomes and Bob's Corrections	359
5.5.5	Why Teleportation Works	361
6	Transpiling	364
6.1	Quantum Fidelity	364
6.1.1	Why Quantum Computers Are Noisy	364
6.1.2	Gate Infidelity	367
6.1.3	Decoherence	368
6.1.3.1	Phase-Flip Errors	368
6.1.3.2	Bit-Flip Errors	374
6.1.3.3	Generic Errors	376
6.1.4	Fidelity of a Quantum Gate	378
6.1.5	Fidelity and Decoherence	382
6.2	Quantum Error Correction	385
6.2.1	Quantum Computation Errors	385
6.2.2	Quantum Error Correction Codes (QECC)	387
6.2.3	Logical Gates	391
6.3	Quantum Transpiling	393
6.3.1	The Quantum Computing Stack	393
6.3.2	Placement	395
6.3.3	Scheduling	397
6.3.4	Routing	405

7	QUBO Formulation	406
7.1	QUBO Problems	406
7.1.1	Introduction to QUBO	406
7.1.2	QUBO as an Energy Minimization Problem	410
7.1.3	Boolean Logic in QUBO	413
7.2	Continuous Variables and Constraints	416
7.2.1	Continuous Variables through Binary Expansion	416
7.2.2	Constraints in QUBO	418
7.2.3	Penalty Terms	420
7.2.4	Penalty Coefficients	428
7.2.5	Equality Constraints in QUBO	429
7.3	From QUBO to Quantum Systems	431
7.3.1	Hamiltonian Recap	432
7.3.2	The Eigenspectrum	436
7.3.3	Why Map QUBO to Quantum Mechanics?	438
7.3.4	Encoding Bit Strings into Quantum States	439
7.3.5	Constructing the QUBO Hamiltonian	440
7.4	The Ising Model	443
7.4.1	QUBO, Hamiltonian, and Ising: Taxonomy	444
7.4.2	Classical Ising Model	446
7.4.3	Quantum Ising Model	450
7.5	Quantum Annealing	451
7.5.1	Annealing Hamiltonian	452
7.5.2	Evolution of the Eigenspectrum	457
7.6	Summary	459
7.7	QUBO Formulations	462
7.7.1	Max-Cut	463
7.7.2	Maximum Independent Set	463
7.7.3	Minimum Vertex Cover	464
7.7.4	Maximum Clique	464
7.7.5	Number Partitioning	465
7.7.6	2-SAT	465
7.7.7	Set Packing	466
7.7.8	Constraint Satisfaction Problem (CSP)	466
8	Variational Quantum Algorithms (VQAs)	467
8.1	Why Variational Algorithms?	467
8.2	General Structure of a VQA	470
8.2.1	Variational Quantum Algorithms	470
8.2.2	Parametric Quantum Circuits (Ansätze)	472
8.2.3	Classical Optimizer	474
8.2.4	Iterative Optimization Loop	475
8.3	Building Blocks of Variational Algorithms	477
8.3.1	Cost Function	477
8.3.2	Ansatz Design	478
8.3.3	Optimizer Choice	480
8.3.4	Hardware Efficient Ansätze	483
8.4	Advantages and Limitations	486
8.4.1	Strengths of Variational Algorithms	486
8.4.2	Weaknesses of Variational Algorithms	488

8.4.3	Barren Plateaus	490
8.4.4	Why Variational Algorithms Matter	497
8.5	Variational Quantum Eigensolver (VQE)	499
8.5.1	The Physical Motivation	499
8.5.2	Hamiltonians and Molecular Energy	500
8.5.3	Variational Principle	503
8.5.4	VQE Objective Function	505
8.6	Quantum Approximate Optimization Algorithm	508
8.6.1	Why QAOA?	508
8.6.2	QAOA Ingredients	510
8.6.3	Structure of the QAOA Ansatz	513
8.6.4	Unitaries from Hamiltonians	516
8.6.5	Cost Hamiltonian and Cost Operator	518
8.6.6	Mixer Hamiltonian and Mixer Operator	521
8.6.7	Alternating Operators and p Layers	523
8.6.8	QUBO Problems for QAOA	525
8.6.9	From QUBO to Cost Hamiltonian	528
8.6.10	Practical QAOA Circuit Construction	531
9	Exams	536
9.1	2024	536
9.1.1	June 20	536
	Simon's Algorithm	536
	Problem solved by Simon's algorithm	536
	Specific exam function	539
	Qubit States and Operations	545
	Single-Qubit state space	545
	Mathematical formalism of quantum gates	548
	No-cloning and no-deleting	552
	QUBO: SAT and Max-Cut problems	555
	SAT problem	555
	Max-Cut problem	564
	Index	577

1 Introduction

1.1 Complex Numbers

Let's not start from math. Let's start from **physics / computation intuition**. Classical bits are represented by two distinct states: 0 and 1. In quantum computing, we have **qubits**, which are essentially **vector of numbers**. But not just any numbers: a qubit is a vector of **complex numbers**. They are needed because quantum systems have **both magnitude and phase**, and complex numbers can represent both. Real numbers can only represent "*how much*" (magnitude), while complex numbers can represent "*how much*" and "*in which direction / phase*" (magnitude and phase).

🔗 Why complex (and not just real) numbers?

1. In quantum computing, we don't store probabilities directly (e.g., 0.5 for a 50% chance). Instead, we store **probability amplitudes**, which are complex numbers. The probability of an outcome is the square of the magnitude of its amplitude:

$$P = |z|^2 \quad (1)$$

2. **Interference.** Quantum states can **combine and cancel**. For example, if we have two states with amplitudes z_1 and z_2 , the combined amplitude is:

$$z_{\text{combined}} = z_1 + z_2$$

If z_1 and z_2 have the same magnitude (*how much*) but opposite phases (*in which direction*), they can cancel each other out, leading to zero probability for that outcome. This is a key feature of quantum mechanics that allows for phenomena like quantum interference, which is essential for quantum algorithms.

In general, complex numbers are needed because **quantum states behave like waves**, not like classical values. Waves have both amplitude and phase, and complex numbers are the natural way to represent them mathematically. This is why quantum mechanics and quantum computing rely heavily on complex numbers.

🔗 What do we actually need?

Not everything. Just a **minimal toolkit**. A complex number is:

$$z = x + iy$$

Where x is the **real part** and y is the **imaginary part**, and i is the **imaginary unit** satisfying $i^2 = -1$. From a geometric interpretation, we can think of z as a point in the 2D plane, where x is the horizontal coordinate (real axis) and y is the vertical coordinate (imaginary axis).

Polar form

Instead of coordinates (x, y) , we can also represent a complex number in **polar form**:

$$z = re^{i\varphi}$$

Where:

- $r = |z| = \sqrt{x^2 + y^2}$ is the **modulus** (or *magnitude*) of the complex number, representing its **distance from the origin in the complex plane**.
- $\varphi = \arg(z) = \tan^{-1}\left(\frac{y}{x}\right)$ is the **argument** (or *phase angle*), representing the **angle between the positive real axis and the line connecting the origin to the point z in the complex plane**.
- The **exponential form** $e^{i\varphi}$ means that we can represent the complex number as a point on the unit circle in the complex plane, where φ is the angle from the positive real axis. The modulus r then scales this point to its actual distance from the origin.

Using **Euler's formula**:

$$e^{i\varphi} = \cos \varphi + i \sin \varphi$$

We can rewrite z as:

$$z = r(\cos \varphi + i \sin \varphi)$$

This form is particularly useful in quantum computing because it makes it easy to understand how complex numbers can represent both magnitude and phase, which are essential for describing quantum states and their evolution.

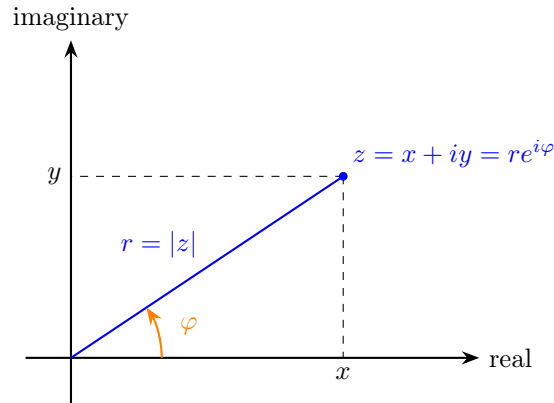


Figure 1: Geometric interpretation of a complex number in the complex plane. The arrow represents the complex number z as a vector from the origin to the point (x, y) . The length of the vector is the modulus r , and the angle it makes with the positive real axis is the argument φ .

Complex Conjugate

The **complex conjugate** of a complex number $z = x + iy$ is denoted as $\bar{z}$ and is defined as:

$$\bar{z} = x - iy$$

Geometrically, the complex conjugate $\bar{z}$ is the **reflection** of z **across the real axis in the complex plane**. This means that if z is represented by the point (x, y) , then $\bar{z}$ is represented by the point $(x, -y)$. The real part remains the same, while the imaginary part changes sign. See Figure 2 for a visual representation of this concept.

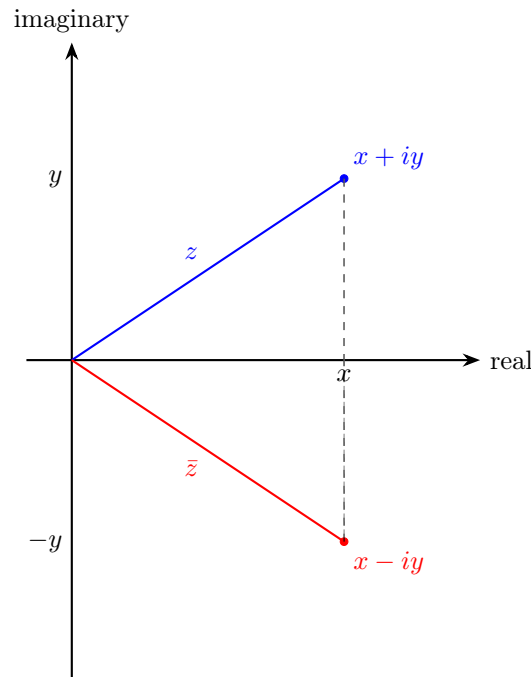


Figure 2: Geometric interpretation of the complex conjugate: reflection across the real axis.

Magnitude

The **magnitude** (or **modulus**) of a complex number $z = x + iy$ is denoted as $|z|$ and is defined as:

$$|z| = \sqrt{x^2 + y^2}$$

And it can also be expressed using the complex conjugate:

$$|z|^2 = z \cdot \bar{z} = (x + iy)(x - iy) = x^2 + y^2$$

The magnitude represents the **distance** of the point z from the origin in the complex plane. It is **always a non-negative real number**. The magnitude is important in quantum mechanics because the probability of an outcome is given by the square of the magnitude of the corresponding probability amplitude. For

example, if a quantum state has an amplitude of z , the probability of measuring that state is $|z|^2$. This is why the magnitude of complex numbers is crucial in quantum computing, as it directly relates to the probabilities of different outcomes when measuring quantum states.

↻ Rotation in the complex plane

Multiplying a complex number by a complex exponential of the form $e^{i\theta}$ corresponds to a **rotation** in the complex plane:

$$z' = z \cdot e^{i\theta} = r e^{i(\varphi+\theta)}$$

Geometrically, this operation:

- **Preserves the magnitude** $|z|$
- **Adds θ to the phase φ**

Therefore, multiplication by $e^{i\theta}$ rotates the complex number by an angle θ around the origin.

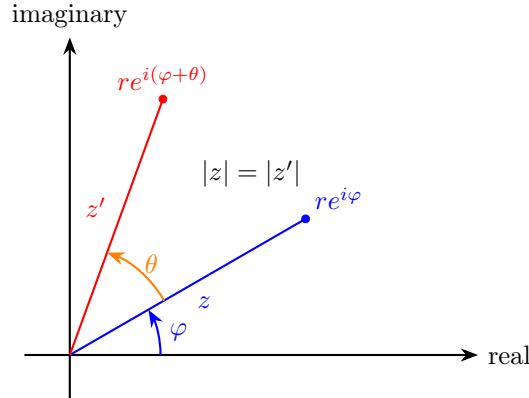


Figure 3: Rotation of a complex number: multiplying by $e^{i\theta}$ rotates the vector by angle θ while preserving its magnitude.

↔ Hermitian (Conjugate Transpose)

Given a complex vector:

$$\mathbf{v} = \begin{bmatrix} a \\ b \end{bmatrix}$$

Its **Hermitian** (or **conjugate transpose**) is defined as:

$$\mathbf{v}^\dagger = [\bar{a} \quad \bar{b}]$$

This operation consists of:

- Taking the **transpose** (column $\rightarrow$ row)
- Taking the **complex conjugate** of each element

1.2 Dirac's Notation

Dirac notation was introduced to make **linear algebra with complex vectors easier, cleaner, and more expressive**, especially for quantum systems.

Let's imagine our work only with standard linear algebra notation, where we have to write out all the vectors and matrices in their full form:

- Vectors:

$$\mathbf{v} = \begin{bmatrix} a \\ b \end{bmatrix}$$

- Conjugate transpose:

$$\mathbf{v}^\dagger = [\bar{a} \quad \bar{b}]$$

- Inner product:

$$\mathbf{v}^\dagger \mathbf{w} = \bar{a}c + \bar{b}d$$

- Matrix-vector multiplication:

$$M\mathbf{v} = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} m_{11}a + m_{12}b \\ m_{21}a + m_{22}b \end{bmatrix}$$

This quickly becomes **cumbersome and hard to read**, especially as we deal with more complex quantum systems. In quantum mechanics we constantly write expressions involving inner products, matrix operations, and state transformations.

✔ **Dirac's idea.** Paul Dirac said “*let's give different roles **different symbols***”. So he introduced:

- **Kets** $|v\rangle$ to represent **column vectors** (quantum states).
- **Bras** $\langle v|$ to represent row vectors (**conjugate transposes**).

Where the operators stay in the middle, and we can write everything in a much cleaner way:

- Vectors (**kets**):

$$|v\rangle = \begin{bmatrix} a \\ b \end{bmatrix} \tag{2}$$

Where the entries are complex numbers and usually we **require** $|v\rangle$ to have unit norm (normalized state).

- Conjugate transpose (**bra**):

$$\langle v| = [\bar{a} \quad \bar{b}] \tag{3}$$

- Inner product:

$$\langle v|w\rangle = \bar{a}c + \bar{b}d \tag{4}$$

The result is a scalar (complex number) that tells us **how much the two states “overlap”**. If $\langle v|w\rangle = 1$ then $|v\rangle$ and $|w\rangle$ are **identical** (up to a global phase, they overlap completely). If $\langle v|w\rangle = 0$ then $|v\rangle$ and

$|w\rangle$ are **orthogonal** (completely different states, no overlap). Otherwise, if $|\langle v|w\rangle| < 1$ then $|v\rangle$ and $|w\rangle$ are partially **similar** (some overlap, but not identical). So the **inner product is basically a notion of “distance” or “similarity” between two quantum states.**

- Matrix-vector multiplication:

$$M|v\rangle = \begin{bmatrix} m_{11} & m_{12} \\ m_{21} & m_{22} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} m_{11}a + m_{12}b \\ m_{21}a + m_{22}b \end{bmatrix} \quad (5)$$

An operator M acting on a state $|v\rangle$ gives us a new state. The result is a new ket.

Even if we ignore physics for now, we can see that Dirac's notation is more expressive and easier to read than standard linear algebra notation.

❓ What is *Chaining* in Dirac notation?

Chaining means writing multiple operations together in a single expression without needing to write out intermediate steps. For example:

$$\langle x|M|y\rangle$$

It is just a **compact way to combine linear algebra operations** (matrix multiplication and inner products) in a single expression. It allows us to write complex quantum mechanical expressions in a much cleaner and more intuitive way.

Without Dirac notation, we would have to write:

$$\mathbf{x}^\dagger M \mathbf{y}$$

That is a **chain of operations**: take the conjugate transpose of $\mathbf{x}$, multiply it by M , and then multiply the result by $\mathbf{y}$. With Dirac notation, we can write this in a much more compact and readable way as $\langle x|M|y\rangle$.

To read $\langle x|M|y\rangle$, we can think of it as:

1. Start with the ket $|y\rangle$ (a quantum state).
2. Apply the operator M to $|y\rangle$, which gives us a new state $M|y\rangle$.
3. Take the inner product of $\langle x|$ with the new state $M|y\rangle$, which gives us a scalar (complex number) that represents the probability amplitude of transitioning from state $|y\rangle$ to state $|x\rangle$ via the operator M . At first, this might seem abstract, but as we go through more examples and applications in quantum mechanics and quantum computing, it will become much clearer how powerful and intuitive Dirac's notation is for describing quantum systems.

2 Single Qubit States

Before introducing qubits, we briefly describe what we mean by a **quantum system**.

A **Quantum System** is a **physical system** (such as a photon, an electron, or an atom) **whose behavior is described by the laws of quantum mechanics**.

In this course, we will focus on the simplest case: systems that can produce **two possible outcomes** when measured. These are called **two-level systems**, and they form the basis of qubits.

2.1 Building and Measuring Qubits (Intuition)

Before introducing the mathematical formalism of qubits, it is useful to develop an **intuitive understanding** of how a simple quantum system behaves when it is **prepared** and **measured**.

🔍 A Simple Quantum Experiment

Consider a very simple experimental setup:

- A **source** emits a quantum particle (for example, a photon).
- The particle passes through a device.
- A **detector** measures the outcome.

Each time we perform the experiment, the detector returns one of two possible results:

$$0 \quad \text{or} \quad 1$$

This motivates the idea of a **two-level system**, which is the simplest type of quantum system.

💡 Key Observation

If we repeat the same experiment many times under identical conditions, we observe that:

- The outcome is **not deterministic**.
- Instead, it is **probabilistic**.

For example, we might obtain:

- Outcome 0 with probability p
- Outcome 1 with probability $1 - p$

This is fundamentally different from classical systems, where the outcome is determined in advance.

❓ What is a Measurement?

A **Measurement** is the process that:

- Extracts **classical information** from a quantum system;
- Produces a **definite outcome** (e.g., 0 or 1).

An important property of quantum measurement is that it **affects the system itself**. After a measurement is performed, the **system is no longer in its original configuration**.

💡 From Intuition to Formalization

This simple experiment suggests that:

- A quantum system can produce **two possible outcomes**
- The outcomes are governed by **probabilities**
- The system must therefore contain some form of **internal state** that determines these probabilities

In the following example, we will introduce a concrete physical example (photon polarization) that allows us to represent this internal state using **vectors with complex coefficients**.

✏ Example 1: Example of a Two-Level Quantum System

To make the previous discussion more concrete, we introduce a simple physical system that behaves as a two-level quantum system: the **polarization of a photon**.

❓ What is a Photon?

A **Photon** is the elementary particle of light. It carries energy and behaves both as a particle and as a wave. In this course, we will use photons as a simple example of a **quantum system**, since their properties can be easily controlled and measured.

❓ What is Polarization?

Polarization describes the direction in which the electric field of a light wave oscillates. In simple terms, it can be thought of as the **orientation** of the light wave. For a photon, polarization can be visualized as a direction in space. In particular, we consider two basic polarization directions:

- **horizontal** ($\rightarrow$)
- **vertical** ($\uparrow$)

These two directions form a **two-level system**, since a measurement can distinguish between them (i.e., we can determine whether the photon is horizontally or vertically polarized).

💡 Superposition of Polarizations

A photon is not restricted to being only horizontal or vertical. In general, it can be in a **combination** of both states:

$$v = a \rightarrow + b \uparrow$$

Where:

- a and b are **complex coefficients**;
- They determine the behavior of the system when measured.

This is the first example of a **quantum state**: a system described by a linear combination of basic states.

❓ Vector Representation

We can represent these polarization states using vectors:

$$\rightarrow = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \uparrow = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

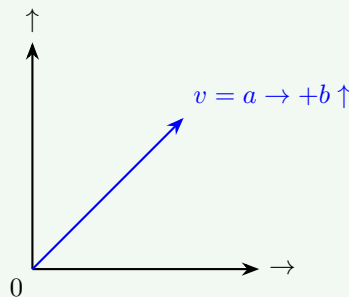
Therefore, a general polarization state becomes:

$$v = \begin{bmatrix} a \\ b \end{bmatrix}$$

This shows that the state of the system can be described as a **vector with complex components**, exactly as introduced in Dirac notation.

📐 Geometric Interpretation

The horizontal and vertical polarization states can be interpreted as two **orthogonal axes** in a two-dimensional space.



The point where the axes intersect represents the origin, and any polarization state can be represented as a **vector starting from the origin**.



In this representation:

- $\rightarrow$ and $\uparrow$ form a **basis**
- Any state is a **linear combination** of these basis states

💡 Measurement with Polarizers

A **Polarizer** is a device that **filters light based on its polarization**. A polarizer acts like a **filter** that allows only one polarization direction to pass and blocks the other. For example:

- A polarizer aligned with the horizontal direction ($\rightarrow$) will allow horizontally polarized light to pass and block vertically polarized light.
- A polarizer aligned with the vertical direction ($\uparrow$) will allow vertically polarized light to pass and block horizontally polarized light.

If the photon is a combination:

$$v = a \rightarrow + b \uparrow$$

And we use a:

- Horizontal polarizer ($\rightarrow$), the photon will pass with probability $|a|^2$ and be absorbed with probability $|b|^2$.
- Vertical polarizer ($\uparrow$), the photon will pass with probability $|b|^2$ and be absorbed with probability $|a|^2$.

This shows that:

- Probabilities are determined by the amplitudes (i.e., the coefficients a and b).
- Measurement **modifies the state of the system** (e.g., if the photon passes through the horizontal polarizer, it will be in the state $\rightarrow$).

2.2 From Polarization to Qubits

The example of photon polarization (page 15) shows that a physical system can be described using two basic states and their linear combinations. We now **abstract** this idea and introduce a **general model that does not depend on a specific physical system**.

🔗 From Physical States to Abstract States

In the polarization example, we used:

$$\rightarrow \quad \text{and} \quad \uparrow$$

As the two basic states of the system. We now replace them with an abstract notation:

$$\rightarrow \rightsquigarrow |0\rangle \quad \uparrow \rightsquigarrow |1\rangle$$

These are called the **computational basis states**. They are just symbols (used in Dirac notation) that represent the two basic states of the system, without referring to any specific physical implementation.

📖 Definition of a Qubit

A **Qubit** is a **two-level quantum system** whose state can be written as:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

It is the **unit of quantum information**, analogous to a classical bit. However, unlike a classical bit that can only be in one of two states (0 or 1), a qubit can exist in **infinitely many possible states**. Indeed, any normalized vector of the form:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

Represents a **valid state** of a qubit. Some **properties of the qubit state** are:

- The coefficients a and b are **complex numbers** (probability amplitudes). They determine the outcome of a measurement:

- $|a|^2$ is the probability of measuring the state $|0\rangle$.
- $|b|^2$ is the probability of measuring the state $|1\rangle$.

Therefore, even though a qubit can be in infinitely many possible states, a measurement (in this basis) can only produce **two possible outcomes**: $|0\rangle$ or $|1\rangle$.

- The state is **normalized**:

$$|a|^2 + |b|^2 = 1$$

Because the probabilities must sum to 1 (i.e., we must get either $|0\rangle$ or $|1\rangle$ when we measure).

Definition 1: Normalized Quantum State

A quantum state $|\psi\rangle$ is said to be **normalized** if its norm is equal to one:

$$\| |\psi\rangle \| = 1$$

Equivalently, using the inner product:

$$\langle \psi | \psi \rangle = 1$$

For a single-qubit state:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

This condition becomes:

$$|a|^2 + |b|^2 = 1 \quad (6)$$

This is required because $|a|^2$ and $|b|^2$ represent measurement probabilities, and the total probability of all possible outcomes must be equal to 1.

- The states $|0\rangle$ and $|1\rangle$ form a **basis** of the vector space describing the qubit.

What is a basis? A **Basis** is a set of vectors that allows us to represent any vector in the space as a linear combination of them. In quantum computing, $|0\rangle$ and $|1\rangle$ are the **basis vectors** and any qubit state is built from them.

In particular, the states $|0\rangle$ and $|1\rangle$ form an **Orthonormal Basis**, meaning that:

- They are **orthogonal**: $\langle 0 | 1 \rangle = 0$.

Why is orthogonality important? Orthogonality means that the two basis states are completely distinguishable by a measurement (not overlapping). If we measure a qubit in the state $|0\rangle$, we will never get $|1\rangle$, and vice versa.

- They are **normalized**: $\langle 0 | 0 \rangle = \langle 1 | 1 \rangle = 1$.

Why is normalization important? Normalization ensures that the probabilities of measurement outcomes are well-defined and sum to 1. It also means that the basis states have unit length in the vector space, which is a fundamental property for quantum states.

🔍 Why This Abstraction Matters

This abstraction is fundamental because it allows us to:

- Describe **any** two-level quantum system (not only photons);
- Use a unified mathematical framework based on vectors;
- Apply the tools of linear algebra to quantum systems.

From now on, we will work with qubits using Dirac notation, without referring to a specific physical implementation.

✎ Example 2: Different Choices of Basis

The computational basis $\{|0\rangle, |1\rangle\}$ is not the only possible choice. A qubit can be described using **any orthonormal basis**.

For example, another important basis is the **Hadamard basis**, defined as:

$$|+\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad |-\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

This shows that the same quantum state can be represented in different bases, depending on the context.

✎ Example 3: Orthogonality of the Hadamard Basis

Show that the Hadamard basis states $|+\rangle$ and $|-\rangle$ are orthogonal.

Proof. Recall:

$$|+\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad |-\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Orthogonal means that their inner product is zero. So, we need to compute $\langle + | - \rangle$ and show that it equals zero:

$$\langle + | - \rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2} (1 - 1) = 0$$

Therefore, $|+\rangle$ and $|-\rangle$ are orthogonal.

QED

✎ Example 4: Change of Basis from Standard $\leftrightarrow$ Hadamard

Given a qubit state in the standard basis:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

We want to express it in the Hadamard basis:

$$|\psi\rangle = x|+\rangle + y|-\rangle$$

Proof. Using the definitions of $|+\rangle$ and $|-\rangle$:

$$|+\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad |-\rangle = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

We can write $|\psi\rangle$ in terms of $|+\rangle$ and $|-\rangle$:

$$|\psi\rangle = a|0\rangle + b|1\rangle = a \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} + b \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} a \\ b \end{bmatrix}$$

Now, we express $|\psi\rangle$ as a linear combination of $|+\rangle$ and $|-\rangle$:

$$|\psi\rangle = x|+\rangle + y|-\rangle$$

Substituting the definitions of $|+\rangle$ and $|-\rangle$:

$$= x \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + y \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Combining the terms:

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} x+y \\ x-y \end{bmatrix}$$

We want this to equal $\begin{bmatrix} a \\ b \end{bmatrix}$, so we set:

$$\frac{1}{\sqrt{2}} \begin{bmatrix} x+y \\ x-y \end{bmatrix} = \begin{bmatrix} a \\ b \end{bmatrix}$$

This gives us the system of equations:

$$x+y = \sqrt{2}a, \quad x-y = \sqrt{2}b$$

Adding the two equations:

$$2x = \sqrt{2}(a+b) \implies x = \frac{a+b}{\sqrt{2}}$$

Subtracting the second from the first:

$$2y = \sqrt{2}(a-b) \implies y = \frac{a-b}{\sqrt{2}}$$

It follows that:

$$|\psi\rangle = x|+\rangle + y|-\rangle = \frac{a+b}{\sqrt{2}}|+\rangle + \frac{a-b}{\sqrt{2}}|-\rangle$$

QED

Example 5: Orthogonal State Construction

Given:

$$|v\rangle = a|0\rangle + b|1\rangle$$

Define:

$$|w\rangle = \bar{b}|0\rangle - \bar{a}|1\rangle$$

Show that $|v\rangle$ and $|w\rangle$ are orthogonal.

Proof. To show that $|v\rangle$ and $|w\rangle$ are orthogonal, we need to compute their inner product $\langle w|v\rangle$ and show that it equals zero. First, we write the inner product converting $|w\rangle$ to its corresponding bra:

$$|w\rangle = \bar{b}|0\rangle - \bar{a}|1\rangle \implies \langle w| = b\langle 0| - a\langle 1|$$

Now we compute:

$$\langle w|v\rangle = (b\langle 0| - a\langle 1|) \cdot (a|0\rangle + b|1\rangle)$$

Expanding this expression:

$$= ba\langle 0|0\rangle + bb\langle 0|1\rangle - aa\langle 1|0\rangle - ab\langle 1|1\rangle$$

Using the orthonormality of the basis states:

$$\langle 0|0\rangle = \langle 1|1\rangle = 1, \quad \langle 0|1\rangle = \langle 1|0\rangle = 0$$

We get:

$$\langle w|v\rangle = \cancel{ba\langle 0|0\rangle} + \cancel{bb\langle 0|1\rangle} - \cancel{aa\langle 1|0\rangle} - \cancel{ab\langle 1|1\rangle} \overset{1}{=} ba - ab$$

Since a and b are complex numbers, multiplication is commutative, so $ba = ab$. Therefore:

$$\langle w|v\rangle = ba - ab = 0$$

Hence, $|v\rangle$ and $|w\rangle$ are orthogonal.

QED

2.3 Single Qubit Measurement

After introducing the mathematical representation of a qubit, we now describe how information is extracted from it through **measurement**.

🔍 What is *Single Qubit Measurement*?

We have a qubit:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

Where a and b are complex numbers such that $|a|^2 + |b|^2 = 1$. Measurement answers the question: “*which classical value do we observe?*”.

A measurement takes a **quantum state** as input and produces a **classical outcome** as output. In the case of a single qubit, the possible outcomes are 0 and 1.

⚠️ Measurement rule (computational basis)

The probability of an outcome is the **squared magnitude of the coefficient of the basis vector**:

$$P(0) = |a|^2, \quad P(1) = |b|^2 \quad (\text{in the computational basis})$$

But which coefficients? This is where the **basis** comes in. A state can be written in different ways depending on the basis. For example, take:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

- If we measure in basis $\{|0\rangle, |1\rangle\}$:
 - Outcome 0 with probability $|a|^2$
 - Outcome 1 with probability $|b|^2$
- If we measure in basis $\{|+\rangle, |-\rangle\}$:
 - Outcome + with probability $|x|^2$
 - Outcome – with probability $|y|^2$

The **coefficients depend on the chosen basis**. Let’s analyze how a basis change affects the coefficients.

🔍 **What is a basis *really*?** Any measurement device (e.g., a polarizer, page 15) has two preferred states (i.e., the states that it can distinguish). A basis is not just a mathematical construct; it corresponds to **what our measurements device can distinguish**. For example:

- Computational basis, the device distinguishes: *are you $|0\rangle$ or $|1\rangle$?*
- Hadamard basis, the device distinguishes: *are you $|+\rangle$ or $|-\rangle$?*

These are **different physical equations**.

❓ **The math behind the measurement.** Measurement is projecting the state vector onto basis vectors. Forget qubits for a second and imagine a 2D real vector $\vec{v}$, and choose axes:

- **Standard axes:** we project onto x -axis and y -axis. We get components: v_x and v_y .
- **Rotated axes:** we rotate axes by 45° and project onto new axes the same vector $\vec{v}$. We get new components: v_+ and v_- .

Different numbers, same vector. The components are **not intrinsic properties of the vector**, but depend on the chosen axes. Back to qubits, **measurement projects the vector $|\psi\rangle$ onto the measurement axes** (i.e., the basis vectors). So $\langle u|\psi\rangle$ is the projection of $|\psi\rangle$ onto $|u\rangle$, and $|\langle u|\psi\rangle|^2$ is the probability of observing outcome u .

✏ **Example 6: Analogy - Measurement as projecting a vector onto different axes**

Consider a classical 2D vector pointing diagonally:

$$\vec{v} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}$$

Case 1: Standard axes (horizontal/vertical). If we describe the vector using the standard basis:

$$\left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right\},$$

Then the vector has equal components along both axes:

$$\vec{v} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

- Projection on horizontal axis: $\frac{1}{\sqrt{2}}$
- Projection on vertical axis: $\frac{1}{\sqrt{2}}$

Case 2: Rotated axes (diagonal basis). Now rotate the coordinate system by 45° and use the basis:

$$\left\{ \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} \right\}$$

In this basis, the same vector becomes:

$$\vec{v} = 1 \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} + 0 \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

- Projection on first axis: 1
- Projection on second axis: 0

Connection with qubits. A qubit behaves exactly in the same way:

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle = |+\rangle$$

- In basis $\{|0\rangle, |1\rangle\}$: equal projections $\Rightarrow$ probabilistic measurement.
- In basis $\{|+\rangle, |-\rangle\}$: full projection on $|+\rangle \Rightarrow$ deterministic measurement.

💡 The state does not change. What changes is the set of axes (basis) onto which the state is projected. Since measurement probabilities are given by these projections, the outcome depends on the chosen basis.

Given the state:

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

- If we measure in the **computational basis**, we get:

- Outcome 0 with probability $\left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}$
- Outcome 1 with probability $\left|\frac{1}{\sqrt{2}}\right|^2 = \frac{1}{2}$

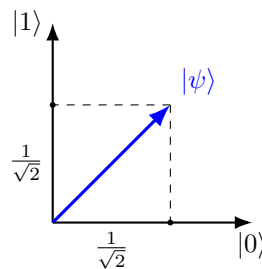


Figure 4: Measurement as projection onto basis vectors.

- If we measure in the **Hadamard basis**, we get:
 - Outcome + with probability $|\langle +|\psi\rangle|^2 = 1$
 - Outcome - with probability $|\langle -|\psi\rangle|^2 = 0$

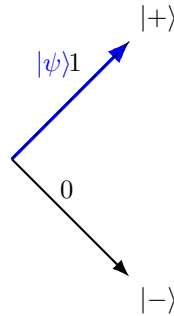


Figure 5: Measurement in the Hadamard basis. The state $|\psi\rangle$ is aligned with $|+\rangle$, so we get outcome $+$ with probability 1. The projection onto $|-\rangle$ is zero, so we get outcome $-$ with probability 0. The axes are rotated by 45° compared to the computational basis.

❓ What happens to the state after measurement? The state **collapses** to the observed outcome. For example, if we measure $|\psi\rangle = a|0\rangle + b|1\rangle$ in the computational basis and observe 0, the state collapses to $|0\rangle$. If we observe 1, the state collapses to $|1\rangle$.

🔗 Two possible situations

As we have seen in page 24, the same state can yield different measurement outcomes depending on the chosen basis. This leads to two important situations:

- **Probabilistic measurement:** If the **state is not aligned with the measurement basis**, we get a probabilistic outcome. For example:

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

Measured in the computational basis gives 0 or 1 with equal probability.

- **Deterministic measurement:** If the **state is aligned with one of the basis vectors**, we get a deterministic outcome. For example:

$$|\psi\rangle = |+\rangle$$

Measured in the Hadamard basis gives $+$ with probability 1 and $-$ with probability 0.

❓ When should we prefer one situation to another? It depends on the task. For example, if we want to encode a classical bit, we prefer deterministic measurement (e.g., $|0\rangle$ or $|1\rangle$). If we want to create superposition states for quantum algorithms, we might want probabilistic measurement (e.g., $|+\rangle$).

Now, if the same quantum state can lead to either deterministic or probabilistic measurement outcomes depending on the chosen basis, we can ask: “*how can be explained this phenomenon?*”. It means that a qubit can simultaneously have non-zero components along multiple basis states. This is a fundamental property of quantum mechanics, and it is what allows quantum computers to perform certain computations more efficiently than classical computers. It is called **superposition**, and we will discuss it in the next section.

2.4 Superposition

We have already seen that:

1. **A qubit is represented as a vector:**

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

Where a and b are complex numbers, and $|0\rangle$ and $|1\rangle$ are the basis states of the qubit.

2. **Measurement probabilities depend on the chosen basis:**

- In the computational basis $\{|0\rangle, |1\rangle\}$, the probabilities of measuring $|0\rangle$ and $|1\rangle$ are $|a|^2$ and $|b|^2$, respectively.
- In another basis, such as the Hadamard basis $\{|+\rangle, |-\rangle\}$, the probabilities of measuring $|+\rangle$ and $|-\rangle$ depend on different coefficients, which can be calculated by expressing $|\psi\rangle$ in terms of the new basis states.

3. **The same state can lead to deterministic or probabilistic outcomes depending on the measurement basis.** For example, if $|\psi\rangle = |0\rangle$, measuring in the computational basis always yield $|0\rangle$ (deterministic):

$$|0\rangle = 1|0\rangle + 0|1\rangle = |0\rangle$$

Whereas measuring in the Hadamard basis yields $|+\rangle$ or $|-\rangle$ with equal probability (probabilistic):

$$|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle)$$

These observations raise an essential question: “*why can a single state exhibit such different behaviors under different measurements?*”. The answer is **superposition**.

🔍 What is Superposition?

Superposition is a fundamental principle of quantum mechanics that **applies to all quantum systems**, not just qubits.

📖 Definition 2: Superposition

Superposition means that a **system can exist in several possible states at the same time until a measurement is made**. It is **basis-dependent**, meaning a state may be a superposition in one basis but not in another, for example:

$$|v\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle$$

Is a superposition with respect to the computational basis $\{|0\rangle, |1\rangle\}$, but it is **not** a superposition in the Hadamard basis $\{|+\rangle, |-\rangle\}$, where it can

be written as:

$$|v\rangle = 1|+\rangle + 0|-\rangle = |+\rangle$$

Formally, let $\{|u\rangle, |u^\perp\rangle\}$ be an orthonormal basis of a single-qubit Hilbert space. A qubit is said to be in **superposition** with respect to this basis if it can be written as:

$$|\psi\rangle = a|u\rangle + b|u^\perp\rangle$$

Where both coefficients a and b are non-zero and satisfy $|a|^2 + |b|^2 = 1$ (normalization condition). If one of the coefficients is zero, the state coincides with one of the basis vectors and the measurements in that basis becomes deterministic, thus it is not a superposition.

More in general, a quantum state $|\psi\rangle$ in a system with multiple possible states can exist in a **linear combination** of these states:

$$|\psi\rangle = c_1|\psi_1\rangle + c_2|\psi_2\rangle + \dots + c_n|\psi_n\rangle$$

Where:

- $|\psi_1\rangle, |\psi_2\rangle, \dots, |\psi_n\rangle$ are **basis states** of the system.
- $c_1, c_2, \dots, c_n$ are **complex probability amplitudes**.
- The system is in **all states at the same time**, with the probability of measuring each state given by $|c_i|^2$.
- The state is **normalized**, meaning:

$$|c_1|^2 + |c_2|^2 + \dots + |c_n|^2 = 1$$

Example 7: Single-Particle Systems (Quantum Mechanics)

In general quantum mechanics, a particle can exist in **multiple positions simultaneously** as a wave function $\Psi(x)$:

$$|\Psi\rangle = \int \Psi(x) |x\rangle dx$$

- The particle is in a **superposition of all possible positions** $|x\rangle$.
- Measurement collapses the wave function to a single position.

❓ How Can a Particle Exist in Multiple States at Once? This question touches the heart of quantum mechanics, where our everyday intuition breaks down. The **key idea** is that a **quantum particle** is not just a tiny ball, it is a **wave function that spreads across multiple possibilities at once**.

1. **Quantum Particles Are Waves, Not Just Points.** In classical physics, we think of particles as tiny, solid objects, like a small ball that always has a precise position and velocity.

In quantum mechanics, however, particles behave more like waves. These waves are described by a wave function $\Psi(x)$, which represents the **probability of finding the particle at different locations**.

The key idea is:

- The **wave function spreads across space**, meaning the **particle does not have a single location** before measurement.
- Instead, it exists in a superposition of all possible locations.

2. **The Double-Slit Experiment: Proof That a Particle Can Be in Two Places at Once.** The double-slit experiment was first performed by Thomas Young in 1801 to demonstrate the **wave nature of light**. Later, in the 20th century, it was repeated with **electrons, atoms**, and even **large molecules**, revealing that **matter itself exhibits wave-like behavior**. The classical expectation for the double-slit experiment is:

- **Classical Particles** (e.g., tiny balls). Imagine firing small balls at a barrier with two slits:
 - Each ball passes through **either** slit 1 **or** slit 2.
 - On a screen behind the slits, we would observe **two bright bands**, corresponding to the two slits.

The result is a simple pattern of two lines, no interference.

- **Classical Waves** (e.g., Water or Light). If instead we send **waves** (like water waves or classical light):
 - Each wave passes through **both slits simultaneously**.
 - The waves emerging from the slits **interfere: constructive interference** creates bright bands, while **destructive interference** creates dark bands.

The result is an **interference pattern** of alternating bright and dark bands.

The quantum reality is even more surprising:

- **Quantum Particles** (e.g., photons or electrons). When the experiment is performed with **single quantum particles** (photons or electrons), something remarkable happens:
 - (a) **Particles are emitted one at a time.**
 - (b) Each particle is detected as a **localized point** on the screen.
 - (c) After many particles are detected, an **interference pattern gradually emerges**.

This means that each particle behaves as if it passes through **both slits simultaneously**. The particle is described by a **wave function**, representing a **superposition of paths**:

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|\text{slit 1}\rangle + |\text{slit 2}\rangle)$$

The interference pattern arises from the **coherent combination** of these two possibilities.

Suppose we place detectors at the slits to determine **which slit** the particle passes through. The interference pattern **disappears** and the screen now shows **two bands**, just like classical particles. This is because measuring the particle's path **entangles** (i.e., correlates) it with the environment (the detector), leading to **decoherence** (loss of quantum coherence). The superposition:

$$\frac{1}{\sqrt{2}} (|\text{slit 1}\rangle + |\text{slit 2}\rangle)$$

Becomes effectively a **classical mixture**:

$$\frac{1}{2} |\text{slit 1}\rangle \langle \text{slit 1}| + \frac{1}{2} |\text{slit 2}\rangle \langle \text{slit 2}|$$

Without coherence, **interference cannot occur**.

In summary, the double-slit experiment demonstrates the principle of quantum superposition. When quantum particles such as photons or electrons are sent one at a time through two slits, they form an interference pattern on a detection screen. This indicates that each particle behaves as if it travels through both slits simultaneously, corresponding to the superposition:

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|\text{slit 1}\rangle + |\text{slit 2}\rangle)$$

If a measurement is performed to determine which slit the particle passes through, the interference pattern disappears due to decoherence, and the system behaves like a classical probabilistic mixture.

Double Slit Experiment explained! by Jim Al-Khalili



3. **Quantum Superposition: More Than Just Probability.** A common misconception is that a particle in superposition is just an **unknown state**, like a coin that is either heads or tails, but we just don't know which. This is wrong, because quantum superposition is much deeper.

A quantum state is a **combination of all possibilities**. *Until measurement*, the system is in **all possible states at once**.

Mathematically, for an electron in **two locations** x_1 and x_2 :

$$|\psi\rangle = a |x_1\rangle + b |x_2\rangle$$

- The electron is **literally in both places simultaneously**.
 - The coefficients a and b are **complex numbers representing the probability amplitudes**.
 - **Interference** between these amplitudes **creates quantum effects that cannot be explained by classical probability**.
4. **Superposition in Quantum Computing.** Quantum computing **directly uses** the fact that a particle can be in multiple states at once.

- A **classical bit** can only be **0 or 1**.
- A **qubit (quantum bit)** can be in a superposition:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

This means a quantum computer can **perform many calculations simultaneously**. Therefore, **superposition allows quantum computers to process information exponentially faster than classical computers for certain tasks**.

5. **Why Don't We See Superposition in Everyday Life?** In our daily experience, objects are **not in multiple states at once** because of a process called **quantum decoherence**.

Quantum superposition is **fragile**. When a quantum system interacts with the environment (air, light, etc.), the superposition **collapses into one definite state**. This is why large objects (like humans or cars) do not appear in multiple places at once.

However, experiments (like the Double-Slit experiment) confirm that superposition is real at microscopic scales (electrons, photons, atoms, and even molecules).

The key **properties of Superposition** are:

1. **Linearity**: Any combination of valid quantum states is also a valid quantum state.
2. **Interference**: Quantum states in superposition can interfere, leading to constructive or destructive interference.
3. **Measurement Collapse**: When measured, the superposition collapses into a single outcome.
4. **Phase Information**: Unlike classical probabilities, quantum superpositions include complex phases that affect interference patterns.

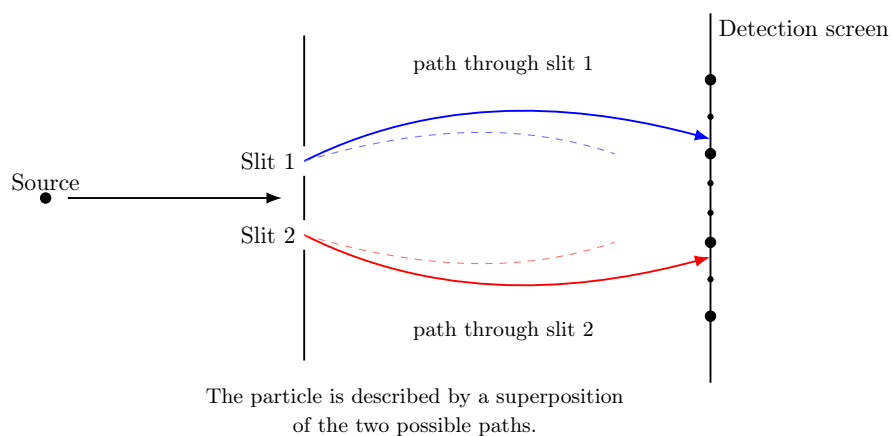


Figure 6: Double-slit experiment, superposition of paths. This figure illustrates the **quantum state of the particle before detection**. Instead of choosing a single path, the particle is described by a **superposition**:

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|\text{slit 1}\rangle + |\text{slit 2}\rangle)$$

The particle is **not** traveling through only one slit, instead, it is described by a **coherent combination** of both possibilities. The diagram emphasizes the **quantum state** rather than the final inference pattern.

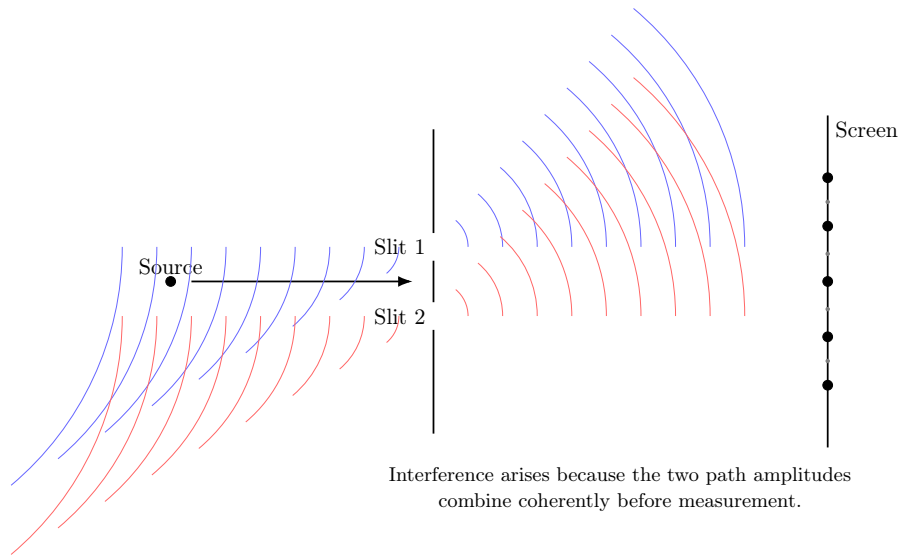


Figure 7: Wave-like representation of the double-slit experiment, interference of amplitudes. This figure focuses on the **wave nature** of the quantum particle. Each slit acts as a **coherent source**, producing wavefronts that overlap and interfere. If $A_1(x)$ and $A_2(x)$ are the probability amplitudes associated with the two slits, the probability of detecting the particle at position x on the screen is:

$$P(x) = |A_1(x) + A_2(x)|^2 = |A_1(x)|^2 + |A_2(x)|^2 + 2\text{Re}(A_1^*(x)A_2(x))$$

The last term is the **inference term**, which can be positive (bright fringes, **constructive interference**) or negative (dark fringes, **destructive interference**). This illustrates how the superposition of paths leads to the characteristic interference pattern observed in the experiment.

2.5 Information Carried by a Single Qubit

This section addresses a fundamental question: *if a qubit can exist in infinitely many states, how much information can it actually convey?*

A general single-qubit state is given by:

$$|\psi\rangle = a|0\rangle + b|1\rangle \quad a, b \in \mathbb{C}, \quad |a|^2 + |b|^2 = 1$$

Because a and b are complex numbers, a qubit can be prepared in **infinitely many distinct states**. This might suggest that a qubit can encode an infinite amount of information. However, this is **not** the case. When we perform a **measurement**, we obtain only a **single classical outcome** (either 0 or 1). Therefore, **a single qubit can yield at most one classical bit of information when measured**. This apparent paradox, **infinite possible states but limited measurable information**, is one of the most important conceptual aspects of quantum information theory.

❓ Why can't we determine the Amplitudes a and b of a single qubit state?

Suppose we are given a qubit in an unknown state $|\psi\rangle$. Can we determine a and b from a single measurement? The answer is **no**, because:

- ✗ Measurement is probabilistic.** A single measurement provides only one outcome (0 or 1), which is insufficient to determine the amplitudes.
- ✗ Measurement disturbs the state.** After measurement, the qubit collapses to the observed basis state, preventing further measurements on the original state.
- ✗ State estimation requires many copies.** To estimate a and b , we need **quantum state tomography**, which relies on performing measurements on **many identically prepared qubits**.

This **irreversible collapse** means that the original information encoded in the amplitudes is generally lost after measurement.

⚠ The No-Cloning Theorem

One might think of copying the qubit before measurement to extract more information. However, quantum mechanics forbids this through the **No-Cloning Theorem** (we will see more about this on page 329).

Theorem 1 (No-Cloning). *It is impossible to create an identical copy of an arbitrary unknown quantum state.*

Proof. Assume there exists a unitary operator U that can clone any state $|\psi\rangle$:

$$U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle \quad \forall |\psi\rangle$$

For two arbitrary states $|\psi\rangle$ and $|\phi\rangle$, linearity of quantum mechanics implies:

$$U(\alpha|\psi\rangle + \beta|\phi\rangle)|0\rangle = \alpha|\psi\rangle|\psi\rangle + \beta|\phi\rangle|\phi\rangle$$

Which is **not equal** to:

$$(\alpha |\psi\rangle + \beta |\phi\rangle)(\alpha |\psi\rangle + \beta |\phi\rangle) = \alpha^2 |\psi\rangle |\psi\rangle + \alpha\beta |\psi\rangle |\phi\rangle + \alpha\beta |\phi\rangle |\psi\rangle + \beta^2 |\phi\rangle |\phi\rangle$$

This contradiction proves that perfect cloning is impossible, since the two expressions cannot be equal for all $|\psi\rangle$ and $|\phi\rangle$. QED

Since we cannot clone a qubit, we cannot create multiple copies to determine its exact state from a single instance.

Feature	Classical Bit	Qubit
Possible states	0 or 1	Qubit
Info from measurement	1 bit	At most 1 bit
Measurement disturbance	No	Yes
Ability to copy	Yes	No (No-Cloning)
State determination	Single observation	Requires many identical copies

Table 1: Comparison of classical bits and qubits in terms of information content and measurement properties.

2.6 State Space of a Single-Qubit System

The **State Space** of a physical system is the set of all possible states that the system can occupy. For a **single qubit**, the state space consists of all vectors of the form:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

- $a, b \in \mathbb{C}$ are **complex probability amplitudes**.
- $|0\rangle$ and $|1\rangle$ form an **orthonormal basis** (the computational basis).
- The **normalization condition** must hold: $|a|^2 + |b|^2 = 1$. This ensures that the total probability of all measurement outcomes is equal to 1.

Degrees of Freedom of a Qubit

At first glance, the coefficients a and b appear to provide **4 real parameters**:

$$\begin{aligned} a = a_r + ia_i &\Rightarrow a_r, a_i \quad (2 \text{ parameters}) \\ b = b_r + ib_i &\Rightarrow b_r, b_i \quad (2 \text{ parameters}) \end{aligned}$$

So initially, it seems that a qubit has **four degrees of freedom**. However, not all of these correspond to physically distinguishable states.

- **Normalization constraint.** Quantum states must be normalized, which imposes the condition:

$$|a|^2 + |b|^2 = a_r^2 + a_i^2 + b_r^2 + b_i^2 = 1$$

Geometrically, it defines the surface of a **unit sphere in four dimension** (also known as a 3-sphere, S^3). This constraint effectively removes **one degree of freedom**, since the amplitudes must lie on the surface of a 3-sphere in 4-dimensional space.

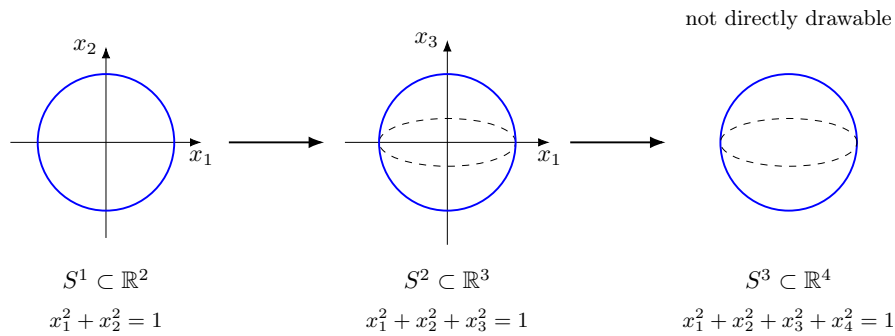


Figure 8: Analogy between the unit circle S^1 , the ordinary sphere S^2 , and the 3-sphere S^3 . The normalized qubit state lives on S^3 because it is specified by four real coordinates constrained by one equation. In this figure, we can only visualize S^1 and S^2 , while S^3 cannot be directly drawn in three-dimensional space, so we represent it with a dashed ellipse to indicate that it is a higher-dimensional object.

- **Global Phase Equivalence.** Two states that differ only by a **global phase** represent the **same physical qubit**:

$$|\psi'\rangle = e^{i\phi} |\psi\rangle$$

For example:

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \quad e^{i\pi/3} |\psi\rangle = \frac{e^{i\pi/3}}{\sqrt{2}} (|0\rangle + |1\rangle)$$

❓ **Why is it physically irrelevant?** Measurement probabilities are given by:

$$P(i) = |\langle i|\psi\rangle|^2$$

If we multiply the state by a global phase:

$$|\langle i|e^{i\gamma}\psi\rangle|^2 = |e^{i\gamma}\langle i|\psi\rangle|^2 = |\langle i|\psi\rangle|^2$$

Because $|e^{i\gamma}| = 1$ for any real γ .

Mathematical proof that $|e^{i\gamma}| = 1$. We can express $e^{i\gamma}$ using Euler's formula:

$$e^{i\gamma} = \cos(\gamma) + i\sin(\gamma)$$

The magnitude of a complex number $z = x + iy$ is given by:

$$|z| = \sqrt{x^2 + y^2}$$

Applying this to $e^{i\gamma}$:

$$\begin{aligned} |e^{i\gamma}| &= \sqrt{(\cos(\gamma))^2 + (\sin(\gamma))^2} \\ &\downarrow \text{using the trigonometric identity} \\ &= \sqrt{\cos^2 \gamma + \sin^2 \gamma} \\ &= \sqrt{1} \\ &= 1 \end{aligned}$$

QED

Therefore, **no experiment can distinguish between $|\psi\rangle$ and $e^{i\phi}|\psi\rangle$** , since **they yield the same measurement probabilities** (due to $|e^{i\phi}| = 1$).

❓ **How does this remove a degree of freedom?** Unlike normalization, the global phase does **not** impose a constrain equation. Instead, it introduces a **redundancy** in our description:

- Many different vectors in S^3 correspond to the **same physical state**.
- All vectors of the norm $e^{i\gamma}|\psi\rangle$ are considered **physically equivalent** to $|\psi\rangle$.

Mathematically, we are **identifying** all these vectors as a single point. This process is called forming a **quotient space**: the set of physically distinct pure qubit states is $S^3 \setminus S^1 \cong S^2$, which is a 2-dimensional surface (the Bloch sphere, see Figure 8, page 36).

After accounting for these constraints, we find that a single qubit state is fully described by **two real parameters** (two degrees of freedom). Therefore, any qubit state can be written in the form:

1. A normalized qubit can be written as:

$$|\psi\rangle = a|0\rangle + b|1\rangle \quad a, b \in \mathbb{C} \quad |a|^2 + |b|^2 = 1$$

2. Any complex number can be written using its **magnitude** and **phase**:

$$a = |a|e^{i\alpha} \quad b = |b|e^{i\beta}$$

Where $|a|$ and $|b|$ are non-negative real numbers, and α and β are real numbers representing the phases. Substituting these into the expression for $|\psi\rangle$ gives:

$$|\psi\rangle = \underbrace{|a|e^{i\alpha}}_a |0\rangle + \underbrace{|b|e^{i\beta}}_b |1\rangle$$

3. We can factor out the common global phase $e^{i\alpha}$:

$$|\psi\rangle = e^{i\alpha} \left(|a| |0\rangle + |b| e^{i(\beta-\alpha)} |1\rangle \right)$$

Since the global phase $e^{i\alpha}$ has **no physical effect**, we can omit it, leading to:

$$|\psi\rangle = |a| |0\rangle + |b| e^{i\phi} |1\rangle$$

Where we define also the **Relative Phase** $\phi = \beta - \alpha$. At this point, the state is characterized by two real magnitudes $|a|$ and $|b|$, and one relative phase ϕ . Mathematically, if:

$$a = |a|e^{i\alpha} \quad b = |b|e^{i\beta}$$

Then:

$$\text{Relative Phase} = \phi = \arg(b) - \arg(a) = \beta - \alpha \quad (7)$$

4. From normalization:

$$|a|^2 + |b|^2 = 1$$

We recognize a familiar trigonometric identity:

$$\cos^2(x) + \sin^2(x) = 1$$

Therefore, we can parametrize the magnitudes using a single angle θ :

$$|a| = \cos\left(\frac{\theta}{2}\right) \quad |b| = \sin\left(\frac{\theta}{2}\right)$$

With $0 \leq \theta \leq \pi$ to ensure that $|a|$ and $|b|$ are non-negative. The reason for using $\frac{\theta}{2}$ instead of θ will become clear when we discuss the Bloch sphere representation (Figure 9, page 40).

5. Substituting the parametrized magnitudes into the state:

$$|\psi\rangle = \underbrace{\cos\left(\frac{\theta}{2}\right)}_{|a|} |0\rangle + e^{i\phi} \underbrace{\sin\left(\frac{\theta}{2}\right)}_{|b|} |1\rangle \quad (8)$$

This is the **canonical parametrization** of a single qubit state.

Regardless of the specific values of θ and ϕ in Equation 8:

- θ : controls the **relative weights of $|0\rangle$ and $|1\rangle$** in the superposition, determining the **probabilities of measuring each state**.
- ϕ : controls the **relative phase between the $|0\rangle$ and $|1\rangle$** components. While it does not affect measurement probabilities, it is crucial for interference effects and the behavior of the qubit under quantum gates (which we will discuss later).

Bloch Sphere Representation of a Qubit

After normalization and removal of the physically irrelevant global phase, a pure single-qubit state depends on only two real parameters. Therefore, it is natural to represent it geometrically as a point on a two-dimensional surface. This surface is the **Bloch sphere**.

Any pure qubit can be written in the form:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle,$$

Where:

- $\theta \in [0, \pi]$ is the **polar angle**,
- $\phi \in [0, 2\pi)$ is the **Azimuthal Angle**, representing the **relative phase**.

This expression gives a one-to-one correspondence between pure qubit states and points on the surface of a unit sphere:

- $|0\rangle$ corresponds to the north pole, since $\theta = 0$:

$$|0\rangle = \cos\left(\frac{0}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{0}{2}\right) |1\rangle = 1 \cdot |0\rangle + 0 \cdot |1\rangle = |0\rangle$$

- $|1\rangle$ corresponds to the south pole, since $\theta = \pi$:

$$|1\rangle = \cos\left(\frac{\pi}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\pi}{2}\right) |1\rangle = 0 \cdot |0\rangle + e^{i\phi} \cdot 1 \cdot |1\rangle = e^{i\phi} |1\rangle \Rightarrow |1\rangle$$

⚠ Attention! $e^{i\phi} |1\rangle$ becomes $|1\rangle$ after **removing the global phase** (physically unobservable), so $|1\rangle$ is indeed at the south pole. In general, all states of the form $e^{i\phi} |1\rangle$ represent the **same physical state** as $|1\rangle$. Hence, $|1\rangle$ corresponds to the **south pole** of the Bloch sphere.

- States on the equator correspond to equal-weight superpositions of $|0\rangle$ and $|1\rangle$.

In particular:

$$\text{Hadamard} \quad |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle),$$

$$\text{Imaginary} \quad |i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle), \quad |-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle),$$

All lie on the equator and differ only by their relative phase.

The Bloch sphere provides an intuitive visualization of a qubit: moving along the vertical direction changes the relative weights of $|0\rangle$ and $|1\rangle$, while rotating around the vertical axis changes the relative phase between them.

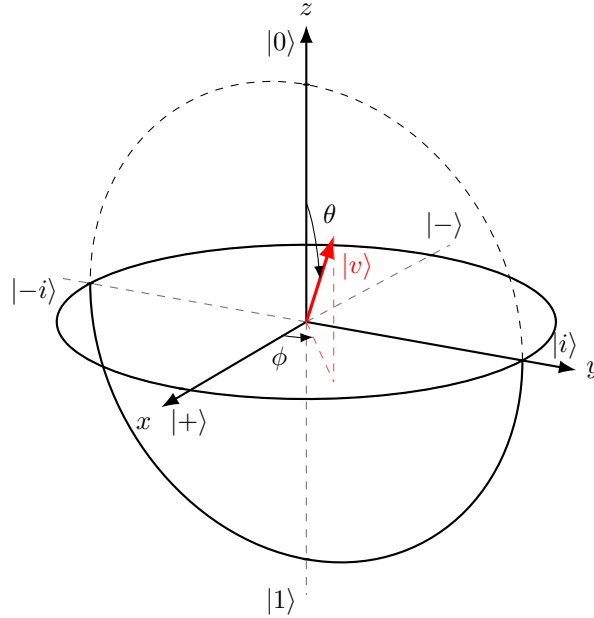


Figure 9: Bloch sphere representation of a pure qubit state. The red vector $|v\rangle$ represents a generic qubit state. Measuring $|v\rangle$ along the z -axis gives probabilities determined by θ , while along x or y -axes reveals information about the relative phase ϕ .

On the Bloch sphere, the axes correspond to **three fundamental measurement bases** for a qubit. Each pair of opposite points represents the **eigenstates of one of the Pauli operators** (we will discuss the Pauli operators in more detail in 3.3.2.2, page 51):

- **z -axis, computational basis** $\{|0\rangle, |1\rangle\}$, eigenstates of σ_z . The states are:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \text{and} \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

These states correspond to the classical bit values 0 and 1.

- $|0\rangle$: North pole of the Bloch sphere $(0, 0, 1)$.
- $|1\rangle$: South pole of the Bloch sphere $(0, 0, -1)$.

Measuring along the z -axis answer the question: “*is the qubit $|0\rangle$ or $|1\rangle$?*”.

- **x -axis, Hadamard basis** $\{|+\rangle, |-\rangle\}$, eigenstates of σ_x . The states are:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \quad \text{and} \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

- $|+\rangle$: $+x$ direction on the equator $(1, 0, 0)$, with $\phi = 0$ (no relative phase).
- $|-\rangle$: $-x$ direction on the equator $(-1, 0, 0)$, with $\phi = \pi$ (relative phase of π between $|0\rangle$ and $|1\rangle$).

These states represent **equal superpositions** of $|0\rangle$ and $|1\rangle$ with **relative phase** 0 and π .

- **y -axis, imaginary basis** $\{|i\rangle, |-i\rangle\}$, eigenstates of σ_y . The states are:

$$|i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) \quad \text{and} \quad |-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle)$$

- $|i\rangle$: $+y$ direction on the equator $(0, 1, 0)$, with $\phi = \frac{\pi}{2}$ (relative phase of $\frac{\pi}{2}$ between $|0\rangle$ and $|1\rangle$).
- $|-i\rangle$: $-y$ direction on the equator $(0, -1, 0)$, with $\phi = -\frac{\pi}{2}$ (relative phase of $-\frac{\pi}{2}$ between $|0\rangle$ and $|1\rangle$).

These states also have equal probabilities of measuring $|0\rangle$ or $|1\rangle$ in the computational basis, but they differ by a **relative phase of $\pm\frac{\pi}{2}$** . This phase is crucial for **interference effects** in quantum algorithms.

3 Single Qubit Gates

3.1 Operations on Qubits

Quantum computing relies on three essential operations:

1. **Logic Gates:** transform quantum states.
2. **Measurements:** extract classical information from qubits.
3. **Initialization Procedures:** prepare qubits in known starting states.

Logic Gates: Reversible Transformations

Quantum **Logic Gates** are the building blocks of quantum circuits. They **manipulate the state of qubits in a controlled and deterministic way**, analogous to classical logic gates (like AND or NOT), but with key quantum differences:

- **Operate on Single or Multiple Qubits.** They can affect one qubit (single-qubit gates) or multiple qubits simultaneously (multi-qubit gates):
 - **Single-Qubit Gates:** These gates, such as the Pauli-X, Y, Z, Hadamard (H), and Phase (S) gates, operate on a single qubit to change its state. We will explore these gates in detail in the next sections.
 - **Multi-Qubit Gates:** These gates, such as the CNOT (Controlled-NOT) and Toffoli gates, operate on multiple qubits and can create *entanglement* (we will discuss this in detail later) between them, which is a key resource in quantum computing.
- **Reversible Transformations.** Unlike many classical gates (e.g., AND), **all valid quantum gates are reversible**. Mathematically, this means they are represented by **unitary matrices** U , satisfying:

$$U^\dagger U = U U^\dagger = I$$

Where $U^\dagger$ is the conjugate transpose of U , I is the identity matrix, and U is a unitary matrix¹. Because of this reversibility, quantum gates **do not lose information** and the **original state can be recovered** by applying $U^\dagger = U^{-1}$ to the output state. This is a fundamental difference from classical logic gates, which can be irreversible (e.g., AND gate loses information about the inputs).

Finally, this property is related to fundamental principles such as the **no-cloning theorem** (page 34, page 329) and the **no-deleting theorem**

¹A matrix U is called **unitary** if its **conjugate transpose equals its inverse**:

$$U^\dagger U = U U^\dagger = I \tag{9}$$

We remind the reader that the definition of the inverse of a matrix is:

$$U^{-1}U = U U^{-1} = I$$

So for a unitary matrix, the conjugate transpose $U^\dagger$ plays the role of the inverse. This means that unitary transformations preserve vector norms, orthogonality, and therefore total probability in quantum mechanics.

(we cannot delete quantum information arbitrarily, since it is preserved under unitary transformations, page 338).

- **State Evolution.** If a qubit is in the state:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

Then after applying a gate U , the new state becomes:

$$|\psi'\rangle = U|\psi\rangle = aU|0\rangle + bU|1\rangle$$

This **transformation is deterministic** and **preserves the normalization of the state** (i.e., $|a|^2 + |b|^2 = 1$ remains true after applying U). The specific effect of the gate depends on its matrix representation, which we will explore in the next sections.

We can think of quantum logic gates as **rotations on the Bloch sphere**, smoothly transforming the qubit's state without destroying its quantum nature.

Measurement: Irreversible Collapse of State

Measurement is the process of extracting information from a qubit. Unlike logic gates, measurement is **irreversible** and fundamentally probabilistic.

- **Irreversible Process.** Once a qubit is measured, its original quantum state is **lost**. This distinguishes measurement from unitary gate operations.

What is a unitary gate? A **Unitary Gate** is a quantum gate represented by a **unitary matrix**. More precisely, a quantum gate U is unitary if (see page 42):

$$U^\dagger U = UU^\dagger = I$$

Where $U^\dagger$ is the conjugate transpose of U , and I is the identity matrix. Since a unitary matrix has an inverse equal to its conjugate transpose, then **every quantum gate is reversible**.

Important Property. A fundamental property of **unitary gates** is that they **preserve similarity transformations**. This means that they preserve **geometric relationships** between quantum states, such as lengths (norms), angles, and inner products:

$$\langle\psi|\phi\rangle = \langle U\psi|U\phi\rangle \quad (10)$$

Where $|\psi\rangle$ and $|\phi\rangle$ are any two quantum states. For example, if two states are orthogonal (i.e., $\langle\psi|\phi\rangle = 0$), then after applying a unitary gate U , they remain orthogonal (i.e., $\langle U\psi|U\phi\rangle = 0$). In other words, if two states are “very close”, they remain “very close” after applying a unitary transformation.

- **State Collapse.** Measuring a qubit in the computational basis $\{|0\rangle, |1\rangle\}$ causes the state:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

To **collapse** to:

- $|0\rangle$ with probability $|a|^2$.
- $|1\rangle$ with probability $|b|^2$.

- **Extraction of Classical Information.** The measurement outcome is a **classical bit** (0 or 1). After measurement, subsequent measurements in the same basis yield the same result with probability 1, since the state has collapsed to a definite state.

Measurement can be described using **projection operators**:

$$M_0 = |0\rangle\langle 0| \quad M_1 = |1\rangle\langle 1|$$

The probability of obtaining outcome k (where $k \in \{0, 1\}$) when measuring $|\psi\rangle$ is given by:

$$\begin{aligned}
 p_k &= \langle \psi | M_k | \psi \rangle \\
 &\downarrow \text{substitute } M_k = |k\rangle\langle k| \\
 &= \langle \psi | (|k\rangle\langle k|) | \psi \rangle \\
 &\downarrow \text{using the associativity of inner products} \\
 &= (\langle \psi | k \rangle) (\langle k | \psi \rangle) \\
 &\downarrow \text{since } \langle \psi | k \rangle = (\langle k | \psi \rangle)^* \\
 &\quad \text{where } * \text{ denotes complex conjugation} \\
 &= (\langle k | \psi \rangle)^* (\langle k | \psi \rangle) \\
 &\downarrow \text{which is the definition of squared magnitude, page 10} \\
 &= |\langle k | \psi \rangle|^2
 \end{aligned}$$

In other words, the probability of measuring outcome k is the **squared magnitude of the amplitude** of $|\psi\rangle$ along the direction $|k\rangle$. And the post-measurement state of the qubit, conditioned on obtaining outcome k , becomes:

$$|\psi_k\rangle = \frac{M_k |\psi\rangle}{\sqrt{p_k}}$$

Measurement is like **asking a question** to the qubit: “are you in state $|0\rangle$ or $|1\rangle$?”. The answer is classical (i.e., 0 or 1), but the act of asking **changes the system**.

Example 1: Calculating Measurement Probabilities

Consider a qubit in the state:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

With $|a|^2 + |b|^2 = 1$.

- Probability of measuring 0. Using the projector:

$$M_0 = |0\rangle\langle 0|$$

We compute:

$$\begin{aligned}
 p_0 &= \langle \psi | M_0 | \psi \rangle \\
 &= \langle \psi | 0 \rangle \langle 0 | \psi \rangle \\
 &= (\langle \psi | 0 \rangle) (\langle 0 | \psi \rangle) \\
 &\downarrow \text{since } \langle \psi | 0 \rangle = (\langle 0 | \psi \rangle)^* \\
 &= |\langle 0 | \psi \rangle|^2 \\
 &= |a|^2
 \end{aligned}$$

- Probability of measuring 1. Using the projector:

$$M_1 = |1\rangle \langle 1|$$

We compute:

$$\begin{aligned}
 p_1 &= \langle \psi | M_1 | \psi \rangle \\
 &= \langle \psi | 1 \rangle \langle 1 | \psi \rangle \\
 &= (\langle \psi | 1 \rangle) (\langle 1 | \psi \rangle) \\
 &\downarrow \text{since } \langle \psi | 1 \rangle = (\langle 1 | \psi \rangle)^* \\
 &= |\langle 1 | \psi \rangle|^2 \\
 &= |b|^2
 \end{aligned}$$

Initialization: Preparing Known States

The **Initialization** is the process of preparing a qubit in a **known and controlled state**, typically $|0\rangle$ or $|1\rangle$. This step is essential because quantum algorithms require a well-defined starting point. It is implemented through various methods, including:

- **Physical Preparation.** Many quantum technologies naturally relax to the ground state $|0\rangle$. Additional gates (e.g., Pauli-X) can be applied to obtain $|1\rangle$ if needed.
- **Initialization via Measurement.** Measuring a qubit projects it into a basis state. If the outcome is not the desired one, a corrective gate can be applied. For example, if measurement yields $|1\rangle$ but $|0\rangle$ is needed, apply an X gate to flip it to $|0\rangle$.

Initialization is the **first step** in any quantum computation: (1) **prepare** qubits in known states, (2) **apply** them using quantum gates, and (3) **measure** to obtain classical results. It is analogous to **resetting a classical register** before starting a computation in classical computing.

3.2 Quantum Logic Gates Overview

In quantum computing, quantum logic gates are the primary tools used to **manipulate qubits**. These gates perform **unitary transformations** on the state of one or more qubits, meaning they **preserve the norm** of the quantum state and are **reversible**. The **design of quantum algorithms** and the **execution of quantum circuits** rely entirely on the sequential application of these gates.

≡ Types of Quantum Gates

Quantum Gates can be classified into:

- **Single-Qubit Gates:** These gates operate on **individual qubits**, modifying their state in isolation.
- **Multiple-Qubit Gates:** These gates act on **two or more qubits simultaneously**, allowing for entanglement and complex correlations between qubits.

In this section we will focus on single qubit gates.

✓ Mathematical Framework: Qubit as Vectors, Gates as Matrices

To describe quantum superposition mathematically, we require a framework that supports linear combinations of states, probabilities derived from inner products, complex amplitudes and phases, and linear state transformations. The mathematical structure satisfying all these requirements is a **complex Hilbert space**. Thus, a qubit is naturally represented as a **unit vector** in a two-dimensional complex Hilbert space $\mathbb{C}^2$.

Using the Dirac notation, the basis states $|0\rangle$ and $|1\rangle$ form an orthonormal basis for this space, satisfying $\langle 0|0\rangle = \langle 1|1\rangle = 1$ and $\langle 0|1\rangle = \langle 1|0\rangle = 0$. Any qubit state can be expressed as a linear combination of these basis states:

$$|\psi\rangle = a|0\rangle + b|1\rangle \quad \text{with } |a|^2 + |b|^2 = 1$$

Two state vectors that differ only by a global phase factor $e^{i\phi}$ represent the same physical qubit, since measurement probabilities remain unchanged (see page 37).

Choosing the computational basis $\{|0\rangle, |1\rangle\}$, we can represent these states as column vectors:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Therefore, the qubit state can be written as:

$$|\psi\rangle = a|0\rangle + b|1\rangle = \begin{bmatrix} a \\ b \end{bmatrix}$$

This vector representation allows us to apply linear algebra techniques to analyze and manipulate qubits.

Once qubits are represented as vectors, transformations of these states must be **linear operators** acting on the vector space. In a chosen basis, linear operators are represented by **matrices**. To preserve normalization and probabilities, these matrices must be **unitary**:

$$U^\dagger U = U U^\dagger = I$$

Thus, quantum gates – the **building blocks of quantum circuits** – are represented as unitary 2×2 matrices that linearly transform the state vector:

$$|\psi'\rangle = U |\psi\rangle$$

❓ Why Matrices? Because quantum mechanics is inherently linear, any physical evolution of a closed quantum system must be described by a **linear operator**. When expressed in a chosen basis (such as the computational basis $\{|0\rangle, |1\rangle\}$), this operator takes the form of a matrix.

☆ Unitary: The Fundamental Constraint

A matrix U is **unitary** if it satisfies the condition:

$$U^\dagger U = U U^\dagger = I \quad (11)$$

Where:

- $U^\dagger$ is the **Hermitian transpose** (conjugate transpose) of U .
- I is the **identity matrix**.

This property ensures that $U^{-1} = U^\dagger$, meaning every quantum operation is **reversible**, and the transformation preserves the **norm** of the state vector. So, quantum gate **must be unitary** to ensure that the evolution of quantum states is consistent with the principles of quantum mechanics (i.e., linearity, reversibility, and preservation of probabilities).

A **general single-qubit unitary matrix** can be expressed as:

$$U = \begin{pmatrix} \alpha & \beta \\ \gamma & \delta \end{pmatrix} \quad \text{with } \alpha, \beta, \gamma, \delta \in \mathbb{C} \quad \text{and} \quad U^\dagger U = I \quad (12)$$

❓ Why must gates be unitary?

1. **No-Cloning Theorem:** It is impossible to create an identical copy of an unknown qubit. Unitary transformations respect this by not duplicating information.
2. **No-Deleting Theorem:** We cannot delete quantum information arbitrarily. Since unitary gates are invertible, they do not erase information.
3. **Preservation of Probability:** The normalization condition $|a|^2 + |b|^2 = 1$ must hold after any operation, and **only unitary matrices preserve this norm.**

Physical Intuition

In classical circuits, logic gates like AND, OR, NOT process bits deterministically. In quantum circuits:

- Gates **rotate** the quantum state on the **bloch sphere**.
- These rotations correspond to **unitary transformations**.
- The **physical implementation** of quantum gates may vary across hardware (e.g., superconducting qubits, trapped ions), but **mathematically**, the **same unitary matrix describes the gate**.

Quantum gates do not correspond to physical gates in the classical sense. They are **abstract operations** realized by **controlled interactions** in quantum systems.

3.3 Main Single-Qubit Gates

3.3.1 Identity Gate (I)

The **Identity Gate**, denoted I , is the most elementary gate in quantum computing. It performs **no change** to the state of a qubit. Its action on a qubit is trivial, yet it **serves important roles in quantum circuits**, particularly **when managing multi-qubit systems** or **timing synchronization**.

√* Matrix Representation

The Identity gate is represented by the 2×2 **identity matrix**:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad (13)$$

Applying the Identity gate to any qubit **leaves the state unchanged**. Formally, for a qubit in state $|\psi\rangle$:

$$I|\psi\rangle = |\psi\rangle$$

To better understand its effect, consider its action on the computational basis:

$$\begin{aligned} I|0\rangle &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle \\ I|1\rangle &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle \end{aligned}$$

Hence, the Identity gate **preserves the state** of the qubit, including any superposition or entangled state it may be in.

📖 Physical Interpretation

Although it may appear useless at first glance, the Identity gate has **conceptual and practical utility**:

- ✓ It acts as a **placeholder** or “*do nothing*” operation when required by **circuit timing**.
- ✓ In **multi-qubit circuits**, it allows one **qubit to remain unchanged** while other qubits are operated on.
- ✓ In **matrix composition** of gates, it serves to **align dimensions** when performing **tensor products** (e.g., $I \otimes H$ applies Hadamard to the second qubit only).

🌀 Bloch Sphere Perspective

In the Bloch sphere (page 40), the **identity gate** represents a **rotation of zero degrees**, meaning the state vector remains exactly where it is. It is the neutral element of quantum transformations.

3.3.2 Pauli Gates

The **Pauli Gates** are a set of fundamental single-qubit quantum gates that play a central role in quantum computing. They are represented by the three **Pauli matrices** X , Y , and Z , and can be interpreted as the quantum analogues of basic operations on a qubit.

Each Pauli gate corresponds to a **rotation of π radians** around one of the principal axes of the Bloch sphere:

- X : rotation around the x -axis (bit flip).
- Z : rotation around the z -axis (phase flip).
- Y : rotation around the y -axis (bit and phase flip).

These operators are **unitary** ($U^\dagger U = U U^\dagger = I$), **Hermitian** ($U = U^\dagger$), and **self-inverse** ($U^2 = I$), and their eigenstates define the three fundamental measurement bases of a qubit. Together, they form the building blocks for describing single-qubit transformations and quantum dynamics. Also, they have eigenvalues ± 1 and satisfy the property:

$$X^2 = Y^2 = Z^2 = I$$

Meaning that applying the same Pauli gate twice returns the qubit to its original state.

3.3.2.1 Rotations Generated by Pauli Operators

In quantum mechanics, rotations of a qubit are generated by the Pauli operators. A rotation around an axis $\hat{n}$ by an angle θ is given by:

$$R_{\hat{n}}(\theta) = e^{-i\frac{\theta}{2}(\hat{n} \cdot \vec{\sigma})} \quad (14)$$

Where $\vec{\sigma} = (X, Y, Z)$ are the Pauli matrices. We will see those matrices in more detail in the next sections, but for now we can just note that they are the generators of rotations around the x , y , and z axes, respectively.

In particular:

$$R_x(\theta) = e^{-i\frac{\theta}{2}X} \quad (15)$$

$$R_y(\theta) = e^{-i\frac{\theta}{2}Y} \quad (16)$$

$$R_z(\theta) = e^{-i\frac{\theta}{2}Z} \quad (17)$$

Using the identity $\sigma^2 = I$, these can be expanded as:

$$R_\alpha(\theta) = \cos\left(\frac{\theta}{2}\right) I - i \sin\left(\frac{\theta}{2}\right) \sigma_\alpha \quad (18)$$

3.3.2.2 Pauli-X (NOT) Gate

The **Pauli-X Gate**, often called the **quantum NOT gate**, is one of the *fundamental* single-qubit quantum gates. It is denoted X and **corresponds to the σ_x Pauli matrix** in quantum mechanics. This gate **flips the state of a qubit, similar to how a classical NOT gate inverts a bit.**

Remark: Pauli Matrix

In quantum mechanics, the **Pauli matrices** are a set of three 2×2 **Hermitian and unitary matrices** that represent **spin operators** for spin- $\frac{1}{2}$ particles and are widely used in quantum computing to describe single-qubit gates. They are:

$$\sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \sigma_y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \quad \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Key properties:

- Hermitian: $\sigma_i^\dagger = \sigma_i$ (observable-related)
- Unitary: $\sigma_i^\dagger \sigma_i = I$ (reversible transformations)
- Involution: $\sigma_i^2 = I$
- Non-commuting: $[\sigma_i, \sigma_j] = 2i\varepsilon_{ijk}\sigma_k$ (with ε the Levi-Civita symbol)

√ Matrix Representation

$$X = \sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (19)$$

This **matrix swaps** the $|0\rangle$ and $|1\rangle$ **basis states**:

$$X|0\rangle = |1\rangle \quad \text{and} \quad X|1\rangle = |0\rangle \quad (20)$$

≡ Action on a General Qubit

Let's consider a qubit in the general state:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

Applying the X gate:

$$X|\psi\rangle = aX|0\rangle + bX|1\rangle = a|1\rangle + b|0\rangle = b|0\rangle + a|1\rangle$$

As a result, the **amplitudes** a and b are **swapped**.

≡ Key Properties of X Gate

The Pauli-X gate satisfies several important mathematical properties:

1. **Unitary**. The X gate is unitary, meaning it preserves the inner product and is reversible:

$$XX^\dagger = I$$

2. **Hermitian**. The X gate is also Hermitian, which means it is equal to its own conjugate transpose:

$$X = X^\dagger$$

3. **Self-Inverse**. The X gate is its own inverse, which means that **applying it twice returns the qubit to its original state**:

$$X^{-1} = X \quad \Rightarrow \quad X^2 = I$$

🔗 Geometric Interpretation: Rotation on the Bloch Sphere

The Pauli-X gate corresponds to a **rotation around the x -axis by π radians** (180°) on the Bloch sphere, denoted as $X = R_x(\pi)$.

To understand this, recall the **general qubit state in spherical coordinates**:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle$$

After a π rotation around the x -axis:

- $\theta \rightarrow \pi - \theta$
- $\phi \rightarrow -\phi$

This **rotation mirrors the qubit across the x -axis**, flipping its position between $|0\rangle$ and $|1\rangle$, and adjusting the phase accordingly.

⚙️ Rotation Interpretation

From Equation 14 (page 50) and Equation 18 (page 50), a rotation around the x -axis by an angle θ is given by:

$$R_x(\theta) = \cos\left(\frac{\theta}{2}\right)I - i\sin\left(\frac{\theta}{2}\right)X$$

For $\theta = \pi$, we obtain:

$$R_x(\pi) = -iX$$

Therefore, the Pauli-X gate corresponds to a **rotation of π around the x -axis, up to a global phase**:

$$X \equiv R_x(\pi)$$

Since global phases are physically irrelevant, X and $R_x(\pi)$ represent the same physical transformation.

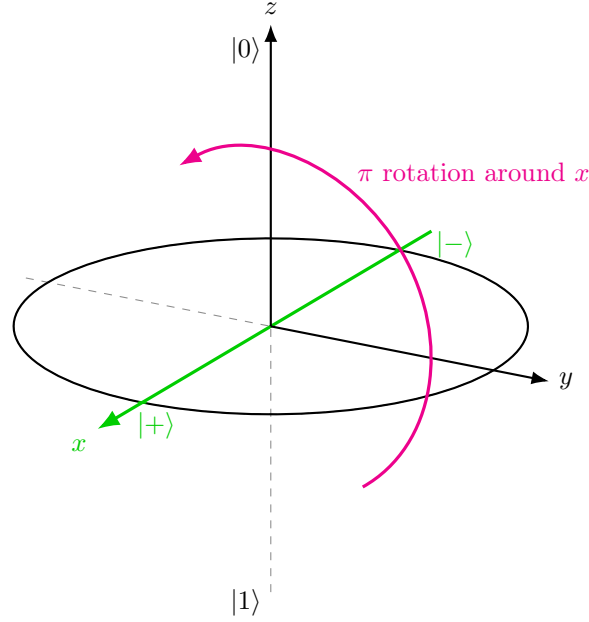


Figure 10: Eigenstates of the Pauli-X gate. Since $|+\rangle$ and $|-\rangle$ lie on the x -axis, they are not moved by a rotation around that axis. The state $|-\rangle$ only acquires a global phase -1 .

Eigenstates

In linear algebra, an **eigenvector** of a matrix (or linear operator) A is a non-zero vector $|v\rangle$ that, when the operator is applied, changes only by a **scalar factor** called the **eigenvalue** λ :

$$A|v\rangle = \lambda|v\rangle$$

In quantum mechanics, we use the term **Eigenstate** instead of eigenvector. So, an eigenstate of an operator U satisfies:

$$U|v\rangle = \lambda|v\rangle$$

An **eigenstate** is a state that remains **unchanged in direction** after the operation is applied, except only its **magnitude** or **phase** may change by a factor of λ . For the Pauli-X gate, solving the eigenvalue equation:

$$X|v\rangle = \lambda|v\rangle$$

Yields two eigenstates:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad \text{with eigenvalue} \quad +1 \Rightarrow X|+\rangle = +1 \cdot |+\rangle = |+\rangle \quad (21)$$

Called the **Plus State**, and:

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \quad \text{with eigenvalue} \quad -1 \Rightarrow X|-\rangle = -1 \cdot |-\rangle = -|-\rangle \quad (22)$$

Called the **Minus State**. These states are also known as **Hadamard basis states** (page 65), **X-basis states**, or **Eigenstates of the Pauli-X operator**.

Example 2: Measurement in the X-basis

For example, if a qubit is in the state $|x\rangle$ and we measure it in the X-basis, the outcome is 1 with probability 1.

Proof. We want to show that if the qubit is in the state:

$$|x\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

And we measure it in the X-basis $\{|+\rangle, |-\rangle\}$, then the **outcome is +1 with probability 1**.

Measuring in the X-basis means using the eigenstates of the Pauli-X operator:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

These correspond to the new possible outcomes: +1 for $|+\rangle$ and -1 for $|-\rangle$.

If the system is in state $|\psi\rangle$, the probability of obtaining the outcome corresponding to basis state $|u\rangle$ is:

$$P(u) = |\langle u|\psi\rangle|^2$$

In our case, the system is in state $|\psi\rangle = |+\rangle$, so:

- Probability of outcome +1 (measuring $|+\rangle$):

$$P(+1) = |\langle +|+\rangle|^2$$

- Probability of outcome -1 (measuring $|-\rangle$):

$$P(-1) = |\langle -|+\rangle|^2$$

Since every normalized state has inner product 1 with itself, we have:

$$\langle +|+\rangle = 1$$

Therefore:

$$P(+) = |\langle +|+\rangle|^2 = |1|^2 = 1$$

So the probability of getting +1 is 1.

If we calculate the probability of getting -1, first write the bras and kets:

$$|+\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle), \quad |-\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

Changing the ket $|+\rangle$ to a bra:

$$\langle -| = \frac{1}{\sqrt{2}} (\langle 0| - \langle 1|)$$

Now we can compute the inner product:

$$\begin{aligned}\langle -|+\rangle &= \frac{1}{\sqrt{2}} (\langle 0| - \langle 1|) \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \\&\downarrow \text{bring out the factors} \\&= \frac{1}{2} (\langle 0| - \langle 1|) (|0\rangle + |1\rangle) \\&\downarrow \text{expand the product} \\&= \frac{1}{2} (\langle 0|0\rangle + \langle 0|1\rangle - \langle 1|0\rangle - \langle 1|1\rangle) \\&\downarrow \text{use orthonormality } (\langle 0|0\rangle = \langle 1|1\rangle = 1, \langle 0|1\rangle = \langle 1|0\rangle = 0) \\&= \frac{1}{2} (1 + 0 - 0 - 1) \\&= 0\end{aligned}$$

Therefore:

$$P(-1) = |\langle -|+\rangle|^2 = |0|^2 = 0$$

So the probability of getting -1 is 0.

QED

3.3.2.3 Pauli-Z (Phase Flip) Gate

The **Pauli-Z Gate**, also called the **Phase Flip Gate**, is one of the core **Pauli operators** used in quantum computing. It acts **only on the phase** of the qubit, leaving the **$|0\rangle$ amplitude unchanged** but **flipping** the sign of the **$|1\rangle$ amplitude**.

√* Matrix Representation

$$Z = \sigma_Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (23)$$

≡ Action on a General Qubit

For a general state:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

Applying Z :

$$Z|\psi\rangle = aZ|0\rangle + bZ|1\rangle = a|0\rangle - b|1\rangle$$

As a result, Z **flips the sign of the $|1\rangle$ amplitude**, this is why it's called a **phase flip**. Finally, it's trivial to show that:

$$Z|0\rangle = |0\rangle \quad (\text{unchanged}), \quad Z|1\rangle = -|1\rangle \quad (\text{sign flip})$$

So the **magnitudes** of the amplitudes remain unchanged and the **probabilities of measurement outcomes** in the standard basis are unaffected, but the **relative phase** between $|0\rangle$ and $|1\rangle$ is altered (since b changes to $-b$).

✎ Example 3: Effect on Superposition States

The Hadamard (or X-basis) states are:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

These two states differ **only by their relative phase**:

- $|+\rangle$: relative phase 0.
- $|-\rangle$: relative phase π .

Applying Z on $|+\rangle$:

$$\begin{aligned} Z|+\rangle &= Z \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) \\ &= \frac{Z|0\rangle + Z|1\rangle}{\sqrt{2}} \\ &= \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\ &= |-\rangle \end{aligned}$$

Applying Z on $|-\rangle$:

$$\begin{aligned} Z|-\rangle &= Z \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right) \\ &= \frac{Z|0\rangle - Z|1\rangle}{\sqrt{2}} \\ &= \frac{|0\rangle + |1\rangle}{\sqrt{2}} \\ &= |+\rangle \end{aligned}$$

The Pauli-Z gate **does not change the probabilities** of measuring $|0\rangle$ or $|1\rangle$ in the computational basis. However, it **changes the relative phase**, transforming $|+\rangle$ into $|-\rangle$ and vice versa.

🔍 Geometric Interpretation: Rotation around the z -axis

On the Bloch sphere, the Pauli-Z gate performs a π radians (180°) rotation around the z -axis. In spherical coordinates:

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + e^{i\phi}\sin\left(\frac{\theta}{2}\right)|1\rangle$$

After Z rotation:

- The angle $\phi \rightarrow \phi + \pi$
- This phase shift **changes the sign of the complex amplitude for $|1\rangle$**

Since **measurement probabilities depend on $|a|^2$ and $|b|^2$** , which **remain unchanged**, the Z gate **does not affect measurement outcomes** in the standard basis. It only affects **interference and future gate interactions**.

⚙️ Rotation Interpretation

From Equation 14 (page 50) and Equation 18 (page 50), a rotation around the z -axis by an angle θ is given by:

$$R_z(\theta) = \cos\left(\frac{\theta}{2}\right)I - i\sin\left(\frac{\theta}{2}\right)Z$$

For $\theta = \pi$, we obtain:

$$R_z(\pi) = -iZ$$

Therefore, the Pauli-Z gate corresponds to a **rotation of π around the z -axis, up to a global phase**:

$$Z \equiv R_z(\pi)$$

Since global phases are physically irrelevant, Z and $R_z(\pi)$ represent the same physical transformation.

⚙️ Key Properties

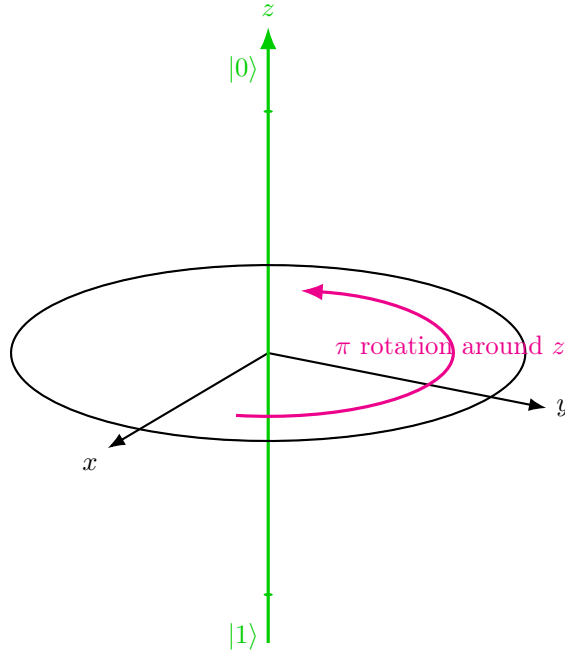


Figure 11: Eigenstates of the Pauli-Z gate. Since $|0\rangle$ and $|1\rangle$ lie on the z -axis, they are not moved by a rotation around that axis. The state $|1\rangle$ only acquires a global phase -1 .

1. **Unitary:** $Z^\dagger = Z^{-1} = Z$. Ensuring normalization is preserved.
2. **Hermitian:** $Z^\dagger = Z$. Meaning it is equal to its own conjugate transpose.
3. **Self-Inverse:** $Z^{-1} = Z \Rightarrow Z^2 = I$. Applying the Pauli-Z gate twice returns the original state.
4. **Eigenvalues and Eigenstates.** The Pauli-Z operator has *eigenvalues* $+1$ and -1 , with corresponding *eigenstates* $|0\rangle$ and $|1\rangle$:

$$Z|0\rangle = +1 \cdot |0\rangle, \quad Z|1\rangle = -1 \cdot |1\rangle$$

This means that $|0\rangle$ and $|1\rangle$ are **unchanged in direction** by the action of Z (up to a possible phase factor). In particular:

- $|0\rangle$ is completely unchanged.
- $|1\rangle$ acquires only a phase factor $-1 = e^{i\pi}$.

Therefore, these states are **invariant under the action of Z** (up to global phase), and represent the **natural states of the operator**.

💡 **Geometric Insight.** On the Bloch sphere, the Pauli-Z gate corresponds to a rotation around the z -axis. The eigenstates $|0\rangle$ and $|1\rangle$ lie exactly on this axis (north and south poles). As a consequence, they are **not affected by the rotation**, which explains why they are eigenstates of Z .

In contrast, any state that is not aligned with the z -axis (for example $|+\rangle$, Figure 9) is rotated by the action of Z . This highlights the difference between:

- **Eigenstates:** unchanged (up to phase)
- **Superposition states:** geometrically rotated

3.3.2.4 Pauli-Y Gate

The **Pauli-Y Gate**, denoted Y , is one of the three fundamental **Pauli matrices** in quantum mechanics. It is less intuitive than Pauli-X and Z, but equally important, especially for its role in complex rotations and quantum interference.

√* Matrix Representation

$$Y = \sigma_Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

≡ Action on a General Qubit

At first glance, this looks similar to the Pauli-X gate because it exchanges $|0\rangle$ and $|1\rangle$. But there is something extra: X swaps the two basis states (i.e., $|0\rangle \leftrightarrow |1\rangle$), while Y swaps them **and** multiplies them by a phases of $\pm i$. So Pauli-Y is not just a bit flip (like Pauli-X), it is a bit flip together with a phase modification.

Given a qubit in state:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

Apply the Y gate:

$$Y|\psi\rangle = aY|0\rangle + bY|1\rangle = a(-i|1\rangle) + b(i|0\rangle) = ib|0\rangle - ia|1\rangle$$

So the **amplitudes are exchanged** (as with Pauli-X), but each one also acquires a **phase factor of i or $-i$** .

🔍 Geometric Interpretation: Rotation on the Bloch Sphere

The Pauli-Y gate corresponds to a **rotation around the y -axis by π radians** (180°). The effect on Bloch sphere coordinates:

- $\theta \rightarrow \pi - \theta$
- $\psi \rightarrow \pi - \psi$

This rotates the qubit across the y -axis, **altering both amplitudes and their phases**.

⚙️ Rotation Interpretation

From Equation 14 (page 50) and Equation 18 (page 50), a rotation around the y -axis by an angle θ is given by:

$$R_y(\theta) = \cos\left(\frac{\theta}{2}\right) I - i \sin\left(\frac{\theta}{2}\right) Y$$

For $\theta = \pi$, we obtain:

$$R_y(\pi) = -iY$$

Therefore, the Pauli-Y gate corresponds to a **rotation of π around the y -axis, up to a global phase**:

$$Y \equiv R_y(\pi)$$

Since global phases are physically irrelevant, Y and $R_y(\pi)$ represent the same physical transformation.

☐ Pauli-Y as a Composite Gate

A very useful identity is:

$$Y = iXZ \quad (24)$$

This means that Pauli-Y can be understood as the combination of:

1. A Pauli-Z gate (phase flip);
2. Followed by a Pauli-X gate (bit flip);
3. Up to a global phase factor of i .

$$XZ = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$$

Now, multiply by i :

$$iXZ = i \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = Y$$

This identity confirms that **Pauli-Y can be viewed as a combination of Pauli-X and Pauli-Z with a global phase factor i** , which does not affect measurement outcomes.

☐ Eigenstates of Y Gate

To find the eigenstates, we solve:

$$Y |v\rangle = \lambda |v\rangle$$

Where λ is the eigenvalue. The result is:

$$|i\rangle = \frac{1}{\sqrt{2}} (|0\rangle + i|1\rangle) \quad (25)$$

$$|-i\rangle = \frac{1}{\sqrt{2}} (|0\rangle - i|1\rangle) \quad (26)$$

With eigenvalues:

$$Y |i\rangle = +1 |i\rangle \quad Y |-i\rangle = -1 |-i\rangle \quad (27)$$

⚡ Key Properties of Y Gate

1. **Unitary**: $Y^\dagger Y = I$. So it is a valid quantum gate that preserves normalization.
2. **Hermitian**: $Y^\dagger = Y$. So it is also an observable, meaning it corresponds to a measurable quantity in quantum mechanics.
3. **Self-inverse**: $Y^2 = I$. Applying the Y gate twice returns the qubit to its original state.

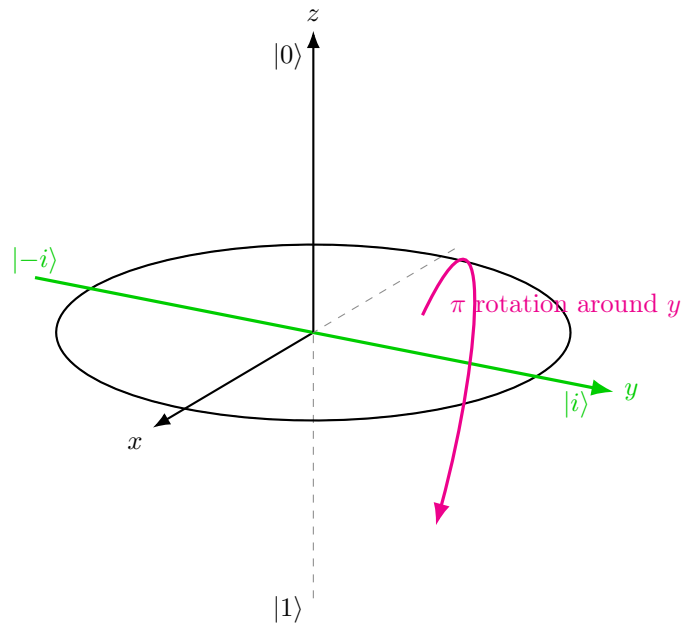


Figure 12: Eigenstates of the Pauli-Y gate. Since $|i\rangle$ and $|-i\rangle$ lie on the y -axis, they are not moved by a rotation around that axis. The state $|-i\rangle$ only acquires a global phase -1 .

3.3.3 Phase Gate (S)

The **Phase Gate (S)** is a single-qubit gate that **adds a phase shift of $\frac{\pi}{2}$ (90°) to the amplitude of the $|1\rangle$ state**. It **leaves $|0\rangle$ unchanged**, like the Pauli-Z gate, but instead of a sign flip (π phase), it **introduces a complex phase factor i** .

√ Matrix Representation

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \quad (28)$$

📖 Action on a General Qubit

For a state:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

Apply S :

$$S|\psi\rangle = a|0\rangle + ib|1\rangle$$

Only the $|1\rangle$ **amplitude gains a phase of i** , affecting interference but **not measurement probabilities** in the computational basis.

$$S|0\rangle = |0\rangle \quad (\text{unchanged}), \quad S|1\rangle = i|1\rangle \quad \left(\text{phase} + \frac{\pi}{2}\right)$$

🌀 Geometric Interpretation: Rotation around z -axis ($\frac{\pi}{2}$)

On the Bloch sphere, S performs a rotation of $\frac{\pi}{2}$ radians around the z -axis. This rotation **shifts the phase angle $\phi \rightarrow \phi + \frac{\pi}{2}$** , altering **how the qubit interferes** in later operations, especially when **Hadamard gates** are involved. In Figure 13, we see how S transforms the state $|+\rangle$ (on the x -axis) into $|i\rangle$ (on the y -axis) through a $\frac{\pi}{2}$ rotation around the z -axis, demonstrating the phase shift's effect on the qubit's state.

↔ Relationship to Pauli-Z

S is often called the **square root of Z** because applying it twice yields the Pauli-Z gate: $S = \sqrt{Z}$. Pauli-Z is a **full phase flip**:

$$e^{i\pi} = -1$$

The S gate is a **partial phase shift**:

$$e^{i\frac{\pi}{2}} = i$$

So:

$$Z = e^{i\pi \cdot |1\rangle\langle 1|} \quad \text{and} \quad S = e^{i\frac{\pi}{2} \cdot |1\rangle\langle 1|}$$

Same transformation, but different phase angles. Let's verify $S^2 = Z$:

$$S \cdot S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & i^2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = Z$$

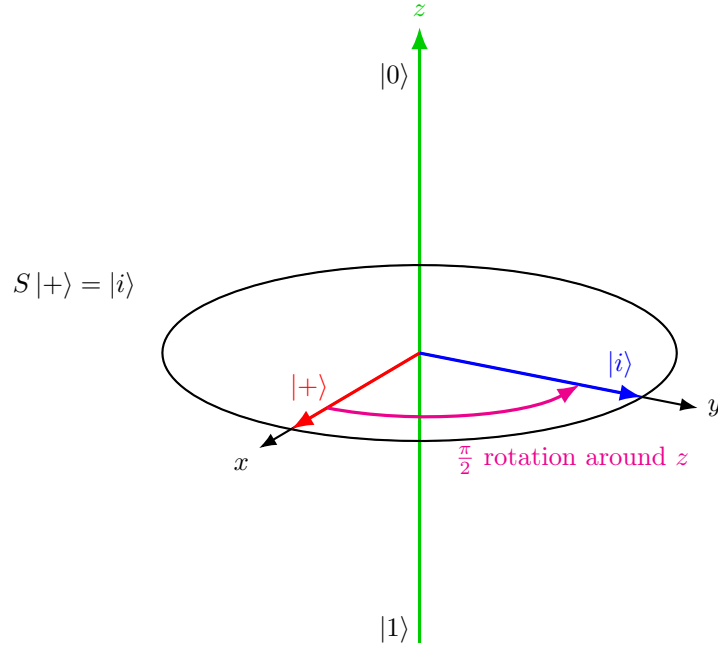


Figure 13: Geometric interpretation of the Phase gate S . The gate performs a rotation of $\frac{\pi}{2}$ around the z -axis, transforming $|+\rangle$ into $|i\rangle$.

Eigenvalues and Eigenstates

The Phase gate S is diagonal in the computational basis, so its eigenstates are $|0\rangle$ and $|1\rangle$:

$$S|0\rangle = 1 \cdot |0\rangle, \quad S|1\rangle = i \cdot |1\rangle$$

Thus, the eigenvalues of S are:

$$\lambda_0 = 1, \quad \lambda_1 = i$$

This means that $|0\rangle$ remains unchanged, while $|1\rangle$ acquires a phase factor $i = e^{i\frac{\pi}{2}}$.

Key Properties

1. **Unitary:** $S^\dagger = S^{-1}$, so normalization is preserved.
2. **Not Hermitian:** $S^\dagger \neq S$, therefore S is not an observable, it is only a transformation.
3. **Inverse:**

$$S^{-1} = S^\dagger = \begin{pmatrix} 1 & 0 \\ 0 & -i \end{pmatrix}$$
4. **Relation with Pauli-Z:** $S^2 = Z$, meaning that S is the square root of the Pauli-Z gate.
5. **Periodicity:** $S^4 = I$, since $i^4 = 1$.
6. **Phase Shift:** S applies a relative phase of $\frac{\pi}{2}$ to the $|1\rangle$ component.

3.3.4 Hadamard Gate (H)

The **Hadamard Gate (H)** changes the basis representation of the qubit. More concretely:

$$H : \{|0\rangle, |1\rangle\} \longleftrightarrow \{|+\rangle, |-\rangle\}$$

It maps computational basis states into equal superpositions and vice versa, allowing phase information to be transformed into measurable probabilities.

√* Matrix Representation

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad (29)$$

≡ Action on Basis States

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle \quad (30)$$

$$H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle \quad (31)$$

The $|+\rangle$ and $|-\rangle$ states are known as the **Hadamard basis**, or **superposition states**, with **equal probability amplitudes** for $|0\rangle$ and $|1\rangle$.

≡ Superposition and Measurement

After applying H to $|0\rangle$, the resulting qubit is:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

This state is a **superposition** of $|0\rangle$ and $|1\rangle$, meaning that the qubit does not have a definite classical value prior to measurement. Instead, it is described by **probability amplitudes** associated with each basis state.

Upon measurement in the computational basis, the state collapses probabilistically:

• **Probability of 0:**

• **Probability of 1:**

$$\left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} \qquad \frac{1}{2}$$

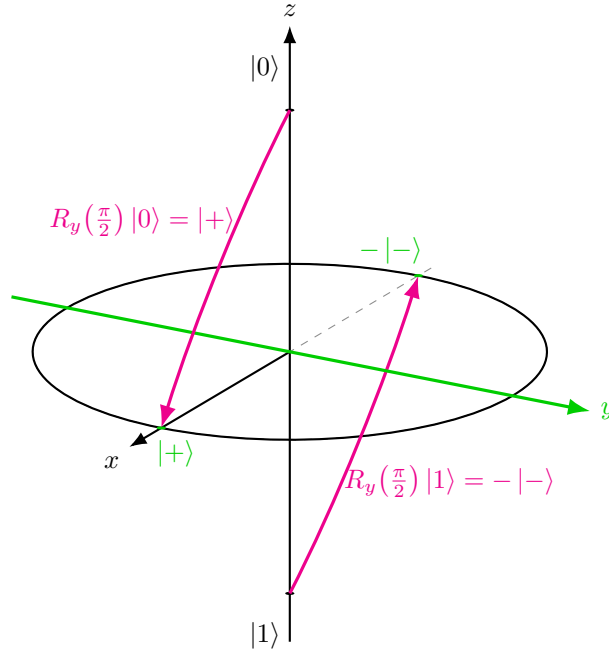
After the measurement:

- The qubit collapses to $|0\rangle$ with probability $\frac{1}{2}$, or to $|1\rangle$ with probability $\frac{1}{2}$.
- The original superposition is **destroyed**, and cannot be recovered.

🔍 Bloch Sphere Interpretation

On the Bloch sphere, the **Hadamard gate performs two sequential rotations**:

1. **Rotation around the y -axis by $\frac{\pi}{2}$ radians (90°): $R_y(\frac{\pi}{2})$.**



The y -axis stays fixed. So the states on the y -axis $|i\rangle$ and $|-i\rangle$ do not move. Instead, the z -axis rotated toward the x -axis: $|0\rangle \rightarrow |+\rangle$ and $|1\rangle \rightarrow -|-\rangle$. So the first step already moves the computational basis from the z -axis to the x axis.

We can verify the matrix representation of $R_y(\frac{\pi}{2})$ by using the exponential of the Pauli Y matrix (see page 50):

$$\begin{aligned}
 R_y(\theta) &= e^{-i\frac{\theta}{2}Y} \\
 &= \cos\left(\frac{\theta}{2}\right) I - i \sin\left(\frac{\theta}{2}\right) Y \\
 &= \cos\left(\frac{\theta}{2}\right) \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} - i \sin\left(\frac{\theta}{2}\right) \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \\
 &= \begin{pmatrix} \cos\left(\frac{\theta}{2}\right) & -\sin\left(\frac{\theta}{2}\right) \\ \sin\left(\frac{\theta}{2}\right) & \cos\left(\frac{\theta}{2}\right) \end{pmatrix}
 \end{aligned}$$

Now, substituting $\theta = \frac{\pi}{2}$ gives us the expected matrix for the first step of the Hadamard decomposition:

$$R_y\left(\frac{\pi}{2}\right) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

- Apply it to $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$:

$$\begin{aligned}
 R_y\left(\frac{\pi}{2}\right)|0\rangle &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\
 &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\
 &= \frac{1}{\sqrt{2}} \left(\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right) \\
 &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \\
 &= \frac{|0\rangle + |1\rangle}{\sqrt{2}} \\
 &= |+\rangle
 \end{aligned}$$

So $|0\rangle$, the north pole of the Bloch sphere, is rotated to the $+x$ -axis, which is the $|+\rangle$ state.

- Apply it to $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$:

$$\begin{aligned}
 R_y\left(\frac{\pi}{2}\right)|1\rangle &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} \\
 &= \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \end{pmatrix} \\
 &= \frac{1}{\sqrt{2}} \left(\begin{pmatrix} 0 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \end{pmatrix} \right) \\
 &= \frac{1}{\sqrt{2}} (|1\rangle - |0\rangle) \\
 &= -\frac{|0\rangle - |1\rangle}{\sqrt{2}} \\
 &= -|-\rangle
 \end{aligned}$$

This means that $|1\rangle$, the south pole of the Bloch sphere, moves to the $-x$ -axis, but with a global phase of -1 . Since global phases do not affect measurement outcomes, we can ignore it and say that $|1\rangle$ is effectively rotated to the $-x$ -axis, which is the $|-\rangle$ state. In simple terms: $-|-\rangle \equiv |-\rangle$ up to a global phase.

2. **Rotation around the x -axis by π radians (180°).** This step is represented by the operator $R_x(\pi)$, which can be expressed as:

$$R_x(\pi) = e^{-i\frac{\pi}{2}X} = \cos\left(\frac{\pi}{2}\right)I - i\sin\left(\frac{\pi}{2}\right)X = -iX$$

Applying $R_x(\pi)$ to the states on the x -axis ($|+\rangle$ and $|-\rangle$) does not change their position on the Bloch sphere, but it does introduce a phase factor

(which becomes a global phase only when applied to the entire state):

$$\begin{aligned} R_x(\pi) |+\rangle &= -iX |+\rangle = -i |+\rangle \\ R_x(\pi) |-\rangle &= -iX |-\rangle = -i(-|-\rangle) = i |-\rangle \end{aligned}$$

Applying $R_x(\pi)$ to the superposition state $a|+\rangle - b|-\rangle$ (obtained from the first rotation) gives us:

$$\begin{aligned} &a|+\rangle - b|-\rangle \\ R_x(\pi)(a|+\rangle - b|-\rangle) &= a(-i|+\rangle) - b(i|-\rangle) \\ &= -ia|+\rangle - ib|-\rangle \\ &= -i(a|+\rangle + b|-\rangle) \end{aligned}$$

Since global phases $-i$ do not affect measurement outcomes, we can ignore them and say that $|+\rangle$ and $|-\rangle$ remain on the x -axis. The second rotation does not change the position of the states on the Bloch sphere, but it is necessary to ensure that the transformation matches the Hadamard gate at the operator level, preserving the correct relative phase for all superpositions.

❓ Why is the second rotation necessary? We **cannot** simply change $-|-\rangle$ to $|-\rangle$ inside a superposition, because that would change only one component and therefore change the **relative phase**. The fix comes from applying $R_x(\pi)$. After the first rotation, we have:

$$a|+\rangle - b|-\rangle$$

Now, applying the Pauli-X operator using the rotation $R_x(\pi)$ (page 50):

$$R_x(\pi) = -iX = -i \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \text{with} \quad X|+\rangle = |+\rangle, \quad X|-\rangle = -|-\rangle$$

Applying $R_x(\pi)$ to the superposition gives us:

$$\begin{aligned} R_x(\pi)(a|+\rangle - b|-\rangle) &= -iX(a|+\rangle - b|-\rangle) \\ &= -i(aX|+\rangle - bX|-\rangle) \\ &= -i(a|+\rangle - b(-|-\rangle)) \\ &= -i(a|+\rangle + b|-\rangle) \end{aligned}$$

Now the factor $-i$ multiplies the **entire state**, so it is a global phase:

$$-i(a|+\rangle + b|-\rangle) \equiv a|+\rangle + b|-\rangle$$

So the important point is:

$$a|+\rangle - b|-\rangle \not\equiv a|+\rangle + b|-\rangle$$

But:

$$R_x(\pi)(a|+\rangle - b|-\rangle) = -i(a|+\rangle + b|-\rangle) \equiv a|+\rangle + b|-\rangle$$

💡 Insight. The second rotation **does not affect the geometric position** of the states, but **corrects the relative phase** structure, ensuring that the transformation is a valid unitary operator acting consistently on the entire Hilbert space.

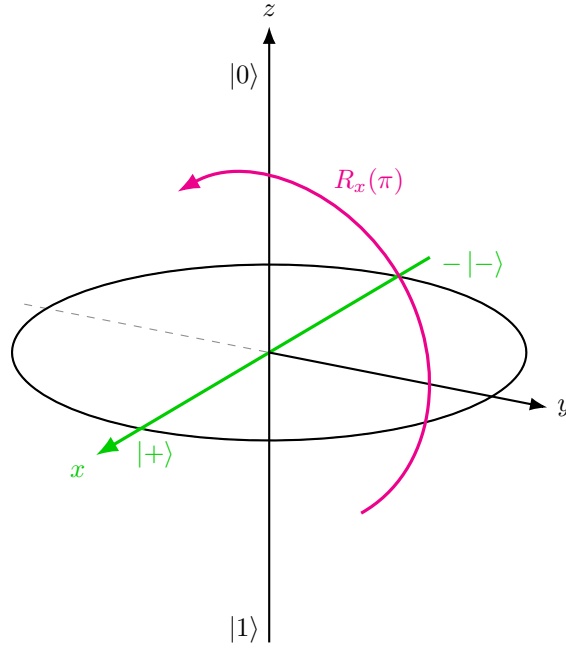


Figure 14: Second step of the Hadamard decomposition: rotation by π around the x -axis. States on the x -axis are not moved on the Bloch sphere; only global phases are introduced.

This brings $|0\rangle$ to the $+x$ -axis ($|+\rangle$) and $|1\rangle$ to the $-x$ -axis ($|-\rangle$). As a result, **qubit moves from pole to equator, entering maximum superposition.**

Eigenvalues and Eigenstates

The Hadamard operator has eigenvalues:

$$\lambda = +1, \quad \lambda = -1$$

with corresponding eigenstates given by the solutions of:

$$H|v\rangle = \lambda|v\rangle$$

These eigenstates are superpositions of $|0\rangle$ and $|1\rangle$, and are aligned along an axis halfway between the x and z directions on the Bloch sphere. Precisely, the eigenvectors are:

$$\begin{aligned} |v_+\rangle &= \cos\left(\frac{\pi}{8}\right)|0\rangle + \sin\left(\frac{\pi}{8}\right)|1\rangle \\ |v_-\rangle &= \sin\left(\frac{\pi}{8}\right)|0\rangle - \cos\left(\frac{\pi}{8}\right)|1\rangle \end{aligned}$$

Where $|v_+\rangle$ corresponds to the eigenvalue $+1$ and $|v_-\rangle$ corresponds to the eigenvalue -1 ; the cosine and sine factors ensure normalization of the eigenstates.

Unlike Pauli gates, the Hadamard eigenstates lie along a **diagonal axis between the computational basis z and the Hadamard basis x** , reflecting its role as a **basis transformation** rather than a simple bit or phase flip.

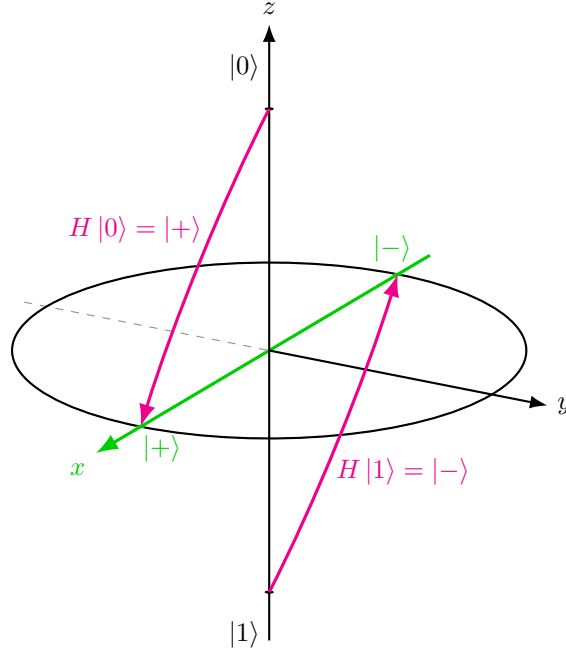


Figure 15: Bloch sphere interpretation of the Hadamard gate. The gate maps $|0\rangle$ to $|+\rangle$ and $|1\rangle$ to $|-\rangle$, moving computational basis states from the z -axis to the x -axis.

⌵ Key Properties

1. **Unitary:** $H^\dagger H = I$, so normalization is preserved.
2. **Hermitian:** $H^\dagger = H$, therefore it corresponds to an observable.
3. **Self-inverse:** $H^2 = I$. Applying the Hadamard gate twice returns the original state.
4. **Basis transformation:** H maps the computational basis $\{|0\rangle, |1\rangle\}$ to the Hadamard basis $\{|+\rangle, |-\rangle\}$:

$$H|0\rangle = |+\rangle, \quad H|1\rangle = |-\rangle$$

5. **Interference enabler:** H converts phase differences into amplitude differences, making them observable through measurement.
6. **Axis mixing:** H transforms Pauli operators as:

$$HZH = X, \quad HXH = Z$$

3.4 Properties

Single-qubit gates exhibit a rich set of mathematical and geometric properties, all of which stem from their nature as unitary transformations on a two-level quantum system. Understanding these properties is key to **predicting gate behavior**, **designing quantum circuits**, and **analyzing quantum algorithms**.

1. All Quantum Gates Are Equivalent to Rotations

A profound insight in quantum computing is that **every single-qubit unitary gate corresponds to a rotation of the qubit's state vector on the Bloch sphere**.

- **Bloch Sphere** is a geometric representation of a qubit's state, where any point on the sphere represents a valid pure state.
- A **unitary operation** on qubit results in **rotating the vector** on this sphere **without changing its length** (preserving probability).

Some examples of rotations:

- **X Gate** → Rotation π around x-axis (section 3.3.2.2, page 51)
- **Z Gate** → Rotation π around z-axis (section 3.3.2.3, page 56)
- **Y Gate** → Rotation π around y-axis (section 3.3.2.4, page 60)
- **S Gate** → Rotation $\frac{\pi}{2}$ around z-axis (section 3.3.3, page 63)
- **H Gate** → Rotation $\frac{\pi}{2}$ around y, then π around x (section 3.3.4, page 65)

❓ Why is this important? Quantum computation is fundamentally about **state rotations**, understanding gates as rotations helps visualize **interference**, **phase shifts**, and **entanglement creation**.

2. Reversibility: Applying a Gate Twice

Many single-qubit gates, especially the Pauli gates, exhibit the property of **involution**: applying the same gate twice **returns the qubit to its original state**. Mathematically it can be expressed as:

$$X^2 = Y^2 = Z^2 = I$$

This is due to the fact that **two 180° rotations around the same axis bring the vector back** to its original position on the Bloch sphere.

Proof that X Gate has involution.

$$X^2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

These identities confirm that **Pauli gates are their own inverses**, and **all unitary gates are reversible**. QED

3. Global Phases Do Not Matter

Quantum gates may **introduce a global phase** (a constant complex factor $e^{i\phi}$) to a state. Physically, **global phases are unobservable**, meaning they do **not affect measurement probabilities**. For example:

- States $|\psi\rangle$ and $e^{i\phi}|\psi\rangle$ are physically indistinguishable.
- The Y gate = iXZ , where i is a global phase that can be ignored in practice.

4. Composite Gates and Commutativity

- Gates can be **combined by matrix multiplication** (note: non commutative in general).
- **Order matters:** $AB \neq BA$ for most gates.
- Exception: **Gate Z and Gate S** are **commuted** because they are both **diagonal phase gates**.

Property	Pauli Gates (X, Y, Z)	Phase Gate (S)	Hadamard (H)
Rotation Axis	x, y, z (π radians)	z ($\frac{\pi}{2}$ radians)	y ($\frac{\pi}{2}$) + x (π)
Involution $G^2 = I$	✓	✗ ($S^2 = Z$)	✓
Self-adjoint $G = G^\dagger$	✓	✗	✓
Unitary	✓	✓	✓
Phase Introduction	Z, S	i for $ 1\rangle$	✓ (interference)
Creates Superposition?	✗ (X, Y, Z alone)	✗	✓

Table 2: Summary of Properties.

3.5 When Does a Gate Create Superposition?

In this section, we will answer the question: *when does a quantum gate create superposition?* More precisely, if we start from a **basis state** $|0\rangle$ or $|1\rangle$, when does a gate produce:

$$a|0\rangle + b|1\rangle \quad \text{with } a, b \neq 0?$$

? What does “*create superposition*” really mean? It means that the output must contain **both basis states**. So:

- $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$ is not a superposition, it's just $|0\rangle$.
- $\begin{pmatrix} 0 \\ 1 \end{pmatrix}$ is not a superposition, it's just $|1\rangle$.
- $\begin{pmatrix} a \\ b \end{pmatrix}$ with $a, b \neq 0$ is a superposition, because it contains both $|0\rangle$ and $|1\rangle$ components. Indeed, we can write it as:

$$a|0\rangle + b|1\rangle = a \begin{pmatrix} 1 \\ 0 \end{pmatrix} + b \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} a \\ b \end{pmatrix}$$

? How does a matrix act on a basis state?

The main idea is that **each column of the matrix tells us where a basis vector goes**. For example, for a matrix:

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$$

The first column $\begin{pmatrix} a_{11} \\ a_{21} \end{pmatrix}$ tells us where $|0\rangle$ goes, and the second column $\begin{pmatrix} a_{12} \\ a_{22} \end{pmatrix}$ tells us where $|1\rangle$ goes. In general, a **gate creates superposition if one of its columns has two non-zero entries**.

⚠ Constraint: Unitarity

Not every matrix can be a valid quantum gate. Quantum gates must be **unitary**, meaning:

$$A^\dagger A = I$$

This imposes strong constraints on the matrix entries. In particular, the columns must be:

- **Normalized**, total probability is 1: $\|\psi\|^2 = |a|^2 + |b|^2 = 1$
- **Orthogonal**, inner product is zero: $\langle v|w\rangle = 0$

As a consequence, some matrices that appear to create superposition are **not physically allowed**, because they are not unitary.

💡 Key Insight

Superposition arises from the **mixing of basis states**. Therefore:

- A gate **creates superposition** if at least one of its columns contains **two non-zero entries**.
- A gate **cannot create superposition** if its columns contain only one non-zero entry (as in diagonal or triangular matrices).

In particular:

- Gates like H (Hadamard) create superposition because they mix $|0\rangle$ and $|1\rangle$.
- Gates like X , Y , Z or S do not create superposition because they only modify phases and do not mix basis states.

Proof that A cannot create superposition. Consider the gate:

$$A = \begin{pmatrix} a_{11} & a_{12} \\ 0 & a_{22} \end{pmatrix}$$

To create superposition from a basis state, at least one column of A must contain two non-zero entries.

Applying A to $|0\rangle$ gives:

$$A|0\rangle = \begin{pmatrix} a_{11} & a_{12} \\ 0 & a_{22} \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} a_{11} \cdot 1 + a_{12} \cdot 0 \\ 0 \cdot 1 + a_{22} \cdot 0 \end{pmatrix} = \begin{pmatrix} a_{11} \\ 0 \end{pmatrix} = a_{11} |0\rangle$$

Therefore, $|0\rangle$ is not mapped to a superposition because the second entry is zero.

Applying A to $|1\rangle$ gives:

$$A|1\rangle = \begin{pmatrix} a_{11} & a_{12} \\ 0 & a_{22} \end{pmatrix} \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} a_{11} \cdot 0 + a_{12} \cdot 1 \\ 0 \cdot 0 + a_{22} \cdot 1 \end{pmatrix} = \begin{pmatrix} a_{12} \\ a_{22} \end{pmatrix} = a_{12} |0\rangle + a_{22} |1\rangle$$

This would be a superposition only if both $a_{12} \neq 0$ and $a_{22} \neq 0$.

However, since A is a quantum gate, it must be unitary (property of quantum gates):

$$A^\dagger A = I$$

Assuming A is real, we have $A^\dagger = A^T$, so:

$$A^T = \begin{pmatrix} a_{11} & 0 \\ a_{12} & a_{22} \end{pmatrix}$$

Hence:

$$A^T A = \begin{pmatrix} a_{11} & 0 \\ a_{12} & a_{22} \end{pmatrix} \cdot \begin{pmatrix} a_{11} & a_{12} \\ 0 & a_{22} \end{pmatrix} = \begin{pmatrix} a_{11}^2 & a_{11}a_{12} \\ a_{11}a_{12} & a_{12}^2 + a_{22}^2 \end{pmatrix}$$

Since A is unitary (i.e., $A^\dagger A = I$), this matrix must equal the identity:

$$\begin{pmatrix} a_{11}^2 & a_{11}a_{12} \\ a_{11}a_{12} & a_{12}^2 + a_{22}^2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

Therefore, to satisfy this equality, the following equations must hold:

$$\begin{cases} a_{11}^2 = 1 \\ a_{11}a_{12} = 0 \\ a_{12}^2 + a_{22}^2 = 1 \end{cases}$$

From $a_{11}^2 = 1$, we get $a_{11} = \pm 1$, so $a_{11} \neq 0$. Therefore, from $a_{11}a_{12} = 0$, it follows that:

$$a_{12} = 0$$

Substituting this into the last equation gives:

$$a_{22}^2 = 1 \quad \Rightarrow \quad a_{22} = \pm 1$$

Hence the matrix becomes diagonal:

$$A = \begin{pmatrix} \pm 1 & 0 \\ 0 & \pm 1 \end{pmatrix}$$

Since A is diagonal, it only modifies the phases of $|0\rangle$ and $|1\rangle$, without mixing them. Thus, when we apply A to $|0\rangle$ and $|1\rangle$, we get:

$$A|0\rangle = \pm|0\rangle, \quad A|1\rangle = \pm|1\rangle$$

So neither $|0\rangle$ nor $|1\rangle$ is mapped to a superposition, because the output states are just $|0\rangle$ or $|1\rangle$ (up to a global phase). Thus, A cannot create superposition (i.e., it cannot produce a state with both $|0\rangle$ and $|1\rangle$ components). QED

3.6 Single-Qubit Quantum Circuits

A **Quantum Circuit** is a **mathematical model** for quantum computation, **composed of sequential operations applied to qubits**. In other words, a quantum circuit is a **model of computation** that describes **how a quantum state evolves step by step**. The operations in a quantum circuit include initializations, unitary gates, and measurements. In this section, we focus on **circuits involving a single qubit**, which form the foundation for understanding multi-qubit systems.

≡ Circuit Elements

A quantum circuit typically consists of three main elements:

1. **Initialization** (initial state)
 - Qubit is prepared in a **known state**, typically $|0\rangle$ or $|1\rangle$.
 - Essential **starting point** for any computation.
2. **Gates**
 - Single-qubit **unitary operations** (e.g., X, Z, H, S).
 - Transform the qubit's state **reversibly**.
3. **Measurement**
 - **Irreversible process**.
 - Collapses the qubit to $|0\rangle$ or $|1\rangle$ with probabilities dictated by the quantum state's amplitudes.

✂ Quantum Circuit Diagram: Visual Language

- **Horizontal lines** represent the **timeline of a single qubit**.

$$|v_{\text{in}}\rangle \text{ ————— } |v_{\text{out}}\rangle$$

- **Boxes** on the line represent **gates** (letters) or **measurements** (tachometer icon).

$$|v_{\text{in}}\rangle \text{ — } \boxed{A} \text{ — } \boxed{B} \text{ — } \boxed{\dots} \text{ — } \boxed{N} \text{ — } \boxed{\text{tachometer icon}}$$

- **Double lines** (if present) represent **classical information** (e.g., measurement result).

$$|v_{\text{in}}\rangle \text{ — } \boxed{A} \text{ — } \boxed{B} \text{ — } \boxed{\text{tachometer icon}} = |v_{\text{out}}\rangle$$

- ⚠ **Left to right:** The circuit evolves over time from left (input) to right (output).

Gate Sequence: Serial Gates

Consider two gates A and B applied in sequence:

$$|v_{\text{in}}\rangle \longrightarrow \boxed{A} \longrightarrow \boxed{B} \longrightarrow |v_{\text{out}}\rangle$$

1. First apply A, then B.
2.  **Attention:** the **order of matrix multiplication is reversed** when we write the combined effect of the gates as a single matrix. The gate that is applied **last** (B) appears **first** in the product.

$$v_{\text{out}} = BAv_{\text{in}}$$

 **Why?** Because first apply A to the input state v_{in} , then apply B to the result of Av_{in} .

3. The **combined effect** of the two gates is equivalent to a single gate $C = BA$:

$$|v_{\text{in}}\rangle \longrightarrow \boxed{BA} \longrightarrow |v_{\text{out}}\rangle \quad \equiv \quad |v_{\text{in}}\rangle \longrightarrow \boxed{A} \longrightarrow \boxed{B} \longrightarrow |v_{\text{out}}\rangle$$

$$v_{\text{out}} = Cv_{\text{in}} = BAv_{\text{in}}$$

Obviously, if we merge the gates into a single gate C , we must respect the order of operations, which is why $C = BA$ and not AB .

Example 4: Hadamard followed by Hadamard

Let's compute H followed by H to $|0\rangle$:

$$|0\rangle \longrightarrow \boxed{H} \longrightarrow \boxed{H} \longrightarrow |v_{\text{out}}\rangle$$

1. Apply H:

$$\begin{aligned} H|0\rangle &= \frac{1}{\sqrt{2}} \cdot \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \cdot \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \cdot \left(\begin{pmatrix} 1 \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \right) \\ &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) = |+\rangle \end{aligned}$$

2. Applying H again, we **restore the original state due to the involution properties** ($H^2 = I$):

$$H(H|0\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle$$

3.7 Outer Product of Kets

In quantum mechanics, **Bra-Ket notation** (introduced by Dirac) is a concise and powerful tool for representing quantum states and operations. It uses **kets** $|\psi\rangle$ for **vectors** (states), and **bras** $\langle\psi|$ for their **Hermitian adjoints** (dual vectors).

Given two kets:

$$|a\rangle = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \quad |b\rangle = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

The **Outer Product** $|a\rangle\langle b|$ is a **matrix**, formed by:

$$|a\rangle\langle b| = |a\rangle \otimes \langle b| = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \begin{pmatrix} \overline{b_1} & \overline{b_2} \end{pmatrix} = \begin{pmatrix} a_1\overline{b_1} & a_1\overline{b_2} \\ a_2\overline{b_1} & a_2\overline{b_2} \end{pmatrix} \quad (32)$$

Where the symbol $\otimes$ is the **tensor product operator**. This operation **maps a vector to a matrix**.

❓ Why is the outer product useful? Because **all quantum gates can be written using outer products of kets**. For example, the **Pauli-X gate** can be expressed as:

$$X = |0\rangle\langle 1| + |1\rangle\langle 0| = \underbrace{\begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix}}_{\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}} + \underbrace{\begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix}}_{\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

The **Inner Product** $\langle b|a\rangle$ is a scalar:

$$\langle a|b\rangle = \begin{pmatrix} \overline{a_1} & \overline{a_2} \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \overline{a_1}b_1 + \overline{a_2}b_2 \quad (33)$$

So:

- **Inner product** $\rightarrow$ **projection**, result is number (scalar, see also page 12).
- **Outer product** $\rightarrow$ **matrix**, result is operator.

≡ Associativity between outer and inner products

A key property of the outer product is how it interacts with inner products. Specifically:

$$(|a\rangle\langle b|)|c\rangle = |a\rangle(\langle b|c\rangle) \quad (34)$$

This identity shows that the outer product acts as an operator: it first computes the inner product $\langle b|c\rangle$ (a scalar), and then scales the vector $|a\rangle$.

Proof the Associativity. In this proof, we will demonstrate that applying the outer product $|a\rangle\langle b|$ to a ket $|c\rangle$ is equivalent to scaling $|a\rangle$ by the inner product $\langle b|c\rangle$.

Let's define:

$$|a\rangle = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \quad |b\rangle = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \quad |c\rangle = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}$$

1. Compute **inner product** $\langle b|c\rangle$:

$$\langle b|c\rangle = \overline{b_1}c_1 + \overline{b_2}c_2 = \alpha$$

Where α is a **scalar** value.

2. Multiply **scalar** α with $|a\rangle$ (vector):

$$|a\rangle \alpha = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \alpha = \begin{pmatrix} a_1 \cdot \alpha \\ a_2 \cdot \alpha \end{pmatrix} = \begin{pmatrix} a_1 \cdot (\overline{b_1}c_1 + \overline{b_2}c_2) \\ a_2 \cdot (\overline{b_1}c_1 + \overline{b_2}c_2) \end{pmatrix}$$

3. Compute outer product $|a\rangle\langle b|$ (matrix):

$$|a\rangle\langle b| = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix} \begin{pmatrix} \overline{b_1} & \overline{b_2} \end{pmatrix} = \begin{pmatrix} a_1\overline{b_1} & a_1\overline{b_2} \\ a_2\overline{b_1} & a_2\overline{b_2} \end{pmatrix}$$

4. Multiply $|a\rangle\langle b|$ by $|c\rangle$:

$$(|a\rangle\langle b|)|c\rangle = \begin{pmatrix} a_1\overline{b_1} & a_1\overline{b_2} \\ a_2\overline{b_1} & a_2\overline{b_2} \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} a_1 \cdot (\overline{b_1}c_1 + \overline{b_2}c_2) \\ a_2 \cdot (\overline{b_1}c_1 + \overline{b_2}c_2) \end{pmatrix}$$

The second and fourth steps yield the same result. This proves that **outer product applied to a ket is equivalent to inner product $\langle b|c\rangle$ scaled by $|a\rangle$** . QED

3.8 Measurement

In quantum computing, **measurement** is a special type of operation that differs fundamentally from unitary gates. While gates perform reversible transformations, **measurement is irreversible**. It allows us to **gain information about the state of a qubit**, but at the price of **collapsing the quantum state**.

√* Measurement Operator: Matrix Form

A **measurement operator** is a **non-unitary, non-invertible** matrix. For a measurement **along a specific direction**, represented by a ket $|k\rangle$, the associated **projector** is (outer product):

$$M_k = |k\rangle\langle k| = \begin{pmatrix} k_1 \\ k_2 \end{pmatrix} \begin{pmatrix} \overline{k_1} & \overline{k_2} \end{pmatrix} = \begin{pmatrix} k_1\overline{k_1} & k_1\overline{k_2} \\ k_2\overline{k_1} & k_2\overline{k_2} \end{pmatrix} \quad (35)$$

This is called a **Projection Operator**, it projects any vector **onto the direction of** $|k\rangle$.

Definition 1: Projection Operator

A **Projection Operator** (or simply **Projector**) is a linear operator that **extracts the component of a quantum state along a specific direction** (or subspace).

For a normalized state $|k\rangle$, the projector onto $|k\rangle$ is defined as:

$$P_k = |k\rangle\langle k| \quad (36)$$

When applied to an arbitrary state

$$|v\rangle = \sum_j c_j |j\rangle,$$

The projector returns only the component of $|v\rangle$ parallel to $|k\rangle$:

$$P_k |v\rangle = (|k\rangle\langle k|) |v\rangle = \langle k|v\rangle |k\rangle$$

For example, let's suppose the state:

$$|v\rangle = a |k\rangle + b |k_\perp\rangle$$

Where $|k\rangle$ is the direction we want to keep and $|k_\perp\rangle$ is the direction we want to discard that is orthogonal to $|k\rangle$ (i.e., $\langle k|k_\perp\rangle = 0$). The projector P_k will keep the component $a |k\rangle$ and remove the component $b |k_\perp\rangle$:

$$\begin{aligned} P_k |v\rangle &= |k\rangle\langle k| (a |k\rangle + b |k_\perp\rangle) \\ &= a |k\rangle\langle k|k\rangle + b |k\rangle\langle k|k_\perp\rangle \\ &\downarrow |k\rangle \text{ is normalized, then } \langle k|k\rangle = 1 \\ &\quad \text{and } |k_\perp\rangle \text{ is orthogonal to } |k\rangle, \text{ then } \langle k|k_\perp\rangle = 0 \\ &= a |k\rangle \underbrace{\langle k|k\rangle}_{=1} + b |k\rangle \underbrace{\langle k|k_\perp\rangle}_{=0} \\ &= a |k\rangle + b |k\rangle \cdot 0 \\ &= a |k\rangle \end{aligned}$$

Therefore, a **projector acts as a filter**: it keeps the part of the state aligned with $|k\rangle$ and removes all orthogonal components. For example, suppose:

$$|v\rangle = 0.8|0\rangle + 0.6|1\rangle$$

Project onto $|0\rangle$:

$$|0\rangle\langle 0| |v\rangle = |0\rangle\langle 0| (0.8|0\rangle + 0.6|1\rangle) = 0.8|0\rangle$$

The $|1\rangle$ part disappears because it is orthogonal to $|0\rangle$, while the $|0\rangle$ part is preserved (but scaled by the coefficient 0.8).

Projection operators satisfy the **idempotence property**:

$$P_k^2 = P_k$$

Meaning that applying the same projector twice has the same effect as applying it once.

⚠ Effect of Measurement

Given a qubit $|\psi\rangle$, applying M_k yields the **(unnormalized) projected vector**:

$$|\psi_k\rangle = M_k |\psi\rangle = |k\rangle \langle k|\psi\rangle = (|k\rangle \langle k|) |\psi\rangle \quad (37)$$

To obtain the **new normalized state**, we **divide by the square root of the probability**:

$$|\psi_k^{\text{norm}}\rangle = \frac{M_k \cdot |\psi\rangle}{\sqrt{\langle \psi | M_k | \psi \rangle}} = \frac{\langle k|\psi\rangle \cdot |k\rangle}{\sqrt{|\langle k|\psi\rangle|^2}} = |k\rangle \quad (38)$$

So after measurement, the qubit **collapses** to $|k\rangle$, the **eigenstate corresponding to the measurement outcome**.

√ Why Normalization is Needed

After applying the measurement operator $M_k = |k\rangle \langle k|$ (Equation 35) to a state $|\psi\rangle$ (Equation 37), we obtain:

$$|\psi_k\rangle = M_k |\psi\rangle = |k\rangle \langle k|\psi\rangle$$

This vector is generally **not normalized**. Its norm is:

$$\| |\psi_k\rangle \|^2 = \langle \psi | M_k | \psi \rangle = |\langle k|\psi\rangle|^2 = p_k$$

Where p_k is the **probability of obtaining outcome k** .

🔍 Deepening: Why $\| |\psi_k\rangle \|^2 = p_k$?

We want to show:

$$\| |\psi_k\rangle \|^2 = \langle \psi | M_k | \psi \rangle = |\langle k|\psi\rangle|^2 = p_k$$

1. **Definition of squared norm.** The squared norm of a ket is:

$$\| |\psi_k\rangle \|^2 = \langle \psi_k | \psi_k \rangle$$

Since $|\psi_k\rangle = M_k |\psi\rangle$, we can write:

$$\| |\psi_k\rangle \|^2 = \underbrace{(M_k |\psi\rangle)^\dagger}_{\langle \psi_k |} \underbrace{(M_k |\psi\rangle)}_{|\psi_k\rangle}$$

We have only expanded the definition of the squared norm and substituted $|\psi_k\rangle$.

2. **Take the dagger $\dagger$.** The dagger of a product of operators is the product of the daggers in reverse order:

$$(AB)^\dagger = B^\dagger A^\dagger$$

Applying this to $(M_k |\psi\rangle)^\dagger$:

$$(M_k |\psi\rangle)^\dagger = \langle \psi | M_k^\dagger$$

And substituting back into the expression for the squared norm:

$$\| |\psi_k\rangle \|^2 = \langle \psi | M_k^\dagger M_k |\psi\rangle$$

3. **Use the Hermitian property of M_k** (explained later page 84). Since M_k is Hermitian, we have $M_k^\dagger = M_k$. Therefore:

$$\| |\psi_k\rangle \|^2 = \langle \psi | M_k M_k |\psi\rangle$$

4. **Use the idempotent property of M_k** (explained later page 84). Since M_k is idempotent, we have $M_k^2 = M_k$. Therefore:

$$\| |\psi_k\rangle \|^2 = \langle \psi | M_k |\psi\rangle$$

5. **Substitute the definition of M_k .** Since $M_k = |k\rangle \langle k|$, we can write:

$$\| |\psi_k\rangle \|^2 = \langle \psi | (|k\rangle \langle k|) |\psi\rangle = \langle \psi | k \rangle \langle k | \psi \rangle$$

6. **Use the property of inner products.** The inner product $\langle \psi | k \rangle$ is the complex conjugate of $\langle k | \psi \rangle$:

$$\langle \psi | k \rangle = \overline{\langle k | \psi \rangle}$$

Therefore (page 10):

$$\| |\psi_k\rangle \|^2 = \overline{\langle k | \psi \rangle} \langle k | \psi \rangle = |\langle k | \psi \rangle|^2 = p_k$$

This is exactly the probability p_k of obtaining outcome k when measuring the state $|\psi\rangle$ with the measurement operator M_k .

⚠ Important. The result of a measurement must be a valid quantum state, and therefore it must be **normalized** (i.e., have unit norm).

To restore normalization, we divide the projected vector by its norm:

$$|\psi_k^{\text{norm}}\rangle = \frac{|\psi_k\rangle}{\| |\psi_k\rangle \|} = \frac{|k\rangle \langle k|\psi\rangle}{\sqrt{|\langle k|\psi\rangle|^2}}$$

Observe that:

$$\frac{\langle k|\psi\rangle}{|\langle k|\psi\rangle|}$$

Is a complex number of magnitude 1, i.e., a **global phase**. **Since global phases do not affect physical states**, we conclude:

$$|\psi_k^{\text{norm}}\rangle = |k\rangle$$

💡 Interpretation Measurement can be understood as a two-step process:

1. **Projection:** extract the component of $|\psi\rangle$ along $|k\rangle$ (this determines the probability p_k).
2. **Renormalization:** rescale the result to obtain a valid quantum state.

📊 Probability of Outcome

The **probability of measuring the qubit in the state $|k\rangle$** is:

$$p_k = \langle \psi | M_k | \psi \rangle = |\langle k | \psi \rangle|^2 = \underbrace{|\overline{k}_1 \cdot \psi_1 + \overline{k}_2 \cdot \psi_2|^2}_{|\langle k | \psi \rangle|^2 \text{ expanded}} \quad (39)$$

This is the **Born rule**, a fundamental postulate of quantum mechanics. It states that the probability of obtaining a specific measurement outcome is given by the **squared magnitude of the inner product between the state being measured and the eigenstate corresponding to that outcome**. In other words, it quantifies how much the state $|\psi\rangle$ (*the state being measured*) overlaps with the state $|k\rangle$ (*the eigenstate corresponding to the outcome*). The more $|\psi\rangle$ is aligned with $|k\rangle$, the higher the probability of measuring k . Conversely, if $|\psi\rangle$ is orthogonal to $|k\rangle$, the probability of measuring k is zero. For example:

$$p_0 = \langle \psi | M_0 | \psi \rangle = |\langle 0 | \psi \rangle|^2 = |1 \cdot \psi_1 + \cancel{0 \cdot \psi_2}| = |\psi_1|$$

$$p_1 = \langle \psi | M_1 | \psi \rangle = |\langle 1 | \psi \rangle|^2 = |\cancel{0 \cdot \psi_1} + 1 \cdot \psi_2| = |\psi_2|$$

These probabilities measure the likelihood of obtaining the outcomes 0 and 1 when measuring the qubit $|\psi\rangle$ in the computational basis.

💬 Remark: $\langle \psi | M_k | \psi \rangle$

This is called a quadratic form in linear algebra, and in quantum mechanics it represents the **probability of the measurement outcome**

k when **measuring the qubit** $|\psi\rangle$ with **measurement operator** M_k .
Suppose a qubit ψ :

$$|\psi\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \quad \text{with } |a|^2 + |b|^2 = 1 \text{ (normalization condition)}$$

If we apply the measurement operator M_0 :

$$M_0 |\psi\rangle = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} a \\ 0 \end{pmatrix}$$

And we finally calculate the bra $\langle\psi|$:

$$\langle\psi| \begin{pmatrix} a \\ 0 \end{pmatrix} = (\bar{a} \quad \bar{b}) \begin{pmatrix} a \\ 0 \end{pmatrix} = \bar{a} \cdot a = |a|^2$$

In other words, this is the probability of measuring 0: p_0 .

≡ Standard Basis Measurement

In quantum computing, we usually measure in the **computational basis** $\{|0\rangle, |1\rangle\}$. The **projectors** are:

$$M_0 = |0\rangle\langle 0| = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \quad M_1 = |1\rangle\langle 1| = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \quad (40)$$

For a **general qubit** $|\psi\rangle = a|0\rangle + b|1\rangle$, the **probabilities** are:

$$p_0 = |a|^2 \quad p_1 = |b|^2 \quad (41)$$

After measurement, the qubit **collapses to** $|0\rangle$ **with probability** $|a|^2$, or to $|1\rangle$ **with probability** $|b|^2$.

☆ Key Properties of Measurement Operator

- **Hermitian**: $M_k^\dagger = M_k$, which means that M_k **is equal to its conjugate transpose**. This implies that M_k has real eigenvalues and orthogonal eigenvectors.

$$\begin{aligned} M_k^\dagger &= (|k\rangle\langle k|)^\dagger \\ &\downarrow \text{ use the rule } (AB)^\dagger = B^\dagger A^\dagger \\ &= (\underbrace{|k\rangle}_A \underbrace{\langle k|}_B)^\dagger \\ &= (\langle k|)^\dagger (|k\rangle)^\dagger \\ &= |k\rangle\langle k| \\ &= M_k \end{aligned}$$

- **Idempotent**: once applied, **applying it again does nothing**:

$$M_k^2 = M_k$$

A simple proof:

$$M_k^2 = |k\rangle |k\rangle \langle k| \langle k| = |k\rangle \langle k| = M_k$$

This reflects that **after projection, the qubit is already in the state $|k\rangle$.**

- **Non-unitary** and **Non-reversible**: measurement destroys superposition and cannot be undone.

4 Multiple Qubit Gates

4.1 Multiple Qubit States

Now that we have a solid understanding of single qubits and their states, we can transition to discussing many qubits as one joint system, rather than just one vector with one qubit.

With **one qubit**, life is simple because the state of the system can be described by a single vector in a two-dimensional complex vector space:

$$|v\rangle = a|0\rangle + b|1\rangle$$

With **two qubits**, something radical happens. We can no longer describe the system by looking at each qubit separately. Instead, we must describe **one global state** that captures the behavior of both qubits together. For example:

$$|v\rangle = c_0|00\rangle + c_1|01\rangle + c_2|10\rangle + c_3|11\rangle$$

This is not just a simple combination of two single-qubit states.

⚠ Exponential Explosion. The first thing to notice is that the number of amplitudes needed to describe the state of a system grows exponentially with the number of qubits.

- For 1 qubit, we need 2 amplitudes: a and b .
- For 2 qubits, we need 4 amplitudes: c_0 , c_1 , c_2 , and c_3 .
- For 3 qubits, we need 8 amplitudes: d_0 , d_1 , d_2 , d_3 , d_4 , d_5 , d_6 , and d_7 .
- For n qubits, we need 2^n amplitudes to fully describe the state of the system.

This is why quantum computing is so powerful: it can **process many possibilities at the same time** and **manipulate them so that only the correct ones survive**. This will become clear when we introduce entanglement (page 95).

📖 The Key New Phenomenon: Entanglement

Entanglement is the key new phenomenon when dealing with multiple qubits. It refers to quantum states that cannot be expressed as a tensor product of individual qubit states. For example, the state:

$$\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Is an entangled state. In such cases, the system must be described globally, and it is not possible to assign independent quantum states to each qubit.

If one qubit is measured, the outcome is random, but it determines the outcome of the other qubit due to the structure of the global state. This leads to strong

correlations between measurements results. However, this does not imply any form of information transfer between the qubits.

Entanglement is a fundamental resource in quantum computing, enabling correlations that have no classical counterpart, especially when combined with interference effects in quantum algorithms.

- Entanglement does not mean that one qubit sends information to the other. The observed correlations arise from the collapse of a **global quantum state**, not from communication between subsystems.
- Before measurement, individual qubits in an entangled state do **not have a well-defined state**. Only the joint system is well-defined.
- Each qubit individually appears **random**, but measurement outcomes are **correlated**.
- Entanglement alone is not the source of quantum computational power; it becomes powerful when combined with **interference** and quantum operations.

⚙️ New tools we need

To handle this new world of multiple qubits, we need new mathematical tools. We will introduce the **tensor product**, which allows us to construct the joint state of multiple qubits; we will also introduce the **multi-qubit gates**, which are operations that can manipulate the joint state of multiple qubits simultaneously; and we will discuss how to **measure** multiple qubits and understand the resulting probabilities.

4.1.1 Tensor Product

Let's start with something simple. As in previous sections, we have a simple one-qubit state. Now, we want to **add a second qubit to the system** to create a multi-qubit state. For simplicity, we will use only two qubits. *How do we describe the **combined system**?* Simply keeping them separate is not enough because we would miss correlations between the two qubits. **We need a way to combine the two qubits into a single object.** The tool for doing so is the **tensor product**.

Example 1: Dice Analogy

Imagine we have two dice. Each die can be in one of six states (1, 2, 3, 4, 5, or 6). If we want to describe the combined state of both dice, we can't just list their individual states separately. Instead, we need to consider all possible combinations of their states. The tensor product allows us to create a new space that includes all these combinations, giving us a complete description of the system.

The **Tensor Product** is a **mathematical operation** that **takes two vectors** (or more generally, two mathematical objects):

$$|v_A\rangle = a_0 |0\rangle + a_1 |1\rangle \quad |v_B\rangle = b_0 |0\rangle + b_1 |1\rangle$$

And **produces a new vector** that represents the combined state of the two systems:

$$\begin{aligned} |v_A\rangle \otimes |v_B\rangle &= (a_0 |0\rangle + a_1 |1\rangle) \otimes (b_0 |0\rangle + b_1 |1\rangle) \\ &= a_0 b_0 |00\rangle + a_0 b_1 |01\rangle + a_1 b_0 |10\rangle + a_1 b_1 |11\rangle \end{aligned} \quad (42)$$

Here, the resulting state $|v_A\rangle \otimes |v_B\rangle$ is a **superposition of all possible combinations of the states of the two qubits**.

❓ Why amplitudes multiply? For example, why is the amplitude of $|00\rangle$ equal to $a_0 b_0$? A qubit is a vector:

$$|v_A\rangle = \begin{pmatrix} a_0 \\ a_1 \end{pmatrix}, \quad |v_B\rangle = \begin{pmatrix} b_0 \\ b_1 \end{pmatrix}$$

The tensor product is **defined** as:

$$|v_A\rangle \otimes |v_B\rangle = \begin{pmatrix} a_0 b_0 \\ a_0 b_1 \\ a_1 b_0 \\ a_1 b_1 \end{pmatrix} \quad (43)$$

So amplitudes multiply because **that's how the tensor product is defined**. **But why is it defined like that?** Because we want a rule that satisfies two requirements:

1. **Linearity.** Quantum mechanics is linear:

$$(a_0 |0\rangle + a_1 |1\rangle) \otimes |v_B\rangle = a_0 (|0\rangle \otimes |v_B\rangle) + a_1 (|1\rangle \otimes |v_B\rangle)$$

The tensor product must be **bilinear**, i.e., linear in each argument, so that superpositions are preserved when combining quantum states.

2. Basis consistency:

$$|0\rangle \otimes |0\rangle = |00\rangle, \quad |0\rangle \otimes |1\rangle = |01\rangle, \quad |1\rangle \otimes |0\rangle = |10\rangle, \quad |1\rangle \otimes |1\rangle = |11\rangle$$

The tensor product must map basis states of individual systems to a consistent basis of the joint system (e.g., $|i\rangle \otimes |j\rangle = |ij\rangle$), preserving the structure of the state space.

This structure ensures that probabilities factorize correctly for independent systems.

💬 Remark: Inner Product of Tensor Products

For composite quantum systems (i.e., systems composed of multiple qubits), the **inner product between tensor product states factorizes**:

$$(\langle a| \otimes \langle b|) (|c\rangle \otimes |d\rangle) = \langle a|c\rangle \cdot \langle b|d\rangle \quad (44)$$

This means that the “similarity” (overlap) between two composite states is the product of the similarities of their individual subsystems (page 12).

For example:

$$(\langle 0| \otimes \langle +|) (|0\rangle \otimes |- \rangle)$$

Has overlap:

$$\langle 0|0\rangle \langle +|- \rangle = 1 \cdot 0 = 0$$

So even though the first qubit is identical, the total states are orthogonal because the second qubits are orthogonal!

💡 **Intuition.** A tensor product state combines independent subsystems. Therefore, the total overlap must take into account the overlap of *each* subsystem simultaneously. **If one subsystem is perfectly distinguishable (overlap 0), then the entire composite states become perfectly distinguishable.**

⚠️ Not all states can be factored

Given two qubits, their joint state lives in a 4-dimensional vector space. However, **not all states** in this 4-dimensional space can be written as a tensor product of two single-qubit states. Precisely, if a multi-qubit state can be written as a **tensor product**, then it is **separable**, and we can recover the individual qubit states. But if it is **not a tensor product**, then it is **entangled**, and we cannot assign well-defined quantum states to each qubit individually.

In general, exists a **necessary (and sufficient) condition** for being a tensor product for two qubits. Given a 2-qubit state:

$$|\psi\rangle = (c_0, c_1, c_2, c_3)$$

Where c_0, c_1, c_2, c_3 are the amplitudes of the basis states $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, respectively. The state $|\psi\rangle$ can be written as a tensor product of two single-qubit states if and only if:

$$c_0 c_3 = c_1 c_2 \quad (45)$$

Equivalently, if we reshape the coefficients into a matrix

$$C = \begin{pmatrix} c_0 & c_1 \\ c_2 & c_3 \end{pmatrix}$$

The state is separable if and only if $\det(C) = 0$. More generally, for n -qubit states, there is **no single simple scalar condition** that characterizes separability. A state is separable **if and only if it can be written as a tensor product of n single-qubit states**, which is a more complex condition to verify. So, the **2-qubit case is simpler to analyze**, but for larger systems, determining whether a state is separable or entangled can be more challenging.

Let's make this more concrete with two examples:

✓ Let's start from a 4D vector:

$$|\psi\rangle = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \end{pmatrix} = \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

We can check if it is a tensor product by verifying the condition in Equation 45:

$$c_0 c_3 = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}, \quad c_1 c_2 = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$$

Since $c_0 c_3 = c_1 c_2$, the state is a tensor product and thus separable.

Now, we can find the two single-qubit states whose tensor product gives us $|\psi\rangle$. We try:

$$|v_A\rangle = \begin{pmatrix} a_0 \\ a_1 \end{pmatrix}, \quad |v_B\rangle = \begin{pmatrix} b_0 \\ b_1 \end{pmatrix}$$

We want:

$$(a_0 b_0, a_0 b_1, a_1 b_0, a_1 b_1) = \left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2} \right) \Rightarrow \begin{cases} a_0 b_0 = \frac{1}{2} \\ a_0 b_1 = \frac{1}{2} \\ a_1 b_0 = \frac{1}{2} \\ a_1 b_1 = \frac{1}{2} \end{cases}$$

Solving this system of equations, we find:

$$a_0 = b_0 = \frac{1}{\sqrt{2}}, \quad a_1 = b_1 = \frac{1}{\sqrt{2}}$$

And this works because:

$$\begin{cases} a_0 b_0 = \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = \frac{1}{2} \\ a_0 b_1 = \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = \frac{1}{2} \\ a_1 b_0 = \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = \frac{1}{2} \\ a_1 b_1 = \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = \frac{1}{2} \end{cases}$$

Therefore:

$$|\psi\rangle = \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right] \otimes \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right]$$

The global 2-qubit state can be written as **two independent qubit states**, confirming that it is separable.

$$|\psi\rangle = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \\ \frac{1}{2} \end{pmatrix} = \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right] \otimes \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right]$$

✗ Let's start from a different 4D vector:

$$|\psi\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

We check the condition for separability:

$$c_0 c_3 = \frac{1}{2}, \quad c_1 c_2 = 0$$

Since $c_0 c_3 \neq c_1 c_2$, the state is not a tensor product and thus is **not separable**. This state is an example of an **entangled state** (**Entanglement** phenomenon), where the two qubits are correlated in such a way that we cannot describe them independently. We will see more about entanglement in the next section, but for now, the key takeaway is that not all multi-qubit states can be factored into tensor products of single-qubit states, and this is a fundamental aspect of quantum mechanics that leads to phenomena like entanglement.

In summary, the tensor product generates only a subset of the full 4-dimensional space of two-qubit states. Separable states have a special multiplicative structure and lie on a lower-dimensional manifold within this space. Most states in the full space do not satisfy this structure and are therefore entangled.

4.1.2 Building a Two-Qubit System

In the previous sections, we have seen how to represent the state of a single qubit using a two-dimensional complex vector. Now, we want to extend this representation to a system of two qubits. To do this, we will use the concept of the tensor product seen in the previous section.

We start from two independent qubits:

$$|v_A\rangle = a_0 |0\rangle + a_1 |1\rangle \quad \text{and} \quad |v_B\rangle = b_0 |0\rangle + b_1 |1\rangle$$

And our **goal is to find a state that encodes all joint possibilities** of the two qubits:

- Both qubits are in state $|0\rangle$: $|00\rangle$
- The first qubit is in state $|0\rangle$ and the second in state $|1\rangle$: $|01\rangle$
- The first qubit is in state $|1\rangle$ and the second in state $|0\rangle$: $|10\rangle$
- Both qubits are in state $|1\rangle$: $|11\rangle$

To represent these joint possibilities, we will use the tensor product of the two qubits' states:

$$|v_A v_B\rangle = |v_A\rangle \otimes |v_B\rangle$$

Expanding this expression, we get:

$$\begin{aligned} |v_A v_B\rangle &= (a_0 |0\rangle + a_1 |1\rangle) \otimes (b_0 |0\rangle + b_1 |1\rangle) \\ &= a_0 b_0 |00\rangle + a_0 b_1 |01\rangle + a_1 b_0 |10\rangle + a_1 b_1 |11\rangle \end{aligned} \quad (46)$$

This state $|v_A v_B\rangle$ is a linear combination of the four possible joint states of the two qubits. Each term in the expansion corresponds to one of the joint possibilities, and the coefficients $a_0 b_0$, $a_0 b_1$, $a_1 b_0$, and $a_1 b_1$ represent the probability amplitudes of each joint state.

We are no longer describing two systems separately, we are describing **one system with 4 possible outcomes**.

General Two-Qubit State

In Equation 46 we built a two-qubit state starting from a **very specific structure**, where the coefficients of the state were products of the coefficients of the individual qubits (i.e., $a_i b_j$). However, we can also consider a more **general two-qubit state**, where the coefficients are not necessarily products of the individual qubits' coefficients:

$$|v\rangle = c_0 |00\rangle + c_1 |01\rangle + c_2 |10\rangle + c_3 |11\rangle \quad (47)$$

Where c_0 , c_1 , c_2 , and c_3 are **arbitrary complex numbers** that represent the probability amplitudes of each joint state.

Obviously, this general two-qubit state **must satisfy the normalization condition**:

$$|c_0|^2 + |c_1|^2 + |c_2|^2 + |c_3|^2 = 1 \quad (48)$$

This means that the sum of the probabilities of all joint states must equal 1, ensuring that the state is properly normalized.

We can also generalize deeper to n qubits, where the state of the **system is a linear combination** of 2^n **basis states**, and the **coefficients** are arbitrary **complex numbers** that satisfy the normalization condition. Mathematically, we can express this as:

$$|v\rangle = \sum_{k=0}^{2^n-1} c_k |k\rangle \quad (49)$$

Where $|k\rangle$ represents the basis states of the n -qubit system, and c_k are the corresponding probability amplitudes. As we have already discussed at page 86, this general form **illustrates the exponential growth of the state space as the number of qubits increases**. However, we can write our one- and two-qubit states as special cases from this general form:

- For $n = 1$ qubit, we have $2^1 = 2$ basis states: $|0\rangle$ and $|1\rangle$, and the state can be written as:

$$|v\rangle = c_0 |0\rangle + c_1 |1\rangle$$

- For $n = 2$ qubits, we have $2^2 = 4$ basis states: $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$, and the state can be written as:

$$|v\rangle = c_0 |00\rangle + c_1 |01\rangle + c_2 |10\rangle + c_3 |11\rangle$$

In summary, a general two-qubit state is a normalized superposition (i.e., the *coefficients satisfy the normalization condition*) of the four basis states (i.e., $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$). Unlike tensor product states, its coefficients are arbitrary and *may not* factorize, leading to entanglement and non-classical correlations between the qubits (like the state in Example at page 91).

4.1.3 Separable vs Entangled States

Let the general state of a two-qubit system be (as defined in Equation 47, page 92):

$$|v\rangle = c_0 |00\rangle + c_1 |01\rangle + c_2 |10\rangle + c_3 |11\rangle$$

The question is: *can we interpret this as two independent qubits?* The answer is **not always**. As we saw on page 88, the answer depends on whether the states are separable or entangled.

☐ Separable States

A state is **separable** if it can be written as a tensor product of two **single-qubit states**. In other words, there exist single-qubit states $|v_A\rangle$ and $|v_B\rangle$ such that:

$$|v\rangle = |v_A\rangle \otimes |v_B\rangle$$

So the system can be split into one state for qubit A and one state for qubit B , and the global state is just the combination of these two states. As consequence, the coefficients factorize as:

$$c_0 = a_0 b_0, \quad c_1 = a_0 b_1, \quad c_2 = a_1 b_0, \quad c_3 = a_1 b_1$$

☐ Entangled States

A state is **entangled** if it cannot be written as a tensor product of two **single-qubit states**. In other words, there do not exist single-qubit states $|v_A\rangle$ and $|v_B\rangle$ such that:

$$|v\rangle \neq |v_A\rangle \otimes |v_B\rangle$$

In this case, the state cannot be separated into two independent qubits, and the coefficients do not factorize as above. *The system is simply one single entity.* For example, the Bell state (we will see in future sections, page 118) is an entangled state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

And the demonstration was written on page 91. In other words, we can only say that the system is in state $|\Phi^+\rangle$, but we cannot say that qubit A is in some state and qubit B is in some state, because the state of the system does not factorize into two independent states.

At this stage, entanglement is introduced as a structural property of quantum states. Its physical meaning and implications become clear only when studying measurement and quantum circuits in later sections.

🔗 Difference. With **separable** states, the *information is stored in the individual qubits*. In contrast, with **entangled** states, the *information is stored in the correlations between the qubits*.

4.1.4 Why Entanglement Matters

Let's answer to the question "*why entanglement matters*" step by step. First of all, the **complexity of quantum states grows exponentially** with the number of qubits, as we have introduced at page 86. As consequence, to describe the system, we need 2^n complex numbers, where n is the number of qubits. This means that for a system of 50 qubits, we would need 2^{50} complex numbers to describe its state, which is an astronomically large number (approximately 1.13×10^{15}). It could be a **problem for classical computers**, but **quantum computers can handle this complexity naturally**.

The reason why quantum computers can handle this complexity naturally is not that they explicitly store all these amplitudes as a classical computer would. Rather, the quantum system itself physically evolves in this exponentially large state space.

Entanglement matters because **most quantum states cannot be decomposed into independent single-qubit states**. In fact, the set of separable states, i.e., states that can be written as a tensor product of individual qubit states, is **very small compared to the set of all possible states**. For two qubits, separable states must satisfy:

$$c_0 c_3 = c_1 c_2$$

Which is a strong structural constraint. Therefore, **most two-qubit states do not satisfy this condition and are entangled**.

❓ But why does this matter? Because **entanglement prevents the system from being compressed into simpler independent components**. The **state must be described globally**. This global structure is difficult to simulate classically, but **quantum mechanics allows us to manipulate it directly through quantum operations**. This is one of the reasons why quantum computers can solve certain problems more efficiently than classical computers.

In summary, entanglement is a fundamental feature of quantum mechanics that prevents a multi-qubit system from being decomposed into independent subsystems. As the number of qubits increases, the state space grows exponentially, and entanglement forces a global description of the system. This global structure can be exploited by quantum algorithms to achieve computational advantages over classical approaches.

4.1.5 Exercise: Normalization of Tensor Product States

Given two qubits in the states $|v_A\rangle$ and $|v_B\rangle$ respectively:

$$|v_A\rangle = a_0 |0\rangle + a_1 |1\rangle, \quad |v_B\rangle = b_0 |0\rangle + b_1 |1\rangle$$

And the combined state of the two qubits is given by the tensor product:

$$|v_A v_B\rangle = |v_A\rangle \otimes |v_B\rangle = a_0 b_0 |00\rangle + a_0 b_1 |01\rangle + a_1 b_0 |10\rangle + a_1 b_1 |11\rangle$$

Show that:

$$|a_0 b_0|^2 + |a_0 b_1|^2 + |a_1 b_0|^2 + |a_1 b_1|^2 = 1$$

Proof. To show that the sum of the squares of the coefficients equals 1, we start expanding the expression:

$$|a_0 b_0|^2 + |a_0 b_1|^2 + |a_1 b_0|^2 + |a_1 b_1|^2$$

Using the property:

$$|xy|^2 = |x|^2 |y|^2$$

Applying this property to each term, we get:

$$\begin{aligned} &= |a_0|^2 |b_0|^2 + |a_0|^2 |b_1|^2 + |a_1|^2 |b_0|^2 + |a_1|^2 |b_1|^2 \\ &= |a_0|^2 (|b_0|^2 + |b_1|^2) + |a_1|^2 (|b_0|^2 + |b_1|^2) \end{aligned}$$

Since $|v_B\rangle$ is a valid quantum state and must be normalized, we have:

$$|b_0|^2 + |b_1|^2 = 1$$

Substituting this back into our expression, we get:

$$\begin{aligned} &= |a_0|^2 \cdot 1 + |a_1|^2 \cdot 1 \\ &= |a_0|^2 + |a_1|^2 \end{aligned}$$

Since $|v_A\rangle$ is also a valid quantum state and must be normalized, we have:

$$|a_0|^2 + |a_1|^2 = 1$$

Therefore, we conclude that:

$$|a_0 b_0|^2 + |a_0 b_1|^2 + |a_1 b_0|^2 + |a_1 b_1|^2 = 1$$

The combined state $|v_A v_B\rangle$ is also a valid quantum state and must be normalized, which is consistent with our result. QED

This exercise demonstrates that the **tensor product of two normalized quantum states is also normalized**, because the squared amplitudes factorize and the normalization conditions of the individual states ensure that the total probability remains 1.

4.2 Introduction to Multiple Qubit Gates

Until now, we learned that a quantum state is a vector in a complex vector space, and for 2 qubits, the state is a vector in $\mathbb{C}^4$:

$$|v\rangle = c_0 |00\rangle + c_1 |01\rangle + c_2 |10\rangle + c_3 |11\rangle$$

Now the question is: *how do we change this state?*

 **The key idea.** A **quantum gate** is just a rule that transforms one state into another. For 1 qubit we already know (page 42):

$$|v'\rangle = U |v\rangle$$

Where U is a 2×2 unitary matrix. For example, the Hadamard gate H (page 65) transforms the state $|0\rangle$ into $|+\rangle$:

$$H |0\rangle = |+\rangle$$

 **What about 2 qubits?** For 2 qubits, we need a 4×4 unitary matrix to transform the state:

$$|v'\rangle = U |v\rangle \quad \text{with} \quad |v\rangle = \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{pmatrix}, \quad U = \begin{pmatrix} u_{00} & u_{01} & u_{02} & u_{03} \\ u_{10} & u_{11} & u_{12} & u_{13} \\ u_{20} & u_{21} & u_{22} & u_{23} \\ u_{30} & u_{31} & u_{32} & u_{33} \end{pmatrix}$$

This means that the gate can change **all amplitudes at once**, not just one at a time as in the single-qubit case. This is the important difference between single and multi-qubit gates: with 1 qubit, we modify only 2 amplitudes, but with 2 qubits, we can modify all 4 amplitudes **simultaneously**. And now, the transformation can mix amplitudes, create correlations, and even generate entanglement between the qubits.

In simple terms, a multi-qubit gate is just a transformation of a larger state vector. Nothing more, nothing less. However, this simple idea has important implications: it allows us to manipulate the global state of a quantum system, including correlations between qubits, which can lead to computational advantages over classical approaches.

4.3 Parallel Gates

Now that we have a way to represent the state of multiple qubits, we can also represent the operations (gates) that act on them. In particular, if we have two qubits $|v_A\rangle$ and $|v_B\rangle$, and two gates A and B , what happens when we want to apply A to $|v_A\rangle$ and B to $|v_B\rangle$ simultaneously?

The answer is given by the **tensor product of quantum gates**:

$$(A \otimes B)(|v_A\rangle \otimes |v_B\rangle) = (A|v_A\rangle) \otimes (B|v_B\rangle) \quad (50)$$

This means that the combined operation of applying A to $|v_A\rangle$ and B to $|v_B\rangle$ is represented by the tensor product of the two gates, $A \otimes B$. So, **applying gates in parallel** corresponds to the tensor product of the individual gates.

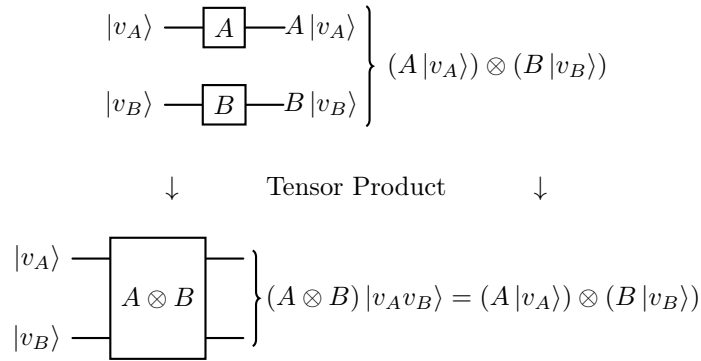


Figure 16: Parallel gates as a tensor product operation.

In simple terms, instead of applying A and B separately, we build one big matrix $A \otimes B$ that acts on the **whole system**. Note that a state is represented as a column vector (e.g., size 4 for two qubits), and a gate is represented as a square matrix (e.g., size 4×4 for two qubits). So *parallel gates* are simply one big transformation of the global state of the system.

❓ What if we want to apply a gate to only one qubit?

Suppose we want to **apply a gate A to the first qubit $|v_A\rangle$ while leaving the second qubit $|v_B\rangle$ unchanged**. We can represent this as:

$$(A \otimes I)(|v_A\rangle \otimes |v_B\rangle) = (A|v_A\rangle) \otimes |v_B\rangle \quad (51)$$

Here, I is the **identity gate** (page 49) that acts on the second qubit, meaning it **leaves it unchanged**. So, to apply a gate to just one qubit in a multi-qubit system, we use the tensor product of that gate with the identity operator on the other qubits.

 Example 2: $H \otimes X$

Let's consider the gates H (Hadamard, page 65):

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

And X (Pauli-X, page 51):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

The tensor product $H \otimes X$ is calculated as follows:

$$\begin{aligned} H \otimes X &= \frac{1}{\sqrt{2}} \left(\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \otimes \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \right) \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \cdot X & 1 \cdot X \\ 1 \cdot X & -1 \cdot X \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \cdot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & 1 \cdot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \\ 1 \cdot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & -1 \cdot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -1 \\ 1 & 0 & -1 & 0 \end{pmatrix} \end{aligned}$$

Now, if we apply $H \otimes X$ to a simple two-qubit state, say $|00\rangle$:

$$\begin{aligned} (H \otimes X) |00\rangle &= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & -1 \\ 1 & 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 + 1 \cdot 0 \\ 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 0 + 0 \cdot 0 \\ 0 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 + -1 \cdot 0 \\ 1 \cdot 1 + 0 \cdot 0 + -1 \cdot 0 + 0 \cdot 0 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} + \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} |01\rangle + \frac{1}{\sqrt{2}} |11\rangle \end{aligned}$$

This can be visually represented as parallel gates acting on the two qubits:

$$\left. \begin{array}{l} |0\rangle \xrightarrow{H} H|0\rangle \\ |0\rangle \xrightarrow{X} X|0\rangle \end{array} \right\} (H|0\rangle) \otimes (X|0\rangle) = |+\rangle \otimes |1\rangle$$

Or as a single global gate:

$$\left. \begin{array}{l} |0\rangle \\ |0\rangle \end{array} \right\} \xrightarrow{H \otimes X} \left. \begin{array}{l} \\ \end{array} \right\} (H \otimes X)|00\rangle = |+\rangle \otimes |1\rangle$$

Where $|+1\rangle$ is $|+\rangle \otimes |1\rangle$. Note that applying the Hadamard gate to the qubit $|0\rangle$ creates the superposition state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, while applying the X gate to $|0\rangle$ flips it to $|1\rangle$.

Example 3: $I \otimes X$

Now, let's consider applying the identity gate I to the first qubit and the X gate to the second qubit. The tensor product is:

$$\begin{aligned} I \otimes X &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \otimes \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \\ &= \begin{pmatrix} 1 \cdot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & 0 \cdot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \\ 0 \cdot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} & 1 \cdot \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \end{pmatrix} \\ &= \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \end{aligned}$$

Applying this to the state $|00\rangle$ gives:

$$\begin{aligned} (I \otimes X)|00\rangle &= \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} \\ &= \begin{pmatrix} 0 \cdot 1 + 1 \cdot 0 + 0 \cdot 0 + 0 \cdot 0 \\ 1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 + 0 \cdot 0 \\ 0 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 + 1 \cdot 0 \\ 0 \cdot 1 + 0 \cdot 0 + 1 \cdot 0 + 0 \cdot 0 \end{pmatrix} \\ &= \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \\ &= |01\rangle \end{aligned}$$

This means that applying I to the first qubit leaves it unchanged as $|0\rangle$, while applying X to the second qubit flips it from $|0\rangle$ to $|1\rangle$, resulting in the combined state $|01\rangle$. Visually, this can be represented as:

$$\left. \begin{array}{l} |0\rangle \text{ --- } \boxed{I} \text{ --- } I|0\rangle \\ |0\rangle \text{ --- } \boxed{X} \text{ --- } X|0\rangle \end{array} \right\} |0\rangle \otimes (X|0\rangle) = |0\rangle \otimes |1\rangle$$

Or as a single global gate:

$$\left. \begin{array}{l} |0\rangle \\ |0\rangle \end{array} \right\} \boxed{I \otimes X} \left. \begin{array}{l} \\ \end{array} \right\} (I \otimes X) |00\rangle = |01\rangle$$

4.4 Hadamard Transform on Multiple Qubits

Applying the Hadamard gate to every qubit is a useful tool in quantum computing, as it creates a superposition of all possible states. Now, let's see how this is done.

We already know that applying the Hadamard gate to a single qubit in the state $|0\rangle$ gives us:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

This means that H creates a **balanced superposition** of the two basis states $|0\rangle$ and $|1\rangle$.

Now, if we apply this idea to two qubits, we can apply the Hadamard gate to each qubit individually. This is done using the tensor product of the Hadamard gates, for example:

$$(H \otimes H)|00\rangle = H|0\rangle \otimes H|0\rangle$$

So it is equivalent to applying the Hadamard gate to each qubit separately. Expanding this, we get:

$$\begin{aligned} (H \otimes H)|00\rangle &= H|0\rangle \otimes H|0\rangle \\ &= \left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \right) \otimes \left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \right) \\ &= \frac{1}{2}(|0\rangle + |1\rangle) \otimes (|0\rangle + |1\rangle) \\ &= \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) \end{aligned}$$

This means that **applying Hadamard gate in parallel generates all possible combinations of the basis states** for the two qubits, creating a superposition of all four possible states.

In general, for n qubits, **applying the Hadamard gate to each qubit will create a superposition of all 2^n possible states.** The resulting state can be expressed as:

$$(H^{\otimes n})|0\rangle^{\otimes n} = \frac{1}{2^{\frac{n}{2}}} \sum_{k=0}^{2^n-1} |k\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} |k\rangle \quad (52)$$

Where $|k\rangle$ represents the basis states of the n -qubit system, and the **sum runs over all possible combinations of the basis states.**

$$\left. \begin{array}{l} |0\rangle \text{---} \boxed{H} \text{---} \\ |0\rangle \text{---} \boxed{H} \text{---} \end{array} \right\} (H \otimes H)|00\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

In summary, **Hadamard on all qubits spreads the state over the entire space of possible states**, creating a uniform superposition that is fundamental in many quantum algorithms.

4.5 Main Multi-Qubit Gates

4.5.1 Controlled NOT (CNOT) Gate

The **Controlled NOT (CNOT) Gate** is a **two-qubit gate** composed of:

- A **control qubit** that **determines** whether the **NOT operation is applied or not**.
- A **target qubit** that is **flipped** (NOT operation) if the control qubit is in the state $|1\rangle$, and remains unchanged if the control qubit is in the state $|0\rangle$.

In simple terms, it is a gate that **performs a NOT operation on the *target* qubit only when the *control* qubit is in the state $|1\rangle$** .

Action on Basis States

The CNOT gate acts on the computational basis states as follows:

Input	Output
$ 00\rangle$	$ 00\rangle$
$ 01\rangle$	$ 01\rangle$
$ 10\rangle$	$ 11\rangle$
$ 11\rangle$	$ 10\rangle$

If the first qubit (control) is $|0\rangle$, the second qubit (target) remains unchanged. If the first qubit is $|1\rangle$, the second qubit is flipped: $|0\rangle \leftrightarrow |1\rangle$.

Matrix Representation

The CNOT gate can be represented as a 4×4 unitary matrix in the computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (53)$$

This matrix indicates that the first two basis states $|00\rangle$ and $|01\rangle$ are unchanged, while $|10\rangle$ and $|11\rangle$ are swapped, reflecting the action of the CNOT gate on the target qubit based on the state of the control qubit.

Action on General States

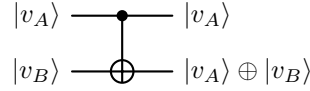
For a general two-qubit state $|v\rangle = c_0 |00\rangle + c_1 |01\rangle + c_2 |10\rangle + c_3 |11\rangle$, the CNOT gate transforms it as follows:

$$\text{CNOT} |v\rangle = c_0 |00\rangle + c_1 |01\rangle + c_2 |11\rangle + c_3 |10\rangle \quad (54)$$

The amplitudes c_0 and c_1 remain unchanged, while c_2 and c_3 are **swapped**, demonstrating the conditional nature of the CNOT gate based on the state of the control qubit.

✂ Circuit Representation

In quantum circuit diagrams, the CNOT gate is typically represented with a solid dot on the control qubit line and a plus sign $\oplus$ (indicating the NOT operation) on the target qubit line, connected by a vertical line.



Here, $|v_A\rangle$ is the control qubit and $|v_B\rangle$ is the target qubit. The output state of the target qubit is the result of the XOR operation between the control and target qubits, reflecting the conditional flipping of the target qubit based on the state of the control qubit.

In summary, **if the control qubit is $|1\rangle$, apply NOT (Pauli-X) to the target qubit; otherwise do nothing.**

⚙ Properties

- **Unitary**

$$\text{CNOT}^\dagger \cdot \text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}^\dagger \cdot \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = I$$

CNOT gate is unitary, so it is a valid quantum gate that preserves normalization of quantum states, and the total probability of all outcomes remains 1.

- **Reversible**

$$\text{CNOT}^{-1} = \text{CNOT}^\dagger = \text{CNOT}$$

The CNOT gate is reversible, meaning that applying the CNOT gate twice will return the system to its original state:

$$\text{CNOT}(\text{CNOT}(|\psi\rangle)) = |\psi\rangle$$

In other words, we can **always undo a CNOT operation without loss of information.**

- **Conditional Operation**

$$\text{CNOT}(|v_A\rangle \otimes |v_B\rangle) \neq \text{CNOT}|v_A\rangle \otimes \text{CNOT}|v_B\rangle$$

The CNOT gate is a conditional operation, meaning that the **action on the target qubit depends on the state of the control qubit**. This is different from parallel gates, where each qubit is acted upon independently:

$$(H \otimes X)|00\rangle = (H|0\rangle) \otimes (X|0\rangle)$$

CNOT **breaks parallelism** because it introduces interaction: the **evolution of one qubit depends on another**, so the **system can no longer be described as independent parts**.

- **Entangling.** CNOT creates *entanglement* **if and only if** the input state has superposition on the control qubit (in the computational basis).

For example:

$$(H \otimes I) |00\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle)$$

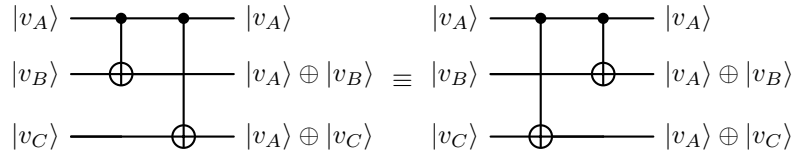
Applying CNOT to this state:

$$\text{CNOT} \left(\frac{1}{\sqrt{2}} (|00\rangle + |10\rangle) \right) = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

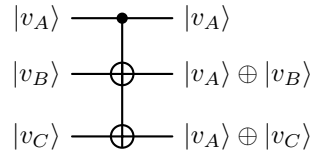
The resulting state is an entangled state (cannot be factored into a tensor product, page 89), demonstrating that CNOT can create entanglement from a superposition state of the control qubit.

✂ Multiple CNOT Gates

Multi-CNOT notation means applying several CNOT gates that share the same control qubit, and these can be represented equivalently as sequential gates or as a single control connected to multiple targets. Visually, this can be represented as:



But it can also be represented as:



The **order of the CNOT gates does not matter** because they **commute when they share the same control qubit**, meaning that the final state of the system will be the same regardless of the order in which the CNOT gates are applied.

4.5.2 Generic Controlled Gate (CU)

The **Generic Controlled Gate (CU)** is a **two-qubit gate** composed of:

- A **control qubit** that determines **whether** the operation is **applied or not** (first qubit).
- A **target qubit** **on which the operation is applied** if the control qubit is in the state $|1\rangle$ (second qubit).
- A **unitary operation** U that is **applied to the target qubit** when the control qubit is in the state $|1\rangle$.

It **generalizes the CNOT gate** (page 103, $U = X$) by allowing **any unitary operation** U to be applied to the target qubit, rather than just the NOT operation.

≡ Action on Basis States

The action of the CU gate on the computational basis states can be described as follows:

Input	Output
$ 00\rangle$	$ 00\rangle$
$ 01\rangle$	$ 01\rangle$
$ 10\rangle$	$ 1\rangle \otimes U 0\rangle$
$ 11\rangle$	$ 1\rangle \otimes U 1\rangle$

If the control qubit is $|0\rangle$, the target qubit remains unchanged. If the control qubit is $|1\rangle$, the unitary operation U is applied to the target qubit, transforming $|0\rangle$ to $U|0\rangle$ and $|1\rangle$ to $U|1\rangle$.

√ Matrix Representation

The CU gate can be represented as a 4×4 unitary matrix in the computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$:

$$CU = C_U = \begin{bmatrix} I & 0 \\ 0 & U \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & U_{00} & U_{01} \\ 0 & 0 & U_{10} & U_{11} \end{bmatrix} \quad (55)$$

Where $U_{00}, U_{01}, U_{10}, U_{11}$ are the elements of the 2×2 unitary matrix U that acts on the target qubit when the control qubit is $|1\rangle$. For the first two basis states $|00\rangle$ and $|01\rangle$, the identity operation I is applied, leaving them unchanged. For the last two basis states $|10\rangle$ and $|11\rangle$, the unitary operation U is applied to the target qubit, resulting in the transformation defined by U .

📖 Action on General States

Let's consider two general single-qubit states:

$$|v_A\rangle = a_0 |0\rangle + a_1 |1\rangle \quad |v_B\rangle = b_0 |0\rangle + b_1 |1\rangle$$

Combining these states into a two-qubit state, we have:

$$|v\rangle = |v_A\rangle \otimes |v_B\rangle = a_0 |0\rangle |v_B\rangle + a_1 |1\rangle |v_B\rangle$$

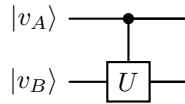
Applying the CU gate to this state, we get:

$$\begin{aligned} \text{CU} |v\rangle &= \text{CU}(a_0 |0\rangle |v_B\rangle + a_1 |1\rangle |v_B\rangle) \\ &= a_0 \text{CU}(|0\rangle |v_B\rangle) + a_1 \text{CU}(|1\rangle |v_B\rangle) \\ &= \underbrace{a_0 |0\rangle |v_B\rangle}_{\text{unchanged}} + \underbrace{a_1 |1\rangle U |v_B\rangle}_{\text{transformed}} \end{aligned}$$

This shows that the CU gate applies the unitary operation U to the target qubit only when the control qubit is in the state $|1\rangle$, while leaving the target qubit unchanged when the control qubit is in the state $|0\rangle$. Each component of the superposition evolves independently according to the value of the control qubit, resulting in a new superposition where different branches have undergone different transformations.

✂ Circuit Representation

In quantum circuit diagrams, the CU gate is typically represented with a **control dot on the control qubit** and a **box labeled with the unitary operation U on the target qubit**. The control dot indicates that the operation is conditional on the state of the control qubit. The circuit representation of the CU gate is as follows:



In summary, if the control qubit is $|1\rangle$, we apply U to the target qubit; otherwise do nothing.

⚙ Properties

The CU gate has the same properties as the CNOT gate because it is a generalization of the CNOT gate. Specifically:

- **Unitary**

$$\text{CU}^\dagger \text{CU} = I$$

- **Reversible**

$$\text{CU}^{-1} = \text{CU}^\dagger$$

- **Conditional Operation**

$$\text{CU}(|v_A\rangle \otimes |v_B\rangle) \neq \text{CU}|v_A\rangle \otimes \text{CU}|v_B\rangle$$

It does not act independently on each qubit, but rather applies a transformation to the target qubit based on the state of the control qubit.

- **Entangling.** The CU gate can create entanglement between the control and target qubits if the unitary operation U and the input state are chosen appropriately. For example, when $U = X$ (i.e., the CNOT gate), we have:

$$\text{CNOT}(|+\rangle |0\rangle) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

4.5.3 SWAP gate

The **SWAP gate** is a **two-qubit gate** that **exchanges the states of two qubits** and has **no control/target distinction**, it is **symmetric**. It simply swaps:

$$|v_A\rangle \otimes |v_B\rangle \xrightarrow{SWAP} |v_B\rangle \otimes |v_A\rangle \quad (56)$$

≡ Action on Basis States

The SWAP gate acts on the basis states as follows:

Input	Output
$ 00\rangle$	$ 00\rangle$
$ 01\rangle$	$ 10\rangle$
$ 10\rangle$	$ 01\rangle$
$ 11\rangle$	$ 11\rangle$

It essentially **swaps the second and third basis states**, while leaving the first and fourth unchanged.

✓✱ Matrix Representation

The matrix representation of the SWAP gate in the computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ is:

$$SWAP = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (57)$$

This matrix clearly shows the **swapping of the second and third basis states**, while the first and fourth remain unchanged.

≡ Action on General States

Let's consider two arbitrary single-qubit states:

$$|v_A\rangle = a_0 |0\rangle + a_1 |1\rangle, \quad |v_B\rangle = b_0 |0\rangle + b_1 |1\rangle$$

The tensor product of these states is:

$$|v_A\rangle \otimes |v_B\rangle = a_0 b_0 |00\rangle + a_0 b_1 |01\rangle + a_1 b_0 |10\rangle + a_1 b_1 |11\rangle$$

Applying the SWAP gate to this state gives:

$$\begin{aligned} SWAP(|v_A\rangle \otimes |v_B\rangle) &= a_0 b_0 |00\rangle + a_1 b_0 |01\rangle + a_0 b_1 |10\rangle + a_1 b_1 |11\rangle \\ &= (b_0 |0\rangle + b_1 |1\rangle) \otimes (a_0 |0\rangle + a_1 |1\rangle) \\ &= |v_B\rangle \otimes |v_A\rangle \end{aligned}$$

This confirms that the SWAP gate indeed **exchanges the states of the two qubits**, as expected.

✂ Circuit Representation

In quantum circuit diagrams, the SWAP gate is typically represented by two crossed lines with $\times$ symbols at the crossing point:

$$\begin{array}{ccc} |v_A\rangle & \text{---} \times & |v_B\rangle \\ |v_B\rangle & \text{---} \times & |v_A\rangle \end{array}$$

This notation visually emphasizes the **swapping of the two qubits' states**.

⚙ Properties

- **Unitary**

$$\text{SWAP}^\dagger \text{SWAP} = I$$

This means that the SWAP gate preserves the inner product and is reversible.

- **Reversible (self-inverse)**

$$\text{SWAP}^2 = I$$

This means that applying the SWAP gate twice will return the system to its original state.

- **Symmetric (no control/target)**. The SWAP gate does not distinguish between the two qubits; it treats them symmetrically. This is in contrast to gates like CNOT, which have a clear control and target qubit.
- **Does NOT create entanglement**. The SWAP gate simply exchanges the states of two qubits and does **not introduce any new correlations** between them. If the input state is separable (not entangled), the output state will also be separable (and vice versa).
- **Can be decomposed into 3 CNOT gates**. The **SWAP gate can be implemented using three CNOT gates**, which is important for practical implementations on quantum hardware that may not have a native SWAP gate.

Proof. We want to transform:

$$|a, b\rangle \rightarrow |b, a\rangle$$

But CNOT only does:

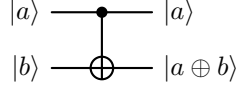
$$|a, b\rangle \rightarrow |a, a \oplus b\rangle$$

So we need to **combine multiple CNOTs** to achieve a SWAP. Let's track what happens to a generic input:

$$|a, b\rangle \quad \text{with } a, b \in \{0, 1\}$$

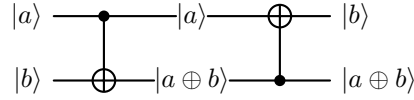
- **First CNOT** (control: first qubit, target: second qubit):

$$|a, b\rangle \xrightarrow{\text{CNOT}} |a, a \oplus b\rangle$$



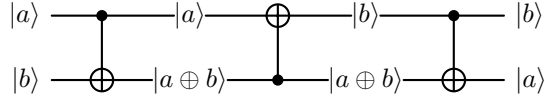
– **Second** CNOT (control: second qubit, target: first qubit):

$$|a, a \oplus b\rangle \xrightarrow{\text{CNOT}} |a \oplus (a \oplus b), a \oplus b\rangle = |b, a \oplus b\rangle$$

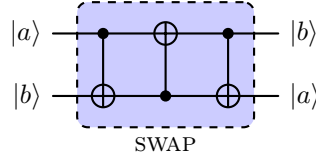


– **Third** CNOT (control: first qubit, target: second qubit):

$$|b, a \oplus b\rangle \xrightarrow{\text{CNOT}} |b, b \oplus (a \oplus b)\rangle = |b, a\rangle$$



In summary, the sequence of operations is:



QED

In general, the SWAP gate **does not create new correlations**; it only **re-orderes the information between qubits**. In other words, it only **changes where the information is stored**.

4.5.4 Controlled-Controlled-NOT (CCNOT) Gate (Toffoli)

The **Controlled-Controlled-NOT (CCNOT) Gate**, also called the **Toffoli Gate**, is a **three-qubit gate** composed of:

- Two **control qubits** that determine whether the gate will be applied.
- One **target qubit** that is flipped (NOT operation) if both control qubits are in the state $|1\rangle$.

It **generalizes the CNOT gate by adding an additional control qubit**, making it a crucial component in quantum computing for implementing more complex operations and algorithms.

📖 Action on Basis States

The CCNOT gate acts on the basis states as follows:

Input	Output
$ 000\rangle$	$ 000\rangle$
$ 001\rangle$	$ 001\rangle$
$ 010\rangle$	$ 010\rangle$
$ 011\rangle$	$ 011\rangle$
$ 100\rangle$	$ 100\rangle$
$ 101\rangle$	$ 101\rangle$
$ 110\rangle$	$ 111\rangle$
$ 111\rangle$	$ 110\rangle$

The target flips **only if both control qubits are $|1\rangle$** , demonstrating the conditional nature of the gate (last two rows of the table). For all other input combinations, the output remains unchanged.

✓✳ Matrix Representation

The matrix representation of the CCNOT gate in the computational basis is an 8×8 (since it operates on three qubits) unitary matrix given by:

$$\text{CCNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \quad (58)$$

This matrix shows that the first six basis states are unchanged (diagonal entries of 1), while the **last two basis states are swapped** (off-diagonal entries of 1 in the last two rows and columns).

📖 Action on General States

For a general three-qubit state:

$$|\psi\rangle = \sum_{i,j,k \in \{0,1\}} \alpha_{ijk} |ijk\rangle$$

The CCNOT gate acts linearly on each component of the superposition and flips the target qubit (the third qubit) if and only if both control qubits are equal to $|1\rangle$. This can be expressed as:

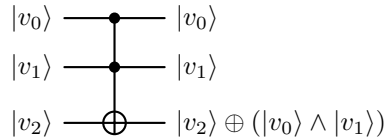
$$\text{CCNOT} |\psi\rangle = \sum_{i,j,k \in \{0,1\}} \alpha_{ijk} |ij(k \oplus (i \cdot j))\rangle \quad (59)$$

Where $\oplus$ denotes the XOR operation, and $i \cdot j = 1$ if and only if $i = j = 1$, and 0 otherwise.

In simple terms, applying CCNOT affects only the c_6 and c_7 coefficients (corresponding to $|110\rangle$ and $|111\rangle$) by swapping them, while all other coefficients remain unchanged.

⌘ Circuit Representation

In quantum circuit diagrams, the CCNOT gate is typically represented by a symbol with two control dots connected to a NOT gate (X gate) on the target qubit. The control dots indicate which qubits are the controls, and the NOT gate indicates the operation performed on the target qubit when both controls are active. It is similar to the CNOT gate but with an additional control dot, making it visually distinct and easy to identify in quantum circuits.



In summary, if both control qubits are $|1\rangle$, apply NOT to the target; otherwise do nothing.

⚙️ Properties

- **Unitary**

$$\text{CCNOT}^\dagger \text{CCNOT} = I$$

- **Reversible**

$$\text{CCNOT}^{-1} = \text{CCNOT}^\dagger$$

- **Classical universal gate (can simulate AND).** CCNOT gate is a universal gate for classical computation, meaning that any classical

logic gate can be constructed using CCNOT gates and single-qubit NOT gates. In particular, it can compute the AND operation in a reversible way:

$$|a, b, 0\rangle \xrightarrow{\text{CCNOT}} |a, b, 0 \oplus (a \cdot b)\rangle = |a, b, a \cdot b\rangle$$

Where a and b are the control qubits, and the target qubit is initialized to $|0\rangle$. After applying the CCNOT gate, the target qubit will be in the state $|1\rangle$ if and only if both a and b are $|1\rangle$, effectively computing the AND of the two control qubits.

Since AND and NOT are sufficient to express any Boolean function, the Toffoli gate can reproduce any classical computation.

- **Introduce multi-qubit interaction.** Multi-qubit interaction refers to operations in which the evolution of one qubit depends on the state of one or more other qubits. Unlike parallel gates, which act independently on each qubit, **interacting gates introduce conditional behavior and correlations between qubits.**

Typical examples include controlled gates such as CNOT and CCNOT, where the transformation applied to a target qubit depends on the value of one or more control qubits. These interactions are essential for creating entanglement and for implementing complex quantum algorithms.

- **Can create entanglement.** Similar to the CNOT gate, also the CCNOT gate can create entanglement between the control and target qubits. For example, if the control qubits are in a superposition state, applying the CCNOT gate can result in an entangled state between the control and target qubits.
- **Not decomposable into a single tensor product.** An interacting multi-qubit gate cannot be expressed as a tensor product of single-qubit operators. That is, in general:

$$U \neq U_A \otimes U_B$$

This means that the gate introduces correlations between qubits and cannot be reduced to independent operations.

In summary, the Toffoli gate performs a conditional operation based on two qubits, enabling multi-qubit logic and more complex correlations.

4.6 Quantum Circuit Representation

In the previous sections, we have been using visual representations of quantum gates and their effects on qubits. However, a quantum circuit can be expressed in a more formal way using a **single unitary matrix** that represents the entire operation of the circuit. This matrix acts on the global state of the system and is obtained by composing the individual gates according to the structure of the circuit. This is known as the **quantum circuit representation**.

Definition 1: Quantum Circuit Representation

A **quantum circuit** can be represented as a **unitary matrix** U that acts on the global state vector of the qubits. The overall operation of the circuit is obtained by composing the unitary matrices corresponding to each layer of gates in the circuit.

Mathematically, if a circuit consists of layers $G_1, G_2, \dots, G_n$ with corresponding unitary matrices $U_1, U_2, \dots, U_n$, then the overall unitary matrix U representing the circuit is given by:

$$U = U_n U_{n-1} \cdots U_2 U_1$$

Where the rightmost operator is applied first.

Each U_i is constructed by combining gates acting in parallel using tensor products, and by extending gates with identity operators on qubits where no operation is applied.

In simple terms, a **quantum circuit is just a structured way to build a single unitary matrix acting on the whole system**. So:

$$|\psi_{\text{out}}\rangle = U |\psi_{\text{in}}\rangle$$

Where $|\psi_{\text{in}}\rangle$ is the initial state of the qubits, $|\psi_{\text{out}}\rangle$ is the final state after applying the circuit, and U is the **product of all the gates in the circuit**.

The 3 Fundamental Rules of Quantum Circuit Representation

1. **Parallel composition.** Gates acting on different qubits are combined using the tensor product:

$$U = A \otimes B$$

2. **Sequential composition.** Gates applied in sequence are combined using matrix multiplication, evaluated from right to left:

$$U = U_n \cdots U_2 U_1$$

So the first gate applied is the rightmost one in the product.

3. **Identity extension.** When a gate does not act on a qubit, the identity operator must be used to extend it:

$$U = I \otimes A$$

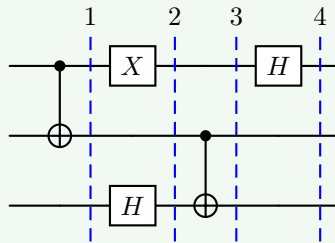
This keeps dimensions consistent when combining gates that act on different subsets of qubits.

 Example 4: Quantum Circuit Representation

Let's consider the following quantum circuit representation:

$$\underbrace{(H \otimes I \otimes I)}_4 \cdot \underbrace{(I \otimes \text{CNOT})}_3 \cdot \underbrace{(X \otimes I \otimes H)}_2 \cdot \underbrace{(\text{CNOT} \otimes I)}_1$$

First of all, the order of the gates is from right to left, so we start with the rightmost gate.



4.7 Entanglement

As discussed on page 95, we have already seen why entanglement exists and matters. In short, describing an n -qubit quantum state requires an exponential number of parameters, meaning that most multi-qubit states are entangled.

If qubits were independent (separable), we would only need $2n$ real numbers, since each qubit would require only 2 real parameters:

$$|v\rangle = a_0 |0\rangle + a_1 |1\rangle$$

Each qubit needs two real parameters, so for n qubits, we would need $2n$ real numbers. However, a general quantum state of n qubits is given by (see page 93):

$$|\psi\rangle = \sum_{k=0}^{2^n-1} c_k |k\rangle$$

This representation requires 2^n complex coefficients (subject to normalization and global phase constraints), corresponding to an exponential number of real degrees of freedom.

In conclusion, most multi-qubit states cannot be described as independent qubits. This phenomenon is called **entanglement** and is a fundamental feature of quantum mechanics with no classical analog.

Definition 2: Entanglement

A multi-qubit state is called **Entangled** if it cannot be written as a tensor product of individual qubit states (i.e., it is not separable). Otherwise, it is called **separable** or **unentangled**. Entanglement can be viewed as a “failure of factorization”, where the state of the whole system cannot be factored into states of its parts.

An important consequence of entanglement is that we cannot assign a separate quantum state to each qubit independently.

Analysis of a Real Example: Bell States

To better understand the concept of entanglement, it is useful to analyze a concrete example. In particular, we consider a class of two-qubit states known as **Bell states**, which provide the simplest and most important examples of entangled states. These states cannot be written as a tensor product of individual qubit states, and therefore explicitly demonstrate the impossibility of decomposing certain multi-qubit states into independent subsystems.

Definition 3: Bell States

The **Bell States** are a set of 4 two-qubit states that form an orthonormal basis of maximally entangled states. They are defined as:

$$\begin{aligned} |\Phi^+\rangle &= \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle) & |\Phi^-\rangle &= \frac{1}{\sqrt{2}} (|00\rangle - |11\rangle) \\ |\Psi^+\rangle &= \frac{1}{\sqrt{2}} (|01\rangle + |10\rangle) & |\Psi^-\rangle &= \frac{1}{\sqrt{2}} (|01\rangle - |10\rangle) \end{aligned} \quad (60)$$

These states are called **maximally entangled** because each qubit, when considered individually, is in a completely mixed (maximally uncertain) state. In other words, Bell states are characterized by the fact that **each individual qubit is in a maximally mixed (completely random) state**, while the **joint state exhibits perfect correlations between the qubits**.

This means that when we measure a single qubit of a Bell state, the outcome is completely random. In other words, there is a 50% chance of getting a 0 and a 50% chance of getting a 1. Therefore, we cannot say that it is mostly a 0 or a 1. However, once we measure a single qubit, a paradox occurs: even though each qubit alone is random, together they are perfectly correlated. In fact, if we measure the first qubit and get $|0\rangle$, we know with certainty that the second qubit is also $|0\rangle$ (in the case of $|\Phi^+\rangle$ and $|\Phi^-\rangle$), or $|1\rangle$ (in the case of $|\Psi^+\rangle$ and $|\Psi^-\rangle$). Thus, we have **maximum uncertainty individually but maximum correlation jointly**.

Example 5: Proof of Entanglement in Bell States

After introducing the Bell states, we would like to **proof that decomposing them into independent qubits is impossible**. In other words, we want to show that:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

Can be written as:

$$|\Phi^+\rangle = |v_A\rangle \otimes |v_B\rangle$$

Proof. Let's assume, for the sake of argument, that $|\Phi^+\rangle$ can be written as a tensor product of two single-qubit states:

$$|\Phi^+\rangle = |v_A\rangle \otimes |v_B\rangle$$

Where:

- $|v_A\rangle = a_0 |0\rangle + a_1 |1\rangle$ is the state of the first qubit.
- $|v_B\rangle = b_0 |0\rangle + b_1 |1\rangle$ is the state of the second qubit.

The tensor product of these two states is:

$$|v_A\rangle \otimes |v_B\rangle = a_0 b_0 |00\rangle + a_0 b_1 |01\rangle + a_1 b_0 |10\rangle + a_1 b_1 |11\rangle$$

For this to equal $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, we must have:

$$a_0b_0 = \frac{1}{\sqrt{2}}, \quad a_1b_1 = \frac{1}{\sqrt{2}}, \quad a_0b_1 = 0, \quad a_1b_0 = 0$$

But here's the problem, if $a_0b_1 = 0$, then either $a_0 = 0$ or $b_1 = 0$:

✗ Case $a_0 = 0$, then $a_0b_0 = 0$, which **contradicts** $a_0b_0 = \frac{1}{\sqrt{2}}$ for $|00\rangle$.

✗ Case $b_1 = 0$, then $a_1b_1 = 0$, which **contradicts** $a_1b_1 = \frac{1}{\sqrt{2}}$ for $|11\rangle$.

Therefore, there is no way to choose a_0, a_1, b_0, b_1 such that the above equations hold simultaneously. This means that $|\Phi^+\rangle$ cannot be written as a tensor product of two single-qubit states, and thus it is an entangled state. **All we can say is that these qubits are correlated and cannot be described independently.** QED

🔍 Entanglement: Intrinsic or Basis-Dependent?

Having established that Bell states are entangled by showing that they cannot be written as a tensor product of individual qubit states, a natural question arises: **is this property intrinsic to the state, or does it depend on the basis in which the state is expressed?** In other words, could a change of basis make an entangled state appear separable? The answer is no. **Entanglement** is not a property of the representation, but of the **structure of the state itself**.

In contrast to entanglement, **superposition depends on the basis we choose**, for example:

$$|0\rangle = \frac{1}{\sqrt{2}}(|+\rangle + |-\rangle)$$

This shows that the state $|0\rangle$ is a **superposition** of $|+\rangle$ and $|-\rangle$ in the **Hadamard basis**, but it is **not a superposition in the computational basis**, indeed:

$$|0\rangle = 1 \cdot |0\rangle + 0 \cdot |1\rangle$$

It is not a superposition because only one basis vector is present.²

Differently, **entanglement** does **not** depend on basis, because it depends on **factorization**, not representation. So we can change basis, rewrite the state, but we **cannot make an entangled state separable**. This prevents a common misunderstanding: one might wonder whether a suitable change of basis could remove entanglement. This is not the case. Entanglement is preserved under local changes of basis, that is, transformations of the form $U_A \otimes U_B$ acting independently on each qubit. Therefore, **entanglement is an intrinsic property of the quantum state and does not depend on the particular representation of each subsystem.**

²A state is a superposition in a given basis if it is written as a combination of multiple basis vectors with non-zero coefficients.

✂ From States to Operations

So far, we have analyzed entanglement as a property of quantum states, showing that certain states cannot be decomposed into independent subsystems. A natural question now arises: *how can such entangled states be generated in practice?* In other words, **which quantum operations are capable of transforming separable states into entangled ones?**

The answer lies in a special class of quantum gates, known as **entangling gates**, which introduce interactions between qubits and allow the creation of correlations that cannot be reduced to independent components.

📖 Definition 4: Entangling Gates

A multi-qubit gate is called **entangling** (**Entangling Gates**) if it cannot be written as a tensor product of single-qubit gates. Equivalently, a gate is entangling if there exists at least one separable input state that it transforms into an entangled state.

Put simply, an **entangling gate is a gate that makes qubits interact and can create entanglement.**

⚠ Attention: Not every input becomes entangled. There exists at least one input that becomes entangled, but not all inputs necessarily do.

In simple terms, an entangling gate is one that can take two or more qubits that are initially uncorrelated (separable) and produce a state where the qubits become correlated in such a way that they cannot be described independently (entangled). For example, the CNOT gate is an entangling gate because it can create Bell states from separable inputs.

✏ Example 6: CNOT as an Entangling Gate

Consider the separable state:

$$|+\rangle \otimes |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |0\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$$

This is a separable state, since it is written as a tensor product of two single-qubit states.

Now apply a CNOT gate (with the first qubit as control and the second as target, page 103):

$$\text{CNOT} \left(\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) \right) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

The resulting state is the Bell state $|\Phi^+\rangle$, which is entangled. Therefore, the **CNOT gate is an entangling gate.**

 Example 7: CNOT Does Not Always Create Entanglement

Not all inputs to an entangling gate produce entangled states. Consider the separable state:

$$|0\rangle \otimes |0\rangle = |00\rangle$$

Applying the CNOT gate:

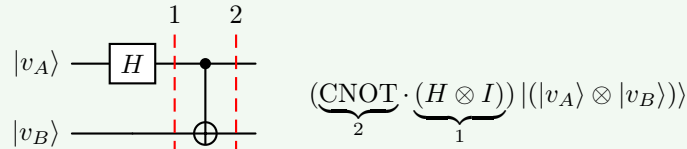
$$\text{CNOT}(|00\rangle) = |00\rangle$$

The state remains separable.

This shows that an entangling gate does not necessarily produce entanglement for every input. However, it is called **entangling** because there exists at least one separable input state that it transforms into an entangled state.

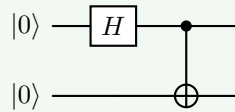
 Example 8: Bell-state generator

Exists a simple quantum circuit that produces **different Bell states depending on the input**. So, for the four input states $|00\rangle$, $|01\rangle$, $|10\rangle$, and $|11\rangle$, the circuit creates the four Bell states $|\Phi^+\rangle$, $|\Phi^-\rangle$, $|\Psi^+\rangle$, and $|\Psi^-\rangle$ respectively. This circuit consists of a Hadamard gate followed by a CNOT gate:



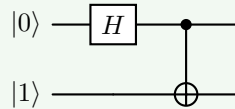
This circuit is a **Bell-state generator** because it can produce any of the four Bell states from the appropriate input.

- Input $|00\rangle$ produces $|\Phi^+\rangle$:



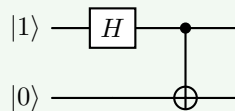
$$|\Phi^+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

- Input $|01\rangle$ produces $|\Phi^-\rangle$:



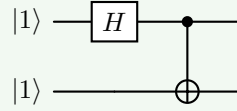
$$|\Phi^-\rangle = \frac{1}{\sqrt{2}} (|00\rangle - |11\rangle)$$

- Input $|10\rangle$ produces $|\Psi^+\rangle$:



$$|\Psi^+\rangle = \frac{1}{\sqrt{2}} (|01\rangle + |10\rangle)$$

- Input $|11\rangle$ produces $|\Psi^-\rangle$:



$$|\Psi^-\rangle = \frac{1}{\sqrt{2}} (|01\rangle - |10\rangle)$$

Example 9: Bell States in the Hadamard Basis

In this exercise, we express the Bell state $|\Phi^+\rangle$ in the Hadamard basis instead of the computational basis.

Bell state $|\Phi^+\rangle$ in the Hadamard basis. Given:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

We aim to re-express this state using the Hadamard basis $\{|+\rangle, |-\rangle\}$, where:

$$|+\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \quad |-\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

1. **Invert the definitions.** To convert from the computational basis to the Hadamard basis, we invert the above equations to express $|0\rangle$ and $|1\rangle$ in terms of $|+\rangle$ and $|-\rangle$:

$$|0\rangle = \frac{1}{\sqrt{2}} (|+\rangle + |-\rangle) \quad |1\rangle = \frac{1}{\sqrt{2}} (|+\rangle - |-\rangle)$$

2. **Substitute into $|00\rangle$ and $|11\rangle$.** We compute the tensor products using the expression above.

- Compute $|00\rangle$:

$$\begin{aligned} |00\rangle &= |0\rangle \otimes |0\rangle \\ &= \left(\frac{1}{\sqrt{2}} (|+\rangle + |-\rangle) \right) \otimes \left(\frac{1}{\sqrt{2}} (|+\rangle + |-\rangle) \right) \\ &= \frac{1}{2} (|++\rangle + |+-\rangle + |-+\rangle + |--\rangle) \end{aligned}$$

- Compute $|11\rangle$:

$$\begin{aligned} |11\rangle &= |1\rangle \otimes |1\rangle \\ &= \left(\frac{1}{\sqrt{2}} (|+\rangle - |-\rangle) \right) \otimes \left(\frac{1}{\sqrt{2}} (|+\rangle - |-\rangle) \right) \\ &= \frac{1}{2} (|++\rangle - |+-\rangle - |-+\rangle + |--\rangle) \end{aligned}$$

3. **Add the two to get $|\Phi^+\rangle$.** Now add the two results:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}} (|00\rangle + |11\rangle)$$

Substituting:

$$\begin{aligned} |\Phi^+\rangle &= \frac{1}{\sqrt{2}} \left[\frac{1}{2} (|++\rangle + |+-\rangle + |-+\rangle + |--\rangle) \right. \\ &\quad \left. + \frac{1}{2} (|++\rangle - |+-\rangle - |-+\rangle + |--\rangle) \right] \\ &= \frac{1}{\sqrt{2}} \cdot \frac{1}{2} (2|++\rangle + 0|+-\rangle + 0|-+\rangle + 2|--\rangle) \\ &= \frac{1}{\sqrt{2}} \cdot \frac{1}{2} (2|++\rangle + 2|--\rangle) \\ &= \frac{1}{\sqrt{2}} (|++\rangle + |--\rangle) \end{aligned}$$

Thus, in the Hadamard basis, we have:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}} (|++\rangle + |--\rangle)$$

Which is a Bell-like state expressed in the Hadamard basis instead of the computational basis. QED

❓ Why single-qubit gates cannot create entanglement?

Entanglement means that two qubits become so strongly correlated that they can no longer be described independently. To create this kind of correlation, the qubits must **interact**. Since single-qubit gates act on each qubit **independently**, they cannot introduce any interaction between qubits.

Suppose we have two qubits $|\psi_A\rangle$ and $|\psi_B\rangle$, and the combined state is:

$$|\psi\rangle = |\psi_A\rangle \otimes |\psi_B\rangle$$

The $|\psi\rangle$ is a **separable** state because it can be written as a tensor product of two single-qubit states (see above). In contrast, suppose we apply a gate U_A to the first qubit, and a gate U_B to the second qubit. The overall operation is:

$$U_A \otimes U_B$$

Applying this to the state $|\psi\rangle$ gives:

$$(U_A \otimes U_B)(|\psi_A\rangle \otimes |\psi_B\rangle) = (U_A |\psi_A\rangle) \otimes (U_B |\psi_B\rangle)$$

The result is **still a tensor product of two separate states**. That means the final state is still separable. Therefore, **single-qubit gates preserve separability, and cannot create entanglement.**

In general, to create entanglement, qubits must interact. We need a gate that acts on both qubits together, such as the CNOT gate. The CNOT gate is the most common example of an entangling gate because under the hood is very simple, but it can create complex correlations between qubits, including the Bell states. CNOT works because it makes the second qubit (the target) depend on the first qubit (the control), thus creating correlations that cannot be reduced to independent states. Instead, single-qubit gates never do this, because each qubit evolves independently.

Deepening: Physical Meaning of Entanglement

We have introduced entanglement as a mathematical property of quantum states, but it also has profound physical implications. We said that *a state is entangled when it cannot be factorized into tensor products of individual subsystem states*. But physics asks “*what does this impossibility mean physically?*”. And the answer is **entanglement means that the subsystems no longer possess independent physical states**. That is the key idea.

❓ Before Entanglement: Independent Systems. Suppose we have two independent qubits:

$$|\psi_A\rangle = a|0\rangle + b|1\rangle \quad |\psi_B\rangle = c|0\rangle + d|1\rangle$$

The joint system is:

$$|\psi_{AB}\rangle = |\psi_A\rangle \otimes |\psi_B\rangle = ac|00\rangle + ad|01\rangle + bc|10\rangle + bd|11\rangle$$

In this case, we can assign a separate state to each qubit, and they evolve independently. The state of the whole system is just the combination of the states of the parts. Physically this means that qubit *A* has its own state, and qubit *B* has its own state, and they do not affect each other.

⚠ Entanglement: The Global State Exists, Local Ones Do Not.

Now consider the Bell state:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

We already know mathematically that this state cannot be written as a tensor product of two single-qubit states (page 118):

$$|\Phi^+\rangle \neq |v_A\rangle \otimes |v_B\rangle$$

But physically this means something deeper:

- There is a state for the **pair**;
- There is NOT a complete independent state for each particle separately.

The **system only has a meaningful description as a whole**. This is the **physical meaning**.

For example, with the Bell state $|\Phi^+\rangle$, the possible outcomes are $|00\rangle$ and $|11\rangle$. So if Alice measures her qubit and gets $|0\rangle$, Bob must get $|0\rangle$ as well. If Alice gets $|1\rangle$, Bob must get $|1\rangle$. The outcomes are perfectly correlated, but neither qubit has a definite state on its own. The state of the whole system is well-defined, but the state of each part is not. This is the essence of entanglement: the **parts do not have independent states; only the whole does.** Entanglement creates strong correlations and the particles behave as parts of a single object.

In summary, entanglement means that multiple quantum particles no longer possess independent physical states; **only the global system has a complete physical description.** Physically this means that the particles are not separate “*objects with separate properties*” anymore, but rather the information is stored in the relationship between them and the measurements become strongly correlated.

4.8 Observables and Measurements

4.8.1 Observables: What Are We Measuring?

Let's start from the basics: *what is an observable?* In quantum mechanics, an **Observable** is a mathematical object representing a measurable physical quantity.

- For example, in classical physics, observables are physical quantities that we can measure, such as position, momentum, energy, etc. (e.g., the position of a particle, the energy of a system).
- In quantum mechanics, the state is no longer a point, but rather a vector (wavefunction) in a complex vector space (Hilbert space). Because of this, observables can no longer be ordinary functions of the state, but instead must be represented as **operators** that act on the state vector.

Definition 5: Observable

In quantum mechanics, an **Observable** is a **Hermitian operator** O associated with a measurable quantity of a quantum system:

$$O = O^\dagger$$

Where $O^\dagger$ denotes the Hermitian adjoint (conjugate transpose) of O .

With *conjugate transpose*, we mean that if O is represented as a matrix, then $O^\dagger$ is obtained by taking the **transpose** of the matrix and then taking the **complex conjugate of each element**. The condition $O = O^\dagger$ ensures that the eigenvalues of O are real, which is necessary for them to correspond to measurable quantities in quantum mechanics.

Why does quantum mechanics specifically choose Hermitian operators for observables?

The answer is simple: **because measurements must produce physically meaningful outcomes**. Specifically, Hermitian operators guarantee: real eigenvalues and orthonormal eigenvectors.

1. **Real eigenvalues**. The eigenvalues of a Hermitian operator are always real numbers, which is **essential because measurement outcomes must be real values**. We cannot measure $3 + 2i$ as a physical quantity, but we can measure 3. Mathematically, if O is a Hermitian operator, then:

$$O |k\rangle = \lambda_k |k\rangle$$

Where λ_k is the possible measurement outcome (eigenvalue) corresponding to the eigenvector $|k\rangle$, and $|k\rangle$ is the state of the system after the measurement.

 Remark

When we write:

$$O |k\rangle = \lambda_k |k\rangle$$

We are choosing $|k\rangle$ to be one of the eigenvectors of O , and λ_k is its corresponding eigenvalue. It is the definition of the eigenvector/eigenvalue relationship.

Suppose O is an operator, so a matrix. An eigenvector $|k\rangle$ of O is a special vector such that, when O acts on it, the result is the **same vector direction**, only multiplied by a number. That number is called the eigenvalue λ_k . So:

$$O |k\rangle = \lambda_k |k\rangle$$

Means applying the operator O to $|k\rangle$ does not change its direction; it only scales it by λ_k .

Example 10: Eigenvectors and Eigenvalues of the Pauli-Z Operator

For example, let's take the Pauli-Z operator:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

And the state $|0\rangle$:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Applying Z to $|0\rangle$:

$$Z |0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1 \cdot |0\rangle = \lambda_0 |0\rangle$$

Therefore, $|0\rangle$ is an eigenvector of Z with eigenvalue $\lambda_0 = 1$. Similarly, applying Z to $|1\rangle$:

$$Z |1\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} = -1 \cdot |1\rangle = \lambda_1 |1\rangle$$

Therefore, $|1\rangle$ is an eigenvector of Z with eigenvalue $\lambda_1 = -1$.

Beyond the mathematical definition, $|0\rangle$ is a special state of the observable Z , measuring Z when the system is in $|0\rangle$ always gives the outcome $+1$. So, when we say that $|0\rangle$ and $|1\rangle$ are eigenvectors of Z , we are saying that these are the states in which the measurement of Z is **perfectly predictable**, yielding the eigenvalues $+1$ and -1 , respectively.

This is a fundamental aspect, because:

✓ If the **system is already in an eigenvector**: there is no uncertainty, and the measurement outcome is known with certainty.

⚠ In contrast, if the **system is not in an eigenvector**: it is a combination of eigenvectors, and measurement becomes **probabilistic**. For example, suppose the qubit is in:

$$|+\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

This is **not** an eigenvector of Z , because applying Z :

$$\begin{aligned} Z|+\rangle &= \frac{1}{\sqrt{2}} \left(\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right) \\ &= \frac{1}{\sqrt{2}} \left(\begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ -1 \end{bmatrix} \right) \\ &= \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\ &= |-\rangle \end{aligned}$$

❓ **Why is $|+\rangle$ not an eigenvector of Z ?** If $|+\rangle$ were an eigenvector, we would have by definition:

$$Z|+\rangle = \lambda|+\rangle$$

For some eigenvalue λ . However, as we calculated, applying Z to $|+\rangle$ gives us $|-\rangle$, which is a different state. Therefore, there is no scalar λ such that $Z|+\rangle = \lambda|+\rangle$, confirming that $|+\rangle$ is not an eigenvector of Z . Physically, this means that if the system is in state $|+\rangle$, measuring Z does not yield a definite outcome; instead, it yields $+1$ with probability 0.5 and -1 with probability 0.5, reflecting the superposition of eigenstates.

In summary, if applying an observable O to a state $|\psi\rangle$ produces the same state multiplied by a scalar:

$$O|\psi\rangle = \lambda|\psi\rangle$$

Then $|\psi\rangle$ is an eigenvector of O , and λ is its eigenvalue. In this case, the state already has a definite value for that observable, so measuring O will return λ with certainty.

✓ **Physical consequence.** If the system is in an eigenvector: the **measurement is deterministic**, the **result is known in advance**, the **outcome is the corresponding eigenvalue**.

Why Hermitian operators have real eigenvalues. Let A be a Hermitian operator:

$$A = A^\dagger$$

Assume that $|x\rangle$ is an eigenvector of A with eigenvalue λ :

$$A|x\rangle = \lambda|x\rangle$$

Consider the quantity:

$$\langle x|A|x\rangle$$

Using the eigenvalue equation (i.e., $A|x\rangle = \lambda|x\rangle$) on the right:

$$\langle x|\underbrace{A|x\rangle}_{\lambda|x\rangle} = \langle x|(\lambda|x\rangle) = \lambda\langle x|x\rangle$$

This is possible because λ is just a scalar, so it can be moved outside bra-ket notation.³ Now take the Hermitian conjugate of the eigenvalue equation:

$$A|x\rangle = \lambda|x\rangle \implies \langle x|A^\dagger = \bar{\lambda}\langle x|$$

Since A is Hermitian:

$$A^\dagger = A$$

Therefore:

$$\langle x|A = \bar{\lambda}\langle x|$$

Applying this on the left of $\langle x|A|x\rangle$:

$$\underbrace{\langle x|A}_{\bar{\lambda}\langle x|}|x\rangle = (\bar{\lambda}\langle x|)|x\rangle = \bar{\lambda}\langle x|x\rangle$$

Combining the two expressions:

$$\lambda\langle x|x\rangle = \bar{\lambda}\langle x|x\rangle$$

Since:

$$\langle x|x\rangle > 0$$

Because the inner product of a vector with itself is always positive (unless the vector is the zero vector, which cannot be an eigenvector), we can divide both sides by $\langle x|x\rangle$:

$$\lambda = \bar{\lambda}$$

Therefore, λ is real.

QED

³Remember that bras and kets can be represented using vectors and matrices, so the usual rules of linear algebra apply. In particular, since λ is a scalar, it can be factored out of an inner product:

$$\langle x|(\lambda|x\rangle) = \lambda\langle x|x\rangle$$

More generally, scalar multiplication commutes with matrix products. Therefore, for matrices A and B :

$$\lambda AB = A\lambda B = AB\lambda$$

Provided that the order of the matrices themselves is preserved.

2. **Orthonormal eigenvectors.** A vector is:

- **Orthogonal** if its inner product with another vector is zero (i.e., they are perpendicular in the vector space). For example, in a 2D space, the vectors $|0\rangle$ and $|1\rangle$ are orthogonal because:

$$\langle 0|1\rangle = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0$$

- **Normalized** if its inner product with itself is equal to one (i.e., it has unit length). For example:

$$\langle 0|0\rangle = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1$$

This property is crucial because quantum measurement requires a basis whose states are both perfectly distinguishable (orthogonal) and probabilistically consistent (normalized).

- ✗ **If eigenvectors were not orthogonal**, different measurement states could partially overlap:

$$\langle v_1|v_2\rangle \neq 0$$

In this case, the measurement process would not be able to distinguish the states perfectly, leading to ambiguity in the interpretation of measurement outcomes.

Example 11: Why Orthogonality Matters

Consider the two states:

$$|v_1\rangle = |0\rangle \quad |v_2\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Their inner product is:

$$\langle v_1|v_2\rangle = \langle 0| \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = \frac{1}{\sqrt{2}}$$

Since:

$$\langle v_1|v_2\rangle \neq 0$$

the two states are not orthogonal. This means that the **states partially overlap**, and therefore a **measurement cannot distinguish them perfectly**. For example, if the system is in state $|v_2\rangle$, there is still a non-zero probability of confusing it with $|v_1\rangle$ during a measurement. Therefore, the measurement outcomes would be ambiguous, and we would not be able to assign a clear physical meaning to the measurement results.

- ✗ **If eigenvectors were not normalized:**

$$\langle v|v\rangle \neq 1$$

Then the Born rule (page 83) would not produce properly normalized probabilities, making the probabilistic interpretation of quantum mechanics inconsistent.

Example 12: Why Normalization Matters

Consider the state:

$$|v\rangle = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

Its norm is:

$$\langle v|v\rangle = \begin{bmatrix} 2 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} = 4$$

Therefore, the state is not normalized.

If we tried to interpret the amplitudes probabilistically using the Born rule, we would obtain:

$$p_0 = |2|^2 = 4$$

Which is **impossible**, since **probabilities must lie between 0 and 1**, and their total sum must equal 1.

To obtain a valid quantum state, we must normalize the vector:

$$|v_{\text{norm}}\rangle = \frac{1}{2} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Therefore, without normalization, we would end up with an invalid quantum state that cannot be interpreted probabilistically, and the measurement outcomes would not correspond to any physical reality.

Why eigenvectors of Hermitian operators are orthogonal. Let A be Hermitian and let:

$$A|x\rangle = \lambda|x\rangle, \quad A|y\rangle = \mu|y\rangle$$

Where $\lambda \neq \mu$ are the eigenvalues corresponding to the eigenvectors $|x\rangle$ and $|y\rangle$, respectively. Consider the inner product:

$$\langle y|A|x\rangle$$

Because we want to show that $|x\rangle$ and $|y\rangle$ are orthogonal, we need to show that $\langle y|x\rangle = 0$. Using the eigenvalue equation on the right:

$$\langle y|A|x\rangle = \langle y|(\lambda|x\rangle) = \lambda\langle y|x\rangle$$

We moved λ outside the inner product because it is a scalar (see the footnote in the previous proof for more details on this step). Now use the Hermitian property on the left:

$$\underbrace{\langle y|A}_{\langle y|A^\dagger} = \mu\langle y|$$

Therefore:

$$\langle y| A |x\rangle = \mu \langle y|x\rangle$$

Combining the two expressions:

$$\lambda \langle y|x\rangle = \mu \langle y|x\rangle$$

Thus:

$$(\lambda - \mu) \langle y|x\rangle = 0$$

Since $\lambda \neq \mu$ by assumption, we must have:

$$\langle y|x\rangle = 0$$

Therefore, the eigenvectors are orthogonal.

QED

We can think of a **Hermitian operator** as a sort of “*machine*” that **decomposes the quantum state into measurable components** (eigenvectors) **and assigns a real value** (eigenvalue) **to each component**, which corresponds to the possible outcomes of a measurement. The orthonormality of the eigenvectors ensures that these outcomes are distinct and can be interpreted probabilistically in a consistent way. In summary, a Hermitian observable tells us:

- *What outcomes are possible when we measure a quantum system.*
- *With respect to which basis we are measuring the system.*
- *Into which states the system will collapse after the measurement.*

Spectral Theorem

So far, we know that an observable O is a Hermitian operator, it has eigenvectors $|k\rangle$, and each eigenvector has an eigenvalue λ_k . So if the system is in $|k\rangle$, measuring O returns λ_k with certainty. Now, a natural question arises: **what happens if the system is in an arbitrary state $|k\rangle$, not necessarily one of the eigenvectors?** To answer this question, the **Spectral Theorem** comes into play by showing that **every observable can be written as a sum of its eigenvectors and eigenvalues**.

Suppose the observable has a set of eigenvectors $|k\rangle$ with corresponding eigenvalues λ_k . Since the eigenvectors form an orthonormal basis, any quantum state $|v\rangle$ can be expressed as a linear combination of these eigenvectors (page 93):

$$|v\rangle = \sum_k c_k |k\rangle$$

Where c_k are complex coefficients. The **Spectral Theorem** states that the observable O can be expressed as:

$$O = \sum_k \lambda_k |k\rangle\langle k| \quad (61)$$

Where each term $\lambda_k |k\rangle\langle k|$ contains two pieces of information:

- **Projector** $|k\rangle\langle k|$ (page 80) onto the eigenvector $|k\rangle$, which extracts the component of the state along $|k\rangle$ during measurement. It is defined as:

$$|k\rangle\langle k| = |k\rangle \langle k|$$

- **Eigenvalue** λ_k , which assigns the measurement outcome associated with that component.

🔍 **Why the Spectral Theorem is important?** Suppose someone gives us an observable O without telling us its eigenvectors or eigenvalues:

$$O = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Without the Spectral Theorem, the physical meaning of the observable would not be immediately apparent. However, by applying the Spectral Theorem, we can decompose O into its eigenvectors and eigenvalues, allowing us to **understand the measurement process and predict the outcomes based on the state of the system**. For example, we can find the eigenvalues and eigenvectors of O :

- To find the eigenvalues, we solve the characteristic equation:

$$\begin{aligned} \det(O - \lambda I) &= 0 \\ \det\left(\begin{bmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{bmatrix}\right) &= 0 \\ (2-\lambda)(2-\lambda) - 1 \cdot 1 &= 0 \\ 4 - 4\lambda + \lambda^2 - 1 &= 0 \\ \lambda^2 - 4\lambda + 3 &= 0 \\ (\lambda - 3)(\lambda - 1) &= 0 \end{aligned}$$

Thus, the eigenvalues are $\lambda_1 = 3$ and $\lambda_2 = 1$.

- Eigenvectors can be found by solving $(O - \lambda I)|k\rangle = 0$ for each eigenvalue:
 - For $\lambda_1 = 3$:

$$\begin{aligned} (O - 3I)|v_1\rangle &= 0 \\ \left(\begin{bmatrix} 2-3 & 1 \\ 1 & 2-3 \end{bmatrix}\right)|v_1\rangle &= 0 \\ \left(\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix}\right)|v_1\rangle &= 0 \end{aligned}$$

Now to find the eigenvector, we need to solve the system of equations:

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

That is a homogeneous system of linear equations:

$$\begin{cases} -x + y = 0 \\ x - y = 0 \end{cases} = \begin{cases} y = x \\ x = y \end{cases}$$

So any vector of the form (x, x) is an eigenvector. We choose the simplest one $(1, 1)$, and then we normalize it:

$$\left\| \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\| = \sqrt{1^2 + 1^2} = \sqrt{2}$$

Therefore, the normalized eigenvector is:

$$|v_1\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

– For $\lambda_2 = 1$:

$$\begin{aligned} (O - 1I) |v_2\rangle &= 0 \\ \left(\begin{bmatrix} 2-1 & 1 \\ 1 & 2-1 \end{bmatrix} \right) |v_2\rangle &= 0 \\ \left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \right) |v_2\rangle &= 0 \end{aligned}$$

We need to solve the system of equations:

$$\begin{cases} x + y = 0 \\ x + y = 0 \end{cases} \Rightarrow x + y = 0 \Rightarrow y = -x \xrightarrow{\text{substitute from } \begin{bmatrix} x \\ y \end{bmatrix}} \begin{bmatrix} x \\ -x \end{bmatrix}$$

So any vector of the form $(x, -x)$ is an eigenvector. We choose the simplest one $(1, -1)$, and then we normalize it:

$$\left\| \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right\| = \sqrt{1^2 + (-1)^2} = \sqrt{2}$$

Therefore, the normalized eigenvector is:

$$|v_2\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

Then we can express O in terms of its spectral decomposition:

$$O = 3 |v_1\rangle\langle v_1| + 1 |v_2\rangle\langle v_2|$$

From this decomposition, we can immediately read off the possible outcomes from the eigenvalues (3 and 1), the measurement basis from the eigenvectors $(|v_1\rangle$ and $|v_2\rangle)$, and the measurement operators from the projectors $(|v_1\rangle\langle v_1|$ and $|v_2\rangle\langle v_2|)$. It also allows us to calculate the probabilities of obtaining each outcome when measuring O on an arbitrary state $|\psi\rangle$ using the Born rule:

$$p_1 = |\langle v_1 | \psi \rangle|^2 \quad p_2 = |\langle v_2 | \psi \rangle|^2$$

Where p_1 and p_2 are the probabilities of obtaining the outcomes 3 and 1, respectively, when measuring the observable O on a state $|\psi\rangle$. And finally the expectation value of O in the state $|\psi\rangle$ can be calculated as:

$$\langle \psi | O | \psi \rangle = \sum_k \lambda_k p_k$$

In this example, this becomes:

$$\langle \psi | O | \psi \rangle = \sum_{k=1}^2 \lambda_k p_k = 3p_1 + 1p_2$$

In summary, the Spectral Theorem transforms an abstract Hermitian matrix into a complete description of a quantum measurement. It reveals:

- The **possible measurement outcomes** (eigenvalues),
- The **corresponding post-measurement states** (eigenvectors),
- The **projectors used to compute probabilities**,
- And the **expectation value** as a weighted average of the outcomes.

4.8.2 Measurement Operators and Projectors

In this section, we will explain **how measurement is actually performed mathematically**.

Measurement Operators

After introducing the Spectral Theorem, we know that any observable can be written as:

$$O = \sum_k \lambda_k |k\rangle\langle k|$$

Where:

- λ_k are the possible measurement outcomes (eigenvalues);
- $|k\rangle$ are the corresponding eigenvectors;
- $|k\rangle\langle k|$ are projection operators onto the eigenvectors.

At this point, a natural question arises: **what role do these projectors play in the measurement process?**

The answer is that each projector:

$$M_k = |k\rangle\langle k|$$

Is called a **Measurement Operator** (page 80). It represents the mathematical operation associated with the measurement outcome λ_k .

Definition 6: Measurement Operator

Given an observable (page 126):

$$O = \sum_k \lambda_k |k\rangle\langle k|$$

The **Measurement Operator** associated with the outcome λ_k is:

$$M_k = |k\rangle\langle k| = \begin{pmatrix} k_1 \\ k_2 \end{pmatrix} \begin{pmatrix} \overline{k_1} & \overline{k_2} \end{pmatrix} = \begin{pmatrix} k_1 \overline{k_1} & k_1 \overline{k_2} \\ k_2 \overline{k_1} & k_2 \overline{k_2} \end{pmatrix}$$

In other words, **each possible measurement outcome has an associated projector** that:

1. **Extracts the component** of the quantum state along the eigenvector $|k\rangle$;
2. **Determines the probability** of obtaining the corresponding eigenvalue λ_k ;
3. **Produces the post-measurement** state after collapse.

For this reason, a measurement operator can be thought of as a **detector** tuned to a specific eigenstate. If the detector corresponding to $|k\rangle$ “clicks”, then:

- The measurement outcome is λ_k ;
- The system collapses to the state $|k\rangle$.

For example, suppose an observable has two eigenstates $|0\rangle$ and $|1\rangle$. Then there are two measurement operators:

$$M_0 = |0\rangle\langle 0|, \quad M_1 = |1\rangle\langle 1|$$

We can imagine them as two detectors:

- Detector M_0 looks for the component of the state along $|0\rangle$.
- Detector M_1 looks for the component of the state along $|1\rangle$.

Suppose the state is:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

This means that part of the state points along $|0\rangle$, and part of the state points along $|1\rangle$. The detector $M_0 = |0\rangle\langle 0|$ asks “*how much of the state is aligned with $|0\rangle$?*” and extracts $\alpha |0\rangle$. The detector $M_1 = |1\rangle\langle 1|$ asks “*how much of the state is aligned with $|1\rangle$?*” and extracts $\beta |1\rangle$.

Example 13: Example: Measurement Operators for Pauli-Z

The Pauli-Z observable can be written as:

$$Z = |0\rangle\langle 0| - |1\rangle\langle 1|$$

Therefore, the associated measurement operators are:

$$M_0 = |0\rangle\langle 0|, \quad M_1 = |1\rangle\langle 1|$$

These operators correspond to the two possible outcomes:

- +1, associated with the eigenstate $|0\rangle$;
- -1, associated with the eigenstate $|1\rangle$.

Suppose the system is in the state:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

Applying the measurement operator M_0 :

$$M_0 |\psi\rangle = |0\rangle\langle 0| (\alpha |0\rangle + \beta |1\rangle) = \alpha |0\rangle$$

This means that the operator M_0 extracts only the component of the state aligned with $|0\rangle$ and removes the orthogonal component $|1\rangle$.

The probability of obtaining the corresponding outcome is:

$$p_0 = \langle \psi | M_0 | \psi \rangle = |\alpha|^2$$

Similarly:

$$p_1 = \langle \psi | M_1 | \psi \rangle = |\beta|^2$$

Therefore, **measurement operators** provide the complete mathematical description of the measurement process: they **isolate the relevant component** of the quantum state, **determine the probability** of each outcome, and **specify the state** after the measurement.

☐ Completeness of the Basis

In the previous example, we defined two measurement operators M_0 and M_1 corresponding to the two eigenstates of the Pauli-Z observable. But what if we had an observable with more eigenstates? Or what if we had a different observable with a completely different set of eigenvectors? Therefore, **how do we know that the measurement operators we defined account for all possible outcomes of the measurement?**

The answer is that the eigenvectors of a Hermitian observable form a **complete orthonormal basis**. This means that any quantum state can be expressed as a linear combination of these eigenvectors:

$$|\psi\rangle = \sum_k c_k |k\rangle$$

Mathematically, this property is expressed by the **Completeness Relation**:

$$\sum_k |k\rangle\langle k| = I$$

Where I is the identity operator (page 49).

☐ Definition 7: Completeness Relation

If $\{|k\rangle\}$ is a complete orthonormal basis, then:

$$\sum_k |k\rangle\langle k| = I \quad (62)$$

Since the measurement operators are defined as:

$$M_k = |k\rangle\langle k|$$

It follows that:

$$\sum_k M_k = I$$

This means that the **set of measurement operators is complete and accounts for all possible outcomes of the measurement.**

🔗 Deepening: When a set of vectors $\{|k\rangle\}$ is a complete orthonormal basis?

A set of vectors $\{|k\rangle\}$ is called a **Complete Orthonormal Basis** when it satisfies **three conditions**:

1. **Normalized**, each vector has length 1: $\langle k|k \rangle = 1$.
2. **Orthogonal**, different vectors are perpendicular: $\langle k|j \rangle = 0$ for $k \neq j$.
3. **Complete**, the vectors span the entire Hilbert space, meaning that every quantum state can be written as a linear combination of them. That is, for any state $|\psi\rangle$, there exist coefficients c_k such that:

$$|\psi\rangle = \sum_k c_k |k\rangle$$

Compactly, the *normalized* and *orthogonal* conditions are written as:

$$\langle k|j \rangle = \delta_{kj} \quad (63)$$

Where δ_{kj} is the **Kronecker delta**, which is 1 if $k = j$ and 0 otherwise:

$$\delta_{kj} = \begin{cases} 1 & k = j \\ 0 & k \neq j \end{cases} \quad (64)$$

And the *completeness* condition is expressed by the completeness relation:

$$\sum_k |k\rangle\langle k| = I$$



Example 14: Single-Qubit Computational Basis

For a single qubit, the computational basis consists of the vectors:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

These vectors are:

- Normalized: $\langle 0|0 \rangle = 1$ and $\langle 1|1 \rangle = 1$.
- Orthogonal: $\langle 0|1 \rangle = 0$.
- Complete: any single-qubit state can be expressed as a linear combination of $|0\rangle$ and $|1\rangle$.

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

This Equation 62 states that the **set of measurement operators is complete**: together, they **describe every possible outcome of the measurement**.

❓ Why is the completeness relation important? To understand why this relation is important, let us apply both sides to an arbitrary state:

$$|\psi\rangle = \sum_k c_k |k\rangle$$

Where c_k are the coefficients of $|\psi\rangle$ in the basis $\{|k\rangle\}$.

Now take the inner product with $\langle k|$ on both sides:

$$\langle k|\psi\rangle = \langle k|\left(\sum_j c_j |j\rangle\right)$$

Using linearity of the inner product:

$$\langle k|\psi\rangle = \sum_j c_j \langle k|j\rangle$$

Since the basis is orthonormal, $\langle k|j\rangle = \delta_{kj}$, so only the term where $j = k$ survives (Equation 63, page 139):

$$\langle k|j\rangle = \delta_{kj} = \begin{cases} 1 & k = j \\ 0 & k \neq j \end{cases}$$

Thanks to the Kronecker delta, all terms in the sum vanish except the one where $j = k$:

$$\langle k|\psi\rangle = c_k \langle k|k\rangle = c_k \cdot 1 = c_k$$

In other words, the bra $\langle k|$ acts as a coefficient extractor: it selects exactly the amount of $|\psi\rangle$ aligned with the basis vector $|k\rangle$.

Finally, apply the sum of all projectors to $|\psi\rangle$:

$$\left(\sum_k |k\rangle\langle k|\right) |\psi\rangle = \sum_k |k\rangle \langle k|\psi\rangle = \sum_k |k\rangle c_k = \sum_k c_k |k\rangle = |\psi\rangle$$

This shows that the **sum of all projectors reproduces the original state exactly**. In other words, **each projector $|k\rangle\langle k|$ extracts one component of the state, and when all these components are added together, the original state is reconstructed without losing any information.** Therefore:

$$\sum_k |k\rangle\langle k| = I$$

Example 15: Computational Basis

For a single qubit, the computational basis is:

$$|0\rangle, \quad |1\rangle$$

The corresponding projectors are:

$$|0\rangle\langle 0| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad |1\rangle\langle 1| = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

Their sum is:

$$|0\rangle\langle 0| + |1\rangle\langle 1| = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

The completeness relation also guarantees that the probabilities of all possible outcomes sum to one. If:

$$p_k = \langle \psi | M_k | \psi \rangle$$

Then:

$$\begin{aligned} \sum_k p_k &= \sum_k \langle \psi | M_k | \psi \rangle \\ &= \langle \psi | \left(\sum_k M_k \right) | \psi \rangle \\ &= \langle \psi | I | \psi \rangle \\ &= \langle \psi | \psi \rangle \\ &= 1 \end{aligned}$$

Therefore, the **completeness relation** ensures that:

- **All possible measurement outcomes** are included;
- **No probability is lost**;
- The **total probability** is always **equal to one**.

4.8.3 Expectation Value

In the previous sections, we have seen how to compute the probability of obtaining a certain measurement outcome when measuring an observable on a quantum state. However, in many cases, we are **interested in the average value of an observable over many measurements**, rather than the probability of specific outcomes. This average value is known as the **expectation value** of the observable.

Definition 8: Expectation Value

Let O be an observable represented by a Hermitian operator, and let $|\psi\rangle$ be a quantum state. The quantity:

$$\langle O \rangle = \langle \psi | O | \psi \rangle = \sum_k \lambda_k p_k \quad (65)$$

Is called the **Expectation Value** of the observable O in the state $|\psi\rangle$.

 **Physical Meaning.** The expectation value $\langle O \rangle$ can be interpreted as the **average result obtained by measuring the observable O many times on identical copies of the state $|\psi\rangle$.**

1. Prepare many identical copies of the state $|\psi\rangle$.
2. Measure the observable O on each copy.
3. Record the outcomes.
4. Compute the arithmetic mean of the recorded outcomes.

As the number of experiments increases, the average converges to the expectation value $\langle \psi | O | \psi \rangle$.

Important Clarification

The expectation value is **not necessarily** one of the possible measurement outcomes. It is the **average (more precisely, the probability-weighted average)** of many outcomes, and can take values that are not eigenvalues of the observable. For example, if possible outcomes are $+1$ with probability 50% and -1 with probability 50%, then the expectation value is:

$$(+1) \cdot \frac{1}{2} + (-1) \cdot \frac{1}{2} = 0$$

Although the actual measurement never returns 0, the expectation value is 0 because it represents the average of many measurements. This means that half of the measurements return $+1$ and the other half return -1 , resulting in an average of 0.

The expectation value tells us the “center of mass” of the measurement distribution. **It summarizes the statistical behavior of the measurement in a single number.**

❓ **Why are we interested in the average value of an observable rather than in the probability of specific outcomes?** Because in many real situations we do not **care about** the result of a single measurement, but about the **overall behavior of the system**. Also, it proves a stable, experimentally meaningful quantity that summarizes the statistical behavior of an observable over many measurements, even when individual outcomes are random.

✂ **How can we compute the expectation value?**

To compute the expectation value $\langle O \rangle$ of an observable O in a state $|\psi\rangle$, we start from the Spectral Theorem (page 132):

$$O = \sum_k \lambda_k |k\rangle\langle k|$$

Substitute this into the expectation value formula (page 142):

$$\begin{aligned} \langle \psi | O | \psi \rangle &= \langle \psi | \left(\sum_k \lambda_k |k\rangle\langle k| \right) | \psi \rangle \\ &\downarrow \text{ use linearity of the inner product} \\ &= \sum_k \lambda_k \langle \psi | k \rangle \langle k | \psi \rangle \end{aligned}$$

Using the property of inner products (page 10):

$$\langle \psi | k \rangle = \overline{\langle k | \psi \rangle} \quad \implies \quad \overline{\langle k | \psi \rangle} \langle k | \psi \rangle = |\langle k | \psi \rangle|^2$$

Here, the Born rule (page 83) states that $|\langle k | \psi \rangle|^2$ is the probability of measuring the outcome λ_k when measuring O on the state $|\psi\rangle$. Therefore, we can rewrite the expectation value as:

$$p_k = |\langle k | \psi \rangle|^2 \quad \implies \quad \langle \psi | O | \psi \rangle = \sum_k \lambda_k p_k \quad (66)$$

This formula shows that the expectation value is the **weighted average** of the possible outcomes λ_k , where each outcome is weighted by its probability p_k . In other words, the **expectation value is the sum of all possible outcomes multiplied by their respective probabilities**, which is consistent with our interpretation of it as an average over many measurements.

4.8.4 Measuring $|+\rangle$ with the Pauli-Z Observable

After introducing observables, projectors, probabilities, and expectation values, it is useful to analyze a concrete example that combines all these concepts.

Consider the quantum state:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

And the Pauli-Z observable:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Calculate the possible measurement outcomes, their probabilities, the post-measurement states, and the expectation value of Z in the state $|+\rangle$.

From the spectral decomposition (page 132) of Z , we know that:

- The **eigenvalues** of Z are $+1$ and -1 :

$$\begin{aligned} \det(Z - \lambda I) &= 0 \\ \det\left(\begin{bmatrix} 1-\lambda & 0 \\ 0 & -1-\lambda \end{bmatrix}\right) &= 0 \\ (1-\lambda)(-1-\lambda) &= 0 \\ \lambda &= \pm 1 \end{aligned}$$

- The **first eigenvector** $|v_1\rangle$ is $|0\rangle$:

$$\begin{aligned} (Z - 1I)|v_1\rangle &= 0 \\ \begin{bmatrix} 1-1 & 0 \\ 0 & -1-1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} &= 0 \\ \begin{bmatrix} 0 & 0 \\ 0 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} &= 0 \\ \begin{cases} 0 \cdot x + 0 \cdot y = 0 \\ 0 \cdot x - 2 \cdot y = 0 \end{cases} &\implies y = 0 \end{aligned}$$

So any vector of the form $\begin{pmatrix} x \\ y \end{pmatrix} \xrightarrow{y=0} \begin{bmatrix} x \\ 0 \end{bmatrix}$ is an eigenvector. We choose the simplest one $(1, 0)$, and then we normalize it:

$$\left\| \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right\| = \sqrt{1^2 + 0^2} = 1$$

Therefore, the normalized eigenvector is:

$$|v_1\rangle = 1 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

- Similarly, the **second eigenvector** $|v_2\rangle$ is $|1\rangle$:

$$\begin{aligned}
 (Z + 1I) |v_2\rangle &= 0 \\
 \begin{bmatrix} 1+1 & 0 \\ 0 & -1+1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} &= 0 \\
 \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} &= 0 \\
 \begin{cases} 2 \cdot x + 0 \cdot y = 0 \\ 0 \cdot x + 0 \cdot y = 0 \end{cases} &\implies x = 0
 \end{aligned}$$

So any vector of the form $\begin{pmatrix} x \\ y \end{pmatrix} \xrightarrow{x=0} \begin{bmatrix} 0 \\ y \end{bmatrix}$ is an eigenvector. We choose the simplest one $(0, 1)$, and then we normalize it:

$$\left\| \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right\| = \sqrt{0^2 + 1^2} = 1$$

Therefore, the normalized eigenvector is:

$$|v_2\rangle = 1 \cdot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = |1\rangle$$

Therefore:

- The possible **measurement outcomes** are **+1** and **-1**;
- The corresponding **eigenstates** are **$|0\rangle$** and **$|1\rangle$** ;
- The **measurement operators** are:

$$M_0 = |0\rangle \langle 0|, \quad M_1 = |1\rangle \langle 1|$$

Measurement Probabilities

The state $|+\rangle$ is an equal superposition of $|0\rangle$ and $|1\rangle$:

$$|+\rangle = \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle$$

Therefore, the amplitudes are:

$$\alpha = \frac{1}{\sqrt{2}}, \quad \beta = \frac{1}{\sqrt{2}}$$

Using the Born rule:

$$p_0 = |\alpha|^2 = \frac{1}{2}, \quad p_1 = |\beta|^2 = \frac{1}{2}$$

Thus:

- There is a 50% probability of obtaining the outcome +1;
- There is a 50% probability of obtaining the outcome -1.

Post-Measurement States

If the measurement outcome is:

- $+1$, the state collapses to $|0\rangle$;
- -1 , the state collapses to $|1\rangle$.

Expectation Value

The expectation value of Z in the state $|+\rangle$ is:

$$\langle + | Z | + \rangle = (+1) \cdot \frac{1}{2} + (-1) \cdot \frac{1}{2} = 0$$

Remark

The expectation value is 0, but this does **not** mean that the measurement can return the value 0. The only possible outcomes are:

$$+1 \quad \text{and} \quad -1$$

The value 0 is simply the probability-weighted average of these outcomes over many repeated measurements.

What if the state to be measured is already an eigenstate of the observable?

Let's assume that now we want to calculate the measurement outcomes, probabilities, post-measurement states, and expectation value when the input state is $|0\rangle$ or $|1\rangle$ instead of $|+\rangle$. How we have seen, $|0\rangle$ and $|1\rangle$ are eigenvectors of Z with eigenvalues $+1$ and -1 , respectively. This means that they are already measurement states of the observable Z . Let's analyze what happens in these cases.

- **Measuring $|0\rangle$:** the measurement probabilities of $|0\rangle$ are:

$$|0\rangle = 1 \cdot |0\rangle + 0 \cdot |1\rangle$$

Therefore:

$$p_0 = |\alpha|^2 = 1, \quad p_1 = |\beta|^2 = 0$$

Using observable Z , there is a 100% probability of obtaining the outcome $+1$ and a 0% probability of obtaining the outcome -1 . Moreover, the (post-)measurement outcome is always $+1$, and the post-measurement state remains $|0\rangle$ because we have 100% probability of collapsing to $|0\rangle$ and 0% probability of collapsing to $|1\rangle$. Finally, the expectation value is:

$$\langle 0 | Z | 0 \rangle = (+1) \cdot 1 + (-1) \cdot 0 = +1$$

And indeed, the expectation value confirms that the measurement outcome is always $+1$.

- **Measuring** $|1\rangle$: the measurement probabilities of $|1\rangle$ are:

$$|1\rangle = 0 \cdot |0\rangle + 1 \cdot |1\rangle$$

Therefore:

$$p_0 = |\alpha|^2 = 0, \quad p_1 = |\beta|^2 = 1$$

Using observable Z , there is a 0% probability of obtaining the outcome $+1$ and a 100% probability of obtaining the outcome -1 . Moreover, the (post-)measurement outcome is always -1 , and the post-measurement state remains $|1\rangle$ because we have 0% probability of collapsing to $|0\rangle$ and 100% probability of collapsing to $|1\rangle$. Finally, the expectation value is:

$$\langle 1|Z|1\rangle = (+1) \cdot 0 + (-1) \cdot 1 = -1$$

And indeed, the expectation value confirms that the measurement outcome is always -1 .

In summary, when a quantum state is already an eigenvector of the observable being measured:

- The **measurement** outcome is **deterministic**;
- The **probability** of the corresponding eigenvalue is **1**;
- The **state does not change** after the measurement;
- The **expectation value** is exactly the associated **eigenvalue**.

In contrast, when the state is a superposition of eigenvectors (as in the case of $|+\rangle$), the measurement outcome is probabilistic and the expectation value becomes a weighted average of the possible outcomes.

4.8.5 Recovering Probabilities from Expectation Values

In the previous sections, we have seen that the expectation value of an observable is the weighted average of its possible measurement outcomes. But, is possible to go in the opposite direction? That is, **if we know the expectation value, can we determine the probabilities of the individual outcomes?** Let's explore how to get **Probabilities from the Expectation Value**.

Let's consider the Pauli-Z observable as an example.

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

With (proof at page 144):

- Eigenvalue $+1$ associated with the state $|0\rangle$;
- Eigenvalue -1 associated with the state $|1\rangle$.

Let:

- p_0 be the probability of measuring the state $|0\rangle$;
- p_1 be the probability of measuring the state $|1\rangle$.

Since Pauli-Z is a valid observable, the probabilities must satisfy:

$$p_0 + p_1 = 1$$

Therefore, the expectation value of Z is:

$$\langle Z \rangle = (+1)p_0 + (-1)p_1 = p_0 - p_1$$

Where $+1$ and -1 are the eigenvalues of Z corresponding to the states $|0\rangle$ and $|1\rangle$, respectively. Instead, p_0 and p_1 are the probabilities of obtaining those states upon measurement.

💡 Now, we are interested in the opposite direction: given the expectation value $m = \langle Z \rangle$ (we change the notation just to be general), can we determine p_0 and p_1 ? The answer is yes, and we can derive the formulas for p_0 and p_1 in terms of m (or $\langle Z \rangle$). We have two relationships:

$$\begin{aligned} p_0 + p_1 &= 1 \\ p_0 - p_1 &= m \end{aligned}$$

We can **solve this system of equations to express p_0 and p_1 in terms of m , and thus recover the probabilities from the expectation value.** Let's do that step by step.

Let's try to solve this system of equations to express p_0 and p_1 in terms of m :

$$\begin{aligned}
 \begin{cases} p_0 + p_1 = 1 \\ p_0 - p_1 = m \end{cases} &= \begin{cases} p_0 = 1 - p_1 \\ p_1 = -m + p_0 \end{cases} \\
 &= \begin{cases} p_0 = 1 - p_1 \\ p_1 = -m + (1 - p_1) \end{cases} \\
 &= \begin{cases} p_0 = 1 - p_1 \\ 2p_1 = -m + 1 \end{cases} \\
 &= \begin{cases} p_0 = 1 - p_1 \\ p_1 = \frac{1-m}{2} \end{cases} \\
 &= \begin{cases} p_0 = 1 - \frac{1-m}{2} \\ p_1 = \frac{1-m}{2} \end{cases} \\
 &= \begin{cases} p_0 = \frac{2-(1-m)}{2} \\ p_1 = \frac{1-m}{2} \end{cases} \\
 &= \begin{cases} p_0 = \frac{1+m}{2} \\ p_1 = \frac{1-m}{2} \end{cases}
 \end{aligned}$$

So, we have:

$$p_0 = \frac{1+m}{2} \quad \text{and} \quad p_1 = \frac{1-m}{2} \quad (67)$$

🔍 Why is this important? This result shows that the expectation value of the Pauli-Z observable contains enough information to reconstruct the complete probability distribution of the measurement outcomes. In other words, by measuring only the average value $m = \langle Z \rangle$, we can determine:

- The probability p_0 of obtaining the state $|0\rangle$;
- The probability p_1 of obtaining the state $|1\rangle$.

⚠️ Not always true in general

For a general observable with more than two outcomes, one expectation value is usually **not enough**. In general, to determine n unknown probabilities, we need n independent equations. Normalization provides one equation, so we typically need additional information such as: more expectation values, higher moments (e.g., $\langle O^2 \rangle$), or other constraints to fully determine the probability distribution.

However, if an observable has only two distinct outcomes a and b , then the same idea works. If:

- Outcome a occurs with probability p ,
- Outcome b occurs with probability $1 - p$,

Then the expectation value is:

$$\langle O \rangle = a \cdot p + b \cdot (1 - p) \xrightarrow{\text{solve for } p} p = \frac{\langle O \rangle - b}{a - b} \quad (68)$$

It finds **the probability of the first outcome** a , and the **probability of the second outcome** b can be found by normalization:

$$1 - p = 1 - \frac{\langle O \rangle - b}{a - b} = \frac{a - \langle O \rangle}{a - b} \quad (69)$$

For example, for the Pauli-Z observable, we have $a = +1$ and $b = -1$, so we can recover the probabilities as:

$$p_0 = P(+1) = \frac{\langle Z \rangle - (-1)}{(+1) - (-1)} = \frac{\langle Z \rangle + 1}{2}$$

$$p_1 = P(-1) = \frac{(+1) - \langle Z \rangle}{(+1) - (-1)} = \frac{1 - \langle Z \rangle}{2}$$

4.9 Heisenberg Representation of Quantum Circuits

Up to now, we have described quantum computation using the **Schrödinger representation**. In this approach, the **quantum state evolves over time**, while the observables remain fixed.

However, there is an alternative but completely equivalent description called the **Heisenberg representation**. In this approach, the **quantum state remains fixed**, while the observables evolve instead.

Both descriptions produce exactly the same physical predictions and measurement outcomes. They are just different mathematical viewpoints of the same underlying physics.

≡ Schrödinger Representation

In the **Schrödinger Representation**, a quantum gate U acts directly on the state:

$$|\psi_1\rangle = U |\psi_0\rangle \quad (70)$$

Where:

- $|\psi_0\rangle$ is the initial state;
- U is the unitary operator representing the quantum circuit;
- $|\psi_1\rangle$ is the final state.

The **observable does not change**. For example, if we measure in the computational basis, we use the Pauli-Z observable Z , which **remains the same before and after applying the circuit**. The expectation value after applying the circuit is:

$$\langle \psi_1 | Z | \psi_1 \rangle$$

In this case:

- The **state changes**;
- The **observable stays fixed**.

≡ Heisenberg Representation

In the **Heisenberg Representation**, the state remains unchanged $|\psi_0\rangle$. Instead, the **observable is transformed** according to:

$$Z \longrightarrow U^\dagger Z U \quad (71)$$

Where:

- $U^\dagger$ is the conjugate transpose of the gate U ;
- Z is the original observable;
- U is the gate itself.

The expectation value is then computed as:

$$\langle \psi_0 | U^\dagger Z U | \psi_0 \rangle$$

In this case:

- The **state stays fixed**;
- The **observable changes**.

❓ Why are the two representations equivalent?

Starting from the Schrödinger representation:

$$|\psi_1\rangle = U |\psi_0\rangle$$

The corresponding bra is:

$$\langle \psi_1 | = \langle \psi_0 | U^\dagger$$

Substituting into the expectation value:

$$\begin{aligned} \langle \psi_1 | Z | \psi_1 \rangle &= (\langle \psi_0 | U^\dagger) Z (U | \psi_0 \rangle) \\ &= \langle \psi_0 | U^\dagger Z U | \psi_0 \rangle \end{aligned}$$

Therefore (and this is the key point), the **expectation value computed in the Schrödinger representation is exactly equal to the expectation value computed in the Heisenberg representation** (see Equation 71):

$$\langle \psi_1 | Z | \psi_1 \rangle = \langle \psi_0 | U^\dagger Z U | \psi_0 \rangle \quad (72)$$

This proves that both representations yield exactly the same measurement result.

🔗 Physical Interpretation

The Schrödinger and Heisenberg representations are two different mathematical viewpoints of the same physical process.

In the **Schrödinger** representation:

- We imagine the quantum state moving through the circuit;
- Gates transform the state;
- Measurements are performed using fixed observables.

In the **Heisenberg** representation:

- We keep the quantum state fixed;
- Gates transform the observables;
- Measurements are performed using transformed observables.

✔ Why is the Heisenberg representation useful?

The Heisenberg representation is useful because it lets us answer a very important question: *what observable is this quantum circuit actually measuring?*

For example, in the Schrödinger representation, suppose we have an initial state $|\psi\rangle$, a quantum circuit H (the Hadamard gate, page 65), and we measure in the computational basis using the observable Z (page 56). *But what does this circuit actually measure?*

To find out, we can switch to the Heisenberg representation and compute the transformed observable:

$$H^\dagger Z H$$

This will give us the new observable that corresponds to the measurement performed by the circuit H . In this case, we find that:

$$H^\dagger Z H = \frac{1}{2} \left(\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) = \frac{1}{2} \begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} = X$$

This means that the circuit H actually measures the observable X (the Pauli- X gate, page 51), not Z . In other words, applying the Hadamard gate before measuring in the computational basis effectively changes our measurement from Z to X .

So, without the Heisenberg representation, we only know the sequence of gates. With the Heisenberg representation, we immediately understand: “*this circuit is actually measuring X !*” This is the real physical meaning of the circuit, and it is crucial for understanding how quantum algorithms work.

Example 16: Analogy with Arithmetic Expressions

Suppose someone gives us:

$$(3 + 5) \times 2 - 16$$

We could compute it step by step. Or we could simplify the whole expression to: 0. The Heisenberg representation does something similar:

- Instead of following every state transformation,
- It simplifies the entire circuit into an equivalent measurement.

Suppose a circuit contains many gates. In the **Schrödinger picture**, we must track the state after each gate. In the **Heisenberg picture**, we ask only: *what observable is measured at the end?* Sometimes the answer is a single operator such as X or Z . Sometimes it is a more complicated operator that involves multiple qubits. But in either case, we get a clear understanding of what the circuit does without having to track the state at every step.

4.10 Clifford Gates

4.10.1 Definition

In the previous section, we introduced the Heisenberg representation of quantum circuits. There, we saw that applying a Hadamard gate H and then measuring an observable Z is equivalent to first transforming the observable Z into X and then measuring X :

$$Z \xrightarrow{H} X$$

This naturally leads to an important question: *what kinds of gates transform simple observables into other simple observables?* A particularly important class of gates with this property is known as the **Clifford gates**.

Definition 9: Clifford Gate

A unitary gate U is called a **Clifford Gate** if, for every Pauli operator P (i.e., $P \in \{I, X, Y, Z\}$), the transformed operator

$$U^\dagger P U = P' \quad (73)$$

Is still a Pauli operator (possibly with an overall sign ± 1).

In other words, **Clifford gates map Pauli operators to other Pauli operators under conjugation.**

For a single qubit, the Pauli operators are:

$$I^4, \quad X, \quad Y, \quad Z.$$

Therefore, a single-qubit Clifford gate must transform each of these operators into another operator from the same set (up to sign).

For example:

$$Z \longrightarrow X$$

Is allowed, because X is still a Pauli operator. However:

$$Z \longrightarrow \frac{X + Y}{\sqrt{2}}$$

Is not allowed, because the **result is not a single Pauli operator**.

Multi-Qubit Extension

For n qubits, Pauli operators are formed by taking tensor products of single-qubit Pauli matrices. For two qubits, examples of Pauli operators include:

$$X \otimes I, \quad I \otimes Z, \quad Y \otimes X, \quad Z \otimes Z$$

⁴Technically, Wolfgang Pauli only defined X , Y , and Z as the Pauli operators because they are the non-trivial ones. However, we include the identity I in the set of Pauli operators for completeness.

For three qubits, examples include:

$$X \otimes I \otimes Z, \quad Y \otimes Y \otimes X$$

In general, an n -qubit Pauli operator has the form:

$$P = P_1 \otimes P_2 \otimes \cdots \otimes P_n$$

Where each P_i is one of the single-qubit Pauli operators $\{I, X, Y, Z\}$.

Example 17: Hadamard Gate is a Clifford Gate

The Hadamard gate H is:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Because H is both Hermitian and unitary, we have $H^\dagger = H$ and $H^{-1} = H$. Therefore, we can compute the conjugation of the Pauli operators by H :

$$HPH = P'$$

In practice, the Hadamard gate transforms the Pauli matrices as follows:

$$\begin{aligned} HXH &= \frac{1}{2} \left(\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) \\ &= \frac{1}{2} \left(\begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) \\ &= \frac{1}{2} \left(\begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix} \right) \\ &= Z \end{aligned} \tag{74}$$

$$\begin{aligned} HZH &= \frac{1}{2} \left(\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) \\ &= \frac{1}{2} \left(\begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) \\ &= \frac{1}{2} \left(\begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix} \right) \\ &= X \end{aligned} \tag{75}$$

$$\begin{aligned} HYH &= \frac{1}{2} \left(\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) \\ &= \frac{1}{2} \left(\begin{bmatrix} i & -i \\ -i & -i \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \right) \\ &= \frac{1}{2} \left(\begin{bmatrix} 0 & 2i \\ -2i & 0 \end{bmatrix} \right) \\ &= -Y \end{aligned} \tag{76}$$

Each result is still a Pauli operator (up to a sign), confirming that the Hadamard gate is indeed a Clifford gate.

 Example 18: CNOT Gate is a Clifford Gate

The CNOT gate (page 103) is:

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

As with the Hadamard gate, we can verify that the CNOT gate is a Clifford gate by checking how it transforms the two-qubit Pauli operators. To do this, we compute the tensor products of the single-qubit Pauli operators with the identity (to create two-qubit Pauli operators) and then conjugate them by the CNOT gate (since the CNOT gate is hermitian, we have $\text{CNOT}^\dagger = \text{CNOT}$):

$$\begin{aligned} X \otimes I &= \begin{bmatrix} 0 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} & 1 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ 1 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} & 0 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \\ \text{CNOT}(X \otimes I)\text{CNOT} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \\ &= X \otimes X \end{aligned} \tag{77}$$

For the Z operator, the CNOT gate does not change the first qubit, so we have $Z \otimes I$:

$$\begin{aligned} Z \otimes I &= \begin{bmatrix} 1 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} & 0 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ 0 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} & -1 \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \end{aligned}$$

$$\begin{aligned}
\text{CNOT}(Z \otimes I)\text{CNOT} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
&= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \\
&= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \\
&= Z \otimes I
\end{aligned} \tag{78}$$

Therefore, the CNOT gate transforms $X \otimes I$ into $X \otimes X$ and leaves $Z \otimes I$ unchanged. By performing similar calculations for all other two-qubit Pauli operators, we can confirm that the CNOT gate maps every two-qubit Pauli operator to another two-qubit Pauli operator (up to a sign):

- $\text{CNOT}(I \otimes X)\text{CNOT} = I \otimes X$
- $\text{CNOT}(I \otimes Z)\text{CNOT} = Z \otimes Z$

These results confirm that the CNOT gate is indeed a Clifford gate.

4.10.2 Non-Clifford Gates

While the Hadamard and CNOT gates are examples of Clifford gates, there are many other gates that do not have this property. The most important example of a non-Clifford gate is the T gate.

The **T Gate** (also called the $\pi/8$ gate) is a **single-qubit gate that applies a specific phase shift to the state $|1\rangle$, while leaving $|0\rangle$ unchanged**. Its matrix representation is:

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1+i}{\sqrt{2}} \end{bmatrix} \quad (79)$$

This means:

$$T|0\rangle = |0\rangle \quad T|1\rangle = e^{i\pi/4}|1\rangle \quad (80)$$

On the Bloch sphere, the T gate corresponds to a rotation around the Z -axis by an angle of $\pi/4$ (or 45 degrees). In terms of the Pauli operators, the T gate transforms them as follows:

$$\begin{aligned} TXT^\dagger &= \frac{1}{\sqrt{2}}(X + Y) \\ TYT^\dagger &= \frac{1}{\sqrt{2}}(Y - X) \\ TZT^\dagger &= Z \end{aligned} \quad (81)$$

Since the T gate transforms the X and Y operators into linear combinations of X and Y (which are not single Pauli operators), the T gate is **not a Clifford gate**.

Deepening: The T Gate is the Square Root of the S Gate

The T gate is the square root of the Phase gate S (which is a Clifford gate) because applying T twice gives exactly S .

If we apply the T gate twice, we have:

$$T^2 = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/2} \end{bmatrix}$$

Where:

$$e^{i\pi/4} \cdot e^{i\pi/4} = \exp\left(\frac{i\pi}{4} + \frac{i\pi}{4}\right) = \exp\left(\frac{2i\pi}{4}\right) = \exp\left(\frac{i\pi}{2}\right) = e^{i\pi/2}$$

Now using Euler's formula, we can express $e^{i\pi/2}$ as:

$$\begin{aligned} e^{i\theta} &= \cos(\theta) + i \sin(\theta) \\ e^{i\pi/2} &= \cos\left(\frac{\pi}{2}\right) + i \sin\left(\frac{\pi}{2}\right) \\ &= 0 + i \cdot 1 \\ &= i \end{aligned}$$

Therefore, we have:

$$T^2 = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} = S$$

Finally, we can also express T as the square root of S :

$$T = \sqrt{S} = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{i} \end{bmatrix}$$

In summary, applying the T gate twice gives us the S gate, confirming that T is indeed the square root of S .

❓ Why are Non-Clifford Gates important?

The real question is: **why can some quantum circuits be simulated efficiently on a classical computer, while others cannot?** To answer this, we must compare two ways of describing a quantum state.

- The **full state vector representation**, where an n -qubit quantum state has the form (page 93):

$$|\psi\rangle = \sum_{k=0}^{2^n-1} c_k |k\rangle$$

This requires storing 2^n complex amplitudes (e.g., 1 qubit requires 2 amplitudes, 2 qubits require 4 amplitudes, etc.), which **grows exponentially** with the number of qubits.

- The **stabilizer representation**. Certain special states can be described much more compactly. Instead of storing all amplitudes, we store a small set of Pauli operators (stabilizers) that uniquely characterize the state. Only n independent stabilizers are needed for n qubits. For example, 10 qubits can be described by just 10 stabilizers, which is a huge reduction compared to the $2^{10} = 1024$ amplitudes needed in the full state vector representation. This representation **grows only linearly** with the number of qubits, making it much more efficient for certain states.

Clifford gates use the stabilizer representation, because transform Pauli operators into other Pauli operators. Therefore, we **start with n stabilizers** (n Pauli operators) that describe the initial state, and **after applying Clifford gates, we still have n stabilizers** that describe the new state. This allows to **leave the state description compact and efficient**.

❓ **What happens at a Non-Clifford Gate?** Consider the T gate. It transforms, for example, the X operator into a linear combination of X and Y :

$$TXT^\dagger = \frac{1}{\sqrt{2}}(X + Y)$$

The result is not a single Pauli operator, but a sum of two Pauli operators. After another Non-Clifford transformation, each term may split again. So **repeated Non-Clifford gates can cause the number of terms to double repeatedly, leading to exponential growth**.

✂ **Computational Consequence.** Classical simulation cost depends on how much information must be stored.

- Clifford circuits keep the description small.
- Non-Clifford circuits cause the description to grow rapidly.

Therefore, if a circuit contains only Clifford gates, a classical computer can efficiently simulate it. In contrast, if the circuit includes Non-Clifford gates such as T , simulation generally becomes exponentially more difficult.

☆ **So, why are Non-Clifford gates important?** If Clifford circuits are classically simulable, then they cannot provide full quantum computational power. To achieve universal quantum computation, we need gates that escape the stabilizer framework. The T gate does exactly this. By adding the T gate (or any other suitable non-Clifford gate) to the Clifford gate set, we obtain a universal set of quantum gates. This allows us to implement arbitrary quantum algorithms, including computations believed to be infeasible for classical computers.

4.10.3 Stabilizer States

As anticipated in the previous section, a **Stabilizer State** is a **quantum state that can be described** not by listing all of its amplitudes, but by **specifying which Pauli operators leave it unchanged**. This is one of the most elegant ideas in quantum computing and is the foundation of the stabilizer formalism used in quantum error correction and efficient simulation.

Definition 10: Stabilizer State

An n -qubit state $|\psi\rangle$ is a stabilizer state if there exists a Pauli operator P (or, more generally, a set of commuting Pauli operators) such that:

$$P|\psi\rangle = |\psi\rangle \quad (82)$$

This means:

- Applying P does not change the state $|\psi\rangle$.
- The state $|\psi\rangle$ is an eigenvector of P .
- The corresponding eigenvalue is $+1$.

The operator P is called a **Stabilizer** because it “*stabilizes*” the state $|\psi\rangle$.

 **Meaning of “*stabilize*”.** Suppose a transformation leaves an object unchanged. Some examples:

- Rotating a perfect sphere leaves it unchanged;
- Reflecting a symmetric figure may leave it unchanged;
- Applying certain Pauli operators may leave a quantum state unchanged.

In quantum mechanics, if the relation $P|\psi\rangle = |\psi\rangle$ holds, then P is a symmetry of the state $|\psi\rangle$, and we say that P stabilizes $|\psi\rangle$. This means that **the state $|\psi\rangle$ is invariant under the transformation represented by P .**

Example 19: $|0\rangle$ and $|1\rangle$

The Pauli-Z gate is:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Applying Z to $|0\rangle$:

$$Z|0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

Therefore, $|0\rangle$ is a stabilizer state and the Pauli-Z operator is one of its stabilizers.

By contrast, applying Z to $|1\rangle$:

$$Z|1\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} = -|1\rangle$$

Shows that $|1\rangle$ is not stabilized by Z (since the eigenvalue is -1), and thus $|1\rangle$ is not a stabilizer state with respect to Z .

Example 20: $|+\rangle$

The state $|+\rangle$ is defined as:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Applying the Pauli-X operator to $|+\rangle$:

$$X|+\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = |+\rangle$$

Therefore, $|+\rangle$ is a stabilizer state and the Pauli-X operator is one of its stabilizers.

Example 21: Bell State

Consider the Bell state (page 118):

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

Applying the operator $X \otimes X$ to $|\Phi^+\rangle$:

$$\begin{aligned} (X \otimes X)|\Phi^+\rangle &= \frac{1}{\sqrt{2}}(X|0\rangle \otimes X|0\rangle + X|1\rangle \otimes X|1\rangle) \\ &= \frac{1}{\sqrt{2}}(|11\rangle + |00\rangle) = |\Phi^+\rangle \end{aligned}$$

And also applying $Z \otimes Z$:

$$\begin{aligned} (Z \otimes Z)|\Phi^+\rangle &= \frac{1}{\sqrt{2}}(Z|0\rangle \otimes Z|0\rangle + Z|1\rangle \otimes Z|1\rangle) \\ &= \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = |\Phi^+\rangle \end{aligned}$$

Therefore, $|\Phi^+\rangle$ is a stabilizer state and both $X \otimes X$ and $Z \otimes Z$ are stabilizers of $|\Phi^+\rangle$.

☰ Multiple Stabilizers

The Equation 82 defines a **stabilizer state with respect to a single Pauli operator** P . However, many quantum states are stabilized by multiple Pauli operators. In fact, for an n -qubit stabilizer state, we typically use n independent commuting stabilizers:

$$P_1, P_2, \dots, P_n$$

Each of these operators satisfies:

$$P_i |\psi\rangle = |\psi\rangle \quad \text{for } i = 1, 2, \dots, n \quad (83)$$

The set of all such stabilizers forms a group under multiplication, known as the **Stabilizer Group**.

✔ Why Stabilizers are powerful

A general n -qubit state requires 2^n complex amplitudes to be fully described. However, a stabilizer state can be completely characterized by its n independent stabilizers, which are much fewer in number. Furthermore, each stabilizer is describable with $O(n)$ symbols (since it is a tensor product of n Pauli operators), leading to an efficient representation of the state. Therefore, the description size grows only polynomially (i.e., $O(n)$) rather than exponentially.

For example, for 100 qubits, a full state-vector simulation requires 2^{100} amplitudes; in contrast a stabilizer formalism requires roughly 100 stabilizers. This is why large Clifford circuits can be simulated efficiently on classical computers.

In summary, instead of describing “*what are all amplitudes of the state?*”, we describe “*which symmetries does the state satisfy?*”. This is often dramatically more efficient.

4.11 Universal set of quantum gates

After studying Clifford and non-Clifford gates, the next question is: *what is the minimum set of gates needed to build any quantum computation?* This leads to the concept of a **universal set of quantum gates**, which is a collection of gates that can be combined to approximate any unitary operation on a quantum system to arbitrary precision.

Definition 11: Universal Set of Quantum Gates

A set of quantum gates is called **universal** if any unitary transformation can be approximated to arbitrary accuracy using only gates from that set.

In other words, a set of gates is universal if using only gates from that set, we can approximate any unitary transformation to arbitrary precision.

$$\text{Universal gate set} \implies \forall U, \exists \text{ a circuit approximating } U$$

Why “Approximate” and not “Exactly”?

In quantum computing, every quantum gate is represented by a **unitary matrix**. For example the Hadamard gate, the T gate, and the CNOT gate. However, the set of all unitary matrices is **continuous** (i.e., there are infinitely many unitary transformations). For example, a single-qubit rotation may involve any real angle $R_z(\theta)$ with infinitely many possible values of θ :

$$R_z(\theta) = \begin{bmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{bmatrix}$$

Here, the angle θ can be 0.1, 0.12345, $\pi/7$ or any other real number. There are infinitely many real numbers, so there are infinitely many possible rotations and therefore infinitely many possible unitary matrices.

 **Why not build hardware for every matrix?** Building a quantum gate for every possible unitary transformation is **impossible**. Imagine trying to create one physical instruction for every possible angle:

- One gate for 0.1 radians
- One gate for 0.12345 radians
- One gate for $\pi/7$ radians
- One gate for 0.00001 radians

And so on forever. This is clearly impractical.

 **The solution: a Finite Gate Set.** Instead of trying to build a gate for every possible unitary, we choose a **small set of standard gates**, for example $\{H, \text{CNOT}, T\}$. These are the only **primitive gates the machine needs to implement directly**. All other operations are constructed by combining them.

⚠ Which gates are in the universal set?

To keep things simple, we could choose Clifford gates, such as H and CNOT, as our set of gates. Although these gates are powerful, they can still be efficiently simulated on a classical computer (page 159). Therefore, **Clifford gates alone are not sufficient for universal quantum computation**. For this reason, in general, a **universal set requires at least one non-Clifford gate**, and the most common choice is the T gate because of its simplicity and its relationship to the S gate (page 158). A **widely used universal set of quantum gates is:**

$$\{H, \text{CNOT}, T\} \quad (84)$$

? Why these three gates are enough?

- **Hadamard H transforms basis states into superpositions.** For example, the $|0\rangle$ state becomes the plus state (page 53):

$$H|0\rangle = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

- **CNOT creates entanglement between qubits.** For example, applying the CNOT to the state $|+0\rangle$ creates the Bell state $|\Phi^+\rangle$:

$$\text{CNOT}(|+0\rangle) = |\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

- **T gate adds a 45° phase and breaks the Clifford-only restriction,** allowing us to access a much larger set of unitary transformations. Because T is a non-Clifford gate, it enables us to perform operations that cannot be efficiently simulated classically.

Hadamard changes basis and creates superposition, CNOT creates entanglement between qubits, and T provides the non-Clifford component required for universality. Together, **they can approximate any unitary operation**.

Other universal gate sets exist, such as:

- Toffoli and Hadamard: $\{H, \text{Toffoli}\}$.
- Single-qubit rotations and CNOT: $\{R_x(\theta), R_y(\theta), R_z(\theta), \text{CNOT}\}$.

📖 How to Generate a Universal Set of Quantum Gates

A universal gate set is obtained by combining two ingredients:

1. **A set of gates that can generate arbitrary single-qubit unitary matrices.**
2. **One entangling two-qubit gates,** usually the CNOT gate (page 103), which can create entanglement between qubits.

If both conditions are satisfied, the gate set is universal. Let's see why this is the case:

1. **Choose Gates that Generate Arbitrary Single-Qubit Unitaries** (e.g., H and T gates). A single qubit can undergo any unitary transformation:

$$U \in U^2 = \{U \in \mathbb{C}^{2 \times 2} : U^\dagger U = I\}$$

To obtain universality, we need to approximate any such 2×2 unitary matrix. A common way is to use a Hadamard gate and a T gate. The pair of gates $\{H, T\}$ can approximate any single-qubit unitary to arbitrary precision.

❓ Why universality? While mathematically there are infinitely many possible 2×2 unitary matrices, our hardware can implement only a **finite, discrete set** of gates exactly (similar to how a digital computer uses a finite instruction set). Therefore, the goal of a *universal* gate set is not to implement every unitary matrix exactly. Instead, **for any target unitary matrix U and for any desired accuracy $\varepsilon > 0$, there must exist a finite sequence of gates from the set whose overall operation V satisfies:**⁵

$$\|U - V\| < \varepsilon$$

In other words, we can **make the approximation error as small as we want**. This ability to approximate any unitary operation to arbitrary precision is what makes a gate set universal.

2. **Add an Entangling (two-qubit) Gate.** Single-qubit gates alone cannot create entanglement (page 123). To manipulate multiple qubits, we **add an entangling gate such as CNOT**. Once CNOT is available, qubits can interact and become entangled.

Combining these two ingredients, the resulting set is universal, meaning that any quantum computation can be approximated using just these gates.

⁵The matrix $U - V$ contains the element-by-element differences, but by itself it is still a matrix, not a single number. To obtain one number representing the size of the difference, we apply a **norm**. It plays exactly the same role as absolute value for numbers. In other words, it means that the implemented operation V is within distance ε of the desired operation U .

4.12 Fault-Tolerant Quantum Gates

So far, we have studied quantum computation in an idealized setting, where quantum gates are represented by perfect unitary matrices acting on perfectly isolated qubits. In practice, however, real quantum hardware is extremely fragile. Physical qubits are affected by noise, decoherence, imperfect control operations, and measurement errors, all of which can corrupt the quantum information stored in the system.

To perform reliable quantum computation, it is therefore necessary to protect quantum information against errors. This is the goal of **quantum error correction**, where a single logical qubit is encoded into many physical qubits in such a way that errors can be detected and corrected without destroying the quantum state.

However, protecting quantum information is not sufficient by itself. We must also be able to apply quantum gates directly on encoded logical qubits while preserving the fault-tolerant structure of the error-correcting code. This introduces the notion of **fault-tolerant quantum gates**, namely gates designed so that local physical errors do not spread uncontrollably through the computation.

In this context, Clifford gates play a particularly important role because many of them admit simple fault-tolerant implementations, often through **transversal operations**. Non-Clifford gates, on the other hand, are significantly more difficult to implement fault-tolerantly, even though they are essential for universal quantum computation. This creates one of the central challenges of modern quantum computing: the gates required for computational universality are also the most expensive and difficult to protect against errors.

In the following sections, we introduce the distinction between physical and logical qubits, the concept of transversal gates, and the reasons why non-Clifford gates such as the T gate represent the main obstacle in fault-tolerant quantum computation.

4.12.1 Logical vs Physical Qubits

⚠ Problem. In an ideal mathematical model of quantum computation, qubits are assumed to evolve perfectly under unitary transformations. Real quantum hardware, however, is inherently **noisy**. Physical qubits interact with the surrounding environment and are affected by decoherence, thermal fluctuations, imperfect control pulses, and measurement errors. As a consequence, **quantum information** stored in a single physical qubit is **extremely fragile**.

✅ Solution. To overcome this limitation, quantum computers do not directly perform large computations on individual physical qubits. Instead, they use **quantum error correction**, where the information of a single qubit is encoded across many physical qubits, forming a **logical qubit**.

- **Physical Qubit.** A physical qubit is the actual hardware-level quantum system used to store information. For example superconducting qubits, trapped ions, neutral atoms or photonic qubits⁶. Physical qubits are directly manipulated by the hardware and implement the elementary quantum operations supported by the quantum processor. However, they are noisy and have limited coherence times.
- **Logical Qubit.** A logical qubit is an encoded qubit constructed from many physical qubits in order to protect quantum information against errors.

Instead of storing the quantum state in a single fragile qubit, the information is distributed redundantly across multiple physical qubits:

$$\text{Many physical qubits} \xrightarrow{\text{Encoding}} \text{One logical qubit}$$

This encoding allows the system to detect errors, correct certain errors and preserve the quantum state for a much longer time.

❓ Why do we need logical qubits? Without error correction, errors accumulate extremely rapidly during a quantum computation. Even a small error probability per gate becomes catastrophic when millions or billions of gates are executed.

Logical qubits make large-scale quantum computation possible by continuously protecting the encoded information while the computation is running.

⚠ The Main Challenge

Encoding a logical qubit is only the first step. We must also be able to apply quantum gates, manipulate information, execute algorithms. Directly on logical qubits without destroying the error-correcting structure. This leads to the concept of **fault-tolerant quantum gates**, which are quantum gates specifically designed to prevent local physical errors from spreading uncontrollably through the encoded system.

⁶It's normal don't know the details of these different physical implementations at this stage. The key point is that they are all examples of physical qubits, which are the building blocks of quantum computers.

4.12.2 Transversal Gates

In fault-tolerant quantum computing, one of the main objectives is to apply logical quantum gates without allowing physical errors to spread uncontrollably through the encoded qubits. Let's suppose one logical qubit⁷ is encoded into many physical qubits:

$$\text{Logical Qubit} \rightarrow q_1, q_2, \dots, q_7$$

It means that a single logical qubit is represented by a collection of physical qubits. Now imagine, for example, that q_3 contains an error, and all the other qubits are fine. This situation is still manageable because we can detect and correct the error in q_3 without affecting the other qubits.

⚠ The Dangerous Situation. Now suppose we apply a complicated gate that strongly couples all qubits together. Then, if q_3 has an error, it can spread to all the other qubits, making the entire logical qubit corrupted and uncorrectable.

✅ The core goal of Fault Tolerance. So the main objective becomes: if one physical qubit fails, the failure should remain localized and not spread to other qubits. This is where transversal gates come into play.

Definition 12: Transversal Gates

A **Transversal Gate** is a logical quantum gate implemented by applying independent operations to corresponding physical qubits of an encoded logical qubit.

Suppose a logical qubit is encoded into n physical qubits. A transversal implementation applies the same unitary operation independently to each physical qubit:

$$U_L = U^{\otimes n}$$

Where:

- U is the physical gate;
- U_L is the resulting logical gate;
- n is the number of physical qubits used in the encoding.

In simple terms, a transversal gate applies operations independently to each physical qubit. So no qubit interacts with another qubit inside the same block. Therefore if q_3 was wrong before, only q_3 will be affected by the gate, and the error will *not* spread to the other qubits.

⁷A logical qubit is an encoded qubit constructed from many physical qubits in order to protect quantum information against errors. Instead of storing the quantum state in a single fragile qubit, the information is distributed redundantly across multiple physical qubits.

Error Containment and Fault Tolerance

The **most important property** of transversal gates is that **they prevent error propagation inside the encoded block.**

Suppose one physical qubit contains an error before the gate is applied. Since each qubit is manipulated independently, the error remains localized and does not spread to many other qubits.

Furthermore, transversal gates are **inherently fault-tolerant**. In general, a fault-tolerant operation must ensure that:

- A small physical error remains small;
- A single faulty qubit does not corrupt the entire logical qubit.

Transversal gates naturally satisfy this requirement **because there are no interactions between qubits within the same code block during the gate operation.** Therefore, even if one qubit experiences an error, it does not affect the others.

Example 22: Logical Hadamard Gate

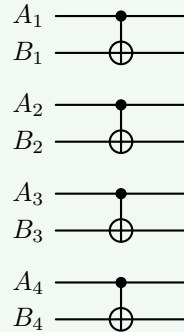
For many quantum error-correcting codes, a **logical Hadamard gate** can be implemented transversally by applying a Hadamard gate independently to every physical qubit:

$$H_L = H^{\otimes n} = \underbrace{H \otimes H \otimes \dots \otimes H}_{n \text{ times}}$$

Thus, the logical operation is realized through many independent physical operations performed in parallel.

Example 23: Logical CNOT Gate

A logical CNOT gate between two encoded logical qubits is often implemented by applying **physical CNOT gates pairwise between corresponding qubits of the two blocks**:



Each pair of physical qubits interacts independently of the others.

4.12.3 Why Non-Clifford Gates Are Difficult

One of the central challenges of fault-tolerant quantum computing is that **most non-Clifford gates cannot be implemented transversally**. While many Clifford gates admit simple fault-tolerant implementations (we saw some examples, like the Hadamard gate and CNOT, in the last section on page 170), non-Clifford gates generally require much more complex procedures.

✔ Clifford Gates and Transversality

As discussed previously, transversal gates are highly desirable because they prevent physical errors from spreading across the encoded logical qubit (page 169).

For many quantum error-correcting codes, **Clifford gates** such as Hadamard gate, Phase gate S , and CNOT gate, can **often be implemented transversally**. This means that the logical operation can be realized by applying independent physical operations to the corresponding physical qubits of the encoded blocks.

⚠ The Problem with Non-Clifford Gates

However, **non-Clifford gates**, such as the T gate, generally **do not admit transversal implementations** that preserve the structure of the error-correcting code. In fact, **if we attempt** to apply a non-Clifford gate transversally, **two problematic situations** may occur:

- ✗ The resulting operation does not correspond to the correct logical gate;
- ✗ Or the encoded structure of the code is broken, allowing errors to propagate uncontrollably through the logical qubit.

The **difficulty of implementing non-Clifford gates in a fault-tolerant manner** is not just a technological limitation of current quantum hardware; it **is also a fundamental theoretical constraint** on the structure of quantum error-correcting codes. The **Eastin-Knill theorem** formalizes this limitation by proving that **no quantum error-correcting code (i.e., no encoding scheme) can realize a universal set of logical gates using only transversal operations**.

Consequently, transversal gates are extremely useful for fault tolerance, but they are not sufficient to implement universal quantum computation. At least one non-Clifford gate must be implemented through more advanced fault-tolerant techniques.

📖 Deepening: Why non-Clifford gates do not admit transversal implementations?

Briefly, because **non-Clifford gates are “too powerful” to fit inside the simple independent structure of transversal operations**. But let us build the intuition behind this statement more carefully.

❓ **What a Transversal Gate really does.** A transversal gate acts

independently on each physical qubit: $U_L = U^{\otimes n}$, where U is a single-qubit operation. Each physical qubit is treated separately, and this is **extremely restrictive** since it **cannot create** any **entanglement** between the physical qubits (since there are no multi-qubit interactions).

? **Why this restriction is good.** Because **independent operations prevent error spreading**. If one physical qubit is wrong, the gate acts independently, then only that qubit remains wrong. So **transversal gates are naturally fault-tolerant**.

⚠ **But there is a problem.** A universal quantum computer must be able to perform extremely rich transformations. In particular, it must be able to do arbitrary rotations on the Bloch sphere, complicated entangling dynamics, and non-stabilizer transformations. And transversal operations are simply too “*rigid*” to generate all of that while preserving the code structure.

⚠ **In fact, there is a no-go theorem.** The **Eastin-Knill theorem** [1] states that **no quantum error-correcting code can have a universal set of transversal gates**. This means that while we can have some transversal Clifford gates, we cannot have a transversal implementation of a non-Clifford gate like T that would allow us to achieve universality. Therefore, we must look for more complex methods to implement non-Clifford gates in a fault-tolerant manner, such as magic state distillation or code switching, which are significantly more resource-intensive than transversal implementations.

In summary, transversal gates are simple, local and safe. Non-Clifford gates require transformations that are too complex to be realized by purely independent local operations while still preserving the error-correcting code. That is the real reason why non-Clifford gates generally cannot be implemented transversally.

✓ Solutions to the Non-Clifford Problem

To overcome the limitations imposed by the Eastin-Knill theorem, researchers have developed several techniques to implement non-Clifford gates in a fault-tolerant manner. Two standard approaches are: **magic state distillation** and **magic state injection**. The main idea is to **avoid applying the non-Clifford gate directly to the encoded logical qubit**. Instead, the **quantum computer prepares a special ancillary quantum state**, called a **magic state**, which can **later be consumed** to indirectly implement the desired logical gate.

However, **this course does not cover these techniques in detail**, as they are quite complex and require a deep understanding of quantum error correction and fault tolerance. The key takeaway is that **while non-Clifford gates are essential for universal quantum computation, their implementation in a fault-tolerant manner is significantly more challenging than that of Clifford gates, and it requires sophisticated methods that go beyond simple transversal operations**.

4.12.4 Physical vs Logical Gate Sets

In quantum computing, it is important to distinguish between:

- **Physical Gate sets**, which correspond to the elementary operations directly implemented by the hardware;
- **Logical Gate sets**, which are the fault-tolerant operations acting on encoded logical qubits (e.g., the Clifford gates).

Although the two descriptions correspond to the same underlying quantum computation, they differ significantly in terms of implementation complexity and fault-tolerant requirements.

Physical (Hardware-Level) Gate Sets

At the hardware level, quantum gates act directly on physical qubits. A typical universal physical gate set consists of:

- **Single-Qubit Gates:** These include rotations around the X, Y, and Z axes (e.g., $R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$).

In general, single-qubit rotations are **easier to implement** than multi-qubit gates because they **act locally on only one physical qubit**. In particular, many hardware platforms can implement $R_z(\theta)$ very efficiently, sometimes even virtually through software phase updates rather than physical operations.

At the physical level, also the T gate is simply a rotation around the Z-axis by $\pi/4$, therefore it is not more difficult to implement than other single-qubit gates.

- **Two-Qubit Gates:** The most common two-qubit gate is the CNOT gate, which is essential for creating entanglement between qubits.

Two-qubit gates such as CNOT are **significantly more difficult** because they require **interactions between qubits**. In general, the **larger the number of qubits involved in a gate, the more complex and error-prone its physical implementation becomes**.

These gates are **implemented through direct hardware control**, such as laser pulses in ion traps or microwave pulses in superconducting qubits.

Logical (Fault-Tolerant) Gate Sets

When quantum information is encoded into logical qubits using quantum error correction, the relevant gate set changes. The focus is no longer only on mathematical universality, but **also on fault-tolerant** implementability. A typical logical fault-tolerant gate set includes (page 164):

- **Clifford gates:**

$$\{H, S, \text{CNOT}\}$$

- **Non-Clifford gates** (e.g., the T gate):

$$\{T\}$$

 Key Distinction

The crucial observation is that the **complexity of a gate depends strongly on whether it acts on**: physical qubits or encoded logical qubits. For example:

- On physical qubits, $T = R_z(\pi/4)$ is a simple single-qubit rotation.
- On logical qubits, T is a non-Clifford logical gate not usually transversally implementable, and therefore it requires magic state distillation and fault-tolerantly complex procedures to implement.

This distinction is one of the main practical challenges of large-scale quantum computing. At the mathematical level, the T gate appears to be a simple phase rotation. However, once quantum information is encoded into logical qubits, implementing the same operation fault-tolerantly becomes extremely resource-intensive.

As a consequence, the **cost of fault-tolerant quantum algorithms is often dominated by**:

- The number of logical T gates required;
- Magic state preparation;
- Error-correction overheads associated with non-Clifford operations.

In summary, the same gate may be easy at the hardware level, but extremely difficult at the logical fault-tolerant level.

4.13 Exercises

In this section, we will provide some exercises to practice the concepts learned in the previous sections.

4.13.1 Single-Qubit States and Probabilities

Let's start with a simple exercise to understand single-qubit states and their associated probabilities.

Exercise 1. Given:

$$|\psi\rangle = \frac{1}{2} |0\rangle + \frac{\sqrt{3}}{2} |1\rangle$$

Compute:

$$P(q=0) \quad \text{and} \quad P(q=1)$$

Solution 1. First of all, we identify the complex probability amplitudes of the state $|\psi\rangle$ (page 36):

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle \Rightarrow |\psi\rangle = \frac{1}{2} |0\rangle + \frac{\sqrt{3}}{2} |1\rangle \Rightarrow \alpha = \frac{1}{2}, \quad \beta = \frac{\sqrt{3}}{2}$$

To compute the probabilities, we use the formula for the probability of measuring a particular state (page 8):

$$P(q=0) = |\alpha|^2 = \left| \frac{1}{2} \right|^2 = \frac{1}{4} = 0.25 = 25\%$$

$$P(q=1) = |\beta|^2 = \left| \frac{\sqrt{3}}{2} \right|^2 = \frac{3}{4} = 0.75 = 75\%$$

As a sanity check, we can verify that the sum of the probabilities equals 1 (the quantum state is normalized):

$$P(q=0) + P(q=1) = 0.25 + 0.75 = 1$$

Exercise 2. Given:

$$|\psi\rangle = \frac{1}{\sqrt{2}} |0\rangle - \frac{1}{\sqrt{2}} |1\rangle$$

Compute:

$$P(q=0) \quad \text{and} \quad P(q=1)$$

Solution 2. As before, we identify the complex probability amplitudes of the state $|\psi\rangle$:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle \Rightarrow |\psi\rangle = \frac{1}{\sqrt{2}} |0\rangle - \frac{1}{\sqrt{2}} |1\rangle \Rightarrow \alpha = \frac{1}{\sqrt{2}}, \quad \beta = -\frac{1}{\sqrt{2}}$$

To compute the probabilities, we use the formula for the probability of measuring a particular state:

$$P(q = 0) = |\alpha|^2 = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} = 0.5 = 50\%$$

$$P(q = 1) = |\beta|^2 = \left| -\frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} = 0.5 = 50\%$$

As a sanity check, we can verify that the sum of the probabilities equals 1:

$$P(q = 0) + P(q = 1) = 0.5 + 0.5 = 1$$

4.13.2 Bloch sphere states

In general, the single-qubit state can be expressed as (page 39):

$$|\psi\rangle = \cos\left(\frac{\theta}{2}\right) |0\rangle + e^{i\phi} \sin\left(\frac{\theta}{2}\right) |1\rangle$$

With:

$$\theta \in [0, \pi], \quad \phi \in [0, 2\pi)$$

And it identifies the main states of the Bloch sphere representation, where θ is the polar angle and ϕ is the azimuthal angle.

Exercise 3. Identify the states of the Bloch sphere representation corresponding to the following pairs of angles (θ, ϕ) :

$$\theta = 0, \quad \phi = 0$$

Solution 3. The state corresponding to $\theta = 0$ and $\phi = 0$ is calculated substituting the values into the general expression for $|\psi\rangle$:

$$\begin{aligned} |\psi\rangle &= \cos\left(\frac{0}{2}\right) |0\rangle + e^{i \cdot 0} \sin\left(\frac{0}{2}\right) |1\rangle \\ &= \cos(0) |0\rangle + e^{0i} \sin(0) |1\rangle \\ &= 1 |0\rangle + 1 \cdot 0 |1\rangle \\ &= |0\rangle \end{aligned}$$

So, the state corresponding to $\theta = 0$ and $\phi = 0$ is $|0\rangle$, which is the north pole of the Bloch sphere.

Exercise 4. Identify the states of the Bloch sphere representation corresponding to the following pairs of angles (θ, ϕ) :

$$\theta = \pi, \quad \phi = 0$$

Solution 4. The state corresponding to $\theta = \pi$ and $\phi = 0$ is calculated substituting the values into the general expression for $|\psi\rangle$:

$$\begin{aligned} |\psi\rangle &= \cos\left(\frac{\pi}{2}\right) |0\rangle + e^{i \cdot 0} \sin\left(\frac{\pi}{2}\right) |1\rangle \\ &= \cos\left(\frac{\pi}{2}\right) |0\rangle + e^{0i} \sin\left(\frac{\pi}{2}\right) |1\rangle \\ &= 0 |0\rangle + 1 |1\rangle \\ &= |1\rangle \end{aligned}$$

So, the state corresponding to $\theta = \pi$ and $\phi = 0$ is $|1\rangle$, which is the south pole of the Bloch sphere.

Exercise 5. Identify the states of the Bloch sphere representation corresponding to the following pairs of angles (θ, ϕ) :

$$\theta = \frac{\pi}{2}, \quad \phi = 0$$

Solution 5. The state corresponding to $\theta = \frac{\pi}{2}$ and $\phi = 0$ is calculated substituting the values into the general expression for $|\psi\rangle$:

$$\begin{aligned}
 |\psi\rangle &= \cos\left(\frac{\pi}{4}\right) |0\rangle + e^{i \cdot 0} \sin\left(\frac{\pi}{4}\right) |1\rangle \\
 &= \frac{1}{\sqrt{2}} |0\rangle + e^{0i} \frac{1}{\sqrt{2}} |1\rangle \\
 &= \frac{1}{\sqrt{2}} |0\rangle + \frac{1}{\sqrt{2}} |1\rangle \\
 &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \\
 &= |+\rangle
 \end{aligned}$$

So, the state corresponding to $\theta = \frac{\pi}{2}$ and $\phi = 0$ is $|+\rangle$, which is located on the equator of the Bloch sphere along the positive x -axis.

Exercise 6. Identify the states of the Bloch sphere representation corresponding to the following pairs of angles (θ, ϕ) :

$$\theta = \frac{\pi}{2}, \quad \phi = \pi$$

Solution 6. The state corresponding to $\theta = \frac{\pi}{2}$ and $\phi = \pi$ is calculated substituting the values into the general expression for $|\psi\rangle$:

$$\begin{aligned}
 |\psi\rangle &= \cos\left(\frac{\pi}{4}\right) |0\rangle + e^{i \cdot \pi} \sin\left(\frac{\pi}{4}\right) |1\rangle \\
 &= \frac{1}{\sqrt{2}} |0\rangle + e^{i\pi} \frac{1}{\sqrt{2}} |1\rangle \\
 &= \frac{1}{\sqrt{2}} |0\rangle - \frac{1}{\sqrt{2}} |1\rangle \\
 &= \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\
 &= |-\rangle
 \end{aligned}$$

The exponential term $e^{i\pi}$ evaluates to -1 due to Euler's formula. Recall the Euler's formula for any angle θ :

$$e^{i\theta} = \cos(\theta) + i \sin(\theta) \tag{85}$$

If we substitute $\theta = \pi$ into Equation 85, we get:

$$e^{i\pi} = \cos(\pi) + i \sin(\pi) = -1 + 0i = -1$$

So, the state corresponding to $\theta = \frac{\pi}{2}$ and $\phi = \pi$ is $|-\rangle$, which is located on the equator of the Bloch sphere along the negative x -axis.

4.13.3 Tensor products and multi-qubit probabilities

Exercise 7. (*Expand a tensor product symbolically*) Given two states:

$$|\psi_1\rangle = \alpha_1 |0\rangle + \beta_1 |1\rangle, \quad |\psi_2\rangle = \alpha_2 |0\rangle + \beta_2 |1\rangle$$

Compute:

$$|\Psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle$$

Solution 7. The tensor product (page 88) is a mathematical operation that takes two vectors and produces a new vector that represents the combined state of the two systems:

$$\begin{aligned} |\Psi\rangle &= |\psi_1\rangle \otimes |\psi_2\rangle \\ &= (\alpha_1 |0\rangle + \beta_1 |1\rangle) \otimes (\alpha_2 |0\rangle + \beta_2 |1\rangle) \\ &= \alpha_1 \alpha_2 |00\rangle + \alpha_1 \beta_2 |01\rangle + \beta_1 \alpha_2 |10\rangle + \beta_1 \beta_2 |11\rangle \end{aligned}$$

Exercise 8. (*Expand a Concrete Two-Qubit State*) Now let's move from the symbolic formula to an actual computation. Given the two qubits:

$$|\psi_1\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \quad \text{and} \quad |\psi_2\rangle = |1\rangle$$

Compute the combined state:

$$|\Psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle$$

Solution 8. As in the previous exercise, we will first substitute:

$$\begin{aligned} |\Psi\rangle &= |\psi_1\rangle \otimes |\psi_2\rangle \\ &= \left(\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right) \otimes |1\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle \otimes |1\rangle + |1\rangle \otimes |1\rangle) \\ &= \frac{1}{\sqrt{2}} (|01\rangle + |11\rangle) \end{aligned}$$

Exercise 9. (*Full-state probabilities*) Given the state from the previous exercise:

$$|\Psi\rangle = \frac{1}{\sqrt{2}} (|01\rangle + |11\rangle)$$

Compute the probabilities:

$$P(q_1 q_2 = 00), \quad P(q_1 q_2 = 01), \quad P(q_1 q_2 = 10), \quad P(q_1 q_2 = 11)$$

Solution 9. The probability of measuring a specific basis state is equal to the squared modulus of its amplitude in the quantum state. Since:

$$|\Psi\rangle = \frac{1}{\sqrt{2}} (|01\rangle + |11\rangle)$$

The amplitudes of the four computational basis states are:

State	Amplitude
$ 00\rangle$	0
$ 01\rangle$	$\frac{1}{\sqrt{2}}$
$ 10\rangle$	0
$ 11\rangle$	$\frac{1}{\sqrt{2}}$

Therefore:

$$\begin{aligned}
 P(q_1 q_2 = 00) &= |0|^2 = 0 \\
 P(q_1 q_2 = 01) &= \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} = 50\% \\
 P(q_1 q_2 = 10) &= |0|^2 = 0 \\
 P(q_1 q_2 = 11) &= \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} = 50\%
 \end{aligned}$$

As expected, the probabilities sum to one:

$$P(00) + P(01) + P(10) + P(11) = 0 + \frac{1}{2} + 0 + \frac{1}{2} = 1$$

Exercise 10. (Compute the probability of measuring individual qubits) We continue from:

$$|\Psi\rangle = \frac{1}{\sqrt{2}} (|01\rangle + |11\rangle)$$

From the previous exercise, we know the full-state probabilities:

$$P(00) = 0, \quad P(01) = \frac{1}{2}, \quad P(10) = 0, \quad P(11) = \frac{1}{2}$$

Now, compute the marginal probabilities for each qubit:

$$P(q_1 = 0), \quad P(q_1 = 1), \quad P(q_2 = 0), \quad P(q_2 = 1)$$

Solution 10. The **Marginal Probability** of a qubit being in a certain state is the **sum of the probabilities of all the full states that include that qubit state**. In other words, it is the **probability of measuring a single qubit**, regardless of the value of the other qubit. Given a two-qubit system, the marginal probabilities are computed as follows:

$$\begin{aligned}
 P(q_1 = 0) &= \sum P(0-) = P(00) + P(01) = 0 + \frac{1}{2} = \frac{1}{2} \\
 P(q_1 = 1) &= \sum P(1-) = P(10) + P(11) = 0 + \frac{1}{2} = \frac{1}{2} \\
 P(q_2 = 0) &= \sum P(-0) = P(00) + P(10) = 0 + 0 = 0 \\
 P(q_2 = 1) &= \sum P(-1) = P(01) + P(11) = \frac{1}{2} + \frac{1}{2} = 1
 \end{aligned}$$

Where $P(0-)$ means the sum of probabilities of all states where the first qubit is 0, and the second qubit can be anything (denoted by the dash, *don't care*). Similarly, $P(-0)$ means the sum of probabilities of all states where the second qubit is 0, and the first qubit can be anything. And so on for the other cases.

4.13.4 Common factors, global phase and relative phase

Before beginning this section, it is important to review the difference between global and relative phase and the meaning of common factors.

Let's consider the following quantum state:

$$|\psi\rangle = \sum_j \alpha_j |j\rangle$$

Where α_j are complex numbers and $|j\rangle$ are the basis states. A **scalar** $c \neq 0$ **multiplying every amplitude** is a **Common Factor**:

$$|\phi\rangle = c|\psi\rangle = \sum_j c\alpha_j |j\rangle \quad (86)$$

Where $|\phi\rangle$ is the new quantum state. However, we must distinguish three different situations where the common factor c can be applied to the quantum state $|\psi\rangle$:

- **Normalization Factor.** Consider the famous quantum state:

$$|+\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

The factor $\frac{1}{\sqrt{2}}$ is necessary because it ensures that the state is normalized:

$$|+\rangle = \underbrace{\frac{1}{\sqrt{2}}}_{\alpha} |0\rangle + \underbrace{\frac{1}{\sqrt{2}}}_{\beta} |1\rangle, \quad \text{where } |\alpha|^2 + |\beta|^2 = 1$$

If we remove it, we would have:

$$|0\rangle + |1\rangle, \quad \text{where } |1|^2 + |1|^2 = 2$$

The state is not normalized anymore, and it cannot be used in quantum computing. Therefore, a **normalization factor cannot simply be discarded**.

- **Global Phase.** A global phase is a common factor of the form $e^{i\gamma}$, whose modulus is always equal to 1:

$$|e^{i\gamma}| = 1$$

Since the global phase does **not change the probabilities of measuring the quantum state**, it can be **discarded**.

Example 24: Global Phase

For example, consider the following quantum state:

$$|\phi\rangle = \frac{i}{\sqrt{2}} (|0\rangle + |1\rangle)$$

Using the Euler's formula, we can rewrite the factor i as:

$$e^{i\theta} = \underbrace{\cos(\theta)}_{\text{must be 0}} + i \underbrace{\sin(\theta)}_{\text{must be 1}} \Rightarrow e^{i\frac{\pi}{2}} = i$$

Therefore, we can rewrite the quantum state as:

$$|\phi\rangle = e^{i\frac{\pi}{2}} \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) = e^{i\frac{\pi}{2}} |+\rangle$$

The two vectors therefore represent the same physical state:

$$|\phi\rangle \sim |+\rangle$$

Where $\sim$ means “*physically equivalent*”. In fact, the global phase can be discarded, and we can write:

$$|\phi\rangle \sim |+\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

Because the measurement probabilities confirm that:

$$P_\phi(0) = P_+(0) = \left| \frac{i}{\sqrt{2}} \right|^2 = \frac{1}{2} \quad \text{and} \quad P_\phi(1) = P_+(1) = \left| \frac{i}{\sqrt{2}} \right|^2 = \frac{1}{2}$$

Thus, the quantum state $|\psi\rangle$ and $e^{i\gamma}|\psi\rangle$ represent the same physical state, and the global phase can be discarded.

- **Relative Phase.** Consider the following quantum states:

$$|+\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \quad \text{and} \quad |-\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

The minus sign in the second state multiplies only the amplitude of the $|1\rangle$ basis state. It is therefore a **Relative Phase**:

$$-1 = e^{i\pi}$$

We **cannot remove** it as a common factor **because it is not applied to the entire state**. Although both states have the same computational-basis probabilities:

$$P_+(0) = P_-(0) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} \quad \text{and} \quad P_+(1) = P_-(1) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2}$$

The two states are **not physically equivalent**:

$$|+\rangle \not\sim |-\rangle$$

Because the relative phase affects the interference of the two states, for example, when we apply the Hadamard gate to both states:

$$H|+\rangle = |0\rangle \quad \text{but} \quad H|-\rangle = |1\rangle$$

Thus, relative phases are **physically relevant**.

In general, suppose we have two quantum states:

$$|\psi\rangle = \sum_j \alpha_j |j\rangle, \quad |\phi\rangle = \sum_j \beta_j |j\rangle$$

If both states are normalized, they represent the same physical state exactly when there is a single number c such that:

$$\beta_j = c\alpha_j \quad \forall j, |c| = 1$$

In other words, all corresponding nonzero amplitudes must have the same ratio:

$$\frac{\beta_0}{\alpha_0} = \frac{\beta_1}{\alpha_1} = \dots = e^{i\gamma}$$

If the ratio is not the same for all amplitudes, then the two states are **not physically equivalent** because they have different relative phases.

Exercise 11. Determine whether the following states represent the same physical state:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |\phi\rangle = \frac{1+i}{2}(|0\rangle + |1\rangle)$$

Solution 11. We need to determine whether there is a single number c such that:

$$\frac{1+i}{2} = c \cdot \frac{1}{\sqrt{2}}$$

Or in other words, we need to determine whether $\frac{1+i}{2}$ differs from $\frac{1}{\sqrt{2}}$ by a global phase.

1. First of all, check that **both states are normalized**:

$$\left| \frac{1}{\sqrt{2}} \right|^2 + \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} + \frac{1}{2} = 1,$$

$$\begin{aligned} \left| \frac{1+i}{2} \right|^2 &= \frac{|1+i|^2}{|2|^2} \\ &= \frac{|1+i|^2}{4} \end{aligned}$$

↓ Remark: $|z| = \sqrt{a^2 + b^2}$ and $|z|^2 = a^2 + b^2$ for $z = a + ib$

↓ For $1+i$, we have $a=1, b=1$

$$\begin{aligned} &= \frac{1^2 + 1^2}{4} \\ &= \frac{2}{4} \end{aligned}$$

$$= \frac{1}{2} \Rightarrow \left| \frac{1+i}{2} \right|^2 + \left| \frac{1+i}{2} \right|^2 = \frac{1}{2} + \frac{1}{2} = 1$$

2. Now, we can **find the factor connecting the two states**. In general, we want to know if one state can be obtained by multiplying the entirety of the other state by one number c :

$$\phi = c\psi$$

Let's substitute the two states:

$$\frac{1+i}{2}(|0\rangle + |1\rangle) = c \cdot \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

We can divide both sides by $(|0\rangle + |1\rangle)$ to remove the basis states:

$$\frac{1+i}{2} = c \cdot \frac{1}{\sqrt{2}}$$

3. Now, we can solve for c :

$$\begin{aligned}\frac{1+i}{2} &= c \cdot \frac{1}{\sqrt{2}} \\ \sqrt{2} \cdot \frac{1+i}{2} &= c \\ c &= \sqrt{2} \cdot \frac{1+i}{2}\end{aligned}$$

And **verify** that multiplying ψ by c gives ϕ (we need to check that c is a global phase):

$$\begin{aligned}\sqrt{2} \cdot \frac{1+i}{2} |\psi\rangle &= \sqrt{2} \cdot \frac{1+i}{2} \cdot \left[\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \right] \\ &= \sqrt{2} \cdot \frac{1+i}{2} \cdot \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \\ &= \cancel{\sqrt{2}} \cdot \frac{1+i}{2} \cdot \frac{1}{\cancel{\sqrt{2}}}(|0\rangle + |1\rangle) \\ &= \frac{1+i}{2}(|0\rangle + |1\rangle) \\ &= |\phi\rangle\end{aligned}$$

So we have found one common factor multiplying the entire state, but this does not mean that the two states are physically equivalent! We need to check that the factor is a global phase, i.e. that its modulus is equal to 1:

$$\begin{aligned}|c| &= \left| \sqrt{2} \cdot \frac{1+i}{2} \right| \\ &= \left| \sqrt{2} \right| \cdot \left| \frac{1+i}{2} \right| \\ &= \sqrt{2} \cdot \frac{|1+i|}{|2|} \\ &= \sqrt{2} \cdot \frac{|1+i|}{2} \\ \downarrow \text{ Remark: } |z| &= \sqrt{a^2 + b^2} \text{ and } |z|^2 = a^2 + b^2 \text{ for } z = a + ib \\ \downarrow \text{ For } 1+i, \text{ we have } a=1, b=1 \\ &= \sqrt{2} \cdot \frac{\sqrt{1^2 + 1^2}}{2} \\ &= \sqrt{2} \cdot \frac{\sqrt{2}}{2} \\ &= \frac{2}{2} = 1\end{aligned}$$

Thus, c **changes only the phase**; it does **not change the magnitude of the amplitudes**. Therefore, c is a global phase:

$$|\phi\rangle = c|\psi\rangle, \quad \text{where } |c| = 1$$

And the states are physically equivalent: $|\phi\rangle \sim |\psi\rangle$.

Exercise 12. Determine whether these states represent the same physical state:

$$|\psi\rangle = \frac{1}{2}(i|00\rangle + |01\rangle - i|10\rangle + |11\rangle), \quad |\phi\rangle = \frac{1}{2}(|00\rangle - i|01\rangle - |10\rangle - i|11\rangle)$$

Solution 12. We need to determine whether there is a single number c such that:

$$|\psi\rangle = c|\phi\rangle$$

$$\frac{1}{2}(i|00\rangle + |01\rangle - i|10\rangle + |11\rangle) = c \cdot \frac{1}{2}(|00\rangle - i|01\rangle - |10\rangle - i|11\rangle)$$

Let's try to find the factor c by comparing the amplitudes of the basis states:

1. First of all, check that **both states are normalized**. We start with $|\psi\rangle$:

$$\underbrace{\left|\frac{1}{2} \cdot i\right|^2}_{|00\rangle} + \underbrace{\left|\frac{1}{2}\right|^2}_{|01\rangle} + \underbrace{\left|\frac{1}{2} \cdot (-i)\right|^2}_{|10\rangle} + \underbrace{\left|\frac{1}{2}\right|^2}_{|11\rangle} = 1$$

$$\left(\sqrt{0^2 + \left(\frac{1}{2}\right)^2}\right)^2 + \frac{1}{4} + \left(\sqrt{0^2 + \left(-\frac{1}{2}\right)^2}\right)^2 + \frac{1}{4} = 1$$

$$\frac{1}{4} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4} = 1$$

$$1 = 1$$

And now for $|\phi\rangle$:

$$\underbrace{\left|\frac{1}{2}\right|^2}_{|00\rangle} + \underbrace{\left|\frac{1}{2} \cdot (-i)\right|^2}_{|01\rangle} + \underbrace{\left|\frac{1}{2} \cdot (-1)\right|^2}_{|10\rangle} + \underbrace{\left|\frac{1}{2} \cdot (-i)\right|^2}_{|11\rangle} = 1$$

$$\frac{1}{4} + \left(\sqrt{0^2 + \left(-\frac{1}{2}\right)^2}\right)^2 + \frac{1}{4} + \left(\sqrt{0^2 + \left(-\frac{1}{2}\right)^2}\right)^2 = 1$$

$$\frac{1}{4} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4} = 1$$

$$1 = 1$$

2. Now, we can **find the factor connecting the two states**, but we need to check that the factor is the same for all amplitudes.

- For the $|00\rangle$ basis state, we have:

$$\frac{1}{2} \cdot i = c \cdot \frac{1}{2} \cdot 1 \quad \Rightarrow \quad c = i$$

- For the $|01\rangle$ basis state, we have:

$$\frac{1}{2} = c \cdot \frac{1}{2} \cdot (-i) \Rightarrow 1 = c \cdot (-i) \Rightarrow c = -\frac{1}{i}$$

- For the $|10\rangle$ basis state, we have:

$$-\frac{1}{2} \cdot i = c \cdot \frac{1}{2} \cdot (-1) \Rightarrow c = i$$

- For the $|11\rangle$ basis state, we have:

$$\frac{1}{2} = c \cdot \frac{1}{2} \cdot (-i) \Rightarrow c = -\frac{1}{i}$$

We can simplify each $c = -\frac{1}{i}$:

$$-\frac{1}{i} = -\frac{1}{i} \cdot \underbrace{\frac{i}{i}}_1 = -\frac{i}{i^2} = -\frac{i}{-1} = i$$

Thus, we have found that the factor c is the same for all amplitudes:

$$c = i$$

3. The previous step shows that the two states are related by a common factor $c = i$. However, we need to check that c is a global phase, i.e. that its modulus is equal to 1:

$$|c| = |i| = \sqrt{0^2 + 1^2} = \sqrt{1} = 1$$

A global phase means that there exists **one single complex number** c , with modulus equal to 1, such that:

$$|\psi\rangle = c|\phi\rangle$$

4.13.5 Unitaries, Pauli Gates, and Eigenstates of Z, X, Y, H

These definitions must become automatic before starting the exercises:

- Z gate (phase flip):

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$Z|0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \cdot 1 + 0 \cdot 0 \\ 0 \cdot 1 + (-1) \cdot 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

$$Z|1\rangle = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 0 + 0 \cdot 1 \\ 0 \cdot 0 + (-1) \cdot 1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} = -|1\rangle$$

- X gate (bit flip):

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$X|0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \cdot 1 + 1 \cdot 0 \\ 1 \cdot 1 + 0 \cdot 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} = |1\rangle$$

$$X|1\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \cdot 0 + 1 \cdot 1 \\ 1 \cdot 0 + 0 \cdot 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

- Y gate (bit and phase flip):

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

$$Y|0\rangle = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \cdot 1 + (-i) \cdot 0 \\ i \cdot 1 + 0 \cdot 0 \end{bmatrix} = \begin{bmatrix} 0 \\ i \end{bmatrix} = i|1\rangle$$

$$Y|1\rangle = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \cdot 0 + (-i) \cdot 1 \\ i \cdot 0 + 0 \cdot 1 \end{bmatrix} = \begin{bmatrix} -i \\ 0 \end{bmatrix} = -i|0\rangle$$

- H gate (Hadamard):

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

Also, a general rule for the action of a unitary gate U on a qubit state $|\psi\rangle$:

$$U(\alpha|0\rangle + \beta|1\rangle)$$

Follow these four steps:

1. Expand using linearity:

$$U(\alpha|0\rangle + \beta|1\rangle) = \alpha U|0\rangle + \beta U|1\rangle$$

2. Apply the gate separately to each basis state.
3. Replace $U|0\rangle$ and $U|1\rangle$ with their known values (from the definitions of the gates).
4. Simplify.

If a common factor such as i appears everywhere, remember what we have learned in the previous section (page 181): it may simply be a **global phase** and can be ignored when calculating probabilities.

Another central equation is:

$$U|\psi\rangle = \lambda|\psi\rangle$$

A state $|\psi\rangle$ is an **eigenstate** of U when applying U does not change its direction: the result is only multiplied by a scalar λ , called the **eigenvalue**. For unitary gates, the eigenvalue has modulus one: $|\lambda| = 1$.

Exercise 13. Which is the eigenstate of the Z gate? What is its eigenvalue?

Solution 13. Using the definition of the Z gate, we have:

$$Z|0\rangle = |0\rangle$$

This means that $|0\rangle$ is an eigenstate of Z with eigenvalue $\lambda = 1$. We also have:

$$Z|1\rangle = -|1\rangle$$

This means that $|1\rangle$ is also an eigenstate of Z , but with eigenvalue $\lambda = -1$. Thus, the Z gate has two eigenstates: $|0\rangle$ and $|1\rangle$, with eigenvalues 1 and -1 , respectively.

Exercise 14. Which is the eigenstate of the X gate? What is its eigenvalue?

Solution 14. Using the definition of the X gate, we have:

$$X|0\rangle = |1\rangle$$

This means that $|0\rangle$ is **not** an eigenstate of X , because applying X to $|0\rangle$ gives $|1\rangle$, which is a different state. Similarly:

$$X|1\rangle = |0\rangle$$

This means that $|1\rangle$ is also **not** an eigenstate of X . We can use $|+\rangle$ and $|-\rangle$, which are defined as:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Let's check if these are eigenstates of the X gate:

$$\begin{aligned} X|+\rangle &= X\left(\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)\right) \\ &= \frac{1}{\sqrt{2}}(X|0\rangle + X|1\rangle) \\ &= \frac{1}{\sqrt{2}}(|1\rangle + |0\rangle) \\ &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \\ &= |+\rangle \end{aligned}$$

Thus, $|+\rangle$ is an eigenstate of the X gate with eigenvalue $\lambda = 1$. Let's check $|-\rangle$:

$$\begin{aligned} X|-\rangle &= X\left(\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right) \\ &= \frac{1}{\sqrt{2}}(X|0\rangle - X|1\rangle) \\ &= \frac{1}{\sqrt{2}}(|1\rangle - |0\rangle) \\ &= -\frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \\ &= -|-\rangle \end{aligned}$$

Thus, $|-\rangle$ is also an eigenstate of the X gate, but with eigenvalue $\lambda = -1$. Therefore, the X gate has two eigenstates: $|+\rangle$ and $|-\rangle$, with eigenvalues 1 and -1 , respectively.

Exercise 15. Which is the eigenstate of the Y gate? What is its eigenvalue?

Solution 15. Similarly, we can check the eigenstates of the Y gate. Let's define the states:

$$|+i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle), \quad |-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle)$$

Let's check if these are eigenstates of the Y gate:

$$\begin{aligned} Y|+i\rangle &= Y\left(\frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)\right) \\ &= \frac{1}{\sqrt{2}}(Y|0\rangle + iY|1\rangle) \\ &= \frac{1}{\sqrt{2}}(i|1\rangle + i(-i|0\rangle)) \\ &= \frac{1}{\sqrt{2}}(i|1\rangle + |0\rangle) \\ &= \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) \\ &= |+i\rangle \end{aligned}$$

Thus, $|+i\rangle$ is an eigenstate of the Y gate with eigenvalue $\lambda = 1$. Let's check $|-i\rangle$:

$$\begin{aligned} Y|-i\rangle &= Y\left(\frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle)\right) \\ &= \frac{1}{\sqrt{2}}(Y|0\rangle - iY|1\rangle) \\ &= \frac{1}{\sqrt{2}}(i|1\rangle - i(-i|0\rangle)) \\ &= \frac{1}{\sqrt{2}}(i|1\rangle - |0\rangle) \\ &= -\frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle) \\ &= -|-i\rangle \end{aligned}$$

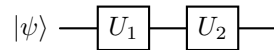
Thus, $|-i\rangle$ is also an eigenstate of the Y gate, but with eigenvalue $\lambda = -1$. Therefore, the Y gate has two eigenstates: $|+i\rangle$ and $|-i\rangle$, with eigenvalues 1 and -1 , respectively.

4.13.6 Circuit notation and gate composition

In this section, we will not introduce any complex exercises. Instead, we will explore how to approach circuit notation and compose gates in a circuit.

→ Consecutive Gates

Consider this circuit:



Physically, this circuit applies the gate U_1 to the state $|\psi\rangle$, and then applies the gate U_2 to the resulting state. But mathematically we need **one equivalent matrix** to represent the entire circuit. This is done by multiplying the matrices of the gates in the order they are applied:

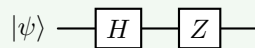
$$U_3 = U_2 U_1$$

Notice the order of multiplication: the **rightmost matrix acts first**.

Why? This rule comes from linear algebra. If $|\psi\rangle$ is a column vector, first we apply U_1 obtaining $U_1 |\psi\rangle$. Then we apply U_2 giving $U_2 (U_1 |\psi\rangle)$. Associativity of matrix multiplication allows us to write this as $(U_2 U_1) |\psi\rangle$, which is equivalent to applying the single gate $U_3 = U_2 U_1$ to the state $|\psi\rangle$.

Example 25: Circuit notation and gate composition

Consider the following circuit:



Physically:

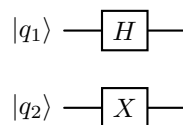
1. Hadamard
2. Pauli-Z

Equivalent matrix:

$$ZH$$

= Parallel Gates

Now let's complicate things a bit. Consider this circuit with two qubits:



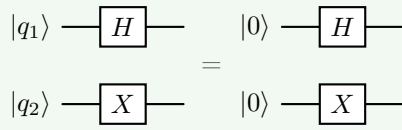
The gates happen **at the same time**. There is no “first” or “second” gate. Instead, we combine them using the **tensor product** (page 88). Thus, the equivalent matrix is:

$$H \otimes X$$

This is the operator acting on the two-qubit system.

 Example 26

Consider the following state: $|\psi\rangle = |00\rangle$. Apply two gates in parallel: Hadamard on the first qubit and Pauli-X on the second qubit. The circuit is:



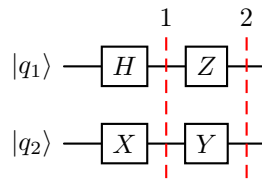
The equivalent matrix is $H \otimes X$. Applying this to the state $|00\rangle$ gives:

$$\begin{aligned} (H \otimes X)|00\rangle &= H|0\rangle \otimes X|0\rangle \\ &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |1\rangle \\ &= \frac{1}{\sqrt{2}}(|01\rangle + |11\rangle) \end{aligned}$$

In general, if we have a circuit with n qubits and we apply n gates in parallel, the equivalent matrix is the tensor product of all the gates:

$$U = U_1 \otimes U_2 \otimes \cdots \otimes U_n$$

Now suppose we have a circuit with **two layers**:



The first layer becomes $H \otimes X$, and the second layer becomes $Z \otimes Y$. The equivalent matrix for the entire circuit is:

$$(Z \otimes Y) \cdot (H \otimes X)$$

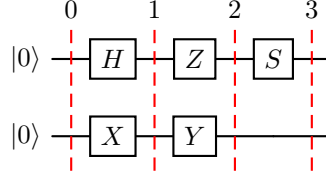
Again, notice that we still multiply **from right to left**, because the first layer acts first, and the second layer acts second.

So there are **two different operations**:

- **Horizontally (along time)**: matrix multiplication.
- **Vertically (same time, different qubits)**: tensor product.

❓ **What if, vertically, one qubit has a gate and the other has a straight line?**

In this case, the scenario is really simple. Let's say we have a circuit with two qubits $|\psi\rangle = |00\rangle$, composed of:



We label each layer with a temporary state $|\psi_i\rangle$, where i is the layer number:

- **Initial state** $|\psi_0\rangle$. Both qubit starts in $|0\rangle$, therefore:

$$|\psi_0\rangle = |0\rangle \otimes |0\rangle = |00\rangle$$

We apply the tensor product because the two qubits are vertically aligned.

- **First layer** $|\psi_1\rangle$. Since the gates are simultaneous, we apply the tensor product of the two gates:

$$|\psi_1\rangle = (H \otimes X) |\psi_0\rangle = (H \otimes X) |00\rangle$$

Remember what we learned on page 187, instead of multiplying two huge matrices, we can apply the two gates separately to each qubit:

$$\begin{aligned} |\psi_1\rangle &= H |0\rangle \otimes X |0\rangle \\ &= \underbrace{\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)}_{|+\rangle} \otimes |1\rangle \end{aligned}$$

💡 **Tip.** At this point **do not expand the tensor product!** Leave it factorized, because it is easier to work with.

- **Second layer** $|\psi_2\rangle$. Again, we apply the tensor product of the two gates:

$$|\psi_2\rangle = (Z \otimes Y) |\psi_1\rangle = (Z \otimes Y) \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |1\rangle \right]$$

- First qubit (Pauli-Z changes only the relative phase of the $|1\rangle$ qubit):

$$Z \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right] = \frac{1}{\sqrt{2}} (Z |0\rangle + Z |1\rangle) = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

- Second qubit (Pauli-Y changes the relative phase of the $|1\rangle$ qubit and flips it):

$$Y |1\rangle = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 + (-i) \cdot 1 \\ i \cdot 0 + 0 \end{bmatrix} = \begin{bmatrix} -i \\ 0 \end{bmatrix} = -i |0\rangle$$

Putting everything together:

$$|\psi_2\rangle = \left[\frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \right] \otimes [-i|0\rangle] = -i \cdot \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes |0\rangle$$

Q Global Phase detected. Before we apply the last gate, we can see that the state has a term $-i$ in front of the tensor product that multiplies **the whole quantum state**. It is not attached only to one basis state (like $|0\rangle - |1\rangle$ on the first qubit). Therefore it has no physical effect and can be discarded. We can rewrite the state as:

$$|\psi_2\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes |0\rangle$$

- **Third layer** $|\psi_3\rangle$. The last layer has a gate on the first qubit and nothing on the second qubit. In this case, we apply the **tensor product of the gate** and the **identity matrix**:

$$|\psi_3\rangle = (S \otimes I) |\psi_2\rangle = (S \otimes I) \left[\frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes |0\rangle \right]$$

- First qubit (phase gate S changes only the relative phase of the $|1\rangle$ qubit):

$$S \left[\frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \right] = \frac{1}{\sqrt{2}} (S|0\rangle - S|1\rangle) = \frac{1}{\sqrt{2}} (|0\rangle - i|1\rangle)$$

- Second qubit (Identity gate does not change the state):

$$I|0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

Remembering that we can apply the two gates separately to each qubit, we can write the final state as:

$$|\psi_3\rangle = \left[\frac{1}{\sqrt{2}} (|0\rangle - i|1\rangle) \right] \otimes |0\rangle$$

Now we are at the end of the circuit, so we can **expand the tensor product** to get the final state:

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|00\rangle - i|10\rangle)$$

From this final state, we can calculate the measurement probabilities and the marginal probabilities of each qubit:

- Measurement probabilities:

$$P(00) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2}$$

$$P(01) = 0$$

$$P(10) = \left| -\frac{i}{\sqrt{2}} \right|^2 = \left| -\frac{1}{\sqrt{2}} \cdot i \right|^2 = \left(\sqrt{0^2 + \left(-\frac{1}{\sqrt{2}} \right)^2} \right)^2 = \frac{1}{2}$$

$$P(11) = 0$$

- Marginal probabilities:

$$P(q_1 = 0) = P(00) + P(01) = \frac{1}{2} + 0 = \frac{1}{2}$$

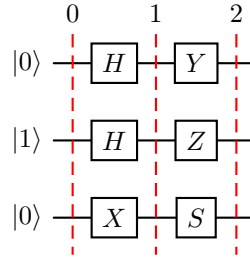
$$P(q_1 = 1) = P(10) + P(11) = \frac{1}{2} + 0 = \frac{1}{2}$$

$$P(q_2 = 0) = P(00) + P(10) = \frac{1}{2} + \frac{1}{2} = 1$$

$$P(q_2 = 1) = P(01) + P(11) = 0 + 0 = 0$$

❓ What if we have a circuit with more than two qubits?

The workflow does not change when the number of qubits increases. The only difference is that the tensor product will have more terms, and the equivalent matrix will be larger. The same rules apply: multiply matrices from right to left for consecutive gates, and use the tensor product for parallel gates. Consider the following circuit with three qubits:



There are three layers, and the equivalent matrix is:

$$(Y \otimes Z \otimes S) \cdot (H \otimes H \otimes X) \cdot (|0\rangle \otimes |1\rangle \otimes |0\rangle)$$

Let's label each layer with a temporary state $|\psi_i\rangle$, where i is the layer number:

- **Initial state** $|\psi_0\rangle$. The three qubits start in $|010\rangle$, therefore:

$$|\psi_0\rangle = |0\rangle \otimes |1\rangle \otimes |0\rangle = |010\rangle$$

The order is always from top to bottom, because the first qubit is the most significant bit and the last qubit is the least significant bit: $|q_1 q_2 q_3\rangle$.

- **First layer** $|\psi_1\rangle$. Since the gates are simultaneous, we apply the tensor product of the three gates:

$$\begin{aligned} |\psi_1\rangle &= (H \otimes H \otimes X) |\psi_0\rangle \\ &= (H \otimes H \otimes X) |010\rangle \\ &= H |0\rangle \otimes H |1\rangle \otimes X |0\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes |1\rangle \end{aligned}$$

💡 Again, leave the tensor product factorized, because it is easier to work with.

- **Second layer** $|\psi_2\rangle$. Again, we apply the tensor product of the three gates:

$$\begin{aligned} |\psi_2\rangle &= (Y \otimes Z \otimes S) |\psi_1\rangle \\ &= (Y \otimes Z \otimes S) \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes |1\rangle \right] \end{aligned}$$

- First qubit (Pauli-Y changes the relative phase of the $|1\rangle$ qubit and flips it):

$$\begin{aligned} Y \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right] &= \frac{1}{\sqrt{2}} (Y|0\rangle + Y|1\rangle) \\ Y|0\rangle &= \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ i \end{bmatrix} = i|1\rangle \\ Y|1\rangle &= \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -i \\ 0 \end{bmatrix} = -i|0\rangle \end{aligned}$$

Therefore:

$$Y \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right] = \frac{1}{\sqrt{2}} (-i|0\rangle + i|1\rangle) = i \cdot \frac{1}{\sqrt{2}} (|1\rangle - |0\rangle)$$

We don't like how the minus sign is in front of the $|0\rangle$ term, so we can factor out a -1 to get:

$$i \cdot \frac{1}{\sqrt{2}} (|1\rangle - |0\rangle) = -i \cdot \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

- Second qubit (Pauli-Z changes only the relative phase of the $|1\rangle$ qubit):

$$Z \left[\frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \right] = \frac{1}{\sqrt{2}} (Z|0\rangle - Z|1\rangle) = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

- Third qubit (phase gate S changes only the relative phase of the $|1\rangle$ qubit):

$$S|1\rangle = i|1\rangle$$

The final state is:

$$|\psi_2\rangle = \left[-i \cdot \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \right] \otimes \left[\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right] \otimes (i|1\rangle)$$

We can simplify the global phase $-i$ and i : $-i \cdot i = -i^2 = 1$. Therefore, the final state is:

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |1\rangle$$

We can also simplify the tensor product of the first two qubits and leave the third qubit factorized:

$$\begin{aligned} |\psi\rangle &= \frac{1}{2} (|0\rangle|0\rangle + |0\rangle|1\rangle - |1\rangle|0\rangle - |1\rangle|1\rangle) \otimes |1\rangle \\ &= \frac{1}{2} (|00\rangle + |01\rangle - |10\rangle - |11\rangle) \otimes |1\rangle \\ &= \frac{1}{2} (|001\rangle + |011\rangle - |101\rangle - |111\rangle) \end{aligned}$$

4.14 Phase Kickback

4.14.1 Prerequisites: Eigenstates and Eigenvalues

Before introducing phase kickback, we briefly review the concepts of **eigenstates** and **eigenvalues**, which play a central role throughout quantum computing. These concepts were already introduced when explaining single-qubit gates, but they become particularly important for understanding controlled unitary operations.

Recall that a quantum gate is represented by a **unitary operator** U . Most quantum states are transformed into different states when the gate is applied. However, there exists a special class of states whose direction in Hilbert space is preserved.

⊗ Eigenstates

A quantum state $|v\rangle$ is called an **eigenstate** (or **eigenvector**) of a unitary operator U if applying the operator changes the state only by a multiplicative constant:

$$U|v\rangle = \lambda|v\rangle$$

Where λ is called the **eigenvalue** associated with the eigenstate $|v\rangle$.

Unlike a generic state transformation:

$$U|\psi\rangle = |\psi'\rangle$$

An eigenstate is not rotated into a different direction. Instead, the operator simply multiplies the entire state by a constant factor.

Therefore, applying U to one of its eigenstates leaves the physical state unchanged except for the multiplicative coefficient.

↻ Eigenvalues of Unitary Operators

Since every quantum gate is represented by a **unitary operator**, its eigenvalues satisfy:

$$|\lambda| = 1$$

Consequently, every eigenvalue can always be written in the form:

$$\lambda = e^{i\phi}$$

Where:

- ϕ is a real number called the **phase angle**;
- $e^{i\phi}$ is a complex number of unit modulus.

Therefore, the eigenvalue does not change the norm of the quantum state; it only introduces a phase factor.

The eigenvalue equation for a quantum gate is therefore usually written as:

$$U |v\rangle = e^{i\phi} |v\rangle$$

Special Case: Real Symmetric Unitary Operators

Some quantum gates, such as the Pauli- X gate, are represented by matrices that are both **real** (i.e., all entries are real numbers) and **symmetric** (i.e., the matrix equals its own transpose).

For this particular class of operators, the eigenvalues must be real. Since unitary eigenvalues always have modulus one, the only possible values are

$$\lambda \in \{+1, -1\}$$

For example, the Pauli- X gate has the two eigenstates:

$$X |+\rangle = + |+\rangle, \quad \text{and} \quad X |-\rangle = - |-\rangle$$

These two eigenstates will become extremely important when we introduce the concept of phase kickback and Deutsch's algorithm in the next sections.

Why are Eigenstates important?

At first sight, multiplying a state by a phase factor may seem uninteresting, since the physical state does not appear to change. However, **phase kickback exploits exactly this property.**

When a unitary operator is applied inside a **controlled operation**, the phase associated with one of its eigenstates can become a **relative phase** between components of a superposition. Unlike a global phase, a relative phase can influence interference and eventually become observable through measurement.

For this reason, the study of phase kickback begins with the eigenvalue equation:

$$U |v\rangle = e^{i\phi} |v\rangle$$

Which forms the mathematical foundation of everything that follows in the next sections.

4.14.2 Eigenvalues of Unitary Operators as Phase Factors

In the previous section, we recalled that an eigenstate $|v\rangle$ of a unitary operator U satisfies:

$$U|v\rangle = \lambda|v\rangle$$

Since U is unitary (i.e., $U^\dagger U = I$), the eigenvalue λ cannot be an arbitrary complex number. It must have modulus one:

$$|\lambda| = 1$$

Therefore, every **eigenvalue of a unitary operator** can be written as

$$\lambda = e^{i\phi}$$

Where $\phi \in \mathbb{R}$ is a phase angle. The eigenvalue equation then becomes:

$$U|v\rangle = e^{i\phi}|v\rangle$$

This means that when a unitary operator acts on one of its eigenstates, it **does not change the direction** of the state vector. It only **multiplies the state by a complex phase factor**.

❓ Why must the eigenvalue have modulus one?

Suppose that $|v\rangle$ is a normalized eigenstate of U :

$$U|v\rangle = \lambda|v\rangle$$

Because U is unitary, it preserves the norm of every state. Therefore:

$$\|U|v\rangle\| = \||v\rangle\|$$

Since $|v\rangle$ is normalized:

$$\||v\rangle\| = 1$$

Using the eigenvalue equation:

$$\|U|v\rangle\| = \|\lambda|v\rangle\|$$

Multiplying a vector by a scalar multiplies its norm by the modulus of that scalar:

$$\|\lambda|v\rangle\| = |\lambda| \||v\rangle\|$$

Therefore:

$$|\lambda| \||v\rangle\| = \||v\rangle\|$$

Since $\||v\rangle\| = 1$:

$$|\lambda| = 1$$

This is necessary because a quantum gate must preserve normalization and therefore preserve total probability.

Complex numbers of modulus one

A complex number of modulus one lies on the unit circle of the complex plane. Using Euler's formula:

$$e^{i\phi} = \cos \phi + i \sin \phi$$

We obtain:

$$|e^{i\phi}| = \sqrt{\cos^2(\phi) + \sin^2(\phi)} = 1$$

Therefore, writing:

$$\lambda = e^{i\phi}$$

Means that the **eigenvalue represents a rotation by an angle ϕ in the complex plane, without changing the magnitude of the state.** The operator changes **only the phase of the eigenstate:**

$$U |v\rangle = e^{i\phi} |v\rangle$$

In other words, the $e^{i\phi}$ changes the phase of the state, but not its probabilities (i.e., the measurement outcomes).

Example 27: Phase eigenvalues

Some common eigenvalues are:

$$1 = e^{i0}, \quad -1 = e^{i\pi}, \quad i = e^{i\pi/2}, \quad -i = e^{-i\pi/2}$$

These phase factors correspond to rotations around the unit circle:

Eigenvalue	Exponential form	Phase angle
1	e^{i0}	0
i	$e^{i\pi/2}$	$\pi/2$
-1	$e^{i\pi}$	π
$-i$	$e^{-i\pi/2}$	$-\pi/2$

This representation is especially useful because different quantum gates can produce different phase angles on their eigenstates.

Example 28: Pauli-X

The eigenstates of the Pauli-X gate are:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Their eigenvalue equations are:

$$X |+\rangle = |+\rangle = e^{i0} |+\rangle, \quad X |-\rangle = -|-\rangle = e^{i\pi} |-\rangle$$

Therefore:

- $|+\rangle$ acquires a phase of 0;

- $|-\rangle$ acquires a phase of π .

This difference will later be essential in phase kickback. A controlled- X acting on a target in $|-\rangle$ can produce a relative minus sign on the control qubit.

Example 29: Phase gate (S)

The phase gate is:

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$$

Its computational-basis states are eigenstates:

$$S|0\rangle = |0\rangle = e^{i0}|0\rangle, \quad S|1\rangle = i|1\rangle = e^{i\pi/2}|1\rangle$$

Therefore:

- $|0\rangle$ has eigenvalue $1 = e^{i0}$;
- $|1\rangle$ has eigenvalue $i = e^{i\pi/2}$.

The state $|1\rangle$ is not changed into a different basis state. It only acquires a phase of $\pi/2$.

 **Real symmetric unitary operators.** If a unitary operator is also real and symmetric, then its eigenvalues are real. This simplifies the analysis of eigenvalues, because we already know that every unitary eigenvalue must satisfy:

$$|\lambda| = 1$$

The only real numbers with modulus one are:

$$\lambda = +1 \quad \text{or} \quad \lambda = -1$$

Equivalently:

$$+1 = e^{i0}, \quad -1 = e^{i\pi}$$

This is why **operators such as X , Z , and H have eigenvalues $+1$ and -1** . The negative eigenvalue should not be interpreted as changing the magnitude of the state. It represents a phase shift of π :

$$-|v\rangle = e^{i\pi}|v\rangle$$

Which does not change the measurement probabilities of the state.

In summary, for an eigenstate $|v\rangle$ of a unitary operator U :

$$U|v\rangle = e^{i\phi}|v\rangle$$

The eigenvalue $e^{i\phi}$:

- Has modulus one;
- Preserves normalization;
- Does not change measurement probabilities by itself;
- Represents a phase rotation;
- Can become computationally useful when converted into a relative phase.

4.14.3 Controlled Unitary Operators

In general, if we apply a unitary operator U directly to one of its eigenstates:

$$U |v\rangle = e^{i\phi} |v\rangle$$

The eigenvalue only creates a **global phase**. Since global phases are physically unobservable, the phase information is effectively “*hidden*”. The natural question is therefore: *can we apply the unitary only to part of the quantum state, so that the phase becomes relative instead of global?* The answer is yes, by using a **controlled unitary operator**.

🔍 Motivation

Suppose we have two qubits:

- A **control qubit**, which decides whether the operation is executed;
- A **target qubit**, on which the unitary U acts.

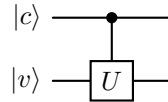
Instead of always applying U to the target qubit:

$$|v\rangle \longrightarrow U |v\rangle = e^{i\phi} |v\rangle$$

We apply it **conditionally**:

- If the control qubit is $|0\rangle$, do nothing to the target qubit;
- If the control qubit is $|1\rangle$, apply U to the target qubit.

Graphically, this is represented as follows:



This simple modification is the key ingredient behind phase kickback.

📖 Definition 13: Controlled Unitary

A **Controlled Unitary Operator** is the two-qubit operator CU that performs:

$$|0\rangle |\psi\rangle \longrightarrow |0\rangle |\psi\rangle, \quad |1\rangle |\psi\rangle \longrightarrow |1\rangle U |\psi\rangle$$

Only the **target qubit** is **modified**. The **control qubit** itself is **never changed**.

Since the operator acts on two qubits, its matrix has dimension 4×4 . It has the block form:

$$CU = \begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix} \quad (87)$$

Where:

- The upper-left block corresponds to the control being $|0\rangle$;

- The lower-right block corresponds to the control being $|1\rangle$.

≡ Action on Computational-Basis States

Assume that the target is an eigenstate:

$$U|v\rangle = e^{i\phi}|v\rangle$$

The two possible values of the control qubit are:

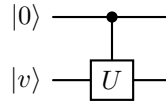
- **Control qubit is $|0\rangle$.** Initially, the two-qubit state is:

$$|0\rangle|v\rangle$$

Applying the controlled unitary, nothing happens:

$$CU|0\rangle|v\rangle = |0\rangle|v\rangle$$

Graphically, this is represented as follows:



Since the control is $|0\rangle$, the “*switch*” remains off.

- **Control qubit is $|1\rangle$.** Initially, the two-qubit state is:

$$|1\rangle|v\rangle$$

Applying the controlled unitary, the target is modified:

$$CU|1\rangle|v\rangle = |1\rangle \otimes U|v\rangle = |1\rangle \otimes e^{i\phi}|v\rangle = e^{i\phi}|1\rangle|v\rangle$$

⚠ Exactly the same eigenvalue $e^{i\phi}$ appears, but **nothing useful** is gained, since it is still a **global phase**.

❓ Why do we still need a Controlled Unitary?

At this point, the controlled operator may seem disappointing because when the control is $|1\rangle$, the eigenvalue is still a global phase (irrelevant). However, notice something important: **the phase appears only when the control qubit is $|1\rangle$** . This means that if the control qubit were somehow simultaneously in a superposition of $|0\rangle$ and $|1\rangle$, the phase would affect only one branch of the computation.

So, instead of choosing a simple computational-basis state for the control qubit, we prepare the control qubit in a superposition:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Now the controlled unitary acts only on the $|1\rangle$ branch of the superposition:

$$CU |+\rangle |v\rangle = \frac{1}{\sqrt{2}} (CU |0\rangle |v\rangle + CU |1\rangle |v\rangle) = \frac{1}{\sqrt{2}} (|0\rangle |v\rangle + e^{i\phi} |1\rangle |v\rangle)$$

Now the phase no longer multiplies the **whole state**, but **only one branch** of the superposition. This transforms the previously invisible global phase into an observable relative phase.

✔ **Factor out $|v\rangle$.** Both branches of the superposition contain the same $|v\rangle$, so we can factor it out:

$$CU |+\rangle |v\rangle = \frac{1}{\sqrt{2}} (|0\rangle + e^{i\phi} |1\rangle) \otimes |v\rangle$$

Now, the phase is **not** multiplying the target qubit anymore, instead $e^{i\phi}$ appears only in front of the $|1\rangle$ component of the **control qubit**. This is the essential mathematical mechanism behind **phase kickback**. The next section will formalize this phenomenon and explain why we say the phase is “*kicked back*” from the target qubit to the control qubit.

4.14.4 Definition and Derivation of Phase Kickback

At this point we have already derived the mathematical result:

$$CU |+\rangle |v\rangle = \frac{|0\rangle + e^{i\phi} |1\rangle}{\sqrt{2}} \otimes |v\rangle$$

Now we can finally give a precise definition of **phase kickback**.

Definition 14: Phase Kickback

Phase Kickback is the quantum phenomenon in which the eigenvalue phase generated by applying a controlled unitary to one of its eigenstates appears as a relative phase on the control qubit instead of remaining on the target qubit.

Equivalently, mathematically we can define phase kickback as the following property of controlled unitary operators:

$$CU |+\rangle |v\rangle = \frac{|0\rangle + e^{i\phi} |1\rangle}{\sqrt{2}} \otimes |v\rangle \quad (88)$$

Where U is a unitary operator, $|v\rangle$ is an eigenstate of U and $e^{i\phi}$ is the corresponding eigenvalue of U , i.e.:

$$U |v\rangle = e^{i\phi} |v\rangle$$

Notice the remarkable result:

- The target state is still $|v\rangle$;
- The phase now modifies the control qubit $|+\rangle$ instead of the target qubit $|v\rangle$.

Formally, start with:

$$|\psi_{\text{in}}\rangle = |+\rangle |v\rangle$$

Expanding the superposition of the control qubit $|+\rangle$ we have:

$$|\psi_{\text{in}}\rangle = \frac{|0\rangle |v\rangle + |1\rangle |v\rangle}{\sqrt{2}}$$

Now we can apply the controlled unitary CU to the input state:

$$\begin{aligned} CU |\psi_{\text{in}}\rangle &= CU \left(\frac{|0\rangle |v\rangle + |1\rangle |v\rangle}{\sqrt{2}} \right) \\ &= \frac{CU |0\rangle |v\rangle + CU |1\rangle |v\rangle}{\sqrt{2}} \\ &= \frac{|0\rangle |v\rangle + |1\rangle U |v\rangle}{\sqrt{2}} \\ &= \frac{|0\rangle |v\rangle + |1\rangle e^{i\phi} |v\rangle}{\sqrt{2}} \\ &= \frac{|0\rangle + e^{i\phi} |1\rangle}{\sqrt{2}} \otimes |v\rangle \end{aligned}$$

❓ Why is it called “kickback”?

If we looked only at the target qubit $|v\rangle$, we would expect that the eigenvalue phase $e^{i\phi}$ would be applied to it:

$$U|v\rangle = e^{i\phi}|v\rangle$$

So one might think: “*the phase belongs to the target*”. However, after applying the controlled operation we obtain:

$$\left(\frac{|0\rangle + e^{i\phi}|1\rangle}{\sqrt{2}} \right) \otimes |v\rangle$$

The target has returned to exactly the same state $|v\rangle$ as before, while the control now carries the phase. It therefore **looks as if the phase has travelled backwards from the target to the control**. This apparent transfer is why the phenomenon is called phase kickback.

⚠ The 4 conditions for phase kickback

Phase kickback **does not happen automatically**. It requires four ingredients:

1. **A controlled unitary**. The operation must be CU . If we simply apply U to a qubit, there is no control branch where a relative phase can appear.
2. **The target must be an eigenstate**. The target must satisfy:

$$U|v\rangle = e^{i\phi}|v\rangle$$

Otherwise, U changes the state itself, not just its phase. For example:

$$X|0\rangle = |1\rangle$$

Is **not** an eigenstate relation. There is no simple phase to kick back because the target state changes from $|0\rangle$ to $|1\rangle$.

3. **The control must be in superposition**. The control must contain at least two branches, typically:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Without superposition, there is only one branch, so every phase is global.

4. **The eigenvalue must contribute a nontrivial phase**. If the eigenvalue is 1, then the phase is trivial and there is no relative phase to kick back. For example, if $e^{i\phi} = 1$, then:

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{|0\rangle + 1 \cdot |1\rangle}{\sqrt{2}}$$

Nothing changes. So the interesting cases are $e^{i\phi} \neq 1$.

❓ **Does the phase literally move?** When we say “*the phase is kicked back*”, we do **not** mean that some physical object travels from one qubit to another. Nothing is moving, instead the mathematics of the tensor product allows us to rewrite:

$$|0\rangle \otimes |v\rangle + e^{i\phi} |1\rangle \otimes |v\rangle$$

As:

$$(|0\rangle + e^{i\phi} |1\rangle) \otimes |v\rangle$$

The phase was always a scalar multiplying one branch of the joint quantum state. So the “kickback” is a **mathematical redistribution of the phase in the tensor-product expression**, not the movement of a physical quantity.

4.14.5 Making the Kicked-Back Phase Observable

In the previous section we discovered the central result of phase kickback:

$$CU |+\rangle |v\rangle = \left(\frac{|0\rangle + e^{i\phi} |1\rangle}{\sqrt{2}} \right) \otimes |v\rangle$$

This is already a remarkable result: the eigenvalue phase has become a **relative phase** on the control qubit. However, a natural question immediately arises: *can we measure this phase directly?* Surprisingly, the answer is **no**. This section explains why the kicked-back phase is **hidden from a direct measurement**, and why quantum algorithms always perform an additional interference step before measuring.

□ The state after phase kickback

Recall the final state after applying the controlled unitary:

$$|\psi\rangle = CU |+\rangle |v\rangle = \left(\frac{|0\rangle + e^{i\phi} |1\rangle}{\sqrt{2}} \right) \otimes |v\rangle$$

Since the target qubit is unchanged, we only need to examine the control qubit:

$$|c\rangle = \frac{|0\rangle + e^{i\phi} |1\rangle}{\sqrt{2}}$$

Everything we want to know is encoded in the phase $e^{i\phi}$. The question is therefore: *if we simply measure this qubit, can we recover ϕ ?*

✖ Measuring in the computational basis

Suppose we perform a standard measurement in the computational basis:

$$\{|0\rangle, |1\rangle\}$$

The amplitudes are:

$$|\alpha\rangle = \frac{1}{\sqrt{2}}, \quad |\beta\rangle = \frac{e^{i\phi}}{\sqrt{2}}$$

Measurement probabilities are obtained from the squared moduli:

- Measuring 0:

$$P(0) = |\alpha|^2 = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2}$$

- Measuring 1 is a little more interesting:

$$P(1) = |\beta|^2 = \left| \frac{e^{i\phi}}{\sqrt{2}} \right|^2$$

Using:

$$|z|^2 = z^* z$$

Where z^* is the complex conjugate of z , we have:

$$\begin{aligned}
 P(1) &= \left(\frac{e^{i\phi}}{\sqrt{2}} \right)^* \left(\frac{e^{i\phi}}{\sqrt{2}} \right) \\
 &= \left(\frac{e^{-i\phi}}{\sqrt{2}} \right) \left(\frac{e^{i\phi}}{\sqrt{2}} \right) \\
 &= \frac{e^{-i\phi} e^{i\phi}}{2} \\
 &= \frac{e^{-i\phi+i\phi}}{2} \\
 &= \frac{e^0}{2} \\
 &= \frac{1}{2}
 \end{aligned}$$

Therefore, the measurement probabilities are:

$$P(0) = P(1) = \frac{1}{2}$$

This means that the measurement outcome is completely independent of the phase ϕ . So a **computational-basis** measurement **cannot distinguish any of these states**. The reason lies in how quantum measurements work: **a measurement does not use the complex amplitudes directly, instead it computes the squared modulus of the amplitudes**. Taking the modulus removes every complex phase factor, and therefore the measurement outcome is completely independent of the phase ϕ .

Phase vs Amplitude Information

A quantum state contains two different kinds of information:

1. **Amplitude:** Determines the probability of measuring a particular state. For example:

$$\frac{\sqrt{3}}{2} |0\rangle + \frac{1}{2} |1\rangle$$

Has:

$$P(0) = \left| \frac{\sqrt{3}}{2} \right|^2 = \frac{3}{4}, \quad P(1) = \left| \frac{1}{2} \right|^2 = \frac{1}{4}$$

These probabilities are *immediately* observable.

2. **Phase.** Compare the two states:

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad \frac{|0\rangle + i|1\rangle}{\sqrt{2}}$$

The amplitudes have identical magnitudes $\frac{1}{\sqrt{2}}$. Yet the two quantum states are different because their **relative phases** are different. The phase is part of the quantum state, but it is **not directly visible through a computational-basis measurement**.

? **Then why is phase useful?** At this point it may seem that phases are useless. After all, if they cannot be measured, *why do quantum algorithms spend so much effort creating them?* The answer is that **phases become observable when different branches of a superposition interfere**. We do not measure the phase itself. Instead, we first **convert the phase into amplitudes**, and **then** measure those amplitudes.

Q The Hadamard gate as a phase detector

The most common gate used to convert phase information into amplitude information is the Hadamard gate. Recall its action:

$$H|+\rangle = |0\rangle, \quad H|-\rangle = |1\rangle$$

Before the Hadamard, the states:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Both have equal amplitudes:

$$P(0) = P(1) = \frac{1}{2}$$

A direct measurement cannot distinguish them. After applying the Hadamard, the states become:

$$|+\rangle \xrightarrow{H} |0\rangle, \quad |-\rangle \xrightarrow{H} |1\rangle$$

Now the measurement probabilities are:

$$P(0) = 1, \quad P(1) = 0$$

For $|+\rangle$, and:

$$P(0) = 0, \quad P(1) = 1$$

For $|-\rangle$. The Hadamard has converted **relative phase** into **observable probability**.

✏ Example 30: Making the Kicked-Back Phase Observable

Suppose phase kickback produces:

$$\frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Suppose phase kickback produces:

$$P(0) = P(1) = \frac{1}{2}$$

No information is revealed. Now apply a Hadamard:

$$H\left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) = |1\rangle$$

A measurement now yields:

$$P(0) = 0, \quad P(1) = 1$$

The phase has become completely observable. Not because we measured it directly, but because interference transformed it into an amplitude difference.

4.14.6 Repeated Controlled Operations

In the previous sections, we saw that if the target qubit is an eigenstate of a unitary operator U :

$$U |v\rangle = e^{i\phi} |v\rangle$$

Then a controlled application of U transfers the eigenvalue phase $e^{i\phi}$ onto the control qubit through the *phase kickback* effect. Now, after having seen how to measure the kicked-back phase, we can ask ourselves *what happens if we apply the controlled operation multiple times?* The answer is really simple: **each application contributes the same phase, so the phases accumulate.** This idea is fundamental because it is exactly what allows algorithms such as Quantum Phase Estimation (QPE) to determine an unknown phase with very high precision.

🔄 Applying a unitary twice

Let's start with the application of a controlled unitary U :

$$U |v\rangle = e^{i\phi} |v\rangle$$

Where:

- $|v\rangle$ is an eigenstate of U ;
- $e^{i\phi}$ is the corresponding eigenvalue, which is a phase factor.

Now apply the same operator a **second time**. We obtain:

$$U^2 |v\rangle = U (U |v\rangle)$$

Substitute the eigenvalue equation:

$$U^2 |v\rangle = U (e^{i\phi} |v\rangle)$$

Since $e^{i\phi}$ is just a complex scalar, it can be factored outside the operator:

$$U^2 |v\rangle = e^{i\phi} U |v\rangle$$

Apply the eigenvalue equation once more:

$$U^2 |v\rangle = e^{i\phi} (e^{i\phi} |v\rangle) = e^{2i\phi} |v\rangle$$

🔄 **Applying a unitary n times.** After applying the unitary n times:

$$U^n = \underbrace{U U \dots U}_{n \text{ times}}$$

The eigenstate satisfies the following equation:

$$U^n |v\rangle = e^{in\phi} |v\rangle \quad (89)$$

The key observation is that **the state never stops being an eigenstate**. Multiplying a vector by a scalar does **not** change its direction in Hilbert space (it only changes its phase). Therefore, after every application, the state is still proportional to $|v\rangle$, so applying U again produces exactly the same eigenvalue. This process repeats indefinitely.

Controlled powers of a unitary

Now recall the phase kickback circuit. Previously we saw that if the target qubit is an eigenstate of U , then a controlled application of U transfers the eigenvalue phase $e^{i\phi}$ onto the control qubit. Now, suppose instead that we use a controlled version of U^n . The control qubit starts in:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

The target is still the eigenstate $|v\rangle$. Initially, the state of the two qubits is:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle|v\rangle + |1\rangle|v\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |v\rangle$$

Applying the controlled operator gives:

$$CU^n|\psi\rangle = \frac{|0\rangle|v\rangle + |1\rangle U^n|v\rangle}{\sqrt{2}}$$

Using the eigenvalue equation for U^n :

$$U^n|v\rangle = e^{in\phi}|v\rangle$$

We can substitute this into the previous equation:

$$CU^n|\psi\rangle = \left(\frac{|0\rangle + e^{in\phi}|1\rangle}{\sqrt{2}} \right) \otimes |v\rangle \quad (90)$$

Notice that the kicked-back phase is now $e^{in\phi}$, instead of $e^{i\phi}$. This is exactly what we expected: **the phase accumulates with each application of the unitary operator.**

Why is this useful?

At first glance, multiplying a phase might not seem very exciting. However, it becomes extremely powerful when we want to **estimate an unknown phase**.

Suppose:

$$U|v\rangle = e^{i\phi}|v\rangle$$

Where ϕ is **completely unknown**. After *phase kickback* we obtain the state:

$$|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)$$

Great, but **can we read ϕ by measuring the qubit?** The answer is no. If we measure in the computational basis, we will always get either $|0\rangle$ or $|1\rangle$ with equal probability (50%). The probabilities do **not** depend on ϕ . So one controlled operation is **not enough** to estimate the phase. Also, if we apply the controlled operation multiple times, we can accumulate the phase:

$$CU^n|\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{in\phi}|1\rangle)$$

And the probabilities of measuring $|0\rangle$ or $|1\rangle$ are still 50% each. So even if we apply the controlled operation multiple times, we still cannot estimate the phase.

However, if we apply **different powers of the unitary** ($U^1, U^2, U^4, \dots$), we can **accumulate different phases on different qubits** (i.e., the phase is accumulated on each control qubit). This is exactly what **Quantum Phase Estimation (QPE)** does: it **uses multiple control qubits, each controlling a different power of the unitary, to accumulate different phases on each control qubit**. Then, by measuring all the control qubits, we can estimate the unknown phase ϕ with very high precision.

4.14.7 Phase Kickback with CNOT

Until now, everything has been abstract:

- Arbitrary unitary U ;
- Arbitrary eigenstate $|v\rangle$;
- Arbitrary eigenvalue $e^{i\phi}$.

Now we'll replace U with one of the most familiar quantum gates: Pauli- X . This example is particularly important because **CNOT is nothing more than a controlled- X gate**, and CNOT appears everywhere in quantum algorithms, including Deutsch's algorithm (that we'll see in the next sections).

🔔 Remember what CNOT really is

The CNOT gate performs the operation:

$$CX = \begin{pmatrix} I & 0 \\ 0 & X \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

This means:

- If the control is $|0\rangle$, do nothing to the target;
- If the control is $|1\rangle$, apply X to the target.

So CNOT is simply **a controlled Pauli- X gate**.

Now, phase kickback only works when the target is an eigenstate of the unitary. So we first ask: *what are the eigenstates of X ?* We know that X is a Pauli matrix:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

To find the eigenstates:

- Write a generic qubit:

$$|v\rangle = \begin{pmatrix} a \\ b \end{pmatrix} \quad a, b \in \mathbb{C}$$

- Apply X to $|v\rangle$:

$$X|v\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} b \\ a \end{pmatrix}$$

- Apply the eigenvalue definition:

$$X|v\rangle = \lambda|v\rangle \quad \Rightarrow \quad \begin{pmatrix} b \\ a \end{pmatrix} = \lambda \begin{pmatrix} a \\ b \end{pmatrix} \quad \Rightarrow \quad \begin{cases} b = \lambda a \\ a = \lambda b \end{cases}$$

- Find the eigenvalues by substituting b in the second equation:

$$a = \lambda b = \lambda(\lambda a) = \lambda^2 a$$

Assuming $a \neq 0$, we can divide both sides by a :

$$1 = \lambda^2 \quad \Rightarrow \quad \lambda = \pm 1$$

- Find the eigenvectors by substituting the eigenvalues in the first equation:

– For $\lambda = 1$:

$$b = \lambda a = 1 \cdot a = a \quad \Rightarrow \quad |v_1\rangle = \begin{pmatrix} a \\ a \end{pmatrix} = a \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Eigenvectors can be multiplied by any nonzero constant and still represent the same direction, so we normalize it:

$$\sqrt{1^2 + 1^2} = \sqrt{2} \quad \Rightarrow \quad |v_1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = |+\rangle$$

– For $\lambda = -1$:

$$b = \lambda a = -1 \cdot a = -a \quad \Rightarrow \quad |v_2\rangle = \begin{pmatrix} a \\ -a \end{pmatrix} = a \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

Normalizing it:

$$\sqrt{1^2 + (-1)^2} = \sqrt{2} \quad \Rightarrow \quad |v_2\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = |-\rangle$$

Therefore, the eigenstates of X are $|+\rangle$ and $|-\rangle$, with eigenvalues $+1$ and -1 , respectively. The phase for each eigenvalue is:

- For $\lambda = +1$: $e^{i\phi} = +1 \quad \Rightarrow \quad \phi = 0$;
- For $\lambda = -1$: $e^{i\phi} = -1 \quad \Rightarrow \quad e^{i\phi} = \cos(\phi) + i\sin(\phi) = -1 \quad \Rightarrow \quad \phi = \pi$.

With these phases, we can now see how phase kickback works with CNOT. We expect that if the target is in $|+\rangle$, the control will remain unchanged (i.e., no phase kickback, $\phi = 0$), while if the target is in $|-\rangle$, the control will acquire a phase of π . Mathematically, we can write:

$$CX |c\rangle |+\rangle = e^{i \cdot 0} |c\rangle |+\rangle = 1 \cdot |c\rangle |+\rangle = |c\rangle |+\rangle \quad (\text{no phase kickback})$$

$$CX |c\rangle |-\rangle = e^{i\pi} |c\rangle |-\rangle = -|c\rangle |-\rangle \quad (\text{phase kickback of } \pi)$$

Example 31: Target in $|+\rangle$

A CX gate with the target in $|+\rangle$ will kick back a phase of 0 to the control qubit. This means that the control qubit will remain unchanged after the operation. Let's see if this is true:

- Control in $|+\rangle$ to allow for superposition of control states:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

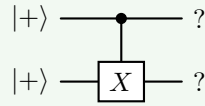
- Target in $|+\rangle$ to be an eigenstate of X with eigenvalue $+1$:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

The initial state is:

$$|\psi\rangle = |+\rangle |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |+\rangle$$

Applying the CX gate (CNOT):

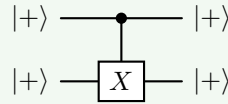


$$CX |\psi\rangle = CX \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |+\rangle \right) = \frac{|0\rangle |+\rangle + |1\rangle X |+\rangle}{\sqrt{2}}$$

Since $X |+\rangle = |+\rangle$, we have:

$$CX |\psi\rangle = \frac{|0\rangle |+\rangle + |1\rangle |+\rangle}{\sqrt{2}} = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |+\rangle = |+\rangle |+\rangle$$

Therefore, the control qubit remains in $|+\rangle$ after the operation, confirming that the phase kickback is 0.



Since there is no phase kickback, the control qubit remains unchanged after the operation.

Example 32: Target in $|-\rangle$

A CX gate with the target in $|-\rangle$ will kick back a phase of π to the control qubit. This means that the control qubit will acquire a phase of $e^{i\pi} = -1$ after the operation. Let's see if this is true:

- Control in $|+\rangle$ to allow for superposition of control states:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

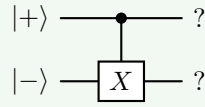
- Target in $|-\rangle$ to be an eigenstate of X with eigenvalue -1 :

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

The initial state is:

$$|\psi\rangle = |+\rangle |-\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |-\rangle$$

Applying the CX gate (CNOT):

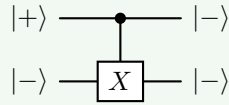


$$CX |\psi\rangle = CX \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |-\rangle \right) = \frac{|0\rangle |-\rangle + |1\rangle X |-\rangle}{\sqrt{2}}$$

Since $X |-\rangle = -|-\rangle$, we have:

$$CX |\psi\rangle = \frac{|0\rangle |-\rangle + |1\rangle (-|-\rangle)}{\sqrt{2}} = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \otimes |-\rangle = |-\rangle |-\rangle$$

Therefore, the control qubit acquires a phase of -1 after the operation, confirming that the phase kickback is π .



Since there is a phase kickback of π , the control qubit changes from $|+\rangle$ to $|-\rangle$ after the operation.

4.14.8 Phase Kickback with Controlled Phase Gates

In the previous section, we have seen the phase kickback using the CNOT gate. Since CNOT is a controlled- X gate, the phase appeared because the target qubit was prepared in an eigenstate of X .

Phase kickback, however, is **not limited to Pauli gates**. It can occur with **any controlled unitary operator**, provided that:

- The *target* qubit is an eigenstate of the unitary;
- The *control* qubit is prepared in a superposition, usually $|+\rangle$.

A particularly simple example is obtained using a **controlled phase gate**.

The **one-qubit Phase Gate** is defined as:

$$P(\phi) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix} \quad (91)$$

Its action is straightforward:

$$P(\phi)|0\rangle = |0\rangle, \quad P(\phi)|1\rangle = e^{i\phi}|1\rangle$$

Unlike the Pauli- X gate, which changes the computational basis states:

$$X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle$$

The phase gate **does not change the basis state**. Instead, it leaves the computational basis unchanged while multiplying the state $|1\rangle$ by the complex phase factor $e^{i\phi}$. For this reason, it is called a *phase gate*.

Eigenstates and Eigenvalues of the Phase Gate. The phase gate is a unitary operator, and its eigenstates are the computational basis states $|0\rangle$ and $|1\rangle$. The corresponding eigenvalues are simply the factors by which the eigenstates are multiplied:

$$P(\phi)|0\rangle = 1 \cdot |0\rangle, \quad P(\phi)|1\rangle = e^{i\phi} \cdot |1\rangle$$

Thus, the eigenvalues of the phase gate are:

$$\lambda_0 = 1, \quad \lambda_1 = e^{i\phi}$$

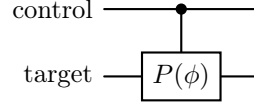
The **Controlled Phase Gate** is a two-qubit gate that applies the phase gate to the target qubit only when the control qubit is in the state $|1\rangle$. Its matrix representation is:

$$CP(\phi) = \begin{pmatrix} I & 0 \\ 0 & P(\phi) \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\phi} \end{pmatrix} \quad (92)$$

As with every controlled gate:

- If the control qubit is $|0\rangle$, the target qubit is left unchanged;
- If the control qubit is $|1\rangle$, the phase gate is applied to the target qubit.

The corresponding quantum circuit is:



Example 33: Target in $|0\rangle$

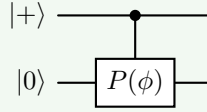
Since:

$$P(\phi)|0\rangle = |0\rangle$$

The eigenvalue is $\lambda = 1$. Therefore, if the target qubit is prepared in the state $|0\rangle$, no phase kickback occurs. The control qubit remains unchanged. Let's verify; prepare the state with the control qubit in $|+\rangle$ and the target qubit in $|0\rangle$:

$$|\psi\rangle = |+\rangle \otimes |0\rangle$$

Applying the controlled phase gate:



$$\begin{aligned}
 CP(\phi)|\psi\rangle &= CP(\phi) \left(\frac{1}{\sqrt{2}} (|0\rangle \otimes |0\rangle + |1\rangle \otimes |0\rangle) \right) \\
 &= \frac{1}{\sqrt{2}} (CP(\phi)|00\rangle + CP(\phi)|10\rangle) \\
 &= \frac{1}{\sqrt{2}} (|00\rangle + |1\rangle P(\phi)|0\rangle) \\
 &= \frac{1}{\sqrt{2}} (|00\rangle + |1\rangle \cdot 1 \cdot |0\rangle) \\
 &= \frac{1}{\sqrt{2}} (|00\rangle + |10\rangle) \\
 &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |0\rangle \\
 &= |\psi\rangle
 \end{aligned}$$

As expected, the state remains unchanged, because the kicked-back phase is $\lambda = 1$: $e^{i \cdot 0} = 1$. Exactly as for CNOT with the target in $|+\rangle$, the phase kickback is trivial.

 Example 34: Target in $|1\rangle$

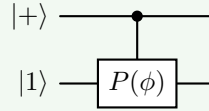
Since:

$$P(\phi) |1\rangle = e^{i\phi} |1\rangle$$

The eigenvalue is $\lambda = e^{i\phi}$. Therefore, if the target qubit is prepared in the state $|1\rangle$, a phase kickback occurs. The control qubit is multiplied by the phase factor $e^{i\phi}$. Let's verify; prepare the state with the control qubit in $|+\rangle$ and the target qubit in $|1\rangle$:

$$|\psi\rangle = |+\rangle \otimes |1\rangle$$

Applying the controlled phase gate:



$$\begin{aligned} CP(\phi) |\psi\rangle &= CP(\phi) \left(\frac{1}{\sqrt{2}} (|0\rangle \otimes |1\rangle + |1\rangle \otimes |1\rangle) \right) \\ &= \frac{1}{\sqrt{2}} (CP(\phi) |01\rangle + CP(\phi) |11\rangle) \\ &= \frac{1}{\sqrt{2}} (|01\rangle + |1\rangle P(\phi) |1\rangle) \\ &= \frac{1}{\sqrt{2}} (|01\rangle + e^{i\phi} |11\rangle) \\ &= \frac{1}{\sqrt{2}} (|0\rangle + e^{i\phi} |1\rangle) \otimes |1\rangle \end{aligned}$$

The target qubit remains unchanged, while the phase has appeared on the **control qubit**. This is precisely the phenomenon of phase kickback. The kicked-back phase is $\lambda = e^{i\phi}$, as expected.

4.14.9 Phase Kickback with Controlled Hadamard

In the previous examples, we studied phase kickback using:

- The controlled- X (CNOT) gate;
- The controlled phase gate $CP(\phi)$.

Both examples were relatively straightforward because their eigenstates were easy to identify. In this final example, we use a **Controlled Hadamard Gate**:

$$CH = \begin{pmatrix} I & 0 \\ 0 & H \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix} \quad (93)$$

Recall that the Hadamard gate is:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Unlike the phase gate, its eigenstates are **not** simply the computational basis states. Instead, the eigenvalues of the Hadamard gate are ± 1 , and the corresponding eigenstates are:

$$|v_+\rangle = \frac{1}{\sqrt{4-2\sqrt{2}}} (|0\rangle + (\sqrt{2}-1)|1\rangle)$$

$$|v_-\rangle = \frac{1}{\sqrt{4+2\sqrt{2}}} (|0\rangle - (\sqrt{2}+1)|1\rangle)$$

To observe phase kickback, we need a **non-trivial phase**. With positive eigenvalues, the phase is trivial (0), so we will use the negative eigenvalue -1 and its corresponding eigenstate $|v_-\rangle$.

Example 35: Phase Kickback with Controlled Hadamard

Let's prepare the state:

- The **control qubit** is in the state $|+\rangle$:

$$|+\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

- The **target qubit** is in the eigenstate $|v_-\rangle$ of the Hadamard gate:

$$|v_-\rangle = \frac{1}{\sqrt{4+2\sqrt{2}}} (|0\rangle - (\sqrt{2}+1)|1\rangle)$$

Applying the controlled Hadamard gate CH to this state will yield:

$$CH(|+\rangle \otimes |v_-\rangle) = \frac{1}{\sqrt{2}} (|0\rangle \otimes |v_-\rangle + |1\rangle \otimes H|v_-\rangle)$$

Since $|v_{-}\rangle$ is an eigenstate of H with eigenvalue -1 , we have:

$$H|v_{-}\rangle = -|v_{-}\rangle$$

Therefore, the state after applying CH becomes:

$$\begin{aligned} CH|+\rangle \otimes |v_{-}\rangle &= \frac{1}{\sqrt{2}} (|0\rangle \otimes |v_{-}\rangle + |1\rangle \otimes H|v_{-}\rangle) \\ &= \frac{1}{\sqrt{2}} (|0\rangle \otimes |v_{-}\rangle + |1\rangle \otimes (-|v_{-}\rangle)) \\ &= \frac{1}{\sqrt{2}} (|0\rangle \otimes |v_{-}\rangle - |1\rangle \otimes |v_{-}\rangle) \\ &= \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes |v_{-}\rangle \\ &= |-\rangle \otimes |v_{-}\rangle \end{aligned}$$

That minus sign then appeared as a **relative phase between the two branches of the control qubit**, which is the essence of phase kickback. The control qubit has been kicked back with a phase of π (or -1) due to the eigenvalue of the target qubit's state under the Hadamard operation.

In general, this example shows that **phase kickback is not tied to any particular quantum gate**. It is a universal consequence of controlled unitary operators. Whenever the target is prepared in an eigenstate of the controlled unitary, the corresponding eigenvalue is transferred to the control qubit as a relative phase. This is the principle that underlies many quantum algorithms, including the **Quantum Phase Estimation** algorithm.

4.15 Deutsch's Algorithm

4.15.1 Problem Definition and the Four Possible Functions

Before learning the algorithm itself, we first need to understand **what problem Deutsch's algorithm is trying to solve**.

The Computational Problem

Suppose someone gives us a mysterious device. We can feed it a bit (0 or 1), and it returns another bit. We **cannot open the device** to see how it works, but we can ask questions by feeding it inputs and observing the outputs. This is what we call a **Black Box** or **Oracle**.

Let's concretize this idea. Imagine the device belongs to another company. Maybe it is an encrypted circuit, a proprietary software routine, a remote server, or simply an expensive experiment. Every time we ask for an output, we pay a cost. The cost could be computation time, money, energy, communication delay, or simply the number of allowed queries (for example, if the device is a quantum experiment that can only be run a limited number of times). Therefore, the **objective is not to reconstruct the function**. Instead, we want to learn **some useful property of the function while asking as few questions as possible**. This is the essence of a **Query Complexity Problem**.

Suppose we are given a function:

$$f : \{0, 1\} \rightarrow \{0, 1\}$$

This notation simply means:

- The input is **one classical bit**;
- The output is **one classical bit**.

Unlike previous examples, **we are not interested in computing the value of the function**. Instead, we only want to discover one global property of the function.

Since there are only two possible inputs, there are only two function evaluations:

$$f(0) \in \{0, 1\} \quad \text{and} \quad f(1) \in \{0, 1\}$$

Deutsch's Algorithm **classifies the function** into one of **two categories**:

- **Constant Function**. A function is *constant* if it **always returns the same output**. Mathematically:

$$f(0) = f(1)$$

There are exactly two constant functions:

- $f(x) = 0$
- $f(x) = 1$

In other words, the function always returns 0 or always returns 1, regardless of the input. Their truth tables are:

x	$f(x) = 0$	$f(x) = 1$
0	0	1
1	0	1

Notice that the output never changes, regardless of the input.

- **Balanced Function.** A function is *balanced* if it returns:
 - 0 exactly once, and
 - 1 exactly once.

Since there are only two possible inputs, this means:

$$f(0) \neq f(1)$$

Again, there are exactly two balanced functions:

- $f(x) = x$
- $f(x) = \bar{x}$

Where $\bar{x}$ is the logical NOT of x . Their truth tables are:

x	$f(x) = x$	$f(x) = \bar{x}$
0	0	1
1	1	0

Unlike constant functions, balanced functions return each output exactly once.

So, instead of identifying exactly which one of the four functions we have, Deutsch proposed an easier question: *is the function constant or balanced?* That sounds almost trivial, but notice something important. To answer this question, we don't need every detail about the function. We only want one **global property** (constant or balanced) of the function.

In other terms, Deutsch's algorithm wants to answer a deeper question: *can a quantum computer determine a global property of an unknown function using fewer queries than any classical computer?*

Definition 15: Deutsch's Algorithm

Deutsch's Algorithm is the first quantum algorithm to demonstrate a genuine quantum computational advantage. Given **oracle** access to an unknown Boolean function (black box):

$$f : \{0, 1\} \rightarrow \{0, 1\}$$

Promised to be either **constant** or **balanced**, the algorithm determines

which of the two cases holds using **a single query** to the quantum oracle. A deterministic classical algorithm requires **two queries** to solve the same problem, because it must evaluate both $f(0)$ and $f(1)$ to determine whether the function is constant or balanced (we will analyze this in detail in the next section).

🧐 **Why should we care today?** The problem itself is not practically useful. Nobody builds quantum computers to classify one-bit functions. The importance of Deutsch's algorithm is that it introduces an idea that appears throughout quantum computing: **rather than learning every output of a function, a quantum algorithm can exploit superposition, interference, and phase kickback to extract a global property directly.** This idea is reused by much more powerful algorithms like Simon's algorithm, Shor's algorithm, and Grover's algorithm. In fact, Deutsch's algorithm is often called the ***Hello World of quantum algorithms*** because it is the simplest example of a quantum algorithm that outperforms classical algorithms.

4.15.2 Classical Solution and Query Complexity

Now that we know the problem about the Deutsch's algorithm, let us ask a natural question: *how many times must a classical computer query the oracle to determine whether the function is constant or balanced?* To answer this question, we first need to introduce the notion of **query complexity**.

Recall that the function is provided as a **black box (oracle)**. We are **not allowed to inspect its implementation**. The only permitted operation is to evaluate the function at a chosen input. For example, we may ask the oracle to evaluate $f(0)$ or $f(1)$. Each evaluation of the oracle is called a **Query**.

Since interacting with the oracle is assumed to be the expensive part of the computation, we measure the efficiency of an algorithm by **counting the number of oracle queries it performs**, rather than the total number of elementary operations. This measure is called the **Query Complexity** of the algorithm.

❓ Can one classical query be enough?

Suppose we decide to evaluate the function only once. For example, we query the oracle with $f(0)$. There are two possible outcomes:

- Case $f(0) = 0$. If the oracle returns 0, there are still two functions consistent with this observation:

Function	$f(0)$	$f(1)$	Type
$f(x) = 0$	0	0	Constant
$f(x) = x$	0	1	Balanced

Both functions produce exactly the same answer for the first query. Therefore, after observing only $f(0) = 0$, we still cannot determine whether the function is constant or balanced.

- Case $f(0) = 1$. Similarly, if the oracle returns 1, there are still two functions consistent with this observation:

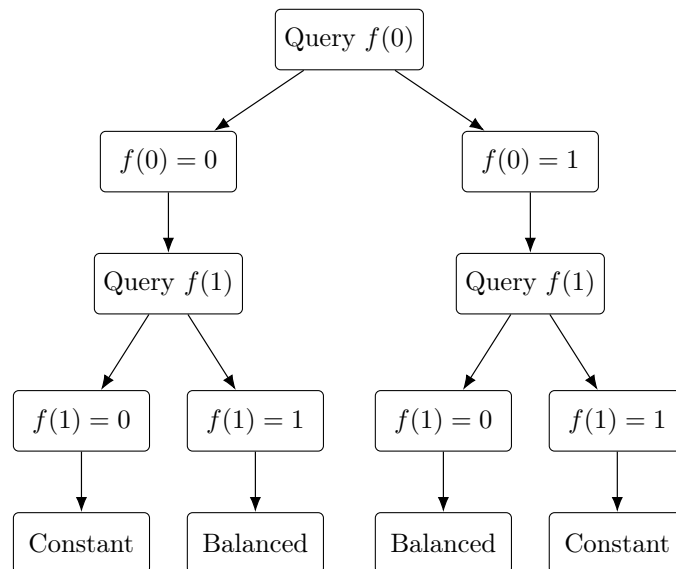
Function	$f(0)$	$f(1)$	Type
$f(x) = 1$	1	1	Constant
$f(x) = \bar{x}$	1	0	Balanced

Again, one query leaves two possible candidates.

⚠️ Why one query is not enough? After a single query, we know only one of the two values: $f(0)$ or $f(1)$. The value of the other input remains unknown.

Since both a constant function and a balanced function can produce exactly the same answer for the queried input, **one classical query can never distinguish the two cases with certainty**. This is true regardless of whether we choose to evaluate $f(0)$ or $f(1)$: one unknown output always remains.

So, to remove the ambiguity, we must evaluate **both** possible inputs. In other words, we need to query the oracle **twice** to determine with certainty whether the function is constant or balanced. The classical strategy can be visualized as the following decision tree:



▲ Classical query complexity. We can now summarize the classical solution. A **deterministic classical algorithm** cannot distinguish between a constant and a balanced function after only one oracle query. It must evaluate both possible inputs $f(0)$ and $f(1)$ requiring **exactly two oracle queries**.

Therefore, the **deterministic classical query complexity of Deutsch's problem is 2**. The remarkable result of Deutsch's algorithm is that a **quantum computer** can **solve the same problem with only one query** to the quantum oracle. And now the natural question arises: *how can a quantum computer possibly do it with only one query?* The answer is provided in the next section.

4.15.3 Quantum Oracle U_f

In the classical version of the problem, the oracle receives one bit x and returns $f(x)$, where f is a function that maps $\{0, 1\}$ to $\{0, 1\}$. Since Deutsch's algorithm is a **quantum algorithm**, we need a **quantum circuit that performs the same computation**. A first idea would be to define a quantum operator satisfying:

$$|x\rangle \mapsto |f(x)\rangle$$

It means that the oracle would take a qubit in state $|x\rangle$ and return a qubit in state $|f(x)\rangle$. Unfortunately, this construction is **not valid in general**.

❓ Why doesn't this work? Every operation performed by a quantum computer must be represented by a **unitary operator**. One fundamental property of unitary operators is that they are **reversible**. In other words, every output state must uniquely determine the corresponding input state. Mathematically:

$$U^{-1}U = UU^{-1} = I$$

Therefore, no two different input states may evolve into the same output state.

If we're not sure why this doesn't work, consider the example of a constant function $f(x) = 0$. Using the naive mapping:

$$|x\rangle \mapsto |f(x)\rangle$$

⚠️ We obtain:

$$\begin{aligned} |0\rangle &\mapsto |f(0)\rangle = |0\rangle \\ |1\rangle &\mapsto |f(1)\rangle = |0\rangle \end{aligned}$$

But this is **not reversible**, since both $|0\rangle$ and $|1\rangle$ are mapped to the same output state $|0\rangle$. Therefore, this **mapping is not a valid quantum operation**.

✅ Preserving the input

The solution is remarkably simple. Instead of replacing the input by the function value, we **keep the original input unchanged** (*data qubit*) and **store the result in a second qubit** (*computation qubit*). The oracle therefore acts on **two qubits** instead of one. The first qubit stores the function input x , while the second qubit stores an auxiliary value, y . The oracle computes the function by modifying only the second qubit (the auxiliary qubit).

The standard **Quantum Oracle** U_f is defined as follows:

$$U_f |x, y\rangle = |x, y \oplus f(x)\rangle \quad (94)$$

Where:

- x is the input bit to the function f . It is called **Data Qubit** and **contains the function input** $|x\rangle$ and **remains unchanged** through the oracle evaluation.

- y is an auxiliary (or computation) bit that is used to **store the result of the function** evaluation. It is called **Computation Qubit** and stores the auxiliary value $|y\rangle$, which becomes $|y \oplus f(x)\rangle$ after the oracle evaluation. It is called also **Ancilla Qubit** or **Target Qubit**.
- $\oplus$ denotes addition modulo 2 (XOR operation).⁸

❓ **Why does XOR make the oracle reversible?** The XOR operation has a crucial property:

$$(a \oplus b) \oplus b = a$$

Applying the same XOR twice restores the original value. Indeed:

$$\begin{aligned} U_f^2 |x, y\rangle &= U_f |x, y \oplus f(x)\rangle \\ &= |x, (y \oplus f(x)) \oplus f(x)\rangle \\ &= |x, y\rangle \end{aligned}$$

Therefore:

$$U_f^{-1} = U_f$$

Meaning that the **oracle is its own inverse**. The transformation is reversible for **every** Boolean function, including the constant ones.

✂ Oracle implementations for the four functions

The four possible Boolean functions correspond to four different quantum circuits:

- $f(x) = 0$, the oracle action is:

$$|x, y\rangle \mapsto |x, y\rangle$$

The circuit implementation is an **identity gate**:

$$\begin{array}{c} |x\rangle \text{ —————} \\ |y\rangle \text{ —————} \end{array}$$

- $f(x) = 1$, the oracle action is:

$$|x, y\rangle \mapsto |x, y \oplus 1\rangle$$

The circuit implementation is a **Pauli-X gate on the computation qubit**:

$$\begin{array}{c} |x\rangle \text{ —————} \\ |y\rangle \text{ — } \boxed{X} \text{ —} \end{array}$$

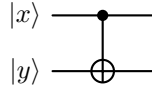
⁸The XOR operation is defined as follows:

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0$$

- $f(x) = x$, the oracle action is:

$$|x, y\rangle \mapsto |x, y \oplus x\rangle$$

The circuit implementation is a **Controlled-NOT** gate, with the **data qubit as control** and the **computation qubit as target**:



- $f(x) = \bar{x}$, the oracle action is:

$$|x, y\rangle \mapsto |x, y \oplus \bar{x}\rangle$$

Since:

$$\bar{x} = 1 \oplus x$$

We can write:

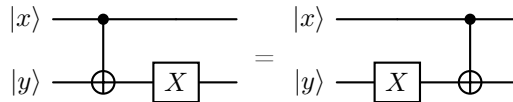
$$y \oplus \bar{x} = y \oplus 1 \oplus x$$

The circuit implementation is:

1. Apply a CNOT from x to y ;
2. Apply a Pauli- X gate to the **computation qubit** y .

The order may also be reversed because an X on the target commutes with this CNOT:

$$CX(I \otimes X) = (I \otimes X)CX$$



Notice an interesting pattern:

- The **constant** functions require **no controlled operation**;
- The **balanced** functions always involve a **controlled-NOT** gate.

At this point this may seem like a coincidence. In the next sections we will see that Deutsch's algorithm exploits this difference using **phase kickback**, allowing the presence or absence of the controlled operation to be detected with a single oracle query.

4.15.4 Preparing the Input State

As we have seen in the previous section, Deutsch's algorithm uses two qubits:

- The **data qubit**, which represents the input x ;
- The **computation qubit**, which allows the oracle to encode $f(x)$ reversibly.

The algorithm begins with:

$$|\psi_0\rangle = |0\rangle |1\rangle$$

The first qubit starts in the state $|0\rangle$ (data qubit), while the second starts in $|1\rangle$ (computation qubit).

⚠ At this point, we could apply the oracle U_f to the state $|\psi_0\rangle$. However, this would not be useful. Indeed:

$$U_f |\psi_0\rangle = U_f |0, 1\rangle = |0, 1 \oplus f(0)\rangle$$

And if $f(0) = 1$:

$$U_f |\psi_0\rangle = |0, 1 \oplus 1\rangle = |0, 0\rangle$$

The second qubit now literally stores the answer, but the Deutsch's algorithm is **not interested in the value of $f(0)$** , because it **does not tell us whether the function is constant or balanced!**

✔ So we need to prepare the input state in a way that allows the oracle to encode the function's behavior on both inputs $x = 0$ and $x = 1$. This is achieved by putting the data qubit in a **superposition** of $|0\rangle$ and $|1\rangle$. Then a Hadamard gate is applied to both qubits:

$$|\psi_1\rangle = (H \otimes H) |0, 1\rangle$$

Using the definition of the Hadamard gate:

$$H |0\rangle = |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad H |1\rangle = |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Where $|+\rangle$ represents the *data qubit* and $|-\rangle$ represents the *computation qubit*. We can rewrite the state $|\psi_1\rangle$ as:

$$\begin{aligned} |\psi_1\rangle &= (H \otimes H) |0, 1\rangle \\ &= H |0\rangle \otimes H |1\rangle \\ &= |+\rangle |-\rangle \\ &= \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\ &= \frac{1}{2} (|0, 0\rangle - |0, 1\rangle + |1, 0\rangle - |1, 1\rangle) \end{aligned}$$

This preparation is not arbitrary. Each of the two states has a precise purpose.

❓ Why is the data qubit prepared in $|+\rangle$?

The data qubit must **allow the oracle to act on both possible inputs**:

$$x = 0 \quad \text{and} \quad x = 1$$

Within the same quantum evolution (i.e., in superposition). The state:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Is an **equal superposition of the two inputs**. Therefore, when the oracle is applied, linearity gives us:

$$\begin{aligned} U_f(|+\rangle \otimes |-\rangle) &= U_f|+\rangle \otimes U_f(|-\rangle) \\ &= U_f\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) \otimes U_f(|-\rangle) \\ &= \left[\frac{1}{\sqrt{2}}U_f(|0\rangle + |1\rangle)\right] \otimes U_f(|-\rangle) \\ &= \frac{1}{\sqrt{2}}U_f(|0, -\rangle + |1, -\rangle) \end{aligned}$$

We simply rewrote the state to demonstrate that the same application of the oracle acts on both branches of the superposition: $|0\rangle$ and $|1\rangle$. In other words, it only means: “*dear oracle, we want to ask about **both possible inputs** $x = 0$ and $x = 1$, so evaluate the function for 0 and 1 simultaneously*”. Thus, the first qubit is **not the answer**; it is simply a **superposition of the two possible questions** we want to ask the oracle.

However, we should be careful with the common statement that the quantum computer “*evaluates both values simultaneously*”. The **algorithm does not directly reveal both** $f(0)$ and $f(1)$. Measurement cannot extract both outputs. Instead, the **oracle encodes a relation** between them **into the relative phase** of the data qubit.

❓ **Why is the data qubit necessary if it doesn't do anything?** At first sight, the data qubit may appear to play a passive role because the oracle never changes its computational basis state. Indeed, in:

$$U_f|x, y\rangle = |x, y \oplus f(x)\rangle$$

The value of x is preserved. However, this does **not** mean that the data qubit remains unchanged. What changes are the **relative phases of its amplitudes**.

Before the oracle, the data qubit is prepared in the equal superposition:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

After the oracle, it becomes:

$$\frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}}$$

Notice that the basis states are still $|0\rangle$ and $|1\rangle$; the oracle has **not** changed the value of x . Instead, it has modified the **relative phase** between the two branches of the superposition. This relative phase is where the information about the function is encoded.

❓ **But how can the oracle modify the data qubit without directly acting on it?** The answer lies in the **computation qubit**. Because it is prepared in the eigenstate $|-\rangle$, the oracle converts the classical output bit $f(x)$ into a phase factor $(-1)^{f(x)}$, which is then transferred to the corresponding branch of the data qubit through **phase kickback**. Let's see how this works in detail.

❓ **Why is the computation qubit prepared in $|-\rangle$?**

Recall that the oracle acts as:

$$U_f |x, y\rangle = |x, y \oplus f(x)\rangle$$

For fixed x , the oracle therefore either (page 231):

- Does nothing to the second qubit when $f(x) = 0$;
- Applies X to the second qubit when $f(x) = 1$.

Indeed:

$$U_f |x, y\rangle = |x\rangle \otimes X^{f(x)} |y\rangle$$

Where $X^{f(x)}$ is the identity when $f(x) = 0$ and the X gate when $f(x) = 1$:

$$X^{f(x)} = \begin{cases} X^0 = I & \text{if } f(x) = 0 \\ X^1 = X & \text{if } f(x) = 1 \end{cases}$$

We want the computation qubit to react differently to these two cases, but without storing the result as an ordinary bit. This is achieved by choosing an eigenstate of X , because eigenstates are only multiplied by a phase factor when acted upon by their corresponding operator. In other words, we want the computation qubit to **convert the classical output bit $f(x)$ into a detectable relative sign**.

The Pauli- X eigenstates are:

$$X |+\rangle = + |+\rangle, \quad \text{and} \quad X |-\rangle = - |-\rangle$$

The state $|-\rangle$ has eigenvalue -1 . Therefore:

$$X^{f(x)} |-\rangle = \begin{cases} X^0 |-\rangle = I |-\rangle = |-\rangle & \text{if } f(x) = 0 \\ X^1 |-\rangle = X |-\rangle = - |-\rangle & \text{if } f(x) = 1 \end{cases}$$

This can be written more compactly as:

$$X^{f(x)} |-\rangle = (-1)^{f(x)} |-\rangle$$

Since the data qubit is in the superposition:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Linearity allows us to write the action of the oracle on the two-qubit state as:

$$\begin{aligned} U_f(|+\rangle|-\rangle) &= \frac{1}{\sqrt{2}} (U_f|0, -\rangle + U_f|1, -\rangle) \\ &= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0, -\rangle + (-1)^{f(1)} |1, -\rangle \right) \\ &= \left(\frac{(-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle}{\sqrt{2}} \right) \otimes |-\rangle \end{aligned}$$

This expression explains why Deutsch's algorithm never learns the individual values $f(0)$ and $f(1)$. The oracle only modifies the relative sign between the two branches:

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}} \longrightarrow \frac{(-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle}{\sqrt{2}}$$

Only the relative phase matters physically.

- **Constant case.** If the two function values are equal, both amplitudes receive the same sign, which corresponds to the state $|+\rangle$ (up to an irrelevant global phase):

$$\begin{aligned} f(0) = f(1) = 0 &\implies \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle \\ f(0) = f(1) = 1 &\implies \frac{-|0\rangle - |1\rangle}{\sqrt{2}} = -\frac{|0\rangle + |1\rangle}{\sqrt{2}} = -|+\rangle = |+\rangle \end{aligned}$$

Notice that the global phase -1 is physically irrelevant, so we can ignore it.

- **Balanced case.** If the two function values are different, the amplitudes acquire opposite signs, which corresponds to the state $|-\rangle$ (again up to a global phase):

$$\begin{aligned} f(0) = 0, f(1) = 1 &\implies \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle \\ f(0) = 1, f(1) = 0 &\implies \frac{-|0\rangle + |1\rangle}{\sqrt{2}} = -\frac{|0\rangle - |1\rangle}{\sqrt{2}} = -|-\rangle = |-\rangle \end{aligned}$$

Again, the global phase -1 is physically irrelevant, so we can ignore it.

Consequently, the oracle's action can be summarized as:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle \quad (95)$$

Where:

- $|x\rangle$ is the data qubit, which explores both inputs $x = 0$ and $x = 1$ simultaneously;

- $|-\rangle$ is the computation qubit, which converts the classical output $f(x)$ into a relative phase factor $(-1)^{f(x)}$ (phase kickback). Its role is therefore to convert $f(x) \in \{0, 1\}$ into the phase factor $(-1)^{f(x)} \in \{+1, -1\}$.

💡 Note that the computation qubit is in the state $|-\rangle$ because its negative eigenvalue transforms the classical output bit $f(x)$ into a detectable relative sign.

In summary, the two qubits are prepared differently because they perform two different jobs:

- $|+\rangle$ explores both inputs $x = 0$ and $x = 1$ simultaneously;
- $|-\rangle$ converts the classical output $f(x)$ into a relative phase factor $(-1)^{f(x)}$ (phase kickback).

4.15.5 Oracle Action and Phase Kickback

This section is the heart of Deutsch's algorithm, even we spoiled it in the previous section. Up to now we've prepared the *right* input state. Here we show **why that preparation was useful**: the oracle no longer stores the function value in the second qubit; it **kicks it back as a phase** onto the first qubit.

In the previous section we prepared the input state:

$$|\psi_1\rangle = |+\rangle |-\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |-\rangle$$

Now we apply the oracle:

$$U_f |x, y\rangle = |x, y \oplus f(x)\rangle$$

At first sight, it may seem that the oracle should modify only the computation qubit, since this is the qubit on which the XOR operation is performed. Surprisingly, once the computation qubit is prepared in the state $|-\rangle$, the oracle leaves it unchanged and instead transfers the information about $f(x)$ into the phase of the data qubit. This phenomenon is known as **phase kickback**.

✂ Oracle Action on the State $|-\rangle$

Recall that:

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

And that the oracle acts as:

$$U_f |x, y\rangle = |x, y \oplus f(x)\rangle$$

Let us evaluate the two possible cases:

- **Case $f(x) = 0$.** The oracle performs no bit flip:

$$U_f |x\rangle |-\rangle = |x\rangle |-\rangle$$

Nothing changes, and the computation qubit remains in the state $|-\rangle$.

- **Case $f(x) = 1$.** The oracle flips the computation qubit:

$$U_f |x\rangle |-\rangle = |x\rangle X |-\rangle$$

Since the Pauli- X operator has eigenvalue -1 on the state $|-\rangle$, we have:

$$X |-\rangle = -|-\rangle \implies U_f |x\rangle |-\rangle = -|x\rangle |-\rangle$$

The computation qubit is still in the state $|-\rangle$; only a minus sign has appeared.

Both cases can therefore be written compactly as:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle \quad \text{where} \quad (-1)^{f(x)} = \begin{cases} +1 & \text{if } f(x) = 0 \\ -1 & \text{if } f(x) = 1 \end{cases} \quad (96)$$

This equation is the key identity behind Deutsch's algorithm.

✂ Applying the Oracle to the Superposition

The data qubit is not in a basis state but in the superposition:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Therefore, the input state to the oracle is:

$$|\psi_1\rangle = |+\rangle |-\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |-\rangle$$

The oracle acts on each branch of the superposition independently:

$$\begin{aligned} U_f |\psi_1\rangle &= \frac{1}{\sqrt{2}} (U_f |0, -\rangle + U_f |1, -\rangle) \\ &= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0, -\rangle + (-1)^{f(1)} |1, -\rangle \right) \end{aligned}$$

Factoring out the unchanged computation qubit ($|-\rangle$), we have:

$$U_f |\psi_1\rangle = \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right) \otimes |-\rangle \quad (97)$$

Notice that:

- The **computation qubit** is exactly the **same as before** ($|-\rangle$);
- All the **information** about the function has been **transferred into the relative signs of the data qubit**.

This is precisely the **phase kickback** mechanism!

The oracle **no longer stores the values** of the function as bits (as in the classical case). Instead, it **encodes** them as **phase factors**:

$$(-1)^{f(0)} \quad \text{and} \quad (-1)^{f(1)}$$

Each branch of the superposition acquires its own sign:

$$\begin{aligned} |0\rangle &\longrightarrow (-1)^{f(0)} |0\rangle \\ |1\rangle &\longrightarrow (-1)^{f(1)} |1\rangle \end{aligned}$$

The quantum state therefore contains both function evaluations simultaneously:

$$\frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right)$$

Not as two readable output bits, but as a **pattern of relative phases**. These relative phases are exactly what the final Hadamard gate will transform into a measurable outcome.

◆ Constant and Balanced Phase Patterns

The four possible functions produce four different phase patterns:

Function	$f(0)$	$f(1)$	Data qubit after the oracle
$f(x) = 0$	0	0	$\frac{ 0\rangle + 1\rangle}{\sqrt{2}} = +\rangle$
$f(x) = 1$	1	1	$-\frac{ 0\rangle + 1\rangle}{\sqrt{2}} = - +\rangle$
$f(x) = x$	0	1	$\frac{ 0\rangle - 1\rangle}{\sqrt{2}} = -\rangle$
$f(x) = \bar{x}$	1	0	$-\frac{ 0\rangle - 1\rangle}{\sqrt{2}} = - -\rangle$

A remarkable pattern immediately appears:

- **Constant functions** produce:

$$\pm |+\rangle$$

Which differ only by a **global phase**.

- **Balanced functions** produce:

$$\pm |-\rangle$$

Which again differ only by a **global phase**.

Since global phases are physically unobservable, the oracle has effectively reduced the four possible functions to **two distinguishable quantum states**:

$$\text{Constant} \longrightarrow |+\rangle \quad \text{and} \quad \text{Balanced} \longrightarrow |-\rangle$$

This is exactly why the algorithm can determine whether the function is constant or balanced with a single oracle query. The only remaining task is to convert the states $|+\rangle$ and $|-\rangle$ into computational basis states, which is accomplished by the final Hadamard gate (next section).

4.15.6 Interference and Final Measurement

After the oracle has been applied, the computation qubit remains in the state $|-\rangle$, while the data qubit contains the information about the function encoded as a pattern of relative phases.

The state of the two qubits is:

$$|\psi_2\rangle = \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right) \otimes |-\rangle$$

⚠ At this point, the computation is almost complete. However, there is still a **problem**: **relative phases cannot be observed directly by measurement**. If we were to measure the data qubit immediately, we would obtain:

$$P(0) = P(1) = \frac{1}{2}$$

Regardless of the values of $f(0)$ and $f(1)$. The information is present in the quantum state, but it is hidden in the interference between the two amplitudes rather than in their magnitudes. Therefore, to reveal this information, **Deutsch's algorithm applies one final Hadamard gate to the data qubit**.

✓ Solution: Applying the Final Hadamard Gate

The final operation of the algorithm is:

$$H \otimes I$$

Which **acts only on the data qubit**. Recall the Hadamard transformations:

$$H|+\rangle = |0\rangle, \quad H|-\rangle = |1\rangle$$

Therefore, if we can determine whether the data qubit is in the state $|+\rangle$ or $|-\rangle$, the Hadamard immediately converts this information into one of the computational basis states.

$$\begin{aligned} |\psi_3\rangle &= (H \otimes I) |\psi_2\rangle \\ &= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} H|0\rangle + (-1)^{f(1)} H|1\rangle \right) \otimes |-\rangle \\ &= \frac{1}{2} \left[(-1)^{f(0)} (|0\rangle + |1\rangle) + (-1)^{f(1)} (|0\rangle - |1\rangle) \right] \otimes |-\rangle \\ &= \frac{1}{2} \left[\left((-1)^{f(0)} + (-1)^{f(1)} \right) |0\rangle + \left((-1)^{f(0)} - (-1)^{f(1)} \right) |1\rangle \right] \otimes |-\rangle \end{aligned} \tag{98}$$

This is the **essence of quantum interference**: **the Hadamard combines the amplitudes of the two branches so that they either reinforce or cancel each other**. From the previous section, we know that the data qubit is in the state $|+\rangle$ if $f(0) = f(1)$ and in the state $|-\rangle$ if $f(0) \neq f(1)$. Therefore, after applying the final Hadamard gate, we have:

- **Constant Functions.** Every constant function leaves the data qubit in the state $\pm|+\rangle$. Applying the Hadamard gives:

$$H(\pm|+\rangle) = \pm|0\rangle$$

The minus sign is merely a global phase and therefore has no physical effect. Consequently, **if the function is constant, the final measurement of the data qubit will always yield $|0\rangle$** . The probability of measuring $|0\rangle$ is 100%, while the probability of measuring $|1\rangle$ is 0% (because the amplitude of $|1\rangle$ is zero). This is a **deterministic outcome** that allows us to conclude that the function is constant with certainty.

- **Balanced Functions.** Similarly, every balanced function leaves the data qubit in the state $\pm|-\rangle$. Applying the Hadamard gives:

$$H(\pm|-\rangle) = \pm|1\rangle$$

Again, the minus sign is a global phase and has no physical effect. Therefore, **if the function is balanced, the final measurement of the data qubit will always yield $|1\rangle$** . The probability of measuring $|1\rangle$ is 100%, while the probability of measuring $|0\rangle$ is 0% (because the amplitude of $|0\rangle$ is zero). This is also a **deterministic outcome** that allows us to conclude that the function is balanced with certainty.

🔍 Understanding the Interference. To understand how the Hadamard gate causes interference, recall the definition of interference on page 371. Although it is placed in the context of quantum annealing, it is not necessary to understand the details of that algorithm to understand interference. What is important are the two images in the example, which show a similar effect created by the Hadamard gate.

The Hadamard gate effectively adds and subtracts the amplitudes of the two computational basis states. Recall that:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

If the amplitudes have the **same sign**, as in:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

They interfere constructively in the first output component and destructively in the second:

$$H|+\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = |0\rangle$$

Conversely, if the amplitudes have opposite signs, as in:

$$|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

The interference occurs in the opposite way:

$$H|-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 0 \\ 2 \end{pmatrix} = |1\rangle$$

Thus, the **Hadamard converts the relative phase introduced by the oracle into a measurable computational-basis outcome**. This is why quantum interference is essential: without it, the phase information would remain inaccessible.

✓ Final Decision Rule

The algorithm now performs a **measurement** of the **data qubit**. The result is deterministic:

- Measuring $|0\rangle$ indicates that the function is constant.
- Measuring $|1\rangle$ indicates that the function is balanced.

Notice that the **computation qubit** is **never measured**. Its sole purpose was to enable phase kickback inside the oracle. After transferring the function values into the relative phases of the data qubit, it no longer carries any useful information for the algorithm.

In summary, the complete evolution of **Deutsch's Algorithm** can now be summarized as:

$$\begin{aligned}
 |0, 1\rangle &\xrightarrow{H \otimes H} |+, -\rangle \\
 &\xrightarrow{U_f} \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right) \otimes |-\rangle \\
 &\xrightarrow{H \otimes I} \begin{cases} |0, -\rangle & \text{if } f \text{ is constant} \\ |1, -\rangle & \text{if } f \text{ is balanced} \end{cases}
 \end{aligned} \tag{99}$$

And all the states:

$$\begin{aligned}
 |\psi_0\rangle &= |0, 1\rangle \\
 |\psi_1\rangle &= |+, -\rangle \\
 |\psi_2\rangle &= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right) \otimes |-\rangle \\
 |\psi_3\rangle &= \begin{cases} |0, -\rangle & \text{if } f \text{ is constant} \\ |1, -\rangle & \text{if } f \text{ is balanced} \end{cases} \quad (\text{up to a global phase})
 \end{aligned}$$

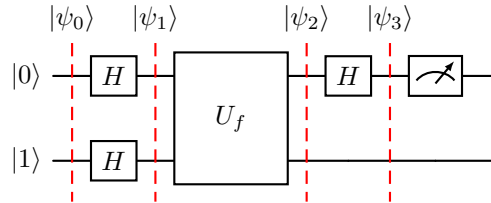


Figure 17: Deutsch's Algorithm. The final measurement of the data qubit reveals whether the function is constant or balanced.

 Example 36: $f(x) = 0$

Let's say we have a constant function $f(x) = 0$:

$$f(0) = f(1) = 0$$

Therefore, the state after the oracle is:

$$\begin{aligned} |\psi_2\rangle &= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right) \otimes |-\rangle \\ &= \frac{1}{\sqrt{2}} \left((-1)^0 |0\rangle + (-1)^0 |1\rangle \right) \otimes |-\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |-\rangle \\ &= |+, -\rangle \end{aligned}$$

Applying the final Hadamard gate gives:

$$|\psi_3\rangle = (H \otimes I) |\psi_2\rangle = (H \otimes I) |+, -\rangle = H |+\rangle \otimes |-\rangle = |0, -\rangle$$

Thus the measured data qubit is $|0\rangle$, indicating that the function is **constant**.

 Example 37: $f(x) = 1$

Let's say we have a constant function $f(x) = 1$:

$$f(0) = f(1) = 1$$

Therefore, the state after the oracle is:

$$\begin{aligned} |\psi_2\rangle &= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right) \otimes |-\rangle \\ &= \frac{1}{\sqrt{2}} \left((-1)^1 |0\rangle + (-1)^1 |1\rangle \right) \otimes |-\rangle \\ &= -\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes |-\rangle \\ &= -|+, -\rangle \end{aligned}$$

Applying the final Hadamard gate gives:

$$|\psi_3\rangle = (H \otimes I) |\psi_2\rangle = (H \otimes I) (-|+, -\rangle) = -H |+\rangle \otimes |-\rangle = -|0, -\rangle$$

Thus the measured data qubit is $|0\rangle$, indicating that the function is also **constant**. The minus sign is a global phase and has no physical effect on the measurement outcome.

 Example 38: $f(x) = x$

Let's say we have a balanced function $f(x) = x$:

$$f(0) = 0, \quad f(1) = 1$$

Therefore, the state after the oracle is:

$$\begin{aligned} |\psi_2\rangle &= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right) \otimes |-\rangle \\ &= \frac{1}{\sqrt{2}} \left((-1)^0 |0\rangle + (-1)^1 |1\rangle \right) \otimes |-\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes |-\rangle \\ &= |-, -\rangle \end{aligned}$$

Applying the final Hadamard gate gives:

$$|\psi_3\rangle = (H \otimes I) |\psi_2\rangle = (H \otimes I) |-, -\rangle = H |-\rangle \otimes |-\rangle = |1, -\rangle$$

Thus the measured data qubit is $|1\rangle$, indicating that the function is **balanced**.

 Example 39: $f(x) = \bar{x}$

Let's say we have a balanced function $f(x) = \bar{x}$:

$$f(0) = 1, \quad f(1) = 0$$

Therefore, the state after the oracle is:

$$\begin{aligned} |\psi_2\rangle &= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle + (-1)^{f(1)} |1\rangle \right) \otimes |-\rangle \\ &= \frac{1}{\sqrt{2}} \left((-1)^1 |0\rangle + (-1)^0 |1\rangle \right) \otimes |-\rangle \\ &= -\frac{1}{\sqrt{2}} (-|0\rangle + |1\rangle) \otimes |-\rangle \\ &= -|-, -\rangle \end{aligned}$$

Applying the final Hadamard gate gives:

$$|\psi_3\rangle = (H \otimes I) |\psi_2\rangle = (H \otimes I) (-|-, -\rangle) = -H |-\rangle \otimes |-\rangle = -|1, -\rangle$$

Thus the measured data qubit is $|1\rangle$, indicating that the function is also **balanced**. The minus sign is a global phase and has no physical effect on the measurement outcome.

4.15.7 Quantum Advantage and Importance

Deutsch's algorithm is the first example of a quantum algorithm that provably outperforms every deterministic classical algorithm for the same problem. Although the speedup is modest, it introduces several of the fundamental ideas that appear in many later quantum algorithms.

🔗 One Quantum Query vs Two Classical Queries

The goal is to determine whether an unknown Boolean function:

$$f : \{0, 1\} \rightarrow \{0, 1\}$$

Is **constant** or **balanced**.

❌ A deterministic classical algorithm must evaluate $f(0)$ and $f(1)$ (**two queries**) to determine whether f is constant or balanced. Because knowing only one output is never sufficient.

✅ Deutsch's algorithm instead prepares a superposition of both inputs:

$$\frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Queries the oracle **once**, and uses phase kickback together with quantum interference to determine whether:

$$f(0) = f(1) \quad (\text{constant})$$

Or:

$$f(0) \neq f(1) \quad (\text{balanced})$$

Thus, Deutsch's algorithm achieves a **quantum advantage** over classical algorithms by requiring only one query to the oracle instead of two.

❓ Why This Is a Query-Complexity Advantage

Notice that the algorithm does **not** reduce the number of elementary quantum gates. Preparing the state, applying Hadamard gates, and performing the measurement all require additional operations.

Instead, the improvement concerns the number of times the algorithm must access the **oracle** U_f . This is called the **query complexity** (or **oracle complexity**) of the problem. Many quantum algorithms are analyzed in exactly this way: ordinary gates are considered inexpensive; oracle evaluations represent the costly resource. Deutsch's algorithm demonstrates that quantum mechanics can reduce the number of oracle queries by exploiting superposition, phase kickback, and interference.

❓ Why the Algorithm Is Historically Important

The computational speedup itself is small:

$$2 \text{ classical queries} \quad \text{vs} \quad 1 \text{ quantum query}$$

Nevertheless, Deutsch's algorithm is historically significant because it is the **first algorithm to demonstrate a genuine quantum computational advantage**. More importantly, it introduces a design pattern that reappears throughout quantum computing:

1. Prepare a superposition of many inputs;
2. Encode information into relative phases using an oracle;
3. Use interference to amplify the desired information;
4. Measure only a global property of the function.

Instead of learning every function value individually, the **algorithm extracts only the property that is needed**.

🔗 Connection to Later Quantum Algorithms

Many of the best-known quantum algorithms follow exactly the same principles introduced by Deutsch's algorithm.

- **Deutsch-Jozsa Algorithm.** The Deutsch-Jozsa algorithm **generalizes the problem** from one input bit to n input bits:

$$f : \{0, 1\}^n \rightarrow \{0, 1\}$$

While maintaining the promise that the function is either constant or balanced. Remarkably, the quantum algorithm still requires only **one oracle query**, whereas a deterministic classical algorithm may require $2^{n-1} + 1$ queries in the worst case.

- **Bernstein-Vazirani Algorithm.** The Bernstein-Vazirani algorithm also relies on phase kickback and interference. Instead of determining whether a function is constant or balanced, its goal is to **determine an unknown n -bit string hidden inside the oracle**. A classical deterministic algorithm requires n oracle queries, while the quantum algorithm determines the entire hidden string using only **one oracle query**.

4.16 Bernstein-Vazirani's Algorithm

4.16.1 Problem Definition and Classical Solution

Bernstein-Vazirani's Algorithm, proposed in 1992 by Ethan Bernstein and Umesh Vazirani, is one of the first examples showing that a quantum computer can solve a problem with dramatically fewer oracle queries than any deterministic classical algorithm.

Like Deutsch's algorithm, it is formulated as an **oracle problem**. The oracle hides some information, and our goal is to discover it by querying the oracle as few times as possible.

🕒 The Hidden Bitstring (the goal)

Suppose there exists an unknown binary string:

$$w \in \{0, 1\}^n$$

Called the **hidden bitstring**. For example, if $n = 5$, a possible hidden bitstring could be:

$$w = 10110$$

The algorithm **knows nothing** about w . It can only obtain information by querying an oracle.

🔧 The Oracle Function

The oracle implements the Boolean function:

$$f_w(x) = w \cdot x \pmod{2} \tag{100}$$

Where:

- ✓ $x \in \{0, 1\}^n$ is an n -bit input string **chosen by us**.
- ❓ w is the **unknown hidden bitstring** (Bernstein-Vazirani's goal).
- $w \cdot x$ denotes the **bitwise inner product modulo 2**. The inner product module 2 is computed by:

$$w \cdot x = (w_1x_1) \oplus (w_2x_2) \oplus \cdots \oplus (w_nx_n)$$

Where multiplication is the ordinary logical AND, and $\oplus$ is the logical XOR. For example:

$$w = 101, \quad x = 110$$

Then:

$$w \cdot x = (1 \cdot 1) \oplus (0 \cdot 1) \oplus (1 \cdot 0) = 1 \oplus 0 \oplus 0 = 1 \oplus 0 = 1$$

Therefore:

$$f_w(110) = 1$$

Similarly, if $x = 011$, then:

$$w \cdot x = (1 \cdot 0) \oplus (0 \cdot 1) \oplus (1 \cdot 1) = 0 \oplus 0 \oplus 1 = 0 \oplus 1 = 1$$

Therefore:

$$f_w(011) = 1$$

🔗 Goal of the Algorithm and Classical Deterministic Solution

Our objective is **not** to evaluate the function for one particular input. Instead, the **goal is to determine the entire hidden string w using as few oracle queries as possible.** Once w is known, we can compute $f_w(x)$ for any input x ourselves, without querying the oracle again. This is the only information hidden inside the oracle.

⚠ Instead, a **deterministic classical algorithm must determine every bit of w .** The simplest strategy is to query the oracle with inputs containing a single 1. For example:

$$x = 100 \dots 0$$

Then:

$$f_w(100 \dots 0) = w_1$$

Likewise, querying the oracle with:

$$x = 010 \dots 0, \quad f_w(010 \dots 0) = w_2$$

And so on, until:

$$x = 000 \dots 1, \quad f_w(000 \dots 1) = w_n$$

Each query reveals exactly one bit of the hidden string. Therefore, **recovering all n bits requires exactly n oracle queries.** For example:

- Query 1: $f_w(1000) = 1$
- Query 2: $f_w(0100) = 0$
- Query 3: $f_w(0010) = 1$
- Query 4: $f_w(0001) = 1$

After 4 oracle calls, the algorithm reconstructs the hidden string:

$$w = 1011$$

In general, an **n -bit hidden string requires n queries to the oracle to be fully determined by a deterministic classical algorithm.**

❓ Why This Problem Is Interesting

At first glance, this problem appears impossible to accelerate: every oracle call seems to reveal only one bit of information about the hidden string. Surprisingly, a quantum computer can exploit **superposition** and **phase kickback** to extract **all n bits simultaneously**. The Bernstein-Vazirani algorithm determines the complete hidden bitstring using **only 1 oracle query**, providing a clear example of a **query-complexity advantage** over deterministic classical computation.

 Key Takeaways 1: Bernstein-Vazirani Problem

Bernstein-Vazirani problem: an oracle hides an unknown bitstring w through the function:

$$f_w(x) = w \cdot x \pmod{2}$$

The objective is to recover w . A deterministic classical algorithm requires n oracle queries (one per bit), whereas the quantum algorithm recovers the entire string with a single oracle query by exploiting superposition and phase kickback.

4.16.2 Quantum Oracle Construction

This section should answer one question: “*how do we build a quantum oracle that computes $f_w(x) = w \cdot x \bmod 2$?*”. The great thing is that the construction is extremely simple. For **each 1 in the hidden string, there is one CNOT gate** in the quantum circuit. Let's see how this works.

The Bernstein-Vazirani algorithm assumes access to a quantum oracle implementing the function:

$$f_w(x) = w \cdot x \bmod 2$$

As in Deutsch's algorithm, the oracle must be represented by a **unitary operator**, since every quantum computation must be reversible.

The oracle acts on **two quantum registers**:

- An **n -qubit data register** containing the input $|x\rangle$.
- A single **ancilla qubit** $|y\rangle$.

Its action is defined as follows:

$$U_{f_w} |x\rangle |y\rangle = |x\rangle |y \oplus f_w(x)\rangle$$

Notice that the **oracle never changes the input register**. It simply computes the function $f_w(x)$ and **XORs the result into the ancilla qubit** $|y\rangle$. This is exactly the same reversible construction used in Deutsch's algorithm, but now the input consists of n qubits instead of one.

Encoding the Hidden Bitstring

The hidden string (algorithm's goal):

$$w = (w_1, w_2, \dots, w_n)$$

Determines the oracle completely. Recall that the function $f_w(x)$ is defined as:

$$f_w(x) = w \cdot x \bmod 2 \implies w_1x_1 \oplus w_2x_2 \oplus \dots \oplus w_nx_n$$

Each term contributes to the final XOR only when the corresponding bit of w is equal to 1 (if $w_i = 0$, then $w_ix_i = 0$ regardless of the value of x_i). Therefore, the **oracle's action is determined entirely by the positions of the 1s in the hidden string**.

This observation leads to a remarkably **simple implementation**. For every position i :

- If $w_i = 1$, then insert a CNOT whose:
 - **Control** qubit is the **data qubit** $|q_i\rangle$ corresponding to the i -th bit of the input $|x\rangle$.
 - **Target** qubit is the **ancilla qubit** $|y\rangle$.
- If $w_i = 0$, then do nothing.

Therefore, **every 1 in the hidden string corresponds to exactly one CNOT gate**. The oracle is simply a collection of CNOT gates, one for each 1 in the hidden string.

❓ Why Does This Work?

Recall that a CNOT flips its target only when its control qubit is equal to 1. Suppose the ancilla initially contains $|y\rangle$. If a control qubit has value $x_i = 1$, the corresponding CNOT performs the operation:

$$y \rightarrow y \oplus 1$$

If instead the control qubit has value $x_i = 0$, the ancilla is left unchanged:

$$y \rightarrow y \oplus 0 = y$$

Applying all the required CNOT gates one after another accumulates the XOR of all selected input bits into the ancilla qubit. The final value of the ancilla is:

$$y \rightarrow y \oplus (w_1x_1) \oplus (w_2x_2) \oplus \dots \oplus (w_nx_n)$$

Which is exactly the desired result:

$$y \rightarrow y \oplus f_w(x)$$

Thus the oracle realizes the unitary transformation:

$$U_{f_w} |x\rangle |y\rangle = |x\rangle |y \oplus f_w(x)\rangle$$

For example, if the hidden string is $w = 10110$, the result is:

Position	w_i	Oracle Action
1	1	CNOT with control $ q_1\rangle$ and target $ y\rangle$
2	0	Do nothing
3	1	CNOT with control $ q_3\rangle$ and target $ y\rangle$
4	1	CNOT with control $ q_4\rangle$ and target $ y\rangle$
5	0	Do nothing

Without knowing w , we do not know which CNOTs are present inside the oracle. The **purpose of the Bernstein-Vazirani algorithm is precisely to discover this hidden pattern** using a single oracle query. When we will introduce the connection with phase kickback, it will become clear how this is possible.

🔗 Connection with Phase Kickback

At this point, the oracle may appear to do nothing more than compute a classical Boolean function. However, the crucial idea is that the ancilla will **not** be initialized in $|0\rangle$. Instead, it will be prepared in the state $|-\rangle$:

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Which is an eigenstate of the Pauli-X operator with eigenvalue -1 .

Because each CNOT applies an X gate to the ancilla whenever its control qubit is 1, the oracle no longer stores the value of $f_w(x)$ in the ancilla. Instead, it transfers that information into a **phase** of the data register through **phase kickback**.

This mechanism is the key to recovering the entire hidden bitstring with only one oracle query and will be examined in detail on page 260.

📌 Key Takeaways 2: Quantum Oracle Construction

The Bernstein-Vazirani oracle is constructed by placing **one CNOT for every position where the hidden bitstring has a 1**, using the corresponding data qubit as the control and the ancilla as the target. Acting on $|x\rangle|y\rangle$, the oracle computes:

$$U_{f_w} |x\rangle |y\rangle = |x\rangle |y \oplus f_w(x)\rangle$$

Where $f_w(x) = w \cdot x \bmod 2$. When the ancilla is later prepared in $|-\rangle$, these CNOTs produce phase kickback instead of storing the function value directly.

✏ Example 40: $w = 101$ Oracle Construction

In general, the hidden string w is unknown, and the oracle is treated as a black box. However, here we would like to illustrate the construction of the oracle for a specific example.

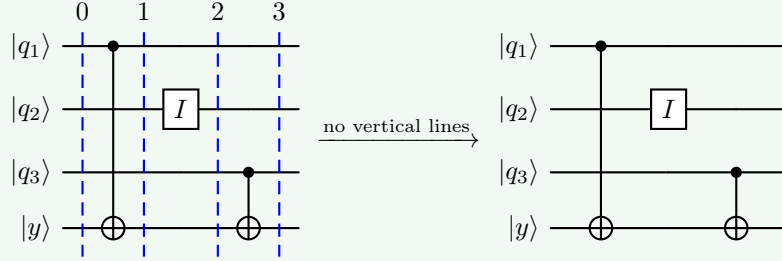
To illustrate how the oracle is constructed, let us consider the hidden bitstring:

$$w = 101$$

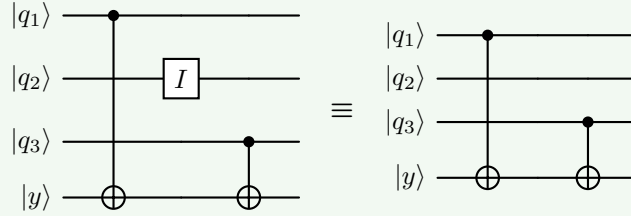
Since the first and third bits of w are equal to 1, while the second bit is 0, the oracle contains exactly **two CNOT gates**:

- One controlled by the first data qubit $|q_1\rangle$ and targeting the ancilla $|y\rangle$.
- One controlled by the third data qubit $|q_3\rangle$ and targeting the ancilla $|y\rangle$.
- The second data qubit $|q_2\rangle$ does not control any CNOT, since the second bit of w is equal to 0.

The resulting oracle circuit has the following structure:



The oracle acts on the computational basis according to Equation 103, page 262. Note that for completeness, the oracle also contains an identity gate on the second qubit, which is not connected to the ancilla, but we can omit it in the circuit diagram, since it does not affect the computation:



✂ **The Corresponding Boolean Function.** The hidden string $w = 101$ defines the function:

$$f_{101}(x_1, x_2, x_3) = (1 \cdot x_1) \oplus (0 \cdot x_2) \oplus (1 \cdot x_3)$$

Since multiplying by zero always gives zero, the expression simplifies to:

$$f_{101}(x_1, x_2, x_3) = x_1 \oplus x_3$$

And this tells us something very important: **the second input bit x_2 does not affect the output of the function at all.** This also explains why the oracle contains CNOT gates only on qubits q_1 and q_3 , while qubit q_2 is not connected to the ancilla (identity gate).

❓ **Input Testing.** We can see the action of the oracle on a few inputs.

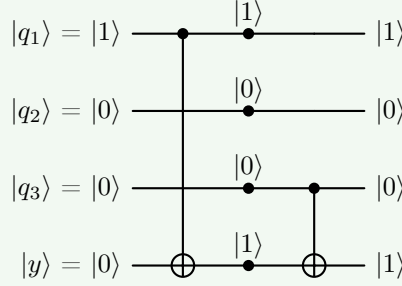
- $x = 100$. The function value (oracle output) is:

$$\begin{aligned} f_{101}(1, 0, 0) &= (1 \cdot 1) \oplus (0 \cdot 0) \oplus (1 \cdot 0) \\ &= 1 \oplus 0 \oplus 0 \\ &= 1 \end{aligned}$$

If the ancilla is initially $|0\rangle$, the oracle produces:

$$U_{f_{101}}(|100\rangle |0 \oplus f_{101}(1, 0, 0)\rangle) = U_{f_{101}}(|100\rangle |0 \oplus 1\rangle) = |100\rangle |1\rangle$$

Only the first CNOT is activated because only the first qubit is equal to 1. The second CNOT is not activated because the third qubit is equal to 0.



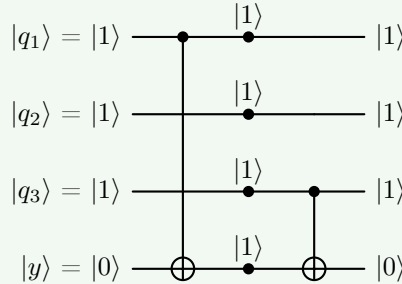
- $x = 111$. The function value (oracle output) is:

$$\begin{aligned} f_{101}(1, 1, 1) &= (1 \cdot 1) \oplus (0 \cdot 1) \oplus (1 \cdot 1) \\ &= 1 \oplus 0 \oplus 1 \\ &= 0 \end{aligned}$$

The first and third CNOT gates are both activated, but the ancilla is flipped twice, so it ends up in the same state as it started:

$$U_{f_{101}}(|111\rangle |0 \oplus f_{101}(1, 1, 1)\rangle) = U_{f_{101}}(|111\rangle |0 \oplus 0\rangle) = |111\rangle |0\rangle$$

Although two CNOT gates were applied, their effects cancel because XORing with 1 twice returns the original value.



What should we notice? This example illustrates a **general rule** for constructing Bernstein-Vazirani oracles. The **hidden string directly specifies which data qubits are connected to the ancilla:**

- Every bit $w_i = 1$ produces a CNOT;
- Every bit $w_i = 0$ produces no gate.

As a result, the oracle computes exactly the XOR of the input bits corresponding to the positions where w contains a 1. In this example, $w = 101$ means:

$$f_{101}(x_1, x_2, x_3) = x_1 \oplus x_3$$

The quantum algorithm will exploit this seemingly simple circuit in an unexpected way: instead of evaluating the function on a single input, it will query the oracle with a **superposition of all possible inputs simultaneously**, allowing the hidden bitstring to be recovered with a single oracle call. We will see how this works in the next section, where we will introduce the connection with phase kickback.

4.16.3 Circuit and Algorithm Steps

The Bernstein-Vazirani algorithm uses a circuit that is almost identical to Deutsch's algorithm (page 243). The main difference is that the single input qubit is replaced by an n -qubit register, allowing the algorithm to process all input bits simultaneously.

The circuit has the following structure:

$$\begin{array}{c} |0\rangle^{\otimes n} \xrightarrow{H^{\otimes n}} U_{f_w} \xrightarrow{H^{\otimes n}} \text{Measure} \\ |1\rangle \xrightarrow{H} \end{array}$$

The first n qubits form the **data register**, while the last qubit is an **ancilla** used during the oracle computation. As in Deutsch's algorithm, the ancilla is prepared in the state:

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Which enables phase kickback.

► Step 1: Initial State

The computation begins with all data qubits initialized to $|0\rangle$ and the ancilla qubit initialized to $|1\rangle$:

$$|\psi_0\rangle = |0\rangle^{\otimes n} \otimes |1\rangle \quad (101)$$

For example, when $n = 3$, the initial state is:

$$|\psi_0\rangle = |000\rangle |1\rangle = |0001\rangle$$

At this point, no information about the hidden string has yet been extracted.

✂ Step 2: First Hadamard Layer

Next, a Hadamard gate is applied to every qubit. The data register becomes an equal superposition of all 2^n possible input strings:

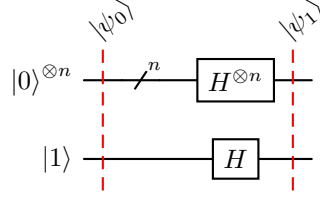
$$|0\rangle^{\otimes n} \longrightarrow \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)^{\otimes n} = |+\rangle^{\otimes n}$$

While the ancilla transforms as:

$$|1\rangle \longrightarrow \frac{|0\rangle - |1\rangle}{\sqrt{2}} = |-\rangle$$

The complete state is therefore:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |-\rangle \quad (102)$$



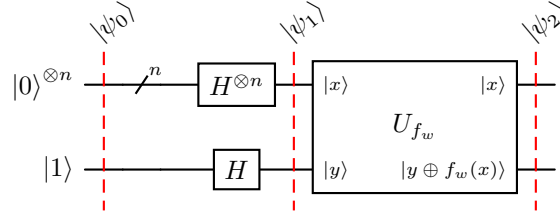
Conceptually, the oracle is now ready to evaluate **every possible input x simultaneously**. This quantum parallelism is the starting point of the algorithm's speedup.

✂ Step 3: Oracle Application

The oracle is applied once: U_{f_w} . Its action is (we will explain in detail on page 262):

$$U_{f_w} |x\rangle |-\rangle = (-1)^{f_w(x)} |x\rangle |-\rangle$$

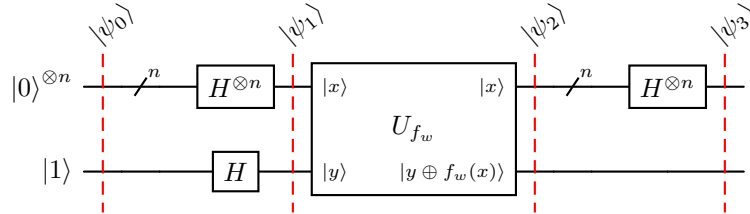
Rather than changing the ancilla, the oracle encodes the value of the Boolean function into the **phase** of each basis state of the superposition.



At this stage, the **hidden bitstring w** has not yet appeared explicitly in the data register. Instead, it is **encoded in the relative phases of the quantum state**. The reason why this happens will be explained in the next section through the phenomenon of **phase kickback**.

✂ Step 4: Final Hadamard Layer

A second layer of Hadamard gates is applied to the data register: $H^{\otimes n}$. This is the crucial **decoding step**.



The Hadamard transform **converts the phase** information introduced by the oracle **into amplitudes of the computational basis states**.

As a result, constructive **interference occurs only for the basis state corresponding to the hidden bitstring $|w\rangle$** , while all other basis states interfere destructively. The mathematical proof of this phenomenon is provided in the next sections.

Step 5: Measurement

Finally, the first n qubits are measured. Unlike many quantum algorithms that produce the correct answer only with high probability, the **Bernstein-Vazirani algorithm is deterministic**.

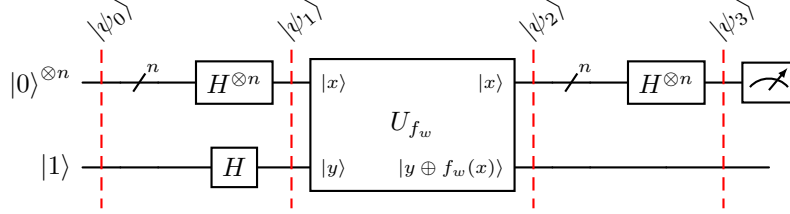


Figure 18: Circuit for the Bernstein-Vazirani algorithm.

The **measurement always returns w** , so the hidden bitstring is recovered after a **single execution of the circuit**. The ancilla qubit is irrelevant at this point and may be ignored.

Why is the Oracle written this way? A quantum gate must be reversible (unitary), the oracle cannot directly use the input x to produce the output $f_w(x)$. Instead, an additional qubit (the ancilla) must be used to store the result of the computation. The oracle is therefore defined as:

$$U_{f_w} |x\rangle |y\rangle = |x\rangle |y \oplus f_w(x)\rangle$$

The input $|x\rangle$ is **preserved** in the output, while the ancilla qubit $|y\rangle$ changes. Also, the **XOR operation $\oplus$ gives the oracle a reversible structure**, since applying the same oracle twice returns the original state:

$$U_{f_w} U_{f_w} |x\rangle |y\rangle = U_{f_w} |x\rangle |y \oplus f_w(x)\rangle = |x\rangle |y \oplus f_w(x) \oplus f_w(x)\rangle = |x\rangle |y\rangle$$

And this is important because **all quantum gates must be reversible**. Instead, suppose we tried to define the oracle as:

$$|x\rangle \mapsto |f(x)\rangle$$

Take a simple example $f(00) = 0$ and $f(11) = 0$. Then we would have:

$$|00\rangle \mapsto |0\rangle \quad \text{and} \quad |11\rangle \mapsto |0\rangle$$

Now imagine we observe the output state $|0\rangle$. Can we determine whether the input was $|00\rangle$ or $|11\rangle$? No, because the information about the input has been destroyed. This is a **non-reversible** operation, therefore the transformation cannot be unitary. Instead, the oracle introduces an additional qubit to store the output, preserving the input $|x\rangle$ in the output state. Again, XOR is used to ensure reversibility:

$$\begin{aligned} |x\rangle |y\rangle &\xrightarrow{U_f} |x\rangle |y \oplus f(x)\rangle \\ &\xrightarrow{U_f} |x\rangle |(y \oplus f(x)) \oplus f(x)\rangle \\ &= |x\rangle |y\rangle \end{aligned}$$

❓ **The oracle writes the function value into the ancilla. However, in Bernstein-Vazirani, we don't care about the ancilla.** Exactly. This is the key point. Initially it looks like the oracle is “*computing the function*” into the ancilla. However, the ancilla is prepared in the state $|-\rangle$, which is an eigenstate of the Pauli- X operator with eigenvalue -1 . Therefore, when the oracle applies the required CNOT operations, the ancilla remains in the state $|-\rangle$. **Instead of storing the function value in the ancilla, the oracle introduces a phase factor:**

$$(-1)^{f_w(x)}$$

Into the joint state:

$$|+\rangle |-\rangle \mapsto (-1)^{f_w(x)} |x\rangle |-\rangle$$

Since the ancilla is unchanged, this phase effectively becomes associated with the data register. This phenomenon is known as **phase kickback** and is the key idea behind the Bernstein-Vazirani algorithm. It will be explained in detail in the next section.

✂ Summary of the Algorithm

The entire procedure can be summarized as follows:

$|\psi_0\rangle$ Prepare the initial state:

$$|0\rangle^{\otimes n} \otimes |1\rangle$$

$|\psi_1\rangle$ Apply Hadamard gates to all qubits, creating a uniform superposition and preparing the ancilla in $|-\rangle$.

$|\psi_2\rangle$ Query the oracle once.

$|\psi_3\rangle$ Apply Hadamard gates to the data register.

4. Measure the data register to obtain the hidden bitstring w .

Remarkably, although the oracle is queried only once, the final measurement reveals all n bits of the hidden string simultaneously.

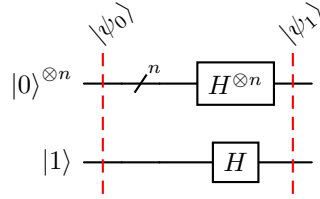
Now the missing piece of the puzzle is to understand **how the oracle converts the hidden string into phase information** and **why the final Hadamard gates decode those phases into the binary representation of w** . We will explore these questions in the next sections, starting with the phenomenon of **phase kickback**.

4.16.4 Phase-Kickback Intuition

The remarkable feature of the Bernstein-Vazirani algorithm is that the **oracle never explicitly writes the hidden bitstring w into the data register**. Instead, it hides that information in the **phases** of the quantum state. This is possible because the ancilla is prepared in a very special state.

The Ancilla is Prepared in $|-\rangle$

After the first Hadamard gate, the ancilla is no longer in the computational basis:



Instead, it is in the state:

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

This state is an eigenstate of the Pauli- X gate with eigenvalue -1 :

$$X|-\rangle = -|-\rangle$$

Therefore, whenever a CNOT attempts to flip the ancilla, the ancilla itself is unchanged (i.e., it remains in the state $|-\rangle$), but the control state acquires a minus sign. This phenomenon is called **phase kickback** and is exactly the same mechanism encountered in Deutsch's algorithm (page 239). Let's see in detail how this works in the Bernstein-Vazirani algorithm.

 **Oracle Action.** The oracle always satisfies:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

But notice something important: the **oracle does not know that the second register is in the state $|-\rangle$** . It simply receives *some quantum state* as the second qubit and applies XOR. Since the Bernstein-Vazirani algorithm prepares the second register (ancilla) in the state $|-\rangle$, the input to the oracle is actually:

$$|x\rangle |-\rangle = \frac{1}{\sqrt{2}} (|x0\rangle - |x1\rangle)$$

Now, since the quantum gates are **linear**, we can rewrite the oracle action as:

$$\begin{aligned} U_f |x\rangle |-\rangle &= U_f \left(\frac{1}{\sqrt{2}} (|x0\rangle - |x1\rangle) \right) \\ &= \frac{1}{\sqrt{2}} (U_f |x0\rangle - U_f |x1\rangle) \\ &= \frac{1}{\sqrt{2}} (U_f |x\rangle |0\rangle - U_f |x\rangle |1\rangle) \end{aligned}$$

At this point, we can apply the oracle definition to each term. Remember that, by definition, the oracle satisfies:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

Thus, if:

- $y = 0$, the expression becomes:

$$U_f |x\rangle |0\rangle = |x\rangle |0 \oplus f(x)\rangle$$

But XOR has a simple property:

y	$f(x)$	$y \oplus f(x)$	
0	0	0	$y = 0 \implies y \oplus f(x) = f(x) \quad \forall f(x)$
0	1	1	
1	0	1	
1	1	0	

Thus, we can rewrite the oracle action as:

$$U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle$$

- $y = 1$, the expression becomes:

$$U_f |x\rangle |1\rangle = |x\rangle |1 \oplus f(x)\rangle$$

Similarly, the XOR property is the same, but on the opposite side:

$$y = 1 \implies y \oplus f(x) = \overline{f(x)} \quad \forall f(x)$$

Thus, we can rewrite the oracle action as:

$$U_f |x\rangle |1\rangle = |x\rangle |\overline{f(x)}\rangle$$

Substituting these results back into the oracle action, we have:

$$\begin{aligned} U_f |x\rangle |-\rangle &= \frac{1}{\sqrt{2}} (|x\rangle |f(x)\rangle - |x\rangle |\overline{f(x)}\rangle) \\ &= \frac{1}{\sqrt{2}} (|x\rangle |f(x)\rangle - |x\rangle |1 \oplus f(x)\rangle) \end{aligned}$$

Leave $1 \oplus f(x)$ as it is for clarity. From this expression, we need to find a way to generalize it to all possible values (1 and 0) of $f(x)$:

- $f(x) = 0$:

$$\begin{aligned} U_f |x\rangle |-\rangle &= \frac{1}{\sqrt{2}} (|x\rangle |0\rangle - |x\rangle |1\rangle) \\ &= \frac{1}{\sqrt{2}} (|x0\rangle - |x1\rangle) \\ &= |x\rangle |-\rangle \end{aligned}$$

Nothing changes in the state of the system, and the **oracle does not introduce any phase change.**

- $f(x) = 1$:

$$\begin{aligned}
 U_f |x\rangle |-\rangle &= \frac{1}{\sqrt{2}} (|x\rangle |1\rangle - |x\rangle |0\rangle) \\
 &= -\frac{1}{\sqrt{2}} (|x\rangle |0\rangle - |x\rangle |1\rangle) \\
 &= -\frac{1}{\sqrt{2}} (|x0\rangle - |x1\rangle) \\
 &= -|x\rangle |-\rangle
 \end{aligned}$$

In this case, the oracle introduces a **phase change of -1** in the state of the system.

Combining these two cases, we need to **find a way to express the oracle action in a single expression that captures both cases**:

$$U_f |x\rangle |-\rangle = \begin{cases} |x\rangle |-\rangle & \text{if } f(x) = 0, \\ -|x\rangle |-\rangle & \text{if } f(x) = 1. \end{cases}$$

The simplest way to express this is:

$$U_f |x\rangle |-\rangle = (-1)^{f(x)} |x\rangle |-\rangle \quad (103)$$

For the skeptics, we can verify:

$$\begin{aligned}
 f(x) = 0 &\implies (-1)^{f(x)} = (-1)^0 = 1 \implies U_f |x\rangle |-\rangle = |x\rangle |-\rangle \\
 f(x) = 1 &\implies (-1)^{f(x)} = (-1)^1 = -1 \implies U_f |x\rangle |-\rangle = -|x\rangle |-\rangle
 \end{aligned}$$

Since we are very meticulous, we can expand $f(x)$ in the case of the Bernstein-Vazirani algorithm using the Equation 100 (page 248):

$$U_f |x\rangle |-\rangle = \underbrace{(-1)^{w \cdot x}}_{\text{where } f(x) = w \cdot x \pmod{2}} |x\rangle |-\rangle \quad (104)$$

Now, we can rewrite the Bernstein-Vazirani algorithm circuit (page page 258) with the explicit oracle action in mind:

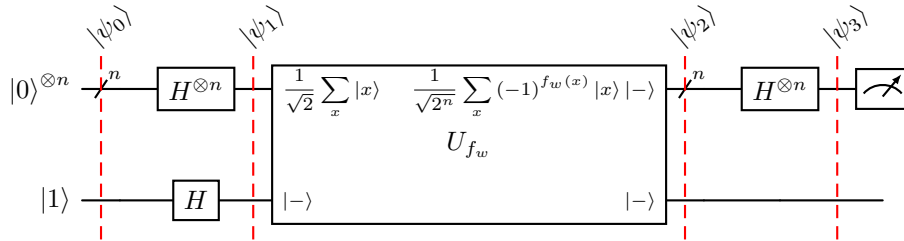


Figure 19: Bernstein-Vazirani algorithm circuit with the explicit oracle action. The oracle is applied to the superposition of all possible inputs, and it introduces a phase change of $(-1)^{f_w(x)}$ to each basis state $|x\rangle$ in the superposition.

Q Looking at One Bit at a Time

The Equation 104 explains the entire algorithm mathematically, but it does not immediately reveal why the hidden bitstring w eventually appears at the output. A much more intuitive way is to examine the oracle **bit by bit**.

Recall that the oracle contains one CNOT gate for every position where $w_i = 1$ (page 251), because the oracle computes the function:

$$f_w(x) = w \cdot x \mod 2 = w_0x_0 \oplus w_1x_1 \oplus \cdots \oplus w_{n-1}x_{n-1}$$

Now consider **what happens to a single data qubit after the first Hadamard layer**. Every data qubit starts in:

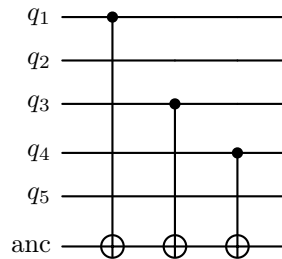
$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Then, there are two possibilities: $w_i = 0$ or $w_i = 1$.

❓ **Why analyze the cases $w_i = 0$ and $w_i = 1$ when they are the goal and we cannot see them?** Someone has already built the oracle. Suppose Nature secretly chooses a hidden bitstring:

$$w = 10110$$

The oracle is then physically built as:

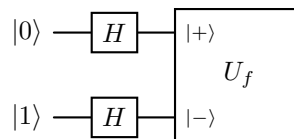


Notice that the oracle contains a CNOT gate for every position where $w_i = 1$. The algorithm **never** sees this circuit, it simply receives the oracle as a black box.

So we analyze the two cases $w_i = 0$ and $w_i = 1$ because **we**, as mathematicians, are **trying to prove that the algorithm works for every possible oracle**.

- Suppose the hidden bit at position i is $w_i = 0$. *What happens?* There is no CNOT gate in the oracle for this position, so the **input data qubit** x_i **remains unchanged**: $|+\rangle$.
- Now suppose instead that the hidden bit at position i is $w_i = 1$. *What happens?* **There is a CNOT gate** in the oracle for this position.

Let's suppose a **single data qubit** and one ancilla qubit.



Before the oracle, the state of the system is:

$$\begin{aligned} |\psi_1\rangle &= |+\rangle |-\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\ &= \frac{1}{2} (|00\rangle - |01\rangle + |10\rangle - |11\rangle) \end{aligned}$$

Now the oracle applies a CNOT gate:

$$\begin{aligned} \text{CNOT}(|+\rangle |-\rangle) &= \frac{1}{2} (|00\rangle - |01\rangle + |11\rangle - |10\rangle) \\ &= \frac{1}{2} (|00\rangle - |01\rangle - |10\rangle + |11\rangle) \end{aligned}$$

Now factor the expression:

$$\begin{aligned} \text{CNOT}(|+\rangle |-\rangle) &= \frac{1}{2} (|0\rangle (|0\rangle - |1\rangle) - |1\rangle (|0\rangle - |1\rangle)) \\ &= \frac{1}{2} (|0\rangle - |1\rangle) (|0\rangle - |1\rangle) \\ &= |-\rangle |-\rangle \end{aligned}$$

Therefore, each bit of the hidden bitstring w determines whether the corresponding qubit ends up in the state $|+\rangle$ or $|-\rangle$ after the oracle.

$$x_i = \begin{cases} |+\rangle & \text{if } w_i = 0, \\ |-\rangle & \text{if } w_i = 1. \end{cases} \quad (105)$$

If $w_i = 0$, the qubit remains in $|+\rangle$; if $w_i = 1$, it flips to $|-\rangle$. This is the **essence of phase kickback**: the oracle encodes the hidden bitstring into the phases of the quantum state without ever explicitly revealing it in the data register.

For example, if the hidden bitstring is $w = 10110$, corresponds to:

$$|-\rangle \otimes |+\rangle \otimes |-\rangle \otimes |-\rangle \otimes |+\rangle$$

The oracle has effectively transformed each hidden bit into a different quantum state of the corresponding data qubit.

The Final Hadamard Decodes the Answer

The last Hadamard layer **converts these phase states back into computational basis states**. Recall the identities:

$$\begin{aligned} H|+\rangle &= |0\rangle \\ H|-\rangle &= |1\rangle \end{aligned}$$

Thus, after the final Hadamard layer, the data qubits will be in the state

$$|w_0\rangle |w_1\rangle |w_2\rangle |w_3\rangle |w_4\rangle = |10110\rangle$$

✂ The Big Picture

The Bernstein-Vazirani algorithm can now be summarized as a simple three-step process:

1. The first Hadamard gates prepare every data qubit in the state $|+\rangle$.
2. The oracle uses phase kickback to change selected qubits into the state $|-\rangle$.
3. The final Hadamard gates convert:

$$|+\rangle \rightarrow |0\rangle \quad \text{and} \quad |-\rangle \rightarrow |1\rangle$$

Revealing the hidden bitstring directly in the computational basis.

This perspective explains why the algorithm succeeds with certainty: the oracle does not merely evaluate a Boolean function; it **encodes each hidden bit into the phase of its corresponding data qubit**, and the final Hadamard layer converts those phases back into ordinary bits.

✓ Key Takeaways 3: Phase Kickback Intuition

The ancilla is prepared in the state $|-\rangle$, so each CNOT in the oracle produces phase kickback instead of flipping the ancilla. As a result:

$$|x\rangle |-\rangle \mapsto (-1)^{f_w(x)=w \cdot x} |x\rangle |-\rangle$$

Meaning that each hidden bit is encoded as a phase on the corresponding data qubit:

$$w_i = 0 \implies x_i = |+\rangle \quad \text{and} \quad w_i = 1 \implies x_i = |-\rangle$$

The final Hadamard layer then decodes these phase states using:

$$H |+\rangle = |0\rangle \quad \text{and} \quad H |-\rangle = |1\rangle$$

Deterministically recovering the hidden bitstring w in the computational basis.

4.16.5 Complete Example for $w = 101$ (Exam Question)

Having understood the construction of the oracle (page 251) and the role of phase kickback (page 260), we can now **execute the entire Bernstein-Vazirani algorithm** for the hidden bitstring:

$$w = 101$$

Rather than focusing on the mathematical derivation, we simply follow the quantum state after each layer of the circuit.

Deepening: Quantum Computing Exam Question

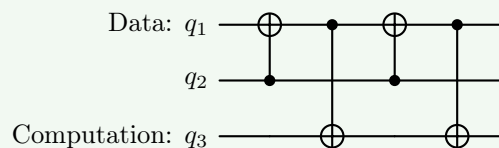
 Note that this is one of the **exam questions for the course**, so it is important to understand and be able to reproduce the steps. The professor will generally ask us to draw the circuit for the Bernstein-Vazirani algorithm and then execute it for a specific hidden bit string w . Another common option is to provide the oracle circuit directly and ask us to draw the complete Bernstein-Vazirani algorithm circuit and execute it for the specific hidden bitstring w obtained from the oracle circuit.

The following is an example of such a question, taken from the June 12, 2026 exam. We will not provide the solution here, but we will go through a complete, worked example for $w = 101$ in the following sections. Understanding the $w = 101$ example will enable us to answer exam questions involving any hidden bitstring w , as with other exam questions concerning the Bernstein-Vazirani algorithm.

Example 41: Exercise taken from the 2026 exam, June 12th

Bernstein-Vazirani Algorithm (8 points) exercise.

- Briefly describe the problem solved by the Bernstein-Vazirani algorithm and draw the circuit of the algorithm in its general form.
- Consider then the oracle U_f implemented by the quantum circuit:



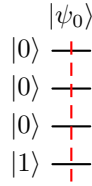
- Draw the full circuit of the Bernstein-Vazirani algorithm applied to the oracle U_f .
- Apply the Bernstein-Vazirani algorithm to the oracle U_f and determine the hidden bitstring encoded in the oracle.

► Step 1: Initial State

The computation starts with three data qubits initialized to $|0\rangle$ and one ancilla qubit initialized to $|1\rangle$:

$$|\psi_0\rangle = |000\rangle |1\rangle$$

The data register (i.e., the first three qubits) contains no information about the hidden bitstring w .



► Step 2: First Hadamard Layer

Hadamard gates are applied to every qubit. The data register becomes:

$$|0\rangle \rightarrow |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

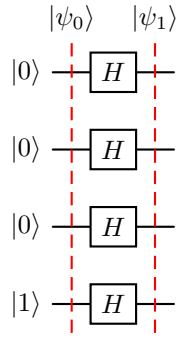
While the ancilla qubit becomes:

$$|1\rangle \rightarrow |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

The quantum state is therefore:

$$|\psi_1\rangle = |+, +, +\rangle |-\rangle$$

At this point, **every data qubit is in an equal superposition**, and the **ancilla is prepared for phase kickback**.



✂ Step 3: Oracle Application

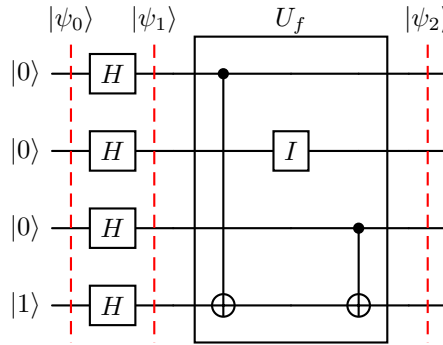
The hidden string is:

$$w = 101$$

So the oracle contains two CNOT gates:

$$f_w(x) = (1 \cdot x_1) \oplus (0 \cdot x_2) \oplus (1 \cdot x_3) = x_1 \oplus x_3$$

One controlled by the first qubit x_1 and one controlled by the third qubit x_3 . Because the ancilla is in the state $|-\rangle$, these CNOTs do not flip the ancilla. Instead, they produce **phase kickback**.



Without providing the full mathematical derivation as we did on page 260, we can take some shortcuts. A data qubit in the state $|+\rangle$ becomes $|-\rangle$ if it is a control for a CNOT gate and remains $|+\rangle$ if it is not. Thus, the first and third qubits become $|-\rangle$, while the second qubit remains $|+\rangle$. Due to phase kickback, the ancilla remains in the state $|-\rangle$.

- The first qubit changes from $|+\rangle$ to $|-\rangle$ because it is a control for a CNOT gate.
- The second qubit remains in the state $|+\rangle$ because it is not a control for any CNOT gate.
- The third qubit changes from $|+\rangle$ to $|-\rangle$ because it is a control for a CNOT gate.
- The ancilla remains in the state $|-\rangle$ due to phase kickback

The resulting quantum state is therefore:

$$|\psi_2\rangle = |-, +, -\rangle |-\rangle$$

Notice that the pattern of plus and minus states now exactly matches the hidden string:

$$w = 101 \iff (|-\rangle, |+\rangle, |-\rangle)$$

The oracle has encoded the hidden bits into the phases of the data qubits.

✂ Step 4: Final Hadamard Layer

The final Hadamard gates convert the phase states back into computational basis states. Using:

$$H|+\rangle = |0\rangle, \quad H|-\rangle = |1\rangle$$

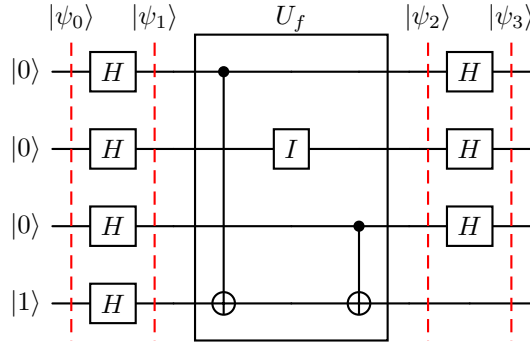
We obtain:

$$|-\rangle \rightarrow |1\rangle, \quad |+\rangle \rightarrow |0\rangle, \quad |-\rangle \rightarrow |1\rangle$$

Therefore:

$$|\psi_3\rangle = |101\rangle|-\rangle$$

The first three qubits now contain exactly the hidden bitstring $w = 101$, while the ancilla remains in the state $|-\rangle$.

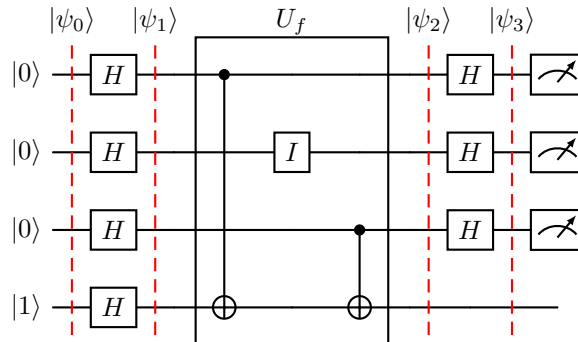


🔍 Step 5: Measurement

Finally, the data register is measured. Since its state is already $|101\rangle$, the measurement outcome is **deterministically** (Bernstein-Vazirani is a deterministic algorithm):

$$\text{Measurement outcome: } 101 \Leftarrow P(101) = 1$$

The ancilla is not measured because it contains no useful information about the solution.



⚠ What happens when the oracle contains gates that affect data qubits, but not ancillas?

Suppose the oracle provided by a *mysterious company* contains a quantum circuit that differs from the classic one we have seen. For instance, it **may contain a CNOT gate controlled by a data qubit that targets another data qubit**. How would we show each step of the Bernstein-Vazirani algorithm in this case? What should the state of the data qubits be after applying the oracle? These questions arise from a similar exercise that was on the **June 12, 2026 exam**.

In the previous sections, we always built the oracle starting from the hidden string:

$$f_w(x) = w \cdot x \pmod{2}$$

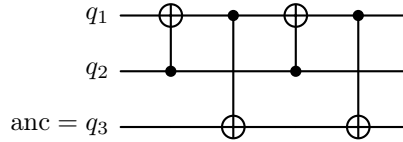
Which naturally leads to **CNOTs from the data qubits to the ancilla**, because the ancilla stores:

$$y \rightarrow y \oplus f_w(x)$$

That is only **one particular implementation of the oracle**. The oracle can be implemented in many different ways, as long as it satisfies the condition:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f_w(x)\rangle$$

Suppose the oracle circuit is:



Where:

- q_1 and q_2 are **data qubits** (data register);
- q_3 is the **computation (ancilla) qubit**.

Notice that there are **four gates**:

1. $\text{CNOT}_{q_2 \rightarrow q_1}$: A CNOT gate controlled by q_2 and targeting q_1 (data qubit);
2. $\text{CNOT}_{q_1 \rightarrow q_3}$: A CNOT gate controlled by q_1 and targeting the ancilla q_3 ;
3. $\text{CNOT}_{q_2 \rightarrow q_1}$: A CNOT gate controlled by q_2 and targeting q_1 (data qubit);
4. $\text{CNOT}_{q_1 \rightarrow q_3}$: A CNOT gate controlled by q_1 and targeting the ancilla q_3 .

The important thing is that the two CNOTs on the first line modify the **data register itself** before it is used to control the CNOTs that target the ancilla.

❓ **Why modify the data qubits?** Because the function may depend on a **transformed version** of the input. Instead of computing simply:

$$f(x) = x_1$$

The circuit may compute:

$$f(x_1 \oplus x_2, x_2)$$

Or some equivalent Boolean expression. The first pair of CNOTs changes the variables that are later used to control the ancilla. So the oracle is implementing a function that depends on a **transformed version of the input**, rather than the input itself:

$$U_f |x, y\rangle = |g(x)\rangle |y \oplus f(g(x))\rangle$$

Where $g(x)$ is the reversible transformation performed on the data qubits.

❓ **How do we recover the hidden string?** After observing the oracle circuit, the clever part begins. Unlike the previous example with naïve oracle circuits, where the hidden bitstring was decoded by simply counting the CNOTs connected to the ancilla, we must now **determine the Boolean function implemented by the circuit**.

1. **Initial data bits.** The data qubits are:

$$q_1 = x_1, \quad q_2 = x_2, \quad q_3 = y$$

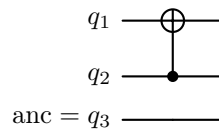
$$\begin{array}{l} q_1 \text{ ---} \\ q_2 \text{ ---} \\ \text{anc} = q_3 \text{ ---} \end{array}$$

2. **First CNOT.** The first CNOT gate is controlled by q_2 and targets q_1 . So we write the XOR operation:

$$\text{CNOT } q_2 \rightarrow q_1 \implies q_1 = x_1 \oplus x_2$$

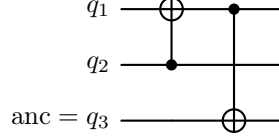
❓ **Why XOR?** Because the CNOT gate flips the target qubit if the control qubit is 1. To replicate this behavior, we use the logical XOR operation, which flips the target bit if the control bit is 1:

Control	Target	XOR Result
0	0	0
0	1	1
1	0	1
1	1	0



3. **First ancilla CNOT.** The next CNOT gate is controlled by q_1 and targets the ancilla q_3 . So the ancilla receives q_1 :

$$\text{CNOT } q_1 \rightarrow q_3 \implies q_3 = y \oplus q_1 = y \oplus (x_1 \oplus x_2)$$

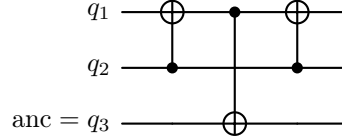


4. **Second CNOT.** The next CNOT gate is controlled by q_2 and targets q_1 , so it affects only the data qubits. The new value of q_1 is:

$$\begin{aligned} \text{CNOT } q_2 \rightarrow q_1 &\implies q_1 = (x_1 \oplus x_2) \oplus x_2 \\ &= x_1 \oplus x_2 \oplus x_2 \\ &\downarrow \text{XOR property: } a \oplus a = 0, \quad \forall a \in \{0, 1\} \\ &= x_1 \oplus 0 \\ &\downarrow \text{XOR property: } a \oplus 0 = a, \quad \forall a \in \{0, 1\} \\ &= x_1 \end{aligned}$$

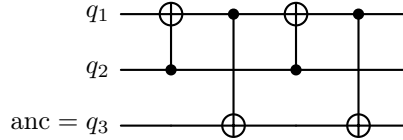
So the second CNOT **undoes the effect of the first CNOT**, and q_1 returns to its original value x_1 . This is a common reversible-computing trick: we can use a CNOT to modify a qubit and then use another CNOT to restore it to its original value. The **ancilla is unaffected** by this second CNOT, so it remains:

$$q_3 = y \oplus (x_1 \oplus x_2)$$



5. **Final ancilla CNOT.** The last CNOT gate is controlled by q_1 and targets the ancilla q_3 . So the ancilla receives q_1 :

$$\begin{aligned} \text{CNOT } q_1 \rightarrow q_3 &\implies q_3 = (y \oplus (x_1 \oplus x_2)) \oplus x_1 \\ &= y \oplus x_1 \oplus x_2 \oplus x_1 \\ &= y \oplus x_2 \end{aligned}$$



Therefore, the oracle implements the function:

$$f(x_1, x_2) = x_2$$

Which means that the hidden string is:

$$w = 01$$

 Key Takeaways 4

If the oracle contains gates that affect also the data qubits:

1. Do **not** try to read the hidden bitstring w directly from the CNOTs.
2. Trace the Boolean value carried by every data qubit after each gate in the oracle circuit.
3. Write the expression that reaches the ancilla.
4. Simplify the XOR expression.
5. Rewrite the result as:

$$f_w(x) = w \cdot x \pmod{2}$$

From which we immediately read the hidden bitstring w .

4.16.6 General Mathematical Derivation

The previous sections explained the Bernstein-Vazirani algorithm intuitively. We now prove mathematically that, for any hidden string:

$$w \in \{0, 1\}^n$$

The final state of the data register (i.e. the first n qubits) is exactly $|w\rangle$. The derivation is based on the n -qubit Hadamard identity:

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle \quad (106)$$

Where:

- $x \in \{0, 1\}^n$ is an n -bit string.
- $y \in \{0, 1\}^n$ is an n -bit string.
- $x \cdot y = x_1 y_1 \oplus \dots \oplus x_n y_n$ is the bitwise inner product of x and y , modulo 2.
- $(-1)^{x \cdot y}$ is a phase factor that is either $+1$ or -1 , depending on the value of $x \cdot y$.

► Initial State

The algorithm starts from:

$$|\psi_0\rangle = |0\rangle^{\otimes n} |1\rangle \quad (107)$$

The first n qubits form the **data register**, while the final qubit is the ancilla.

► After the First Hadamard Layer

We apply:

$$H^{\otimes n} H$$

Where the first n Hadamards act on the data register, and the final Hadamard acts on the ancilla. Thus, for the data register we have:

$$H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{0 \cdot x} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$$

Instead, for the ancilla we have:

$$H |1\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) = |-\rangle$$

Thus, the state after the first Hadamard layer is:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle |-\rangle \quad (108)$$

The **data register** is now in a **uniform superposition of all possible input strings**, while the ancilla is in the state $|-\rangle$, which is prepared for phase kickback.

✂ After the Oracle

The oracle satisfies:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

Since the ancilla is in the state $|-\rangle$, **phase kickback** gives (complete derivation on page 262):

$$U_f |x\rangle |-\rangle = (-1)^{f(x)=w \cdot x} |x\rangle |-\rangle$$

By linearity, the complete state after the oracle is:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{w \cdot x} |x\rangle |-\rangle \quad (109)$$

At this point, the **hidden string is encoded entirely in the phases of the data register.**

✂ Applying the Final Hadamard Layer

We now apply $H^{\otimes n}$ to the data register:

$$|\psi_3\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{w \cdot x} H^{\otimes n} |x\rangle |-\rangle$$

Using the parallel Hadamard identity:

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle$$

We obtain:

$$|\psi_3\rangle = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{y \in \{0,1\}^n} (-1)^{w \cdot x} (-1)^{x \cdot y} |y\rangle |-\rangle$$

Rearranging the sums:

$$|\psi_3\rangle = \frac{1}{2^n} \sum_{y \in \{0,1\}^n} \left(\sum_{x \in \{0,1\}^n} (-1)^{w \cdot x} (-1)^{x \cdot y} \right) |y\rangle |-\rangle$$

Thus, the amplitude of each basis state $|y\rangle$ is:

$$\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{w \cdot x} (-1)^{x \cdot y}$$

Because the exponents are computed modulo 2:

$$(-1)^a (-1)^b = (-1)^{a \oplus b}$$

Therefore, we can combine the exponents:

$$(-1)^{w \cdot x} (-1)^{x \cdot y} = (-1)^{w \cdot x \oplus x \cdot y}$$

The scalar product modulo 2 is linear, so we can factor out x :

$$w \cdot x \oplus x \cdot y = x \cdot (w \oplus y)$$

Thus, the state after the final Hadamard layer is:

$$|\psi_3\rangle = \frac{1}{2^n} \sum_{y \in \{0,1\}^n} \left(\sum_{x \in \{0,1\}^n} (-1)^{x \cdot (w \oplus y)} \right) |y\rangle |-\rangle \quad (110)$$

Everything now depends on the inner sum:

$$\sum_{x \in \{0,1\}^n} (-1)^{x \cdot (w \oplus y)}$$

✓ Measurement

At this point, there are two cases:

- If $w = y$, then $w \oplus y = 0^n$, therefore the inner sum is:

$$\sum_{x \in \{0,1\}^n} (-1)^{x \cdot 0^n} = \sum_{x \in \{0,1\}^n} 1 = 2^n$$

So it becomes a giant sum of 1 terms, until it reaches 2^n . The amplitude of $|w\rangle$ is therefore:

$$\frac{1}{2^n} \cdot 2^n = 1$$

Deepening: How did we measure the amplitude?

After the final Hadamard layer we have:

$$|\psi_3\rangle = \frac{1}{2^n} \sum_{y \in \{0,1\}^n} \left(\sum_{x \in \{0,1\}^n} (-1)^{x \cdot (w \oplus y)} \right) |y\rangle |-\rangle$$

This equation shows an interesting structure:

$$\begin{aligned} |\psi_3\rangle = & A(00 \dots 0) |00 \dots 0\rangle + \\ & A(00 \dots 1) |00 \dots 1\rangle + \\ & \dots + A(11 \dots 1) |11 \dots 1\rangle \end{aligned}$$

Where $A(y)$ is the amplitude of the basis state $|y\rangle$:

$$A(y) = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{x \cdot (w \oplus y)}$$

So **each basis state has its own amplitude**. Now, *what is the amplitude of the basis state $|y\rangle$?* There are 2^n possibilities, and instead of writing all of them, we compute them generically. If $y = w$, then $w \oplus y = 0^n$, and the amplitude of $|w\rangle$ is:

$$A(w) = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{x \cdot 0^n} = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} 1 = \frac{1}{2^n} \cdot 2^n = 1$$

- If $y \neq w$, then $w \oplus y \neq 0^n$, because at least one bit of $w \oplus y$ is equal to 1. As x ranges over all 2^n possible n -bitstrings, exactly half of the values satisfy $x \cdot (w \oplus y) = 0$ and the other half satisfy $x \cdot (w \oplus y) = 1$. Therefore, the inner sum contains equally many $+1$ and -1 terms:

$$\sum_{x \in \{0,1\}^n} (-1)^{x \cdot (w \oplus y)} = 2^{n-1} - 2^{n-1} = 0$$

The amplitude of every state $|y\rangle$ with $y \neq w$ **vanishes**. This is the mathematical expression of **destructive interference**.

Only the term with $y = w$ has nonzero amplitudes. Therefore:

$$|\psi_3\rangle = \frac{1}{2^n} (2^n |w\rangle) |-\rangle = |w\rangle |-\rangle$$

Thus:

$$|\psi_{\text{final}}\rangle = |w\rangle |-\rangle \quad (111)$$

The data register is exactly in the computational basis state corresponding to the hidden bitstring. Consequently, **measuring the data register will yield the hidden string w with probability 1.**

📌 Key Takeaways 5

After the oracle, the state is:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{w \cdot x} |x\rangle |-\rangle$$

Applying $H^{\otimes n}$ gives the amplitude of each basis state $|y\rangle$:

$$A(y) = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{x \cdot (w \oplus y)}$$

For $y = w$, the sum equals 2^n , so the amplitude is 1. For every $y \neq w$, the positive and negative terms cancel, so the amplitude is 0. Therefore:

$$|\psi_{\text{final}}\rangle = |w\rangle |-\rangle$$

And measurement recovers the hidden string w with probability 1 (deterministically).

4.16.7 Complexity and Importance

The Bernstein-Vazirani algorithm is mainly important because it gives a very **clear example of quantum query advantage**. The computational problem itself is simple:

$$f_w(x) = w \cdot x \pmod{2}$$

Where $w \in \{0,1\}^n$ is unknown. The goal is to recover the complete hidden bitstring w .

⚡ Classical Query Complexity

A deterministic classical algorithm can **recover one bit of w per query**. Using the standard basis vectors:

$$e_1 = (1, 0, 0, \dots, 0), \quad e_2 = (0, 1, 0, \dots, 0), \quad \dots, \quad e_n = (0, 0, 0, \dots, 1)$$

We obtain:

$$f_w(e_1) = w_1, \quad f_w(e_2) = w_2, \quad \dots, \quad f_w(e_n) = w_n$$

Therefore, recovering all n bits requires n oracle queries. Hence the classical query complexity is:

$$Q_{\text{classical}} = n \rightarrow \mathcal{O}(n)$$

This is an exact deterministic result: after n queries, the hidden bitstring is known with certainty.

✅ Quantum Query Complexity

The quantum algorithm prepares a superposition of all possible inputs, **queries the oracle once**, and uses phase kickback and interference to recover the entire string. After the final Hadamard layer, the state is:

$$|\psi_{\text{final}}\rangle = |w\rangle |-\rangle$$

Where the first n qubits are in the state $|w\rangle$, and the last qubit (ancilla) is in the state $|-\rangle$. Therefore, one measurement of the data register returns w with certainty (probability 1). Thus, the quantum query complexity is:

$$Q_{\text{quantum}} = 1 \rightarrow \mathcal{O}(1)$$

🚀 What Kind of Speedup is This?

The Bernstein-Vazirani algorithm gives an **n -to-1 reduction in oracle queries**. This is a real quantum advantage in the **query model**, but it is important to state it correctly.

It is not an exponential speedup in the ordinary running-time sense. The quantum circuit still uses:

- n data qubits;

- $n + 1$ Hadamard gates before the oracle;
- n Hadamard gates after the oracle;
- Up to n CNOTs inside the oracle;
- n measurements.

Therefore, the total gate cost still grows with n . The advantage is specifically that the expensive black-box function U_f is queried only once. So the correct statement is that the Bernstein-Vazirani algorithm provides a query-complexity advantage, not a constant running time advantage.

🔗 Relationship with Deutsch's Algorithm

Bernstein-Vazirani is closely related to Deutsch's algorithm. Both algorithms use:

- An ancilla initialized in $|1\rangle$.
- A Hadamard gate preparing the ancilla in $|-\rangle$.
- An oracle of the form:

$$|x\rangle |y\rangle \mapsto |x\rangle |y \oplus f(x)\rangle$$

- Phase kickback to encode the function value in the phase of the data register.
- A final Hadamard transformation to recover the hidden information.
- Deterministic measurement: the hidden information is recovered with probability 1.

The main difference is the problem being solved.

- Deutsch's algorithm uses one data qubit and determines whether a Boolean function is constant or balanced.
- Bernstein-Vazirani uses n data qubits and recovers an entire hidden bit-string:

$$w \in \{0, 1\}^n$$

Thus, Bernstein-Vazirani is a multi-qubit extension of the phase-kickback idea used in Deutsch's algorithm.

🔗 Relationship with Simon's Algorithm

Bernstein-Vazirani and Simon's algorithm are both hidden-structure problems, but they recover different kinds of information.

- **Bernstein-Vazirani.** The function is:

$$f_w(x) = w \cdot x \pmod{2}$$

The hidden object is a bitstring w , and the algorithm returns it directly with **one query**: $|w\rangle$.

- **Simon's algorithm.** As we will see in the next section, the function satisfies the promise:

$$f(x) = f(x \oplus a)$$

Where $a \neq 0$ is a hidden period. A single run does not return a . Instead, it returns a string y satisfying:

$$a \cdot y = 0 \pmod{2}$$

The **circuit must be repeated several times** to collect enough independent equations to reconstruct a .

Therefore, **Bernstein-Vazirani returns the hidden string directly; Simon returns constraints on the hidden string.** Bernstein-Vazirani is simpler and deterministic after one run, while Simon introduces the more advanced idea of recovering hidden structure from repeated inference constraints.

✓ Key Takeaways 6

The Bernstein-Vazirani problem hides a bitstring:

$$w \in \{0, 1\}^n$$

Inside the Boolean function:

$$f_w(x) = w \cdot x \pmod{2}$$

Classically, the bits of w are recovered one at a time, requiring n queries.

The quantum algorithm prepares:

$$|0\rangle^{\otimes n} |1\rangle$$

Applies Hadamard gates, and queries the oracle once. Because the ancilla is in $|-\rangle$, the oracle introduces the phase:

$$(-1)^{w \cdot x}$$

The final Hadamard layer decodes this phase pattern:

$$|+\rangle \mapsto |0\rangle, \quad |-\rangle \mapsto |1\rangle$$

The resulting state is:

$$|\psi_{\text{final}}\rangle = |w\rangle |-\rangle$$

So measuring the data register returns the hidden bitstring w with certainty (probability 1).

4.17 Simon's Algorithm

4.17.1 Problem Definition and Classical Solution

Simon's algorithm is another **hidden-bitstring problem**, so there is a similar structure with Bernstein-Vazirani (page 248). However, the way the hidden string is encoded is completely different. In Bernstein-Vazirani we had:

$$f_w(x) = w \cdot x \pmod{2}$$

And wanted to recover w by querying the oracle. In Simon's problem, the hidden bitstring instead represents an **XOR period** of a function.

The Problem

We are given a black-box function:

$$f : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

So both the input and output are n -bit strings. There exists an unknown, non-zero bitstring:

$$a \in \{0, 1\}^n, \quad a \neq 0^n$$

That we **want to discover**. The function is promised to satisfy:

$$f(x) = f(y) \iff x = y \quad \text{or} \quad x = y \oplus a \quad (112)$$

Where $\oplus$ is the bitwise XOR operation:

y	a	$y \oplus a$
0	0	0
0	1	1
1	0	1
1	1	0

This is **Simon's Promise**. The **goal** of Simon's algorithm is therefore simply to **find the hidden bitstring** a . But let's understand *what the promise actually means* before thinking about how to solve the problem.

What does Simon's promise mean?

Suppose $n = 2$. Then there are four possible inputs: 00, 01, 10, and 11. We **don't know how f was constructed**. We can only give it inputs and observe the outputs. For example, imagine this black box function:

x	$f(x)$
00	10
01	10
10	01
11	01

Notice that the outputs repeat:

$$f(00) = f(01) = 10 \quad \text{and} \quad f(10) = f(11) = 01$$

So there are two **collisions**: 00 and 01 collide, and 10 and 11 collide.

✓ **Simon tells us that these collisions aren't random.** The Simon's promise tells us that exists some secret nonzero string a such that whenever two **different** inputs have the same output, they differ exactly by a . So look at our first collision: $f(00) = f(01)$. The two inputs are 00 and 01. If we XOR them together, we get:

$$00 \oplus 01 = 01$$

Therefore the secret string a must be 01. Now check the other collision: $f(10) = f(11)$. Again XOR the inputs:

$$10 \oplus 11 = 01$$

Same result! So we have found the hidden string $a = 01$.

☐ **Another way to see it.** If the hidden string is $a = 01$, then Simon promises that every x has a "partner" $x \oplus a$ that collides with it. Start with $x = 00$:

$$00 \oplus 01 = 01$$

Therefore Simon's promise requires:

$$f(00) = f(01)$$

Let's continue with $x = 10$:

$$10 \oplus 01 = 11$$

So Simon's promise requires:

$$f(10) = f(11)$$

And that's exactly what we observed in the table above. So the promise is satisfied.

❓ **What if a were different?** Suppose instead that the hidden string was $a = 11$. Then the pairs would change. For $x = 00$:

$$00 \oplus 11 = 11$$

So Simon's promise would require:

$$f(00) = f(11)$$

And for $x = 01$:

$$01 \oplus 11 = 10$$

So Simon's promise would require:

$$f(01) = f(10)$$

But this is **not what we observed** in the table above. Therefore a cannot be 11. In fact, if we check all the other possibilities, we will find that $a = 01$ is the only one that satisfies Simon's promise.

Finally, suppose **we don't know** a . We query the mysterious black-box function and observe the outputs:

$$f(01) = 10 \quad \text{and} \quad f(00) = 10$$

We have found a collision! Simon's promise guarantees that two different inputs with the same output must differ by the hidden string a . So we can XOR the inputs together to find a :

$$01 \oplus 00 = 01$$

And we solved the problem.

✔ **If we find a collision, we immediately find a .** In the previous explanation, we found a collision by brute force. We queried the function for all possible inputs and observed the outputs. But this is **not efficient**! Suppose classically we query the function and happen to find two different inputs $x_1 \neq x_2$ that collide:

$$f(x_1) = f(x_2)$$

Simon's promise guarantees that:

$$x_2 = x_1 \oplus a$$

But then we can isolate a to find a general formula for the hidden string:

$$\begin{aligned} x_2 &= x_1 \oplus a \\ x_2 \oplus x_1 &= x_1 \oplus a \oplus x_1 \\ &\downarrow \quad x_1 \oplus x_1 = 0 \\ x_2 \oplus x_1 &= a \end{aligned}$$

And we obtain the hidden string a :

$$a = x_1 \oplus x_2 \tag{113}$$

And this is extremely important, because suppose we query the oracle and eventually discover a collision:

$$f(001) = f(110)$$

Then immediately we can find the hidden string:

$$a = 001 \oplus 110 = 111$$

So the classical problem can be rephrased as: *how many queries do we need before we find two different inputs producing the same output?*

= The equivalent formulation

The promise can therefore be written more intuitively as:

$$f(x) = f(x \oplus a), \quad \forall x \in \{0, 1\}^n \quad (114)$$

Take an arbitrary x . Its “partner” is $y = x \oplus a$. By Simon's promise:

$$f(x) = f(y), \quad \Rightarrow \quad f(x) = f(x \oplus a)$$

This is why a is often called the **hidden XOR period** (i.e., because f repeats itself every a). It plays a role analogous to an ordinary period. For example, for a familiar periodic function we might have: $f(x) = f(x + r)$ for some period r .

? Why is the function 2-to-1?

A function is called **2-to-1** if every output has exactly two inputs that map to it. For example:

$$f(00) = 10 \quad \text{and} \quad f(01) = 10$$

Here, **two inputs** 00 and 01 produce **one output** 10. Now, Simon's promise guarantees that this happens systematically. Suppose $a = 01$ and take $x = 00$. Its partner is:

$$x \oplus a = 00 \oplus 01 = 01$$

And Simon's promise says:

$$f(00) = f(01)$$

So that's one pair of inputs producing the same output: $\{00, 01\}$. Now take $x = 10$. Its partner is:

$$x \oplus a = 10 \oplus 01 = 11$$

And Simon's promise says:

$$f(10) = f(11)$$

So that's another pair of inputs producing the same output: $\{10, 11\}$. Therefore, for $n = 2$, we might have the following mapping:

Inputs	Output
00, 01	10
10, 11	01

There are 4 inputs but only 2 distinct outputs. So the function is 2-to-1.

In general, there are 2^n possible n -bit inputs. Since we group them **two at a time**:

$$\frac{2^n}{2} = 2^{n-1} \quad (115)$$

There are 2^{n-1} distinct outputs. For example, for $n = 2$ we have $2^2 = 4$ inputs, which are grouped into $2^{2-1} = 2$ pairs.

⚡ Deterministic classical solution

We don't know a , but we know that if we ever find a collision:

$$f(x_1) = f(x_2), \quad x_1 \neq x_2$$

Then we're finished:

$$a = x_1 \oplus x_2$$

So, *how many oracle queries do we need to find a collision?*

Suppose $n = 3$. There are $2^3 = 8$ possible inputs. Because Simon's function is 2-to-1, those 8 inputs are grouped into $2^{3-1} = 4$ pairs of hidden pairs. For example, perhaps $a = 001$. Then the pairs are:

$$\{000, 001\}, \quad \{010, 011\}, \quad \{100, 101\}, \quad \{110, 111\}$$

Each pair produces one common output. Now imagine we're extremely unlucky (**worst-case scenario**). We query the oracle and get 000, then 010, then 100, and then 110. We've made **4 queries**, but notice what happened: we selected exactly **one member from each pair**. Therefore we haven't seen a collision yet. But now we've exhausted all 4 pairs. *What happens with query number 5?* Whatever remaining input we choose, it must belong to one of the pairs we've already touched.

For example, if we choose 101, we already queried its partner 100:

$$f(100) = f(101)$$

Collision! Therefore for $n = 3$, we need at most $2^{3-1} + 1 = 5$ queries to find a collision.

In general, with n bits there are 2^n inputs. Because they come in pairs, there are 2^{n-1} pairs. In the **worst possible case**, we first query one member of each pair 2^{n-1} times, and we still haven't found a collision. But the next query must be a member of one of the pairs we've already seen, so we will find a collision. Therefore, in the worst case, we need at most:

$$2^{n-1} + 1 = O(2^n) \tag{116}$$

💬 Remark: Why do we write $O(2^n)$?

Because Big-O ignores constant factors and lower-order constants. We have:

$$2^{n-1} + 1$$

But we can rewrite 2^{n-1} as:

$$2^{n-1} = \frac{2^n}{2}$$

Therefore, we can write:

$$2^{n-1} + 1 = \frac{1}{2} \cdot 2^n + 1$$

And in Big-O notation, we ignore constant factors and lower-order terms, so we write:

$$2^{n-1} + 1 = O(2^n)$$

For example:

n	$2^{n-1} + 1$
3	5
10	513
20	524'289
30	536'870'913

It grows exponentially with n .

❖ Randomized classical solution

The deterministic classical solution is **exponential**, but we can do better with a **randomized approach**. The deterministic result asks: *how many queries do we need to guarantee a collision even if we're maximally unlucky?* The answer is $2^{n-1} + 1, O(2^n)$. But now suppose we're willing to say: ***we're choose inputs randomly. How many queries before a collision becomes likely?*** This is a classic **birthday problem**.

The **Birthday Problem** is a famous problem in probability theory. There are 365 possible birthdays. We might initially think “*to have a good chance that two people share a birthday, perhaps we need around 365 people*”. But that's not true. With only **23 people**, the probability that at least two share a birthday is already above 50%.

Why? Because we're not comparing everyone against one particular person. We're comparing **every person with every other person**. With k people, the number of pairs of people is:

$$\binom{k}{2} = \frac{k(k-1)}{2} \approx \frac{k^2}{2}$$

So the number of possible comparisons grows roughly like k^2 . Therefore, the probability of a collision grows much faster than we might initially expect.

❓ **What plays the role of the 365 birthdays in Simon?** In Simon, the “birthday categories” (i.e., the number of distinct outputs) are the 2^{n-1} **input pairs** created by the hidden string a . Suppose we have 2^n possible inputs. Simon's promise partitions them into 2^{n-1} pairs:

$$\{x, x \oplus a\}$$

Both members of a pair give the same output:

$$f(x) = f(x \oplus a)$$

So finding a collision is equivalent to randomly selecting **both members of one of these hidden pairs**. Now suppose we randomly query k distinct inputs. Among those k inputs there are:

$$\binom{k}{2} = \frac{k(k-1)}{2} \approx \frac{k^2}{2}$$

Possible pairs of queried inputs. For any particular pair of random distinct inputs x_i, x_j , the chance that they happen to be Simon partners is roughly $\frac{1}{2^n}$. Because once x_i is chosen, there is exactly **one special other input** $x_i \oplus a$ among roughly 2^n possible inputs. Therefore, with approximately $\frac{k^2}{2}$ opportunities for a collision, the expected number of collisions is roughly:

$$\frac{k^2}{2} \cdot \frac{1}{2^n}$$

A collision becomes reasonably likely when this reaches order 1:

$$\frac{k^2}{2^{n+1}} \sim 1$$

Ignoring constant factors for complexity:

$$k^2 \sim 2^n$$

Therefore:

$$k \sim \sqrt{2^n} = 2^{n/2} \quad \Rightarrow \quad k = O(2^{n/2}) \quad (117)$$

In contrast to the deterministic classical solution, which requires $O(2^n)$ queries, the randomized classical solution “*only*” requires $O(2^{n/2})$ queries. This is a **quadratic speedup** over the deterministic approach.

 Example 42: $n = 10$

Suppose $n = 10$. Then there are $2^{10} = 1024$ possible inputs. Because Simon's function is 2-to-1, those 1024 inputs are grouped into $2^{10-1} = 512$ hidden pairs. If we randomly query about:

$$k = \sqrt{2^{10}} = \sqrt{1024} = 32$$

Inputs, those 32 inputs create roughly:

$$\binom{32}{2} = \frac{32 \cdot 31}{2} = 496$$

Different pairs of queried inputs. So despite querying only about 32 of the 1024 inputs, we already have **hundreds of chances** that one of our queried pairs happens to be a hidden Simon pair. That's the birthday-paradox effect. Therefore, with only about 32 random queries, we have a good chance of finding a collision and discovering the hidden string a .

✔ What Simon's quantum algorithm changes

Simon found a way to **avoid searching directly for a collision**. The quantum algorithm uses **only $O(n)$ oracle queries**. But there is an important difference from Bernstein-Vazirani. In **Bernstein-Vazirani**, we **queried** the oracle **once** and got the hidden string w immediately. Instead, in Simon's algorithm, **each execution gives us some bitstring y** satisfying:

$$a \cdot y = 0 \pmod{2}$$

That's one equation involving the unknown hidden string a . We need to repeat the algorithm $O(n)$ times until we have enough independent equations to solve for a . We'll derive *why the circuit produces exactly these equations* later, but for now let's just accept that this is what happens. The important point is that Simon provides an **exponential query-complexity advantage** over classical algorithms.

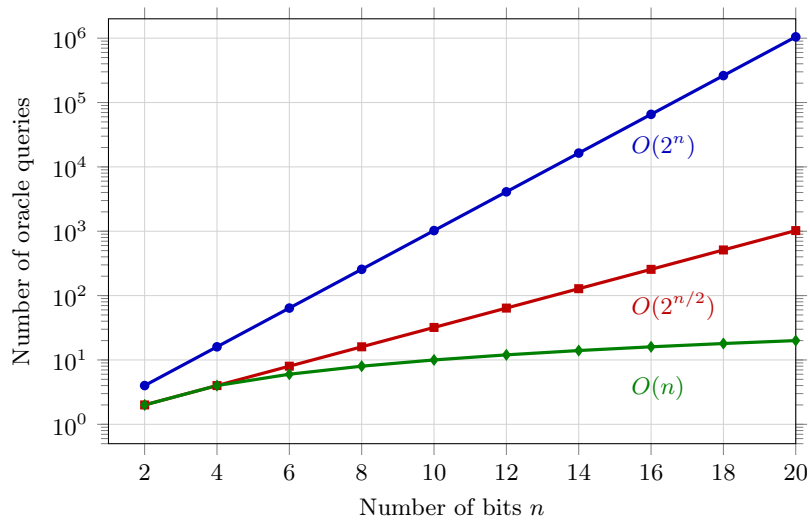


Figure 20: **Query-complexity scaling for Simon's problem.** A deterministic classical strategy requires exponentially many oracle queries, while randomized classical search reduces the complexity to approximately $O(2^{n/2})$ through the birthday-paradox effect. Simon's quantum algorithm requires only $O(n)$ oracle queries.

	Bernstein-Vazirani	Simon
Hidden string	w	$a \neq 0$
Structure	$f_w(x) = w \cdot x$	$f(x) = f(x \oplus a)$
Meaning	hidden coefficients	hidden XOR period
Classical	$O(n)$ queries	randomized $O(2^{n/2})$
Quantum	$O(1)$ queries	$O(n)$ queries
One run gives	w	y s.t. $a \cdot y = 0$
Post-processing	none	solve $O(n)$ equations for a

Table 3: **Comparison of Bernstein-Vazirani and Simon's.** It's interesting to note that **Simon uses more quantum queries than Bernstein-Vazirani**, but achieves a much stronger asymptotic separation (exponential vs. linear) between classical and quantum query complexity.

📌 Key Takeaways 7

Simon's algorithm considers a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ for which there exists an unknown non-zero bitstring a satisfying $f(x) = f(y)$ if and only if $x = y$ or $x = y \oplus a$. Therefore, $f(x) = f(x \oplus a)$, and a represents a hidden XOR period. The goal is to recover a . Classically, finding a collision requires $O(2^{n/2})$ randomized queries, whereas Simon's quantum algorithm requires $O(n)$ oracle queries.

Also, the classical algorithm **searches for such a collision**:

$$f(x_1) = f(x_2), \quad x_1 \neq x_2 \quad \Rightarrow \quad a = x_1 \oplus x_2$$

Simon's quantum algorithm, instead, will take a completely different route: it will extract **linear constraints on a** using interference.

4.17.2 Quantum Circuit and State Evolution

Now that the problem is clear, we can look at what **one execution of Simon's quantum circuit** actually does. The main conceptual difference from Bernstein-Vazirani is important from the beginning: one Simon run does **not** return a . Instead, one run will eventually give us a bitstring y containing **partial information about a** .

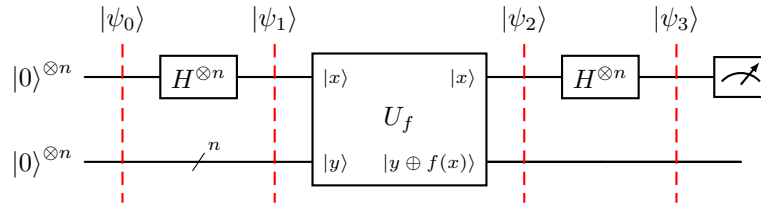


Figure 21: Quantum circuit for Simon's algorithm.

🔌 Step 1: Initial state

Simon uses **two registers**, each containing n qubits:

$$|\psi_0\rangle = |0\rangle^{\otimes n} \otimes |0\rangle^{\otimes n}$$

So there are $2n$ qubits in total. It is useful to give the registers different jobs:

$$\underbrace{|0\rangle^{\otimes n}}_{\text{input register}} \otimes \underbrace{|0\rangle^{\otimes n}}_{\text{output register}}$$

For example, if $n = 3$, the initial state is:

$$|\psi_0\rangle = \underbrace{|000\rangle}_{\text{input}} \otimes \underbrace{|000\rangle}_{\text{output}}$$

⚡ Step 2: Hadamards on the first register

We apply $H^{\otimes n}$ only to the **first** register. We move from a definite computational basis input into the **Hadamard basis**, expressed as a **superposition over all computational strings**. We already know that (from page 256) that:

$$H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$$

Therefore, after the Hadamards, the state of the system is:

$$|\psi_1\rangle = \underbrace{\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle}_{\text{input register}} \otimes \underbrace{|0\rangle^{\otimes n}}_{\text{output register}}$$

The first register is now an equal superposition of **every possible input**. For example, for $n = 2$:

$$\begin{aligned} |\psi_1\rangle &= \frac{1}{\sqrt{2^2}} \sum_{x \in \{0,1\}^2} |x\rangle \otimes |00\rangle \\ &= \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) \otimes |00\rangle \end{aligned}$$

So far, nothing about f or a has entered the computation. The first register is in a superposition of all possible inputs, and the second register is still in the initial state $|0\rangle^{\otimes n}$.

Step 3: Apply Simon's oracle

Classically, we think of a function as:

$$x \rightarrow f(x)$$

So we might initially want a quantum oracle that does:

$$|x\rangle \rightarrow |f(x)\rangle$$

The **problem** is that a quantum gate must be **unitary**, and therefore **reversible**. But a general function f may not be reversible. Simon is actually a perfect example because f is **2-to-1**:

$$f(x) = f(x \oplus a)$$

Meaning that two different inputs x and $x \oplus a$ map to the same output. For example, we could have:

$$f(00) = 10, \quad f(01) = 11$$

If our quantum transformation were:

$$|00\rangle \rightarrow |10\rangle, \quad |01\rangle \rightarrow |10\rangle$$

We've **lost the information** about whether the input was 00 or 01. We couldn't reverse the operation. So this cannot be a unitary quantum gate.

✓ **The standard quantum oracle.** To make an arbitrary classical function reversible, we keep x and introduce a second register:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

This is the oracle defined on page 230. The **first register is unchanged**:

$$|x\rangle \rightarrow |x\rangle$$

The **second register is modified**:

$$|y\rangle \rightarrow |y \oplus f(x)\rangle$$

So conceptually:

$$(x, y) \longrightarrow (x, y \oplus f(x))$$

This transformation is reversible because XOR is its own inverse (i.e., $y \oplus f(x) \oplus f(x) = y$).

🔗 Why do we initialize the second register to $|0\rangle^{\otimes n}$? Here's an interesting trick. If we initialize the second register to $y = 0^n$, then the oracle does:

$$U_f |x\rangle |0^n\rangle = |x\rangle |0^n \oplus f(x)\rangle$$

And since by definition of XOR:

$$0^n \oplus f(x) = f(x)$$

We can see that the second register is now in the state $|f(x)\rangle$:

$$U_f |x\rangle |0^n\rangle = |x\rangle |f(x)\rangle$$

So initializing the second register to zero makes the oracle behave as if it simply computes $f(x)$ into that register:

$$|x\rangle |0\rangle \rightarrow |x\rangle |f(x)\rangle$$

While still preserving x , and therefore preserving reversibility.

Example 43: Applying the oracle to a specific input

Suppose $x = 10$ and $f(10) = 01$. Apply the oracle to the state $|10\rangle |00\rangle$:

$$\begin{aligned} U_f |10\rangle |00\rangle &= |10\rangle |00 \oplus f(10)\rangle \\ &= |10\rangle |00 \oplus 01\rangle \\ &= |10\rangle |01\rangle \end{aligned}$$

The first register is unchanged, and the second register now contains $f(10) = 01$. And since XOR is its own inverse, we can reverse the operation:

$$\begin{aligned} U_f |10\rangle |01\rangle &= |10\rangle |01 \oplus f(10)\rangle \\ &= |10\rangle |01 \oplus 01\rangle \\ &= |10\rangle |00\rangle \end{aligned}$$

So the oracle is unitary and reversible, and it is a valid quantum gate.

🔗 Connect this to Simon's superposition. Before the oracle, Simon has the state:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |0\rangle^{\otimes n}$$

The important thing is that the oracle is linear. Therefore it acts on **every term of the superposition independently**. If we apply the oracle to $|\psi_1\rangle$,

we get:

$$\begin{aligned}
 |\psi_2\rangle &= U_f |\psi_1\rangle \\
 &= U_f \left(\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |0\rangle^{\otimes n} \right) \\
 &\quad \downarrow (a+b)c = ac + bc \\
 &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} U_f (|x\rangle \otimes |0\rangle^{\otimes n}) \\
 &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} \underbrace{|x\rangle}_{\text{input register}} \otimes \underbrace{|f(x)\rangle}_{\text{output register}}
 \end{aligned}$$

So the oracle leaves the **first register unchanged**, and the **second register now contains $f(x)$** for every possible input x in superposition:

$$U_f : \underbrace{|x\rangle}_{\text{untouched}} \otimes \underbrace{|0\rangle}_{\text{workspace}} \rightarrow \underbrace{|x\rangle}_{\text{untouched}} \otimes \underbrace{|f(x)\rangle}_{\text{output}}$$

Step 4: Final Hadamards

After the oracle, the state of the system is:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |f(x)\rangle$$

And therefore, the quantum state now contains the desired structure:

$$f(x) = f(x \oplus a)$$

For example, if $n = 2$ and $a = 01$, then the oracle has created the state:

$$|\psi_2\rangle = \frac{1}{2} (|00\rangle |f(00)\rangle + |01\rangle |f(01)\rangle + |10\rangle |f(10)\rangle + |11\rangle |f(11)\rangle)$$

So a is encoded in the fact that certain terms are paired. But there is an important problem: **we cannot simply measure this state and expect to obtain a** . If we measured the first register now, we'd just obtain some x (e.g., $x = 00$, or $x = 01$, or $x = 10$, or $x = 11$). One random x tells us essentially nothing about a .

Therefore, the idea is: *can we transform the state so that **wrong answers cancel out** and only outcomes containing information about a remain measurable?* That's exactly what **interference** gives us. The final Hadamards cause the amplitudes associated with the pair x and $x \oplus a$ to meet at the same possible output y . Then there are two possibilities:

✗ If their amplitudes have **opposite signs**:

$$+A + (-A) = 0$$

They **destructively interfere** and the probability of measuring that y is zero.

✓ If they have the **same sign**:

$$+A + (+A) = 2A$$

They **constructively interfere** and the probability of measuring that y is increased.

For each input x :

$$\begin{aligned} |\psi_3\rangle &= H^{\otimes n} |\psi_2\rangle \\ &= H^{\otimes n} \left(\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |f(x)\rangle \right) \\ &\downarrow (a+b)c = ac + bc \\ &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} H^{\otimes n} |x\rangle \otimes |f(x)\rangle \\ &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} \left(\frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle \otimes |f(x)\rangle \right) \\ &= \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle \otimes |f(x)\rangle \end{aligned}$$

Notice that **each x now contributes an amplitude to every possible y** . That's what allows interference. For a particular y , their contributions look like $(-1)^{x \cdot y}$, and $(-1)^{(x \oplus a) \cdot y}$. Depending on y , these two amplitudes either have the same sign (constructive interference) or opposite signs (destructive interference). And this inference removes exactly the y 's that do not satisfy:

$$a \cdot y = 0 \pmod{2}$$

We will see this in more detail in the next section.

💬 **Remark: Explain** $H|x\rangle = \frac{1}{\sqrt{2}} \sum_{y \in \{0,1\}} (-1)^{xy} |y\rangle$

We know that the Hadamard gate is defined as:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

And we can see that it acts on the computational basis states as:

$$\begin{aligned} H|0\rangle &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \\ H|1\rangle &= \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \end{aligned}$$

We can combine both equations into one formula. Let $x \in \{0, 1\}$, then:

$$H|x\rangle = \frac{1}{\sqrt{2}} \sum_{y \in \{0,1\}} (-1)^{xy} |y\rangle$$

Let's verify it:

- If $x = 0$, then $(-1)^{0 \cdot y} = (-1)^0 = 1$, $\forall y \in \{0, 1\}$, and we get:

$$H|0\rangle = \frac{1}{\sqrt{2}} \sum_{y \in \{0,1\}} 1 \cdot |y\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

- If $x = 1$, then $(-1)^{1 \cdot y} = (-1)^y$, and we get:

$$H|1\rangle = \frac{1}{\sqrt{2}} \sum_{y \in \{0,1\}} (-1)^y \cdot |y\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Now take two qubits. Suppose $x \in \{0, 1\}^2$, then we can write $x = x_1 x_2$ with $x_1, x_2 \in \{0, 1\}$. Then:

$$|x\rangle = |x_1 x_2\rangle = |x_1\rangle \otimes |x_2\rangle$$

Applying a Hadamard to both qubits means

$$H^{\otimes 2} |x_1 x_2\rangle = H|x_1\rangle \otimes H|x_2\rangle$$

Use the one-qubit formula on each:

$$H|x_1\rangle = \frac{1}{\sqrt{2}} \sum_{y_1} (-1)^{x_1 y_1} |y_1\rangle, \quad H|x_2\rangle = \frac{1}{\sqrt{2}} \sum_{y_2} (-1)^{x_2 y_2} |y_2\rangle$$

Tensor them:

$$H^{\otimes 2} |x_1 x_2\rangle = \left(\frac{1}{\sqrt{2}} \sum_{y_1} (-1)^{x_1 y_1} |y_1\rangle \right) \otimes \left(\frac{1}{\sqrt{2}} \sum_{y_2} (-1)^{x_2 y_2} |y_2\rangle \right)$$

Multiply the normalization factors:

$$\frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = \frac{1}{2}$$

Therefore

$$H^{\otimes 2} |x_1 x_2\rangle = \frac{1}{2} \sum_{y_1, y_2} (-1)^{x_1 y_1} (-1)^{x_2 y_2} |y_1 y_2\rangle$$

Now use

$$(-1)^a (-1)^b = (-1)^{a+b}$$

So

$$H^{\otimes 2} |x_1 x_2\rangle = \frac{1}{2} \sum_{y_1, y_2} (-1)^{x_1 y_1 + x_2 y_2} |y_1 y_2\rangle$$

But the exponent can be written as a **dot product**:

$$x \cdot y = x_1 y_1 + x_2 y_2 = \sum_{i=1}^2 x_i y_i$$

Therefore

$$H^{\otimes 2} |x_1 x_2\rangle = \frac{1}{2} \sum_{y_1, y_2} (-1)^{x \cdot y} |y_1 y_2\rangle = \frac{1}{2} \sum_{y \in \{0,1\}^2} (-1)^{x \cdot y} |y\rangle$$

 **Now take n qubits.** Suppose $x \in \{0,1\}^n$, then we can write:

$$x = x_1 \cdot x_2 \cdots x_n$$

Then:

$$|x\rangle = |x_1 x_2 \cdots x_n\rangle = |x_1\rangle \otimes |x_2\rangle \otimes \cdots \otimes |x_n\rangle$$

Applying Hadamard to every qubit means:

$$H^{\otimes n} |x\rangle = H |x_1\rangle \otimes H |x_2\rangle \otimes \cdots \otimes H |x_n\rangle$$

Each qubit contributes:

$$\frac{1}{\sqrt{2}} \sum_{y_i=0}^1 (-1)^{x_i y_i} |y_i\rangle$$

Therefore all n normalization factors multiply to:

$$\frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} \cdots \frac{1}{\sqrt{2}} = \frac{1}{\sqrt{2^n}}$$

And the signs multiply:

$$\begin{aligned} (-1)^{x_1 y_1} \cdot (-1)^{x_2 y_2} \cdots (-1)^{x_n y_n} &= (-1)^{x_1 y_1 + x_2 y_2 + \cdots + x_n y_n} \\ &= (-1)^{\sum_{i=1}^n x_i y_i} \\ &= (-1)^{x \cdot y} \end{aligned}$$

Remember that **in Simon we work modulo 2**, so the sum in the exponent is modulo 2 as well:

$$x \cdot y = \left(\sum_{i=1}^n x_i y_i \right) \bmod 2$$

Which is equivalent to writing:

$$x \cdot y = x_1 y_1 \oplus x_2 y_2 \oplus \cdots \oplus x_n y_n$$

For example, if $n = 2$:

$$x = 11, \quad y = 11$$

Ordinary dot product:

$$1 \cdot 1 + 1 \cdot 1 = 2$$

But modulo 2:

$$2 \bmod 2 = 0$$

Equivalently:

$$1 \oplus 1 = 0$$

Therefore in the Hadamard formula:

$$(-1)^{x \cdot y} = (-1)^0 = +1$$

Therefore we can write:

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle$$

Step 5: Measurement

We are interested in measuring the **first register** because it contains information about a . The measurement returns some:

$$y \in \{0,1\}^n$$

That y is guaranteed to satisfy:

$$a \cdot y = 0 \pmod{2} \tag{118}$$

This is **very different from Bernstein-Vazirani**. In BV, one run of the circuit returned a directly. In Simon, one run of the circuit returns a y that is **orthogonal to a** (i.e., $a \cdot y = 0$). So y is **not the hidden string**. It's a **constraint** on the hidden string.

For example, suppose $n = 3$ and we measure $y = 101$. Then we know that:

$$a \cdot y = 0 \quad \Rightarrow \quad a_1 \cdot (1) \oplus a_2 \cdot (0) \oplus a_3 \cdot (1) = 0$$

This is equivalent to:

$$a_1 \oplus a_3 = 0$$

That's one piece of information about a . If we run Simon again, we will get a different y (e.g., $y = 011$), which gives us another constraint:

$$a_2 \oplus a_3 = 0$$

After enough runs, we will have enough constraints to solve for a . The “*Recovering the Hidden Period a* ” section will explain how many runs are needed and how to solve for a using the collected constraints.

📌 Key Takeaways 8

The equation chain for Simon's algorithm is:

$$\begin{aligned}
 |\psi_0\rangle &= |0\rangle^{\otimes n} \otimes |0\rangle^{\otimes n} \\
 &\xrightarrow{H^{\otimes n} \otimes I^{\otimes n}} \\
 |\psi_1\rangle &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |0\rangle^{\otimes n} \\
 &\xrightarrow{U_f} \\
 |\psi_2\rangle &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle \otimes |f(x)\rangle \\
 &\xrightarrow{H^{\otimes n} \otimes I^{\otimes n}} \\
 |\psi_3\rangle &= \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle \otimes |f(x)\rangle
 \end{aligned}$$

And the result of measuring the first register is a y that satisfies:

$$y : a \cdot y = 0 \pmod{2}$$

4.17.3 Why Simon's Algorithm Works

In the previous section, we established the circuit and arrived at:

$$|\psi_3\rangle = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle |f(x)\rangle$$

We also said that the purpose of those Hadamards is to create **interference** between inputs related by the hidden period. Now we can prove exactly how that happens. Simon turns the general phenomenon of interference into a very precise filter.

✂ Reorganizing the Sum Using Simon's Promise

After the final Hadamards, the state is:

$$|\psi_3\rangle = \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle |f(x)\rangle$$

At first sight, there is no explicit term involving $x \oplus a$. However, the sum over x iterates over **every possible n -bit string**. Since $a \in \{0,1\}^n$, for every x in the sum:

$$x \oplus a \in \{0,1\}^n$$

Therefore, if the sum contains the contribution indexed by x :

$$(-1)^{x \cdot y} |y\rangle |f(x)\rangle$$

Then somewhere else in the same sum it necessarily also contains the contribution indexed by $x \oplus a$:

$$(-1)^{(x \oplus a) \cdot y} |y\rangle |f(x \oplus a)\rangle$$

The transformation $x \mapsto x \oplus a$ does not introduce new elements: it simply **permutes the elements of $\{0,1\}^n$** . Thus, we can reorganize the original sum into pairs x and $x \oplus a$. Now Simon's promise becomes useful. It guarantees that:

$$f(x) = f(x \oplus a)$$

Hence the two terms of each pair contributes to the **same final basis state** $|y\rangle |f(x)\rangle$. We can therefore combine their amplitudes:

$$\begin{aligned} & (-1)^{x \cdot y} |y\rangle |f(x)\rangle + (-1)^{(x \oplus a) \cdot y} |y\rangle |f(x \oplus a)\rangle \\ &= \left[(-1)^{x \cdot y} + (-1)^{(x \oplus a) \cdot y} \right] |y\rangle |f(x)\rangle \end{aligned}$$

Thus, the combined amplitude associated with a Simon pair is proportional to:

$$(-1)^{x \cdot y} + (-1)^{(x \oplus a) \cdot y}$$

Now the question is: *do these two amplitudes reinforce each other, or cancel each other?* That depends entirely on $a \cdot y$.

✔ **Simplify** $(x \oplus a) \cdot y$

We want to simplify the expression $(x \oplus a) \cdot y$ because our actual goal is **not** to understand x or y , but rather to **extract information about the unknown secret a** . And simplifying $(x \oplus a) \cdot y$ will allow us to see how a affects the relative sign of the two amplitudes.

At this point, we have obtained the combined amplitude:

$$(-1)^{x \cdot y} + (-1)^{(x \oplus a) \cdot y}$$

And we want to answer the question *for a given y , do these two amplitudes add or cancel?* Right now that's difficult to see because the second exponent mixes everything together: $(x \oplus a) \cdot y$. We want to expose explicitly **how a affects the relative sign** of the two terms.

So we rewrite the second exponent $(-1)^{(x \oplus a) \cdot y}$ using arithmetic over $\mathbb{Z}_2$ (i.e., modulo 2):

$$\begin{aligned} (x \oplus a) \cdot y &= \sum_{i=1}^n (x_i \oplus a_i) y_i \pmod{2} \\ &= \sum_{i=1}^n (x_i + a_i) y_i \pmod{2} \\ &= x \cdot y + a \cdot y \pmod{2} \end{aligned}$$

💬 **Remark: $\oplus$ vs $+$**

The equality:

$$x_i \oplus a_i \equiv x_i + a_i \pmod{2}$$

Holds in **arithmetic over $\mathbb{Z}_2$** . Indeed, let's check the truth table:

x_i	a_i	$x_i \oplus a_i$	$x_i + a_i \pmod{2}$
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	0

Thus, $1 \oplus 1 = 0$, while ordinary integer addition gives $1 + 1 = 2$. Reducing modulo 2 makes them equal.

Therefore, we can rewrite the combined amplitude as:

$$\begin{aligned} (-1)^{x \cdot y} + (-1)^{(x \oplus a) \cdot y} &= (-1)^{x \cdot y} + (-1)^{x \cdot y + a \cdot y} \\ &\downarrow \text{since } (-1)^{p+q} = (-1)^p (-1)^q \text{ for any integers } p, q \\ &= (-1)^{x \cdot y} + (-1)^{x \cdot y} (-1)^{a \cdot y} \\ &= (-1)^{x \cdot y} [1 + (-1)^{a \cdot y}] \end{aligned}$$

This is the expression we wanted! Because now something very interesting happens. Since we're working modulo 2, $a \cdot y$ can only be 0 or 1. Therefore, there are only **two cases**:

- $a \cdot y = 0$, then:

$$(-1)^{a \cdot y} = (-1)^0 = 1 \implies 1 + (-1)^{a \cdot y} = 1 + 1 = 2$$

Our combined amplitude becomes $2(-1)^{x \cdot y}$, which is **non-zero**:

$$(-1)^{x \cdot y} [1 + (-1)^{a \cdot y}] = (-1)^{x \cdot y} \cdot 2 = 2(-1)^{x \cdot y}$$

Therefore, the two contributions reinforce each other (**constructive interference**), and the corresponding y is potentially **measurable**.

- $a \cdot y = 1$, then:

$$(-1)^{a \cdot y} = (-1)^1 = -1 \implies 1 + (-1)^{a \cdot y} = 1 - 1 = 0$$

The entire combined amplitude becomes 0:

$$(-1)^{x \cdot y} [1 + (-1)^{a \cdot y}] = (-1)^{x \cdot y} \cdot 0 = 0$$

The two contributions cancel each other (**destructive interference**), and the corresponding y is **not measurable**.

We have therefore shown that:

- $a \cdot y = 0 \implies$ constructive interference.
- $a \cdot y = 1 \implies$ destructive interference.

And therefore:

$$P(y) > 0 \iff a \cdot y = 0 \pmod{2}$$

Or in words, **after the final Hadamards, Simon's interference removes every y that is not orthogonal to the hidden string a modulo 2.**

🔍 Why are all valid y 's equally likely? This is one another interesting point. The **interference condition doesn't prefer one valid y over another!** For every y satisfying constructive interference:

$$a \cdot y = 0 \pmod{2}$$

The factor:

$$1 + (-1)^{a \cdot y}$$

Has the same magnitude, 2. Therefore **all surviving y 's have equal probability.** For a non-zero a , exactly half of all 2^n bitstrings satisfy $a \cdot y = 0$, so there are 2^{n-1} valid y 's, each with probability:

$$P(y) = \frac{1}{2^{n-1}}$$

 Example 44: Constructive and Destructive Interference

We have $a = 01$. For $n = 2$, there are four possible measurement outcomes:

$$y \in \{00, 01, 10, 11\}$$

Let's calculate $a \cdot y$ for each of them:

- $y = 00$: $a \cdot y = 0 \cdot 0 + 1 \cdot 0 = 0$ (constructive interference)
- $y = 01$: $a \cdot y = 0 \cdot 0 + 1 \cdot 1 = 1$ (destructive interference)
- $y = 10$: $a \cdot y = 0 \cdot 1 + 1 \cdot 0 = 0$ (constructive interference)
- $y = 11$: $a \cdot y = 0 \cdot 1 + 1 \cdot 1 = 1$ (destructive interference)

Therefore, the only valid measurement outcomes are $y = 00$ and $y = 10$, each with probability:

$$P(y = 00) = P(y = 10) = \frac{1}{2^{2-1}} = \frac{1}{2}$$

 What did the quantum circuit actually accomplish?

Before the final Hadamards, the secret was encoded indirectly as:

$$f(x) = f(x \oplus a)$$

That isn't something we can conveniently read from a single measurement. The final Hadamards cause the paired inputs to interfere:

$$(-1)^{x \cdot y} + (-1)^{(x \oplus a) \cdot y} = (-1)^{x \cdot y} [1 + (-1)^{a \cdot y}]$$

The interference converts that hidden pairing into a completely different kind of information:

$$f(x) = f(x \oplus a) \xrightarrow{H^{\otimes n} + \text{inference}} a \cdot y = 0 \pmod{2}$$

So one execution of the quantum circuit effectively transforms **hidden periodic structure** into **one linear constraint on the hidden string**. However, **we still haven't found a** . We've only measured one y satisfying $a \cdot y = 0$. The natural next question is: *how many such y 's do we need, and how do we solve the measuring equations to reconstruct a ?* We'll answer that in the next section.

4.17.4 Recovering the Hidden Period

The previous section established that after the final Hadamards and measurement of the **first register**, we don't obtain a . Instead, we obtain some random bitstring y satisfying:

$$a \cdot y = 0 \pmod{2}$$

The question is now: *if the circuit doesn't give us a , how can we actually recover a ?* The answer is: **run the circuit multiple times and turn the measurements into a system of linear equations.**

⚠ One execution gives one constraint, not a

Suppose the first register has $n = 3$ qubits. Immediately before measurement, several valid y 's may still be in superposition:

$$|\psi\rangle = \cdots |000\rangle + \cdots |011\rangle + \cdots |101\rangle + \cdots |110\rangle + \cdots$$

Where all surviving y 's satisfy $a \cdot y = 0 \pmod{2}$. But **when we measure**, the superposition **collapses to one of them**. Let's say we measure $y = 110$. We don't get a list of all valid y 's, we get only that single classical bitstring. And that one bitstring gives us exactly the relationship:

$$a \cdot 110 = 0 \pmod{2}$$

Expanding:

$$a_1 \cdot 1 \oplus a_2 \cdot 1 \oplus a_3 \cdot 0 = 0 \pmod{2}$$

Because $a_3 \cdot 0 = 0$, we can simplify to:

$$a_1 \oplus a_2 = 0 \pmod{2}$$

That's **one equation**.

The important point is that the n measured bits $y_1 \dots y_n$ (e.g., 110) do **not** individually tell us n different things about a . They're all coefficients of **one dot-product equation**:

$$a_1 \cdot y_1 \oplus a_2 \cdot y_2 \oplus \cdots \oplus a_n \cdot y_n = 0 \pmod{2}$$

For example, measuring $y = 110$ does **not** mean that $a_1 = 0$, $a_2 = 0$, and $a_3 = 0$. Instead, 110 means "*whatever the secret a is, we know that it is orthogonal to 110 modulo 2*". In other words, a must satisfy the equation $a_1 \oplus a_2 = 0 \pmod{2}$.

🔄 **So we execute the circuit again.** Since only one run gives us one equation, we need to run the circuit multiple times. **Every execution starts again and produces another random valid y .**

For example, perhaps:

$$y^{(1)} = 110$$

And on another execution:

$$y^{(2)} = 011$$

Then we obtain two equations:

$$a \cdot 110 = 0 \pmod{2}, \quad a \cdot 011 = 0 \pmod{2}$$

Expanding:

$$a_1 \oplus a_2 = 0 \pmod{2}, \quad a_2 \oplus a_3 = 0 \pmod{2}$$

Remember that a_1 , a_2 , and a_3 are bits, so each can only be 0 or 1. Let's check every possibility:

a_1	a_2	$a_1 \oplus a_2$
0	0	0
0	1	1
1	0	1
1	1	0

So XOR produces 0 exactly when the two bits are equal. Therefore, the first equation $a_1 \oplus a_2 = 0$ is true if and only if $a_1 = a_2$. Similarly, the second equation $a_2 \oplus a_3 = 0$ is true if and only if $a_2 = a_3$. Combining these two equations, we have:

$$a_1 = a_2 = a_3$$

Since they're bits, there are only two ways all three can be equal:

- $a_1 = a_2 = a_3 = 0$, which means $a = 000$. But Simon's promise tells us that $a \neq 0$, so this is the **trivial case** that we discard.
- $a_1 = a_2 = a_3 = 1$, which means $a = 111$. This is the **non-trivial case** that we accept as the solution.

We've recovered the hidden period a by running the circuit multiple times (in this case, twice).

Writing everything as a matrix

Instead of writing many equations individually, we collect our measured y 's as rows of a matrix.

Suppose we measured:

$$y^{(1)}, y^{(2)}, \dots, y^{(k)}$$

We construct the matrix Y by stacking them as rows:

$$Y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(k)} \end{bmatrix}$$

Where each $y^{(i)}$ is a row vector of length n . The equations $a \cdot y^{(i)} = 0$ can then all be written together as a single matrix equation:

$$Ya = 0 \pmod{2} \tag{119}$$

For our previous example, if we measured $y^{(1)} = 110$ and $y^{(2)} = 011$, then:

$$Y = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \quad a = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

And the matrix equation $Ya = 0$ is equivalent to the two equations we derived earlier:

$$\begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \pmod{2}$$

Which gives us the same two equations:

$$\begin{cases} a_1 \oplus a_2 = 0 \\ a_2 \oplus a_3 = 0 \end{cases}$$

So the matrix equation $Ya = 0$ is just a **compact notation** for all the constraints collected from the quantum circuit measurements.

⚠ Why do the equations need to be linearly independent?

The linear independence of the equations is crucial because the circuit returns **random** valid y 's.

Imagine we run it 3 times and obtain: 110, 110, and 110 again. We technically have three measurements, but all three tell us exactly the same thing: $a_1 \oplus a_2 = 0$. We haven't gained three pieces of information, we've only gained **one**.

There are other subtle situations. Suppose two actual executions gave:

$$y^{(1)} = 110, \quad y^{(2)} = 011$$

They give the constraints:

$$a \cdot 110 = 0, \quad a \cdot 011 = 0$$

Now suppose we **really execute the quantum circuit a third time**, and the measurement happens to give $y^{(3)} = 101$. At first glance, 101 looks new. But notice:

$$110 \oplus 011 = 101$$

So the third vector is a linear combination of the first two. So **before even considering the third equation**, the first two equations already mathematically imply the third.

That's why we're interested not simply in the **number of measurements**, but in the number of **linearly independent measured y 's**.

🔍 **How many independent equations do we need?** There are n unknown bits:

$$a_1, a_2, \dots, a_n$$

At first we might think we need n independent equations. But there's something special here. Every equation is homogeneous (i.e., the right-hand side is 0):

$$a \cdot y^{(i)} = 0$$

Therefore, the trivial solution $a = 0^n$ is **always** a solution. But Simon explicitly promises that:

$$a \neq 0^n$$

What we actually want is enough constraints so that the solution space becomes **one-dimensional**, which means that there are only two solutions: the trivial $a = 0^n$ and the **non-trivial a that we want: $\{0^n, a\}$** . For a nonzero n -bit a , we therefore generally want **$n - 1$ linearly independent constraints**.

We start with n freely selectable bits, but every **independent** equation removes one free choice. After $n - 1$ independent equations, only one free choice remains:

$$n - (n - 1) = 1$$

That one free choice is exactly the difference between the trivial solution 0^n and the non-trivial solution a :

$$2^n \xrightarrow{1 \text{ constraint}} 2^{n-1} \xrightarrow{2 \text{ constraints}} 2^{n-2} \xrightarrow{3 \text{ constraints}} \dots \xrightarrow{n-1 \text{ constraints}} 2^1 = 2$$

And these two solutions are exactly the trivial 0^n and the non-trivial a that we want.

❓ How do we actually solve $Ya = 0$? With ordinary **Gaussian elimination**, except that every operation is done modulo 2. So the quantum computer runs Simon's circuit repeatedly and measures the first register:

$$y^{(1)}, y^{(2)}, \dots, y^{(k)}$$

Each $y^{(i)}$ measurement guarantees:

$$a \cdot y^{(i)} = 0 \pmod{2}$$

Then send/store those ordinary classical bitstrings on a **normal classical computer** that builds the matrix Y and solves the linear system $Ya = 0 \pmod{2}$ using Gaussian elimination.

Once we have enough independent y 's, the solutions reduce to exactly two possibilities: the trivial 0^n and the non-trivial a . We discard the trivial solution (Simon's promise guarantees that $a \neq 0^n$) and accept the non-trivial solution as the hidden period a .

🔧 Complexity of Simon's algorithm

For **Gaussian elimination** on Simon's system, the standard complexity is $O(n^3)$ classical operations, assuming we're solving an $n \times n$ -scale system.

Why? Roughly, Gaussian elimination has three nested dimensions of work:

$$\underbrace{n}_{\text{pivot columns}} \times \underbrace{n}_{\text{rows}} \times \underbrace{n}_{\text{entries per row}} = O(n^3)$$

In Simon, we have about $O(n)$ measured y 's, each containing n bits, so Y is roughly an $n \times n$ binary matrix. The operations are particularly simple because we're over modulo 2: row addition is just XOR. So for the whole Simon algorithm, distinguish:

- **Quantum oracle queries:** $O(n)$. We need $n - 1$ linearly independent constraints to identify the one-dimensional solution space. Since the measured y 's are random, more than $n - 1$ executions may be necessary due to repeated or linearly dependent measurements. Nevertheless, $O(n)$ executions are sufficient with high probability.
- **Classical post-processing:** $O(n^3)$ using standard Gaussian elimination. Because we need to solve the linear system $Ya = 0$ using Gaussian elimination.

However, the $O(n^3)$ classical post-processing does not eliminate Simon's exponential advantage, since Gaussian elimination still requires only polynomial time. Simon's algorithm uses $O(n)$ quantum oracle queries, followed by $O(n^3)$ classical post-processing, whereas the best randomized classical approach requires $O(2^{n/2})$ oracle queries.

✓ Key Takeaways 9

- A single execution of Simon's circuit does **not** directly reveal the hidden period a . Instead, measuring the first register produces a random bitstring y satisfying

$$a \cdot y = 0 \pmod{2}$$

- Each measured y provides **one linear constraint** on the unknown bits of a :

$$a_1 y_1 \oplus a_2 y_2 \oplus \cdots \oplus a_n y_n = 0$$

- Repeating the quantum circuit produces several constraints, which can be collected as the binary linear system

$$Ya = 0 \pmod{2}$$

- What matters is not simply the number of measurements, but their **linear independence**. Repeated or linearly dependent y 's provide no new information.
- For an n -bit hidden period, $n - 1$ linearly independent constraints reduce the solution space to

$$\{0^n, a\}$$

Simon's promise $a \neq 0^n$ then uniquely identifies the hidden period.

- The system $Ya = 0$ is solved classically using **Gaussian elimination over $\text{GF}(2)$** , where row addition corresponds to XOR.

- Because the measurements are random, exactly $n - 1$ circuit executions are not guaranteed to be sufficient. However, $O(n)$ executions are sufficient with high probability.
- Simon's algorithm is therefore naturally divided into two stages:

$$\underbrace{\text{quantum circuit}}_{\text{generate constraints}} + \underbrace{\text{classical Gaussian elimination}}_{\text{recover } a}$$

- Its quantum query complexity is $O(n)$, while standard Gaussian elimination requires $O(n^3)$ classical operations. Both are polynomial, whereas the best randomized classical collision-finding approach requires $O(2^{n/2})$ oracle queries.

4.17.5 Examples

Let's start with a simple example. For $n = 1$, the hidden string has only one bit. Simon requires $a \neq 0$, so the only possibility is immediately $a = 1$. This means the Simon promise:

$$f(x) = f(x \oplus a)$$

Becomes:

$$f(x) = f(x \oplus 1)$$

For $x = 0$, we have $0 \oplus 1 = 1$, so $f(0) = f(1)$. Thus, for $n = 1$, a valid Simon function must actually be constant on the two inputs.

0. Now follow the circuit. The **initial state** is:

$$|\psi_0\rangle = |0\rangle |0\rangle$$

$$\begin{array}{c} |0\rangle \text{ ---} \\ |0\rangle \text{ ---} \end{array}$$

The first qubit is the input register, the second is the output/work register.

1. Apply the **Hadamard gate** to the first qubit:

$$|\psi_1\rangle = H |0\rangle |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle |0\rangle + |1\rangle |0\rangle)$$

$$\begin{array}{c} |0\rangle \text{ ---} \boxed{H} \text{ ---} \\ |0\rangle \text{ ---} \end{array}$$

2. Apply the **oracle** U_f :

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

To both terms of the superposition, we apply the oracle:

$$U_f \underbrace{|0\rangle}_{|x\rangle} \underbrace{|0\rangle}_{|y\rangle} = |0\rangle |0 \oplus f(0)\rangle = |0\rangle |f(0)\rangle$$

$$U_f |1\rangle |0\rangle = |1\rangle |0 \oplus f(1)\rangle = |1\rangle |f(1)\rangle$$

$$|\psi_2\rangle = \frac{1}{\sqrt{2}}(|0\rangle |f(0)\rangle + |1\rangle |f(1)\rangle)$$

Now, since $n = 1$, recall the promise:

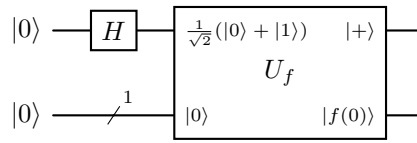
$$f(x) = f(x \oplus a)$$

Where the hidden period must satisfy $a \neq 0$. For $n = 1$, a consists of only one bit, so there are only two possibilities: $a = 0$ or $a = 1$. Since $a \neq 0$, we must have $a = 1$. Now take $x = 0$, Simon's promise says

$$f(0) = f(0 \oplus a) \xrightarrow{a=1} f(0) = f(0 \oplus 1) = f(1)$$

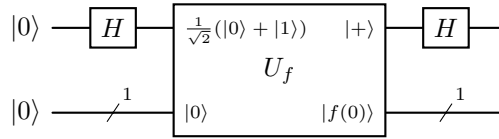
Thus, we can simplify the state $|\psi_2\rangle$:

$$\begin{aligned}
 |\psi_2\rangle &= \frac{1}{\sqrt{2}}(|0\rangle |f(0)\rangle + |1\rangle |f(1)\rangle) \\
 &= \frac{1}{\sqrt{2}}(|0\rangle |f(0)\rangle + |1\rangle |f(0)\rangle) \\
 &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) |f(0)\rangle \\
 &= |+\rangle |f(0)\rangle
 \end{aligned}$$



3. Apply the **Hadamard gate** to the first qubit again:

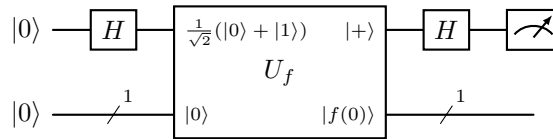
$$|\psi_3\rangle = H |+\rangle |f(0)\rangle = |0\rangle |f(0)\rangle$$



4. **Measure** the first qubit. The result is always $y = 0$ with probability 1. And this satisfies Simon's condition:

$$a \cdot y = 1 \cdot 0 = 0 \pmod{2}$$

In this special case, we can immediately recover the hidden period $a = 1$ from the measurement result $y = 0$. But this is a trivial case, and for $n > 1$ we will need to repeat the algorithm multiple times to recover the hidden period.



What if $n = 2$?

The $n = 2$ case is the **first really meaningful Simon example**, because now the algorithm gives a nontrivial constraint and we can actually recover the hidden period from measurement results.

Let's consider the **hidden period** $a = 01$. The Simon promise says:

$$f(x) = f(x \oplus a) = f(x \oplus 01)$$

So the inputs are paired as follows:

$$f(00) = f(00 \oplus 01) = f(01)$$

$$f(01) = f(01 \oplus 01) = f(00)$$

$$f(10) = f(10 \oplus 01) = f(11)$$

$$f(11) = f(11 \oplus 01) = f(10)$$

More compactly, we can write the function as:

$$f(00) = f(01) \quad \text{and} \quad f(10) = f(11)$$

A **valid oracle** can therefore **assign one output to the first pair and another output to the second pair**. We do not care what the actual outputs are; only the equality structure matters.

0. The **initial state** is:

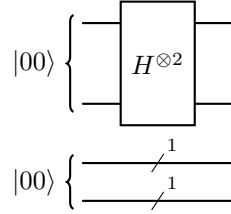
$$|\psi_0\rangle = |00\rangle |00\rangle$$

$$|00\rangle \left\{ \begin{array}{l} \text{---} \\ \text{---} \end{array} \right.$$

$$|00\rangle \left\{ \begin{array}{l} \text{---} \\ \text{---} \end{array} \right.$$

1. Apply the **Hadamard gate** to the first two qubits:

$$|\psi_1\rangle = H^{\otimes 2} |00\rangle |00\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) |00\rangle$$



2. Apply the **oracle U_f** :

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

The oracle acts on both registers: it preserves the first register $|x\rangle$, while the second register is transformed according to $|y\rangle \mapsto |y \oplus f(x)\rangle$.

$$U_f |\psi_1\rangle = \frac{1}{2} (U_f \underbrace{|00\rangle}_{|x\rangle} \underbrace{|00\rangle}_{|y\rangle} + U_f \underbrace{|01\rangle}_{|x\rangle} \underbrace{|00\rangle}_{|y\rangle} + U_f \underbrace{|10\rangle}_{|x\rangle} \underbrace{|00\rangle}_{|y\rangle} + U_f \underbrace{|11\rangle}_{|x\rangle} \underbrace{|00\rangle}_{|y\rangle})$$

Since:

$$|y \oplus f(x)\rangle = |00 \oplus f(x)\rangle = |f(x)\rangle$$

Because 00 is the identity for bitwise XOR, we can simplify the oracle action:

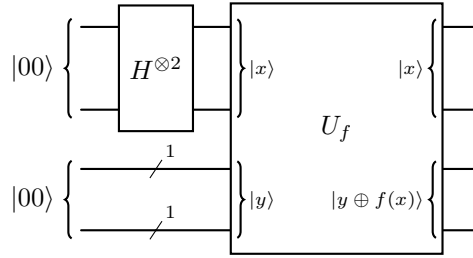
$$|\psi_2\rangle = \frac{1}{2} (|00\rangle |f(00)\rangle + |01\rangle |f(01)\rangle + |10\rangle |f(10)\rangle + |11\rangle |f(11)\rangle)$$

But at the beginning of this exercise, we established that:

$$f(00) = f(01) \quad \text{and} \quad f(10) = f(11)$$

Thus, we can simplify the state $|\psi_2\rangle$:

$$\begin{aligned} |\psi_2\rangle &= \frac{1}{2}(|00\rangle |f(00)\rangle + |01\rangle |f(01)\rangle + |10\rangle |f(10)\rangle + |11\rangle |f(11)\rangle) \\ &= \frac{1}{2}(|00\rangle |f(00)\rangle + |01\rangle |f(00)\rangle + |10\rangle |f(10)\rangle + |11\rangle |f(10)\rangle) \\ &= \frac{1}{2}(|00\rangle + |01\rangle) |f(00)\rangle + \frac{1}{2}(|10\rangle + |11\rangle) |f(10)\rangle \end{aligned}$$



3. Apply the **Hadamard gate** to the first two qubits again:

$$\begin{aligned} |\psi_3\rangle &= (H^{\otimes 2} \otimes I^{\otimes 2}) |\psi_2\rangle = \frac{1}{2} (H^{\otimes 2} |00\rangle + H^{\otimes 2} |01\rangle) |f(00)\rangle + \\ &\quad \frac{1}{2} (H^{\otimes 2} |10\rangle + H^{\otimes 2} |11\rangle) |f(10)\rangle \end{aligned}$$

Recall that:

$$H^{\otimes 2} |00\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

$$H^{\otimes 2} |01\rangle = \frac{1}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle)$$

$$H^{\otimes 2} |10\rangle = \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle)$$

$$H^{\otimes 2} |11\rangle = \frac{1}{2}(|00\rangle - |01\rangle - |10\rangle + |11\rangle)$$

Thus, we can substitute these into the expression for $|\psi_3\rangle$:

- $\alpha = H^{\otimes 2} |00\rangle + H^{\otimes 2} |01\rangle$:

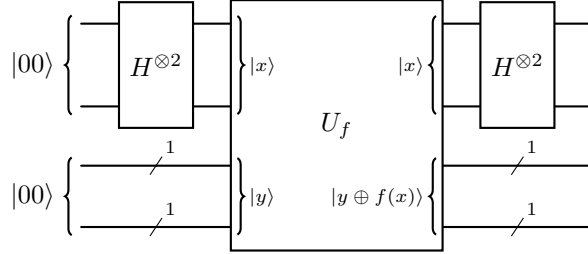
$$\begin{aligned} \alpha &= \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) + \frac{1}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle) \\ &= \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle + |00\rangle - |01\rangle + |10\rangle - |11\rangle) \\ &= \frac{1}{2}(2|00\rangle + 2|10\rangle) \\ &= \frac{1}{2}(2(|00\rangle + |10\rangle)) \\ &= |00\rangle + |10\rangle \end{aligned}$$

- $\beta = H^{\otimes 2} |10\rangle + H^{\otimes 2} |11\rangle$:

$$\begin{aligned}
 \beta &= \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle) + \frac{1}{2}(|00\rangle - |01\rangle - |10\rangle + |11\rangle) \\
 &= \frac{1}{2}(|00\rangle + |01\rangle - |10\rangle - |11\rangle + |00\rangle - |01\rangle - |10\rangle + |11\rangle) \\
 &= \frac{1}{2}(2|00\rangle - 2|10\rangle) \\
 &= |00\rangle - |10\rangle
 \end{aligned}$$

Substituting these back into the expression for $|\psi_3\rangle$, we have:

$$|\psi_3\rangle = \frac{1}{2}(|00\rangle + |10\rangle)|f(00)\rangle + \frac{1}{2}(|00\rangle - |10\rangle)|f(10)\rangle$$



4. **Measure** the first two qubits. We want the probability of measuring the **first register** as 00 or 10.

- For $y = 00$, there are two branches containing $|00\rangle$:

$$\frac{1}{2} |00\rangle |f(00)\rangle \quad \text{and} \quad \frac{1}{2} |00\rangle |f(10)\rangle$$

From Simon's full promise:

$$f(x) = f(x') \iff x = x' \text{ or } x = x' \oplus a$$

With $a = 01$, 00 is paired only with 01. Thus 10 belongs to 11. This means that:

$$f(00) \neq f(10)$$

Because $f(00)$ and $f(10)$ are distinct classical bitstrings, the corresponding computational-basis states are orthogonal:

$$\langle f(00) | f(10) \rangle = 0$$

Therefore the two branches cannot interfere with one another when we measure only the first register. Extracting the relevant part of the state from $|\psi_3\rangle$, we have:

$$|00\rangle \otimes \left(\frac{1}{2} |f(00)\rangle + \frac{1}{2} |f(10)\rangle \right)$$

Thus, the probability of measuring $y = 00$ is:

$$\begin{aligned}
 P(y = 00) &= \left\| \frac{1}{2} |f(00)\rangle + \frac{1}{2} |f(10)\rangle \right\|^2 \\
 &= \frac{1}{4} \langle f(00)|f(00)\rangle + \frac{1}{4} \langle f(10)|f(10)\rangle + \\
 &\quad \frac{1}{4} \langle f(00)|f(10)\rangle + \frac{1}{4} \langle f(10)|f(00)\rangle \\
 &\downarrow \text{since } \langle f(00)|f(00)\rangle = \langle f(10)|f(10)\rangle = 1 \text{ (page 80)} \\
 &\downarrow \text{since } \langle f(00)|f(10)\rangle = \langle f(10)|f(00)\rangle = 0 \text{ (page 80)} \\
 &= \frac{1}{4} + \frac{1}{4} + 0 + 0 = \frac{1}{2}
 \end{aligned}$$

- For $y = 10$, there are two branches containing $|10\rangle$:

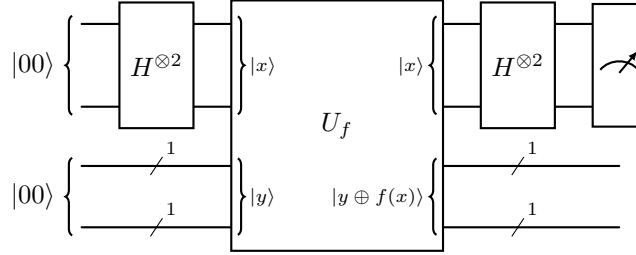
$$\frac{1}{2} |10\rangle |f(00)\rangle \quad \text{and} \quad -\frac{1}{2} |10\rangle |f(10)\rangle$$

Again:

$$|10\rangle \otimes \left(\frac{1}{2} |f(00)\rangle - \frac{1}{2} |f(10)\rangle \right)$$

Thus, the probability of measuring $y = 10$ is:

$$P(y = 10) = \left\| \frac{1}{2} |f(00)\rangle - \frac{1}{2} |f(10)\rangle \right\|^2 = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$$



And for the other possible first-register values $y = 01$ and $y = 11$, their amplitudes have canceled completely, so the probability of measuring them is zero:

$$P(y = 00) = \frac{1}{2}, \quad P(y = 10) = \frac{1}{2}, \quad P(y = 01) = 0, \quad P(y = 11) = 0$$

Finally, let's check that the measurement results satisfy the constraint (explained on page 294, and page 297):

$$a \cdot y = 0 \pmod{2}$$

For $y = 00$, we have:

$$a \cdot y = 01 \cdot 00 \pmod{2} = 0(0) + 1(0) \pmod{2} = 0 \pmod{2}$$

For $y = 10$, we have:

$$a \cdot y = 01 \cdot 10 \pmod{2} = 0(1) + 1(0) \pmod{2} = 0 \pmod{2}$$

Indeed, both measurement results satisfy the constraint. In fact, they undergo constructive interference, which allows us to measure them. In contrast, the other two possible results, $y = 01$ and $y = 11$, are destructively interfered with and thus have zero probability of being measured:

$$a \cdot y = 01 \cdot 01 \pmod{2} = 0(0) + 1(1) \pmod{2} = 1 \pmod{2}$$

$$a \cdot y = 01 \cdot 11 \pmod{2} = 0(1) + 1(1) \pmod{2} = 1 \pmod{2}$$

4.17.6 The Linear-Algebra Interpretation

In previous sections, we discussed the constraints that the hidden string must satisfy:

$$a \cdot y = 0 \pmod{2}$$

Here we discuss the linear-algebra interpretation of this constraint. First of all, the condition is called **orthogonality** between the hidden string a and the measured string y . We weren't introducing a new property of Simon's algorithm, but rather we were **renaming something we already knew**.

❓ **What does “orthogonal” mean here?** We already know ordinary orthogonality. For real vectors, for example:

$$u = (1, 1), \quad v = (1, -1)$$

Their ordinary dot product is:

$$u \cdot v = 1 \cdot 1 + 1 \cdot (-1) = 0$$

So they are orthogonal. In Simon, however, our vectors are **bitstrings** and arithmetic is performed over:

$$GF(2) = \{0, 1\}$$

So for:

$$a = a_1 a_2 \dots a_n, \quad y = y_1 y_2 \dots y_n$$

We define the dot product as:

$$a \cdot y = a_1 y_1 \oplus a_2 y_2 \oplus \dots \oplus a_n y_n$$

If the result is $a \cdot y = 0$, we say that y is orthogonal to a over $GF(2)$.⁹ For example, if $a = 01$, for $y = 10$ we have:

$$a \cdot y = 01 \cdot 10 = 0(1) \oplus 1(0) = 0 \oplus 0 = 0$$

So $y = 10$ is orthogonal to $a = 01$:

$$01 \perp 10 \text{ over } GF(2)$$

This is **not geometric perpendicularity in ordinary Euclidean space**. It's algebraic orthogonality according to the mod-2 dot product.

❓ **If every measured y must be orthogonal to a , which y 's are actually possible?** That leads us to the **set of all possible Simon measurements**. We know that Simon can only produce y 's satisfying:

$$a \cdot y = 0$$

So for a fixed hidden string a , we define the set of all possible measurements as:

$$a^\perp = \{y \in \{0, 1\}^n \mid a \cdot y = 0\} \tag{120}$$

⁹ $GF(2)$ is the formal mathematical name for the arithmetic system we are using when we say “modulo 2”. It is the acronym for **Galois Field** with 2 elements. The two elements, in our case, are simply 0 and 1.

In words, take **all possible n -bit strings** y and keep only those satisfying the orthogonality condition. This is called the **orthogonal component** of a over $GF(2)$.

For example, if $n = 2$ and $a = 01$, there are four possible y 's:

$$00, 01, 10, 11$$

We can check which ones are orthogonal to a :

$$01 \cdot 00 = 0$$

$$01 \cdot 01 = 1$$

$$01 \cdot 10 = 0$$

$$01 \cdot 11 = 1$$

So the set of all possible measurements is:

$$a^\perp = \{00, 10\}$$

And that's exactly what we found explicitly running the circuit in the previous section. So the linear-algebra interpretation and the quantum interference calculation are describing **the same result**.

❓ How large and how structured is the set of all possible measurements? We have just defined the “*orthogonal component*” of a over $GF(2)$, but **how many independent choices are there inside $a^\perp$?** Let's look at the $n = 3$ case. If $a = 111$, the condition for $y = y_1y_2y_3$ to be orthogonal to a is:

$$a \cdot y = 0$$

Therefore, we have:

$$111 \cdot y_1y_2y_3 = 1(y_1) \oplus 1(y_2) \oplus 1(y_3) = y_1 \oplus y_2 \oplus y_3 = 0$$

Originally, y has three freely selectable bits: y_1 , y_2 , and y_3 . But the equation imposes one constraint on them. For example, if we choose y_1 and y_2 freely, then y_3 is determined by the equation:

$$y_3 = y_1 \oplus y_2$$

So only **two bits can be chosen independently**, and the third bit is determined by the first two. Hence, the dimension is governed by the number of independent bits:

$$\dim(a^\perp) = 3 - 1 = 2$$

In general, y contains n bits/degrees of freedom, but the condition of orthogonality:

$$a \cdot y = 0$$

Provides **one nontrivial linear constraint** because Simon guarantees that $a \neq 0$. Therefore, the dimension of the orthogonal component is:

$$n \text{ degrees of freedom} - 1 \text{ constraint} = n - 1$$

In our previous example, the constraint $a \cdot y = 0$ forces y_3 to be determined by y_1 and y_2 , so we lose one degree of freedom, and the dimension is:

$$\dim(a^\perp) = n - 1 \quad (121)$$

This is the **dimension of the set of all possible measurements** for a given hidden string a . In other words, it means that every possible measurement y can be described using $n - 1$ **independent binary choices**. Because each independent choice can be either 0 or 1, the **total number of elements** in the set is:

$$|a^\perp| = 2^{n-1} \quad (122)$$

For example, if $n = 3$, and $a = 111$, the set of all possible measurements is:

$$a^\perp = \{000, 011, 101, 110\}$$

And its dimension is:

$$\dim(a^\perp) = 3 - 1 = 2$$

But its size (number of possible measurements) is:

$$|a^\perp| = 2^{3-1} = 4$$

🌀 **If we don't know a , but Simon gives us y 's, can we recover a ?** So far we have been looking at the problem from the perspective:

a is fixed, and we are finding all possible y 's

We found that the set of all possible measurements is:

$$a^\perp = \{y \in \{0, 1\}^n \mid a \cdot y = 0\}$$

And we found that its dimension is:

$$\dim(a^\perp) = n - 1$$

Now, here, we **reverse the direction**. Suppose we run the circuit several times and obtain:

$$y^{(1)}, y^{(2)}, \dots, y^{(m)}$$

Every one belongs to $a^\perp$, so they all satisfy:

$$a \cdot y^{(i)} = 0 \quad \text{for } i = 1, 2, \dots, m$$

We know that the dimension of $a^\perp$ is:

$$\dim(a^\perp) = n - 1$$

That means there can be $n-1$ **independent directions** inside the space of valid measurements. So if Simon eventually gives us $n - 1$ **linearly independent** measurements:

$$y^{(1)}, y^{(2)}, \dots, y^{(n-1)}$$

They collectively describe the whole space of valid measurements $a^\perp$. And now comes the key reversal: *if we know the entire space $a^\perp$, which is the vector a orthogonal to all vectors in that space?* The answer is: **the hidden string a itself**. In other words, if we know the entire space of valid measurements, we can recover the hidden string a .

 Example 45: $n = 3$ and $a = 111$

Suppose we have $n = 3$, and the hidden string is unknown:

$$a = a_1 a_2 a_3$$

First run. Suppose Simon measures;

$$y^{(1)} = 110$$

We know from everything we proved earlier that every measurement must satisfy:

$$a \cdot y = 0$$

Therefore, the first measurement gives us the equation:

$$110 \cdot a = 0$$

Which gives:

$$1a_1 \oplus 1a_2 \oplus 0a_3 = a_1 \oplus a_2 = 0$$

From XOR properties, $a_1 \oplus a_2 = 0$ means that $a_1 = a_2$. But that's not enough to determine a yet. We need more measurements.

But suppose we would like to stop here and try to guess a . We know that $a_1 = a_2$:

$$\begin{cases} a_1 = a_2 \\ a_3 \text{ is free} \end{cases}$$

This means that a can be either:

$$a = 000, \quad a = 110, \quad a = 001, \quad a = 111$$

But Simon guarantees that $a \neq 0$, so we can eliminate 000. So we are left with three possibilities:

$$a = 110, \quad a = 001, \quad a = 111$$

At this point, we cannot determine a uniquely. We need more measurements!

Second run. Suppose we run Simon again measure:

$$y^{(2)} = 011$$

This gives us the equation:

$$011 \cdot a = 0$$

Which gives:

$$0a_1 \oplus 1a_2 \oplus 1a_3 = a_2 \oplus a_3 = 0$$

From XOR properties, $a_2 \oplus a_3 = 0$ means that $a_2 = a_3$. Now we have two equations:

$$\begin{cases} a_1 \oplus a_2 = 0 \\ a_2 \oplus a_3 = 0 \end{cases} \Rightarrow \begin{cases} a_1 = a_2 \\ a_2 = a_3 \end{cases} \Rightarrow a_1 = a_2 = a_3$$

So the hidden string must be either 000 or 111. But Simon guarantees that $a \neq 0$, so the only solution is:

$$a = 111$$

And indeed, if we check the dimension of the space of valid measurements, we find that it is $n - 1 = 2$, so two independent measurements are enough to recover a .

4.17.7 Why Simon's Algorithm Is Important

The central reason is the enormous difference in **query complexity** between the classical and quantum approaches. Recall the problem statement:

$$f : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

With the Simon's promise:

$$f(x) = f(y) \iff x = y \quad \text{or} \quad y = x \oplus a$$

Where $a \neq 0^n$ is a secret bitstring. Our goal is to discover the hidden period a .

 **Classical randomized:** $O(2^{n/2})$ (page 286). We already saw that classically we essentially need to find a **collision**:

$$f(x_1) = f(x_2), \quad x_1 \neq x_2$$

Once we find one:

$$a = x_1 \oplus x_2$$

But until we encounter such a collision, evaluating $f(x)$ doesn't directly reveal the hidden pairing. Using the **birthday-paradox** argument, a **randomized classical algorithm need** approximately:

$$O(2^{n/2})$$

Oracle queries to obtain a collision with substantial probability.

For intuition, imagine $n = 100$:

$$2^{n/2} = 2^{50} \approx 1.1 \times 10^{15}$$

Simon's algorithm, on the other hand, requires **only** 100 queries to the oracle. That's an enormous asymptotic separation.

 **Quantum:** $O(n)$. The quantum algorithm approaches the problem completely differently. It does **not** search for a collision. Instead, each execution gives us a random y satisfying:

$$a \cdot y = 0 \pmod{2}$$

$$\text{run 1} \rightarrow y^{(1)} \rightarrow a \cdot y^{(1)} = 0 \pmod{2}$$

$$\text{run 2} \rightarrow y^{(2)} \rightarrow a \cdot y^{(2)} = 0 \pmod{2}$$

$\vdots$

$$\text{run } n \rightarrow y^{(n)} \rightarrow a \cdot y^{(n)} = 0 \pmod{2}$$

After collecting enough independent constraints, we solve the resulting linear system classically. Thus the quantum oracle-query is **linear**:

$$O(n)$$

Compared to the classical randomized approach, this is an **exponential query-complexity separation**. And the word *query* matters. As we discussed before, we shouldn't casually claim that every implementation of Simon has total runtime $O(n)$. Query complexity asks specifically: *how many times do we need to call the black-box function/oracle?* In other words, the quantum algorithm is exponentially more efficient in terms of oracle queries than any classical randomized algorithm.

Simon is a hidden-structure problem

The quantum computer is **not trying to learn the table of f** :

$$x \mapsto f(x)$$

We don't care whether, for example, $f(000) = 101$ or $f(000) = 011$. We care about a **global relationship shared by all inputs**:

$$f(x) = f(x \oplus a)$$

That's the hidden structure. Think of f as an enormous table with 2^n rows. Classically, we're evaluating individual entries and hoping eventually to notice two equal outputs. Simon instead extracts information about the **global symmetry of the entire function**:

$$x \leftrightarrow x \oplus a$$

This is why calling a a **hidden period** makes sense: $x \rightarrow x \oplus a$ leaves the function unchanged.

For an **ordinary periodic function**, T is a period if:

$$f(x + T) = f(x) \quad \forall x$$

For example:

$$\sin(x + 2\pi) = \sin(x) \quad \forall x$$

So 2π is a period. And we can think of T as a **shift that leaves the function unchanged**:

$$x \xrightarrow{+T} x + T \quad \text{but} \quad f(x) \rightarrow f(x + T) = f(x)$$

Thus Simon has the same idea, but the domain is bitstrings:

$$x \in \{0, 1\}^n$$

And the relevant operation is XOR (the promise):

$$f(x \oplus a) = f(x), \quad \forall x, \quad a \neq 0^n$$

Ordinary periodicity	Simon periodicity
$f(x + T) = f(x)$	$f(x \oplus a) = f(x)$
shift by T	XOR-shift by a
$T = \text{period}$	$a = \text{hidden period}$

🔗 Connection with Bernstein-Vazirani

Both algorithms have a hidden bitstring:

$$\text{Bernstein-Vazirani: } w \quad \text{Simon: } a$$

But they reveal it in very different ways. In Bernstein-Vazirani:

$$f_w(x) = w \cdot x \pmod{2}$$

After one quantum query and the final Hadamards, we obtain $|w\rangle$. So measurement gives w **directly**. Simon does not do that. Instead, one execution gives a y satisfying:

$$a \cdot y = 0 \pmod{2}$$

So Simon gives **information about** a , rather than a itself. This is an important conceptual progression:

$$\text{Deutsch} \rightarrow \text{BV} \rightarrow \text{Simon}$$

The quantum algorithms we're seeing are becoming increasingly sophisticated in **what measurement reveals**.

❓ Why interference is essential

In Simon, after the oracle, we have a superposition of all x :

$$\frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle |f(x)\rangle$$

At this point, the hidden period is encoded in the state because:

$$x \text{ and } x \oplus a$$

Are correlated with the same output:

$$f(x) = f(x \oplus a)$$

But if we simply measure x , we get essentially a random input. So the information is **present**, but not yet in a directly useful measurement distribution. That's what the final $H^{\otimes n}$ is for.

Recall our derivation. For the paired inputs x and $x \oplus a$, the amplitudes for a candidate measurements y combine as:

$$(-1)^{x \cdot y} + (-1)^{(x \oplus a) \cdot y} = (-1)^{x \cdot y} (1 + (-1)^{a \cdot y})$$

Now:

- If $a \cdot y = 0$, then

$$1 + (-1)^{a \cdot y} = 2$$

So the amplitudes **reinforce** (constructive interference).

- If $a \cdot y = 1$, then

$$1 + (-1)^{a \cdot y} = 0$$

So the amplitudes **cancel** (destructive interference).

❓ **Why the final Hadamards reveal global structure.** The oracle acts coherently across all x 's (due to the first Hadamards). Then, it establishes the hidden correlation:

$$x \leftrightarrow x \oplus a$$

The **final Hadamards cause those different computational paths to overlap**. And when paths overlap, amplitudes add or cancel. As a consequence, instead of learning something like:

$$f(101) = 011$$

The measurement tells us something about the entire function:

$$a \cdot y = 0 \pmod{2}$$

That equation is true because of the global periodic structure:

$$f(x) = f(x \oplus a)$$

Across the whole domain.

That's a **recurring quantum-algorithm pattern** we've already seen in Deutsch and Bernstein-Vazirani: **superposition allows the oracle to encode a global pattern, while interference converts that pattern into measurable amplitudes**. Thus, Simon does not use quantum mechanics to evaluate $f(x)$ faster; it uses interference to reveal a hidden global symmetry of f .

This is why Simon is such an important algorithm. It demonstrates that, for a suitably structured oracle problem, a **quantum computer can extract global structure with only $O(n)$ queries** where a randomized classical algorithm needs $O(2^{n/2})$ queries.

4.17.8 Simon as a Fourier / Period-Finding Algorithm

We've been describing Simon using Hadamard gates $H^{\otimes n}$, but there is a deeper interpretation: $H^{\otimes n}$ is the quantum Fourier transform appropriate to $\mathbb{Z}_2^n$.

❓ What is $\mathbb{Z}_2^n$?

It is essentially the mathematical structure of our n -bit strings:

$$\{0, 1\}^n$$

For example, for $n = 2$, we have:

$$\mathbb{Z}_2^2 = \{00, 01, 10, 11\}$$

The operation is component-wise addition modulo 2, which is equivalent to the XOR operation. So throughout Simon, whenever we've written $x \oplus a$, we've actually been performing addition in the group $\mathbb{Z}_2^n$. That's why Simon's promise:

$$f(x) = f(x \oplus a)$$

Can be interpreted as saying: f possesses a hidden periodic symmetry under addition by a .

❓ Where does Fourier analysis enter in?

That's the fun part: we've already been doing Fourier analysis! Recall the formula we've used repeatedly:

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \mathbb{Z}_2^n} (-1)^{x \cdot y} |y\rangle$$

The factors $(-1)^{x \cdot y}$ are the **Fourier phases** appropriate to the group $\mathbb{Z}_2^n$. So the final Hadamards **aren't just an arbitrary trick** that happens to produce interference. Mathematically, $H^{\otimes n}$ is **performing the Fourier transform** for the particular algebraic structure in which Simon's period lives.

Thus, for bitstrings with XOR, this is called the **Fourier transform over $\mathbb{Z}_2^n$** , or equivalently the **Walsh-Hadamard transform**.

≡ Simon as a hidden subgroup problem

There's an even more abstract interpretation, but we only need the basic idea. Consider the set:

$$H = \{0^n, a\}$$

It forms a **small subgroup** of $\mathbb{Z}_2^n$, and Simon's function is constant on pairs of the form:

$$\{x, x \oplus a\}$$

These pairs are precisely the **cosets** (i.e., shifted copies) associated with that hidden subgroup. So, **Simon hides the subgroup H inside the equality structure**

of f . The quantum algorithm uses Fourier sampling to obtain information that lets us reconstruct that hidden structure.

In other words, the subgroup is the set of XOR shifts that don't change the function's output f . Obviously the trivial subgroup $\{0^n\}$ is always hidden, but the promise of Simon's problem is that there is a **non-trivial** hidden subgroup $\{0^n, a\}$, and the quantum algorithm is able to find it efficiently.

 Example 46: Simon's hidden subgroup

Suppose Simon's secret is $a = 01$. The possible inputs are:

$$\{00, 01, 10, 11\}$$

Simon promises that:

$$f(x) = f(x \oplus a)$$

So the function is constant on the pairs:

$$\underbrace{\{00, 01\}}_{00 \oplus 01 = 01}, \quad \underbrace{\{10, 11\}}_{10 \oplus 01 = 11}$$

Therefore, the domain is partitioned.

Now, the **hidden subgroup** is:

$$H = \{00, 01\}$$

And it is special because if we combine its elements using XOR, we never leave the original set:

$$\begin{aligned} 00 \oplus 00 &= 00, \\ 00 \oplus 01 &= 01, \\ 01 \oplus 00 &= 01, \\ 01 \oplus 01 &= 00 \end{aligned}$$

So H has a closed little structure under XOR. The other coset is obtained by taking an element outside H , for example 10, and XORing it with every element of H :

$$10 \oplus H = \{10 \oplus 00, 10 \oplus 01\} = \{10, 11\}$$

Therefore, the two cosets are:

$$\underbrace{\{00, 01\}}_H \quad \text{and} \quad \underbrace{\{10, 11\}}_{10 \oplus H}$$

The **second coset is not a subgroup**. In particular, it does not contain the identity element 00 and it is not closed under XOR. For example:

$$10 \oplus 11 = 01 \notin \{10, 11\}$$

⚠ Subgroup vs. Coset. A **subgroup** is a **group inside the larger group**, while a **coset** is a **shifted copy of a subgroup**. In this case, H is a subgroup because it contains the identity 00, and XORing any two elements of H keeps us inside H . In contrast, $10 \oplus H$ XORs with **every element of H** and produces:

$$10 \oplus H = \{10, 11\}$$

It is not a subgroup because it does not contain the identity 00, but it is a coset because it is a shifted copy of H .

❓ Why is it called hidden? Simple, because we **don't know a** . If we knew a , we could easily find the subgroup $H = \{0^n, a\}$ instantly. But the oracle doesn't tell us H . Instead, we can only query f , and the structure of f indirectly reveals that:

$$f(x) = f(x \oplus a)$$

Then:

$$H = \{0^n, a\}$$

Because **finding this subgroup is essentially equivalent to finding a** .

⚠ Ordinary Period Finding

Suppose instead that our inputs aren't n -bit strings under XOR. Imagine:

$$f : \mathbb{Z}_N \rightarrow \mathbb{Z}_N$$

With an ordinary modular period r :

$$f(x) = f((x + r) \bmod N)$$

For example, suppose $N = 12$ and $r = 3$. Then we'd have patterns like:

$$\begin{aligned} f(0) &= f(3) = f(6) = f(9), \\ f(1) &= f(4) = f(7) = f(10), \\ f(2) &= f(5) = f(8) = f(11) \end{aligned}$$

Compare that with Simon:

$$f(x) = f(x \oplus a)$$

The conceptual problem is extremely similar: **there is a hidden period, we need to recover it**. But the underlying mathematical structure has changed. Simon has $\mathbb{Z}_2^n$ as its domain, while the ordinary period finding problem has $\mathbb{Z}_N$ as its domain. And **this is why we need something more general than Hadamards**.

❓ Why isn't $H^{\otimes n}$ enough anymore? Because the Hadamard only introduces phases of $+1$ and -1 . We can see that directly from $(-1)^{x \cdot y}$. Those phases are perfect for binary/XOR structure. But suppose the period lives in something

like $\mathbb{Z}_N$, where N is not a power of 2. Then we need richer phases corresponding to the N -th roots of unity:

$$1, \quad e^{2\pi i/N}, \quad e^{4\pi i/N}, \quad \dots$$

That motivates the general **Quantum Fourier Transform**:

$$QFT_N |x\rangle = \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} e^{2\pi i xy/N} |y\rangle$$

We'll explore this in more detail in the next sections, but for now, compare it with Simon:

$$H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \mathbb{Z}_2^n} (-1)^{x \cdot y} |y\rangle$$

They have the **same broad structure**: a **superposition over y with phases depending on x and y** . The difference is that Simon's underlying group is $\mathbb{Z}_2^n$, for which the Fourier transform happens to be simply $H^{\otimes n}$.

5 Limits of Quantum Information

5.1 No-Cloning Principle

5.1.1 Why Cloning Matters

Briefly, the **no-cloning principle** states that it is **impossible to create a perfect copy of an arbitrary unknown quantum state**.

At first sight this may seem like a strange technical limitation, because in classical computing copying information is *trivial*: we can easily copy files, duplicate RAM, clone hard disks, send packets over the network, etc. However, **in quantum computing, copying an arbitrary qubit is fundamentally impossible**. This is **not a technological limitation**, instead it is a direct **consequence of the mathematical structure** of quantum mechanics.

In classical computing, a bit can be in one of two states: 0 or 1. In quantum computing, a qubit instead can be in infinitely many states, because it can be in any superposition of 0 and 1:

$$|\psi\rangle = a|0\rangle + b|1\rangle$$

With complex amplitudes a and b such that $|a|^2 + |b|^2 = 1$. So a qubit is not just “0” or “1”, but it represents a point on the Bloch sphere, meaning there are infinitely many possible states. **If arbitrary cloning were possible**, we could take an unknown state $|\psi\rangle$ and create as many copies of it as we want. This would allow us to perform measurements on the copies to learn about the original state $|\psi\rangle$ without disturbing it, **violating the fundamental principles of quantum mechanics**.

Relation with Unitary

The no-cloning principle theorem is deeply connected to the fact that quantum evolution is **unitary**. Quantum gates are represented by unitary matrices:

$$U^\dagger U = U U^\dagger = I$$

Unitary operations preserve norms, probabilities and inner products between states. **Cloning would violate this preservation property**. Because, intuitively, if two partially similar states must remain partially similar after evolution (i.e., their inner product must be preserved), then cloning would instead make them “more distinguishable” (i.e., their inner product would change), which is a contradiction. We will see the formal proof of the no-cloning principle in the next section.

Relation with No-Signaling

The no-cloning principle is also related to the no-signaling principle, because **breaking the no-cloning principle would violate the no-signaling principle**.

Suppose Alice and Bob share an entangled Bell pair:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Normally, Bob cannot determine whether Alice has measured her qubit or not, because his local measurements always appear random (50% probability of measuring 0 and 50% probability of measuring 1). But *imagine* Bob could clone his qubit infinitely many times. Then, he could:

1. Create many identical copies of his qubit.
2. Perform measurements/statistics on all copies.
3. Infer whether the global quantum state collapsed (i.e., whether Alice measured her qubit). Using the principle of reductio ad absurdum, if Alice does not measure, then Bob will measure random results (0, 1, 0, 1, etc.). When Alice measures and collapses the state to $|0\rangle$, then Bob's qubit will instantly become $|0\rangle$, and therefore, all of his copies will be $|0\rangle$.

This would allow instantaneous communication between Alice and Bob, faster than the speed of light, which is forbidden by the no-signaling principle (violation of relativity, no information can travel faster than the speed of light). Therefore, the **impossibility of cloning helps preserve the no-signaling principle**.

Relation with Quantum State Tomography

Another consequence of breaking the no-cloning principle would be that we would be able to store an infinite amount of classical information inside one qubit.

The idea is the following. Because a qubit can assume infinitely many states, one could theoretically encode a very long classical string into amplitudes (e.g., the binary expansion of π):

$$|\psi_{\text{message}}\rangle$$

But there is a problem: we **cannot directly read the amplitudes** from a single qubit, because measurement collapses the state. If we measure the qubit, we only get one classical outcome (e.g., 0 or 1), and not the full amplitudes a and b . So normally, the information is not accessible.

However, exists a technique called **quantum state tomography** that:

1. Prepares many identical copies¹⁰ of the same state $|\psi_{\text{message}}\rangle$.
2. Measures them in different bases (e.g., X, Y, Z).

¹⁰Quantum State Tomography does not violate the no-cloning principle, because it does not create perfect copies of an unknown state. Instead, it prepares many identical copies of a **known** state $|\psi_{\text{message}}\rangle$, which is allowed by quantum mechanics. For example, if we have a quantum circuit that prepares the state $|\psi_{\text{message}}\rangle$, we can run that circuit many times to get many copies of $|\psi_{\text{message}}\rangle$, which is perfectly fine. The no-cloning principle only forbids creating perfect copies of an **unknown** state, not preparing many copies of a **known** state.

3. Reconstructs the amplitudes of the original state $|\psi_{\text{message}}\rangle$ statistically.

In simple terms, given many copies of the same unknown state, estimate what the state was.

Now, imagine there were **no no-cloning theorem**. Then we could do the following:

1. Take the single qubit $|\psi_{\text{message}}\rangle$ that encodes the message.
2. Clone it infinitely many times to get many copies of $|\psi_{\text{message}}\rangle$. This would be possible because we are assuming that the no-cloning principle does not hold.
3. Perform measurements/statistics on all copies to reconstruct the amplitudes of the original state $|\psi_{\text{message}}\rangle$, and therefore read the message encoded in the amplitudes.

So cloning would transform ONE unknown qubit into enough data for tomography, which would allow us to read the message encoded in the amplitudes. This would be a huge problem, because we could encode an infinite amount of classical information into one qubit and then read it using cloning and tomography.

Example 1: Why Cloning Matters

✓ **Normal world (no-cloning theorem exists).** Alice wants to send Bob one qubit:

$$|\psi_{\text{message}}\rangle = \sqrt{0.73148291} |0\rangle + \sqrt{0.26851709} |1\rangle$$

Suppose these decimals secretly encode a super secret message. Bob receives the qubit. Now Bob asks: “*what are the exact amplitudes of the state $|\psi_{\text{message}}\rangle$?*” Bob measures the qubit, but measurement only gives him one classical bit (0 or 1). For example, if Bob measures 0, then the state collapses to $|0\rangle$, and the original amplitudes are lost. So from ONE measurement, Bob cannot reconstruct 0.73148291 exactly. So the hidden information is inaccessible to Bob, and the message is safe.

Hypothetical world (no-cloning theorem does not exist).

Now suppose cloning is allowed. Bob receives the qubit $|\psi_{\text{message}}\rangle$. Now Bob **can clone** the qubit, so he runs a mysterious cloning machine that creates many identical clones of $|\psi_{\text{message}}\rangle$:

$$|\psi\rangle \rightarrow |\psi\rangle |\psi\rangle |\psi\rangle |\psi\rangle |\psi\rangle \dots$$

Now he has millions of copies of $|\psi_{\text{message}}\rangle$. Now Bob can decrypt the super secret message. How? Simple:

1. **Measure many copies in the computational basis.** Bob has millions of copies. Suppose he observes (after many measurements) that 731482 times he gets 0 and 268517 times he gets 1. So Bob can estimate the probabilities of measuring 0 and 1 as 0.731482 and 0.268517. Bob is already very close to the original amplitudes, but he can do better.

2. **Measure in other bases.** Bob is smart, and he knows that measuring in the computational basis only gives him the magnitudes of the amplitudes, but not their phases. So Bob also measures in the X and Y bases to get more information about the state. By combining all the measurement results, Bob can reconstruct the original state $|\psi_{\text{message}}\rangle$ with very high precision, and therefore read the super secret message encoded in the amplitudes.

⚠ The Problem. Alice effectively transmitted thousands, millions, theoretically infinite classical digits using only one qubit, that should not be physically possible (because in physics, the amount of extractable classical information from a physical system must be finite).

🔗 Relation with Quantum Error Correction

In classical systems, redundancy is simple: we can copy bits to create redundancy for error correction. For example, we can encode a bit as three bits:

$$0 \rightarrow 000 \quad 1 \rightarrow 111$$

Then, if one bit gets flipped, we can still recover the original bit by majority voting:

$$000 \rightarrow 000 \quad 001 \rightarrow 000 \quad 010 \rightarrow 000 \quad 100 \rightarrow 000$$

Then majority voting allows us to correct single bit errors. However, in **quantum computing we cannot simply duplicate qubits** and we cannot create copies of arbitrary quantum states. So **quantum error correction cannot protect information by simply copying qubits**. Instead, it uses more complex techniques (e.g., entanglement, syndrome measurements, etc.) to protect quantum information without violating the no-cloning principle.

In summary, the no-cloning theorem is not an isolated curiosity. It is one of the foundational constraints that shape all of quantum information theory. It is connected to reversibility, unitarity, causality, measurement, entanglement, quantum cryptography, quantum error correction. **Without the no-cloning theorem, the entire structure of quantum information would collapse.**

5.1.2 Formal Statement

Definition 1: No-Cloning Theorem

The **No-Cloning Theorem** states that there **does not exist a universal quantum gate capable of perfectly copying an arbitrary unknown quantum state**.

Formally, suppose we have:

- A qubit in an unknown state: $|x\rangle$.
- And a second qubit initialized to: $|0\rangle$.

We would like a quantum gate U such that:

$$U |x\rangle |0\rangle = |x\rangle |x\rangle \quad (123)$$

For every possible quantum state $|x\rangle$. The **No-Cloning Theorem states that such a gate U cannot exist.**

Proof. We now prove that the cloning gate U :

$$U |x\rangle |0\rangle = |x\rangle |x\rangle$$

Cannot exist for an arbitrary quantum state $|x\rangle$.

1. **Assume that cloning is possible.** Suppose there exists a unitary gate U such that:

$$U |x\rangle |0\rangle = |x\rangle |x\rangle$$

And also:

$$U |y\rangle |0\rangle = |y\rangle |y\rangle$$

So U should clone both $|x\rangle$ and $|y\rangle$.

2. **Take the inner product before cloning.** Before cloning, the two input states are:

$$|x\rangle |0\rangle \quad \text{and} \quad |y\rangle |0\rangle$$

Their inner product is:

$$\langle x|y\rangle \langle 0|0\rangle$$

Since $|0\rangle$ is a normalized state, we have $\langle 0|0\rangle = 1$. Thus, the inner product before cloning is:

$$\langle x|y\rangle$$

3. **Take the inner product after cloning.** After cloning, the states become:

$$|x\rangle |x\rangle \quad \text{and} \quad |y\rangle |y\rangle$$

Their inner product is:

$$\langle x|y\rangle \langle x|y\rangle = (\langle x|y\rangle)^2$$

Now we use the tensor product property of inner products, which states that (page 89):

$$(A \otimes B)(C \otimes D) = (AC) \otimes (BD)$$

Applying this to our inner product, we get:

$$(\langle x| \otimes \langle x|)(|y\rangle \otimes |y\rangle) = (\langle x|y\rangle) \otimes (\langle x|y\rangle)$$

Which simplifies to:

$$(\langle x|y\rangle)^2$$

4. **Use unitary.** Valid quantum gates must be unitary, which means they preserve inner products. Therefore, the inner product before and after applying U must be the same:

$$\langle x|y\rangle = (\langle x|y\rangle)^2$$

This is only possible if $\langle x|y\rangle = 0$ or $\langle x|y\rangle = 1$. But for arbitrary states, this is not generally true. Thus, the assumption that cloning is possible leads to a contradiction.

Let's try to understand why this is a contradiction. Suppose two states are $|x\rangle$ and $|y\rangle$, and suppose $|\langle x|y\rangle|$ (i.e., the magnitude of the inner product) is 0.8. It means that the states are not identical, but they are still “quite similar”. They are partially distinguishable, but not perfectly. Now, if cloning were possible, the inner product after cloning would be still 0.8. However, we found that the inner product after cloning would be $(0.8)^2 = 0.64$. This means that the states have become more distinguishable after cloning, which is a contradiction because unitary operations cannot change the inner product between states:

$$|\langle x|y\rangle| = 0.8 \xrightarrow{\text{clone}} |\langle x|y\rangle|^2 = 0.64 \xrightarrow{\text{clone}} |\langle x|y\rangle|^3 = 0.512 \dots$$

QED

Deepening: Alternative Proof of No-Cloning Theorem

We can also prove the no-cloning theorem using a different approach, by directly applying the unitarity of U to the cloning operation. This alternative proof is often more concise and elegant.

Proof. Suppose, by contradiction, that there exists a cloning gate U such that:

$$U |x\rangle |0\rangle = |x\rangle |x\rangle$$

For an arbitrary state $|x\rangle$.

Since U is supposed to be universal, it must also clone another arbitrary state $|y\rangle$:

$$U |y\rangle |0\rangle = |y\rangle |y\rangle$$

At this point, we have two equations describing the action of U on two different input states.

Now, the **only thing we know about unitary operators is that they preserve inner products** (page 43). So if we want to derive a **contradiction from unitarity**, we must compute an inner product. If the inner product before cloning is different from the inner product after cloning, then we have a contradiction, which means that U cannot be unitary, and therefore cannot be a valid quantum gate.

Let's compute the inner product before and after cloning.

- **Before cloning**, the inner product between the two input states is:

$$(U|x\rangle|0\rangle) \cdot (U|y\rangle|0\rangle)$$

But the inner product between two states is written as (page 78):

$$\begin{aligned} (U|x\rangle|0\rangle) \cdot (U|y\rangle|0\rangle) &= (U|x\rangle|0\rangle)^\dagger (U|y\rangle|0\rangle) \\ &= (\langle 0| \langle x| U^\dagger) (U|y\rangle|0\rangle) \\ &\downarrow \text{Remove parentheses} \\ &= \langle 0| \langle x| U^\dagger U |y\rangle|0\rangle \\ &\downarrow \text{Use unitarity: } U^\dagger U = I \\ &= \langle 0| \langle x|y\rangle|0\rangle \end{aligned}$$

- **After cloning**, the inner product between the two output states is:

$$(|x\rangle|x\rangle) \cdot (|y\rangle|y\rangle)$$

Using the tensor product property of inner products:

$$\begin{aligned} (|x\rangle|x\rangle) \cdot (|y\rangle|y\rangle) &= (|x\rangle|x\rangle)^\dagger (|y\rangle|y\rangle) \\ &= (\langle x| \langle x|) (|y\rangle|y\rangle) \\ &= \langle x|y\rangle \langle x|y\rangle \\ &= (\langle x|y\rangle)^2 \end{aligned}$$

Thus, **if cloning were possible**, we would have:

$$\langle 0| \langle x|y\rangle|0\rangle = (\langle x|y\rangle)^2$$

Using the tensor product property of inner products again:

$$\langle \psi| \langle \phi|\chi\rangle|\psi\rangle = \langle \phi|\chi\rangle \langle \psi|\psi\rangle$$

Remarkably, the inner product $\langle \phi|\chi\rangle$ is just a complex number (page 78), so we can pull it out of the inner product. So, we can simplify the left side:

$$\underbrace{\langle 0| \langle x|y\rangle|0\rangle}_{=\langle x|y\rangle \langle 0|0\rangle} = (\langle x|y\rangle)^2$$

Since $\langle 0|0 \rangle = 1$, we have:

$$\langle x|y \rangle = (\langle x|y \rangle)^2 \iff \begin{cases} \langle x|y \rangle = 0 \\ \langle x|y \rangle = 1 \end{cases}$$

So unitarity forces the inner product to satisfy this equation. However, this equation is not true for all possible values of $\langle x|y \rangle$. It only holds if $\langle x|y \rangle = 0$ or $\langle x|y \rangle = 1$. But for arbitrary states, $\langle x|y \rangle$ can take any value between 0 and 1, so this is a contradiction. Therefore, a universal cloning gate U cannot exist. QED

❓ Why CNOT Does NOT Clone Qubits

A common confusion is that the CNOT gate seems to copy the control qubit into the target qubit (page 103). This is true only when the control qubit is a computational basis state, i.e. $|0\rangle$ or $|1\rangle$. It is not true for a generic superposition.

Classically, CNOT is often written as:

$$\text{CNOT}(v_A, v_B) = (v_A, v_A \oplus v_B)$$

Where $\oplus$ denotes XOR. ⚠️ So, if the second qubit is initialized to $|0\rangle$, one *may* be tempted to write:

$$\text{CNOT}(v_A, |0\rangle) = (v_A, v_A \oplus 0) = (v_A, v_A)$$

This is true because the XOR of any bit with 0 is the bit itself:

v_A	$ 0\rangle$	$v_A \oplus 0$
$ 0\rangle$	$ 0\rangle$	$ 0\rangle$
$ 1\rangle$	$ 0\rangle$	$ 1\rangle$

For example, if we start with $v_A = |1\rangle$ and the second qubit is $|0\rangle$, applying CNOT gives:

$$\text{CNOT}(|1\rangle, |0\rangle) = (|1\rangle, |1\rangle \oplus |0\rangle) = (|1\rangle, |1\rangle)$$

This looks **like cloning**: the first qubit v_A appears to be copied into the second qubit.

However, the mistake is that this notation is valid only for computational basis states $|0\rangle$ and $|1\rangle$, not for arbitrary superpositions. The informal notation:

$$\text{CNOT}(v_A, v_B) = (v_A, v_A \oplus v_B)$$

Can be used only when the inputs are basis states. For a generic superposition, we must use the full linear algebra representation of CNOT, which does not clone the state.

✔ **Correct Quantum Computation.** Let the control qubit be a generic superposition:

$$v_A = a_0 |0\rangle + a_1 |1\rangle$$

And let the target qubit be initialized to $|0\rangle$. The joint input state is:

$$v_A |0\rangle = (a_0 |0\rangle + a_1 |1\rangle) |0\rangle$$

Using the tensor product:

$$v_A |0\rangle = a_0 |00\rangle + a_1 |10\rangle$$

Now apply CNOT linearly. CNOT leaves $|00\rangle$ unchanged and maps $|10\rangle$ to $|11\rangle$:

$$\text{CNOT}(a_0 |00\rangle + a_1 |10\rangle) = a_0 |00\rangle + a_1 |11\rangle$$

So the real output is:

$$a_0 |00\rangle + a_1 |11\rangle$$

If CNOT had truly cloned the qubit, the output should instead be:

$$v_A v_A = (a_0 |0\rangle + a_1 |1\rangle)(a_0 |0\rangle + a_1 |1\rangle)$$

Expanding:

$$v_A v_A = a_0^2 |00\rangle + a_0 a_1 |01\rangle + a_1 a_0 |10\rangle + a_1^2 |11\rangle$$

However, these two states are not the same:

$$a_0 |00\rangle + a_1 |11\rangle \neq a_0^2 |00\rangle + a_0 a_1 |01\rangle + a_1 a_0 |10\rangle + a_1^2 |11\rangle$$

Therefore, CNOT does not clone a superposition. More fundamentally, a true universal cloning operation cannot exist because it would violate the unitary structure of quantum mechanics. Indeed, in general:

$$a_0 |00\rangle + a_1 |11\rangle$$

Cannot be factorized as a tensor product of two independent single-qubit states. The second state $v_A v_A$ would be two independent qubits in the same state. This is the key distinction: CNOT can create correlation or entanglement, but it does not create an independent copy of an unknown qubit. In summary, the CNOT output is entangled, while the true cloned state would be two qubits in the same state.

In other words, CNOT preserves quantum coherence between basis components, while true cloning would require duplicating the amplitudes themselves, which is forbidden by unitary quantum evolution.

5.2 No-Deleting Principle

The **No-Deleting Principle** is the dual counterpart of the no-cloning theorem. While the no-cloning theorem states that it is impossible to perfectly copy an arbitrary unknown quantum state, the no-deleting principle states that it is also **impossible to perfectly delete quantum information through a unitary operation**.

In other words:

- Quantum information **cannot** be freely **copied**;
- Quantum information **cannot** be freely **destroyed**.

Like the no-cloning theorem, the no-deleting principle is **deeply connected to the unitary structure** of quantum mechanics.

Definition 2: No-Deleting Principle

Suppose we want a quantum gate U that deletes a qubit state:

$$U |x\rangle = |0\rangle$$

For an arbitrary unknown state $|x\rangle$.

Equivalently, if we have two identical copies of a state, we may wish to delete one copy while leaving the other unchanged:

$$U |x\rangle |x\rangle = |x\rangle |0\rangle \quad (124)$$

The **No-Deleting Theorem** states that **no universal unitary gate** capable of performing this operation **can exist** for arbitrary unknown quantum states.

Intuitively, this means that **quantum information cannot simply disappear through reversible quantum evolution**.

As with the no-cloning theorem, we can prove the no-deleting principle by contradiction, using the fact that unitary operators preserve inner products (see page 334).

Proof. Suppose, by contradiction, that there exists a deleting gate U such that:

$$U |x\rangle = |0\rangle$$

For an arbitrary quantum state $|x\rangle$.

Since U is supposed to be universal, it must also delete another arbitrary state $|y\rangle$:

$$U |y\rangle = |0\rangle$$

At this point, we have two equations describing the action of U on two different input states.

Now, the only thing we know about unitary operators is that they **preserve inner products**. Therefore, if U is unitary, the inner product between the input states must be equal to the inner product between the corresponding output states.

- **Before deleting**, the inner product between the two input states is:

$$(U|x\rangle) \cdot (U|y\rangle)$$

Using the definition of inner product:

$$\begin{aligned} (U|x\rangle) \cdot (U|y\rangle) &= (U|x\rangle)^\dagger (U|y\rangle) \\ &= \langle x|U^\dagger U|y\rangle \\ &\downarrow \text{since } U \text{ is unitary, } U^\dagger U = I \\ &= \langle x|y\rangle \end{aligned}$$

- **After deleting**, both output states become $|0\rangle$, so the inner product is:

$$|0\rangle \cdot |0\rangle$$

Again:

$$\begin{aligned} |0\rangle \cdot |0\rangle &= |0\rangle^\dagger |0\rangle \\ &= \langle 0|0\rangle \\ &\downarrow \text{since } |0\rangle \text{ is a normalized state (page 19)} \\ &= 1 \end{aligned}$$

Since unitary operators preserve inner products:

$$\langle x|y\rangle = 1$$

But this is **not true in general for arbitrary quantum states**. It is true only if:

$$|x\rangle = |y\rangle$$

Therefore, such a **universal deleting gate U cannot exist.**

QED

5.3 No-Signaling Principle

Quantum entanglement creates strong correlations between distant particles (i.e., the measurement of one particle instantaneously affects the state of the other, regardless of the distance between them, page 124). However, these correlations cannot be used to transmit information instantaneously.

This idea is known as the **no-signaling principle**: although measurements on **entangled systems are correlated, they cannot be exploited for faster-than-light communication**.

⚠ The Communication Problem

Suppose Alice and Bob are at opposite ends of the universe. They each possess one qubit of the Bell pair (page 118):

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

The first qubit belongs to Alice, while the second belongs to Bob. More precisely, Alice owns the **first subsystem** of the bipartite state and Bob owns the **second subsystem**.

$$\begin{aligned} |00\rangle &= |0\rangle_A \otimes |0\rangle_B & |11\rangle &= |1\rangle_A \otimes |1\rangle_B \\ |\Phi^+\rangle &= \frac{|0\rangle_A \otimes |0\rangle_B + |1\rangle_A \otimes |1\rangle_B}{\sqrt{2}} \end{aligned}$$

Because the two qubits are **entangled**, measuring one qubit instantaneously determines the state of the other. For example:

- If Alice measures $|0\rangle$, Bob's qubit immediately collapses to $|0\rangle$;
- If Alice measures $|1\rangle$, Bob's qubit immediately collapses to $|1\rangle$.

⚠ At first sight, this **seems to violate relativity**. One may wonder:

“Can Bob detect instantly whether Alice has already measured her qubit?”

If the answer were yes, then **Alice could communicate information to Bob instantaneously**. For example, Alice could decide:

- If Alice measures her qubit $\Rightarrow$ transmit bit 1;
- If Alice does not measure her qubit $\Rightarrow$ transmit bit 0.

This **would allow faster-than-light communication**, contradicting special relativity.

The **no-signaling principle states that this is impossible**: although entanglement creates instantaneous correlations, **Bob cannot locally determine whether Alice has performed a measurement or not**.

Definition 3: No-Signaling Principle

The **No-Signaling Principle** states that quantum **entanglement cannot be used to transmit information instantaneously** between distant observers.

More precisely, local measurements performed on one subsystem of an entangled quantum system do not allow another observer to determine whether a measurement has already been performed on the distant subsystem.

Equivalently, although entangled measurements produce strong correlations, the local measurement statistics observed by one party are independent of the actions performed by the other party.

Proof. We want to understand whether Bob can locally detect whether Alice has already measured her qubit or not.

Alice and Bob share the Bell pair:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} = \frac{|0\rangle_A \otimes |0\rangle_B + |1\rangle_A \otimes |1\rangle_B}{\sqrt{2}}$$

The question is **whether Alice's measurement changes the probabilities observed by Bob**. If Bob's probabilities changed, then **Alice could use her measurement choice to send information instantaneously**.

Case 1: Alice does not measure.

If Alice does not measure her qubit, the global state remains:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

If Bob measures his qubit in the computational basis, he obtains:

$$P_B(0) = \frac{1}{2}, \quad P_B(1) = \frac{1}{2}$$

So Bob sees a **completely random result**.

Case 2: Alice measures.

Now suppose Alice measures her qubit first in the computational basis. There are **two possible outcomes**:

- With probability $\frac{1}{2}$, Alice obtains $|0\rangle$, and the global state collapses to:

$$|00\rangle$$

In this case, Bob's qubit becomes $|0\rangle$.

- With probability $\frac{1}{2}$, Alice obtains $|1\rangle$, and the global state collapses to:

$$|11\rangle$$

In this case, Bob's qubit becomes $|1\rangle$.

Therefore, from Bob's point of view:

$$P_B(0) = \frac{1}{2}, \quad P_B(1) = \frac{1}{2}$$

So **Bob again sees a completely random result.**

Conclusion.

In both cases:

$$P_B(0) = P_B(1) = \frac{1}{2}$$

Therefore, **Bob's local measurement statistics are the same whether Alice has measured or not.** This means Bob cannot infer Alice's action by measuring only his own qubit. Thus, no usable information is transmitted instantaneously, and the no-signaling principle is preserved. QED

❓ What are we trying to prove? We start with the Bell state:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}} = \frac{|0\rangle_A \otimes |0\rangle_B + |1\rangle_A \otimes |1\rangle_B}{\sqrt{2}}$$

Alice has the first qubit, while Bob has the second qubit. Now suppose Alice is billions of light-years away from Bob. The concern is that if Alice measures her qubit, the joint entangled state changes, and Bob's qubit can subsequently be described by a definite state correlated with Alice's outcome. Can Bob detect this change? If yes, then Alice could use her measurement choice to send information to Bob faster than light, for example:

- If Alice measures her qubit $\Rightarrow$ transmit bit 1;
- If Alice does not measure her qubit $\Rightarrow$ transmit bit 0.

This would violate the relativity principle.

❓ What is the relativity principle? It is not a topic of this course, but it is fundamental to understand the motivation behind the no-signaling principle. The **Einstein's Relativity Principle** core idea is: **no information, influence, or signal can travel faster than the speed of light.** This is not merely a technological limitation, but a fundamental law of spacetime.

Now, related to quantum mechanics, why is this principle relevant? Imagine Alice is on Earth and Bob is on Mars. Suppose Alice could send a message instantly. Then there would exist observers for whom: Bob receives the message before Alice sends it, which creates a paradox. In some reference frames, the **effect would occur before the cause**, but this contradicts our understanding of causality. Therefore, Einstein looked at this and thought¹¹: “*wait, did*

¹¹We will discuss the EPR paradox later, but it is interesting to note that Einstein historically disliked the idea that measuring one particle could apparently affect another distant particle instantaneously. He famously referred to this phenomenon as “spukhafte Fernwirkung” (spooky action at a distance), expressing his discomfort with the idea that quantum mechanics could allow for such non-local effects. As we will see later, Schrödinger coined the term “entanglement” to describe this phenomenon, and it has since become a fundamental aspect of quantum mechanics. Nevertheless, Einstein's concerns about entanglement's implications for locality and causality remain a topic of discussion and debate in the foundations of quantum mechanics.

something travel from Alice to Bob faster than light?". This is the motivation behind the no-signaling principle: yes, the correlations are real, but Alice cannot choose the outcome she obtains, so she cannot force Bob's qubit to collapse to a specific state. Therefore, Bob cannot detect whether Alice has measured her qubit or not, and no information is transmitted faster than light. In simple terms, **Entanglement creates correlations, but it does not allow for Communication**. Entanglement makes distant measurement outcomes correlated, but the no-signaling principle guarantees that these correlations cannot be controlled to transmit information.

5.4 EPR Paradox and Quantum Correlation

5.4.1 Definition of EPR Paradox

In 1935, Albert Einstein, Boris Podolsky, and Nathan Rosen published a paper entitled *Can Quantum-Mechanical Description of Physical Reality Be Considered Complete?*. [2] Their goal was not to disprove quantum mechanics, but to **show** that the **theory appeared to be incomplete**.

The source of their concern was a peculiar phenomenon predicted by quantum mechanics: two particles could become so strongly correlated that measuring one particle would instantaneously determine the state of the other, regardless of the distance separating them (explained in the proof of the no-signaling principle, page 340).

Einstein found this deeply troubling. According to special relativity, **no information can travel faster than the speed of light**. Yet entanglement seemed to imply an instantaneous connection between distant particles, a phenomenon Einstein famously called “*spooky action at a distance*” (*spukhafte Fernwirkung*).

The EPR paper sparked one of the most important debates in the history of physics. Shortly after its publication, Erwin Schrödinger analyzed the EPR argument and realized that it revealed what he considered the most distinctive feature of quantum mechanics. In a series of papers published the same year, he introduced the term *Verschränkung* (“entanglement”), which is now known as quantum entanglement (page 117).

Interestingly, Schrödinger largely shared Einstein’s discomfort with the standard interpretation of quantum mechanics. However, while Einstein viewed the EPR argument as evidence that the theory was incomplete, **Schrödinger recognized entanglement as a fundamental property of quantum systems**.

The central **question raised by the EPR paradox** is therefore:

*If measuring Alice’s qubit immediately determines the state of Bob’s qubit, **does quantum mechanics allow faster-than-light communication?***

As we have already seen through the no-signaling principle (page 340), the **answer is no**. Entanglement creates correlations between distant measurements, but these correlations cannot be used to transmit information.

Precisely, Einstein’s concern was fundamentally:

*If measuring Alice’s qubit instantaneously determines the state of Bob’s qubit, **what is actually happening?***

In other words, **how can nature produce these instantaneous correlations?** Einstein was not primarily asking about the possibility of faster-than-light communication, but rather about the underlying mechanism that could explain the observed correlations without violating relativity. He suspected

that Bob's particle already possessed some hidden information before the measurement, which would determine the outcome of Alice's measurement without any need for instantaneous influence.

Modern Perception of the EPR Paradox

To understand the paradox in a modern quantum-computing setting, consider the Bell state:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Suppose Alice and Bob each possess one qubit of this entangled pair and are separated by a very large distance.

If Alice measures her qubit in the computational basis and obtains:

$$|0\rangle$$

Then Bob's qubit immediately collapses to:

$$|0\rangle$$

Similarly, if Alice measures:

$$|1\rangle$$

Then Bob's qubit immediately collapses to:

$$|1\rangle$$

At first sight, this appears to imply an instantaneous influence between distant particles. How can Bob's qubit "know" which outcome Alice obtained if they may be separated by billions of light-years? This is the modern formulation of the EPR paradox.

Definition 4: Einstein-Podolsky-Rosen (EPR) Paradox

The **Einstein-Podolsky-Rosen (EPR) Paradox** is the **apparent contradiction** between quantum **entanglement** and the **classical principle of locality**.

More precisely, if two particles are prepared in an entangled state, a measurement performed on one particle appears to instantaneously determine the state of the other, regardless of the distance separating them.

This raises the question of whether quantum mechanics is a complete description of physical reality, or whether additional hidden information is required to explain these correlations without instantaneous influence.

5.4.2 Entanglement and Quantum Correlation

The EPR paradox highlights a fundamental feature of entangled systems: measurement outcomes may appear completely random to individual observers, yet exhibit strong correlations when compared afterward.

In simple terms, Alice and Bob each observe **random outcomes** when measuring their respective qubits. However, when they later compare their results, they discover that these outcomes are **correlated in a way that appears difficult to reconcile with classical intuition**.

❓ Alice and Bob obtain correlated results. Why is that surprising?

At first glance, it might seem that correlated results are not surprising at all. After all, **classical systems can also exhibit correlations**.

🎲 **Classical example.** Suppose two coins are prepared in such a way that they always show the same face. If one observer sees Heads, they immediately know that the other coin must also be Heads. Likewise, if one observer sees Tails, the other must see Tails. This is an example of a **classical correlation**.

In **classical physics**, such **correlations** are explained by assuming that the **system possessed predetermined properties** before any observation was made. In our example, the two coins were already both Heads or both Tails before anyone looked at them.

💡 **Einstein's intuition.** Einstein believed that the **correlations observed in entangled particles might be explained in exactly the same way**. According to this view, the particles could carry some form of **hidden information that predetermines the outcome of future measurements**. If such hidden information existed, the apparent mystery of entanglement would disappear: measuring one particle would simply reveal information that was already present from the beginning.

Quantum entanglement exhibits a superficially similar phenomenon. Individual measurement outcomes appear random, yet they become strongly correlated when compared afterward.

⚠️ **The difference.** Unlike classical correlations, which are completely determined by pre-existing properties of the system, **quantum correlations depend on the measurement basis chosen by the observers**. This means that **changing how Alice and Bob perform their measurements can change the observed correlations in a way that has no classical analogue**.

Therefore, the existence of correlations is not itself surprising. The **real challenge is determining whether quantum correlations can be explained using the same classical intuition** that explains correlated coins, cards, or dice, or whether they **represent an entirely new physical phenomenon**.

This question leads directly to the distinction between **classical correlation** and **quantum correlation**.

5.4.3 Classical vs. Quantum Correlation

✦ Classical Correlation

A **Classical Correlation** arises from **information** that **already exists before any measurement** is performed.

In a classical system, correlated outcomes can be explained by assuming that the **system possesses predetermined properties**. These properties are often called **Hidden Variables**, because they **determine the outcome of future observations** even if the observers do not know their values.

✎ Example 2: Classical Correlation

For example, consider again two coins prepared so that they always show the same face. Before any observation takes place, the system is already in one of two possible configurations:

- Heads - Heads;
- Tails - Tails.

The measurement does not create the correlation. It merely reveals information that was already present.

Therefore, knowing the outcome observed by one observer immediately allows the other outcome to be inferred. No instantaneous influence is required, because the **correlation existed before the measurement**.

✦ Quantum Correlation

A **Quantum Correlation** arises from **measurements performed on an entangled quantum system**.

As in the classical case, each observer sees random outcomes individually. However, the observed **correlations** possess a remarkable property: they **depend on the measurement basis chosen by the observers**.

✎ Example 3: Quantum Correlation

For example, if Alice and Bob share the Bell state

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

And both measure in the same basis, their outcomes are perfectly correlated. However, if they choose different measurement bases, the observed correlations change.

The Main Difference

In a **classical system**, changing how an observer looks at the system does not alter the underlying correlation. The **correlation is determined entirely by the hidden variables** that existed **before the measurement**.

In contrast, **quantum correlations** depend on the **measurements that are actually performed**. The observed correlations cannot always be explained by assuming that the particles carried predetermined answers before the measurement.

This distinction is what makes quantum correlation fundamentally different from classical correlation and lies at the heart of the EPR debate.

Bell Pair Example

The distinction between classical and quantum correlation becomes clearer when we analyze an entangled Bell pair. Consider the Bell state:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

This state can also be expressed in the Hadamard basis as (see the full derivation in section 4.7):

$$|\Phi^+\rangle = \frac{|++\rangle + |--\rangle}{\sqrt{2}}$$

Suppose Alice and Bob each possess one qubit of this entangled pair:

$$|\Phi^+\rangle_{AB} = \frac{|00\rangle_{AB} + |11\rangle_{AB}}{\sqrt{2}} = \frac{|+\rangle_A \otimes |+\rangle_B + |-\rangle_A \otimes |-\rangle_B}{\sqrt{2}} = \underbrace{\frac{|0\rangle_A \otimes |0\rangle_B + |1\rangle_A \otimes |1\rangle_B}{\sqrt{2}}}_{\text{classical correlation}}$$

Example 4: Same Basis Measurements

Suppose Alice and Bob both measure in the computational basis $\{|0\rangle, |1\rangle\}$.

Alice measures first and obtains:

$$|0\rangle \quad \text{or} \quad |1\rangle$$

With equal probability:

$$P(0) = P(1) = \frac{1}{2}$$

If Alice obtains:

$$|0\rangle$$

The Bell state collapses to:

$$|00\rangle$$

Therefore Bob's qubit becomes:

$$|0\rangle$$

Similarly, if Alice obtains:

$$|1\rangle$$

The Bell state collapses to:

$$|11\rangle$$

And Bob's qubit becomes:

$$|1\rangle$$

Consequently, **Bob always obtains the same outcome as Alice.**

Although each observer sees **random outcomes individually**, they discover **perfect correlation when they later compare their measurement results**.

Example 5: Different Basis Measurements

Now suppose Alice measures in the Hadamard basis:

$$\{|+\rangle, |-\rangle\}$$

Where

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Bob, however, continues measuring in the computational basis.

Alice obtains either:

$$|+\rangle \quad \text{or} \quad |-\rangle$$

With probability:

$$\frac{1}{2}$$

If Alice measures $|+\rangle$, Bob's qubit collapses to:

$$|+\rangle$$

If Alice measures $|-\rangle$, Bob's qubit collapses to:

$$|-\rangle$$

However, **Bob is measuring in the computational basis.** For both $|+\rangle$ and $|-\rangle$:

$$P(0) = P(1) = \frac{1}{2}$$

Therefore Bob obtains:

$$0 \quad \text{or} \quad 1$$

With equal probability, regardless of Alice's outcome.

When Alice and Bob later compare their results, they find that the **perfect correlation** observed in the previous experiment has **disappeared**.

💡 In a classical hidden-variable model, the correlation would be determined entirely by pre-existing information carried by the particles. In contrast, the Bell-pair example shows that the **observed quantum correlation depends on the measurement basis chosen by the observers** (same basis, perfect correlation; different basis, no correlation). This basis dependence is one of the fundamental signatures of quantum correlation.

5.4.4 Entanglement vs. Quantum Correlation

Throughout the discussion of the EPR paradox, we have repeatedly encountered two closely related concepts: **entanglement** and **quantum correlation**. Although they are strongly connected, they describe different aspects of a quantum system.

 **Entanglement.** We have already defined **entanglement** (page 117) as a property of the quantum state itself. A system is entangled when its state cannot be written as a tensor product of the states of its individual subsystems.

For example, the Bell state:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Is entangled because it cannot be expressed as:

$$|\psi_A\rangle \otimes |\psi_B\rangle$$

The important point is that entanglement is an intrinsic property of the system. It exists independently of whether anybody performs a measurement (see the discussion on page 119).

 **Quantum Correlation.** **Quantum Correlation** refers to the **statistical relationship observed between measurement outcomes**. Unlike entanglement, quantum correlation is **not a property of the state** alone. It **depends on how the system is measured**. For instance, Alice and Bob may share the same entangled Bell pair, yet the observed correlations change depending on the measurement bases they choose.

Entanglement vs. Quantum Correlation

Entanglement does not depend on the measurement basis:

- Entanglement is a **property of the quantum system**.
- It exists before any measurement is performed.

Quantum correlation, on the other hand, depends on the measurement basis:

- Quantum correlation is a **property of the measurement outcomes**.
- Different measurement bases may produce different observed correlations.

In summary:

- **Entanglement** is a property of the quantum state and does not depend on how the system is measured.
- **Quantum correlation** is a property of the measurement outcomes and depends on the measurement basis chosen by the observers.

5.5 Quantum Teleportation

5.5.1 Problem Definition and Required Resources

Let's imagine Alice has a qubit:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

And she wants Bob to have **exactly the same quantum state**. Alice does **not** know what α and β are. The state could be anything: $|0\rangle$, $|1\rangle$, $|+\rangle$, $|-\rangle$, or any point on the Bloch sphere. So the protocol must work for **every possible qubit**.

❓ **Why can't Alice simply measure it?** This is the first obstacle. If Alice measures the qubit she obtains only 0 or 1. Suppose the state:

$$|\psi\rangle = \frac{\sqrt{3}}{2}|0\rangle + \frac{1}{2}|1\rangle$$

After measuring, she might get $|0\rangle$ or $|1\rangle$. The original amplitudes α and β are gone forever. So **measurement destroys the information we wanted to send**.

❓ **Why can't Alice simply tell Bob the amplitudes?** Because she doesn't know them. Even worse, if she did know them (α and β), she would need to send an infinite amount of information to Bob. A qubit contains a **continuous** amount of information (a point on the Bloch sphere), not just one classical bit. So Alice cannot write in a message "*Hey Bob, use $\alpha = 0.834$ and $\beta = 0.551e^{i0.42}$* ", because she has no way to discover those numbers from a single copy of the qubit.

❓ **Why can't she copy the qubit?** We already know the answer. The No-Cloning Theorem (page 34) says: "***there is no quantum gate that can copy an arbitrary unknown quantum state***". This is exactly why teleportation is so surprising.

❓ **So what is teleportation actually trying to do? And why is this surprising?**

It is **not** send the particle. It is **transfer the quantum state**.

Imagine Alice owns *electron A* and Bob owns *electron B*. At the end, Alice's electron **does not** have the state anymore; Bob's electron **does**. The particle never moved, only the **state** moved. This is why it is called **State Teleportation**.

It is surprising because Bob receives $|\psi\rangle$ without Alice ever sending the qubit, the amplitudes, or even a copy of the qubit. Instead, they only exchange one entangled pair and two classical bits.

❓ The three resources required for teleportation

We still don't know how to do teleportation, but first we need to understand the resources required. Teleportation is impossible if Alice starts with only one qubit. She needs three resources:

1. **The unknown qubit.** This is the object we want to teleport.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

We can think of it as someone handing Alice a mysterious box and saying: “*Please make Bob end up with this exact quantum state*”. The important point is that **Alice does not know the values of α and β** . She only knows that the state has the general form:

$$\alpha|0\rangle + \beta|1\rangle$$

The protocol must work for **every possible qubit**.

2. **A shared Bell pair.** This is the most mysterious resource. Before teleportation even begins, Alice and Bob already share an entangled pair of qubits:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

This Bell pair contains **no information about the unknown qubit $|\psi\rangle$** . It is prepared **before** Alice even receives the unknown qubit.

It is a sort of **quantum communication channel**. In classical communication, we might need an Ethernet cable, Wi-Fi, or optical fiber. Teleportation needs something different: **shared entanglement**. Without this Bell pair, there is **no quantum resource** connecting Alice and Bob.

3. **Two classical bits.** After Alice measures, she gets one of only four outcomes: 00, 01, 10, or 11. She sends those two bits to Bob. **Bob uses those two bits to perform a correction on his qubit.** This is the only classical communication in the protocol. It is important to note that **the classical bits do not contain any information about α and β** . They only tell Bob which correction to apply.

≡ Qubit Ordering Convention

In the teleportation protocol, we will always order the qubits as follows:

1. The unknown qubit $|q_\psi\rangle$ (Alice's qubit).
2. Alice's half of the Bell pair $|q_A\rangle$ (Alice's qubit).
3. Bob's half of the Bell pair $|q_B\rangle$ (Bob's qubit).

This is just notation, it prevents mistakes.

5.5.2 Shared Bell Pair and Initial Three-Qubit State

In this section, we will prepare the starting point for the quantum teleportation protocol:

- Alice will have the unknown qubit.
- Alice and Bob will share an entangled Bell pair.
- The entire three-qubit system will be written as one combined state.

⌘ Before anything happens

At the beginning, **there are three qubits**.

$$\text{Alice} = \underbrace{q_\psi}_{?} \quad \underbrace{q_A}_0 \quad \text{Bob} = \underbrace{q_B}_0$$

The first qubit is the unknown state:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

The other two are initially just $|00\rangle$. Alice owns **two qubits** and Bob owns **one qubit**.

🌀 Alice and Bob create an entangled pair

Now Alice and Bob prepare the Bell state (page 118):

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (125)$$

The Bell pair **does not contain** the unknown state. It is simply a resource that Alice and Bob decide to share **before** teleportation begins. Think of it like preparing the communication channel in advance.

🧠 **Why specifically a Bell pair?** At this point, it is not clear why we need to use a Bell pair. The answer will become clear in the next section, when we see how the teleportation protocol works. For now, we simply accept that Alice and Bob start with one maximally entangled pair of qubits.

✂ **Build the complete state.** The complete quantum system is simply the tensor product of the unknown state and the Bell pair:

$$|\Psi_1\rangle = |\psi\rangle \otimes |\Phi^+\rangle = (\alpha|0\rangle + \beta|1\rangle) \otimes \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (126)$$

Now, if we distribute the tensor product exactly like ordinary multiplication, there are four products:

$$\begin{aligned} \alpha|0\rangle \otimes |00\rangle &= \alpha|000\rangle \\ \alpha|0\rangle \otimes |11\rangle &= \alpha|011\rangle \\ \beta|1\rangle \otimes |00\rangle &= \beta|100\rangle \\ \beta|1\rangle \otimes |11\rangle &= \beta|111\rangle \end{aligned}$$

Putting it all together, we have the initial three-qubit state:

$$|\Psi_1\rangle = \frac{1}{\sqrt{2}} (\alpha |000\rangle + \alpha |011\rangle + \beta |100\rangle + \beta |111\rangle) \quad (127)$$

Now we have a complete description of the three-qubit system, and we are ready to start the teleportation protocol.

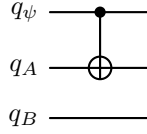
5.5.3 Alice's Operations and Mathematical Derivation

The teleportation protocol applies two gates to the first two qubits of the three-qubit state $|\Psi_1\rangle$ (Alice's qubit and her half of the Bell pair). The first gate is a CNOT gate, which uses Alice's qubit as the control and her half of the Bell pair as the target. The second gate is a Hadamard gate applied to Alice's qubit. Let's explain the effect of these gates on the state $|\Psi_1\rangle$.

✂ Apply the CNOT

The **first operation is a CNOT gate applied to the first two qubits**. The **control is the first qubit q_ψ** and the **target is the second qubit q_A** . So the second qubit flips only when the first qubit is 1. Therefore, **only the terms $\beta|100\rangle$ and $\beta|111\rangle$ will be affected** by the CNOT gate. The first term $\beta|100\rangle$ will become $\beta|110\rangle$, while the second term $\beta|111\rangle$ will become $\beta|101\rangle$. The other terms remain unchanged.

$$|\Psi_{CX}\rangle = \frac{1}{\sqrt{2}} (\alpha|000\rangle + \alpha|011\rangle + \beta|110\rangle + \beta|101\rangle) \quad (128)$$



Notice that **Bob's qubit, the third one, is never touched by Alice's operations, so it remains unchanged**. The state of the system after the CNOT gate is $|\Psi_{CX}\rangle$.

✂ Apply the Hadamard to the first qubit

Now **Alice applies a Hadamard gate to her qubit q_ψ** . Recall that the Hadamard gate transforms the basis states as follows:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

For each term in $|\Psi_{CX}\rangle$, we apply the Hadamard transformation to the first qubit. So, for convenience, we can rewrite the state $|\Psi_{CX}\rangle$ as:

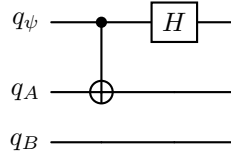
$$|\Psi_{CX}\rangle = \frac{1}{\sqrt{2}} (\alpha|0\rangle \otimes (|00\rangle + |11\rangle) + \beta|1\rangle \otimes (|10\rangle + |01\rangle))$$

Now we apply the Hadamard gate to the first qubit:

$$\begin{aligned} H|\Psi_{CX}\rangle &= \frac{1}{\sqrt{2}} (\alpha H|0\rangle \otimes (|00\rangle + |11\rangle) + \beta H|1\rangle \otimes (|10\rangle + |01\rangle)) \\ &= \frac{1}{\sqrt{2}} \left(\alpha \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes (|00\rangle + |11\rangle) + \right. \\ &\quad \left. \beta \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes (|10\rangle + |01\rangle) \right) \\ &= \frac{1}{2} (\alpha(|0\rangle + |1\rangle) \otimes (|00\rangle + |11\rangle) + \beta(|0\rangle - |1\rangle) \otimes (|10\rangle + |01\rangle)) \end{aligned}$$

Expanding this expression, we get:

$$\begin{aligned}
 |\Psi_H\rangle &= \frac{1}{2} (\alpha |0\rangle |00\rangle + \alpha |0\rangle |11\rangle + \alpha |1\rangle |00\rangle + \alpha |1\rangle |11\rangle + \\
 &\quad \beta |0\rangle |10\rangle + \beta |0\rangle |01\rangle - \beta |1\rangle |10\rangle - \beta |1\rangle |01\rangle) \\
 &= \frac{1}{2} (\alpha |000\rangle + \alpha |011\rangle + \alpha |100\rangle + \alpha |111\rangle + \\
 &\quad \beta |010\rangle + \beta |001\rangle - \beta |110\rangle - \beta |101\rangle)
 \end{aligned}$$



Even if it is correct, it **does not reveal teleportation**. To see teleportation, we can rearrange the terms by grouping them according to the owner of the first two qubits (Alice's) and the last qubit (Bob's):

- Alice's bits 00:

$$\begin{aligned}
 &\alpha |000\rangle + \beta |001\rangle \\
 &|00\rangle \otimes (\alpha |0\rangle + \beta |1\rangle) \\
 &\quad \underbrace{\hspace{1.5cm}}_{|\psi\rangle}
 \end{aligned}$$

- Alice's bits 01:

$$\begin{aligned}
 &\alpha |011\rangle + \beta |010\rangle \\
 &|01\rangle \otimes (\alpha |1\rangle + \beta |0\rangle) \xrightarrow{\text{convention}} |01\rangle \otimes (\beta |0\rangle + \alpha |1\rangle)
 \end{aligned}$$

- Alice's bits 10:

$$\begin{aligned}
 &\alpha |100\rangle - \beta |101\rangle \\
 &|10\rangle \otimes (\alpha |0\rangle - \beta |1\rangle)
 \end{aligned}$$

- Alice's bits 11:

$$\begin{aligned}
 &\alpha |111\rangle - \beta |110\rangle \\
 &|11\rangle \otimes (\alpha |1\rangle - \beta |0\rangle) \xrightarrow{\text{convention}} |11\rangle \otimes (-\beta |0\rangle + \alpha |1\rangle)
 \end{aligned}$$

Putting all together, we can write the state after Alice's operations as:

$$\begin{aligned}
 |\Psi_H\rangle = |\Psi_2\rangle &= \frac{1}{2} [|00\rangle \otimes (\alpha |0\rangle + \beta |1\rangle) + \\
 &\quad |01\rangle \otimes (\beta |0\rangle + \alpha |1\rangle) + \\
 &\quad |10\rangle \otimes (\alpha |0\rangle - \beta |1\rangle) + \\
 &\quad |11\rangle \otimes (-\beta |0\rangle + \alpha |1\rangle)]
 \end{aligned} \tag{129}$$

❓ What does this equation mean? At first sight, it may not be clear what this equation means. However, it is crucial to understand that the first two qubits belong to Alice, while the last qubit belongs to Bob (state in parentheses). The equation shows that the **state of Bob's qubit depends on the measurement outcome of Alice's qubits.**

In other words, the equation says:

- If Alice later measures 00, Bob has:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

- If Alice later measures 01, Bob has:

$$\beta|0\rangle + \alpha|1\rangle$$

- If Alice later measures 10, Bob has:

$$\alpha|0\rangle - \beta|1\rangle$$

- If Alice later measures 11, Bob has:

$$-\beta|0\rangle + \alpha|1\rangle$$

So Bob does not always have exactly the state $|\psi\rangle$, but he always has a state related to $|\psi\rangle$ by a simple Pauli gate.

In summary, **Alice's two operations have transformed the total system so that Bob's qubit is now 1 of 4 known variants of the original unknown state.** The next section explains how Alice's two classical bits tell Bob which correction to apply.

5.5.4 Measurement Outcomes and Bob's Corrections

Everything so far was just preparation. This is where the teleportation suddenly becomes interesting.

Remember where we left off in the previous section: Alice has a composite system of two qubits, the first one being the qubit she wants to teleport and the second one being her half of the shared Bell pair. She has just applied a CNOT gate and a Hadamard gate to her two qubits:

$$\begin{aligned} |\Psi_H\rangle = |\Psi_2\rangle = \frac{1}{2} [& |00\rangle \otimes (\alpha|0\rangle + \beta|1\rangle) + \\ & |01\rangle \otimes (\beta|0\rangle + \alpha|1\rangle) + \\ & |10\rangle \otimes (\alpha|0\rangle - \beta|1\rangle) + \\ & |11\rangle \otimes (-\beta|0\rangle + \alpha|1\rangle)] \end{aligned}$$

Before Alice measures her two qubits, **all four branches coexist in superposition**.

We have also seen that after measurement, **only one** of the four branches will **survive**, and the other three will be destroyed.

Let's imagine Alice **measures her two qubits and gets the outcome $|00\rangle$** . In this case, the state immediately becomes:

$$|\Psi_3\rangle = |00\rangle (\alpha|0\rangle + \beta|1\rangle)$$

Now look only at Bob's qubit, which is the last one in the tensor product. It is exactly the **same as the original qubit that Alice wanted to teleport** (state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$). In other words, Alice only needs to send 00 to Bob, and he will know that his qubit is **already in the correct state** (or formally, he can apply the identity operator to it, which does nothing).

❓ What if Alice measures a different outcome?

Let's complicate things a bit. Suppose Alice measures a different outcome:

- If Alice measures $|01\rangle$, the state becomes:

$$|\Psi_3\rangle = |01\rangle (\beta|0\rangle + \alpha|1\rangle)$$

In this case, the amplitudes of Bob's qubit are swapped:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \longrightarrow |\psi'\rangle = \beta|0\rangle + \alpha|1\rangle$$

Therefore, Bob needs to apply an operator that swaps the amplitudes back to their original order. We know that the **Pauli-X gate swaps the amplitudes of a qubit, so Bob can apply it to his qubit to recover the original state**:

$$|\psi'\rangle = \beta|0\rangle + \alpha|1\rangle \xrightarrow{X} |\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

So, if Alice measures $|01\rangle$, she sends 01 to Bob, and he knows that he needs to apply the Pauli-X gate to his qubit to recover the original state.

- If Alice measures $|10\rangle$, the state becomes:

$$|\Psi_3\rangle = |10\rangle (\alpha |0\rangle - \beta |1\rangle)$$

In this case, Bob's qubit has the same amplitudes as the original state, but the amplitude of $|1\rangle$ has a negative sign. Therefore, Bob needs to apply an operator that flips the sign of the amplitude of $|1\rangle$.

For this purpose, we can use the **Pauli-Z gate**, which flips the sign of the amplitude of $|1\rangle$:

$$|\psi'\rangle = \alpha |0\rangle - \beta |1\rangle \xrightarrow{Z} |\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

So, if Alice measures $|10\rangle$, she sends 10 to Bob, and he knows that he **needs to apply the Pauli-Z gate to his qubit to recover the original state.**

- Finally, if Alice measures $|11\rangle$, the state becomes:

$$|\Psi_3\rangle = |11\rangle (-\beta |0\rangle + \alpha |1\rangle)$$

In this case, Bob's qubit has the amplitudes swapped and the amplitude of $|0\rangle$ has a negative sign. Therefore, Bob needs to apply an operator that swaps the amplitudes and flips the sign of the amplitude of $|0\rangle$.

To fix this issue, Bob can choose two strategies:

- **Apply the Pauli-X gate first to swap the amplitudes, and then apply the Pauli-Z gate to flip the sign of the amplitude of $|0\rangle$** (or vice versa, since the two gates commute):

$$|\psi'\rangle = -\beta |0\rangle + \alpha |1\rangle \xrightarrow{X} \alpha |0\rangle - \beta |1\rangle \xrightarrow{Z} |\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

- Or simply (cleanly) **apply the Pauli-Y gate**, which swaps the amplitudes and flips the sign of the amplitude of $|0\rangle$ in one step:

$$\begin{aligned} |\psi\rangle &= Y |\psi'\rangle \\ &= Y (-\beta |0\rangle + \alpha |1\rangle) \\ \downarrow \text{Recall: } Y |0\rangle &= i |1\rangle, \quad Y |1\rangle = -i |0\rangle \\ &= -\beta (i |1\rangle) + \alpha (-i |0\rangle) \\ &= -\beta (i |1\rangle) - \alpha (i |0\rangle) \\ &= -i (\underbrace{\alpha |0\rangle + \beta |1\rangle}_{|\psi\rangle}) \\ \downarrow \text{Global phase } -i &\text{ can be physically ignored} \\ &= \alpha |0\rangle + \beta |1\rangle \end{aligned}$$

At the beginning of the protocol we might expect: “*Bob has absolutely no information about the state of his qubit, so how can he possibly know what to do?*” But after Alice performs **only two gates**, Bob's qubit is **already one of four versions of the correct state**. The quantum information has already reached Bob, but the only thing still missing is **which of the four versions is it?** And that's all the two classical bits communicate. They do **not** carry the amplitudes α and β . They only answer the question: “*Bob, which correction should you apply?*”.

5.5.5 Why Teleportation Works

Now that we've completed the mathematics, this last section should answer a single question: *why didn't we break any law of quantum mechanics?* Let's revisit each principle.

✔ The quantum state is transferred, not copied

We could think that we copied the state of Alice's qubit to Bob's qubit:

$$\text{Alice: } |\psi\rangle \quad \text{Bob: } |0\rangle \quad \xrightarrow{\text{teleportation}} \quad \text{Alice: } |\psi\rangle \quad \text{Bob: } |\psi\rangle$$

But we didn't. Teleportation does **not** do this, instead it destroys the state of Alice's qubit and transfers it to Bob's qubit:

$$\text{Alice: } |\psi\rangle \quad \text{Bob: Bell qubit} \quad \xrightarrow{\text{teleportation}} \quad \text{Alice: (measured)} \quad \text{Bob: } |\psi\rangle$$

Alice no longer possesses the state $|\psi\rangle$, and Bob does. The state has been **moved**, not duplicated. This is why teleportation **does not violate the no-cloning theorem**.

❓ Why are the two classical bits necessary?

Suppose Alice refuses to send the message to Bob. Then Bob waits forever because he only knows that he has a Bell qubit, which is one of the four possible states: $|\psi\rangle$, $X|\psi\rangle$, $Z|\psi\rangle$, or $XZ|\psi\rangle$. But he has no idea which one. He cannot guess, nor can he measure his qubit to find out, because measuring it would destroy the state. He is stuck.

Only when Alice sends 00, 01, 10, or 11 can Bob apply the correct Pauli gate to recover the state $|\psi\rangle$. This is why **the two classical bits are necessary**. They do not encode the amplitudes α and β , but they encode the **correction** that Bob must apply to his qubit to recover the state $|\psi\rangle$.

🌀 Why doesn't this allow faster-than-light communication?

At first glance it looks like something magical happened: Alice measures; instantly Bob's qubit becomes 1 of the 4 states. ***Could Alice send information faster than light?*** The answer is obviously no.

Imagine Alice measures her qubit. Bob immediately looks at his own qubit. Can he tell whether Alice obtained 00 or 11? Or 01 or 10? No, he cannot. Can he determine whether he should apply gate X , Y or Z , or nothing? No, he cannot. Thus, can he recover the original state $|\psi\rangle$ without Alice? No! He must wait for the classical message from Alice. This is why **teleportation does not allow faster-than-light communication**. Classical communication is still necessary to complete the teleportation process, and it cannot travel faster than light.

▲ The Bell pair is a unique resource

At the beginning we had the resource:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$$

One might ask: “*can Alice and Bob teleport another qubit using the same Bell pair?*”. The answer is **no**. After Alice’s measurement, the entanglement is gone. The Bell pair has been “*spent*” and cannot be reused. If Alice and Bob want to teleport another qubit, they must first create a **new Bell pair**. It is a sort of battery: before teleportation it is fully charged (shared entanglement); during teleportation its energy is used; after teleportation it is discharged and cannot be reused. This is why **the Bell pair is a unique resource**.

❓ If we still have to send classical information, what’s the point of teleportation?

Let’s imagine the classical world. Suppose Alice wants Bob to have $|\psi\rangle$. Could she simply send him the values α and β over the internet? No, because **she doesn’t know them**. Remember that Alice never learns α , β , because if she did, the state would collapse and she would lose the information.

Now, let’s think about this more practically. Suppose Alice has a quantum state:

$$|\psi\rangle = 0.834|0\rangle + 0.551e^{i0.42}|1\rangle$$

How would Alice tell Bob this? She would have to send $\alpha = 0.834$ and $\beta = 0.551e^{i0.42}$, but she cannot because the qubit is unknown to her. She has only one copy of the qubit, so measuring destroys it. So classical communication alone is useless.

The advantage is **not faster communication**. It is something much stranger: quantum mechanics lets us transfer an **unknown quantum state** from one qubit to another, without ever knowing what the state is. This is something that is impossible in classical physics.

❓ **Where is this useful?** If teleportation were only a way of sending qubits from one room to another, it would be an expensive curiosity. Its real applications are in **quantum networks**.

Imagine a situation where there are two quantum computers, one in Milan (Quantum Computer A) and one in New York (Quantum Computer B). They share a Bell pair. Suppose computer A has produced a qubit during some computation. Instead of physically transporting the fragile qubit through a noisy optical fiber, it can teleport its state to quantum computer B. This is one of the building blocks of the future **quantum internet**.

How two quantum computers can share a Bell pair is a topic beyond the scope of this course. In practice, one half of an entangled Bell pair can be physically distributed to a remote quantum computer using quantum communication

channels, such as optical fibers or satellite links. Establishing and maintaining long-distance entanglement is one of the central challenges in the development of the future quantum internet and remains an active area of research.

6 Transpiling

6.1 Quantum Fidelity

6.1.1 Why Quantum Computers Are Noisy

In all the previous chapters, we implicitly assumed that quantum gates exactly as their mathematical definitions. For example:

- Hadamard gate:

$$H|0\rangle = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Behaves as a perfect beam splitter, splitting the input state into an equal superposition of $|0\rangle$ and $|1\rangle$.

- Pauli-X gate:

$$X|0\rangle = |1\rangle$$

Behaves as a perfect NOT gate, flipping the state of the qubit from $|0\rangle$ to $|1\rangle$.

- CNOT gate:

$$\begin{aligned} \text{CNOT}|00\rangle &= |00\rangle, & \text{CNOT}|01\rangle &= |01\rangle \\ \text{CNOT}|10\rangle &= |11\rangle, & \text{CNOT}|11\rangle &= |10\rangle \end{aligned}$$

Behaves as a perfect Controlled-NOT gate, flipping the target qubit if the control qubit is in the state $|1\rangle$ and leaving it unchanged if the control qubit is in the state $|0\rangle$.

Mathematically these transformations **are exact**. However, in a real quantum computer, gates are implemented by physical hardware:

- Microwave pulses (superconducting qubits);
- Laser pulses (trapped ions);
- Electromagnetic fields;
- Control electronics;

Because the **hardware is imperfect** (e.g., due to manufacturing defects, environmental noise, or limitations in control precision), the **resulting operation is only an approximation of the ideal gate**. As a consequence:

$$U_{\text{physical}} \neq U_{\text{ideal}} \xrightarrow{\text{better}} U_{\text{physical}} \approx U_{\text{ideal}}$$

This discrepancy leads to **errors** in the quantum computation, which can accumulate over multiple gates and degrade the performance of quantum algorithms.

▲ The Two Main Sources of Noise

- **Gate Infidelity.** Gate infidelity means that the gate physically applied by the hardware is not exactly the gate we intended to execute.

Suppose we want to apply a Hadamard gate:

$$U_{\text{ideal}} = H$$

Ideally, we would like the physical implementation to match the ideal gate:

$$H|0\rangle = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

However, due to **imperfections** like inaccurate control pulses, calibration errors, electronic noise, manufacturing imperfections, the hardware may actually apply:

$$U_{\text{physical}} = H + \varepsilon$$

Where ε is a **small error**. The resulting state is therefore slightly different from the desired one. After thousands of gates, these small errors can **accumulate** and significantly affect the final result.

❓ **Why Multi-Qubit Gates Are Worse?** Intuitively, multi-qubit gates (like CNOT) are **more complex to implement** than single-qubit gates (like Hadamard) because they require precise control over interactions between multiple qubits. For this reason, the **fidelity of multi-qubit gates is typically lower** than that of single-qubit gates:

$$\text{Fidelity}_{\text{CNOT}} < \text{Fidelity}_{\text{Hadamard}}$$

This is one reason why transpilers try to minimize the number of CNOTs and other multi-qubit operations. However, we will understand this in more detail in the next chapters.

- **Decoherence.** A qubit is never perfectly isolated from the surrounding environment. Even if we do not intentionally measure it, the environment continuously interacts with it: electromagnetic radiation, thermal fluctuations, vibrations, surrounding particles, etc. These **interactions gradually destroy the quantum properties of the qubit**. In summary, decoherence is the process by which a quantum system progressively loses its “*quantumness*” and starts behaving more like a classical object (e.g., a bit instead of a qubit).

For example, imagine preparing a qubit in the state $|+\rangle$:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

Initially the qubit exhibits quantum superposition, interference, and phase relationships. However, as time passes, interactions with the environment disturb the phase information. Eventually, the qubit behaves more like a classical probabilistic bit than a true quantum state. The information

is not necessarily destroyed instantly, but the quantum coherence gradually leaks into the environment, making it harder to maintain the desired quantum state over time.

⚠ Why is Decoherence a serious problem? A quantum algorithm often requires many sequential operations:

$$U_n U_{n-1} \cdots U_2 U_1$$

Where each operation takes a certain amount of time to execute. If the algorithm runs for too long, decoherence may occur before the computation finishes. **When decoherence becomes dominant**, the superposition is lost, the interference is lost, the entanglement is lost, and the **quantum advantage disappears**.

Hardware Limitations

Both gate infidelity and decoherence originate from the same fundamental fact: **quantum computers are physical devices**. Unlike an ideal mathematical model, qubits are not perfectly isolated, gates are not perfectly accurate, and measurements are not perfectly reliable. Therefore, every real quantum computer has a finite error rate, coherence time, and computational depth. These limitations ultimately restrict how deep a quantum circuit can be before errors become overwhelming.

In summary, quantum computers are noisy because the **gates are imperfectly implemented** and the **qubits decohere over time**.

6.1.2 Gate Infidelity

In the mathematical model of quantum computing, quantum gates are represented by ideal unitary matrices that transform quantum states exactly as predicted by theory.

For example, the Hadamard gate is defined as:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

And when applied to the $|0\rangle$ state, it always produces the $|+\rangle$ state:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle$$

In theory, this transformation is **exact**.

Real quantum computers, however, do not manipulate abstract vectors and matrices. They manipulate physical systems through microwave pulses, laser pulses, electromagnetic fields, and control electronics. As a consequence, the operation implemented by the hardware is never perfectly identical to the ideal gate (as we have seen in the previous section).

Ideal Gate vs Physical Gate

It is useful to distinguish between:

- **Ideal gate**: the mathematical operation specified by the algorithm;
- **Physical gate**: the operation actually implemented by the hardware.

Ideally we would like:

$$U_{\text{physical}} = U_{\text{ideal}}$$

But in practice we have:

$$U_{\text{physical}} \approx U_{\text{ideal}}$$

But there is always a small discrepancy due to imperfections in the hardware. This **difference between the ideal and physical implementation** is called **Gate Infidelity**.

6.1.3 Decoherence

In the previous section, we introduced decoherence as the gradual loss of the quantum properties of a qubit due to interactions with the surrounding environment (page 364). Now, the natural question is: *how does decoherence actually modify a quantum state?* To understand this, we need to introduce two types of errors, because decoherence can manifest itself in different ways.

6.1.3.1 Phase-Flip Errors

One common consequence of decoherence is **Dephasing**, also called **Loss of Phase**.

Consider the qubit:

$$|\psi\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle$$

The probabilities are:

$$P(0) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2} \quad P(1) = \left| \frac{1}{\sqrt{2}} \right|^2 = \frac{1}{2}$$

The important observation is that a quantum state contains more information than probabilities. It **also contains the relative phase between the amplitudes**. For example, the states:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

Have exactly the same probabilities:

$$P(0) = P(1) = \frac{1}{2}$$

However, they **behave differently** in interference experiments because their **relative phase is different**.

❓ What does Dephasing do? And, **⚠️ why is it hard to notice?** During dephasing, the **probabilities remain unchanged**, but the **relative phase becomes randomized**. This means that:

- **The coherence between the states is lost.**

❓ What is Coherence? A quantum state is **coherent** if the **different components** of the superposition **maintain a fixed phase relationship**.

For example:

$$|\psi\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad |\psi\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

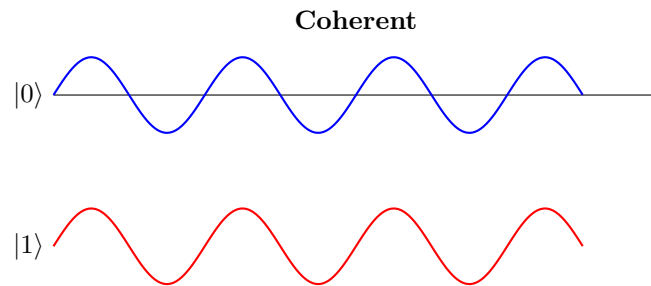
Are coherent states because the relative phase between $|0\rangle$ and $|1\rangle$ is well-defined. Also, the state:

$$|\psi\rangle = \sqrt{0.8}|0\rangle + e^{i\pi/3}\sqrt{0.2}|1\rangle$$

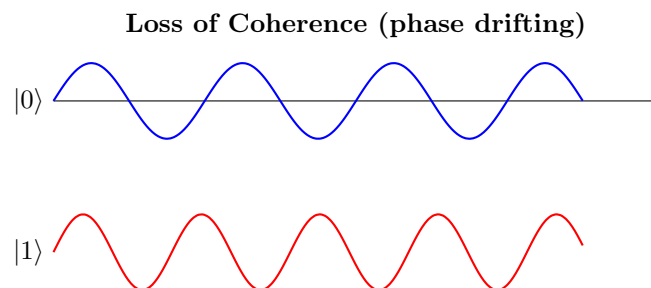
Is also coherent because the relative phase between $|0\rangle$ and $|1\rangle$ is well-defined. The **amplitudes don't matter** for coherence, the important thing is that the **phase relationship is known and stable**.

💡 **Wave analogy for coherence.** The following figures should not be interpreted as the actual amplitudes of a qubit. They are only a visual analogy. The two waves represent the two components of a quantum superposition and illustrate whether their relative phase relationship remains stable over time.

✅ **Coherent state.** In a coherent quantum state, the relative phase between the **amplitudes remains well-defined and stable**. In the figure, the two waves remain perfectly synchronized: peaks align with peaks and valleys align with valleys. Although the amplitudes may evolve, their phase relationship is preserved. This stable phase relationship is what allows quantum interference to occur.

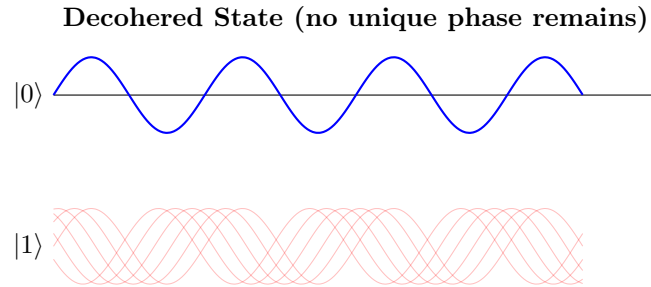


⚠️ **Loss of Coherence.** When a qubit interacts with its environment, the relative phase between the amplitudes gradually becomes uncertain. In the figure, the **two waves lose their synchronization over time**. The peaks and valleys no longer align, representing the loss of a well-defined phase relationship. This **gradual loss of phase synchronization is called dephasing** and is one of the main mechanisms responsible for decoherence. Once coherence is lost, interference effects disappear and the quantum state behaves more like a classical probabilistic system.



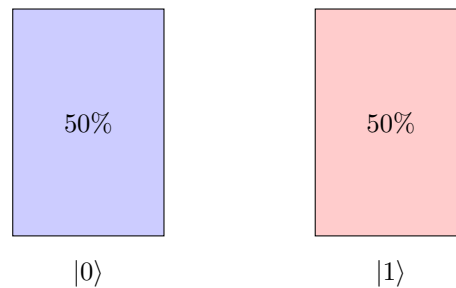
⚠️ **Complete loss of coherence.** If the dephasing process continues, the **relative phase becomes completely randomized**. In the figure, the two waves are no longer synchronized at all, and there is no stable phase relationship between them. This represents a fully decohered state, where the phase information has been completely lost. The system

can no longer exploit quantum interference and therefore behaves similarly to a classical probabilistic mixture.



⚠ Classical Mixture. After **complete decoherence**, all **phase information is lost**, and the **quantum state can be described as a classical mixture of states**. In this case, the system behaves as if it were in state $|0\rangle$ with probability 50% and in state $|1\rangle$ with probability 50%. The state no longer exhibits any quantum interference effects and can be treated as a classical probabilistic system. A great analogy is a coin that before the decoherence process was in a superposition of heads and tails, but after decoherence, it behaves like a classical coin that is either heads or tails with certain probabilities.

Classical Mixture



Notice that this is **fundamentally different from the coherent state**:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}$$

Which also produces 50% probabilities when measured. The difference is that the **coherent state contains phase information and can exhibit interference**, whereas the classical mixture contains only probabilities.

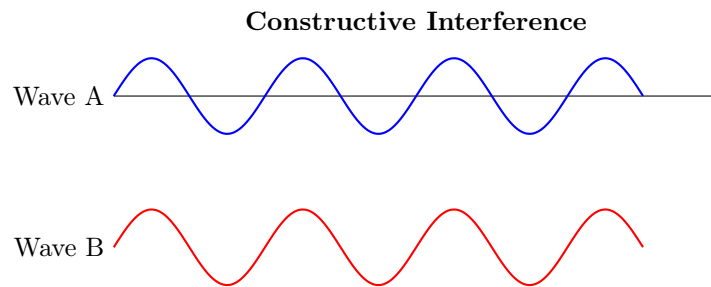
In summary, a coherent quantum state and a classical probabilistic mixture **may produce the same measurement probabilities**, but **only the coherent state contains phase information and can therefore exhibit interference effects**.

- **Interference effects disappear.**

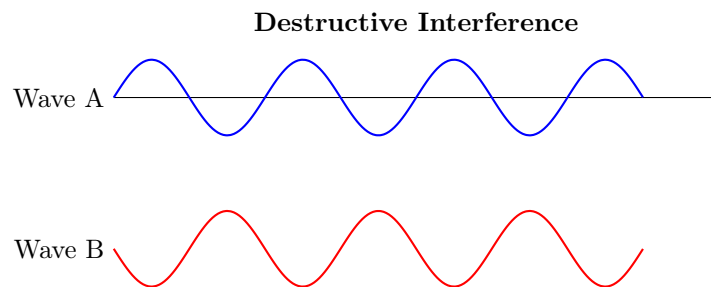
❓ What is Interference? In the previous point, we mentioned that the loss of coherence leads to the disappearance of interference effects. *But what exactly are interference effects?* **Interference** is the **phenomenon by which quantum amplitudes combine**.

Unlike classical probabilities, which simply add together, quantum amplitudes can be positive, negative or complex. Therefore, when **amplitudes are added** together, they can either **reinforce each other** (called **Constructive Interference**) or **cancel each other out** (called **Destructive Interference**).

An example of constructive interference is shown in the figure below. The two waves represent the two components of a quantum superposition. When they are in phase (peaks align with peaks and valleys align with valleys), their amplitudes reinforce each other, leading to a stronger overall amplitude, and therefore a higher probability of certain measurement outcomes (because probabilities are obtained by taking the square of the amplitude).



On the other hand, when the two waves are out of phase (peaks align with valleys), their amplitudes cancel each other out, leading to a weaker overall amplitude, and therefore a lower probability of certain measurement outcomes.



⚠️ Decoherence and interference. Decoherence does not necessarily destroy the probabilities immediately. Instead, it destroys the **phase information**. Without phase information, interference effects disappear, and the quantum state behaves more like a classical probabilistic system.

- **The state becomes more classical.** As coherence is lost and interference effects disappear, the quantum state gradually behaves more like a classical probabilistic mixture. In the limit of complete decoherence, the state can be described as a classical mixture of states, where the system is in one state with a certain probability and in another state with another probability.

Dephasing is **invisible in the computational basis** because the probabilities of measuring $|0\rangle$ or $|1\rangle$ remain unchanged. However, the damage only becomes visible when the algorithm relies on interference effects, which require coherence. This is one reason why decoherence is so dangerous. It can silently destroy the relative phases of a state without affecting the measurement probabilities in the computational basis, making it **difficult to detect until it's too late**.

❓ Which mathematical error changes the phase of a quantum state?

In other words, *which mathematical error corresponds to dephasing?*

The answer is the Pauli-Z gate:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Which acts as follows:

$$Z|0\rangle = |0\rangle \quad Z|1\rangle = -|1\rangle$$

Where the minus sign in front of $|1\rangle$ represents a phase shift of π (or 180 degrees). It is more intuitive to see the effect of the Z gate on the Bloch sphere, page 58. However, with Pauli-Z, the probabilities of measuring $|0\rangle$ or $|1\rangle$ remain unchanged, but the relative phase between the two states is flipped. This is why it's called a **Phase-Flip Error**.

⚠️ Why is it dangerous? Per se, the Pauli-Z gate is not dangerous, and a **single application does not destroy coherence**. For example, applying a Pauli-Z gate transforms $(|+\rangle)$ into $(|-\rangle)$, but the resulting state is still perfectly coherent.

However, in a real quantum computer the environment does not apply one neat Pauli-Z gate. Instead, interactions with the environment introduce many random phase perturbations over time. These random phase changes can be modeled as phase-flip (Z) errors. As they accumulate, the relative phase between the components of a superposition becomes increasingly uncertain, producing **dephasing**. The consequence is the loss of coherence and, therefore, the disappearance of the interference effects on which quantum algorithms rely.

❓ If the Pauli-Z gate is a valid quantum operation, why is it associated with errors? Let's clarify this point with an analogy. For example, when we write $\text{NOT}(0) = 1$, there is nothing wrong with the NOT operation. The gate is doing exactly what it's supposed to do. However, suppose our program says to store the value 0 in a variable, but electrical noise flips the bit to 1. Now the bit has undergone a NOT operation, but this is not what we intended. The NOT operation is a valid operation, but in this context, it represents an error because

it changes the value in an unintended way. Exactly the same thing happens in quantum computing.

Suppose our algorithm contains $Z|+\rangle = |-\rangle$. We intentionally changed the phase, and everything is correct. However, suppose our algorithm says “*do nothing*”, but the environment interacts with the qubit and changes its phase. The effect on the state is $|+\rangle \rightarrow |-\rangle$, which is exactly what a Z gate would have done. Now, we call this a **phase-flip error** because the phase changed even though we never requested that operation. The Z gate is a valid quantum operation, but in this context, it represents an error because it changes the phase in an unintended way.

6.1.3.2 Bit-Flip Errors

While dephasing affects the **relative phase** of a quantum state, amplitude damping affects the **probabilities (populations)** of the basis states. In practice, **Amplitude Damping** occurs when the **qubit exchanges energy with the environment**. For example, initially the state:

$$|\psi\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} = |+\rangle$$

With probabilities:

$$P(0) = P(1) = \frac{1}{2}$$

After amplitude damping, the state might evolve into:

$$|\psi\rangle = \sqrt{0.8}|0\rangle + \sqrt{0.2}|1\rangle$$

With probabilities:

$$P(0) = 0.8 \quad P(1) = 0.2$$

The population of $|1\rangle$ has decreased. This is amplitude damping. Specifically, this **form of amplitude damping** is called **Energy Relaxation** because the qubit loses energy to the environment. In this case, the state tends to relax towards the ground state $|0\rangle$.

❓ Why does Energy Relaxation occur? Energy relaxation occurs because the qubit is not perfectly isolated from its surroundings. The qubit can interact with the environment, exchanging energy in the process. For example, if the qubit is implemented using a superconducting circuit, it can lose energy by emitting photons into the surrounding electromagnetic field. This process causes the excited state $|1\rangle$ to decay towards the ground state $|0\rangle$, leading to amplitude damping.

⚠️ Another type of amplitude damping: Thermal Excitation. In some cases, the **qubit can also gain energy from the environment**, leading to **Thermal Excitation**. For example, if the **qubit is in a warm environment** (i.e., at a higher temperature), it can **absorb thermal energy** and transition from the ground state $|0\rangle$ to the excited state $|1\rangle$. This process also **leads to amplitude damping because it changes the population of the basis states**.

❓ Where does the Bit-Flip come from? Suppose Energy Relaxation occurs:

$$|1\rangle \rightarrow |0\rangle$$

What happened is that the logical value changed from 1 to 0. Instead, suppose Thermal Excitation occurs:

$$|0\rangle \rightarrow |1\rangle$$

Now the logical value changed from 0 to 1. In both cases, the logical value of the qubit has flipped. This is exactly what the Pauli- X gate does:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Which acts as follows:

$$X |0\rangle = |1\rangle \quad X |1\rangle = |0\rangle$$

Therefore, we can model amplitude damping as a **Bit-Flip Error** because it changes the logical value of the qubit. Just like with phase-flip errors, a single application of the X gate does not necessarily destroy the quantum state, but in a real quantum computer, interactions with the environment can introduce many random bit-flip errors over time, leading to significant amplitude damping and loss of quantum information.

⚠ Important Caveat. Amplitude damping is not literally a bit-flip; we use **bit-flips as a convenient mathematical model**. A Pauli- X gate is a perfect bit-flip and treats the two states symmetrically $|0\rangle \iff |1\rangle$. Same behavior in both directions. Instead, the nature of amplitude damping is asymmetric. Usually, the qubit $|1\rangle$ tends to decay towards $|0\rangle$ (energy relaxation), and the reverse process (thermal excitation) is less common. Because $|1\rangle$ is typically the **higher-energy state**, while $|0\rangle$ is the **lower-energy state**, and the qubit naturally tends to lose energy to the environment rather than gain it. Therefore, while we can model amplitude damping as a bit-flip error for simplicity, it's important to remember that the underlying physical processes are more complex and not perfectly symmetric.

⚠ Important Caveat. Amplitude damping is not literally a bit-flip; rather, we use **bit-flips as a convenient mathematical model**. A Pauli- X gate is a perfect bit-flip and treats the two states symmetrically:

$$|0\rangle \leftrightarrow |1\rangle$$

In other words, the behavior is identical in both directions. Amplitude damping, however, is generally asymmetric. Usually, the qubit $|1\rangle$ tends to decay towards $|0\rangle$ (energy relaxation), while the reverse process $|0\rangle \rightarrow |1\rangle$ (thermal excitation) is less likely. This happens because $|1\rangle$ is typically the **higher-energy state**, whereas $|0\rangle$ is the **lower-energy state**. Consequently, a qubit naturally tends to lose energy to the environment rather than gain it. Therefore, although amplitude damping is often modeled and corrected as an X -type (bit-flip) error, the underlying physical mechanism is more complex and not perfectly symmetric. In summary, a bit-flip describes *what happens* to the logical value of the qubit, while amplitude damping describes *why it happens* physically.

6.1.3.3 Generic Errors

So far we have discussed two idealized errors:

- Phase-flip errors (Pauli- Z) that flip the phase of a qubit (page 368);
- Bit-flip errors (Pauli- X) that flip the state of a qubit (page 374).

However, real quantum hardware is messy (i.e., noisy) and a qubit may experience electromagnetic noise, thermal fluctuations, imperfect control pulses and interactions with the environment. Therefore, the resulting **disturbance** is generally **not** a **perfect phase-flip or bit-flip error**. Instead, the error can be some arbitrary transformation.

To describe this situation, we introduce an **Error Operator** E that acts on the qubit exactly like a **quantum gate**:

$$|\psi\rangle \rightarrow E|\psi\rangle \quad (130)$$

But with the crucial difference that E is **unwanted**. At first glance this *seems* disastrous because there are infinitely many possible error operators. Therefore, *how could we possibly correct all of them?*

✔ Decomposition into Pauli Operators

The key insight is that **every** 2×2 complex matrix can be written as a linear combination of four special matrices:

$$I, \quad X, \quad Y, \quad Z$$

These matrices form a basis for the space of single-qubit operators (page 50). So **any error operator** E **can be written as**:

$$E = \alpha_0 I + \alpha_1 X + \alpha_2 Y + \alpha_3 Z$$

Where the **coefficients** α_i with $i = 0, 1, 2, 3$ are complex numbers that **depend on the specific physical error**.

Theorem 2 (Pauli Basis Decomposition). *The set:*

$$\{I, X, Y, Z\}$$

Forms a basis for the vector space of all 2×2 complex matrices. Therefore, any single-qubit operator E can be uniquely written as:

$$E = \alpha_0 I + \alpha_1 X + \alpha_2 Y + \alpha_3 Z \quad (131)$$

Where $\alpha_0, \alpha_1, \alpha_2, \alpha_3 \in \mathbb{C}$ are complex coefficients that depend on the specific operator E .

❓ Meaning of Each Term

Each term in the decomposition has a clear physical interpretation:

- **Identity term** (I): represents the case where no error occurs, and the qubit remains unchanged.

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- **Bit-flip term** (X): represents the probability amplitude of a bit-flip error, which flips the state of the qubit from $|0\rangle$ to $|1\rangle$ and vice versa (page 374).

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

- **Phase-flip term** (Z): represents the probability amplitude of a phase-flip error, which flips the phase of the qubit from $|1\rangle$ to $-|1\rangle$ while leaving $|0\rangle$ unchanged (page 368).

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

- **Combined bit-and-phase-flip term** (Y): represents the probability amplitude of an error that combines both bit-flip and phase-flip effects, flipping the state of the qubit and changing its phase simultaneously, $Y = iXZ$.

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$$

✅ Why Correcting X and Z Errors is Sufficient

Since Y can be expressed as a combination of X and Z (i.e., $Y = iXZ$), if we can correct for X and Z errors, we can also correct for Y errors. Therefore, **correcting for X and Z errors is sufficient to correct for any arbitrary error E** . This is a fundamental principle in quantum error correction, allowing us to design codes that can protect against a wide range of errors by focusing on correcting just these two types of errors. Depending on the source, the related theorem has many names. However, the important point is that **if a quantum error-correcting code can correct all Pauli errors, then it can correct any arbitrary single-qubit error**. We will discuss quantum error-correcting codes in future sections.

Theorem 3 (Discretization Pauli Error, Error Discretization, Pauli Error Expansion, Digitization of Quantum Errors). *Let Equation 131:*

$$E = \alpha_0 I + \alpha_1 X + \alpha_2 Y + \alpha_3 Z$$

Be an arbitrary error acting on a physical qubit. If a QECC (quantum error-correcting code) can correct each Pauli operator I , X , Y and Z , individually, then it can correct the arbitrary error E as well.

6.1.4 Fidelity of a Quantum Gate

We have defined the types of errors that can occur in quantum gates. But *how can we measure how well a quantum gate performs?* The **fidelity** of a quantum gate is the answer to this question. It is a measure of how close the actual operation performed by the gate is to the ideal operation.

🔍 Motivation

In an ideal quantum computer, every gate would be implemented perfectly. If we want to apply a gate U_I (where I stands for “*ideal*”), then the **output should be exactly**:

$$y_I = U_I x \quad (132)$$

Where x is the input qubit and the y_I is the *ideal* output. Unfortunately, real hardware is noisy (as we have seen in the previous sections). The gate physically implemented on the processor is slightly different from the ideal one: $U_I \neq U_P$ (where P stands for “*physical*”). Therefore, the **actual output** is:

$$y_P = U_P x \quad (133)$$

This output may differ from the ideal one due to imperfect control electronics, calibration errors, decoherence, and other sources of noise like environmental interactions. Given this situation, we ask ourselves: **how close is y_P to y_I ?** How closely does the physical output y_P produced by the real gate U_P resemble the ideal output y_I we would get from a perfect gate U_I ? We introduce the **fidelity** to answer this question.

📖 Definition 1: Fidelity

The **Fidelity** of a quantum gate is a measure of **how closely the actual operation performed by the gate matches the ideal operation**. It is defined as:

$$F = |\langle y_I | y_P \rangle|^2 \quad (134)$$

Where $|y_I\rangle$ is the ideal output state and $|y_P\rangle$ is the actual output state produced by the physical gate. We use the inner product $\langle y_I | y_P \rangle$ to quantify the **overlap between the two states**, and the **square** of its magnitude gives us a **value between 0 and 1** that represents the fidelity. The fidelity ranges from 0 to 1, where:

- $F = 1$ means the **actual output perfectly matches the ideal output** (i.e., the **gate is perfect**).
- $F = 0$ means the **actual output is completely orthogonal to the ideal output** (i.e., the **gate is completely wrong**).
- $0 < F < 1$ indicates that the **actual output is partially close** to the ideal output, with higher values indicating better performance (i.e., the **gate is somewhat good**).

We can also write the fidelity more explicitly in terms of the gates. Using

the Equation 132 and Equation 133:

$$y_I = U_I x \quad \text{and} \quad y_P = U_P x$$

Writing the conjugate transpose of y_I (bra):

$$\langle y_I | = \langle U_I x | \Rightarrow \langle y_I | = \langle x | U_I^\dagger$$

Substituting this into the fidelity definition:

$$F = |\langle y_I | y_P \rangle|^2 = \left| \langle x | U_I^\dagger U_P | x \rangle \right|^2 \quad (135)$$

⚠ Why isn't one fidelity value enough?

Suppose we build a terrible **physical gate**:

$$U_P = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

This gate does nothing to the input state. Now suppose the **ideal gate** should be:

$$U_I = X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

Let's test the fidelity using only $|+\rangle$:

$$U_I |+\rangle = X |+\rangle = |+\rangle$$

Because $|+\rangle$ is an eigenstate of X . Meanwhile:

$$U_P |+\rangle = I |+\rangle = |+\rangle$$

Calculating the fidelity:

$$F = |\langle + | X^\dagger I | + \rangle|^2 = |\langle + | X I | + \rangle|^2 = |\langle + | X | + \rangle|^2 = |\langle + | + \rangle|^2 = 1$$

Perfect fidelity! But we know that U_P is a terrible gate. In fact, if we test the fidelity with $|0\rangle$:

$$U_I |0\rangle = X |0\rangle = |1\rangle$$

While:

$$U_P |0\rangle = I |0\rangle = |0\rangle$$

Calculating the fidelity:

$$F = |\langle 0 | X^\dagger I | 0 \rangle|^2 = |\langle 0 | X I | 0 \rangle|^2 = |\langle 0 | X | 0 \rangle|^2 = |\langle 0 | 1 \rangle|^2 = 0$$

Terrible fidelity! The same gate appears perfect and terrible depending on which input state we choose. So asking “*what is the fidelity of this gate?*” is incomplete, instead we should ask “*for which input state?*”.

✔ The Solution: Average Fidelity

Instead of testing a single input state, we conceptually test **all possible input states**. Remember:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle$$

Can be any point on the Bloch sphere. So rather than computing the fidelity for a single state (Equation 135):

$$F(|\psi\rangle) = \left| \langle\psi| U_P^\dagger U_I |\psi\rangle \right|^2$$

For one specific $|\psi\rangle$, we **average over all possible input states**:

$$F_{\text{avg}} = \int \left| \langle\psi| U_P^\dagger U_I |\psi\rangle \right|^2 d\psi \quad (136)$$

This is the **Average Fidelity** of the quantum gate. The notation $d\psi$ indicates a **uniform average over all possible quantum states**. For a single qubit, every pure state can be represented as a point on the Bloch sphere. Therefore, the average fidelity **measures the performance of the gate over the entire Bloch sphere rather than for a specific input state.**

In other words, we conceptually:

1. Choose a quantum state $|\psi\rangle$;
2. Apply both the ideal gate U_I and the physical gate U_P ;
3. Compute the fidelity;
4. Repeat the process for all possible states on the Bloch sphere;
5. Average the results.

The resulting value provides a single number that characterizes the overall quality of the gate, independently of the particular input state chosen.

❓ **Can we really test all possible states?** In practice, this is **impossible because the Bloch sphere contains infinitely many states**. Fortunately, it can be shown (proof outside the scope of this course) that the average fidelity can be computed directly from the matrices describing the ideal and physical gates:

$$F_{\text{avg}} = \frac{\left| \text{Tr} \left(U_P^\dagger U_I \right) \right|^2 + d}{d(d+1)} \quad (137)$$

Where Tr is the **trace of a matrix**, a sum of the diagonal entries:

$$\text{Tr}(A) = \sum_i A_{ii}$$

And:

$$d = 2^n$$

Is the **dimension of the Hilbert space** and n is the **number of qubits** involved in the gate.

💡 **Meaning of d .** For a single-qubit gate:

$$d = 2^1 = 2$$

Because a qubit lives in a two-dimensional Hilbert space. For a two-qubit gate:

$$d = 2^2 = 4$$

Because the computational basis contains four states:

$$|00\rangle, |01\rangle, |10\rangle, |11\rangle$$

More generally, an n -qubit system has

$$2^n$$

Basis states, hence the dimension of its Hilbert space is

$$d = 2^n$$

✅ **Interpretation.** The average fidelity replaces an impossible average over infinitely many quantum states with a simple calculation involving only the matrices of the ideal and physical gates. This allows hardware vendors and researchers to assign a single fidelity value to a gate and compare different quantum processors.

6.1.5 Fidelity and Decoherence

Imagine we start with a gate that takes time t_{gate} to execute. During this time, decoherence attacks our qubit. Therefore, the longer the gate lasts, the more time decoherence has to damage the qubit's state. **More decoherence means more errors, which means lower fidelity.**

Precisely, the fidelity measures how accurately a physical gate reproduces the ideal gate (page 378). One important source of infidelity is decoherence (page 368). Since a gate requires a **finite execution time** t_{gate} , the qubit is exposed to environmental noise while the gate is running. The longer the gate duration compared to the **decoherence time** t_{dec} , the higher the probability that decoherence introduces errors. Consequently, **longer gates tend to have lower fidelity**. A **simple approximation** is:

$$p_{\text{gate}} \approx \frac{t_{\text{gate}}}{t_{\text{dec}}} \quad (138)$$

Where:

- t_{dec} is the **Decoherence Time**. It measures **how long a qubit can preserve its quantum state before the environment destroys it**. For superconducting qubits, usually the decoherence time is on the order of microseconds (μs):

$$t_{\text{dec}} \approx 200\mu s$$

- t_{gate} is the **Gate Time**. It measures **how long a quantum gate takes to execute**. Generally, X gates take around 20 ns, H gates around 30 ns, and CNOT gates around 300 ns. The important assumption is $t_{\text{gate}} \ll t_{\text{dec}}$, because otherwise the qubit would lose coherence while the gate is still running, leading to very low fidelity.
- p_{gate} is the **Gate Error Probability**. It estimates **the likelihood that a gate operation fails due to decoherence**. The longer the gate duration relative to the decoherence time, the higher the error probability, which reduces the overall fidelity of the quantum operation.

For example, suppose $t_{\text{dec}} = 100\mu s$ and $t_{\text{gate}} = 1\mu s$. Then:

$$p_{\text{gate}} \approx \frac{1\mu s}{100\mu s} = 0.01$$

This means that the gate has a 1% chance of introducing an error due to decoherence, which corresponds to a fidelity of approximately 99%. If we can reduce the gate time to $0.5\mu s$, then:

$$p_{\text{gate}} \approx \frac{0.5\mu s}{100\mu s} = 0.005$$

This means the gate error probability drops to 0.5%, improving the fidelity to approximately 99.5%. Therefore, **optimizing gate times is crucial for enhancing the fidelity of quantum operations.**

☆ **Quality Ratio.** The **Quality Ratio** is a metric that answers the question: *how many gate operations can we perform before decoherence becomes significant?* It is defined as:

$$\text{Quality Ratio} = \frac{t_{\text{dec}}}{t_{\text{gate}}} \quad (139)$$

This is simply the inverse of the gate error probability:

$$\text{Quality Ratio} = \frac{1}{p_{\text{gate}}}$$

A higher quality ratio indicates that we can perform more gate operations before decoherence significantly degrades the qubit's state. For instance, if $t_{\text{dec}} = 100\mu s$ and $t_{\text{gate}} = 1\mu s$, then the quality ratio is:

$$\text{Quality Ratio} = \frac{100\mu s}{1\mu s} = 100$$

This means we can perform approximately 100 gate operations before decoherence significantly impacts the qubit's state. If we reduce the gate time to $0.5\mu s$, the quality ratio increases to:

$$\text{Quality Ratio} = \frac{100\mu s}{0.5\mu s} = 200$$

This means we can perform approximately 200 gate operations before decoherence becomes significant, effectively doubling the number of operations we can execute with high fidelity. Therefore, optimizing gate times not only reduces error probabilities but also increases the quality ratio, allowing for more complex quantum computations before decoherence limits performance. In general, a quality ratio of $10^3 - 10^4$ is the target for fault-tolerant quantum computing, as it allows for sufficient error correction to mitigate the effects of decoherence.

✔ **Updated Fidelity Formula.** The **Fidelity** of a quantum gate can be approximated by considering the gate error probability due to decoherence. If we assume that the only source of infidelity is decoherence, then the fidelity can be expressed as:

$$F_{\text{gate}} \approx 1 - p_{\text{gate}} = 1 - \frac{t_{\text{gate}}}{t_{\text{dec}}} \quad (140)$$

This formula indicates that the fidelity decreases linearly with the ratio of gate time to decoherence time. For example, if $t_{\text{dec}} = 100\mu s$ and $t_{\text{gate}} = 1\mu s$, then the fidelity is:

$$F_{\text{gate}} \approx 1 - \frac{1\mu s}{100\mu s} = 0.99$$

This means the gate has a fidelity of approximately 99%. If we reduce the gate time to $0.5\mu s$, the fidelity improves to:

$$F_{\text{gate}} \approx 1 - \frac{0.5\mu s}{100\mu s} = 0.995$$

This means the gate has a fidelity of approximately 99.5%. Therefore, optimizing gate times is crucial for enhancing the fidelity of quantum operations, as it directly reduces the error probability associated with decoherence.

💡 **Threshold Theorem.** Every physical quantum gate introduces a small probability of error. Without error correction, these errors accumulate and eventually destroy the computation.

The **Threshold Theorem** states that if the physical **error rate** of the quantum hardware is **below a certain threshold value** (equivalently, if the gate **fidelity is sufficiently high**), then **quantum error correction can remove errors faster than they are generated**.

As a consequence, arbitrarily long quantum computations become possible using logical qubits and fault-tolerant protocols, even though the underlying physical qubits are noisy and have finite decoherence times.

For example, if every second 10 errors appear, and we can correct 20 errors per second, then the system stays healthy. But if every second 10 errors appear, and we can only correct 5 errors per second, then the system becomes unhealthy and eventually fails. Therefore, maintaining an error rate below the threshold is crucial for the viability of quantum computing.

6.2 Quantum Error Correction

Before discussing quantum compilation and transpiling, we must understand a fundamental limitation of current quantum hardware. Unlike classical computers, quantum computers are extremely sensitive to noise. Errors continuously affect qubits during computation, making it impossible to execute long quantum algorithms reliably without some form of error correction.

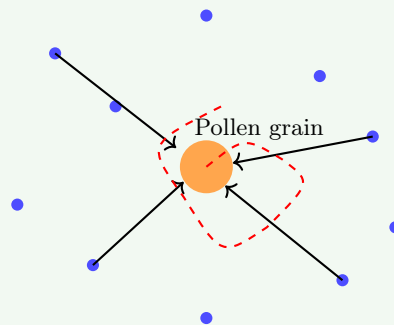
6.2.1 Quantum Computation Errors

Real quantum computers are physical devices and therefore interact with their surrounding environment. Even when carefully isolated, external disturbances can affect the quantum state of a qubit, as discussed in previous sections. Examples of these disturbances include electromagnetic interference, thermal fluctuations, and microscopic vibrations of surrounding particles.

Example 1: Brownian motion

The **Brownian Motion** is the **random motion of particles caused by thermal energy**.

In 1827, the Scottish botanist Robert Brown observed tiny pollen grains suspended in water. Instead of remaining still, they jittered randomly. At the time nobody understood why, but later it was discovered that water molecules were constantly hitting the pollen grain from random directions. The pollen grain was so small that these microscopic collisions became visible.



Random Brownian trajectory

Illustration of Brownian motion. A microscopic pollen grain suspended in a fluid is continuously struck by surrounding molecules. Because the collisions occur randomly and from different directions, the grain follows an irregular trajectory (red dashed line). Brownian motion provides direct evidence that matter at finite temperature is constantly moving at the microscopic scale.

 **How is linked to thermal energy?** Brownian motion is a direct consequence of the thermal energy of the surrounding environment.

At finite temperature, particles in a fluid are in constant motion due to their thermal energy. This motion leads to random collisions with suspended particles, causing the observed Brownian motion. The intensity of Brownian motion increases with temperature, as higher thermal energy results in more vigorous particle movement and more frequent collisions. Indeed, if we boil the water, the Brownian motion becomes more pronounced as the water molecules move faster and collide more frequently with the pollen grain.

❓ Amazing, but how is it relevant for quantum computing?

The actual **enemy is not Brownian motion** itself, but the fact that the **thermal environment constantly interacting with the qubit**. Brownian motion is simply an intuitive **example showing that matter at finite temperature is never perfectly still, but rather in constant motion**. This means that **qubits are continuously affected by their environment**, leading to errors in quantum computations. Even if we try to isolate the qubits as much as possible, they will still be subject to random disturbances from the surrounding environment, making it impossible to achieve perfect error-free quantum computation.

❓ Why Error Correction is necessary

Quantum algorithms often require millions of gate operations. Without a protection mechanism, continuous interaction with the environment would cause decoherence (page 368) and accumulate gate errors (page 367) during computation, making it **highly unlikely that the correct result would be obtained**.

However, there is hope. The **Threshold Theorem** (page 384) discussed previously provides the curcial insight that reliable **quantum computation is still possible** if errors are corrected continuously and the physical error rate remains below a certain threshold.

For this reason, modern quantum computers cannot rely solely on physical qubits (page 168). Instead, they require **Quantum Error Correction Codes** (QECCs), which encode quantum information redudantly across many physical qubits in order to detect and correct errors before they destroy the computation.

-  Noise → Errors → Decoherence → Computation Failure
-  Noise → Quantum Error Correction → Reliable Computation

6.2.2 Quantum Error Correction Codes (QECC)

After understanding that physical qubits suffer from decoherence and gate errors, we need a **mechanism to preserve quantum information during a computation**. **Quantum Error Correction Codes (QECCs)** achieve this by introducing **redundancy**: instead of storing information in a single physical qubit, the information is distributed across many qubits.

We previously discussed this concept on page 168, where we compared logical and physical qubits. Here, however, we present the concept with more detail because we now have a better understanding of possible errors and their impact on quantum computations.

- A **Physical Qubit** is a **real qubit implemented by hardware**, such as superconducting qubits, trapped-ion qubits or photonic qubits. Unfortunately, physical qubits are directly affected by decoherence (page 368), thermal noise (page 385), and gate imperfections (infidelity, page 367). Therefore, a **physical qubit is unreliable if used alone** for quantum computation, as it is prone to errors that can lead to incorrect results.
- To obtain reliable computation, quantum information is not stored in a single physical qubit. Instead, one creates a **Logical Qubit** $|q_L\rangle$ which is **encoded into many physical qubits**. For instance, a logical qubit can be encoded into 5, 7, or even hundreds of physical qubits, depending on the specific error correction code used.

The user of the quantum computer thinks in terms of logical qubits, while the hardware manipulates the underlying physical qubits.

⚠ How can errors be detected without collapsing the state?

Suppose our logical qubit is encoded into some physical qubits:

$$q_0, q_1, q_2, q_3, q_4$$

An error occurs on one of the physical qubits, say q_2 . However, since the logical qubit is encoded across all five physical qubits, enough information remains in the qubits to recover the original state. The question is, *how do we know that an error occurred and which qubit was affected?*

✖ The first naïve solution could be to measure all the physical qubits to check for errors. However, this approach is flawed because **measuring a qubit collapses its quantum state**, which would destroy the quantum information we are trying to protect.

At this point, we can **exploit the redundancy introduced by the logical encoding**. Consider a simple example where:

$$|0_L\rangle = |000\rangle \quad |1_L\rangle = |111\rangle$$

Where L stands for “logical”. In this encoding, the logical qubit $|0_L\rangle$ is represented by three physical qubits all in the state $|0\rangle$, and the logical qubit $|1_L\rangle$ is

represented by three physical qubits all in the state $|1\rangle$. A generic logical qubit is therefore:

$$|\psi_L\rangle = \alpha |000\rangle + \beta |111\rangle$$

Notice that in a **valid encoded state** all physical qubits agree with each other: they are either all 0 or all 1.

Now, suppose that a **bit-flip error** affects the second physical qubit. The error acts on the entire quantum state:

$$\alpha |000\rangle + \beta |111\rangle \longrightarrow \alpha |010\rangle + \beta |101\rangle$$

In this case, the second qubit has flipped from 0 to 1 in the first term and from 1 to 0 in the second term. The resulting state is **no longer a valid encoded state** because the physical qubits do not all agree with each other.

✔ **Solution: Syndrome.** Rather than asking “*what is the quantum state?*” which would destroy the computation, we only ask: “*are neighbouring qubits equal or different?*”. For instance, we can check if q_0 and q_1 are the same, and if q_1 and q_2 are the same. This way, we can detect that q_2 is different from the others without measuring the actual state of any qubit. In other words, the **pattern of answers identifies the location of the error without revealing the coefficients α and β of the logical qubit.** The collection of these answers is called the **syndrome**.

Definition 2: Syndrome

A **Syndrome** is a classical piece of information that indicates the **presence and type of error without revealing the logical quantum state**. A *syndrome* is obtained by performing a **Syndrome Measurement**, which simply collects information about the error, a sort of “*error diagnosis*”. However, it is a **quantum measurement** and therefore **causes a collapse**. But it **collapses only the error-related degrees of freedom** (the syndrome subspace), not the logical quantum information encoded in the qubit. Therefore the logical state survives while the error information becomes known.

For instance, if we have the logical state:

$$\alpha |0\rangle + \beta |1\rangle$$

The syndrome does **not** tell us α or β , the syndrome only tells us things like “*error detected: yes; location: qubit 5; type: bit-flip*”.

❓ **How can we perform syndrome measurements without collapsing the logical state?**

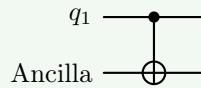
The key insight is that we can perform **indirect measurements** by coupling the data qubits to an additional qubit, called an **Ancilla Qubit**, and then measuring the ancilla. This way, we can extract information about the error

without directly measuring the data qubits, thus preserving the logical quantum state.

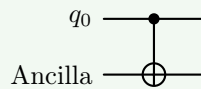
Usually the ancilla qubit starts in a known state, such as $|0\rangle$. We then apply a series of quantum gates that entangle the ancilla with the data qubits in such a way that the state of the ancilla after the interaction contains information about the syndrome.

Example 2: Syndrome Measurement with an Ancilla Qubit

We want to know if q_0 and q_1 are the same or different. We initialize the ancilla qubit in the state $|0\rangle$ and apply a CNOT gate with q_1 as the control and the ancilla as the target:



If q_1 is in the state $|0\rangle$, the ancilla remains in $|0\rangle$. If q_1 is in the state $|1\rangle$, the ancilla flips to $|1\rangle$. Next, we apply another CNOT gate with q_0 as the control and the ancilla as the target:



After these two gates, the state of the ancilla will be $|0\rangle$ if q_0 and q_1 are the same (both $|0\rangle$ or both $|1\rangle$) and $|1\rangle$ if they are different. The final value becomes $\text{Ancilla} = q_0 \oplus q_1$, which is the syndrome for this particular check.

Finally, we **measure the ancilla qubit**. The measurement outcome gives the syndrome, which can be used to determine whether an error occurred and, in many cases, which qubit was affected.

An important observation is that the syndrome does not exist beforehand. The ancilla qubit is the physical system that temporarily stores the syndrome information so that it can be measured without directly measuring the data qubits.

Definition 3: Ancilla Qubit

An **Ancilla Qubit** is an **auxiliary qubit** introduced to assist a quantum computation. In QECC, ancilla qubits are used to interact with data qubits, extract information about errors without collapsing the data qubits' states, and help correct errors. They **do not store the logical quantum information** being protected.

In summary, QECC makes reliable quantum computation possible, but at a **significant cost**. A logical qubit is not implemented by a single physical qubit. Instead, it requires many physical qubits, including both data qubits and ancilla

qubits used for syndrome extraction and error correction.

As a consequence, the number of physical qubits required by a quantum computer grows dramatically. In practice, a **single logical qubit may require tens or even hundreds of physical qubits**. This phenomenon is known as the **Blow-Up Problem**.

Definition 4: Quantum Error Correction Code (QECC)

A **Quantum Error Correction Code (QECC)** is a **technique used to protect quantum information** from noise and decoherence. A logical qubit is encoded into multiple physical qubits, introducing redundancy that allows errors to be detected and corrected.

Error detection is performed through **syndrome measurements**, typically using **ancilla qubits**, so that information about errors can be extracted *without* directly measuring the logical quantum state.

By continuously detecting and correcting errors, QECC enables reliable quantum computations despite the presence of imperfect hardware.

6.2.3 Logical Gates

Originally, before Quantum Error Correction Codes (QECCs), life was easy: physical qubit, apply gate (e.g., Hadamard), get result. After QECC, things are different. A logical qubit is encoded into many physical qubits, and now the question is: “*how do we apply a gate to the logical qubit if the logical qubit is spread across many physical qubits?*”.

Logical Gates

To apply a gate to a logical qubit, we need to define the gate operation on the logical qubit level. So, each fundamental gate needs to be defined on logical qubits. But the way the logical operation is performed depends on the logical qubit implementation. There is not one universal logical H gate. The implementation depends on the code used to encode the logical qubit.

Definition 5: Logical Gate

A **Logical Gate** is a quantum **operation acting on logical qubits**. Since a logical qubit is encoded into multiple physical qubits, a logical gate is implemented through a coordinated sequence of operations on the underlying physical qubits.

Physical Implementation of Logical Gates

In superconducting quantum computers, logical gates are ultimately implemented as **microwave pulses** sent through a cryogenic chip. The physical hardware consists of superconducting circuits, resonators, control lines, and Josephson junctions operating at temperatures close to absolute zero.

Superconducting qubits are the most common type of qubit used in current quantum computers (e.g., IBM, Google, Rigetti). The superconducting qubits are typically implemented as cryogenic qubits, which operate at extremely low temperatures 10–20 millikelvins (0.01°C above absolute zero, $\approx -273.15^\circ\text{C}$). At these temperatures, the superconducting materials exhibit almost zero thermal noise, allowing for coherent quantum operations (page 385). The gates are implemented via microwave pulses that manipulate the energy levels of the superconducting qubits.

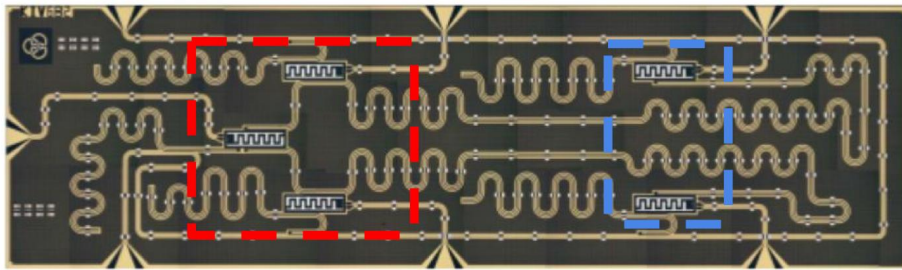


Figure 22: Photograph of a **superconducting quantum chip** used in quantum computing. [3] The squiggly lines are the microwave transmission lines/resonators that control the qubits. The highlighted regions contain the superconducting qubit circuitry and the microwave resonators/control structures used to implement quantum gates through microwave pulses.

6.3 Quantum Transpiling

6.3.1 The Quantum Computing Stack

Before studying placement, scheduling, routing, and optimization, we need to understand **where transpiling sits inside the quantum-computing workflow**. We can summarize the whole process with five layers: (1) the quantum processor, (2) the quantum programming language, (3) the quantum algorithm, (4) the quantum compilation (transpiling), and (5) the quantum execution. The role of the *transpiler* is to act as the **bridge between the abstract quantum algorithm and the physical hardware**. We will see that the **transpiler is responsible for taking a high-level quantum algorithm, expressed in a quantum programming language, and transforming it into a hardware-compatible circuit** that satisfies the constraints of a specific quantum processor.

In general, a quantum computer is not just a chip. Many software and hardware layers cooperate to transform a high-level algorithm into physical operations executed on real qubits. Let's briefly describe the main layers of the quantum computing stack:

1. **Quantum Processor.** The **lowest layer** is the **quantum processor**. A quantum processor is the **physical device that contains the qubits used for computation**. The processor is the actual **hardware where quantum states evolve according to the laws of quantum mechanics**.
2. **Quantum Programming Language.** Writing microwave pulses directly would be impossible for programmers. Instead, we use a **quantum programming language** (similar to classical programming languages). A quantum programming language allows us to describe a circuit using quantum instructions such as H , $CNOT$, and X . The **language hides the physical details of the hardware and provides an abstract description of the computation**.
3. **Quantum Algorithm.** A quantum algorithm is the **circuit that we want to execute**. At this stage we only think about the mathematical algorithm, without worrying about the hardware connectivity (i.e., which qubits can interact with which), gate durations (i.e., decoherence times), physical errors (i.e., noise), and qubit locations (i.e., where the qubits are physically located on the chip). The algorithm represents the **ideal computation**.
4. **Quantum Compilation** (Transpiling). This is the central topic of the chapter. A **quantum algorithm cannot usually be executed directly on hardware**. The circuit must first be adapted to the specific quantum processor. This adaptation process is called: **quantum compilation** or **Transpiling**. We will see four main tasks of the transpiler: (1) **placement**, (2) **scheduling**, (3) **routing**, and (4) **optimization** (use heuristics to merge gates or rearrange operations). In general, the transpiler takes an ideal circuit and transforms it into another circuit that performs the same computation, respects hardware constraints, minimizes errors, and minimizes execution time.

5. **Quantum Execution.** After transpilation, the circuit is finally converted into physical control signals. It is the translation of gate-level instructions into signals sent to the processor. For superconducting qubits this means: generating microwave pulses, sending them to specific qubits, controlling interactions between qubits, performing measurements. This is the **stage where the abstract circuit becomes a real physical experiment.**

In general, the **transpiler is the middle layer that makes the whole system work.**

6.3.2 Placement

In an ideal quantum circuit we freely write operations between any pair of qubits, like $\text{CNOT}(q_1, q_8)$. However, a real quantum processor is not fully connected. Physical qubits occupy fixed positions on a chip and a two-qubit gate can generally be executed only between adjacent qubits. Therefore, before execution, the transpiler must decide: *which logical qubit should be assigned to which physical qubit?* This task is called **placement**.

Definition 6: Placement

Placement is the process of **mapping the qubits of a quantum circuit onto the physical qubits of the processor**. In other words, it is the process of **deciding which logical qubit should be assigned to which physical qubit**. The result is the initial configuration used by the hardware-aware transpiler to perform the subsequent steps of routing and scheduling.

 **The Goal of Placement.** A good placement tries to **position interacting qubits close together**. The reason of this goal is simple: if **two qubits are far apart, they must later be moved using SWAP operations**. SWAP operations are costly because they require multiple gates, larger circuit depth, more noise and lower fidelity. Therefore, placement tries to **reduce future routing costs** before routing even begins.

How do Transpilers understand how far apart qubits are?

To quantify how far two qubits are, the transpiler uses the **Manhattan distance**.

The **Manhattan distance**¹² is simply the **distance we travel if we can only move along the grid lines of the chip** (up/down and left/right). Suppose we have two qubits at positions $A = (x_1, y_1)$ and $B = (x_2, y_2)$. The Manhattan distance between them is given by:

$$d_M(A, B) = |x_1 - x_2| + |y_1 - y_2| \quad (141)$$

For example, if $A = (1, 2)$ and $B = (4, 6)$, then the Manhattan distance is:

$$d_M(A, B) = |1 - 4| + |2 - 6| = 3 + 4 = 7$$

This means that to move from qubit A to qubit B , we would need to perform 7 operations if they are not directly connected.

 **Why Manhattan distance?** Suppose a qubit must move four positions to reach another qubit. The qubit must perform approximately four hops. Since routing (page 405) is implemented through SWAP gates, the number of **hops**

¹²The Manhattan distance takes its name from the grid-like structure of the streets in Manhattan, where we can only move along the streets (up/down and left/right) to get from one point to another.

is directly related to the number of additional operations required. For this reason, minimizing Manhattan distance approximately minimizes routing cost.

In summary, the goal of the placement can be summarized as follows: find the placement that minimizes the sum of Manhattan distances over all qubit pairs involved in multi-qubit gates. Conceptually:

$$\text{Cost} = \sum_{\text{all interacting pairs}} d_M$$

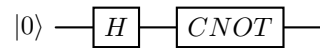
A placement with a smaller total cost will usually require fewer SWAP operations later.

6.3.3 Scheduling

After placement (page 395), the transpiler knows **where each qubit is located** on the processor. However, another question remains: “*when should each gate be executed?*”. At first sight, one might think that all gates could be executed simultaneously, but this is not the case. In fact, there are scheduling issues primarily caused by **data dependencies**.

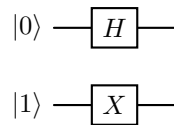
Data Dependencies

A gate can only be executed if all the qubits it requires already contain the correct input state. For example:



The CNOT cannot be executed before the Hadamard gate. Because the CNOT uses the state produced by the Hadamard gate. The second operation therefore **depends** on the first one. This relationship is called a **Data Dependency**. Therefore, the two gates must be executed sequentially, and the **scheduler must respect this ordering**.

Fortunately, not all gates depend on each other. For example:



The two gates act on different qubits. Neither gate needs the output of the other. Therefore they can be **executed simultaneously**. This is a **source of parallelism** in quantum circuits.

In general, **Out-of-Order execution is possible** as long as the computation remains correct. This means the scheduler can **reorder operations only if the final quantum state remains unchanged**. The scheduler cannot move gates around the circuit arbitrarily; it must always respect the dependencies between operations.

Scheduling Objectives

The scheduler tries to achieve two goals:

- **Exploit as much parallelism as possible** to minimize the circuit depth.
- **Reduce the effects of decoherence** (page 368) by executing the circuit as quickly as possible.

These goals sometimes conflict with each other, and for this reason different scheduling policies exist:

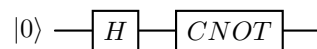
- **As Soon As Possible (ASAP) Scheduling.** Execute an operation immediately when all its required input qubits are available. In other words, when the gate becomes executable, the scheduler executes it right away.
 ✔ The main **advantages** of ASAP scheduling are: maximizes parallelism, reduces total circuit execution time, and keeps hardware busy
- **As Late As Possible (ALAP) Scheduling.** Instead of executing operations immediately, the scheduler delays them as much as possible while still preserving correctness. Conceptually, when the gate needs to be executed, the scheduler checks if it can be delayed without violating any dependencies. If it can be delayed, the scheduler postpones its execution until the last possible moment.
 ✔ The main **advantages** of ALAP scheduling are: reduces idle time of qubits, minimizes the lifetime of quantum states, and can help to **mitigate decoherence effects**.

❓ Which scheduling policy is better?

The choice depends by the final goal of the scheduling.

- **Total execution time of the circuit.** For this quantity, **ASAP is usually better** because it tries to execute everything immediately and therefore often minimizes the overall circuit depth. In simple terms, **ASAP** scheduling tries to **minimize total circuit duration**.
- **Lifetime of a specific quantum state.** Instead, for this quantity, **ALAP is usually better** because it tries to delay operations as much as possible, which can help to reduce the time that a particular quantum state needs to be maintained, thus reducing the effects of decoherence on that state. In simple terms, **ALAP** scheduling tries to **minimize idle lifetime of quantum states**.

Let's suppose we have the following circuit:

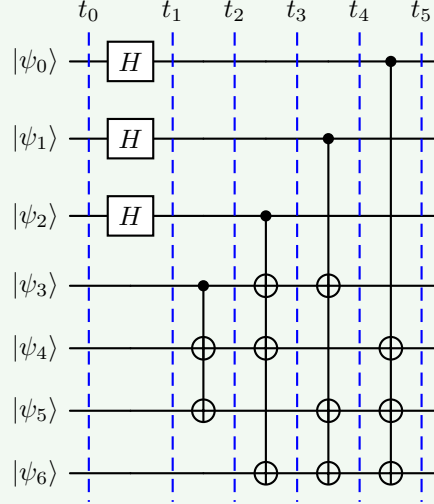


✗ With **ASAP scheduling**, the Hadamard gate H would be executed immediately at time 0, and the CNOT gate would be executed later at time 100 due to other operations that need to be executed in between. Then the superposition created by H must survive for 100 time units, but this **increases the risk of decoherence**.

✔ Instead, **ALAP scheduling** would try to delay the execution of the Hadamard gate H as much as possible, while still ensuring that the CNOT gate can be executed at time 100. This means that H would be executed at time 90, just before the CNOT gate, which reduces the time that the superposition needs to be maintained and therefore **reduces the risk of decoherence**.

Example 3: ASAP & ALAP Scheduling

Consider the following quantum circuit:



Now, let's analyze how the scheduling would work under both policies. The operations must include the initialization of the qubits, the application of the Hadamard gates, and the execution of the CNOT gates.

ASAP Scheduling

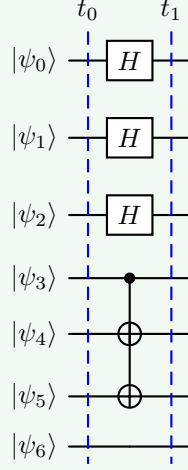
With **ASAP**, all qubits are initialized immediately at time 0 because there are no dependencies:

INIT $|\psi_0\rangle, |\psi_1\rangle, |\psi_2\rangle, |\psi_3\rangle, |\psi_4\rangle, |\psi_5\rangle, |\psi_6\rangle$ at time 0

Once the qubits are initialized, every gate can be executed as soon as its dependencies are satisfied. The main idea is: “*if a gate can run now, run it now*”.

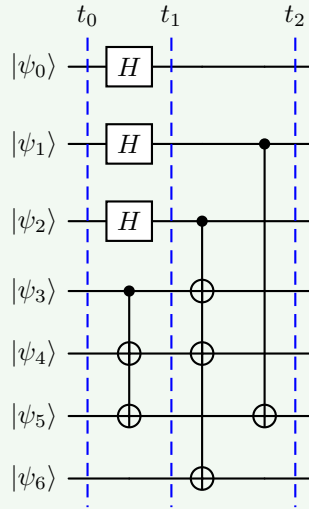
1. At time 1, the Hadamard gates on $|\psi_0\rangle$, $|\psi_1\rangle$, and $|\psi_2\rangle$ can be executed immediately because they only depend on the initialization of the qubits. So we execute them at time 1. Regarding the qubits $|\psi_3\rangle$, $|\psi_4\rangle$ and $|\psi_5\rangle$, we can execute the CNOT gates that depend on them as soon as the Hadamard gates on $|\psi_0\rangle$, $|\psi_1\rangle$ and $|\psi_2\rangle$ are executed.

INIT $|\psi_0\rangle, |\psi_1\rangle, |\psi_2\rangle, |\psi_3\rangle, |\psi_4\rangle, |\psi_5\rangle, |\psi_6\rangle$
 $H(|\psi_0\rangle), H(|\psi_1\rangle), H(|\psi_2\rangle), \text{CNOT}(|\psi_3\rangle, |\psi_4\rangle), \text{CNOT}(|\psi_3\rangle, |\psi_5\rangle)$



2. At time 2, we can execute the CNOT gates that depend on $|\psi_2\rangle$ and $|\psi_1\rangle$ as soon as the Hadamard gates on $|\psi_0\rangle$, $|\psi_1\rangle$ and $|\psi_2\rangle$ are executed. So we execute them at time 2. Regarding the qubits $|\psi_3\rangle$, $|\psi_4\rangle$ and $|\psi_5\rangle$, we can execute the CNOT gates that depend on them as soon as the CNOT gates that depend on them are executed. However, we cannot execute the CNOT gate that depends on $|\psi_6\rangle$ because it depends on the CNOT gate that depends on $|\psi_1\rangle$. And also, the CNOT gate that depends on $|\psi_1\rangle$ cannot be executed completely at time 2 because it depends on the CNOT gate that depends on $|\psi_2\rangle$. So we execute it partially at time 2, and we will execute the remaining part at time 3.

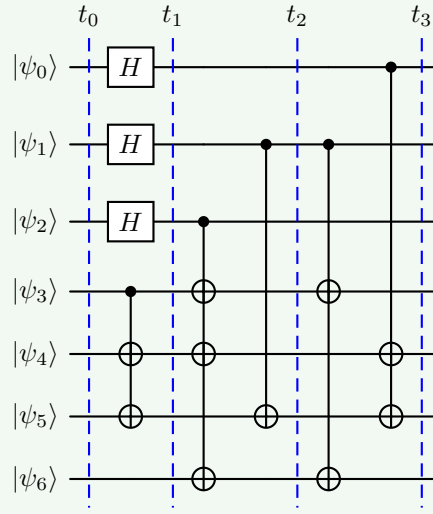
$$\text{CNOT}(|\psi_2\rangle, |\psi_3\rangle), \text{CNOT}(|\psi_2\rangle, |\psi_4\rangle), \\ \text{CNOT}(|\psi_2\rangle, |\psi_5\rangle), \text{CNOT}(|\psi_1\rangle, |\psi_6\rangle)$$



3. At time 3, we can execute the CNOT gates that depend on $|\psi_1\rangle$ and

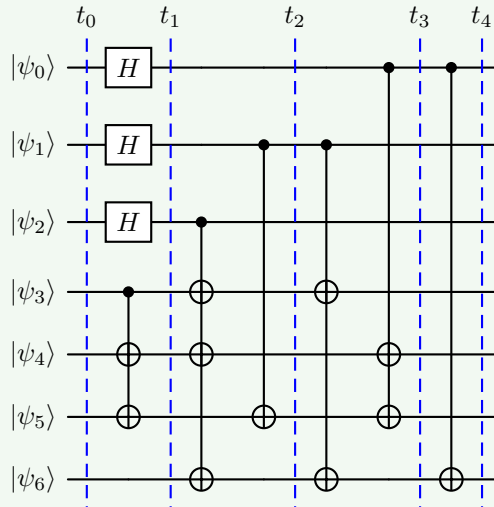
$|\psi_0\rangle$ as soon as the CNOT gates that depend on $|\psi_2\rangle$ and $|\psi_1\rangle$ are executed. Regarding the qubits $|\psi_3\rangle$, $|\psi_4\rangle$ and $|\psi_5\rangle$, we can execute the CNOT gates that depend on them as soon as the CNOT gates that depend on them are executed. However, we cannot execute the CNOT gate that depends on $|\psi_6\rangle$ because it depends on the CNOT gate that depends on $|\psi_1\rangle$. So we execute it partially at time 3, and we will execute the remaining part at time 4.

$$\text{CNOT}(|\psi_1\rangle, |\psi_3\rangle), \text{CNOT}(|\psi_1\rangle, |\psi_6\rangle), \\ \text{CNOT}(|\psi_0\rangle, |\psi_4\rangle), \text{CNOT}(|\psi_0\rangle, |\psi_5\rangle)$$



4. Finally, at time 4, we can execute the CNOT gate that depends on $|\psi_6\rangle$ as soon as the CNOT gate that depends on $|\psi_1\rangle$ is executed.

$$\text{CNOT}(|\psi_0\rangle, |\psi_6\rangle)$$

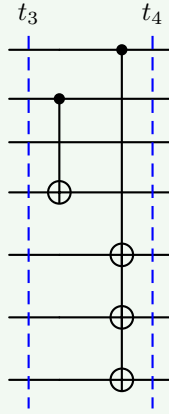


ALAP Scheduling

With **ALAP**, the scheduler works with the opposite philosophy: “*if a gate can be delayed safely, delay it*”. Differently from ASAP scheduling, we simply create a schedule starting from the end of the circuit and moving backwards.

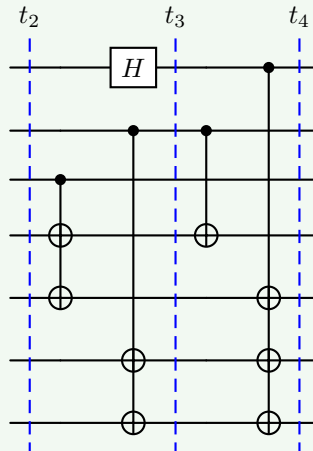
- At time 4, we start with the last CNOT gate. The remaining free time slots are filled with the CNOT gates that depend on $|\psi_1\rangle$ and $|\psi_3\rangle$.

$$\text{CNOT}(|\psi_0\rangle, |\psi_4\rangle), \text{CNOT}(|\psi_0\rangle, |\psi_4\rangle), \text{CNOT}(|\psi_0\rangle, |\psi_4\rangle) \\ \text{CNOT}(|\psi_1\rangle, |\psi_3\rangle)$$



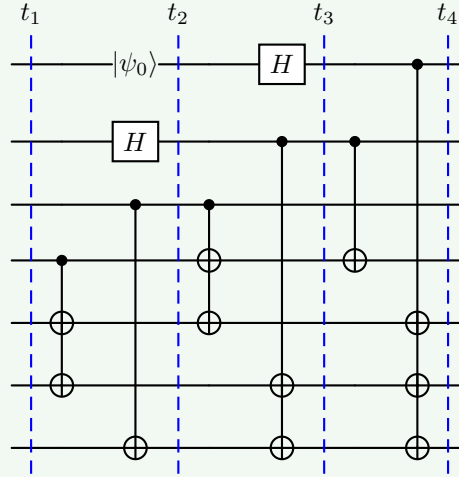
- At time 3, we continue with the same logic, taking the left piece of the circuit and filling the remaining free time slots.

$$H(|\psi_0\rangle), \text{CNOT}(|\psi_1\rangle, |\psi_5\rangle), \text{CNOT}(|\psi_1\rangle, |\psi_6\rangle), \\ \text{CNOT}(|\psi_2\rangle, |\psi_3\rangle), \text{CNOT}(|\psi_2\rangle, |\psi_4\rangle)$$



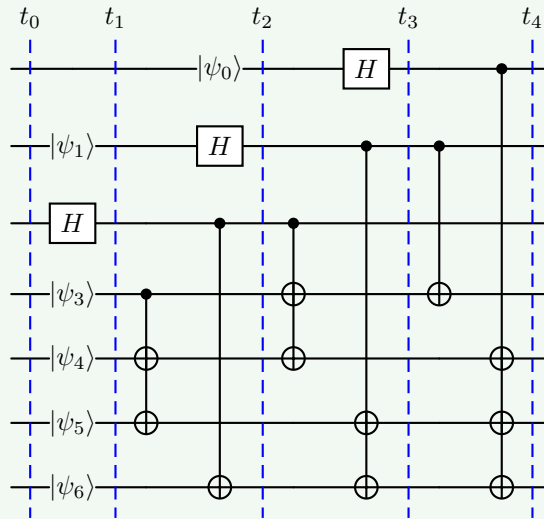
2. At time 2:

INIT $|\psi_0\rangle, H(|\psi_1\rangle), \text{CNOT}(|\psi_2\rangle, |\psi_6\rangle),$
 $\text{CNOT}(|\psi_3\rangle, |\psi_4\rangle), \text{CNOT}(|\psi_3\rangle, |\psi_5\rangle)$



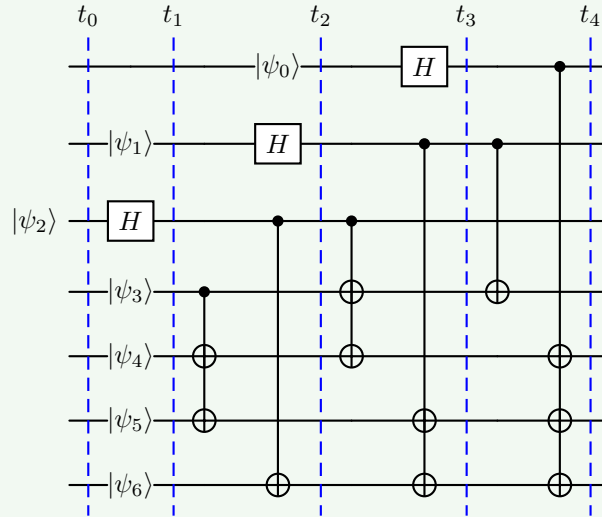
1. At time 1:

INIT $|\psi_1\rangle, |\psi_3\rangle, |\psi_4\rangle, |\psi_5\rangle, |\psi_6\rangle, H(|\psi_2\rangle)$



0. At time 0:

INIT $|\psi_2\rangle$



In general, the ASAP version is more aggressive because it does everything as early as possible. Instead, the ALAP version is more conservative because it delays operations until they are needed. Both schedules preserve the same logical computation. The difference is not **what** gates are executed, but **when** they are executed.

In summary, ASAP is useful to start operations early and exploit parallelism, while ALAP is useful to reduce idle time between the moment a qubit is prepared and the moment it is used. That is why scheduling is not only a performance problem, but also a **fidelity problem** because it can affect the overall quality of the computation by **influencing how long qubits need to maintain their quantum states**, which in turn affects the **likelihood of errors due to decoherence**.

6.3.4 Routing

To understand routing, we need to recall the fundamental hardware constraint: a multi-qubit gate can only be executed between adjacent qubits (i.e., qubits that are directly connected in the hardware architecture). For example, suppose the algorithm requires a $\text{CNOT}(|\psi_3\rangle, |\psi_5\rangle)$ gate, but the placement step maps the qubits onto the processor such that $|\psi_3\rangle$ and $|\psi_5\rangle$ are not adjacent:

$$\begin{bmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & & \end{bmatrix}$$

Here v_3 is in the first row and v_5 is in the second row, so they are **not adjacent**. Therefore the CNOT cannot be executed directly.

✔ **Solution: Routing.** If qubits are not adjacent, how do we move them? Using SWAP gates. A **SWAP gate** exchanges the quantum states of two adjacent qubits (page 109):

$$\text{SWAP}(|\psi_i\rangle, |\psi_j\rangle) = |\psi_j\rangle \otimes |\psi_i\rangle$$

Thus, the qubits themselves do not physically move, but their quantum states are exchanged. The **Routing** is the **process of inserting SWAP gates into the circuit** to ensure that **all multi-qubit gates operate on adjacent qubits**. The goal of routing is to find an efficient sequence of SWAP gates that minimizes the overhead in terms of additional gates and circuit depth.

⚠ **Why Routing is Expensive?** Routing solves the connectivity problem, but it introduces additional gates. For instance, to execute $\text{CNOT}(|\psi_3\rangle, |\psi_5\rangle)$, we might need to perform a sequence of SWAP gates to bring $|\psi_3\rangle$ and $|\psi_5\rangle$ adjacent to each other.

- Each **SWAP gate increases the circuit depth** (longer execution time);
- Every additional **gate introduces also error probability**. This means that the **fidelity** (page 383):

$$F_{\text{gate}} \approx 1 - p_{\text{gate}}$$

Decreases with the number of gates, so more gates means lower fidelity.

- A **deeper circuit takes longer to execute**, which means that qubits remain active for a larger fraction on their decoherence time, **increasing the likelihood of errors due to decoherence**.

🔗 **Connection with Placement.** Now we can finally understand why placement was important. Placement tries to minimize the Manhattan distance (page 395) between qubits that interact. *Why?* Because **smaller distances imply fewer SWAP gates**, which means **less routing overhead, higher fidelity, and shorter execution time**. Therefore, placement and routing are tightly connected: **a good placement reduces the amount of routing required later**.

In summary, routing is the process of inserting SWAP operations to bring non-adjacent qubits together so that multi-qubit gates can be executed on real hardware.

7 QUBO Formulation

7.1 QUBO Problems

In the previous sections of the course, we showed how quantum computers manipulate information using qubits and quantum gates. But a fundamental question remains: *which problems are we trying to solve with these quantum computers?*

One important class of problems is **optimization problems**, where the goal is to find the best solution among many possible alternatives.

Many **optimization problems** arising in science, engineering, logistics, finance, and machine learning can be **reformulated into** a common **mathematical framework** called **Quadratic Unconstrained Binary Optimization (QUBO)**. The importance of QUBO is that a wide variety of seemingly different optimization tasks can be expressed using the same mathematical structure.

This common formulation is particularly useful in quantum computing because it can be directly connected to physical systems, Hamiltonians, and quantum annealing algorithms. For this reason, QUBO serves as a bridge between classical optimization problems and quantum optimization methods.

7.1.1 Introduction to QUBO

Many real-world problems can be formulated as optimization problems. Given a set of possible solutions, the **goal is to find the one that minimizes** (or **maximizes**) a certain **objective function**.

Examples include: scheduling tasks, routing vehicles¹³, portfolio optimization (finance), resource allocation (logistics), and machine learning optimization (training models). A **challenge** is that these **problems often appear very different from one another**. Instead of developing a completely different solution method for every problem, it is useful to express them using a common mathematical formulation.

One of the most important formulations is the **Quadratic Unconstrained Binary Optimization (QUBO) model**. In a QUBO problem, the solution is represented by a collection of binary variables:

$$x_i \in \{0, 1\}$$

And the **objective is to find the assignment of these variables that minimizes a cost (or energy) function**. So QUBO is a **mathematical framework** for

¹³The Vehicle Routing Problem (VRP) is an optimization problem that asks: “*how should a fleet of vehicles serve a set of customers at the lowest cost while satisfying operational constraints?*”. It is one of the most important problems in logistics, transportation, and supply chain management. We are given a depot (warehouse, distribution center, etc.), a set of customers that need service, and a fleet of vehicles. We need to determine: (1) which customers each vehicle should visit; (2) the order in which each vehicle should visit them; and (3) when the visits should occur (in some variants).

expressing optimization problems in terms of binary variables and a quadratic cost function.

✚ Mathematical Formulation

The general QUBO formulation is¹⁴:

$$y = \sum_i a_i x_i + \sum_{i>j} q_{ij} x_i x_j \quad (142)$$

Or, more compactly, in matrix form:

$$\arg \min_x y = \mathbf{x} Q \mathbf{x}^T \quad (143)$$

Where:

- $\mathbf{x}$ is a vector of **binary variables**. The **variables** of a QUBO problem can **only assume two values**:

$$x_i \in \{0, 1\}$$

Each variable therefore behaves similarly to a boolean variable: 0 represents **false** and 1 represents **true**. The number of variables is denoted by n , which represents the size of the optimization problem.

📌 Useful Property of Binary Variables

Since the variables are binary, $x_i \in \{0, 1\}$, we always have $x_i^2 = x_i$. Indeed $0^2 = 0$ and $1^2 = 1$. This seemingly simple property is **extremely important because it allows many optimization expressions to be simplified and rewritten in QUBO form**. We will see examples of this in the next sections.

- $Q \in \mathbb{R}^{n \times n}$ is a matrix containing the coefficients of the problem. It contains all **coefficients defining the optimization problem**. The matrix may be:
 - **Symmetric**, meaning $q_{ij} = q_{ji}$ for all i and j (i.e., the values above and below the diagonal are the same).
 - **Upper triangular**, meaning $q_{ij} = 0$ for all $i > j$ (i.e., only the values on and above the diagonal are non-zero).

Also:

- The **diagonal elements** $q_{ii} = a_i$ represent the **contribution of individual variables**;
- While the **off-diagonal elements** q_{ij} represent the **interaction between pairs of variables**.
- y is the **objective function to be minimized**.

¹⁴The coefficients a_i and q_{ij} are sometimes called **biases** and **couplers**, respectively. The origin of this terminology will become clear when introducing the Ising model (page 446).

❓ Understanding the QUBO Acronym

The name **QUBO** stands for **Quadratic Unconstrained Binary Optimization**. Each word describes an important characteristic of the formulation. Understanding the meaning of the acronym helps clarify both the capabilities and limitations of QUBO models:

- **Quadratic.** A QUBO objective function can contain:
 - Linear terms $a_i x_i$, which represent the contribution of individual variables.
 - Quadratic terms $q_{ij} x_i x_j$, which represent the interaction between pairs of variables.

However, it cannot contain higher-order terms such as cubic $x_i x_j x_k$ or quartic $x_i x_j x_k x_l$, or any polynomial term of degree greater than two. For this reason, QUBO problems are said to be **quadratic**. The restriction to quadratic interactions is particularly important because it allows the problem to be mapped naturally onto physical systems whose energy depends on pairwise interactions (e.g., Ising models in physics, we will see examples of this in the next sections).

- **Unconstrained.** In its pure form, a **QUBO problem does not include explicit constraints**. For example, expressions such as $x_1 + x_2 \leq 1$ or $x_1 = x_2$ cannot be imposed directly. Instead, **constraints must be incorporated indirectly by modifying the objective function and introducing suitable penalty terms**. This topic will be discussed in more detail in the next sections, page 418.

❓ **Why can't QUBO include explicit constraints?** In a normal optimization problem, suppose we want to minimize $\min f(x)$ subject to $x_1 + x_2 = 1$. This is a **constrained optimization problem** because the solver knows objective function $f(x)$ and the constraints $x_1 + x_2 = 1$ and can use both to find the optimal solution.

In contrast, the **QUBO solver** receives only $\min \mathbf{x}Q\mathbf{x}^T$ and has no information about any constraints. Therefore, the solver cannot enforce the constraint $x_1 + x_2 = 1$ directly. Instead, we need to modify the objective function to include a **penalty term that discourages solutions that violate the constraint**. For example, we could add a term like $P(x_1 + x_2 - 1)^2$ to the objective function, where P is a large positive constant. This way, any solution that does not satisfy $x_1 + x_2 = 1$ will incur a large penalty, effectively enforcing the constraint indirectly.

- **Binary.** The **decision variables are binary** $x_i \in \{0, 1\}$. Each variable can therefore assume only two possible values: 0 or 1. This property makes QUBO particularly suitable for representing logical decisions such as: yes / no; true / false; selected / not selected; active / inactive. The binary nature of the variables will later allow a direct connection with qubits and spin systems.
- **Optimization.** The goal of a QUBO problem is to find the **variable assignment that minimizes the objective function**:

$$\arg \min_x y = \arg \min_x \mathbf{x}Q\mathbf{x}^T$$

Among all possible configurations of the binary variables, we search for the **one associated with the lowest cost** (or energy) value.

This interpretation is particularly important because later in the chapter the objective function will be viewed as an **energy landscape**, and solving the optimization problem will become equivalent to finding the **ground state** of a quantum system.

❓ Why is QUBO Important? One of the **main advantages** of the QUBO formulation is that many **difficult optimization problems** can be **rewritten using this common framework**. Once a problem has been converted into QUBO form, the same optimization machinery can be applied regardless of the original application domain. In other words, the **true strength of QUBO is the ability to serve as a universal language for optimization problems**.

For this reason, technologies capable of solving QUBO problems efficiently may have a significant impact across many fields of science and engineering.

7.1.2 QUBO as an Energy Minimization Problem

A useful way to think about a QUBO problem is not in terms of optimization, but in terms of **energy**. Instead of saying “*find the variable assignment that minimizes the objective function*”, we can equivalently say “***find the configuration of variables with the lowest energy***”. In fact, the objective function:

$$y = \mathbf{x}Q\mathbf{x}^T$$

Can be interpreted as an **energy function** associated with a particular configuration of the binary variables. Every possible assignment of the variables corresponds to a different energy value.

✂ Configurations and Energies

Consider a simple problem with two **binary variables**:

$$x_1, x_2 \in \{0, 1\}$$

There are **four possible configurations** of these variables:

$$(0, 0), (0, 1), (1, 0), (1, 1)$$

For each configuration, the **QUBO function produces** a numerical value:

$$y(x_1, x_2)$$

This value can be interpreted as the **energy associated with that configuration**.

Configuration	Energy
(0, 0)	$y(0, 0) = E_1$
(0, 1)	$y(0, 1) = E_2$
(1, 0)	$y(1, 0) = E_3$
(1, 1)	$y(1, 1) = E_4$

The optimization problem then becomes the **search for the configuration with the smallest energy**. In physics, the **state** with the **lowest possible energy** is called the **Ground State**. Thus, solving a QUBO problem is equivalent to finding:

$$\mathbf{x}_{\text{opt}} = \arg \min_{\mathbf{x}} \mathbf{x}Q\mathbf{x}^T$$

Where $\mathbf{x}_{\text{opt}}$ is the optimal solution, i.e., the configuration corresponding to the ground state of the energy function. **This terminology is extremely important** because later we will **map the QUBO objective function to a quantum Hamiltonian**. At that point, the optimization problem will become simply the problem of finding the ground state of a quantum system. In other words, the **mathematical problem remains the same**; only the **interpretation changes**.

📖 Energy Landscape (useful for visualization)

It is often useful to visualize a QUBO problem as an **energy landscape**. Each possible **configuration of the variables** corresponds to a **point in this landscape**, while the **value of the objective function determines its height**:

- ✖ High energy: **undesirable** solution.
- ⚖ Low energy: **desirable** solution.
- ✔ Minimum energy: **optimal** solution (ground state).

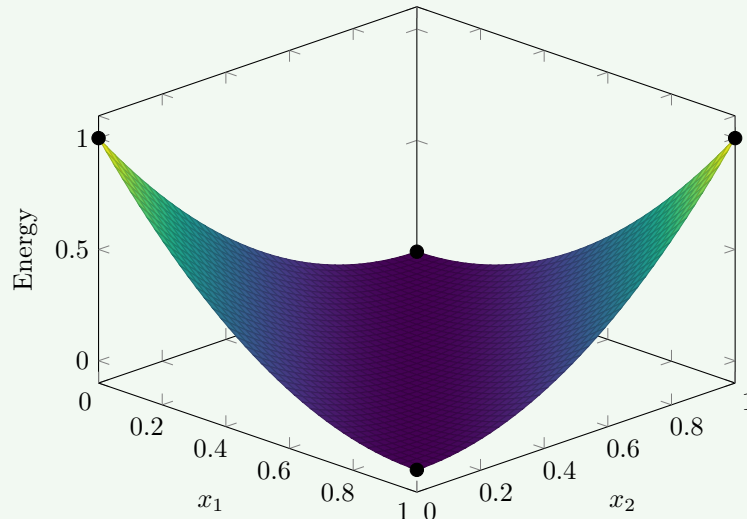
Conceptually, solving the optimization problem is like searching for the deepest valley in the landscape. The global minimum corresponds to the solution of the QUBO problem.

📌 Example 1: QUBO Energy Landscape

For example, consider the following QUBO function:

$$E(x_1, x_2) = 1 - x_1 - x_2 + 2x_1x_2$$

This function assigns an energy value to each of the four configurations of the binary variables x_1 and x_2 . We can visualize this energy landscape in a 3D plot, where the x and y axes represent the **binary variables**, and the z axis represents the **energy value**. The points corresponding to the four configurations will have different heights based on their energy values, and the **optimal solutions will be the points with the lowest height** (energy).



In summary, the QUBO formulation can be viewed as an energy minimization problem, where the objective function represents an energy landscape. The goal is to find the configuration of binary variables that corresponds to the lowest energy, which is equivalent to solving the original optimization problem. This

perspective is particularly useful when we later map QUBO problems to quantum Hamiltonians, as it allows us to leverage concepts from quantum mechanics to find optimal solutions.

7.1.3 Boolean Logic in QUBO

One of the **strengths** of the QUBO formulation is that **logical conditions can be translated into energy functions**. The **general strategy** is remarkably simple:

1. **Identify** the **configurations** that satisfy the desired condition.
2. **Assign** them the **lowest** possible **energy**.
3. **Assign** **higher energy to all other** configurations.
4. **Construct** a **quadratic function** that reproduces those energy values.

The **minimum of the resulting** energy function will then correspond exactly to the **solutions we want**.

✂ Example: XOR

Let's see how this works in practice with a simple example: the **logical XOR** of two binary variables x_1 and x_2 . The XOR condition is satisfied only when exactly one of x_1 and x_2 is 1: $a \neq b$. The truth table for XOR is:

x_1	x_2	XOR(x_1, x_2)
0	0	0
0	1	1
1	0	1
1	1	0

The goal is to find the assignments that make the boolean function true.

🌀 **From Logic to Energy.** Instead of directly solving the logical expression, we assign energies to each configuration. A natural choice is:

- **Energy 0** for the **desired** solutions (those that satisfy the XOR condition): (0, 1) and (1, 0).
- **Energy 1** for the **undesired** solutions (those that do not satisfy the XOR condition): (0, 0) and (1, 1).

This gives us the following energy landscape:

x_1	x_2	XOR	Energy
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

Now the optimization problem is equivalent to finding the configurations with minimum energy.

✂ **Constructing the Energy Function.** We now need to find a quadratic function $E(a, b)$ that reproduces the energy values of the table. For this simple case, we can build it incrementally:

1. The configuration $(a, b) = (0, 0)$ must have energy 1. So a constant term $E = 1$ is a good starting point. However, all four configurations would then have energy 1, so additional terms are required.
2. Next, notice that the configurations $(0, 1)$ and $(1, 0)$ should have energy 0. Subtracting a and b lowers the energy whenever one of the variables becomes 1:

$$E(a, b) = 1 - a - b$$

But now the configuration $(1, 1)$ has energy -1 , which is too low.

3. To fix last configuration, we add a quadratic term $2ab$ that increases the energy when both variables are 1:

$$E(a, b) = 1 - a - b + 2ab$$

Now the energy function correctly assigns energy 0 to the desired configurations and energy 1 to the undesired ones:

x_1	x_2	$E(x_1, x_2)$
0	0	$1 - 0 - 0 + 2(0)(0) = 1$
0	1	$1 - 0 - 1 + 2(0)(1) = 0$
1	0	$1 - 1 - 0 + 2(1)(0) = 0$
1	1	$1 - 1 - 1 + 2(1)(1) = 1$

Therefore, the logical problem of finding the XOR of two binary variables has been transformed into an optimization problem of minimizing the energy function:

$$E(a, b) = 1 - a - b + 2ab$$

The minimum energy configurations correspond exactly to the solutions of the original logical problem.

⚠ Equivalence of Formulations. In QUBO, the absolute value of the energy is usually irrelevant. What matters is where the minima are located. In other words, once we have a function that correctly identifies the desired configurations as minima, *is the energy function unique?* The answer is **no**. Different energy functions may represent the same optimization problem, provided that they preserve the same ground states.

In this example, the formulation could be modified by adding or subtracting a constant without changing the location of the minima. For instance, we could add a constant term -1 to the energy function:

$$E'(a, b) = E(a, b) - 1 = (1 - a - b + 2ab) - 1 = -a - b + 2ab$$

Evaluating the new function:

a	b	$E'(a, b)$
0	0	0
0	1	-1
1	0	-1
1	1	0

These two formulations are equivalent in terms of optimization, since they have the same minima. The choice between them is a matter of convenience and does not affect the solution of the problem. More generally, **two QUBO formulations** are considered **equivalent** if they **preserve the same ground states**, even when their energy values differ.

This flexibility is extremely useful when constructing QUBO models. A modeler is free to choose the energy function that is easiest to derive, combine with constraints, or implement, as long as the desired solutions remain the ground states.

✂ Visualizing the Energy Landscape

The **energy values associated with the four possible assignments can be visualized as a discrete energy landscape**. Each point corresponds to one configuration of the binary variables, while the vertical axis represents the energy assigned by the QUBO function.

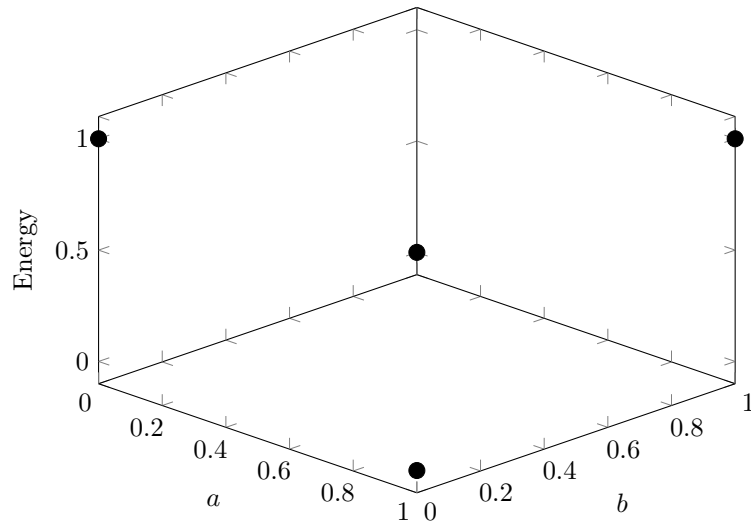


Figure 23: Discrete energy landscape of the QUBO formulation for the boolean function $a \neq b$. The ground states correspond to the minimum-energy configurations $(0, 1)$ and $(1, 0)$.

7.2 Continuous Variables and Constraints

7.2.1 Continuous Variables through Binary Expansion

This section addresses the first practical limitation of QUBO: **variables must be binary**. Up to now this was not a problem because we were working with Boolean variables $a, b \in \{0, 1\}$. However, most real-world optimization problems involve quantities such as temperatures, distances, costs, production levels, or resource allocations, which are usually **represented by continuous or integer-valued variables**. The natural question is therefore, “*how can a QUBO formulation represent variables that are not binary?*”. The answer is **binary expansion**.

Binary Expansion

Recall how ordinary binary numbers are represented. For example, the number 13 can be represented in binary as:

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = (1101)_2$$

The same principle can be used inside a QUBO formulation. Instead of representing a variable x directly, we write:

$$x = \sum_k 2^k x_k \quad x_k \in \{0, 1\} \quad (144)$$

Each binary variable x_k determines whether the corresponding power of two contributes to the value of x .

Suppose we want to represent a real variable using binary variables. One possible encoding is:

$$x = \sum_{k=-2}^2 2^k x_k$$

Where k ranges from -2 to 2 . Expanding the powers of two, we get:

$$x = \frac{1}{4}x_{-2} + \frac{1}{2}x_{-1} + x_0 + 2x_1 + 4x_2$$

Each coefficient corresponds to a different binary weight.

- **What values can be represented?** Since each variable is binary $x_i \in \{0, 1\}$, the smallest representable value is obtained when all variables are zero, $x_{\min} = 0$. The largest representable value is obtained when all variables are one, $x_k = 1$ for all k , which gives:

$$x_{\max} = \frac{1}{4} + \frac{1}{2} + 1 + 2 + 4 = \frac{31}{4} = 7.75$$

Therefore the representation can describe values between 0 and 7.75.

- **Precision.** The smallest coefficient is $\frac{1}{4}$, as a consequence, the representable values differ by multiples of 0.25 . This quantity is called the **resolution** (or *precision*) of the representation. Adding more negative powers of two would increase the precision, while adding more positive powers of two would increase the maximum representable value.

❓ **Who decides the range and precision of the representation?** The **modeler decides both the expansion and the precision**. For instance, suppose our original optimization problem contains a variable $x \in [0, 5]$. To convert the problem into QUBO, we must decide how to encode x using binary variables. If we choose the same expansion as before, we can represent values up to 7.75, which is sufficient to cover the range of x . In general, **higher precision requires more binary variables**, which **increases the size of the QUBO problem** and can make it **more difficult to solve**. Therefore, there is a trade-off between precision and computational complexity that the modeler must consider when choosing the binary expansion.

In general, **Binary Expansion** is simply the idea of **representing a number as a sum of powers of two**. It is the binary equivalent of decimal notation. In the context of QUBO, it allows us to **replace one continuous** (or integer) **variable with several binary variables**.

7.2.2 Constraints in QUBO

After the first major limitation of the QUBO formulation, which is that it only allows for binary variables (page 416), we have another one: *how can we express constraints if QUBO is “unconstrained”?*

⚠ How can we express constraints in QUBO?

Many optimization problems contain **constraints that restrict the set of valid solutions**. For example, we may require:

$$x_1 + x_2 \leq 1 \quad \text{or} \quad x_1 = x_2$$

In traditional optimization, such constraints can be written explicitly and enforced by the solver. QUBO formulations, however, are **unconstrained** optimization problems, where the objective function has the form:

$$\arg \min_{\mathbf{x}} \mathbf{x} Q \mathbf{x}^T$$

And there is no place where additional constraints can be directly specified.

✔ **Solution.** Since we cannot forbid invalid solutions directly (i.e., we cannot write constraints explicitly), we use a different strategy: **make invalid solutions energetically expensive**. This is achieved through a **penalty function**. So, instead of solving:

$$\arg \min_{\mathbf{x}} \mathbf{x} Q \mathbf{x}^T \quad \text{subject to constraints}$$

We solve:

$$\arg \min_{\mathbf{x}} (\mathbf{x} Q \mathbf{x}^T + \gamma \cdot \text{penalty}(\mathbf{x})) \quad (145)$$

Where:

- $\text{penalty}(\mathbf{x})$ is a **function that measures the violation of the constraints** (see more on page 420).

In general, a good penalty function should satisfy two conditions:

- ✔ $\text{penalty}(\mathbf{x}) = 0$ if $\mathbf{x}$ **satisfies** the constraints (i.e., if $\mathbf{x}$ is a feasible solution).
- ✗ $\text{penalty}(\mathbf{x}) > 0$ if $\mathbf{x}$ **violates** the constraints (i.e., if $\mathbf{x}$ is an infeasible solution).

As a result, feasible solutions preserve their original energy, while infeasible solutions receive an additional energy cost.

- γ is a **coefficient** controlling **how strongly violations are punished** (see more on page 428).

In summary, the underlying idea is identical to the boolean example we saw on page 413. For XOR, we assigned *low energy* to desired configurations and *high energy* to undesired ones. Here, we do the same: *feasible solutions* receive no penalty (i.e., they keep their original energy), while *infeasible solutions* receive a penalty that increases their energy, making them less attractive to the optimizer. In both cases, the **goal is to shape the energy landscape** so that the ground state corresponds to the solution we want.

 Example 2: Enforcing a Simple Constraint

Suppose we have the constraint $x_1 + x_2 \leq 1$. This means that at most one of the variables can be 1. For instance, the configuration:

$$(x_1, x_2) = (1, 1)$$

Violates the constraint because: $1 + 1 > 1$. Applying the penalty trick, instead of explicitly forbidding this assignment, we can increase its energy. Conceptually:

Configuration	Valid?	Energy Penalty
(0, 0)	✓	0
(0, 1)	✓	0
(1, 0)	✓	0
(1, 1)	✗	Large

The optimizer will naturally avoid the last configuration because it has a higher energy.

 Practical Warning

Because constraints are enforced indirectly through penalties, the resulting **solution should always be checked for feasibility after optimization**. A poorly chosen penalty coefficient may allow an infeasible solution to appear energetically attractive. Therefore, it is crucial to verify that the solution satisfies the original constraints, especially when using a small penalty coefficient γ .

7.2.3 Penalty Terms

A **Penalty Term** is an **additional contribution** to the objective function whose purpose is to **discourage solutions that violate a constraint**. The general form is:

$$\arg \min_{\mathbf{x}} (\mathbf{x}Q\mathbf{x}^T + \gamma \cdot \text{penalty}(\mathbf{x})) \quad (146)$$

Where the penalty function is designed such that:

$$\text{penalty}(\mathbf{x}) = \begin{cases} 0 & \text{if } \mathbf{x} \text{ satisfies the constraints} \\ > 0 & \text{otherwise} \end{cases} \quad (147)$$

And γ controls the strength of the penalty.

✂ Common Constraints

In the following, we will see some constraints and their corresponding penalty terms, in particular, we will see how to build the penalty term for the following constraints:

- $x_1 + x_2 \leq 1$ (page 420)
- $x_1 \leq x_2$ (page 423)
- $x_1 + x_2 \geq 1$ (page 421)
- $x_1 = x_2$ (page 424)
- $x_1 + x_2 = 1$ (page 421)
- $x_1 + x_2 + x_3 \leq 1$ (page 425)

✂ Example 3: $x_1 + x_2 \leq 1$

Let's consider the **constraint**:

$$x_1 + x_2 \leq 1$$

Means that at most one variable can be 1. If we see at the truth table, we can see that the only way to violate the constraint is to have both x_1 and x_2 equal to 1:

x_1	x_2	$x_1 + x_2$	Constraint Satisfied?	Penalty Desired
0	0	0	✓	0
0	1	1	✓	0
1	0	1	✓	0
1	1	2	✗	positive

Thus, the only simple way to define a penalty term for this constraint is to multiply x_1 and x_2 together, since the product will be 1 only when both variables are 1, which is the only case that violates the constraint. The **penalty term** can be defined as:

$$x_1x_2 \quad x_1x_2 = \begin{cases} 1 & \text{if } x_1 = x_2 = 1 \\ 0 & \text{otherwise} \end{cases}$$

 Example 4: $x_1 + x_2 \geq 1$

Let's consider the **constraint**:

$$x_1 + x_2 \geq 1$$

Means that at least one variable must be 1. Let's write the truth table for this constraint:

x_1	x_2	$x_1 + x_2$	Constraint Satisfied?	Penalty Desired
0	0	0	✗	positive
0	1	1	✓	0
1	0	1	✓	0
1	1	2	✓	0

The question becomes, *can we build a polynomial that is 1 only when both variables are 0?* Yes, we can. Let's think it this way: we want a function that is *true* only when both variables are 0. Logically, we can write this as:

$$\text{penalty}(x_1, x_2) = (x_1 = 0) \wedge (x_2 = 0) = \neg x_1 \wedge \neg x_2$$

Mathematically, a $\neg$ can be represented as $1 - x_i$, since x_i is a binary variable:

x	$1 - x$
0	1
1	0

Thus, we can substitute the $\neg$ with $1 - x_i$:

$$\text{penalty}(x_1, x_2) = (1 - x_1) \wedge (1 - x_2)$$

Finally, we can use the same trick as before to represent the $\wedge$ with a product:

a	b	$a \wedge b$
0	0	0
0	1	0
1	0	0
1	1	1

Thus, we can write the **penalty term** as:

$$\text{penalty}(x_1, x_2) = (1 - x_1)(1 - x_2) = 1 - x_1 - x_2 + x_1x_2$$

 Example 5: $x_1 + x_2 = 1$

Let's consider the **constraint**:

$$x_1 + x_2 = 1$$

Means that exactly one variable must be 1. Let's write the truth table for this constraint:

x_1	x_2	$x_1 + x_2$	Constraint Satisfied?	Penalty Desired
0	0	0	✗	positive
0	1	1	✓	0
1	0	1	✓	0
1	1	2	✗	positive

Differently from the previous cases, here we have two cases that violate the constraint: when both variables are 0 and when both variables are 1. Thus, we need to find a function that is 1 in both cases. Again, we can use the logical representation to find the function:

$$[(x_1 = 0) \wedge (x_2 = 0)] \vee [(x_1 = 1) \wedge (x_2 = 1)]$$

We already know how to represent the $\wedge$ and the $\neg$, but we also need to represent the $\vee$. The $\vee$ truth table is the following:

a	b	$a \vee b$
0	0	0
0	1	1
1	0	1
1	1	1

At first, it might seem that we cannot represent the $\vee$ with a polynomial, since the only case that is 0 is when both variables are 0. However, we can use the **dual form of De Morgan's law for disjunction**:

$$a \vee b = \neg(\neg a \wedge \neg b)$$

And the truth table for this expression is the following:

a	b	$\neg a$	$\neg b$	$\neg a \wedge \neg b$	$\neg(\neg a \wedge \neg b)$
0	0	1	1	1	0
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	0	1

Now we can substitute the $\neg$ with $1 - x_i$ and the $\wedge$ with a product:

$$\text{penalty}(x_1, x_2) = [(1 - x_1)(1 - x_2)] \vee [x_1 x_2]$$

Finally, we can substitute the $\vee$ with the dual form of De Morgan's law:

$$\text{penalty}(x_1, x_2) = 1 - [1 - (1 - x_1)(1 - x_2)] \cdot [1 - x_1 x_2]$$

Which can be simplified to:

$$\begin{aligned}
 \text{penalty}(x_1, x_2) &= 1 - [1 - (1 - x_1)(1 - x_2)] \cdot [1 - x_1x_2] \\
 &= 1 - [1 - (1 - x_2 - x_1 + x_1x_2)] \cdot [1 - x_1x_2] \\
 &= 1 - [x_2 + x_1 - x_1x_2] \cdot [1 - x_1x_2] \\
 &= 1 - (x_2 - x_1x_2^2 + x_1 - x_1^2x_2 - x_1x_2 + x_1^2x_2^2) \\
 &= 1 - x_2 + x_1x_2^2 - x_1 + x_1^2x_2 + x_1x_2 - x_1^2x_2^2
 \end{aligned}$$

Now, it might seem so complicated, but we can simplify it even more. Since x_i is a binary variable, we can substitute x_i^2 with x_i :

$$\begin{aligned}
 \text{penalty}(x_1, x_2) &= 1 - x_2 + x_1x_2^2 - x_1 + x_1^2x_2 + x_1x_2 - x_1^2x_2^2 \\
 &= 1 - x_2 + x_1x_2 - x_1 + x_1x_2 + x_1x_2 - x_1x_2 \\
 &= 1 - x_1 - x_2 + 2x_1x_2
 \end{aligned}$$

We got it! Let's check the truth table to be sure:

- $\text{penalty}(0, 0) = 1 - 0 - 0 + 2(0)(0) = 1$ (constraint not satisfied)
- $\text{penalty}(0, 1) = 1 - 0 - 1 + 2(0)(1) = 0$ (constraint satisfied)
- $\text{penalty}(1, 0) = 1 - 1 - 0 + 2(1)(0) = 0$ (constraint satisfied)
- $\text{penalty}(1, 1) = 1 - 1 - 1 + 2(1)(1) = 1$ (constraint not satisfied)

Everything checks out! So the **penalty term** can be defined as:

$$\text{penalty}(x_1, x_2) = 1 - x_1 - x_2 + 2x_1x_2$$

Example 6: $x_1 \leq x_2$

Let's consider the **constraint**:

$$x_1 \leq x_2$$

The constraint means that x_1 can be 1 only if x_2 is also 1, otherwise, if x_1 is 0, x_2 can be either 0 or 1. Let's write the truth table for this constraint:

x_1	x_2	Constraint Satisfied?	Penalty Desired
0	0	✓	0
0	1	✓	0
1	0	✗	positive
1	1	✓	0

The only case that violates the constraint is when x_1 is 1 and x_2 is 0. Thus, we need to find a function that is 1 only in this case. We can use

the logical representation to find the function:

$$\text{penalty}(x_1, x_2) = (x_1 = 1) \wedge (x_2 = 0) = x_1 \wedge \neg x_2$$

Now we can substitute the $\neg$ with $1 - x_2$:

$$\text{penalty}(x_1, x_2) = x_1 \wedge (1 - x_2)$$

Finally, we can substitute the $\wedge$ with a product:

$$\text{penalty}(x_1, x_2) = x_1(1 - x_2) = x_1 - x_1x_2$$

Let's check the truth table to be sure:

- $\text{penalty}(0, 0) = 0 - 0 = 0$ (constraint satisfied)
- $\text{penalty}(0, 1) = 0 - 0 = 0$ (constraint satisfied)
- $\text{penalty}(1, 0) = 1 - 0 = 1$ (constraint not satisfied)
- $\text{penalty}(1, 1) = 1 - 1 = 0$ (constraint satisfied)

Everything checks out! So the **penalty term** can be defined as:

$$\text{penalty}(x_1, x_2) = x_1 - x_1x_2$$

Example 7: $x_1 = x_2$

Let's consider the **constraint**:

$$x_1 = x_2$$

The constraint means that both variables must be the same. Let's write the truth table for this constraint:

x_1	x_2	Constraint Satisfied?	Penalty Desired
0	0	✓	0
0	1	✗	positive
1	0	✗	positive
1	1	✓	0

The only cases that violate the constraint are when x_1 is 0 and x_2 is 1, or when x_1 is 1 and x_2 is 0. Thus, we need to find a function that is 1 in both cases. We can use the logical representation to find the function:

$$\text{penalty}(x_1, x_2) = [(x_1 = 0) \wedge (x_2 = 1)] \vee [(x_1 = 1) \wedge (x_2 = 0)]$$

Now we can substitute the $\neg$ with $1 - x_i$ and the $\wedge$ with a product:

$$\begin{aligned}
&= [(1 - x_1)x_2] \vee [x_1(1 - x_2)] \\
&= 1 - [1 - (1 - x_1)x_2] \cdot [1 - x_1(1 - x_2)] \\
&= 1 - [1 - x_2 + x_1x_2] \cdot [1 - x_1 + x_1x_2] \\
&= 1 - (1 - x_1 + x_1x_2 - x_2 + x_1x_2 - x_1x_2^2 + x_1x_2 - x_1^2x_2 + x_1^2x_2^2) \\
&\downarrow \text{ since } x_i^2 = x_i \\
&= 1 - (1 - x_1 + x_1x_2 - x_2 + x_1x_2 - x_1x_2 + x_1x_2 - x_1x_2 + x_1x_2) \\
&\downarrow \text{ simplify} \\
&= 1 - (1 - x_1 + 2x_1x_2 - x_2) \\
&= x_1 + x_2 - 2x_1x_2
\end{aligned}$$

Let's check the truth table to be sure:

- $\text{penalty}(0, 0) = 0 + 0 - 0 = 0$ (constraint satisfied)
- $\text{penalty}(0, 1) = 0 + 1 - 0 = 1$ (constraint not satisfied)
- $\text{penalty}(1, 0) = 1 + 0 - 0 = 1$ (constraint not satisfied)
- $\text{penalty}(1, 1) = 1 + 1 - 2 = 0$ (constraint satisfied)

Everything checks out! So the **penalty term** can be defined as:

$$\text{penalty}(x_1, x_2) = x_1 + x_2 - 2x_1x_2$$

Example 8: $x_1 + x_2 + x_3 \leq 1$

Let's consider the **constraint**:

$$x_1 + x_2 + x_3 \leq 1$$

The constraint means that at most one variable can be 1. Let's write the truth table for this constraint:

x_1	x_2	x_3	$x_1 + x_2 + x_3$	Constraint Satisfied?
0	0	0	0	✓
0	0	1	1	✓
0	1	0	1	✓
0	1	1	2	✗
1	0	0	1	✓
1	0	1	2	✗
1	1	0	2	✗
1	1	1	3	✗

The constraint is satisfied when at most one variable is set to true (i.e., equals to one). Here, instead of trying to find a logical representation of the penalty term, we can think more abstractly. Invalid states are all

cases where **at least two variables are 1**:

$$110, 101, 011, 111$$

From this, we can see that 110 is simply:

$$x_1x_2(1 - x_3)$$

But we could include 111 as well, since it also violates the constraint. Thus, we can write something like $11-$ where the $-$ means that the variable can be either 0 or 1, don't care about it. This can be represented as:

$$(x_1 = 1) \wedge (x_2 = 1) \wedge - = x_1x_2$$

Now, the remaining invalid states are 101 and 011, which can be represented as:

$$101 \rightarrow (x_1 = 1 \wedge x_2 = 0 \wedge x_3 = 1) \quad 011 \rightarrow (x_1 = 0 \wedge x_2 = 1 \wedge x_3 = 1)$$

But again, we don't care if x_2 is 0 or 1 in 101, and similarly, we don't care if x_1 is 0 or 1 in 011. Thus, we can simply write:

$$101 \rightarrow (x_1 = 1 \wedge x_3 = 1) \quad 011 \rightarrow (x_2 = 1 \wedge x_3 = 1)$$

Which can be represented as:

$$101 \rightarrow x_1x_3 \quad 011 \rightarrow x_2x_3$$

Finally, we can combine all the invalid states together to get the penalty term:

$$\text{penalty}(x_1, x_2, x_3) = x_1x_2 + x_1x_3 + x_2x_3$$

Let's check the truth table to be sure:

- $\text{penalty}(0, 0, 0) = 0 + 0 + 0 = 0$ (constraint satisfied)
- $\text{penalty}(0, 0, 1) = 0 + 0 + 0 = 0$ (constraint satisfied)
- $\text{penalty}(0, 1, 0) = 0 + 0 + 0 = 0$ (constraint satisfied)
- $\text{penalty}(0, 1, 1) = 0 + 0 + 1 = 1$ (constraint not satisfied)
- $\text{penalty}(1, 0, 0) = 0 + 0 + 0 = 0$ (constraint satisfied)
- $\text{penalty}(1, 0, 1) = 0 + 1 + 0 = 1$ (constraint not satisfied)
- $\text{penalty}(1, 1, 0) = 1 + 0 + 0 = 1$ (constraint not satisfied)
- $\text{penalty}(1, 1, 1) = 1 + 1 + 1 = 3$ (constraint not satisfied)

Everything checks out! So the **penalty term** can be defined as:

$$\text{penalty}(x_1, x_2, x_3) = x_1x_2 + x_1x_3 + x_2x_3$$

In general, a constraint is converted into a QUBO penalty by building a polynomial that is:

$$\text{penalty}(\mathbf{x}) = 0$$

For all **feasible assignments**, and:

$$\text{penalty}(\mathbf{x}) > 0$$

For all **infeasible assignments**. Therefore, **constraints** are not imposed explicitly: they are transformed into **energy costs**. The optimizer is still free to choose any binary assignment, but **infeasible assignments become energetically unfavorable**.

A useful way to construct penalty terms is:

1. Identify the **assignments that violate** the constraint;
2. **Build a detector polynomial** for each forbidden assignment or forbidden pattern;
3. **Sum all detector polynomials** to get the overall penalty function;
4. *Optionally* multiply the result by a penalty coefficient $\gamma > 0$.

For binary variables, the basic translations are:

$$\begin{aligned} x_i = 1 &\rightarrow x_i, & x_i = 0 &\rightarrow 1 - x_i, \\ \text{AND} &\rightarrow a \wedge b \rightarrow ab, & \text{OR} &\rightarrow \neg(\neg x_1 \wedge \neg x_2) \rightarrow 1 - (1 - x_1)(1 - x_2) \end{aligned}$$

Thus, the **penalty term acts as an energy detector** for constraint violations: feasible solutions have zero extra energy, while infeasible solutions receive a positive energy penalty.

7.2.4 Penalty Coefficients

Suppose our original objective is simply:

$$\min y(x)$$

And we have a constraint:

$$x_1 + x_2 \leq 1$$

We know the penalty is (page 420):

$$\text{penalty}(x) = x_1 x_2$$

However, in QUBO we need to minimize:

$$\min y(x) + \gamma \cdot \text{penalty}(x)$$

Where γ is the **penalty coefficient**.

❓ Why do we need γ ? Because the objective function and the penalty may have **completely different scales**. For example, let's say that the feasible solution has $y(x) = 100$ and a penalty of 0, while the infeasible solution has a much smaller objective value, say $y(x) = 1$, but a penalty of 1. **Without** a coefficient, the **infeasible solution would be preferred**, which is not what we want.

⚠️ γ must be chosen carefully!

- **Penalty too small.** Suppose we have $\gamma = 0.1$. Then, the infeasible solution would have a total cost of $1 + 0.1 \cdot 1 = 1.1$, which is still less than the feasible solution's cost of 100. Therefore, the infeasible solution would still be preferred. In general, **if the penalty is too small, it may lead to the selection of unfeasible solutions.**
- **Penalty large enough.** Suppose we have $\gamma = 100$. Then, the infeasible solution would have a total cost of $1 + 100 \cdot 1 = 101$, which is greater than the feasible solution's cost of 100. Therefore, the feasible solution would be preferred. In general, **if the penalty is large enough, it will ensure that feasible solutions are preferred over infeasible ones.**
- **Penalty too large.** Suppose we have $\gamma = 1000$. Then, the infeasible solution would have a total cost of $1 + 1000 \cdot 1 = 1001$, which is much greater than the feasible solution's cost of 100. Therefore, the feasible solution would be strongly preferred. In general, **if the penalty is too large, it may overwhelm the problem and lead to non optimal solutions.**

In practice, γ is simply an **hyperparameter** because there is no formula that tells us the perfect value for it. The modeler chooses it. However, we can **start with a penalty coefficient close to one and fine tune from there**.

7.2.5 Equality Constraints in QUBO

Up to now, we have seen examples such as (page 421):

$$x_1 + x_2 = 1$$

With penalty terms such as:

$$1 - x_1 - x_2 + 2x_1x_2$$

However, *how did we know that this was the correct penalty?*

Consider a generic optimization problem:

$$\arg \min_{\mathbf{x}} \mathbf{x} Q \mathbf{x}^T$$

Subject to the constraint (a *generic linear equality constraint*):

$$A\mathbf{x} = b$$

Where A is a matrix and b is a vector. The first part $\mathbf{x} Q \mathbf{x}^T$ is the **objective function**. The second part $A\mathbf{x} = b$ is an **equality constraint** that we would like to enforce.

⚠ The Problem. However, QUBO is **unconstrained**. Therefore we cannot directly enforce the constraint $A\mathbf{x} = b$ inside the optimizer. Instead, we must **transform the constraint into an energy penalty**.

✂ Measuring the Violation. The expression $A\mathbf{x} - b$ tells us how much the constraint is violated. If the constraint is **satisfied**:

$$A\mathbf{x} = b$$

Then the violation is zero (ground state):

$$A\mathbf{x} - b = 0$$

If the constraint is **violated**, then the violation is non-zero (excited state):

$$A\mathbf{x} - b \neq 0$$

⚠ Making the Error Always Positive. We want to penalize the violation of the constraint. However, the violation $A\mathbf{x} - b$ can be **positive or negative**. To make it **always positive**, the simplest way is to **square the violation**:

$$\text{penalty}(\mathbf{x}) = \gamma \cdot (A\mathbf{x} - b)^T (A\mathbf{x} - b) \quad (148)$$

We take the transpose of the violation and multiply it by itself, because we need to convert a vector of violations into a single scalar penalty by taking its squared Euclidean norm. In simple terms, we are summing the squares of the individual violations to get a single penalty value that represents the overall

violation of the constraint. However, for the sake of simplicity, we can also write the penalty as:

$$\text{penalty}(\mathbf{x}) = \gamma \cdot (\mathbf{Ax} - b)^2$$

Where the square is applied element-wise to the vector of violations, and then we sum the resulting values to get the total penalty.

In summary, when we have an **equality constraint**:

$$\arg \min_{\mathbf{x}} \mathbf{x}Q\mathbf{x}^T \quad \text{subject to} \quad \mathbf{Ax} = b$$

We have a beautiful universal rule to transform it into a penalty term:

$$\text{penalty}(\mathbf{x}) = \gamma \cdot (\mathbf{Ax} - b)^T (\mathbf{Ax} - b)$$

Therefore, we can rewrite the original optimization problem as:

$$\arg \min_{\mathbf{x}} \mathbf{x}Q\mathbf{x}^T + \gamma \cdot (\mathbf{Ax} - b)^T (\mathbf{Ax} - b) \quad (149)$$

Where γ is the penalty coefficient. In general, **to convert an equality constraint into a QUBO penalty, compute the constraint violation and square it.**

Example 9: Applying the Universal Rule for Equality Constraints in QUBO

Let's consider the constraint $x_1 + x_2 = 1$ (page 421). Following the universal rule, we can write the penalty term as:

- Move everything to the left-hand side:

$$x_1 + x_2 - 1 = 0$$

- Square the violation:

$$(x_1 + x_2 - 1)^2$$

- Expand the square:

$$x_1^2 + 2x_1x_2 + x_2^2 - 2x_1 - 2x_2 + 1$$

- Since $x_1^2 = x_1$ and $x_2^2 = x_2$ for binary variables, we can simplify the expression to:

$$1 - x_1 - x_2 + 2x_1x_2$$

As we can see, we have obtained the same penalty term that we had before. This is not a coincidence, but rather a consequence of the universal rule for transforming equality constraints into penalty terms.

7.3 From QUBO to Quantum Systems

So far, we have studied QUBO purely as a mathematical formulation for optimization problems. We have seen how objective functions, Boolean logic, and constraints can all be represented through an energy function whose minimum corresponds to the desired solution. At this point, however, a fundamental question arises: *why describe an optimization problem in terms of energy?*

The answer is that **energy is a central concept in physics**. Physical systems naturally evolve toward low-energy configurations, and quantum systems are no exception. If we can encode the energy landscape of a QUBO problem into a quantum system, then **finding the minimum of the QUBO** becomes **equivalent** to finding the **lowest-energy state of that quantum system**.

This observation provides the bridge between:

- **Classical Optimization**, represented by a QUBO objective function;
- **Quantum Mechanics**, where the energy of a system is described by a Hamiltonian.

The goal of the next sections is therefore to show **how a QUBO energy landscape can be translated into a quantum-mechanical description**. Ultimately, minimizing a QUBO problem will become equivalent to finding the ground state of a suitable Hamiltonian.

7.3.1 Hamiltonian Recap

Before connecting QUBO problems to quantum systems, we need to introduce one of the most important concepts in quantum mechanics: the Hamiltonian. Very informally, the **Hamiltonian** is the **mathematical object** that describes the energy of a quantum system and determines how the system evolves over time.

❓ What is a Hamiltonian?

First of all, the Hamiltonian is an **operator**. An operator is simply a mathematical object that acts on another object and transforms it in some way.

For example, the derivative is an operator:

$$D[f(x)] = \frac{d}{dx}f(x)$$

Here, the operator D takes a function $f(x)$ and produces its derivative $\frac{d}{dx}f(x)$. For instance, if:

$$f(x) = x^2 \Rightarrow D[f(x)] = 2x$$

Similarly, a Hamiltonian takes a quantum state and acts on it. Imagine a physical system, such as a collection of interacting qubits. At any instant, the system has some energy. The Hamiltonian is the mathematical object that tells us:

1. **How much energy every possible state has**
2. **How the system evolves over time**

So, the Hamiltonian is the *rulebook* that assigns an energy to every possible state of a system.

The landscape introduced for QUBO (page 415) is a great **analogy**. For XOR, we had an energy landscape where each point corresponds to a particular assignment of the variables x_1 and x_2 . Each state (point) has an associated energy. The Hamiltonian does exactly the same thing for a quantum system. In some sense, the Hamiltonian is the **mathematical representation of the energy landscape**.

✂ Quantum Mechanics View

Historically, in **classical mechanics**, the Hamiltonian is often written as:

$$H = T + V$$

Where T is the kinetic energy and V is the potential energy. For example, for a ball:

$$H = \frac{1}{2}mv^2 + mgh$$

It represents the total energy of the system, which is the sum of kinetic and potential energy, called Hamiltonian.

However, in **quantum mechanics** things become more complicated. A quantum state is represented by a vector $|\psi\rangle$. The Hamiltonian becomes a **matrix** (more generally, an **operator**): $\hat{H}$. Instead of assigning a single number directly, it **acts on quantum states** $\hat{H}|\psi\rangle$. The result tells us information about the **energy of the state** and **how the state evolves over time**. Unlike quantum gates, the Hamiltonian describes the physical system, exists naturally within it, and determines its energy and evolution. It does not perform a computation or change the state in a specific way.

A Why does the Hamiltonian determine time evolution?

We have seen that the Hamiltonian assigns an energy to every state naturally. But it also **determines how the system evolves over time**. Let's briefly explain why.

We already know that a quantum state is $|\psi\rangle$. Suppose at time $t = 0$ we have a state $|\psi(0)\rangle$. Now, the question: *what will the state be one second later?* In classical mechanics we would use Newton's laws to determine the future state, like $F = ma$, which tells us how the position and velocity of an object change over time. Instead, in quantum mechanics, we use the Schrödinger equation:

$$i\hbar \frac{\partial}{\partial t} |\psi(t)\rangle = \hat{H} |\psi(t)\rangle \quad (150)$$

The important thing is not the formula itself (which we will not analyze in detail in this course), but the fact that **Schrödinger equation predicts future quantum state**. And the **object appearing inside the Schrödinger equation is the Hamiltonian**. Therefore, if we know the Hamiltonian, we can predict the future evolution of the quantum system.

In other words, **if we know the Hamiltonian $\hat{H}$ of a quantum system, then the Schrödinger equation tells us how the quantum state evolves in time**. Therefore, knowing the **Hamiltonian allows us to predict the future** state of the system.

? But how does a Hamiltonian store energies?

First of all, let's remember where we came from. We built an energy landscape (page 410) where each state (configuration of variables) has an associated energy. The optimization problem is:

$$\min E(x)$$

Where we want to find the state with the lowest energy.

? Now, what do we need from a quantum system? If we want a quantum system to solve this problem, we need a way to tell the quantum system: “Hey, for this state, the energy is E_1 . For that state, the energy is E_2 . For that other state, the energy is E_3 ”. In other words, we need a way to **encode the energy landscape into the quantum system**. So, *how do we store these energies inside a quantum system?* Simple, we use the Hamiltonian.

❓ **But how does a Hamiltonian store energies?** This is where eigenvalues and eigenvectors come into play. When we have a Hamiltonian $\hat{H}$, we can find its eigenvalues and eigenvectors:

$$\hat{H} |\phi_i\rangle = E_i |\phi_i\rangle \quad (151)$$

First of all, we apply the Hamiltonian to the eigenvector $|\phi_i\rangle$. The result is the same vector $|\phi_i\rangle$ multiplied by a number E_i . This number E_i is called the **eigenvalue** corresponding to the **eigenvector** $|\phi_i\rangle$.

For example, if we have a Hamiltonian $\hat{H}$:

$$\hat{H} = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$$

And the state:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Applying the Hamiltonian to the state gives us:

$$\hat{H} |0\rangle = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = 1 |0\rangle$$

Here, the eigenvalue is $E_0 = 1$ corresponding to the eigenvector $|0\rangle$. Similarly, for the state:

$$|1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Applying the Hamiltonian gives us:

$$\hat{H} |1\rangle = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 3 \end{bmatrix} = 3 |1\rangle$$

Here, the eigenvalue is $E_1 = 3$ corresponding to the eigenvector $|1\rangle$. It means that the state $|0\rangle$ has energy 1 and the state $|1\rangle$ has energy 3.

The key point is that **the energy of a quantum system is encoded in the eigenvalues of the Hamiltonian**. Indeed, if $|\psi_i\rangle$ is an eigenvector of the Hamiltonian, then the corresponding eigenvalue E_i is the energy of that state.

⚠️ **Hamiltonian must be Hermitian**

First of all, a matrix is **Hermitian** if it is **equal to its conjugate transpose**:

$$H = H^\dagger$$

Where $\dagger$ means the transpose and complex conjugate of the matrix. For example:

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix} = A^\dagger$$

Similarly, the complex matrix:

$$B = \begin{bmatrix} 1 & i \\ -i & 2 \end{bmatrix} \rightarrow B^T = \begin{bmatrix} 1 & -i \\ i & 2 \end{bmatrix} \rightarrow B^\dagger = \begin{bmatrix} 1 & i \\ -i & 2 \end{bmatrix} = B$$

An interesting property of Hermitian matrices is that they have **real eigenvalues**. Taking the previous example, the eigenvalues of A are $2 \pm \sqrt{5}$, which are real numbers. The eigenvalues of B are ≈ 2.61 and ≈ 0.38 , which are also real numbers. Also, the **eigenvectors corresponding to different eigenvalues are orthogonal** (page 12).

❓ Why must energy be real? Suppose we measure the energy of a particle. The experimental result will always be a real number. Now imagine the result of the measurement returned a complex number, like $3 + 7i$ joules. *What would that even mean physically? Is the particle more energetic because of the $7i$? Can we extract $7i$ Joules of work?* There is no physical interpretation for a complex energy. Therefore, **measurable quantities must be real**, and energy is a measurable quantity, $E_i \in \mathbb{R}$. This implies that the **Hamiltonian must have real eigenvalues**. In quantum mechanics, this is guaranteed by requiring that the Hamiltonian is a **Hermitian operator**.

❓ Why is it useful that the energy eigenstates of the Hamiltonian are orthogonal? Let's **suppose** energy eigenstates were **not orthogonal**. Imagine we have two energy eigenstates $|\phi_1\rangle$ and $|\phi_2\rangle$ with two different energies. If they were not orthogonal, then projecting onto one state would partially affect the other. Then, the decomposition:

$$|\psi\rangle = c_1 |\phi_1\rangle + c_2 |\phi_2\rangle$$

Would become *messy* (i.e., the coefficients c_1 and c_2 would not be independent). The coefficients would not be easy to extract, and the energy of the state would not be well-defined.

Instead, **orthogonality gives clean probabilities**. Suppose:

$$|\psi\rangle = c_1 |\phi_1\rangle + c_2 |\phi_2\rangle$$

Because the eigenstates are orthonormal:

$$\langle\psi_0|\psi_1\rangle = 0$$

We can *easily* extract the coefficients:

$$P(E_0) = |c_1|^2, \quad P(E_1) = |c_2|^2$$

No cross terms appear. Precisely, **each energy eigenstate evolves independently**. Again, because the eigenstates are orthogonal, the evolution of one state does not affect the other.

In simple terms, if our Hamiltonian has two energies $E_1 \neq E_2$, when we measure energy, we want the outcomes to be either E_1 or E_2 , with probabilities $P(E_1)$ and $P(E_2)$. If the eigenstates were not orthogonal, the measurement outcomes would not be well-defined, and we would not be able to assign clear probabilities to each energy level. Therefore, **orthogonality of energy eigenstates is crucial for a consistent physical interpretation of quantum mechanics**.

7.3.2 The Eigenspectrum

Before connecting QUBO problems to quantum systems, we need to understand one final concept: the **eigenspectrum** of a Hamiltonian. The definition of eigenvectors and eigenvalues is the following. Given a matrix A , an eigenvector x and its corresponding eigenvalue λ satisfy the following equation:

$$Ax = \lambda x$$

For a Hamiltonian, the same concept appears as (page 434):

$$\hat{H} |\psi_i\rangle = E_i |\psi_i\rangle$$

Where $\hat{H}$ is the Hamiltonian matrix, $|\psi_i\rangle$ is the i -th eigenvector (also called eigenstate), and E_i is the corresponding eigenvalue (also called energy level). Therefore, the **possible measurable energies of a quantum system are the eigenvalues of its Hamiltonian**.

❓ What does the $\hat{H} |\psi_i\rangle = E_i |\psi_i\rangle$ equation mean? It means that when the Hamiltonian acts on the state $|\psi_i\rangle$, the result is the same state multiplied by a scalar E_i . In other words, the **state is not transformed into a different state; it is only scaled**. This is exactly the defining property of an eigenvector. The associated scalar E_i is the energy of that state.

📖 Stationary States. The importance of the eigenvalue equation is not only mathematical. In quantum mechanics, the eigenvectors of the Hamiltonian correspond to **physically special states**, called **stationary states**, while the **eigenvalues correspond to the possible measurable energies of the system**. Therefore, studying the eigenspectrum of the Hamiltonian allows us to understand the energy structure of the quantum system.

Let's start from a state that is an eigenvector:

$$H |v_1\rangle = E_1 |v_1\rangle$$

This state has **one single energy** E_1 . There aren't multiple energies involved. Because there is only one energy, the time evolution is:

$$|v_1(t)\rangle = e^{-iE_1 t/\hbar} |v_1\rangle$$

The equation above is derived from the Schrödinger equation, which describes how quantum states evolve over time. It is not important to understand the details of the derivation, but the key point is that the whole state gets multiplied by the same phase factor $e^{-iE_1 t/\hbar}$. Nothing inside the state can “*get out of sync*”, therefore all measurement probabilities remain unchanged. This is why eigenstates are called **stationary states**: they do **not change their measurement probabilities over time**. They evolve only through a global phase factor. Since global phases have no observable effect, all measurement probabilities remain unchanged over time.

However, consider a superposition of two eigenstates:

$$|\psi\rangle = \frac{1}{\sqrt{2}} |v_1\rangle + \frac{1}{\sqrt{2}} |v_2\rangle$$

Here there are **two energies** involved E_1 and E_2 . The time evolution is:

$$|\psi(t)\rangle = \frac{1}{\sqrt{2}}e^{-iE_1t/\hbar}|v_1\rangle + \frac{1}{\sqrt{2}}e^{-iE_2t/\hbar}|v_2\rangle$$

Now the two components of the state evolve with different phase factors. The first component rotates with E_1 , while the second component rotates with E_2 . Then, if $E_1 \neq E_2$, they rotate at different rates. This changes the relative phase between the two components, and that can change measurement probabilities. So the **state is not stationary**.

❓ But what does Eigenspectrum mean?

The collection of all eigenvalues of a Hamiltonian is called the **Eigenspectrum** of the Hamiltonian. Let's suppose $\hat{H}$ has n eigenvalues $E_1, E_2, \dots, E_n$. The eigenspectrum is the set $\{E_1, E_2, \dots, E_n\}$. Each energy level is associated with one or more eigenstates.

Furthermore, among all eigenvalues $E_1, E_2, \dots, E_n$, the **smallest one** is called the **Ground-State Energy**:

$$E_{\text{ground}} = \min_i E_i \quad (152)$$

And the corresponding eigenvector is called the **ground state** (page 410). Instead, all higher energy levels are called **Excited States**.

❓ **Why do we care about the Eigenspectrum for a QUBO?** Recall what we want from a QUBO problem:

$$\min E(x)$$

We want the lowest energy configuration. Now look at a quantum system:

$$\hat{H}|\psi\rangle = E|\psi\rangle$$

The **lowest energy is exactly the smallest eigenvalue of the Hamiltonian**. Therefore, if we can encode the QUBO problem into a Hamiltonian, then the solution to the QUBO problem is exactly the ground state of that Hamiltonian. This is the key connection between QUBO problems and quantum systems. By studying the eigenspectrum of the Hamiltonian, we can find the optimal solution to the original QUBO problem.

7.3.3 Why Map QUBO to Quantum Mechanics?

At this point we have two seemingly unrelated descriptions of a problem.

On one side, we have a **QUBO optimization problem** (page 406):

$$\arg \min_{\mathbf{x}} E(\mathbf{x}) = \arg \min_{\mathbf{x}} \mathbf{x} Q \mathbf{x}^T$$

Where the goal is to find the binary assignment $\mathbf{x}$ that minimizes the energy function $E(\mathbf{x})$. Here, we are still in the classical mathematical formulation of the problem, and we have not yet made any connection to quantum mechanics, QUBO is not a quantum concept, but rather a classical optimization problem.

On the other side, quantum mechanics describes physical systems through a Hamiltonian $\hat{H}$ (page 432), whose eigenvalues represent the possible energy levels of the system.

The main observation is that both descriptions revolve around the same concept: **energy**.

 **The Central Idea.** Recall what we do when solving a **QUBO problem**. We assign an energy value to every possible configuration (e.g., state 00 has energy $E(00)$, state 01 has energy $E(01)$, etc.). The **solution** is simply the state with the **lowest energy**. Now, consider a quantum system. It also possesses energy levels $E_0, E_1, E_2, \dots$, and naturally tends to occupy its lowest-energy configuration, called the **ground state** (page 410), because it is the most stable. Therefore, we can think of a quantum system as an **energy minimization process**, where the system evolves towards its ground state, which corresponds to the minimum energy configuration.

Quantum System = Energy Minimization Process

 **The Opportunity.** This similarity (both QUBO and quantum systems are about energy minimization) opens up a powerful opportunity: **we can leverage the natural energy-minimizing behavior of quantum systems to solve QUBO problems**. Precisely, instead of solving the QUBO problem directly using a classical algorithm, we can:

1. Encode the QUBO energy landscape into a Hamiltonian.
2. Build a quantum system described by that Hamiltonian.
3. Let the quantum system search for its ground state.
4. Read the solution from the resulting quantum state.

Therefore, instead of viewing the optimization problem as a purely mathematical object, we reinterpret it as a physical system. The optimization then becomes:

$$\text{Find minimum of } E(\mathbf{x}) \Rightarrow \text{Find ground state of } \hat{H}$$

Or more precisely:

$$\text{Minimizing a QUBO} \iff \text{Finding the ground state of a Hamiltonian}$$

7.3.4 Encoding Bit Strings into Quantum States

 The idea of finding the ground state of a Hamiltonian to solve a QUBO problem is powerful, but it raises an important question: *how do we encode the binary variables of the QUBO problem into quantum states?*

In a QUBO problem, every candidate solution is represented by a binary string:

$$\mathbf{x} = (x_1, x_2, \dots, x_n) \quad x_i \in \{0, 1\}$$

For example, the values 00, 01, 10, and 11 are possible solutions for a problem with two binary variables. However, a **Hamiltonian acts on quantum states** rather than classical bit strings. Therefore, **before defining the Hamiltonian, we must establish a correspondence between classical configurations and quantum states.**

Mapping an Entire Bit String

Let's start with the simplest case: a classical bit. It can take on two values, 0 and 1:

$$x_i \in \{0, 1\}$$

It is simply mapped to a quantum state of a single qubit:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \leftrightarrow 0, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \leftrightarrow 1$$

This is simply the standard computational basis of quantum computing.

In general, a QUBO solution contains n bits:

$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

Each bit is mapped to a qubit. The entire string is then represented using the tensor product of the individual qubits:

$$|\mathbf{x}\rangle = |x_1\rangle \otimes |x_2\rangle \otimes \dots \otimes |x_n\rangle$$

Therefore, in general a bit string of length n corresponds to a quantum state in a 2^n -dimensional Hilbert space.

Example 10: Encoding the Bit String 10

Consider the bit string $x = 10$.

$$1 \iff |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad 0 \iff |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The corresponding two-qubit state is:

$$|10\rangle = |1\rangle \otimes |0\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

7.3.5 Constructing the QUBO Hamiltonian

In the previous section, we established a one-to-one correspondence between classical bit strings and computational basis states:

$$\mathbf{x} \longrightarrow |x\rangle$$

For example:

$$00 \longrightarrow |00\rangle \quad 01 \longrightarrow |01\rangle \quad 10 \longrightarrow |10\rangle \quad 11 \longrightarrow |11\rangle$$

We now want to go one step further. The **goal** is to **construct a Hamiltonian whose energy levels reproduce exactly the QUBO energy landscape**.

Example 11: How to Construct the QUBO Hamiltonian

Suppose we have a business problem, for example, *choose which warehouses to open* in order to minimize costs; or *choose which stocks to buy* in order to maximize returns; or *schedule jobs on machines* in order to minimize total processing time. We define binary variables $x_i \in \{0, 1\}$ where $x_i = 1$ means “open warehouse i ” or “buy stock i ” or “schedule job i on machine j ”, and $x_i = 0$ means the opposite.

Now we build a QUBO model of the problem, which is a quadratic function of the binary variables (page 407):

$$y = \sum_i a_i x_i + \sum_{i>j} q_{ij} x_i x_j$$

The goal is to find the binary string $\mathbf{x}$ that minimizes y . The coefficients come from the application domain and encode the costs, returns, processing times, etc. associated with the different choices. For example, suppose:

$$y = 3x_1 + 5x_2 - 4x_1x_2$$

Where the numbers 3, 5, and -4 might represent the costs of opening warehouses 1 and 2, and the cost reduction if both warehouses are opened together. Now, we evaluate y for all possible binary strings:

- State $|00\rangle$: $y(x_1 = 0, x_2 = 0) = 0$
- State $|01\rangle$: $y(x_1 = 0, x_2 = 1) = 5$
- State $|10\rangle$: $y(x_1 = 1, x_2 = 0) = 3$
- State $|11\rangle$: $y(x_1 = 1, x_2 = 1) = 3 + 5 - 4 = 4$

Now, we want to construct a Hamiltonian $\hat{H}$ because we want to leverage the natural energy-minimization properties of quantum systems (page 438). We are looking for a Hamiltonian such that:

$$\begin{aligned} \hat{H} |00\rangle &= 0 |00\rangle & \hat{H} |01\rangle &= 5 |01\rangle \\ \hat{H} |10\rangle &= 3 |10\rangle & \hat{H} |11\rangle &= 4 |11\rangle \end{aligned}$$

In other words, we want the Hamiltonian to assign the same energy values to the computational basis states as the QUBO function does to the corresponding binary strings. This way, the ground state of the Hamiltonian will correspond to the optimal solution of the QUBO problem.

In this example, the ground state would be $|00\rangle$ with energy 0, which means that the optimal solution is to not open any warehouses (or not buy any stocks, or not schedule any jobs, depending on the context).

In the previous example, we reached the point where we have a clear goal: we want to construct a Hamiltonian $\hat{H}$ that reproduces the energy landscape of the QUBO function. The next step is to figure out how to actually build such a Hamiltonian.

💡 The Key Idea. We need a **matrix** that **recognizes a specific basis state** and **assigns the corresponding QUBO energy only to that state**, while leaving all other basis states unaffected. The only way to achieve this is to use a **projector** (page 80) onto that specific basis state:

$$H_Q = \hat{H} = \sum_{x \in \{0,1\}^n} E(x) |x\rangle\langle x| \quad (153)$$

Where $E(x)$ is the energy assigned by the QUBO function to the binary string x (the y value in the previous example), and $|x\rangle\langle x|$ is the projector onto the computational basis state $|x\rangle$.

❓ Why do we need projectors? In general, we want a Hamiltonian operator such that:

$$\hat{H} |x\rangle = E(x) |x\rangle$$

For example, with the previous example, we want:

$$\hat{H} |00\rangle = 0 |00\rangle \quad \hat{H} |01\rangle = 5 |01\rangle \quad \hat{H} |10\rangle = 3 |10\rangle \quad \hat{H} |11\rangle = 4 |11\rangle$$

This means that we need a matrix that, when applied to $|00\rangle$, gives $0 |00\rangle$; when applied to $|01\rangle$, gives $5 |01\rangle$; and so on. Simply:

$$\hat{H} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 4 \end{bmatrix}$$

To **construct** such a **matrix**, we can **use projectors**. For instance, the projector onto $|01\rangle$ is:

$$|01\rangle\langle 01| = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

Therefore, if we want to assign energy 5 to $|01\rangle$, we can simply use $5 |01\rangle\langle 01|$:

$$5 |01\rangle\langle 01| = 5 \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 5 \end{bmatrix}$$

And so on for the other basis states. By **summing** all these **projectors weighted by the corresponding energies**, we can construct the **desired Hamiltonian** that **reproduces the QUBO energy landscape**. For example, applying this hamiltonian matrix to the state $|01\rangle$ gives:

$$\hat{H} |01\rangle = 0 \underbrace{|00\rangle\langle 00|}_{\langle 00|01\rangle=0} |01\rangle + 5 \underbrace{|01\rangle\langle 01|}_{\langle 01|01\rangle=1} |01\rangle + 3 \underbrace{|10\rangle\langle 10|}_{\langle 10|01\rangle=0} |01\rangle + 4 \underbrace{|11\rangle\langle 11|}_{\langle 11|01\rangle=0} |01\rangle = 5 |01\rangle$$

This is exactly what we wanted: the Hamiltonian assigns energy 5 to the state $|01\rangle$, which corresponds to the QUBO energy for the binary string 01, and **leaves all other states unaffected thanks to the orthonormality of the computational basis states**.¹⁵

✓ What Have We Achieved?

We can now summarize the entire construction. **Starting from a QUBO problem**, we first **encode** each **binary string** x as a **computational basis** state $|x\rangle$. Then, using the QUBO energy function $E(x)$, we construct the **Hamiltonian**:

$$\hat{H} = \sum_{x \in \{0,1\}^n} E(x) |x\rangle\langle x|$$

The purpose of this Hamiltonian is to **reproduce exactly the original QUBO energy landscape**. In particular:

$$\hat{H} |x\rangle = E(x) |x\rangle$$

Therefore:

- The **computational basis** states $|x\rangle$ become the **eigenstates** of the Hamiltonian;
- The **QUBO energies** $E(x)$ become the **eigenvalues** of the Hamiltonian;
- The **eigenspectrum** (i.e., the set of all eigenvalues) of the Hamiltonian coincides with the **energy landscape of the QUBO problem**.

As a consequence, minimizing the QUBO function $\min_x E(x)$ is equivalent to finding the lowest-energy eigenstate (the **ground state**) of the Hamiltonian:

$$x^* = \arg \min_x E(x) \iff |x^*\rangle \text{ is a ground state of } \hat{H}$$

This is the fundamental bridge between classical optimization and quantum mechanics. ✗ Instead of **searching** directly for the **optimal binary string**, ✓ we can reformulate the problem as the **search** for the **ground state of a quantum system**. This observation is the foundation of quantum optimization techniques such as the Ising model and quantum annealing, which will be introduced in the next sections.

¹⁵A set of vectors (in this case, the computational basis states) is orthonormal (page 19) when it satisfies two conditions: (1) different vectors are perpendicular (their dot product is zero) $\langle v_i | v_j \rangle = 0$ for $i \neq j$ (*orthogonality*); and (2) each vector has unit length $\langle v_i | v_i \rangle = 1$ (*normalization*). In general, an orthonormal basis satisfies $\langle v_i | v_j \rangle = \delta_{ij}$, where δ_{ij} is the Kronecker delta (page 139), which is 1 if $i = j$ and 0 otherwise. This property ensures that **each projector only affects its corresponding basis state and does not interfere with others**.

7.4 The Ising Model

In the previous section, we established a bridge between QUBO optimization problems and quantum systems by constructing a Hamiltonian whose ground state corresponds to the optimal solution of the QUBO problem.

However, the Hamiltonian introduced so far is mainly a conceptual construction. Real quantum optimization devices are typically described using a different, but closely related, mathematical model known as the **Ising model**.

The Ising model originates from statistical mechanics and was originally developed to describe magnetic materials. Today, it plays a central role in quantum optimization because many quantum annealers and optimization-oriented quantum devices naturally implement **Ising Hamiltonians**.

Therefore, before studying quantum annealing, we need to understand how the concepts introduced so far relate to the Ising formulation and how QUBO problems can be represented within that framework.

7.4.1 QUBO, Hamiltonian, and Ising: Taxonomy

At this point, several related concepts have been introduced:

- QUBO (page 406)
- Hamiltonian (page 432)
- Ising Model (only a brief introduction on the previous page)

Since they all involve energy functions and optimization, it is easy to confuse their roles. The purpose of this section is to clarify the relationship between them.

QUBO: The Optimization Problem

A **Quadratic Unconstrained Binary Optimization (QUBO)** problem is a mathematical formulation used to describe optimization problems. It specifies:

$$\min_x E(x)$$

Where the variables are binary:

$$x_i \in \{0, 1\}$$

The QUBO itself does **not** describe a physical system. Instead, it describes the objective function that we want to optimize. Examples include: scheduling, portfolio optimization, routing, and resource allocation problems. Thus, QUBO belongs primarily to the world of **optimization theory**.

Hamiltonian: The Energy Operator

A **Hamiltonian** is the mathematical object used in quantum mechanics to describe the energy of a physical system.

For a quantum state $|\psi\rangle$, the Hamiltonian determines: the possible energy levels of the system, the time evolution of the system. Mathematically:

$$\hat{H} |\psi_i\rangle = E_i |\psi_i\rangle$$

The Hamiltonian therefore belongs to the world of **physics**.

Ising Model: A Particular Energy Formulation

As we will see in the next section, the **Ising Model**, from the German physicist Ernst Ising, is a specific *mathematical model* that was originally developed to describe magnetic materials.

Instead of using binary variables:

$$x_i \in \{0, 1\}$$

It uses spin variables:

$$s_i \in \{-1, +1\}$$

The Ising Model provides a particular way of **writing the energy of a system as a function of the states of its spins**. In other words, the **Ising model is a specific mathematical form for constructing Hamiltonians**.

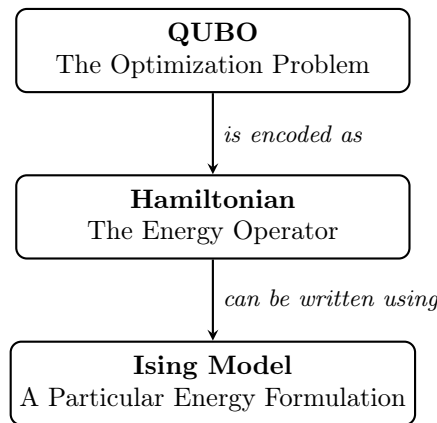


Figure 24: Relationship between QUBO, Hamiltonian, and Ising model.

? We have already seen how to construct a Hamiltonian from a QUBO. So why do we need the Ising model?

On page 440 we built a Hamiltonian directly from the QUBO energy landscape using projectors:

$$\hat{H} = \sum_{x \in \{0,1\}^n} E(x) |x\rangle\langle x|$$

✓ This was extremely useful for understanding the QUBO to Quantum Mechanics connection.

⚠ However, real **quantum annealers** (we will see them in the next sections) **do not directly implement arbitrary diagonal Hamiltonians**. Instead, they typically implement Hamiltonians based on the **Ising formulation**.

⚠ Furthermore, Hamiltonians are beautiful objects, even mathematically, but they are **not practical** for solving real-world problems. Let's imagine a problem with 100 variables, $n = 100$. Then, the Hamiltonian would be a $2^{100} \times 2^{100}$ matrix (because it acts on a Hilbert space of dimension 2^n). This is an astronomically large matrix, and nature doesn't naturally give us a machine where we can say “*please implement this arbitrary diagonal matrix*”.

Therefore, the next step is to **study the Ising model** itself and **understand how QUBO problems can be expressed using that formulation**.

7.4.2 Classical Ising Model

The **Ising Model** is a **mathematical model** originally introduced in statistical mechanics to describe **ferromagnetism**, that is, how microscopic magnetic particles interact with one another.

The central idea is extremely simple:

- We have a collection of particles.
- Each particle behaves like a tiny magnet.
- Each magnet can point either up (+1) or down (−1).

The value describing **where the tiny magnet points** is the **Spin** variable, denoted as s_i for the i -th particle. Unlike QUBO variables, which are binary:

$$x_i \in \{0, 1\}$$

The **Ising model** uses **spin variables** that can take values of either +1 or −1:

$$s_i \in \{-1, +1\}$$

≡ The Ising Energy Function

The energy of the system is given by the **Ising Energy Function**:

$$H_{\text{Ising}} = \sum_i h_i s_i + \sum_{i>j} J_{ij} s_i s_j \quad (154)$$

At first glance it looks complicated, but it is actually composed of two very simple parts.

- **Local Fields:**

$$\sum_i h_i s_i$$

The coefficient h_i represents an external magnetic field acting on spin i . We can think of it as a preference (also called “*bias*”, because it biases the spin toward a particular direction). This **coefficient encourages the spin to align in a certain direction**:

- If $h_i > 0$, the field encourages s_i to be −1 (spin **down**).
- If $h_i < 0$, the field encourages s_i to be +1 (spin **up**).

❓ **Why does a positive h_i encourage s_i to be −1? Isn’t that counterintuitive?** Seemingly, it might be counterintuitive at first, but remember that the **energy function is something we want to minimize**. If $h_i > 0$, then having $s_i = -1$ will contribute a negative term to the energy, thus lowering it. Conversely, if $s_i = +1$, it would contribute a positive term, increasing the energy. Therefore, a positive h_i encourages s_i to be −1 to minimize the energy.

Suppose $h_i = 5$, then the contribution to the energy from this term would be:

- If $s_i = +1 \rightarrow 5 \cdot (+1) = +5$ (increases energy)
- If $s_i = -1 \rightarrow 5 \cdot (-1) = -5$ (decreases energy)

Thus, the system will *prefer* $s_i = -1$ to minimize the energy when h_i is positive.

❓ **Why is it called “local fields”?** The word **local** is there because this term acts on **one spint at a time**. Each h_i only affects the energy contribution of the corresponding spin s_i , without directly involving any other spins. In contrast, the second term involves interactions between pairs of spins, which is why we call it **spin interactions**.

• **Spin Interactions:**

$$\sum_{i>j} J_{ij} s_i s_j$$

This term describes **how pairs of spins influence each other**. The coefficient J_{ij} is called the **coupling strength** between spins i and j (also called “*coupler*”, because it *couple*s the spins together and creates a connection between qubits).

❓ **Why $i > j$ and not $i \neq j$?** The notation $i > j$ is just a convention to avoid double-counting the interactions. Suppose we have 3 spins: s_1 , s_2 , and s_3 ; if we use $i \neq j$, we would get:

$$J_{12}s_1s_2 + \underbrace{J_{21}s_2s_1}_{\text{duplicate}} + J_{13}s_1s_3 + \underbrace{J_{31}s_3s_1}_{\text{duplicate}} + J_{23}s_2s_3 + \underbrace{J_{32}s_3s_2}_{\text{duplicate}}$$

But since $s_i s_j$ is the same as $s_j s_i$, we would be **counting each interaction twice**.

❓ **What does J_{ij} represent?** The coefficient J_{ij} represents the strength of the interaction between spins i and j .

Remember $s_i, s_j \in \{-1, +1\}$, so the product $s_i s_j$ can only be:

$$s_i s_j = \begin{cases} +1 & \text{same direction} \\ -1 & \text{opposite directions} \end{cases}$$

Because:

s_i	s_j	$s_i s_j$
+1	+1	+1
+1	-1	-1
-1	+1	-1
-1	-1	+1

Thus, the value of J_{ij} determines **whether the spins prefer to align in the same direction or in opposite directions**. Remember that we want to **minimize the energy**, therefore:

- * **$J_{ij} > 0$** : if the spins are aligned in the same direction ($s_i s_j = +1$), the energy contribution is $+J_{ij}$, which increases the energy. If they are in opposite directions ($s_i s_j = -1$), the contribution is $-J_{ij}$, which decreases the energy. Therefore, the system prefers $s_i s_j = -1$, meaning the **spins prefer to align in opposite directions** (one up and one down).

- * **$J_{ij} < 0$** : if the spins are aligned in the same direction ($s_i s_j = +1$), the energy contribution is $-J_{ij}$ (since J_{ij} is negative, this actually decreases the energy). If they are in opposite directions ($s_i s_j = -1$), the contribution is $+J_{ij}$ (which increases the energy). Therefore, the system prefers $s_i s_j = +1$, meaning the **spins prefer to align in the same direction** (both up or both down).

In contrast to the *local fields*, which act on individual spins, the **spin interactions** describe how pairs of spins affect each other.

❓ Where do h_i and J_{ij} come from? In the original physical context of the Ising model, h_i represents an external magnetic field applied to the system, while J_{ij} represents the intrinsic interaction strength between neighboring spins. Therefore, they are parameters that physicists would determine based on the physical properties of the material being studied.

However, in quantum optimization, we can treat h_i and J_{ij} as **tunable parameters** that we can set to encode specific optimization problems. By choosing appropriate values for h_i and J_{ij} , we can represent a wide variety of combinatorial optimization problems within the framework of the Ising model.

❓ If h_i and J_{ij} are tunable parameters, are the spins s_i also tunable? No, the spin variables s_i are **not tunable parameters**; they are the **variables we want to optimize over**. The goal is to find the configuration of spins (i.e., the values of s_i) that minimizes the energy function H_{Ising} . In other words, we choose h_i and J_{ij} so that the energy function represents our optimization problem, and then we **search for the spin configuration that yields the lowest energy, which corresponds to the optimal solution of the problem we are trying to solve**.

In other words, in optimization problems, the coefficients h_i and J_{ij} define the energy landscape, while the spin variables s_i represent the candidate solution. Solving the problem means finding the spin configuration that minimizes the Ising Hamiltonian.

❓ Wait, the Ising model equation looks a lot like the QUBO energy function. Are they related?

The QUBO energy function is given by (page 407):

$$E(x) = \sum_i a_i x_i + \sum_{i>j} q_{ij} x_i x_j$$

Similarly, the Ising energy function is given by:

$$H_{\text{Ising}} = \sum_i h_i s_i + \sum_{i>j} J_{ij} s_i s_j$$

Notice that they have the same structure: a linear term (the first sum) and a quadratic term (the second sum).

QUBO	Ising
$x_i \in \{0, 1\}$	$s_i \in \{-1, +1\}$
a_i	h_i
q_{ij}	J_{ij}

This similarity is really important, because the Ising energy function is essentially another way of writing an optimization problem, just like the QUBO energy function. The only **major difference is the domain of the variables**: QUBO uses binary variables (0 and 1), while the Ising model uses spin variables (-1 and $+1$).

7.4.3 Quantum Ising Model

In the previous section, we studied the classical **Ising model**, where each spin is represented by a classical variable:

$$s_i \in \{-1, +1\} \quad \forall i \in \{1, \dots, n\}$$

Where n is the number of spins in the system. The energy of the system was:

$$H_{\text{Ising}} = \sum_i h_i s_i + \sum_{i>j} J_{ij} s_i s_j$$

This equation assigns an energy to each configuration of the spins. However, the spins are still classical objects that can only be in one of two possible states.

⚙ From Classical Spins to Quantum Spins

To obtain a quantum model, we replace the classical spin variables s_i with quantum operators. But the natural question is: *which quantum operators should we use to represent the spins?*

We want something that behaves like a spin. Since a spin can be in two states:

$$s_i \in \{-1, +1\}$$

We look for a quantum operator whose eigenvalues are also $\{-1, +1\}$. The Pauli Z operator is a perfect candidate for this:

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Because its eigenvectors are $|0\rangle$ and $|1\rangle$:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

And its eigenvalues are $+1$ and -1 :

$$Z|0\rangle = +1|0\rangle, \quad Z|1\rangle = -1|1\rangle$$

This means that we can represent the spin s_i as the operator Z_i , where Z_i acts on the i -th qubit:

Classical Spin	Quantum Spin
+1	$ 0\rangle$
-1	$ 1\rangle$

This mapping allows us to rewrite the classical Ising Hamiltonian in terms of quantum operators:

$$\hat{H}_{\text{Ising}} = \sum_i h_i Z_i + \sum_{i>j} J_{ij} Z_i Z_j \quad (155)$$

The structure is identical to the classical case, the only difference is that the classical variables have been replaced by quantum operators.

7.5 Quantum Annealing

In the previous sections, we established that a QUBO problem can be transformed into an Ising Hamiltonian whose ground state corresponds to the optimal solution of the original optimization problem.

However, knowing the Hamiltonian is not enough. We still need a mechanism capable of driving the quantum system toward its ground state.

Quantum Annealing is a **computational paradigm designed precisely for this purpose**. Instead of searching directly among all possible solutions, the system is initialized in the ground state of a simple Hamiltonian and then slowly evolved toward the Hamiltonian that encodes the optimization problem.

If this evolution is performed sufficiently slowly, the system tends to remain in its instantaneous ground state throughout the process. As a consequence, the final state of the system approximates the ground state of the problem Hamiltonian, which corresponds to the optimal solution of the original QUBO problem.

If these concepts seem too abstract, don't worry! We will cover the specifics of quantum annealing in the following sections.

7.5.1 Annealing Hamiltonian

Suppose we start with a QUBO:

$$E(\mathbf{x}) = \mathbf{x}Q\mathbf{x}^T$$

For example:

$$E(x_1, x_2) = x_1 + x_2 - 2x_1x_2$$

This is just a classical optimization problem. Now we convert it into an Ising formulation (page 446):

$$H_{\text{Ising}} = \sum_i h_i s_i + \sum_{i>j} J_{ij} s_i s_j$$

Then we move from classical spins to quantum spins, and we get the quantum Ising Hamiltonian (page 450):

$$\hat{H}_{\text{Ising}} = \sum_i h_i Z_i + \sum_{i>j} J_{ij} Z_i Z_j$$

Now we have built a quantum Hamiltonian that encodes the optimization problem. We have a mathematical operator whose energies correspond to the QUBO cost. For example:

$$h_1 = 2, \quad h_2 = -1, \quad J_{12} = 3$$

With the Hamiltonian:

$$\hat{H}_{\text{Ising}} = 2Z_1 - Z_2 + 3Z_1Z_2$$

To find the ground state of this Hamiltonian, we can compute the energy for each configuration of the spins. For example:

s_1	s_2	$E(s_1, s_2)$
+1	+1	$2(1) - 1(1) + 3(1)(1) = 4$
+1	-1	$2(1) - 1(-1) + 3(1)(-1) = -2$
-1	+1	$2(-1) - 1(1) + 3(-1)(1) = -4$
-1	-1	$2(-1) - 1(-1) + 3(-1)(-1) = 0$

The ground state is the configuration with the lowest energy, which is $s_1 = -1$ and $s_2 = +1$, with energy -4 . This means that the optimal solution to the original QUBO problem is $x_1 = 0$ and $x_2 = 1$ (since $s_i = 2x_i - 1$).

⚠ The Problem. If the number of spins is small, like in the example above, we can easily compute the energy for each configuration and find the ground state. However, as the number of spins increases, the number of configurations grows exponentially (2^n), making it infeasible to compute the energy for all configurations. So, the question is: *instead of checking all 2^n configurations one by one, can a quantum system naturally evolve toward the ground state?*.

✓ The main idea of Quantum Annealing

Instead of starting directly from the problem Hamiltonian $\hat{H}_{\text{Ising}}$, we start from a very simple Hamiltonian whose ground state is easy to prepare. Then we slowly transform that simple Hamiltonian into the Hamiltonian encoding our optimization problem. If the evolution is *slow enough*, the quantum system tends to remain in its ground state throughout the process. At the end, the ground state contains the solution of the optimization problem.

Definition 1: Quantum Annealing

The name *annealing* comes from metallurgy. Imagine a piece of metal: (1) heat it up; (2) at high temperature, atoms move freely; (3) cool it down slowly; (4) the atoms settle into a low-energy configuration. This process is called **annealing**.

In quantum computing, we can *mimic* processes using **quantum evolution** instead of temperature. The system starts in the ground state of the easy Hamiltonian and, if we evolve slowly enough, ends in the ground state of the problem Hamiltonian.

Therefore, **Quantum Annealing** is a technique that finds the solution of an optimization problem by encoding the problem into a Hamiltonian and then slowly transforming an easy quantum system into that problem Hamiltonian, hoping the system continuously follows the ground state until it reaches the optimal solution.

The whole method relies on the **Adiabatic Theorem**, which states that if a quantum system starts in the ground state and the Hamiltonian changes sufficiently slowly, the system remains in the instantaneous ground state.

To achieve this, **quantum annealing** uses a time-dependent Hamiltonian of the form:

$$H(s) = -\frac{A(s)}{2} \left(\sum_i \hat{\sigma}_x^{(i)} \right) + \frac{B(s)}{2} \left(\sum_i h_i \hat{\sigma}_z^{(i)} + \sum_{i>j} J_{ij} \hat{\sigma}_z^{(i)} \hat{\sigma}_z^{(j)} \right) \quad (156)$$

Called the **Transverse-Field Ising Model (TFIM)**. At first glance this looks intimidating, but it is simply the **sum of two Hamiltonians**: the **Ising term** (the second part), which is along the Z direction because contains Pauli- Z operators, and the **transverse field term** (the first part), which is along the X direction because contains Pauli- X operators. The name **transverse field** comes from the fact that the first term creates a magnetic field that is perpendicular (transverse) to the direction of the Ising interactions (which are along the Z axis).

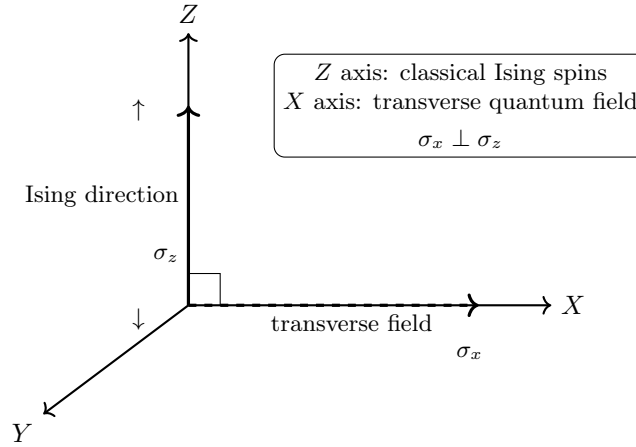


Figure 25: The quantum field acts along the X direction, perpendicular to the Ising Z direction.

- **Initial Hamiltonian:**

$$-\frac{A(s)}{2} \left(\sum_i \hat{\sigma}_x^{(i)} \right)$$

This first term is called **initial Hamiltonian** and its role is to create an easily-prepared quantum state.

❓ **What is $\hat{\sigma}_x^{(i)}$?** $\hat{\sigma}_x^{(i)}$ is the Pauli-X operator acting on the i -th qubit. It is called the **Transverse Field** and it creates quantum fluctuations (spin flips) that allow the system to explore the energy landscape.

✅ **The Standard Choice: Pauli-X Operator** The most common choice for the initial Hamiltonian is to use the Pauli-X operator, which creates a superposition of all possible states. This is because we **want to start with a state that is not biased toward any particular solution**, allowing the system to explore the entire solution space. Formally, the initial Hamiltonian is often chosen as:

$$H_A = - \sum_{i=1}^n X_i \quad (157)$$

Where X_i is the Pauli-X operator acting on the i -th qubit, and H_A is the initial Hamiltonian.

❓ **Why Pauli-X operator and not Pauli-Z?** Suppose we choose $H = -Z$. The eigenstates of Z are $|0\rangle$ and $|1\rangle$, which are also the computational basis states, and the eigenvalues are:

$$-Z|0\rangle = -1|0\rangle, \quad -Z|1\rangle = +1|1\rangle$$

Therefore, the ground state of $-Z$ is $|0\rangle$. But this is a **problem**, because the system is **already in the ground state** of the problem Hamiltonian (since $|0\rangle$ is also an eigenstate of Z). This means that the system will not evolve and we won't be able to find the solution.

On the other hand, Pauli-X operator **tries to flip the spin** and creates a superposition of states. For example, choosing $H = -X$, the eigenstates of X are:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

With eigenvalues:

$$-X|+\rangle = -1|+\rangle, \quad -X|-\rangle = +1|-\rangle$$

Therefore, the ground state of $-X$ is $|+\rangle$, which is a superposition of $|0\rangle$ and $|1\rangle$.

Roughly speaking, the superposition state $|+\rangle$ does not correspond to a single candidate solution. Instead, it places the system in a quantum state that contains **all possible computational basis** states with equal amplitude. For n qubits:

$$|+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$$

Therefore, the system is not initially biased toward any particular solution. The transverse field generated by the Pauli-X operators continuously induces spin flips and quantum fluctuations, **allowing the system to explore different regions of the energy landscape**. During the annealing process, these fluctuations are gradually reduced while the problem Hamiltonian is turned on, ideally guiding the system toward the ground state corresponding to the optimal solution.

- **Problem Hamiltonian:**

$$\frac{B(s)}{2} \underbrace{\left(\sum_i h_i \hat{\sigma}_z^{(i)} + \sum_{i>j} J_{ij} \hat{\sigma}_z^{(i)} \hat{\sigma}_z^{(j)} \right)}_{H_P}$$

This second term is called **problem Hamiltonian** and it exactly the Ising Hamiltonian we saw before (page 450). It contains the **information about the optimization problem we want to solve**. The coefficients h_i and J_{ij} are derived from the original QUBO problem, and they encode the cost function we want to minimize.

Frequently, the annealing Hamiltonian is written in a more **compact form**:

$$H(s) = A(s)H_A + B(s)H_P \quad (158)$$

Where H_A is the initial Hamiltonian and H_P is the problem Hamiltonian, and $A(s)$ and $B(s)$ are the time-dependent weights.

❓ **What are $A(s)$ and $B(s)$?** These functions **control the annealing process**. They are **time-dependent weights** that determine how much influence the initial Hamiltonian and the problem Hamiltonian have at any given time.

Typically, at the **beginning** of the annealing process ($s = 0$), we have:

$$A(0) \gg B(0)$$

This means that the system is dominated by the initial Hamiltonian, the Transverse Field, and the exploration of the energy landscape is strong. As **time progresses**, we gradually decrease $A(s)$ and increase $B(s)$, so that at the end of the annealing process, we have:

$$A(s) \ll B(s)$$

This means that the system is dominated by the problem Hamiltonian and by the optimization problem we want to solve. The hope is that if we change $A(s)$ and $B(s)$ **slowly enough**, the system will remain in its ground state throughout the evolution, and at the end, it will be in the ground state of the problem Hamiltonian, which corresponds to the optimal solution of the original QUBO problem.

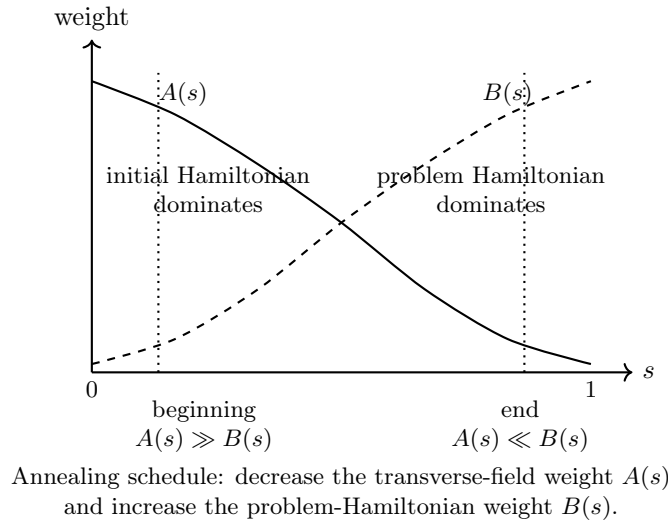


Figure 26: Typical annealing schedule controlled by the functions $A(s)$ and $B(s)$.

7.5.2 Evolution of the Eigenspectrum

Before discussing the eigenspectrum during annealing, let us recall a key result from page 431, precisely from page 436. Given a Hamiltonian:

$$\hat{H} |\psi_i\rangle = E_i |\psi_i\rangle$$

The eigenvalues E_i represent the possible energy levels of the system, while the eigenvectors $|\psi_i\rangle$ are the corresponding stationary states.

❓ Why are stationary states important? We previously saw that if a system is in an eigenstate of the Hamiltonian:

$$\hat{H} |\psi_i\rangle = E_i |\psi_i\rangle$$

Then its evolution is given by:

$$|\psi(t)\rangle = e^{-i\hat{H}t/\hbar} |\psi_i\rangle = e^{-iE_it/\hbar} |\psi_i\rangle$$

The state $|\psi_i\rangle$ only acquires a global phase factor $e^{-iE_it/\hbar}$, which does not affect the physical properties of the state. Therefore, its measurement probabilities do not change over time. This is why we call $|\psi_i\rangle$ a *stationary state*: it remains unchanged in terms of observable properties as time evolves.

❓ What changes during Quantum Annealing?

During Quantum Annealing, the Hamiltonian of the system is time-dependent:

$$H(s) = A(s)H_A + B(s)H_P$$

As s changes from 0 to 1, the **Hamiltonian evolves during the computation**. At the beginning ($s = 0$), the system is in the ground state of H_A , which is easy to prepare. As s increases, the Hamiltonian gradually transforms into H_P , whose ground state encodes the solution to our optimization problem.

Since the Hamiltonian changes with time, **its eigenvalues also change**. Instead of having fixed energies E_i , we now have:

$$E_0(s) \leq E_1(s) \leq E_2(s) \leq \dots$$

Which depend on the annealing parameter s . The **collection of these energy levels is called the eigenspectrum**. The energy E_i is the energy of the i -th eigenstate at the specific moment s of the annealing process.

It is crucial to understand that the **lowest energy level** is $E_0(s)$, which corresponds to the **instantaneous ground state**.

- $E_0(s)$ is the ground-state energy;
- $E_1(s)$ is the energy of the first excited state;
- $E_2(s)$ is the energy of the second excited state;
- ...

So instead of a single energy landscape, we can imagine a family of landscapes that slowly deform as s goes from 0 to 1.

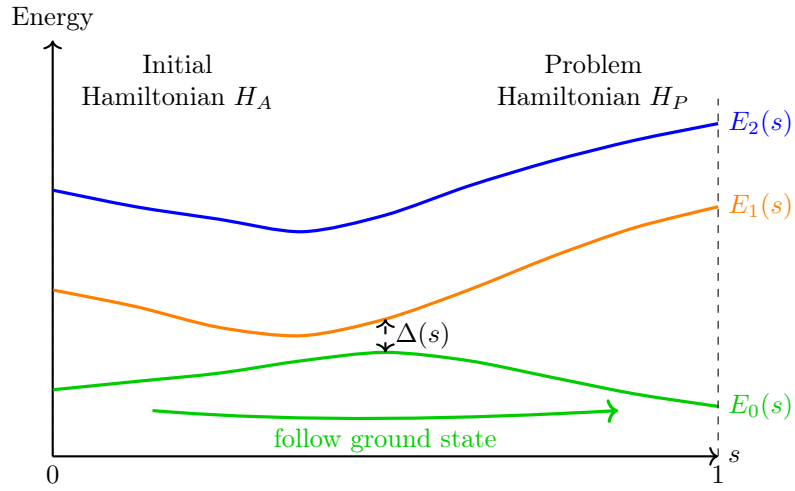


Figure 27: Schematic representation of the evolution of the eigenspectrum during Quantum Annealing. The curves represent the energy levels $E_0(s)$, $E_1(s)$, and $E_2(s)$ as functions of the annealing parameter s . The gap $\Delta(s)$ between the ground state and the first excited state is called the **Spectral Gap**, and it **measures how far the ground state is from the first excited state**. Gap is important because the smaller the gap becomes ($\Delta \approx 0$), the easier it is for the system to jump from the ground state to an excited state, which can lead to errors in the computation.

7.6 Summary

The goal of this chapter was to understand how a classical optimization problem can be transformed into a quantum optimization problem.

We started from the **Quadratic Unconstrained Binary Optimization (QUBO)** formulation (page 406):

$$\min_{\mathbf{x}} \mathbf{x} Q \mathbf{x}^T$$

Where the unknown variables are binary:

$$x_i \in \{0, 1\}$$

A QUBO problem describes an energy landscape in which each binary configuration has an associated cost (or energy). The objective is to find the configuration with minimum energy. QUBO is extremely important because many combinatorial optimization problems can be expressed in this form.

Since QUBO problems are **unconstrained**, **constraints cannot be imposed directly**. Instead, they are incorporated through penalty terms, which increase the energy of infeasible solutions while leaving feasible solutions unchanged. In this way, the optimization process naturally favors valid configurations.

The key step is establishing a connection between optimization and quantum mechanics. Each binary string (page 439):

$$\mathbf{x} \in \{0, 1\}^n$$

Is mapped to a computational basis state:

$$\mathbf{x} \longrightarrow |x\rangle$$

Using this correspondence, the QUBO energy function can be encoded into a Hamiltonian (page 432) whose eigenvalues coincide with the energies of the original optimization problem. Consequently, minimizing the QUBO becomes equivalent to finding the ground state of the Hamiltonian.

$$\arg \min E(\mathbf{x}) \iff \text{ground state of } \hat{H}$$

To better describe physical optimization systems, the problem is rewritten using the **Ising model** (page 443). Instead of binary variables:

$$x_i \in \{0, 1\}$$

The Ising formulation uses spin variables:

$$s_i \in \{-1, +1\}$$

The classical Ising energy is:

$$H_{\text{Ising}} = \sum_i h_i s_i + \sum_{i>j} J_{ij} s_i s_j$$

Where:

- h_i are the local fields (biases),
- J_{ij} are the interaction strengths (couplers).

Thus, QUBO and Ising are simply two equivalent ways of representing the same optimization problem.

The classical spins are then replaced by quantum operators, producing the quantum Ising Hamiltonian (page 450):

$$\hat{H}_{\text{Ising}} = \sum_i h_i Z_i + \sum_{i>j} J_{ij} Z_i Z_j$$

In this formulation, the eigenstates of the Hamiltonian correspond to candidate solutions, while the ground state corresponds to the optimal solution.

Finding the ground state directly is generally computationally difficult. This motivates the use of **Quantum Annealing** (page 451). The system is initially prepared in the ground state of a simple Hamiltonian:

$$H_A = - \sum_i X_i$$

Whose ground state is easy to construct. The Hamiltonian is then slowly transformed into the problem Hamiltonian that encodes the optimization problem.

The complete annealing process is described by the **Transverse-Field Ising Hamiltonian**:

$$H(s) = -\frac{A(s)}{2} \left(\sum_i \sigma_x^{(i)} \right) + \frac{B(s)}{2} \left(\sum_i h_i \sigma_z^{(i)} + \sum_{i>j} J_{ij} \sigma_z^{(i)} \sigma_z^{(j)} \right)$$

Initially, the transverse-field term dominates and creates a superposition over many possible solutions. As the annealing progresses, the problem Hamiltonian gradually takes over until the final state encodes the solution of the optimization problem.

The success of quantum annealing is explained through the evolution of the eigenspectrum (page 457). During the annealing process, the Hamiltonian possesses a sequence of energy levels:

$$E_0(s) \leq E_1(s) \leq E_2(s) \leq \dots$$

Where $E_0(s)$ is the instantaneous ground-state energy. If the evolution is sufficiently slow and the energy gap between the ground state and the excited states remains large enough, the system remains in the ground state throughout the evolution and eventually reaches the optimal solution.

In summary, the complete chain saw in this chapter is:

1. Optimization Problem
2. QUBO

3. Hamiltonian
4. Ising Model
5. Quantum Annealing
6. Ground State
7. Optimal Solution

This transformation is the fundamental idea behind quantum optimization algorithms and quantum annealing hardware.

7.7 QUBO Formulations

In this section we present a few known QUBO formulations.

❓ What is a QUBO formulation?

A **Quadratic Unconstrained Binary Optimization (QUBO)** formulation rewrites an optimization problem as the minimization of a quadratic function of binary variables:

$$\min_{\mathbf{x} \in \{0,1\}^n} E(\mathbf{x}) = \min_{\mathbf{x} \in \{0,1\}^n} \mathbf{x}^T Q \mathbf{x}$$

The variables encode candidate decisions, while the energy function assigns lower values to better solutions. Since the formulation is **unconstrained**, hard constraints are typically converted into **penalty terms**: the penalty is zero when a constraint is satisfied and positive when it is violated. A coefficient such as P or p controls the strength of that penalty.

The formulas below are useful as ready-made encodings. In practice, the usual workflow is:

1. **Choose binary variables** to encode the problem.
2. **Write the objective function** in terms of those variables.
3. **Add penalty terms** for any hard constraints.
4. **Expand** the resulting expression into a **quadratic polynomial**.

The coefficients of the resulting polynomial can then be placed in the QUBO matrix Q .

📋 General Penalty Conversion Rules

Classical Constraint	Equivalent Penalty
$x + y \leq 1$	$P(xy)$
$x + y \geq 1$	$P(1 - x - y + xy)$
$x + y = 1$	$P(1 - x - y + 2xy)$
$x \leq y$	$P(x - xy)$
$x_1 + x_2 + x_3 \leq 1$	$P(x_1x_2 + x_1x_3 + x_2x_3)$
$x = y$	$P(x + y - 2xy)$

Table 4: A few known constraint/penalty pairs.

The following notation is used in the formulas below:

- $G = (V, E)$ is an undirected graph.
- In graph problems, assume $(i, j) = (j, i)$ and then consider each edge only once.

- The complement graph is

$$\bar{G} = (V, \bar{E}), \quad \bar{E} = \{(i, j) : i \neq j, (i, j) \notin E\}$$

- In Set Packing, c_i is the capacity of set S_i , $B \in \{0, 1\}^{m \times n}$ is a constraint matrix (m constraints and n variables), and $\vec{1}$ is a vector of n elements, all equal to 1.
- In CSPs, C is a symmetric cost matrix, A is a constraint matrix, and b is a target vector.

7.7.1 Max-Cut

🔍 What it solves. Partition the nodes of a graph into two sets S and $V - S$ so as to maximize the number of edges crossing between them.

📖 Variable definition.

$$x_i = \begin{cases} 1, & i \in S, \\ 0, & \text{otherwise.} \end{cases}$$

💰 Cost function.

$$\min - \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j) \quad (159)$$

⚠️ Penalty. None.

7.7.2 Maximum Independent Set

🔍 What it solves. Find the largest set of nodes S with no edges connecting them.

📖 Variable definition.

$$x_i = \begin{cases} 1, & i \in S, \\ 0, & \text{otherwise.} \end{cases}$$

💰 Cost function.

$$\min - \sum_i^{|V|} x_i \quad (160)$$

⚠️ Penalty.

$$p \sum_{(i,j) \in E} x_i x_j, \quad p = 2 \quad (161)$$

7.7.3 Minimum Vertex Cover

🔍 What it solves. Find the smallest set of nodes C that touches every edge in the graph.

📋 Variable definition.

$$x_i = \begin{cases} 1, & i \in C, \\ 0, & \text{otherwise.} \end{cases}$$

💰 Cost function.

$$\min \sum_i^{|V|} x_i \quad (162)$$

⚠️ Penalty.

$$p \sum_{(i,j) \in E} (1 - x_i - x_j + x_i x_j), \quad p = |V| \quad (163)$$

7.7.4 Maximum Clique

🔍 What it solves. Find the largest fully connected subset of nodes (clique) C .

📋 Variable definition.

$$x_i = \begin{cases} 1, & i \in C, \\ 0, & \text{otherwise.} \end{cases}$$

💰 Cost function. This is the Maximum Independent Set formulation applied to the complement graph $\bar{G}$:

$$\min - \sum_i^{|V|} x_i \quad (164)$$

⚠️ Penalty.

$$p \sum_{(i,j) \in \bar{E}} x_i x_j, \quad p = 2 \quad (165)$$

7.7.5 Number Partitioning

🔍 What it solves. Partition a set of numbers Z into two groups Z_1, Z_2 such that the sums of the elements in Z_1 and Z_2 are as equal as possible.

📖 Variable definition.

$$x_i = \begin{cases} 1, & z_i \in Z_1, \\ 0, & z_i \in Z_2. \end{cases}$$

💰 Cost function.

$$\min \left(\sum_i^{|Z|} (2x_i - 1)z_i \right)^2 \quad (166)$$

⚠️ Penalty. None.

7.7.6 2-SAT

🔍 What it solves. Find a Boolean assignment that satisfies all clauses with two literals each.

📖 Variable definition. One binary variable x_i per Boolean literal.

💰 Cost function. Sum of the penalties associated with the individual clauses.

⚠️ Clause penalties.

$$\begin{aligned} x_i \vee x_j &\Rightarrow 1 - x_i - x_j + x_i x_j, \\ x_i \vee \bar{x}_j &\Rightarrow x_j - x_i x_j, \\ \bar{x}_i \vee \bar{x}_j &\Rightarrow x_i x_j \end{aligned} \quad (167)$$

7.7.7 Set Packing

🔍 What it solves. Select the most valuable disjoint subset of sets under mutual exclusion constraints.

📖 Variable definition.

$$x_i = \begin{cases} 1, & S_i \text{ is selected,} \\ 0, & \text{otherwise.} \end{cases}$$

💰 Cost function.

$$\min - \sum_i^n c_i x_i, \quad \text{subject to} \quad Bx \leq \vec{1} \quad (168)$$

⚠️ Penalty. Use the first and fifth formulas in Table 4, with:

$$p = \sum_i^n c_i \quad (169)$$

7.7.8 Constraint Satisfaction Problem (CSP)

🔍 What it solves. Minimize a cost function while satisfying linear equality constraints.

📖 Variable definition.

$$x_i \in \{0, 1\}$$

💰 Cost function.

$$\min x^T C x, \quad \text{subject to} \quad Ax = b \quad (170)$$

⚠️ Penalty.

$$p(Ax - b)^T(Ax - b) \quad (171)$$

8 Variational Quantum Algorithms (VQAs)

8.1 Why Variational Algorithms?

Most famous quantum algorithms assume quantum computers were perfect. For example: qubits remain coherent; gates are applied correctly; we can execute long circuits; we can use many qubits. However, on page 364 we discovered something unpleasant: **real quantum computers are noisy**. Qubits lose information because of decoherence, and gates are not perfect.

✖ **Can't we use error correction?** In principle, we could use quantum error correction (page 385) to mitigate noise. However there is a **problem**. A single logical qubit may require tens or hundreds of physical qubits to be protected against errors, and this could raise the **blow-up problem** (i.e., the number of physical qubits required to implement a quantum algorithm becomes unmanageable). So **today's quantum computers do not yet have enough qubits to run large fault-tolerant computations**.

📖 NISQ devices

The term **NISQ (Noisy Intermediate-Scale Quantum)** was introduced by John Preskill to **describe the current generation of quantum computers**. These machines are:

- **Noisy**. Qubits are extremely fragile. They suffer from decoherence, gate errors, measurement errors, and environmental disturbances. The longer the circuit, the larger the error becomes.
- **Intermediate Scale**. Today's devices contain tens of qubits, hundreds of qubits are expected in the near future, and at most a few thousand physical qubits are expected in the next decade. This sounds impressive, but it is far from what is needed for large-scale fault-tolerant quantum computing.

📖 Definition 1: NISQ Device

A **NISQ (Noisy Intermediate-Scale Quantum) device** is a **current generation quantum computer** characterized by a limited number of qubits and significant noise sources such as decoherence and gate errors. Because these devices cannot reliably execute long fault-tolerant quantum algorithms, hybrid approaches such as VQE and QAOA were developed specifically for them.

🔧 Example 1: An Analogy for the NISQ Problem

Imagine owning a Formula 1 engine. The engine can theoretically reach 350 km/h. Amazing. But suppose the fuel tank leaks after 10 seconds and we cannot finish the race. The engine is powerful, but the hardware is not mature enough. Current quantum computers are exactly like that:

- Quantum mechanics allows us to design powerful algorithms.
- Current hardware is not mature enough to run these algorithms without errors.

💡 The Key Idea: Hybrid Quantum-Classical Approaches

Researchers asked: “*instead of waiting for perfect quantum computers, can we already do something useful with today’s noisy machines?*” This is the birth of **variational quantum algorithms (VQAs)**, which are designed to be robust against noise and to work within the limitations of NISQ devices.

The core idea behind variational quantum algorithms is to **combine quantum and classical computing resources** in a hybrid approach. The quantum computer is used for a specific part of the computation, while classical methods wrap this quantum subroutine and optimize it.

❓ **Why split the computation?** Because the **two machines have different strengths**. Classical computers are very good at optimization, arithmetic, memory management, parameter updates, and decision making. Instead, quantum computers are potentially very good at representing large quantum states, exploiting superposition, exploiting entanglement, and estimating quantum properties.

❓ **What does the Classical Computer do?** The **classical computer acts like a coach**. Suppose the quantum circuit has a parameter θ . The quantum computer runs and returns a cost function $C(\theta)$. The classical computer takes this cost, updates the parameter θ (e.g., using gradient descent), and sends the new parameter back to the quantum computer. This process is repeated until the optimizer (i.e., the classical computer) finds the optimal parameter θ^* .

In this way, the quantum computer is not executing a long, complex algorithm on its own. Instead, it is performing many short executions of a parameterized quantum circuit, which is more feasible on NISQ devices. The classical computer handles the optimization and decision-making, while the quantum computer provides the quantum advantage where it can.

🔧 Example 2: Analogy

Think about a Formula 1 race. *Who wins the race?* Not only the driver, but also the engineers, the strategists, and the pit crew (i.e., the whole team). The driver does what only the driver can do, while the engineers do everything else.

🔧 Example 3: Optimizing the Quantum Circuit, Not the Solution

Imagine we want to find the minimum of a function $f(x)$. Classically we would directly try different values of x . However, in a Variational Algorithm (VQA), we do something slightly different. The parameter θ controls a quantum circuit and the optimizer changes θ until the cost function becomes minimal. So we are not directly optimizing the solution. We are optimizing the **quantum circuit that generates the solution**.

8.2 General Structure of a VQA

8.2.1 Variational Quantum Algorithms

A **Variational Quantum Algorithm (VQA)** is a hybrid algorithm in which:

1. A **parameterized quantum circuit** generates candidate solutions to a problem;
2. A **classical optimizer** evaluates those solutions;
3. The process is repeated until the best parameters are found, which correspond to the optimal solution of the problem.

The key idea is that we do **not directly program the solution**. Instead, we program a **circuit whose behavior depends on adjustable parameters**.

The Three Components

Every VQA contains exactly three fundamental ingredients:

- **Parametric Quantum Circuit (Ansatz)**. The **quantum computer executes a circuit containing adjustable parameters**. For example, if the circuit contains a rotation gate $R_y(\theta)$, then θ is a parameter that can be tuned to change the behavior of the circuit. The quantum computer therefore acts as a **state generator**, which produces different quantum states depending on the values of the parameters. We will see the ansatz in detail in the next section.
- **Classical Optimizer**. The optimizer **chooses the values of the parameters**. It is a sort of coach that guides the quantum computer towards the optimal solution. The optimizer asks itself: “*which θ should I try next?*”. As we have seen, the optimizer runs on a classical computer, and it can be implemented using various algorithms, such as gradient descent, genetic algorithms, or Bayesian optimization.
- **Iterative Execution**. The VQA is an **iterative process**: choose parameters, run the quantum circuit, measure the output, evaluate the cost function, update the parameters, and repeat if necessary. This loop continues until some stopping condition is reached.

Why Variational?

The name *variational* comes from the fact that the **parameters vary**. During the execution of the algorithm, we are constantly changing the parameters of the quantum circuit in order to find the optimal solution. Thus, the term *variational* emphasizes the **algorithm’s dynamic nature**. Rather than running a fixed circuit, the **algorithm searches through a family of quantum states generated by different parameter values**.

In summary, **Variational Quantum Algorithms (VQAs)** are **hybrid quantum-classical algorithms** consisting of three main components: a parameterized quantum circuit called an Ansatz, a classical optimizer that searches

for the best parameters, and an iterative execution loop. The quantum computer evaluates candidate solutions, while the classical optimizer updates the circuit parameters according to the measured results until an optimal solution is found.

8.2.2 Parametric Quantum Circuits (Ansätze)

Ansatz (or **Ansätze**) is a physical/mathematical term that refers to an **educated guess** for the **form of the solution**. Instead of searching among **all possible quantum states** (or in general, all possible solutions), we can restrict our search to a **subset of states** that are generated by a **parametric quantum circuit**, also called **ansatz**.

In other words, an **Ansatz** is a **parametric quantum circuit**, meaning a circuit whose behavior depends on one or more adjustable parameters. Specifically, some quantum gates contain parameters that can be changed during optimization. However, since the quantum circuit is executed on a quantum computer, the **optimizer** (which is executed on a classical computer) **cannot modify the circuit structure**, but only the values of the parameters.

❓ What is parametrized in a parametric quantum circuit?

Some gates, such as the Hadamard gate, are fixed and do not contain any parameters. On the other hand, gates like the rotation gates (e.g., $R_x(\theta)$, $R_y(\theta)$, $R_z(\theta)$, page 50) have parameters (in this case, the angle θ) that can be adjusted during the optimization process. The optimizer will modify these parameters to find the optimal solution to the problem at hand. This is extremely important because it means that with **one single quantum circuit** structure, we can **explore a wide range of quantum states by simply changing the parameters**, allowing us to efficiently search for the optimal solution **without having to design a new circuit for each possible state**.

❓ What exists the ansatz?

The output of an ansatz is not the solution itself, but rather a **quantum state candidate** that is generated by the parametric quantum circuit.

Let's suppose we have a Hamiltonian H whose ground state is $|\psi_0\rangle$. The ansatz generates a quantum state $|\psi(\theta)\rangle$ that depends on the parameters θ . The optimizer will minimize the expectation value $\langle\psi(\theta)|H|\psi(\theta)\rangle$ with respect to the parameters θ , until hopefully:

$$|\psi(\theta^*)\rangle \approx |\psi_0\rangle$$

⚠ However, there is **no guarantee** that the ansatz will be able to generate a state that is close to the ground state $|\psi_0\rangle$. An ansatz does **not solve the problem**. It **defines the set of solutions that the optimizer is allowed to search through**.

🔗 Expressibility

The **expressibility** of an ansatz is a measure of **how well the ansatz can represent a wide range of quantum states**. Different ansätze have different expressibility, and different complexity. Intuitively, an ansatz with higher expressibility can represent a larger variety of quantum states, because it has a larger search

space. However, a more expressive ansatz usually means a more complex quantum circuit, which can be more difficult to optimize, may require more quantum resources (e.g., more qubits, more gates, etc.), and may be more prone to noise and errors. Therefore, there is a **trade-off between expressibility and complexity** when designing an ansatz for a VQA.

Since the ansatz will be **executed on a NISQ device**, it should be **shallow** (i.e., it should have a small depth) and must **use gates that are native to the hardware**. In general, if the ansatz expressibility is too low, it may not be able to represent the optimal solution, and the optimization process will fail to find a good solution. On the other hand, if the ansatz expressibility is too high, noise destroys the computation. Therefore, designing an ansatz with the right balance of expressibility and complexity is crucial for the success of a VQA.

8.2.3 Classical Optimizer

The **Optimizer** is a component whose goal is to **find the parameters of the quantum circuit that optimize the desired objective function**. It **never manipulates qubits** because it is a completely classical algorithm running on a classical computer.

❓ What criteria does the optimizer use to determine the optimal parameters?

The optimizer is not guessing randomly. After each execution it **receives a score**, which is the **value of the objective function** (e.g., the expectation value of the Hamiltonian) for the current parameters, called also **cost function**.

This means that the **optimizer is guided by the cost function**, which provides feedback on how good the current parameters are. So the same ansatz (i.e., the same quantum circuit structure) can be used for completely different problems simply by changing the cost function.

⚙️ How does the optimizer update the parameters?

Usually, the optimizer is an **iterative algorithm** that updates the parameters in each iteration based on the cost function value. The optimization process continues until a stopping criterion is met, such as reaching a maximum number of iterations or achieving a cost function value below a certain threshold. Some common algorithms include gradient-based methods (e.g., gradient descent, Adam, etc.) and gradient-free methods (e.g., Nelder-Mead, COBYLA, etc.).

In summary, the Classical **Optimizer** is the classical component of a Variational Quantum Algorithm (VQA). Its role is to find the parameters of the ansatz that optimize the objective function computed from the measurement results. After each execution of the quantum circuit, the optimizer receives the cost value, updates the parameters, and repeats the process. The specific problem is defined by the objective function, while the choice of optimizer is relatively flexible.

8.2.4 Iterative Optimization Loop

The main idea behind VQAs is to leverage the power of quantum computers to evaluate a cost function that is hard to compute classically, and then use a classical optimizer to find the optimal parameters that minimize this cost function. This process is iterative, as the classical optimizer updates the parameters based on the cost function evaluations from the quantum computer.

The **iterative optimization loop** can be summarized as follows:

1. **Initialize Parameters.** We start with some **initial guess** for the parameters θ of the parametric quantum circuit (ansatz). This can be done randomly, based on some heuristic, or using prior knowledge about the problem.
2. **Run the Quantum Circuit.** We execute the parametric quantum circuit on the quantum computer with the current parameters θ to prepare a quantum state $|\psi(\theta)\rangle$. This state is the **candidate solution** currently being tested.
3. **Measure.** The quantum computer **measures the output state**. Here, the important point is that the **measurement outcomes must be classical information**. Because the **optimizer cannot directly read the quantum state**. It only receives measurement results. This step is crucial, as it bridges the quantum and classical parts of the algorithm.
4. **Compute the Cost Function.** Now that we have the measurement outcomes, we ask: “*was this candidate solution good or bad?*” To answer this, we compute the **cost function** $C(\theta)$ based on the measurement results. The cost function is designed such that its value reflects how well the current parameters θ perform with respect to the problem we are trying to solve. For example, in VQE, the cost function is the expectation value of the Hamiltonian, which we want to minimize. The key point is that the **optimizer does not need to understand quantum mechanics**; it **only needs to know the score** (cost) associated with the current parameters.
5. **Feed the Result to the Optimizer.** The computed cost function value $C(\theta)$ is then **fed into the classical optimizer**. It stores this information and uses it (later) to decide where to search for better parameters θ' in the next iteration.
6. **Update Parameters.** The classical optimizer **updates the parameters** θ based on the cost function evaluations it has received so far. This update can be done using various optimization algorithms, such as gradient descent, Nelder-Mead, or Bayesian optimization. The optimizer’s goal is to find new parameters θ' that will hopefully yield a lower cost function value in the next iteration.
7. **Repeat.** The loop then repeats from step 2 with the new parameters θ' . This iterative **process continues until a stopping criterion is met**, such as a maximum number of iterations, convergence of the cost function, or reaching a satisfactory solution.

In a Variational Quantum Algorithm, the parameters of the ansatz are initialized and the quantum circuit is executed. The output is measured and used to compute a cost function. The cost value is sent to a classical optimizer, which updates the parameters and runs the circuit again. This quantum-classical feedback loop continues iteratively until a termination condition is reached, producing the parameters that optimize the objective function.

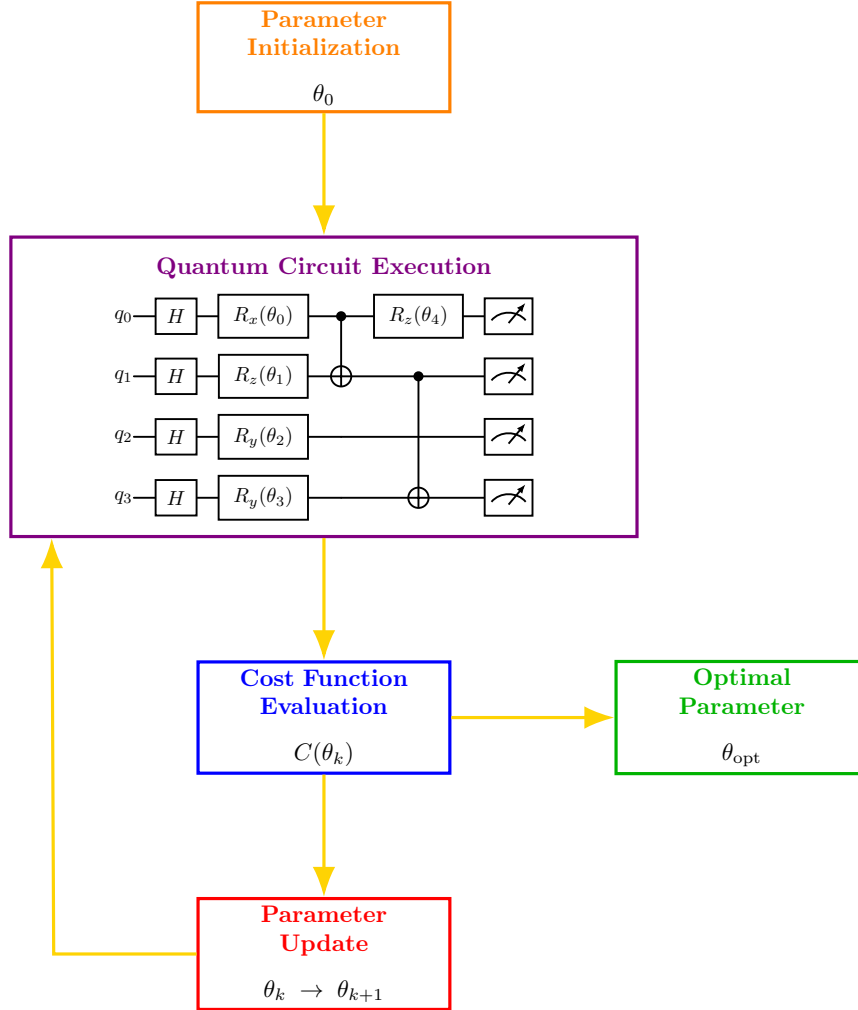


Figure 28: Schematic representation of the iterative optimization loop in a Variational Quantum Algorithm. The process starts with parameter initialization, followed by quantum circuit execution, cost function evaluation, and parameter update. The loop continues until convergence to the optimal parameters θ_{opt} .

8.3 Building Blocks of Variational Algorithms

8.3.1 Cost Function

A **Cost Function** is a **numerical measure of the quality of a candidate solution**. It maps the quantum state generated by the ansatz to a classical number that the optimizer can evaluate. The cost function is designed so that its optimum value corresponds to the solution of the original problem.

📖 The Cost Function Defines the Problem

In traditional algorithms, it is the circuit that defines the problem. In contrast, in variational quantum algorithms, it is the cost function that defines the problem. The quantum circuit (ansatz) is just a tool to explore the solution space, while the **cost function encodes the specific problem we want to solve**. For example, in VQE, the cost function is the expectation value of the Hamiltonian, which we want to minimize to find the ground state energy.

✅ What Makes a Good Cost Function?

The **minimum of the cost function must correspond to the solution of the problem**. If this is not true, the optimizer may find a minimum that has nothing to do with the problem we want to solve. Therefore, the cost function must be carefully designed to ensure that its minimum corresponds to the desired solution. Additionally, the cost function should be efficiently computable on a quantum computer, as it will be evaluated many times during the optimization process.

↔ The Cost Function Connects Quantum and Classical Worlds

The cost function serves as the **bridge between the quantum and classical parts of the algorithm**. The quantum computer produces measurements; the cost function takes these measurements and produces a classical number that the optimizer can use. This is a crucial point: the **optimizer does not need to understand quantum mechanics; it only needs to know how to evaluate the cost function based on the measurement outcomes**. The **cost function translates quantum information into a form that classical optimization algorithms can work with**.

In summary, the **Cost Function** is the quantity optimized by a Variational Quantum Algorithm (VQA). It evaluates the quality of the quantum state generated by the ansatz and provides a numerical score to the classical optimizer. The cost function must be designed so that its minimum (or maximum, depending on the formulation) corresponds to the solution of the problem. Therefore, the cost function effectively defines the problem being solved by the variational algorithm. For this reason, the cost function is the **most important part**. **Even if we have a fantastic optimizer and a powerful ansatz, if the cost function is poorly designed, we will solve the wrong problem or fail to find the correct solution.**

8.3.2 Ansatz Design

An Ansatz is not just any parameterized quantum circuit. It must be carefully designed so that:

1. The **solution can be represented**. The first requirement is **expressibility**. A highly expressive ansatz can generate many different quantum states.
2. The **circuit can run on real hardware**. The second requirement is **hardware compatibility**. The ansatz should use gates that the hardware can implement efficiently. This is crucial because every additional gate can introduce noise, errors, and decoherence issues, which is exactly the problem of NISQ devices.
3. The **optimizer can find the solution**. The third requirement is **shallow circuit depth**. A shallow circuit is less prone to noise and can be optimized more easily by the classical optimizer. Deep circuits may increase the chances of noise, gate errors, and decoherence.

 **The Fundamental Tradeoff.** There is a tension between *expressibility* and *noise*. If the ansatz is too simple, the search space may not contain the optimal solution. However, if the ansatz is too complex, it may be too noisy to run on NISQ devices.

Therefore, designing an ansatz is a delicate balance between these two factors. The **goal is to find an ansatz that is expressive enough to capture the solution while being shallow enough to be executed on real hardware without excessive noise.**

If any of these fail, the VQA may fail.

Problem-Agnostic vs Problem-Specific

There are two main approaches to ansatz design:

- **Problem-Agnostic Ansatz.** This approach uses a **general-purpose ansatz that is not tailored to any specific problem**. Examples include the hardware-efficient ansatz, which consists of layers of parameterized single-qubit rotations and entangling gates. The **advantage** of this approach is that it can be **applied to a wide range of problems without modification**. However, it may **require more parameters** and **deeper circuits** to achieve good performance, which can be problematic on NISQ devices.
- **Problem-Specific Ansatz.** This approach designs an **ansatz that is tailored to the specific problem being solved**. For example, in quantum chemistry, one can use the Unitary Coupled Cluster (UCC) ansatz, which is based on classical coupled cluster theory. The **advantage** of this approach is that it can be **more efficient** and require **fewer parameters** than a problem-agnostic ansatz. However, it may require **more domain knowledge** and may **not be applicable to other problems**.

In summary, Ansatz design is the process of choosing a parameterized quantum circuit suitable for a variational algorithm. An ideal ansatz should be shallow,

compatible with the available hardware, and sufficiently expressive to represent the solution of the problem. Since the ansatz defines the search space explored by the optimizer, it must be capable of generating the desired solution for some choice of parameters. However, greater expressibility usually comes at the cost of deeper circuits and increased noise, creating a tradeoff between accuracy and hardware limitations.

8.3.3 Optimizer Choice

So far, we have introduced two of the three fundamental building blocks of a Variational Quantum Algorithm: the **cost function** (page 477), which defines the optimization objective, and the **ansatz** (page 472 and page 478), which determines the family of quantum states that can be explored. The third essential component is the **Classical Optimizer**, whose role is to **search for the values of the variational parameters that minimize the cost function.**

At first glance, it may seem surprising that a *quantum* algorithm relies on a classical optimization procedure. However, this is precisely what makes Variational Quantum Algorithms (VQAs) **hybrid algorithms**. The quantum processor is responsible for preparing quantum states and measuring the objective function, while the classical computer analyzes the measurement results and decides how the circuit parameters should be modified in the next iteration. The **optimizer** therefore **acts as the “decision-making” component of the algorithm.**

✂ The optimization problem

Suppose our ansatz contains several parameterized rotation gates:

$$R_y(\theta_1), \quad R_z(\theta_2), \quad R_x(\theta_n), \quad \dots$$

The collection of all parameters can be represented as a vector:

$$\boldsymbol{\theta} = (\theta_1, \theta_2, \dots, \theta_m)$$

For a given choice of parameters, the quantum circuit prepares a quantum state:

$$|\psi(\boldsymbol{\theta})\rangle$$

After measuring the circuit, we evaluate the corresponding cost function:

$$C(\boldsymbol{\theta})$$

The optimization problem is therefore simply:

$$\min_{\boldsymbol{\theta}} C(\boldsymbol{\theta}) \tag{172}$$

The objective of the classical optimizer is to discover the parameter vector that minimizes this quantity. Therefore, the optimization problem being solved is determined entirely by the cost function, while the **optimizer is simply the algorithm used to search for the minimum.**

❓ How does the optimizer work?

Unlike the quantum processor, the optimizer never manipulates quantum states directly. Instead, it only **receives numerical values**. A typical optimization cycle consists of the following steps:

1. Choose a set of parameters $\boldsymbol{\theta}_0$ and send them to the quantum processor;

2. Execute the quantum circuit;
3. Measure the output;
4. Compute the cost function $C(\theta_0)$;
5. Decide how the parameters should be modified to improve the cost function (here is where the optimizer's algorithm comes into play);
6. Repeat the process.

This optimization loop continues until a stopping criterion is satisfied, such as reaching a maximum number of iterations or observing that the cost function no longer improves.

❓ Why is the optimizer necessary?

One might wonder why we cannot simply compute the best parameters directly. The reason is that the **relationship between the parameters and the cost function** is generally **highly nonlinear** and **extremely complex**.

In other words, changing even a single parameter affects the entire quantum state produced by the circuit, which in turn changes all measurement probabilities. Consequently, the optimizer has no explicit formula for determining the optimal parameters. Instead, it must **explore the parameter space by repeatedly trying different parameter values and observing how the cost changes**.

This is essentially the same idea used in many classical machine learning algorithms: rather than solving an equation analytically, the optimizer progressively improves the solution through repeated evaluations (i.e., gradient-based methods).

✂ Different optimizers can be used

In general, the **choice of optimizer is relatively free**. Unlike the ansatz, which must be implemented on the quantum hardware, the optimizer runs entirely on a classical computer. Therefore, many different optimization algorithms can be employed. Some common examples include *Gradient Descent*, *Stochastic Gradient Descent (SGD)*, *Adam*, *COBYLA*, *Nelder-Mead*, *SPSA (Simultaneous Perturbation Stochastic Approximation)*. Fortunately, this course does not require knowing the details of these algorithms. The important concept is that the optimizer is not fixed: **different optimization methods can be combined with the same variational circuit**.

✅ How to choose an optimizer? Which characteristics should it have?

Although many optimization algorithms exist, a good optimizer for Variational Quantum Algorithms (VQAs) should possess several desirable properties.

- First, it should require **as few evaluations of the quantum circuit as possible**. Each circuit execution is expensive because it involves preparing a quantum state and performing many measurements. Therefore, since each evaluation of the cost function requires executing the quantum circuit (often many times due to repeated measurements), a good optimizer should find the optimal parameters using as few quantum circuit executions as possible.
- Second, it should **converge rapidly toward a good solution**. A slow optimizer that takes many iterations to improve the cost function may not be practical, especially when each iteration requires a costly quantum circuit execution.
- Finally, it should **avoid becoming trapped in local minima**, where the optimization stops even though a better solution exists elsewhere.

For these reasons, the choice of optimizer has a significant impact on the practical performance of a Variational Quantum Algorithm. An appropriate optimizer helps reduce the number of calls to the quantum computer and prevents the optimization from getting stuck in poor solutions.

In summary, the classical optimizer is the component of a Variational Quantum Algorithm responsible for **searching for the optimal values of the variational parameters**. It receives the measured value of the cost function, updates the circuit parameters, and repeatedly executes this optimization loop until convergence. Unlike the ansatz and the quantum circuit, the optimizer is entirely classical, and many different optimization algorithms can be employed. The optimizer itself does not determine which problem is being solved; that role belongs exclusively to the objective (cost) function. Its responsibility is simply to **find the parameter values that minimize that function as efficiently as possible**.

8.3.4 Hardware Efficient Ansätze

In the previous sections, we learned that the ansatz is one of the three fundamental building blocks of a Variational Quantum Algorithm. However, **there is not a single universal ansatz**: many different circuit architectures have been proposed, each characterized by different gate arrangements, parameterizations, and expressive power.

Among the various possibilities, one of the most widely used is the **Hardware Efficient Ansatz (HEA)**. As its name suggests, this ansatz is specifically designed to be **easy to execute on real quantum hardware**, making it particularly suitable for today's NISQ devices.

❓ Why do we need a Hardware Efficient Ansatz?

When designing an ansatz, one might think that **adding more gates would always produce a better quantum circuit** because it allows the preparation of a larger variety of quantum states. While this is true in principle, there is an important **trade-off**.

⚠️ Recall that current quantum computers are affected by: decoherence (page 368), gate errors, measurement errors. As the **circuit becomes deeper**, these **errors accumulate** and progressively **destroy the quantum state** before the computation finishes. For this reason, an ideal ansatz should satisfy two seemingly conflicting objectives:

- ✓ It should be **expressive** enough to represent the solution;
- ✓ It should **remain simple** enough to be executed reliably on noisy hardware.

The Hardware Efficient Ansatz (HEA) was introduced precisely to **achieve this compromise**.

💡 Main Idea

The Hardware Efficient Ansatz is **one of the many possible ansatz architectures**. Its construction is remarkably simple; it consists of **alternating layers** of:

1. The first type of layer contains **single-qubit parameterized rotation gates** (i.e., R_y , R_z), where the rotation angles are variational parameters optimized by the classical optimizer.
2. After rotating the individual qubits, the circuit applies **entangling two-qubit gates**. These are usually CNOT gates (although other native two-qubit gates may also be used). These gates **create entanglement** among the qubits, enabling the circuit to represent quantum states that cannot be expressed as independent single-qubit states. **Without** these entangling layers, the ansatz would **only generate product states, severely limiting its expressive power**.

This basic block is then **repeated several times**, usually denoted by a depth parameter D . Each repetition introduces additional variational parameters and increases the expressive power of the circuit. However, it also increases the circuit depth, making the algorithm more susceptible to noise.

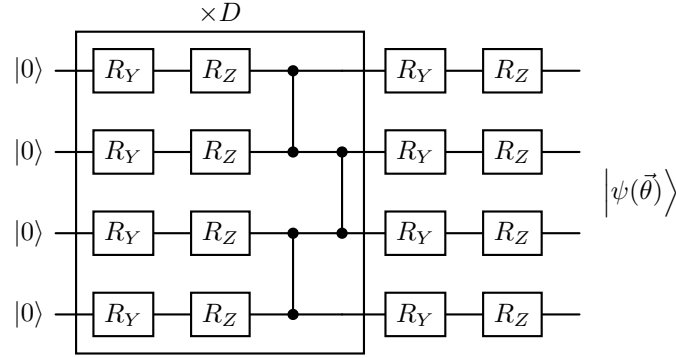


Figure 29: Generic Hardware Efficient Ansatz on 4 qubits. The final R_Y and R_Z gates are an additional final rotation layer. They are placed after the repeated entangling block because, after entanglement has been created, we may still want to independently adjust each qubit before measuring or evaluating the cost function.

❓ Why is it called Hardware Efficient?

The adjective *hardware efficient* comes from the fact that the **circuit is specifically designed to match the capabilities of existing quantum hardware**. Instead of requiring arbitrary quantum operations, it uses:

- Rotation gates that are directly supported by most quantum processors;
- Simple entangling gates (typically CNOTs) that correspond to the hardware's native two-qubit interactions.

As a result, the circuit: contains relatively few gates; is shallow; can be executed with fewer errors than more complicated ansatzes. This is particularly important for NISQ devices, where every additional gate increases the probability of errors.

✅ Advantages

The Hardware Efficient Ansatz offers several important advantages:

- ✓ **Easy to implement**, since it uses only standard quantum gates.
- ✓ **Compatible with existing hardware**, exploiting the native gate set of the processor.
- ✓ **Shallow circuits**, reducing the effects of decoherence and gate noise.
- ✓ **Highly flexible**, as increasing the depth D allows more expressive quantum states to be prepared.

⚠ Limitations

Although very practical, the Hardware Efficient Ansatz also has some drawbacks. Because it is **designed to be problem-independent**, it **does not exploit any specific knowledge about the problem being solved**. As a consequence:

- ✗ It may require many parameters;
- ✗ Optimization can become more difficult;
- ✗ Increasing the depth too much can introduce excessive noise and contribute to optimization challenges such as **barren plateaus**, which will be discussed in the next sections.

Thus, the Hardware Efficient Ansatz represents a compromise between **hardware compatibility** and **expressive power**.

In summary, the Hardware Efficient Ansatz (HEA) is one of the most commonly used ansatz architectures for Variational Quantum Algorithms. It is constructed by alternating layers of **parameterized single-qubit rotation gates** and **entangling two-qubit gates**, repeating this pattern a number of times determined by the circuit depth D . The use of simple, native quantum gates makes the ansatz particularly suitable for current NISQ devices, where shallow circuits are essential to mitigate noise. While increasing the depth improves the expressive power of the circuit, it also increases its susceptibility to errors and makes the optimization process more challenging.

8.4 Advantages and Limitations

8.4.1 Strengths of Variational Algorithms

Now that we have introduced the main components of variational quantum algorithms, we can discuss their strengths and limitations. We move from **how VQAs work** to **why they became the dominant NISQ algorithms**.

The main strengths of VQAs are:

1. **Shallow circuits are more resilient to noise.**
2. **Repeated sampling and optimization make them robust to errors.**
3. **They are highly flexible.**

For these reasons, they are considered **the most promising algorithms for NISQ devices**. Let's examine each point.

🛡️ Shallow Circuits Are More Resilient to Noise

This is probably the biggest advantage. Recall the main limitation of NISQ hardware: **more gates lead to more accumulated errors, which lowers the probability of obtaining the correct result**.

A traditional quantum algorithm may require thousands of gates, whereas a VQA typically executes a much shorter circuit. Because the **circuit is shorter, qubits spend less time evolving, fewer gate errors accumulate, and decoherence has less time to affect the computation**. Hence, shallow circuits lead to less noise and **more reliable execution**.

🔄 Repeated Sampling and Optimization Improve Robustness

At first glance, **repeatedly executing the circuit may seem inefficient**. However, repetition provides an important benefit.

Instead of trusting a single noisy measurement, the algorithm runs the circuit many times and **estimates the expectation value from multiple samples**. The optimizer then uses this more reliable estimate. So the repeated execution is not wasted effort, it **helps reduce the impact of statistical fluctuations and measurement noise**. This is why repeated sampling and optimization make VQAs more robust to errors.

✂️ Ample Flexibility

One of the nicest features of VQAs is that the **framework is very general**. Remember the three building blocks: **ansatz, cost function, and optimizer**. Each can be **modified independently**. This modularity allows researchers to adapt the algorithm to many different applications without redesigning everything from scratch.

In summary, VQAs are the most promising algorithms for NISQ devices. Despite the three major limitations of NISQ devices (noise, few qubits, and limited circuit depth, page 467), **VQAs address these limitations** by using shallow circuits, which reduce decoherence and gate errors; hybrid optimization, which allows for repeated measurements and better estimation of the cost function; and flexibility, which allows different ansätze, optimizers, and cost functions to be combined for various applications. Rather than trying to eliminate noise completely, VQAs are designed to **work despite the presence of noise**.

8.4.2 Weaknesses of Variational Algorithms

As discussed in the previous section, VQAs seem to be the perfect solution for NISQ devices. However, **VQAs are not perfect**. We use them **because they are currently the best option for NISQ hardware**, not because they are theoretically superior to all quantum algorithms.

The main weaknesses of VQAs are:

1. They are **approximate methods**.
2. The iterative process is **slow and resource-intensive**.
3. They offer **few formal guarantees**.
4. The **classical optimizer may fail** to find good parameters.

Let's understand each of them.

🕒 They Are Approximate Methods

Unlike algorithms such as Shor's algorithm (we will discuss later), VQAs generally do **not** guarantee the exact solution. Instead, they **search for the best solution found** rather than the **true optimal solution**. For example, in VQE, the algorithm might return an energy value that is very close to the ground state energy but not exact. Similarly, in QAOA, the algorithm may find a solution that is close to the optimal Max-Cut but not necessarily the best possible one. This is why QAOA is called the **Quantum Approximate Optimization Algorithm**. We will discuss QAOA in more detail in the next section.

⌘ The Iterative Process is Slow and Resource-Intensive

This is perhaps the most practical drawback. The optimization loop (page 475) requires many iterations, and each iteration involves preparing the quantum state, executing the circuit, measuring, and often repeating the circuit many times to estimate expectation values.

This makes **VQAs computationally expensive and unlikely to show a quantum advantage on current devices**. The total number of circuit executions can become enormous, making the process slow and resource-intensive. But *why is it resource-intensive?* Because one **evaluation of the cost function is usually not one execution**. For example, in VQE, estimating the expectation value of the Hamiltonian requires many repeated measurements. So one optimization step actually involves multiple circuit executions, and when multiplied by hundreds or thousands of optimization iterations, the total number of circuit executions can become enormous.

❓ Few Formal Guarantees

Classical optimization is difficult. Even if the **ansatz is expressive** and the **cost function is correct**, there is **no guarantee that the optimizer will reach the global optimum**. It may stop too early or converge to a poor solution. Unlike algorithms such as Shor's algorithm, which are mathematically guaranteed to produce the correct answer (assuming ideal hardware), VQAs often lack such guarantees.

⚠️ The Optimizer May Fail

This is closely related to the previous point. If the cost landscape has local minima, the **optimizer may converge to a local minimum instead of the global minimum**. Since the gradient becomes zero at a local minimum, the optimizer may incorrectly conclude that it has found the best solution, even though a better solution exists elsewhere. This is why optimizer choice is important and why optimization itself is an active research area. However, this weakness is not a fundamental limitation of VQAs, but rather a classical optimization problem. In fact, the next section will discuss a specific reason why the classical optimizer may fail: **barren plateaus**.

In general, all these weaknesses are consequences of one fundamental fact: **VQAs are optimization problems**. **Finding the best parameters is often the hardest part of the algorithm**. Despite these drawbacks, VQAs remain the **most practical algorithms for current NISQ devices** because they can be executed on noisy, shallow quantum hardware.

8.4.3 Barren Plateaus

The Barren Plateau is a fundamental challenge in Variational Quantum Algorithms (VQAs), and the idea is very intuitive once we understand what a gradient represents.

🔗 What is a Gradient?

Suppose we have a **cost function** $C(\theta)$ where θ is one **parameter** of the Ansatz. The optimizer wants to know: “*if I change θ slightly, will the cost improve?*”. The answer is given by the **gradient**:

$$\frac{\partial C}{\partial \theta}$$

Because the gradient measures **how sensitive the cost function is to changes in the parameters**, it tells the optimizer which direction to move θ to improve the cost.

💡 Imagine we are hiking on a mountain. The slope tells us *which* direction is downhill and *how* steep the terrain is. The optimizer uses the gradient in exactly the same way.

- If the **gradient** is **large**, the optimizer knows that a small change in θ will lead to a significant improvement in the cost function (i.e., the cost will **decrease quickly**).
- If the **gradient** is **small**, the optimizer cannot determine whether changing θ will improve the cost function, as the change will be negligible.

🔗 What is a Barren Plateau?

Suppose our ansatz has **only one parameter** θ . For every value of θ , we can compute $C(\theta)$. For example:

θ	Cost
0.0	7.76
0.5	5.98
1.0	3.66
1.5	2.70
2.0	2.56
2.5	1.79
3.0	2.15
3.5	3.61
4.0	4.44

If we plot the cost function, the optimizer likes this because it immediately sees “*go to the right, the cost decreases*”. The slope (gradient) is large.

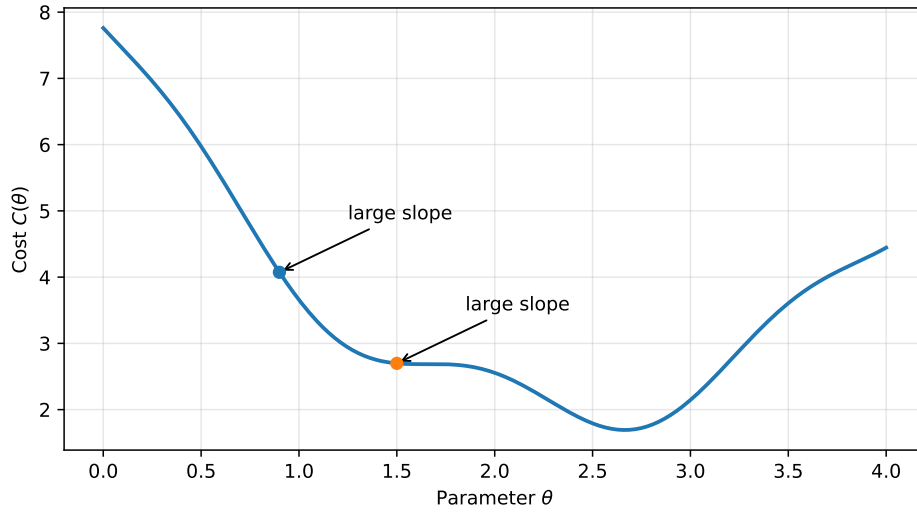


Figure 30: A cost function with a large gradient. The optimizer can easily determine which direction to move θ to improve the cost.

Now suppose the cost values are:

θ	Cost
0.0	5.00
0.5	4.9992431975046925
1.0	5.000989358246623
1.5	4.9994634270819995
$\vdots$	$\vdots$

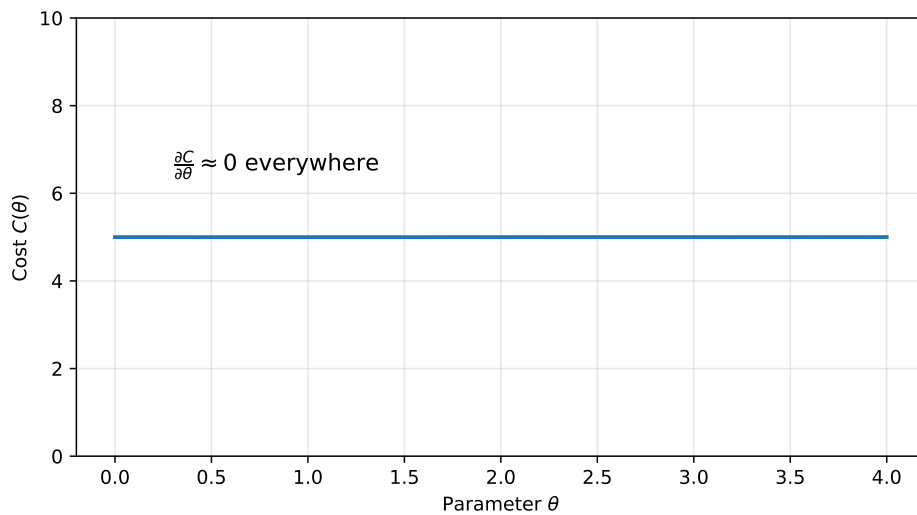


Figure 31: A cost function with a small gradient. The optimizer cannot determine which direction to move θ to improve the cost.

Everything is almost identical. The optimizer asks: “*if I increase θ , do I improve?*”. The answer is: “*I have no idea*”. The optimizer is **stuck**. This is a **barren plateau**!

The gradient is simply $\frac{\partial C}{\partial \theta}$, and it tells us “*how much does the cost change if we slightly change θ ?*”. For large gradients, the optimizer can easily determine which direction to move θ to improve the cost. For small gradients, the optimizer cannot determine which direction to move θ to improve the cost.

It is called **barren plateau** because the cost landscape is almost perfectly flat, like a desert. The optimizer is like a hiker trying to find the lowest point in an endless flat desert. Similarly, “barren” refers to the fact that there is **nothing useful** in the landscape to guide the optimizer.

📖 **A more realistic picture.** Actually, the landscape is **not perfectly flat**. There are small hills and valleys, but they are so small that the optimizer cannot distinguish them from numerical noise:

$$\frac{\partial C}{\partial \theta} \approx 0$$

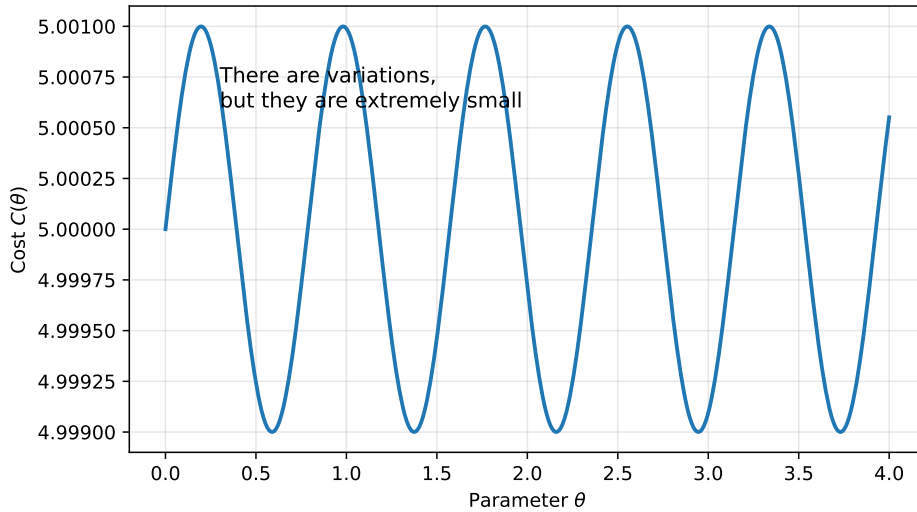


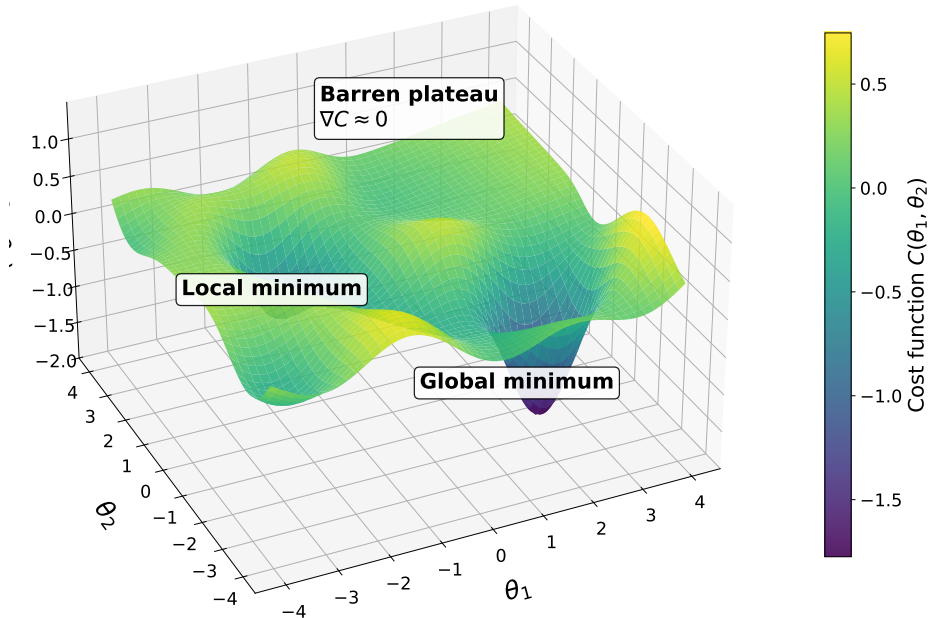
Figure 32: A barren plateau is not perfectly flat. There are small hills and valleys, but they are so small that the optimizer cannot distinguish them from numerical noise.

📖 **Even more realistic: many parameters.** In reality, the ansatz has **many** parameters:

$$\theta = (\theta_1, \theta_2, \dots, \theta_n)$$

So the cost function is not a 2D curve $C(\theta)$, it is a surface in a very high-dimensional space $C(\theta_1, \theta_2, \dots, \theta_n)$. In this case, a barren plateau becomes almost a **flat table**; the optimizer is trying to find the lowest point on that

table, but every direction looks almost identical. In essence, **a barren plateau is not a property of the optimizer, it is a property of the cost function landscape.**



❓ Why do Barren Plateaus occur?

We have seen what a barren plateau is, but we have not yet explained **why** they occur. Let's try to understand the reason behind this phenomenon.

- **What is the optimizer trying to do?** The optimizer is trying to minimize:

$$C(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle$$

Where $C(\theta)$ is the cost function, and $\langle \psi(\theta) | H | \psi(\theta) \rangle$ is the **expectation value** of the Hamiltonian H with respect to the quantum state $|\psi(\theta)\rangle$. In other words, it answers the question: **“if our system is in the state $|\psi(\theta)\rangle$, what energy should we expect to measure?”**. It is simply the average energy of the system in that state.

✏ Example 4: Expectation Value

For example, suppose our state $|\psi(\theta)\rangle$ is 50% ground state $|\psi_0\rangle$ and 50% first excited state $|\psi_1\rangle$. Then the expected energy is:

$$0.5E_0 + 0.5E_1$$

Where E_0 is the energy of the ground state and E_1 is the energy of the first excited state. The expectation value:

$$\langle \psi(\theta) | H | \psi(\theta) \rangle = 0.5E_0 + 0.5E_1$$

It is literally a **weighted average** of the energies.

The Hamiltonian H assigns an energy to every possible quantum state. Then, the Ansatz circuit generates candidate states $|\psi(\theta)\rangle$ by varying the parameters θ . Finally, the optimizer tries to find the parameters θ that minimize the expected energy. The optimizer is trying to find the **ground state** of the Hamiltonian H (i.e., the state with the lowest energy E_0). So the Hamiltonian acts almost like a **judge**, the ansatz proposes every state with an energy, and the optimizer tries to find the state with the lowest energy.

Geometrically, the optimizer is trying to find the **lowest point** in a very high-dimensional landscape. The landscape is defined by the cost function $C(\theta)$, which assigns a height (energy) to every point (parameter configuration) in the landscape. To **reach the lowest point**, the optimizer needs to know which direction to move in the landscape. The **gradient is the only information that tells the optimizer which direction to move**:

$$\nabla C(\theta) = \left(\frac{\partial C}{\partial \theta_1}, \frac{\partial C}{\partial \theta_2}, \dots, \frac{\partial C}{\partial \theta_n} \right)$$

For each parameter θ_i , the gradient tells the optimizer how much the cost function changes if we slightly change θ_i . After computing the gradient, the optimizer updates the parameters θ in the direction of the negative gradient to decrease the cost function. We do not present the details of the optimization algorithm here, but the key point is that **the optimizer relies on the gradient to find the lowest point in the landscape**.

- **Add more parameters and many qubits.** Imagine 20 qubits and 100 parameters. The optimizer is trying to find the **lowest point in a 100-dimensional landscape**. The gradient is a vector with 100 components, and each component tells the optimizer how much the cost function changes if we slightly change one of the parameters. As we increase the number of qubits, the number of parameters increases, and the **landscape becomes more complex**. The optimizer has to explore a larger space, and it becomes more likely that it will encounter regions where the gradient is very small.
- **Averaging effect.** Since the cost function tries to minimize the expectation value of the Hamiltonian:

$$C(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle$$

The optimizer is effectively averaging over many possible states. As the number of qubits increases, the number of possible states grows exponentially. Many positive and negative contributions to the gradient cancel each other out, leading to a very small overall gradient. This is known as the **averaging effect**, and it is a key reason why barren plateaus occur.

Example 5: Averaging Effect

For example, consider a simple case where the Hamiltonian has

two terms:

$$H = H_1 + H_2$$

The cost function is:

$$C(\theta) = \langle \psi(\theta) | H_1 | \psi(\theta) \rangle + \langle \psi(\theta) | H_2 | \psi(\theta) \rangle$$

The gradient is:

$$\nabla C(\theta) = \nabla \langle \psi(\theta) | H_1 | \psi(\theta) \rangle + \nabla \langle \psi(\theta) | H_2 | \psi(\theta) \rangle$$

Each term contributes to the total gradient. For small systems, these contributions usually combine to produce a meaningful gradient. However, as the number of qubits and parameters increases, the ansatz explores an exponentially larger Hilbert space. The individual gradient contributions become increasingly random and tend to cancel statistically. As a result, the total gradient approaches zero almost everywhere in the parameter space, producing a barren plateau.

Because all these cancellations occur, changing θ slightly no longer changes the expectation value very much. The optimizer cannot determine which direction to move θ to improve the cost function, and it becomes stuck in a barren plateau (the landscape is almost flat!).

❓ Why does increasing the number of qubits make it worse?

With n qubits, the Hilbert space has dimension 2^n .

Qubits	State Space
2	$2^2 = 4$
5	$2^5 = 32$
10	$2^{10} = 1'024$
50	$2^{50} \approx 1.13 \times 10^{15}$

The state grows **exponentially**. As the space becomes enormous, random states become almost indistinguishable in terms of their average properties, and the expectation value changes less and less when we perturb the parameters. Thus, the **gradient vanishes exponentially with the system size**.

⚠️ Notice that the barren plateau is a **dangerous phenomenon** because it can prevent the optimizer from finding the global optimum. But, the minimum still exists! The problem is that **almost the entire landscape is so flat that the optimizer cannot find the narrow valley where the optimum lies**. So the issue is not the existence of the solution (i.e., the global optimum), but rather the **difficulty of finding it** due to the flatness of the landscape.¹⁶

¹⁶This is very similar to the vanishing gradient problem in deep neural networks, where the gradient becomes very small in deep layers, making it difficult for the optimizer to update the weights and learn effectively. The Artificial Neural Networks and Deep Learning course explains it in more detail.

✔ Strategies to avoid Barren Plateaus

There are three main strategies to mitigate the barren plateau problem:

1. **Better Parameter Initialization.** Instead of choosing completely random parameters (random θ), **choose parameters closer to promising regions**. The *promising regions* are those where the **gradient is larger**, allowing the optimizer to make progress. This can be achieved by using prior knowledge about the problem or by employing advanced initialization techniques that take into account the structure of the ansatz and the Hamiltonian. This increases the chance of **starting outside a barren plateau**.
2. **Better Ansatz Design.** Some ansätze naturally produce flatter landscapes because they are too expressive or too deep. Designing a suitable ansatz can preserve meaningful gradients.
3. **Better Optimizer Choice.** Some optimizers can handle small gradients more effectively than others. Although no optimizer completely solves the problem, choosing an appropriate one can improve convergence.

Unfortunately, there is **no universal solution** to the barren plateau problem. We need to carefully consider the ansatz, initialization, and optimizer for each specific problem. The barren plateau is a fundamental challenge in VQAs, and ongoing research continues to explore new strategies to mitigate its effects.

📖 Definition 2: Barren Plateau

A **Barren Plateau** is a **region of the optimization landscape where the gradient of the cost function is extremely close to zero**.

As the number of qubits increases, the gradient decreases exponentially, making it very difficult for the classical optimizer to determine how to update the circuit parameters. Consequently, the optimizer may fail to find the global optimum.

Common mitigation strategies include careful parameter initialization, suitable ansatz design, and choosing an efficient optimizer.

8.4.4 Why Variational Algorithms Matter

After studying all the strengths and weaknesses, a natural question arises: *if VQAs are approximate, slow, and suffer from barren plateaus... why do we study them at all?* The answer is because they are much more than just an algorithm. They are also a **research paradigm** for learning how to exploit today's quantum computers.

Variational Quantum Algorithms (VQAs) are important because they:

1. **Studying the Computational Power of Quantum Devices.** Today's quantum computers are still experimental. We still do not fully know:
 - Which problems they solve efficiently;
 - Which circuit structures work best;
 - How hardware behaves under realistic workloads.

VQAs provide an ideal testing framework. So, instead of asking “*can this quantum computer factor 2048-bit integers?*”, we ask “*how well can it optimize a small Hamiltonian?*” or “*how much entanglement can it reliably generate?*”. VQAs therefore serve as **benchmarks** for quantum hardware.

2. **Developing Methods for Noisy Quantum Computers.** The problem with NISQ devices is that they have a limited number of qubits and significant noise sources such as decoherence and gate errors. If we waited for perfect quantum computers, research would almost stop. Instead, **VQAs encourage researchers to develop techniques specifically for imperfect devices.** Many techniques that are now standard in quantum computing (e.g., error mitigation, circuit compilation, and noise-aware optimization) were first explored within the VQA framework.

So VQAs are not only useful for solving problems, they are also a laboratory for developing new quantum computing techniques.

3. **They May Still Be Useful in the Future.** VQAs may still be useful in the future for problems for which we do not yet have a dedicated quantum algorithm. Probably the optimizer will need to be part of the quantum circuit to ensure scalability.

Today, the optimizer is classical: quantum circuits are run on a quantum computer, and the results are fed into a classical optimizer to update the parameters. This works for today's relatively small problems.

But imagine a future quantum computer with millions of logical qubits. The optimizer may need to adjust millions of parameters. Sending information back and forth between quantum and classical computers could become the bottleneck. The communication overhead and the classical optimization itself may no longer scale well.

One possible future solution is to implement the **quantum circuit** and the **optimizer** on the **same quantum computer**. The idea is that part (or

all) of the optimization could itself be performed quantum mechanically. However, even if this is possible, many optimization problems may still not have a dedicated quantum algorithm and designing a specialized algorithm for every problem is unrealistic. So, **VQAs may still be the best option for many problems in the future.**

In summary, Variational Quantum Algorithms (VQAs) are important because they allow researchers to study the computational capabilities of current quantum devices and develop techniques to cope with limited resources and noise. Although they were originally motivated by the NISQ era, they may remain useful in the future for optimization problems that lack dedicated quantum algorithms. As quantum hardware scales, the optimization process itself may eventually need to become quantum to ensure scalability.

8.5 Variational Quantum Eigensolver (VQE)

8.5.1 The Physical Motivation

⚠ The Fundamental Problem. Before introducing what the Variational Quantum Eigensolver (VQE) is, it is important to understand the physical motivation behind it. The VQE is used to find the **minimum-energy state of a molecule by solving the Schrödinger equation**. Before understanding the mathematics, we first need to understand **why this problem is important**. Suppose we have a molecule. For example H_2 or H_2O or CH_4 . The first a chemist asks is: *is this molecule stable?* And if it is stable, *what is its energy?*

❓ Why does energy matter? Nature has a very simple rule: systems naturally evolve toward lower energy states. For example, if you have a ball on a hill, it will roll down to the bottom of the hill. The same principle applies to molecules: electrons and nuclei arrange themselves until the molecule reaches its **lowest-energy configuration**. That configuration is called the **ground state**.

⚠ The Problem: Classical Difficulty. Almost every important property of a molecule depends on its ground state. For example, the color of a molecule, its reactivity, and its stability all depend on the ground state. Therefore, if we want to understand a molecule, we need to find its ground state.

However, finding the ground state is **very difficult** for classical computers. The reason is that the number of possible configurations of electrons grows exponentially with the number of electrons in the molecule. For example, for a molecule with 10 electrons, there are $2^{10} = 1024$ possible configurations. For a molecule with 20 electrons, there are $2^{20} = 1,048,576$ possible configurations. For a molecule with 100 electrons, there are $2^{100} \approx 1.27 \times 10^{30}$ possible configurations! This exponential growth makes it impossible for classical computers to find the ground state of large molecules.

✅ The Solution: Quantum Computers. A molecule is made of electrons and nuclei, which are **quantum particles**. Therefore, we cannot describe them using classical mechanics. We must solve quantum mechanics. Also, due to the exponential growth of the number of configurations, we need a quantum computer to solve this problem efficiently. Thus, this is exactly the motivation originally proposed by Richard Feynman: “*a quantum system should be simulated by another quantum system*”. Instead of storing exponentially many amplitudes on a classical computer, a **quantum computer naturally evolves according to quantum mechanics**.

❓ Where does VQE fit? Unfortunately, today’s quantum computers are NISQ devices. So we cannot simply run a huge quantum simulation. Instead, the quantum computer is used to prepare candidate quantum states and classical computer is used to optimize the parameters of the quantum state. **Together they search for the lowest-energy state**. That **hybrid algorithm is the Variational Quantum Eigensolver (VQE)**.

8.5.2 Hamiltonians and Molecular Energy

Inside each molecule, there are nuclei and electrons that interact with each other through electromagnetic forces. The question is: *how much energy does this molecule have?* But more importantly: *which arrangement of the electrons corresponds to the minimum energy?*

❓ Why can't we compute it directly? In principle, we could compute the energy of a molecule directly from the laws of quantum mechanics. However, for realistic molecules this becomes computationally intractable because every particle contributes to the total energy and all particles interact with one another.

More precisely:

- Each electron has **kinetic energy**;
- Each electron interacts with **every nucleus**;
- Each electron interacts with **every other electron**;
- Nuclei also interact with **each other**.

So the total energy is **not** simply the **sum of independent contributions**. It is the result of a **huge number of coupled quantum interactions**.

✅ Why introduce the Hamiltonian? The Hamiltonian is **not** introduced because writing many energy terms is inconvenient. It is introduced because **quantum mechanics is formulated in terms of operators acting on quantum states**. In general, the Hamiltonian is the operator that **contains all the information about the energy of the system** (page 432, where a system could be a molecule, an atom, a spin chain, a quantum computer, etc). Here, we use the Hamiltonian to describes the **physical energy of a molecule**; it contains **all** contributions to the molecular energy.

❓ What does the Hamiltonian contain? In the context of VQE, the Hamiltonian contains **all the energy contributions of the molecule**. More precisely, it represents: the **kinetic energy** of all the particles, and the **potential energy** of all the particles. Together, these describe the molecule's total energy. The Hamiltonian is therefore a compact mathematical representation of the molecule's entire energy landscape.

✂ The Time-Independent Schrödinger Equation

Now that we have introduced the operator (Hamiltonian) that contains all the information about the molecule's energy, the next question is: *how do we obtain the energies encoded inside the Hamiltonian?* The answer is given by the **Time-Independent Schrödinger Equation**:

$$H|\psi\rangle = E|\psi\rangle \quad (173)$$

The Schrödinger equation is the mathematical tool that **extracts the energies from the Hamiltonian**. The equation is simply an **eigenvalue equation**, so

it tells us: *find the special states $|\psi\rangle$ for which applying the Hamiltonian simply multiplies the state by a number E .* The number E is the **energy** of the state $|\psi\rangle$.

- H is the **Hamiltonian operator**, which describes the total energy of the molecule;
- $|\psi\rangle$ is the **quantum state of the molecule**, a possible configuration of the electrons. In simple terms, “*how are the electrons arranged?*”;
- E is the **energy associated** with that particular quantum state $|\psi\rangle$.

Solving the Schrödinger equation is equivalent to select the **special states** that satisfy the Hamiltonian. These states are the **eigenstates** of the Hamiltonian. Each one has a **well-defined energy** E , which is the corresponding **eigenvalue**. It is similar to the algebraic case:

$$A\vec{v} = \lambda\vec{v}$$

Where A is a matrix, $\vec{v}$ is an eigenvector, and λ is the corresponding eigenvalue. In the quantum case, the Hamiltonian is an operator, the quantum state is an eigenvector, and the energy is the corresponding eigenvalue.

💡 The power of the time-independent formulation. With the time-independent Schrödinger equation we are **not** studying how the state evolves over time. Instead, we are looking for the special states that have a **well-defined, constant energy**. The solution gives us a state where the energy is fixed and **does not change into another energy level**. This is why these states are also known as **stationary states**.

In other words, the time-independent Schrödinger equation finds the **stationary (stable) quantum states** of the Hamiltonian. These states have a **well-defined energy** that remains *constant over time*. This is why they are considered stable and are the states of interest in VQE.

❓ Which stationary state are we looking for? Solving the time-independent Schrödinger equation does not produce a single solution. Instead, it produces a set of **eigenstates**:

$$|\psi_0\rangle, |\psi_1\rangle, |\psi_2\rangle, \dots$$

Each associated with an energy eigenvalue

$$E_0, E_1, E_2, \dots$$

Where, by convention:

$$E_0 \leq E_1 \leq E_2 \leq \dots$$

The state with the **lowest energy** $|\psi_0\rangle$ is called the **ground state**. It represents the most stable configuration of the molecule because physical systems naturally tend to occupy the minimum-energy state. All the remaining eigenstates:

$$|\psi_1\rangle, |\psi_2\rangle, \dots$$

Are called **excited states** (page 437), since they possess higher energies and can only be occupied if additional energy is supplied to the system.

❓ **Why does VQE only care about these states?** Because VQE wants to find the **ground state**, which is **one of the stationary states**, specifically the one with the **lowest energy**.

 Deepening: How can we represent the molecular Hamiltonian on a quantum computer?

A quantum computer does **not** understand arbitrary matrices. It understands quantum gates built from Pauli operators. Therefore, before running VQE, the **molecular Hamiltonian is rewritten as a sum of tensor products of Pauli matrices**.

❓ **Why tensor products?** Because a molecule involves **multiple qubits**. The tensor product (page 88) is the mathematical operation that combines multiple qubits into a single quantum state. Each qubit can be represented by a Pauli operator, and the tensor product allows us to describe the interactions between these qubits in the Hamiltonian.

❓ **Why a sum?** Each tensor product represents **one contribution** to the Hamiltonian. The total Hamiltonian is obtained by adding them together.

In general, although the Hamiltonian represents the physical energy of the molecule, before it can be processed by a quantum computer it is expressed as a **sum of tensor products of Pauli matrices**, since Pauli operators form the natural building blocks of quantum circuits.

In summary, in VQE, the energy of a molecule is represented by the Hamiltonian H , a quantum operator that includes the kinetic and potential energies of all the particles in the system. The physical states of the molecule are obtained by solving the time-independent Schrödinger equation $H|\psi\rangle = E|\psi\rangle$, where $|\psi\rangle$ is a quantum state and E is its associated energy. Since this equation has the form of an eigenvalue problem, the solutions correspond to the eigenstates of the Hamiltonian and their associated energy levels.

Therefore, the goal of VQE is not to compute all the eigenvalues of the Hamiltonian. Instead, it focuses on finding the **minimum eigenvalue** and its corresponding eigenstate:

$$E_{\min} = E_0, \quad |\psi_{\min}\rangle = |\psi_0\rangle$$

Once the ground state has been identified, many important physical and chemical properties of the molecule can be determined.

8.5.3 Variational Principle

This section presents a **theorem that makes VQE possible**. Recall the logical chain established in previous sections: we need to find the ground state of a Hamiltonian, which is equivalent to determining its minimum eigenvalue. Now, the problem is: “*how can we find the minimum eigenvalue of a Hamiltonian?*”

⚠ Diagonalization is not feasible for large Hamiltonians! An n -qubit quantum system lives in a Hilbert space of dimension 2^n , therefore the Hamiltonian matrix has size $2^n \times 2^n$. For example, a system of $n = 100$ qubits would require a Hamiltonian matrix of size $2^{100} \times 2^{100}$, which is approximately $10^{30} \times 10^{30}$. One naïve approach would be to find the minimum eigenvalue of the Hamiltonian by **diagonalizing** it. Diagonalizing a matrix involves **computing all its eigenvalues and eigenvectors**. However, the **computational cost grows polynomially with the matrix size**, while the **matrix size itself grows exponentially with the number of qubits**. Therefore, the **overall computation becomes exponential**.

✓ Variational Principle saves us

Wait a minute, do we really need $\lambda_0, \lambda_1, \lambda_2, \dots, \lambda_{2^n-1}$? No, we only need the **minimum eigenvalue** λ_0 . So, instead of solving the complete eigenvalue problem (finding all eigenvalues), we only want to find the **minimum eigenvalue**.

The **Rayleigh-Ritz Variational Principle**¹⁷ saves us from diagonalization. It states that:

$$\langle \psi | H | \psi \rangle \geq \lambda_0 \quad \text{for any normalized state } |\psi\rangle$$

We remark that the expectation value $\langle \psi | H | \psi \rangle$ (page 142) represents the **average value obtained if we repeatedly measure the observable H on many identical copies of the state $|\psi\rangle$** . In other words, it is the expected energy of the quantum state $|\psi\rangle$.

Theorem 4 (Rayleigh-Ritz Variational Principle). *Let H be a Hermitian operator (Hamiltonian) with eigenvalues $\lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_{2^n-1}$ and corresponding eigenvectors $|\psi_0\rangle, |\psi_1\rangle, \dots, |\psi_{2^n-1}\rangle$. For **any normalized state** $|\psi\rangle$, the expectation value of H satisfies:*

$$\langle \psi | H | \psi \rangle \geq \lambda_0 \tag{174}$$

Where λ_0 is the minimum eigenvalue of H . Equality holds if and only if $|\psi\rangle$ is an eigenvector corresponding to λ_0 .

Thus, the ground-state energy λ_0 is the **lowest possible energy**, nothing can go below it. Obviously, this means that the ground state satisfies:

$$\langle \psi_{\min} | H | \psi_{\min} \rangle = \underbrace{\lambda_{\min}}_{\lambda_0}$$

¹⁷The word “variational” comes from the fact that we **vary** the trial state $|\psi\rangle$ to minimize the expectation value $\langle \psi | H | \psi \rangle$. We keep varying the parameters until the energy is as low as possible.

❓ **Why is the Variational Principle incredibly useful?** Because it completely changes the problem. Originally we wanted to diagonalize the Hamiltonian to find its minimum eigenvalue, which is **computationally expensive**. Now we only need to minimize $\langle \psi | H | \psi \rangle$ over all possible normalized states $|\psi\rangle$, which is **computationally feasible**. We shifted from a problem that is **exponentially hard** to a problem that is **polynomially hard**. Now, we need to solve an **optimization problem** that is still hard, but at least it is feasible.

🔗 Connection with the Ansatz

The Ansatz is nothing more than a state generator. Instead of trying every possible quantum state (which is impossible), we define a family of states $|\psi(\theta)\rangle$ parameterized by θ . The ansatz is literally a circuit that prepares these states.

In other words, since the theorem needs a normalized state $|\psi\rangle$, *where does this state come from?* It comes from the Ansatz. The Ansatz is a circuit that prepares a quantum state $|\psi(\theta)\rangle$ parameterized by θ , then we measure the expectation value:

$$\langle \psi(\theta) | H | \psi(\theta) \rangle$$

Simply put, the ansatz is a parameterized quantum circuit that generates trial quantum states. Thanks to the Variational Principle, VQE only needs to search among these trial states for the one with the smallest expected energy, instead of diagonalizing the exponentially large Hamiltonian.

In summary, the Variational Quantum Eigensolver (VQE) is based on the Rayleigh-Ritz Variational Principle, which states that for any normalized quantum state $|\psi\rangle$, the expectation value of the Hamiltonian satisfies $\langle \psi | H | \psi \rangle \geq \lambda_{\min}$, where $\lambda_{\min}$ is the minimum eigenvalue of the Hamiltonian. Equality is achieved only when $|\psi\rangle$ is the ground state. Therefore, if the ansatz can represent the ground state, minimizing the expectation value is equivalent to finding the ground-state energy of the molecule.

8.5.4 VQE Objective Function

In the previous section, the Rayleigh-Ritz Variational Principle told us:

$$\langle \psi | H | \psi \rangle \geq \lambda_{\min}$$

But now, *what exactly should the optimizer minimize?* The answer is the **VQE objective function**.

The **VQE Objective Function** is defined as:

$$\arg \min_{\theta} C(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle \quad (175)$$

Where:

- $C(\theta)$ is the cost function. It measures how well the current parameters θ are performing.
- θ are the parameters of the quantum gates. They are simply the angles of the parameterized gates. The optimizer never changes the circuit structure. It only changes these angles.
- H is the Hamiltonian of the molecule.
- $|\psi(\theta)\rangle$ is the state prepared by the Ansatz circuit. Obviously, if we change the parameters θ , we obtain a **different quantum state**. So the ansatz is actually a **family of quantum states** $|\psi(\theta)\rangle$.

Once the circuit prepares the state $|\psi(\theta)\rangle$, we evaluate its energy by measuring the expectation value of the Hamiltonian H :

$$\langle \psi(\theta) | H | \psi(\theta) \rangle$$

After that, we feed the result to a classical optimizer, which will minimize the energy by changing the parameters θ of the quantum circuit. The optimizer will repeat this process until it finds the optimal parameters θ^* that minimize the energy:

$$\theta^* = \arg \min_{\theta} C(\theta) \rightarrow C(\theta^*) = \langle \psi(\theta^*) | H | \psi(\theta^*) \rangle \approx \lambda_{\min}$$

- $\arg \min_{\theta}$ is the operator that returns the parameters θ^* that minimize the cost function $C(\theta)$.

❓ How does the quantum computer compute this expectation value?

The molecular Hamiltonian is actually a **sum of many Pauli strings**. For example:

$$H = 0.5I - 1.2Z_0 + 0.8X_0X_1 - 0.3Y_0Y_1 + 0.6Z_0Z_1$$

Notice that every term is a coefficient multiplied by a tensor product of Pauli operators. This **decomposition comes from mapping the molecular Hamiltonian onto qubits** (using transformations such as Jordan-Wigner or Bravyi-Kitaev, techniques not covered in this course). Now comes the beautiful mathematical trick. Expectation values are **linear**, which means that we can compute

the expectation value of the Hamiltonian by computing the expectation value of each Pauli string separately and then summing them up:

$$\begin{aligned}\langle\psi(\theta)|H|\psi(\theta)\rangle &= 0.5\langle\psi(\theta)|I|\psi(\theta)\rangle - 1.2\langle\psi(\theta)|Z_0|\psi(\theta)\rangle \\ &\quad + 0.8\langle\psi(\theta)|X_0X_1|\psi(\theta)\rangle - 0.3\langle\psi(\theta)|Y_0Y_1|\psi(\theta)\rangle \\ &\quad + 0.6\langle\psi(\theta)|Z_0Z_1|\psi(\theta)\rangle\end{aligned}$$

Or more compactly:

$$\langle H \rangle = 0.5 \langle I \rangle - 1.2 \langle Z_0 \rangle + 0.8 \langle X_0X_1 \rangle - 0.3 \langle Y_0Y_1 \rangle + 0.6 \langle Z_0Z_1 \rangle$$

Measuring the expectation value of a Pauli string is easy. We prepare the state $|\psi(\theta)\rangle$ and then measure **both qubits in the computational Z basis**. For example, suppose after many repetitions on the Pauli string $\langle Z_0Z_1 \rangle$ we get the following measurement results:

Outcome 00	Frequency 0.4
Outcome 01	Frequency 0.1
Outcome 10	Frequency 0.2
Outcome 11	Frequency 0.3

Now assign the eigenvalues of Z to the measurement outcomes:

Outcome 00	Eigenvalue $Z 0\rangle = +1 0\rangle \rightarrow +1$
Outcome 01	Eigenvalue $Z 1\rangle = -1 1\rangle \rightarrow -1$
Outcome 10	Eigenvalue $Z 0\rangle = +1 0\rangle \rightarrow +1$
Outcome 11	Eigenvalue $Z 1\rangle = -1 1\rangle \rightarrow -1$

Now we can compute the expectation value of Z_0Z_1 as follows:

$$\begin{aligned}\langle Z_0Z_1 \rangle &= 0.4 \cdot (+1) + 0.1 \cdot (-1) + 0.2 \cdot (-1) + 0.3 \cdot (+1) \\ &= 0.4 - 0.1 - 0.2 + 0.3 = 0.4\end{aligned}$$

❓ How do we measure the expectation value of Pauli strings that contain X or Y operators? The quantum computer naturally measures in the Z basis only.¹⁸ So we simply rotate the basis **before** measuring. For X operators, we apply a Hadamard gate H to rotate the basis from X to Z (page 155):

$$HXH = Z$$

For Y operators, we apply a $S^\dagger$ gate followed by a Hadamard gate to rotate the basis from Y to Z :

$$HS^\dagger YSH = Z$$

Finally, the classical computer combines all the measured values:

$$\langle H \rangle = 0.5 \langle I \rangle - 1.2 \langle Z_0 \rangle + 0.8 \langle X_0X_1 \rangle - 0.3 \langle Y_0Y_1 \rangle + 0.6 \langle Z_0Z_1 \rangle$$

¹⁸The measurement apparatus of a quantum computer is designed to directly distinguish the two **physical basis states** of the qubit, denoted by $|0\rangle$ and $|1\rangle$. These states form the **computational basis**, which is also the eigenbasis of the Pauli Z operator. Therefore, the hardware naturally performs measurements in the Z basis. In contrast, the eigenstates of the Pauli X and Y operators are superpositions of the computational basis states and cannot be directly distinguished by the measurement device.

This final number is the **cost function** $C(\theta)$ that the classical optimizer will minimize by changing the parameters θ of the quantum circuit.

In summary, in VQE, the objective function is the expectation value of the Hamiltonian with respect to the state prepared by the parameterized ansatz:

$$C(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle = \sum_i c_i \langle \psi(\theta) | P_i | \psi(\theta) \rangle = \sum_i c_i \langle P_i \rangle_\theta$$

Where c_i are the Hamiltonian coefficients, and H is decomposed as:

$$H = \sum_i c_i P_i$$

Each P_i is a Pauli string, for example $P_i = X_0 X_1$, $P_i = Y_0 Y_1$, or $P_i = Z_0 Z_1$. The classical optimizer searches for the parameters θ that minimize this expectation value. By the Rayleigh-Ritz Variational Principle, the minimum possible expectation value equals the ground-state energy if the ansatz can represent the ground state. Since the Hamiltonian is expressed as a sum of Pauli operators, its expectation value is obtained by measuring the corresponding Pauli observables and combining the results.

8.6 Quantum Approximate Optimization Algorithm

8.6.1 Why QAOA?

In the Variational Quantum Algorithms (VQAs) framework, the VQE and QAOA algorithms are **hybrid quantum-classical algorithms**. It means that the quantum computer executes a parameterized quantum circuit (ansatz), the classical computer updates its parameters, and the process repeats until convergence. **QAOA is a Variational Algorithm for combinatorial optimization problems on NISQ devices and is a special case of VQE.**

The problem

Imagine we work in industry. Every week we receive a different optimization problem:

- Routing trucks (e.g., UPS, FedEx)
- Scheduling workers (e.g., Amazon, Walmart)
- Assigning frequencies (e.g., AT&T, Verizon)
- Portfolio optimization (e.g., BlackRock, Vanguard)
- Max-Cut (e.g., Facebook, Twitter)
- Maximum Independent Set (e.g., Google, Microsoft)
- SAT (e.g., IBM, Intel)
- Graph partitioning (e.g., Netflix, Spotify)

We could design a different quantum algorithm for each one of these problems, but it would be very inefficient. Instead, we want to design a **general-purpose quantum algorithm** that can solve all these problems. In other words, **one optimization framework that works for many problems**. This is exactly the goal of QAOA.

The idea

Instead, industry wants something different. They want an algorithm that is:

- | | |
|------------|-----------------------------|
| • Generic | • Easy to adapt |
| • Reusable | • Requires minimal redesign |

QAOA takes an optimization problem, represents it as a Hamiltonian, and then run the same algorithm to find the solution. This is the fundamental idea of QAOA: the algorithm **does not change**; only the **Hamiltonian changes**.

 This method has a great advantage: **every combinatorial optimization problem can be represented in the same mathematical form.**

✖ However, nothing comes for free. To make every optimization problem look identical, we must **remove all problem-specific information**. So, we **lose the knowledge of the specific problem structure, which classical algorithms often exploit**. For example, classical Max-Cut algorithms know graphs, edges and connectivity; after converting everything into a Hamiltonian, QAOA only “sees” a Hamiltonian, it no longer knows what the vertices represent, why two variables interact, what the original optimization problem looked like. Everything becomes an **energy minimization problem**.

In summary, the **Quantum Approximate Optimization Algorithm (QAOA)** is a **hybrid Variational Quantum Algorithm (VQA)** designed to **solve combinatorial optimization problems on NISQ quantum computers**. It is a special case of VQE. Rather than designing a different quantum algorithm for every optimization problem, QAOA converts the problem into a Cost Hamiltonian H_C and applies the same optimization framework. Its main advantage is that many optimization problems can be expressed in the same Hamiltonian form, making the algorithm general and reusable. The drawback is that converting every problem into a generic Hamiltonian loses the original problem structure, which efficient classical algorithms often exploit.

8.6.2 QAOA Ingredients

In this section, we will see the high-level architecture of the Quantum Approximate Optimization Algorithm (QAOA). We will discuss **what components QAOA needs**, before explaining how each one works.

In general, we can think of QAOA as a recipe. To make a dish, we need ingredients. To run QAOA, we need ingredients as well. The ingredients of QAOA are:

- Optimization problem (the problem we want to solve)
- Cost Hamiltonian H_C
- Mixer Hamiltonian H_M
- Quantum circuit
- Classical optimizer
- Best parameters (γ, β)
- Approximate optimal solution (the solution we get from QAOA)

Let's go through each of these ingredients in detail.

1. **Build the Cost Hamiltonian.** We convert our optimization problem into H_C . The ground state of the Cost Hamiltonian should be the solution of the problem we want to solve. In other words, it answers the question: “*what are we trying to optimize?*”.

In optimization we usually define a **cost function** $C(x)$ that we want to minimize. QAOA replaces that classical cost function with H_C . So, instead of minimizing $C(x)$, we **minimize the expected energy** of H_C :

$$\langle H_C \rangle = \langle \psi | H_C | \psi \rangle$$

For example, for every possible solution:

Solution	Energy
A	12
B	7
C	1
D	9

The **lowest energy corresponds to the optimal solution**. So the Cost Hamiltonian simply answers: “*how good is this solution?*”.

2. **Build the Mixer Hamiltonian.** Now suppose the quantum computer is currently representing solution B. *How can it move toward solution C?* It **needs a mechanism that changes the quantum state**. That mechanism is the **Mixer Hamiltonian**. It doesn't know what the best solution is, it just moves the quantum state around.

In other words, the **Mixer Hamiltonian** explores the space of possible quantum states. It tries to solve the local minima problem by moving the quantum state around.

❓ Why both the Cost Hamiltonian and the Mixer Hamiltonian?

Because they do different jobs. The Cost Hamiltonian **evaluates** how good a solution is, while the Mixer Hamiltonian **explores** the space of possible solutions. The two work together to find the best solution.

- Cost Hamiltonian: *How good is this solution?*
- Mixer Hamiltonian: *What other solutions should we try?*

3. **Construct the Quantum Circuit.** Once we have both Hamiltonians, they must be **transformed into an actual quantum circuit**. This raises an obvious question; if Hamiltonians are matrices and quantum computers execute gates, *how do we convert one into the other?* The answer will be discussed in the next sections.

In general, we will see that (page 516):

$$U = e^{-i\alpha H}$$

Allows us to convert a Hamiltonian into a unitary operator, which can then be implemented as quantum gates. However, for now, just know that we can convert Hamiltonians into unitary operators, and then into quantum circuits.

4. **Use a Classical Optimizer.** Just like VQE, QAOA is a **hybrid algorithm**. The quantum computer alone is **not enough**. So we need to choose a classical optimizer and begin to iterate. **Why?** Because the circuit contains unknown parameters (γ, β) that represent the angles of rotation for the Cost and Mixer Hamiltonians (we will see this in the next sections). The optimizer decides:
 - Which values to try;
 - Whether the new result is better;
 - How to update the parameters.
5. **Optimization Objective.** Finally, *what are we minimizing?* Exactly the same objective as VQE: the expected energy of the Cost Hamiltonian. The classical optimizer will try to find the best parameters (γ, β) that minimize the expected energy of H_C :

$$\langle H_C \rangle = \langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle \rightarrow \arg \min_{\theta} \langle \psi(\theta) | H_C | \psi(\theta) \rangle$$

Where the parameters θ are the angles (γ, β) of the Cost and Mixer Hamiltonians.

Therefore, QAOA is searching for **the values of γ and β that produce the quantum state with the lowest expected energy**. Since the Cost Hamiltonian was built so that the lowest energy corresponds to the optimal solution, this means that QAOA is searching for the best solution to the optimization problem.

So to summarize, the entire algorithm can be summarized as follows:

1. Optimization problem
2. Build Cost Hamiltonian H_C
3. Add Mixer Hamiltonian H_M
4. Build parameterized circuit
5. Choose (γ, β)
6. Run circuit
7. Measure
8. Compute $\langle \psi | H_C | \psi \rangle$
9. Classical optimizer
10. Update (γ, β)
11. Repeat

Notice that this is essentially the **same hybrid optimization loop as VQE**. The only major difference is the **structure of the ansatz**: in VQE the ansatz is largely user-defined, whereas in QAOA the ansatz is **completely determined by the Cost Hamiltonian and the Mixer Hamiltonian**. That distinction will become the central topic of the next section.

8.6.3 Structure of the QAOA Ansatz

In general, the Ansatz is a **parameterized quantum circuit whose parameters are optimized by a classical optimizer**. In Variational Quantum Eigensolver (VQE), the user could choose the circuit architecture. For example, the user could choose a hardware-efficient Ansatz or a problem-inspired Ansatz (i.e. a circuit that is inspired by the problem Hamiltonian).

In QAOA, the ansatz is **not arbitrary**. The **ansatz is determined by the Cost Hamiltonian, the Mixer Hamiltonian and their parameters γ and β** . So unlike VQE, we do **not** invent our own circuit. The structure is fixed. Only the parameter values change. See Figure 33 (page 515) for a visual representation of the QAOA ansatz.

❓ Why do we start with Hadamard gates? The circuit always begins with $H^{\otimes n}$. In other words, **every qubit receives one Hadamard gate**. Because we want to start from $|0\rangle^{\otimes n}$, and transform it into $|+\rangle^{\otimes n}$. This creates the uniform superposition:

$$|+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$$

For example, suppose we have 3 binary variables. The possible solutions are:

$$000, 001, 010, 011, 100, 101, 110, 111$$

Initially, the quantum state $|000\rangle$ represents only one candidate. After the Hadamards, the quantum computer simultaneously represents all 8 candidates. This is the starting point of the optimization.

🔄 The repeated block. After initialization, QAOA **repeatedly applies** the Cost Hamiltonian (H_C) and the Mixer Hamiltonian (H_M) until it has executed p **blocks**. This alternating structure is the defining characteristic of QAOA.

Each block has two parameters: γ and β , $\gamma, \beta \in \mathbb{R}^p$. These are the **parameters (vectors) optimized by the classical optimizer**. Each layer has **its own pair of parameters**.

❓ What is p ? The parameter p is called the **depth** of the QAOA circuit. It indicates **how many Cost-Mixer pairs are applied**.

- **Larger p** means a circuit more expressive and potentially more accurate, but also deeper, noisier, and slower to run. It also means more parameters to optimize, which can be challenging for the classical optimizer.
- **Smaller p** means a circuit that is less expressive and potentially less accurate, but also shallower, less noisy, and faster to run. It also means fewer parameters to optimize, which can be easier for the classical optimizer.

❓ Why repeat the block? One of the main questions about the QAOA architecture is *why the Cost-Mixer block needs to be repeated*. Imagine the Cost Hamiltonian alone. It tries to move the state toward lower energy. Then the

Mixer slightly perturbs the state, allowing exploration. Then the Cost Hamiltonian improves it again. Then another exploration, and another optimization, and so on. So the repeated block allows the algorithm to **explore and optimize** the solution space. The more blocks, the more exploration and optimization can occur. This alternation is exactly what gives the algorithm its name: **Quantum Approximate Optimization Algorithm (QAOA)**.

❓ And the optimizer? Where does it fit in? Notice the feedback loop in Figure 33 (page 515). After measuring, the optimizer computes the expectation value of the Cost Hamiltonian:

$$\langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle$$

And proposes new values for γ and β . The circuit architecture never changes. **Only the parameters change.** This is another important difference from general VQE.

It is important that only the parameters change because it means the ansatz is determined by the Hamiltonians and not the user.. The user does not invent a circuit. The user only chooses the Hamiltonians and the depth p . The rest is determined by the problem. This means that if tomorrow we solve Max-Cut, and the day after we solve Max-3-SAT, the overall shape of the circuit is always the same (the same Cost-Mixer block repeated p times). The **only things that change are**:

- The **Cost Hamiltonian** (because the optimization problem changes);
- The optimized values of γ_i and β_i .

The Mixer Hamiltonian keeps the same mathematical form in standard QAOA.

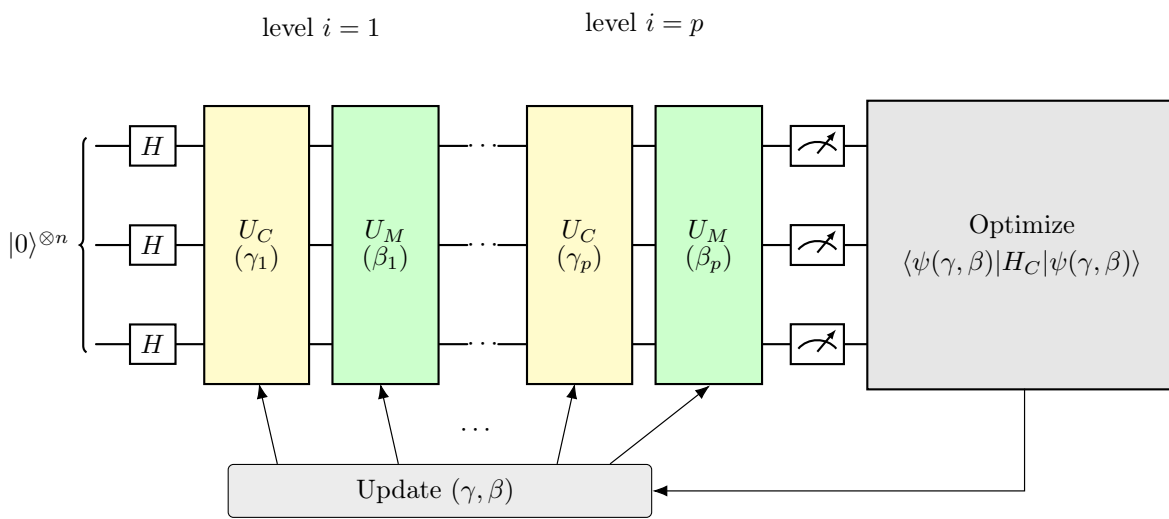


Figure 33: Structure of the QAOA ansatz.

8.6.4 Unitaries from Hamiltonians

Until now, we’ve talked about Hamiltonians: Cost Hamiltonian H_C and Mixer Hamiltonian H_M . But a **quantum computer cannot execute Hamiltonians**. It executes **quantum gates** (unitary operators). So the obvious question is: *how do we transform a Hamiltonian into something that a quantum computer can execute?* The answer is the bridge between **physics** (Hamiltonians) and **quantum circuits** (gates).

⚠ The problem. Suppose we have built the Cost Hamiltonian H_C for a given problem. Great! *But now what?* Can we execute H_C on IBM Quantum computers? No, we cannot. Quantum hardware understands only **unitary operators** (quantum gates). So we need a translation from Hamiltonians to unitary operators. The question is: *how do we do that?*

✓ The fundamental theorem

Theorem 5. A unitary operator U can be represented as the exponential of a Hamiltonian H :

$$U = e^{-i\alpha H} = \exp(-i\alpha H) \quad (176)$$

This equation comes from **quantum mechanics**. Remember that the time evolution of a quantum system is given by the Schrödinger equation:

$$i\hbar \frac{\partial}{\partial t} |\psi(t)\rangle = H |\psi(t)\rangle$$

The solution of this equation is (assuming H is time-independent):

$$|\psi(t)\rangle = e^{-iHt/\hbar} |\psi(0)\rangle$$

Where the operator $U(t) = e^{-iHt/\hbar}$ is exactly the **time-evolution operator**, which tells us how the quantum state evolves after a physical time t .

Unlike VQE, where physical time t is considered, in QAOA, the Hamiltonian H_C does not describe a physical system evolving naturally. Rather, it is an artificial Hamiltonian that encodes the objective function of an optimization problem. We still call it “*time-evolution*” because, mathematically, it is the same operation. For this reason, the time-evolution operator can be generalized to the QAOA case, since we are **not concerned with physical time and only need a unitary generated by a Hamiltonian**. Thus, we can write:

$$U(t) = e^{-iHt/\hbar} \Rightarrow U = e^{-i\alpha H}, \quad \alpha = \frac{t}{\hbar}$$

❓ Why introduce α ? Because α becomes a **free parameter**. So instead of saying “*let the system evolve for $t = 0.73$ ns*”, we say “*apply the unitary with parameter $\alpha = 0.37$* ”, since U may represent a quantum gate, a variational layer, a QAOA cost operator or a simulation step. In other words, α is a **scalar variational parameter** that we can optimize to find the best solution to our problem.

The α parameter controls the **strength of the unitary operator**. For example, if $\alpha = 0$, then $U = e^{-i0H} = I$, which means that the unitary does nothing.

If α is very large, then U will have a strong effect on the quantum state, causing a noticeable evolution.

Example 6: Connection with rotations

The unitary operator $U = e^{-i\alpha H}$ can be interpreted as a rotation in the Hilbert space. The parameter α controls the angle of rotation, and the Hamiltonian H determines the axis of rotation.

For example, we can retrieve all the Pauli rotations (page 50) from the general formula $U = e^{-i\alpha H}$ by choosing the appropriate Hamiltonian H :

- Pauli-X rotation:

$$R_X(\theta) = e^{-i\frac{\theta}{2}X} = \exp\left(-i\frac{\theta}{2}\begin{pmatrix}0 & 1 \\ 1 & 0\end{pmatrix}\right)$$

- Pauli-Y rotation:

$$R_Y(\theta) = e^{-i\frac{\theta}{2}Y} = \exp\left(-i\frac{\theta}{2}\begin{pmatrix}0 & -i \\ i & 0\end{pmatrix}\right)$$

- Pauli-Z rotation:

$$R_Z(\theta) = e^{-i\frac{\theta}{2}Z} = \exp\left(-i\frac{\theta}{2}\begin{pmatrix}1 & 0 \\ 0 & -1\end{pmatrix}\right)$$

8.6.5 Cost Hamiltonian and Cost Operator

Up to now we know that:

- Every optimization problem becomes a **Cost Hamiltonian** H_C ;
- Every Hamiltonian generates a unitary through $U = e^{-iHt}$.

Now we specialize this to the Cost Hamiltonian H_C , obtaining the first half of every QAOA layer: the **Cost Operator** $U_C(\gamma) = e^{-i\gamma H_C}$, where γ is a variational parameter.

🌀 **From Hamiltonian to Cost Operator.** The previous section introduced the general rule:

$$U = e^{-iHt}$$

Now we simply apply it to the **Cost Hamiltonian**. Since our Hamiltonian is H_C , its corresponding unitary is:

$$U_C(\gamma_i) = e^{-i\gamma_i H_C} = \exp(-i\gamma_i H_C) \quad (177)$$

Where $\gamma_i \in \mathbb{R}$ is the parameter associated with the i -th QAOA layer. This unitary operator (**quantum gate**) is called the **Cost Operator** (or **Cost Unitary**, or **Cost Evolution Operator**), which evolves the quantum state according to the Cost Hamiltonian.

❓ **Why is it called the Cost Hamiltonian.** Like in QUBO, we always begin with an optimization problem:

$$\min_x C(x)$$

The function $C(x)$ assigns a **cost** to every candidate solution x . The best solution is simply the one with the smallest cost. In QAOA we replace this classical function by H_C . Instead of assigning a cost, it assigns an energy. So, it is called the **Cost Hamiltonian** because it is a **quantum version of the classical cost function**.

Also, the **Cost Hamiltonian encodes the problem we want to solve**. Notice that H_C does **not** solve the optimization problem. Instead, it is a mathematical description of the problem. For example, suppose we have Max-Cut; we encode the entire problem in H_C . Then, we use QAOA to find the best solution. In other words, H_C is a **problem descriptor** and the graph linked to the Max-Cut problem disappears from the QAOA algorithm. The graph is only used to build H_C and then it is no longer needed.

🕒 **The goal of the Cost Operator.** The Cost Operator is defined as:

$$U_C(\gamma_i) = e^{-i\gamma_i H_C}$$

This operator **evolves the quantum state according to the energy landscape defined by the Cost Hamiltonian**. Thus:

- The **Cost Hamiltonian** defines which configurations are good or bad (their energies);

- The **Cost Operator** modifies the quantum amplitudes based on those energies.

It does **not immediately produce the optimal solution** after one application. Instead, it is **one step in the repeated QAOA evolution**.

🔍 **What does γ_i control?** Earlier we introduced the generic evolution (page 516):

$$U = e^{-i\alpha H}$$

Where α is a real number that controls the evolution. In the case of the Cost Operator, we have:

$$U_C(\gamma_i) = e^{-i\gamma_i H_C}$$

Where γ_i is a real number that controls **how strongly the Cost Hamiltonian acts during the i -th QAOA layer**. This parameter is a **variational parameter** that is optimized by the classical optimizer. The optimizer will find the best γ_i that produces the best solution.

⚠️ The implementation problem

The implementation of the Cost Operator **with suitable gates is a non-trivial step**. Because the Cost Operator:

$$U_C(\gamma_i) = e^{-i\gamma_i H_C}$$

Is still a **huge matrix** containing $2^n \times 2^n$ elements. We cannot implement it directly on a quantum computer. The quantum computer never materializes the entire matrix. Instead, it implements the same transformation through a sequence of elementary gates.

✅ So now the question is: **how do we implement the Cost Operator with suitable gates?** Instead of treating H_C as one giant matrix, we can write it as a **sum of tensor products of Pauli operators**:

$$I, \quad \sigma_X, \quad \sigma_Y, \quad \sigma_Z$$

Once the Hamiltonian is decomposed in this form, the corresponding Cost Operator can be implemented using elementary rotation gates.

Therefore, we can **write the Cost Hamiltonian H_C as a sum of tensor products of Pauli operators**:

$$H_C = \sum_k c_k P_k \tag{178}$$

Where c_k are real coefficients and each P_k is a tensor product of Pauli matrices. At this point, we can implement the corresponding unitary by implementing the evolution generated by each Pauli term.

In summary, the Cost Hamiltonian H_C is the quantum representation of the optimization problem. It assigns an energy to each candidate solution, and its

ground state corresponds to the optimal solution. In each QAOA layer, the Cost Hamiltonian generates the Cost Operator:

$$U_C(\gamma_i) = e^{-i\gamma_i H_C}$$

Where γ_i is a variational parameter. Since arbitrary Hamiltonians cannot be directly executed on quantum hardware, H_C is decomposed into sums of tensor products of Pauli operators, allowing the Cost Operator to be implemented using standard quantum gates such as rotation gates.

8.6.6 Mixer Hamiltonian and Mixer Operator

In the previous section we saw the **Cost Hamiltonian**, whose purpose is to **encode the optimization problem**. Now we introduce the Mixer Hamiltonian. Its purpose is **not** to optimize the cost function, but to **continuously move the quantum state through the search space**, preventing the algorithm from getting trapped in a fixed configuration.

The Mixer Hamiltonian is fundamental. Without it, the Cost Operator would repeatedly evolve the state according to the same energy landscape. The problem is that **nothing would encourage the algorithm to explore different candidate solutions**. Instead, the state could become concentrated around a limited region of the search space. Instead, the **Mixer Hamiltonian** solves exactly this problem. Its purpose is **to explore the space of possible quantum states (candidate solutions)**.

Cost vs Mixer

The role of the Cost and Mixer Hamiltonians are complementary. Let's summarize their differences in the following table:

Cost Hamiltonian	Mixer Hamiltonian
Encodes the optimization problem	Explores the search space
Depends on the problem	Usually fixed
Built from QUBO/Ising	Built from Pauli-X operators
Tries to favor good solutions	Creates transitions between solutions

Mixer Hamiltonian

The standard QAOA mixer is:

$$H_M = \sum_j \sigma_X^j \quad (179)$$

Where each σ_X^j **applies a Pauli-X operator only to the j -th qubit, while the identity acts on all the others:**

$$\sigma_X^j = I \otimes I \otimes \cdots \otimes \underbrace{\sigma_X}_{j\text{-th}} \otimes \cdots \otimes I$$

For example, with three qubits:

$$\begin{aligned} \sigma_X^1 &= \sigma_X \otimes I \otimes I \\ \sigma_X^2 &= I \otimes \sigma_X \otimes I \\ \sigma_X^3 &= I \otimes I \otimes \sigma_X \end{aligned}$$

Therefore, the **Mixer Hamiltonian** is simply a sum of one Pauli-X operator for every qubit:

$$H_M = X_1 + X_2 + \cdots + X_n$$

❓ **Why Pauli- X ?** The Pauli- X operator is a quantum gate that flips the state of a qubit. In the context of QAOA, suppose our current candidate solution is:

$$01010$$

Applying an X operation on one qubit changes one variable in the candidate solution. For example, applying X on the first qubit gives:

$$11010$$

The Mixer Hamiltonian therefore **creates transitions between different bit strings**. It allows the algorithm to leave the current candidate solution and explore nearby ones (i.e., those that differ by a single bit flip). This is why the mixer is associated with **exploration**. Unlike the Cost Hamiltonian, which only assigns energies, the **Mixer Hamiltonian actively mixes amplitudes between computational basis states**.

☰ The Mixer Operator

Exactly as we did for the Cost Hamiltonian, we can define the **Mixer Operator** as the unitary evolution generated by the Mixer Hamiltonian:

$$U_M(\beta_i) = e^{-i\beta_i H_M} \quad \text{with } \beta_i \in \mathbb{R} \quad (180)$$

❓ **Why is the Mixer Hamiltonian fixed?** Unlike the Cost Hamiltonian, which depends on the optimization problem, the **Mixer Hamiltonian is usually the same for every problem**. Because exploration is a generic operation. Regardless of whether we solve a Max-Cut problem, a Traveling Salesman Problem, or any other combinatorial optimization problem, we still **want to move through the space of candidate solutions**. Therefore, the **same exploration mechanism works for many optimization problems**.

✂ **How is it implemented?** Unlike the Cost Hamiltonian, the Mixer Hamiltonian is already extremely simple to implement. Since it is a sum of Pauli- X operators:

$$H_M = \sum_j \sigma_X^j$$

And we already know how to implement the unitary evolution of a Pauli- X operator:

$$R_x(\theta) = e^{-i\frac{\theta}{2}\sigma_X}$$

Each σ_X^j term corresponds to an R_x rotation on the j -th qubit. Therefore, the Mixer Operator is **implemented as a layer of parallel $R_x(\beta_i)$ gates applying the same rotation angle to every qubit**. With “parallel” we mean that all the R_x gates can be **applied simultaneously**, since they act on different qubits. This parallelism means also that β_i is a **single parameter for the entire layer**, not one per qubit. The Mixer Operator is therefore implemented as a single layer of R_x gates, all with the same rotation angle β_i . The optimizer will search for a vector of angles $\vec{\beta} = (\beta_1, \beta_2, \dots, \beta_p)$ for the p layers of the QAOA ansatz.

8.6.7 Alternating Operators and p Layers

Until now, we have seen the two main operators of QAOA independently:

- The **Cost Operator**:

$$U_C(\gamma) = e^{-i\gamma H_C}$$

- The **Mixer Operator**:

$$U_M(\beta) = e^{-i\beta H_M}$$

The natural question now is: *how are these operators actually used together to build QAOA?* And the answer is very simple: they are **applied alternately, layer after layer**. This alternating structure is exactly what gives the algorithm its name: *Quantum Approximate Optimization Algorithm*.

✂ **One QAOA layer.** Let's recall the QAOA circuit from Figure 33 (page 515). One **QAOA layer** is simply the application of the cost operator followed by the mixer operator, as shown in Figure 33:

$$U_C(\gamma_i) = e^{-i\gamma_i H_C} \quad \text{and} \quad U_M(\beta_i) = e^{-i\beta_i H_M}$$

Mathematically, a single QAOA layer can be expressed as:

$$U(\gamma_i, \beta_i) = U_M(\beta_i)U_C(\gamma_i) = e^{-i\beta_i H_M} \cdot e^{-i\gamma_i H_C} \quad (181)$$

The ansatz for a single layer of QAOA with two qubits is shown in the following circuit diagram:

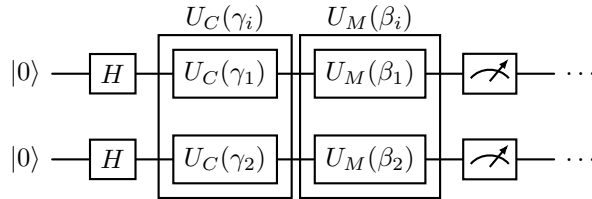


Figure 34: QAOA ansatz with one layer ($p = 1$) for two qubits ($n = 2$).

❓ **Why this order?** Initially, the algorithm prepares $|+\rangle^{\otimes n}$, which is an equal superposition of all possible solutions.¹⁹ At this point, every solution has the same probability, we have maximum exploration of the solution space, and no preference for good or bad solutions. Suppose we applied the mixer first; the state $|+\rangle^{\otimes n}$ is actually an eigenstate of the mixer Hamiltonian H_M , and thus:

$$U_M(\beta) |+\rangle^{\otimes n} = e^{-i\psi} |+\rangle^{\otimes n}$$

Where $e^{-i\psi}$ is only a **global phase**, which has **no physical effect**. So the first mixer would essentially do **nothing useful**.

Instead, we first apply the cost operator, which **modifies the amplitudes of the states based on their cost**. This is the first step in guiding the algorithm

¹⁹Actually, the algorithm prepares the state $|0\rangle^{\otimes n}$ and then applies a Hadamard gate to each qubit to create the superposition.

towards better solutions. Then, we apply the mixer operator, which **mixes the amplitudes of the states**, allowing for exploration of new solutions while still favoring those with lower costs.

✂ **The complete QAOA layer.** We can **extend** the single layer Equation 181 to p layers, where each layer has its own parameters γ_i and β_i . The complete QAOA circuit with p layers can be expressed as:

$$\left| \psi(\vec{\gamma}, \vec{\beta}) \right\rangle = U_M(\beta_p) U_C(\gamma_p) \cdots U_M(\beta_1) U_C(\gamma_1) |+\rangle^{\otimes n} \quad (182)$$

Where:

- $\vec{\gamma} = (\gamma_1, \gamma_2, \dots, \gamma_p)$ are the **parameters** for the **cost operators**.
- $\vec{\beta} = (\beta_1, \beta_2, \dots, \beta_p)$ are the **parameters** for the **mixer operators**.
- Parameter p is the **depth** of the QAOA circuit, which controls the number of alternating layers of cost and mixer operators. It is defined as the number of Cost-Mixer pairs in the circuit.

8.6.8 QUBO Problems for QAOA

This section is where QAOA reconnects with everything we saw in QUBO Formulation (page 406). In fact, there is almost no new mathematics here, but rather we answer an important practical question: *which optimization problems can we naturally solve with QAOA?* The answer is: **QUBO problems**.

❓ Why QUBO? At the beginning of the QAOA section, we saw that given an optimization problem, we can construct a cost Hamiltonian H_C and a mixer Hamiltonian H_M , run the QAOA circuit, and measure the output state to get a solution. But there is one missing step. *How do we build the Cost Hamiltonian?* The answer is simple: given an optimization problem, we can always write it in QUBO form, and then we can construct the Cost Hamiltonian from the QUBO matrix Q . Therefore, **QUBO is not the algorithm**, but rather the **standard mathematical representation** used before constructing the Cost Hamiltonian.

❓ But why QUBO and not some other representation? In this course, we will focus on QUBO because it is well suited for QAOA. The reason is that **QUBO already has exactly the mathematical structure that we need to construct a Hamiltonian**. Let's see why.

📖 The general QUBO formulation. Recall that a QUBO problem (page 406) is defined as follows:

$$\min_{x \in \{0,1\}^n} x^T Q x$$

Where x is a binary vector $x \in \{0,1\}^n$ and Q is an $n \times n$ matrix containing the linear and quadratic coefficients of the objective function.²⁰ Also, we know that the main diagonal entries Q_{ii} represent the **linear terms** x_i (i.e., contribution of individual variables), while the off-diagonal entries Q_{ij} represent the **quadratic terms** $x_i x_j$ (i.e., interaction between pairs of variables).

Example 7: QUBO representation of a polynomial

Suppose our object function is:

$$E(x) = x_1 + x_2 - 2x_1 x_2$$

This polynomial is the optimization problem. Now, let's encode it into a matrix. We want to find a way to represent:

$$E(x) = x^T Q x = x_1 + x_2 - 2x_1 x_2 \quad \text{with } x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

There are **two** ways to write this polynomial as a quadratic form.

1. **Symmetric matrix.** Substitute the values of x_1 and x_2 into the polynomial and rewrite it as a quadratic form:

$$E(x) = x^T \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} x$$

²⁰The matrix Q contains all the coefficients describing the optimization problem.

Where the matrix Q is composed as follows:

- The diagonal entries $Q_{11} = 1$ and $Q_{22} = 1$ correspond to the linear terms x_1 and x_2 .
- The off-diagonal entries $Q_{12} = -1$ and $Q_{21} = -1$ correspond to the quadratic term $-2x_1x_2$.

Now compute Qx :

$$Qx = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 - x_2 \\ -x_1 + x_2 \end{bmatrix}$$

Then compute $x^T Qx$:

$$\begin{aligned} x^T Qx &= \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} x_1 - x_2 \\ -x_1 + x_2 \end{bmatrix} \\ &= x_1(x_1 - x_2) + x_2(-x_1 + x_2) \\ &= x_1^2 - x_1x_2 - x_2x_1 + x_2^2 \\ &= x_1^2 + x_2^2 - 2x_1x_2 \end{aligned}$$

So we have successfully represented the polynomial as a quadratic form using a symmetric matrix. However, notice something: the interaction appeared **twice** $-x_1x_2$ and $-x_2x_1$, is stored in two different entries of the matrix.

2. **Upper triangular.** Instead, we can write the polynomial as a quadratic form using an upper triangular matrix:

$$E(x) = x^T \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} x$$

Where the matrix Q is composed as follows:

- The diagonal entries $Q_{11} = 1$ and $Q_{22} = 1$ correspond to the linear terms x_1 and x_2 .
- The off-diagonal entry $Q_{12} = -2$ corresponds to the quadratic term $-2x_1x_2$.

Now compute Qx :

$$Qx = \begin{bmatrix} 1 & -2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} x_1 - 2x_2 \\ x_2 \end{bmatrix}$$

Then compute $x^T Qx$:

$$\begin{aligned} x^T Qx &= \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} x_1 - 2x_2 \\ x_2 \end{bmatrix} \\ &= x_1(x_1 - 2x_2) + x_2(x_2) \\ &= x_1^2 - 2x_1x_2 + x_2^2 \\ &= x_1^2 + x_2^2 - 2x_1x_2 \end{aligned}$$

So we have successfully represented the polynomial as a quadratic form using an upper triangular matrix. Notice that the interaction appeared **once** $-2x_1x_2$, and is stored in a single entry of the matrix.

So when we construct Q , we have to decide **how to store the interaction coefficients**: symmetric (split the coefficient equally between Q_{ij} and Q_{ji}) or upper triangular (store the coefficient in a single entry Q_{ij}). The choice is arbitrary.

In summary, QUBO provides a standard mathematical formulation for many combinatorial optimization problems. A QUBO problem is written as:

$$\min_{x \in \{0,1\}^n} x^T Q x$$

Where x is a binary vector and Q contains the linear and quadratic coefficients of the objective function. Since this formulation can be systematically converted into spin variables and then into a Cost Hamiltonian, QUBO is particularly well suited for QAOA. The matrix Q may be symmetric or upper triangular because the interaction terms $x_i x_j$ are symmetric.

8.6.9 From QUBO to Cost Hamiltonian

We have:

- An optimization problem;
- The optimization problem is expressed as a QUBO problem;
- The Q matrix is known and contains the coefficients of the QUBO problem.

But a quantum computer cannot optimize binary variables $x_i \in \{0, 1\}$ directly. Instead, it evolves **quantum states** under a **Hamiltonian**. So in this section we will see *how to convert a classical QUBO into a quantum Cost Hamiltonian*.

🌀 Step 1: Binary variables $\rightarrow$ Spin variables (QUBO $\rightarrow$ Ising model)

A quantum computer does **not naturally understand binary variables** $x_i \in \{0, 1\}$. It naturally manipulates **qubits**, whose computational basis states are eigenstates of the Pauli-Z operator. The eigenvalues of the Pauli-Z operator are ± 1 , which is not the same as the binary values 0 and 1, but are exactly the values of **spin variables**! So we first rewrite the optimization problem using spins.

Suppose we have a QUBO problem:

$$\min_x x^T Q x \quad \text{s.t. } x_i \in \{0, 1\}$$

Substitute the binary variables x_i with spin variables s_i using the relation:

$$s = 2x - 1 \quad \Longleftrightarrow \quad x = \frac{1 + s}{2} \quad (183)$$

After substituting $x_i = \frac{1+s_i}{2}$ into the QUBO objective function (page 407):

$$y = \sum_i a_i x_i + \sum_{i>j} q_{ij} x_i x_j$$

Every QUBO can be rewritten as an Ising model (page 446):

$$\underbrace{y}_{H_{\text{Ising}}} = c + \sum_i S_{ii} s_i + \sum_{i>j} S_{ij} s_i s_j$$

Where:

- c is a constant term that does not depend on the spin variables s_i ;
- S_{ii} are the local fields (linear coefficients);
- S_{ij} are the coupling strengths (quadratic coefficients).

Notice that the Ising model has the same structure as the QUBO problem: a linear term (the first sum) and a quadratic term (the second sum). The only difference is that the variables are now spins $s_i \in \{-1, +1\}$ instead of binary variables $x_i \in \{0, 1\}$. Also, the matrix Q is **replaced** by a new matrix S that contains the **coefficients of the Ising model**, but **describe exactly the same optimization problem** as the original QUBO.

🔗 Step 2: From Ising Model to Cost Hamiltonian

The Ising model is a classical model that describes the energy of a system of spins. However, the spins are still classical objects that can only be in one of two possible states. To “quantize” the Ising model, we replace the classical spin variables s_i with quantum operators. The natural choice for this is the Pauli-Z operator, which has eigenvalues ± 1 and acts on qubits (page 450). The resulting quantum Hamiltonian is called the **Cost Hamiltonian**:

$$H_C = \sum_i S_{ii} \sigma_Z^i + \sum_{i>j} S_{ij} \sigma_Z^i \sigma_Z^j \quad (184)$$

Nothing of difficult has happened here: we have **simply replaced the classical spin variables s_i with quantum operators σ_Z^i** , which act on the i -th qubit. This simple **replacement is allowed** because the **Pauli-Z operator preserves the mathematical structure of the optimization problem** while turning it into a quantum operator:

$$Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle, \quad \text{with eigenvalues } +1 \text{ and } -1$$

The resulting Hamiltonian H_C is a matrix of size $2^n \times 2^n$, where n is the number of qubits. This exponential growth in size is due to the fact that each term σ_Z^i acts on a different qubit, and the total Hilbert space is the tensor product of all the qubits ($I \otimes I \otimes \dots \otimes \sigma_Z^i \otimes \dots \otimes I$).

The Cost Hamiltonian is a quantum operator that encodes the same optimization problem as the original QUBO, but in a form that can be manipulated by a quantum computer. Also, it is **diagonal** and **enumerates the energy of all possible states**, which is exactly what we need for optimization.

❓ **Why does the constant disappear?** The constant term c does not appear in the Hamiltonian because **adding a constant to every energy level does not change which state has the lowest energy**. Since optimization only depends on the ordering of the energies, the constant term can be safely ignored.

In summary, to obtain the Cost Hamiltonian from a QUBO problem, we follow these steps:

1. The QUBO formulation is rewritten in terms of spin variables using the transformation $s = 2x - 1$. This produces a spin objective function of the form:

$$y = c + \sum_i S_{ii} s_i + \sum_{i>j} S_{ij} s_i s_j$$

2. Each spin variable s_i is replaced by the corresponding Pauli-Z operator σ_Z^i , obtaining the Cost Hamiltonian:

$$H_C = \sum_i S_{ii} \sigma_Z^i + \sum_{i>j} S_{ij} \sigma_Z^i \sigma_Z^j$$

The constant term can be discarded because it shifts all energies equally.

The resulting Hamiltonian is diagonal, acts on quantum states, and its ground state corresponds to the optimal solution of the original optimization problem.

At this point, we have successfully built the Cost Hamiltonian. The remaining question is purely practical: *how do we implement $U_C(\gamma) = e^{-i\gamma H_C}$ (i.e., the unitary operator corresponding to the Cost Hamiltonian) as an actual quantum circuit?* The next section answers this.

8.6.10 Practical QAOA Circuit Construction

Up to now, we have built the entire mathematical pipeline: optimization problem, QUBO formulation, spin formulation, Cost Hamiltonian H_C , Cost Operator $U_C(\gamma)$. There is only one question left: *how do we actually draw the quantum circuit?*

What we have now is the Cost Hamiltonian:

$$H_C = \sum_i S_{ii} Z_i + \sum_{i < j} S_{ij} Z_i Z_j$$

And the corresponding Cost Operator (page 518):

$$U_C(\gamma) = e^{-i\gamma H_C}$$

How do we implement this exponential using elementary gates? The answer is really simple: **implement each term of the Hamiltonian separately.**

Step 1: Linear Terms

This is the easy part. Consider the first term of the Hamiltonian, the linear terms:

$$\sum_i S_{ii} Z_i$$

Its corresponding evolution operator is:

$$e^{-i\gamma S_{ii} Z_i} = \exp(-i\gamma S_{ii} Z_i)$$

And the rotation operator is:

$$R_Z(\theta) = e^{-i\frac{\theta}{2} Z}$$

Immediately, we can see that **every linear term Z_i is just an R_Z rotation with an angle proportional to the coefficient S_{ii} and the variational parameter γ .** Thus, for every linear term $S_{ii} Z_i$, we can implement the corresponding evolution operator $e^{-i\gamma S_{ii} Z_i}$ using a single-qubit R_Z rotation on qubit i with angle γS_{ii} .

$$q_i \text{ --- } \boxed{R_Z(\gamma S_{ii})} \text{ ---}$$

Figure 35: Circuit implementing the evolution operator $e^{-i\gamma S_{ii} Z_i}$ for a single linear term $S_{ii} Z_i$.

🔗 Step 2: Interaction Terms

Now consider the second term of the Hamiltonian, the interaction terms:

$$\sum_{i < j} S_{ij} Z_i Z_j$$

This is slightly different, because it acts simultaneously on two qubits. There is **no native quantum gate** that implements this operator, therefore we must built it. The “*difficulty*” derives from the fact that a single R_Z rotation only “*looks at*” one qubit, but the Hamiltonian term $Z_i Z_j$ depends on **both** qubits. Therefore, we need to temporarily encode the joint information of both qubits onto one qubit, apply a single R_Z rotation, and then restore the original basis. This is an elegant trick.

In general, for every interaction $S_{ij} Z_i Z_j$, we can implement the corresponding evolution operator $e^{-i\gamma S_{ij} Z_i Z_j}$ using the following circuit.

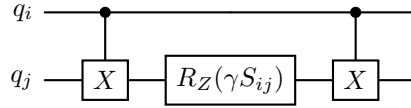
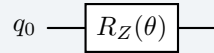


Figure 36: Circuit implementing the evolution operator $e^{-i\gamma S_{ij} Z_i Z_j}$ for a single interaction term $S_{ij} Z_i Z_j$.

❓ **Why does this circuit work?** This is a beautiful trick. The **first CNOT computes the parity between the two qubits**. The R_Z rotation applies a phase depending on that parity. The second CNOT undoes the temporary entanglement created by the first CNOT. The result is that the two qubits are rotated in phase depending on their parity, which is exactly what the operator $e^{-i\gamma S_{ij} Z_i Z_j}$ does.

🔗 Deepening: Why does a single R_Z rotation not work?

A single R_Z gate only sees one qubit. Suppose we apply:



The phase applied depends **only on** q_0 . It has absolutely no information about the state of any other qubit. However, $Z_i Z_j$ depends on both qubits. Consider the interaction $Z_i Z_j$ and evaluate it:

State	Z_i	Z_j	$Z_i Z_j$
$ 00\rangle$	+1	+1	+1
$ 01\rangle$	+1	-1	-1
$ 10\rangle$	-1	+1	-1
$ 11\rangle$	-1	-1	+1

Notice that the phase should depend on **both qubits together**. For example, the state $|00\rangle$ and $|11\rangle$ receive the **same** phase, while the states $|01\rangle$ and $|10\rangle$ receive another phase. Thus, a single R_Z gate is not enough, because it only sees one qubit. We need to temporarily encode the joint information of both qubits onto one qubit, apply a single R_Z rotation, and then restore the original basis. This is exactly what the circuit above does.

At this point, we can summarize the translation rules from Hamiltonian terms to quantum gates:

- For every linear term $S_{ii}Z_i$, apply a single-qubit R_Z rotation on qubit i with angle γS_{ii}
- For every interaction term $S_{ij}Z_iZ_j$, apply a two-qubit circuit with two CNOTs and a single R_Z rotation on qubit j with angle γS_{ij} , as shown in the figure above.

Hamiltonian Term	Circuit
$S_{ii}Z_i$	One R_Z gate
$S_{ij}Z_iZ_j$	CNOT $\rightarrow R_Z \rightarrow$ CNOT

Example 8: Building a simple Cost Layer

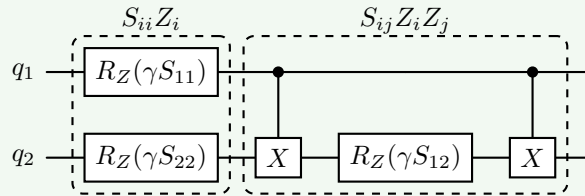
Suppose the spin Hamiltonian is:

$$H_C = S_{11}Z_1 + S_{22}Z_2 + S_{12}Z_1Z_2$$

The Cost Layer (i.e. the Cost Operator $U_C(\gamma)$) is:

$$U_C(\gamma) = e^{-i\gamma H_C} = e^{-i\gamma(S_{11}Z_1 + S_{22}Z_2 + S_{12}Z_1Z_2)}$$

Using the translation rules, we can build the corresponding quantum circuit:



Notice the structure. First, all the **single-qubit** contributions, then all the **interaction** contributions.

❓ What about the Mixer Layer?

The Mixer Layer is much simpler because it is always the same and it is linear (i.e. it only has single-qubit terms). Recall that the Mixer Hamiltonian is:

$$H_M = \sum_i X_i$$

And the corresponding Mixer Operator is:

$$U_M(\beta) = e^{-i\beta H_M} = e^{-i\beta \sum_i X_i}$$

The translation rule is the **same as for the linear terms of the Cost Hamiltonian**: **for every X_i term, apply a single-qubit R_X rotation on qubit i with angle β** . Therefore, the Mixer Layer is simply a layer of R_X rotations, one for each qubit. Notice that the angle of the rotation is the variational parameter β , which is the same for all qubits! Thus, we can parallelize the Mixer Layer and apply all the R_X rotations at the same time:

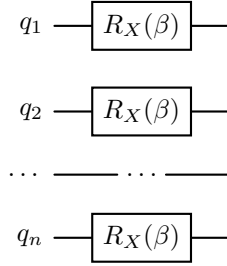


Figure 37: Circuit implementing the Mixer Layer $U_M(\beta)$ for n qubits.

Q A useful observation

Notice something elegant. The **shape** of the QAOA circuit is almost **independent of the optimization problem**. In fact, the structure is always the same: Hadamards (to create a superposition), followed by p layers of Cost and Mixer Operators, and finally measurements. What changes are only:

- The coefficients S_{ii} which determine the R_Z rotation angles in the Cost Layer;
- The coefficients S_{ij} which determine which $Z_i Z_j$ interaction blocks are present and their rotation angles in the Cost Layer;
- The variational parameters γ and β which determine the rotation angles in the Cost and Mixer Layers, respectively.

Therefore, changing from Max-Cut to Maximum Independent Set does **not** require inventing a new algorithm. We **simply build a different Cost Hamiltonian, and the corresponding Cost Layer changes automatically.**

In summary, the QAOA is a **general-purpose quantum algorithm** extremely powerful that takes a classical optimization problem, expresses it as a QUBO, converts it into a Cost Hamiltonian, transforms that Hamiltonian into a quantum circuit, alternates it with a fixed Mixer circuit, and optimizes the circuit parameters (γ, β) using a classical optimizer.

1. **Input:** A classical optimization problem, for example Max-Cut or Maximum Independent Set.
2. **QUBO Formulation:** Express the problem as a QUBO, which is a quadratic polynomial of binary variables.
3. **Spin Formulation:** Convert the QUBO into a spin Hamiltonian, which is a polynomial of Pauli Z operators (Ising model).
4. **Cost Hamiltonian:** Build the Cost Hamiltonian H_C from the spin formulation (“*quantumize*” the Ising model).
5. **Build Cost Layer:** Build the Cost Layer $U_C(\gamma)$ from the Cost Hamiltonian, using the translation rules for linear and interaction terms. In particular, for every linear term $S_{ii}Z_i$, apply a single-qubit R_Z rotation on qubit i with angle γS_{ii} , and for every interaction term $S_{ij}Z_iZ_j$, apply a two-qubit circuit with two CNOTs and a single R_Z rotation on qubit j with angle γS_{ij} .
6. **Build Mixer Layer:** Build the Mixer Layer $U_M(\beta)$, which is always the same: for every X_i term, apply a single-qubit R_X rotation on qubit i with angle β . This can be parallelized, applying all the R_X rotations at the same time.
7. **Repeat:** Repeat the Cost and Mixer Layers p times, where p is the depth of the QAOA circuit.
8. **Measure:** Measure the qubits in the computational basis to obtain a bit-string, which is a candidate solution to the original optimization problem.
9. **Classical Optimization:** Use a classical optimizer to adjust the variational parameters (γ, β) to maximize the expected value of the Cost Hamiltonian, which corresponds to finding better solutions to the optimization problem.

9 Exams

9.1 2024

9.1.1 June 20

🔗 Exercise 1 (Simon's Algorithm).

1. State the problem that is solved by the Simon's algorithm. Briefly describe the algorithm, draw its quantum circuit and state its speedup compared to a classical algorithm.
2. Consider a function $f(\{0, 1\}^2) \rightarrow \{0, 1\}^2$ such that $f(00) = f(10)$, $f(01) = f(11)$. Use the Simon's algorithm to compute its period.

Motivate your answer by showing all stages of the computation.

✅ Solution 1 (Problem solved by Simon's algorithm). We are given a black-box function:

$$f : \{0, 1\}^n \rightarrow \{0, 1\}^n$$

With the promise that there exists a hidden non-zero string:

$$a \in \{0, 1\}^n, \quad a \neq 0^n$$

Such that:

$$f(x) = f(y) \iff x = y \quad \text{or} \quad x = y \oplus a$$

Equivalently:

$$f(x) = f(x \oplus a)$$

For every $x \in \{0, 1\}^n$. The objective is to determine the hidden period a . Because $a \neq 0$, every output has exactly two corresponding inputs: x and $x \oplus a$. So the domain is divided into cosets/pairs:

$$\{x, x \oplus a\}$$

Classical vs quantum complexity. Classically, we need to find two different inputs x_1, x_2 , producing the same output:

$$f(x_1) = f(x_2)$$

Then:

$$a = x_1 \oplus x_2$$

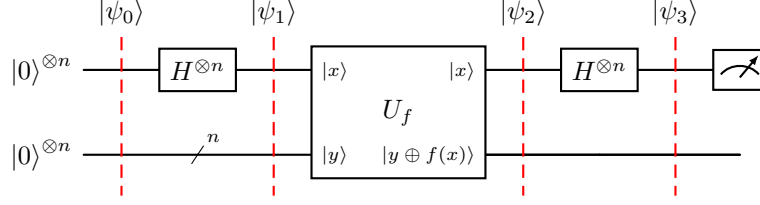
Because this is essentially a collision-search problem, the randomized classical query complexity is:

$$O\left(2^{n/2}\right)$$

Simon's quantum algorithm requires only:

$$O(n)$$

Oracle calls, because each execution gives one linear constraint on a . Therefore Simon gives an **exponential speedup** over classical algorithms.



Simon circuit.

The state evolution in the general case is as follows:

- **Initial state:**

$$|\psi_0\rangle = |0\rangle^{\otimes n} |0\rangle^{\otimes n}$$

- **First Hadamards:**

$$|\psi_1\rangle = H^{\otimes n} |\psi_0\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle |0\rangle^{\otimes n}$$

- **Oracle:**

$$\begin{aligned} |\psi_2\rangle &= U_f |\psi_1\rangle \\ &= U_f \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle |0\rangle^{\otimes n} \\ &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} U_f |x\rangle |0\rangle^{\otimes n} \\ &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle |f(x)\rangle \end{aligned}$$

- **Second Hadamards:**

$$\begin{aligned} |\psi_3\rangle &= H^{\otimes n} |\psi_2\rangle \\ &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} H^{\otimes n} |x\rangle |f(x)\rangle \\ &= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} \left(\frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle \right) |f(x)\rangle \\ &= \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle |f(x)\rangle \end{aligned}$$

- **Measurement.** Group together x and $x \oplus a$. Since:

$$f(x) = f(x \oplus a)$$

Their combined amplitude contains:

$$(-1)^{x \cdot y} + (-1)^{(x \oplus a) \cdot y}$$

Using the property:

$$(x \oplus a) \cdot y = x \cdot y \oplus a \cdot y$$

Rewriting the amplitude:

$$\begin{aligned} (-1)^{x \cdot y} + (-1)^{(x \oplus a) \cdot y} &= (-1)^{x \cdot y} + (-1)^{x \cdot y \oplus a \cdot y} \\ &= (-1)^{x \cdot y} + (-1)^{x \cdot y} (-1)^{a \cdot y} \\ &= (-1)^{x \cdot y} (1 + (-1)^{a \cdot y}) \end{aligned}$$

Therefore, if $a \cdot y = 1$, the amplitude is zero (destructive interference):

$$1 + (-1)^{a \cdot y} = 1 + (-1) = 0$$

If $a \cdot y = 0$, the amplitude is non-zero (constructive interference):

$$1 + (-1)^{a \cdot y} = 1 + 1 = 2$$

Hence measurements can only return y such that:

$$a \cdot y = 0 \pmod{2}$$

We repeat the algorithm until we have $n - 1$ independent equations, then solve the over $GF(2)$ to recover a .

💡 Remark: Why do we want destructive interference?

Before the final Hadamards, the first register contains information about **all possible inputs** x . The problem is that if we simply measured it, we would essentially obtain a random x , which tells us almost nothing about the hidden period a .

The purpose of the final $H^{\otimes n}$ is therefore **not just to create another superposition**. It makes the different computational paths interfere so that the hidden structure of f becomes observable.

For every pair:

$$x, \quad x \oplus a,$$

Simon's promise tells us:

$$f(x) = f(x \oplus a)$$

Because they correspond to the **same output state**, their amplitudes can interfere.

For a candidate measurement y , the two contributions are:

$$(-1)^{x \cdot y} + (-1)^{(x \oplus a) \cdot y} = (-1)^{x \cdot y} (1 + (-1)^{a \cdot y})$$

Now there are two cases:

$a \cdot y$	Interference	Result
0	constructive	amplitude survives
1	destructive	amplitude = 0

So destructive interference acts as a filter:

$$a \cdot y = 1 \implies P(y) = 0$$

And therefore measurement can only give:

$$a \cdot y = 0$$

That is the deep reason we care about destructive interference: **it removes outcomes that do not contain the constraint we need.**

In general, one of the most important patterns to remember is: **superposition alone does not give a quantum advantage. Interference is what makes the useful answers more likely and the useless answers less likely or impossible.** Simon is an especially clear example because the unwanted outcomes are **completely cancelled.**

For example, if:

$$a = 10 \implies a \cdot y = y_1$$

The four possible y 's start conceptually as:

$$y = 00, 01, 10, 11$$

Interference eliminates the two with $y_1 = 1$, leaving only:

$$y = 00, 01, \cancel{10}, \cancel{11}$$

And both of these remaining y 's satisfy the constraint:

$$a \cdot y = 0$$

So the quantum computer is **not directly revealing** a . It uses destructive interference to prevent us from observing strings inconsistent with a . Repeating the experiment gives enough surviving constraints to reconstruct a . In other words, **destructive interference removes measurement outcomes inconsistent with the hidden period.**

✔ **Solution 1 (Specific exam function).** We have:

$$f : \{0, 1\}^2 \rightarrow \{0, 1\}^2$$

And:

$$f(00) = f(10), \quad f(01) = f(11)$$

We want to determine the hidden period a . Before doing any quantum computation, we can already see what it must be. Since Simon's promise tells us:

$$f(x) = f(x \oplus a)$$

We also know that any value XORed with 00 returns the same value:

x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

Thus, the only way to satisfy:

$$f(00) = f(10)$$

Is if:

$$00 \oplus a = 10 \quad \Rightarrow \quad a = 10$$

We are confident that this is the correct answer, so let's try the other pair:

$$f(01) = f(11)$$

We can see that:

$$01 \oplus a = 11 \quad \Rightarrow \quad 01 \oplus 10 = 11$$

$$11 \oplus a = 01 \quad \Rightarrow \quad 11 \oplus 10 = 01$$

Thus, both pairs are **consistent** with $a = 10$. Now let's see if the quantum algorithm gives the same answer.

Step 1. Initial state. Since $n = 2$, there are four qubits total:

$$|\psi_0\rangle = |00\rangle |00\rangle$$

Step 2. First Hadamards. Apply $H^{\otimes 2}$ to the first register:

$$\begin{aligned} |\psi_1\rangle &= H^{\otimes 2} |\psi_0\rangle \\ &= H^{\otimes 2} |00\rangle |00\rangle \\ &= (H|0\rangle \otimes H|0\rangle) \otimes |00\rangle \\ &= \left(\frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \right) \otimes |00\rangle \\ &= \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) \otimes |00\rangle \end{aligned}$$

We omit the identity on the second register for brevity:

$$(H^{\otimes 2} \otimes I^{\otimes 2}) |\psi_0\rangle \tag{185}$$

Step 3. Oracle. The oracle leaves unchanged the first register and applies $y \oplus f(x)$ to the second register. Since the second register is initialized to $|00\rangle$, it will simply be replaced by $f(x)$:

$$\begin{aligned} |\psi_2\rangle &= U_f |\psi_1\rangle \\ &= U_f \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) \otimes |00\rangle \end{aligned}$$

Now we apply the oracle to each term. The first register remains unchanged, while the second register is replaced by $f(x)$. We follow the oracle's definition:

$$U_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$$

$$\begin{aligned} |\psi_2\rangle &= U_f \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) \otimes |00\rangle \\ &= \frac{1}{2} (|00\rangle |f(00)\rangle + |01\rangle |f(01)\rangle + |10\rangle |f(10)\rangle + |11\rangle |f(11)\rangle) \\ &\downarrow \text{Since } f(00) = f(10) \text{ and } f(01) = f(11) \\ &= \frac{1}{2} (|00\rangle |f(00)\rangle + |01\rangle |f(01)\rangle + |10\rangle \underbrace{|f(10)\rangle}_{|f(00)\rangle} + |11\rangle \underbrace{|f(11)\rangle}_{|f(01)\rangle}) \\ &= \frac{1}{2} (|00\rangle |f(00)\rangle + |01\rangle |f(01)\rangle + |10\rangle |f(00)\rangle + |11\rangle |f(01)\rangle) \\ &\downarrow \text{Group terms with the same output } f(x) \\ &= \frac{1}{2} [(|00\rangle + |10\rangle) |f(00)\rangle + (|01\rangle + |11\rangle) |f(01)\rangle] \end{aligned}$$

Step 4. Apply the second $H^{\otimes 2}$. We apply $H^{\otimes 2}$ to the first register:

$$\begin{aligned} |\psi_3\rangle &= (H^{\otimes 2} \otimes I^{\otimes 2}) |\psi_2\rangle \\ &= \frac{1}{2} [H^{\otimes 2} (|00\rangle + |10\rangle) |f(00)\rangle + H^{\otimes 2} (|01\rangle + |11\rangle) |f(01)\rangle] \end{aligned}$$

Now we compute each term separately:

- From the theory, we know that:

$$H |0\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle), \quad H |1\rangle = \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle)$$

- Therefore, the combination $|00\rangle$ is:

$$\begin{aligned} H^{\otimes 2} |00\rangle &= H |0\rangle \otimes H |0\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \\ &= \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) \end{aligned}$$

- The combination $|10\rangle$ is:

$$\begin{aligned} H^{\otimes 2} |10\rangle &= H |1\rangle \otimes H |0\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \\ &= \frac{1}{2} (|00\rangle + |01\rangle - |10\rangle - |11\rangle) \end{aligned}$$

- The combination $|01\rangle$ is:

$$\begin{aligned} H^{\otimes 2} |01\rangle &= H |0\rangle \otimes H |1\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\ &= \frac{1}{2} (|00\rangle - |01\rangle + |10\rangle - |11\rangle) \end{aligned}$$

- Finally, the combination $|11\rangle$ is:

$$\begin{aligned} H^{\otimes 2} |11\rangle &= H |1\rangle \otimes H |1\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \otimes \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \\ &= \frac{1}{2} (|00\rangle - |01\rangle - |10\rangle + |11\rangle) \end{aligned}$$

Before substituting these results back into the expression for $|\psi_3\rangle$, we can simplify the two terms:

- The first term $|f(00)\rangle$:

$$\begin{aligned} H^{\otimes 2} (|00\rangle + |10\rangle) &= \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) + \\ &\quad \frac{1}{2} (|00\rangle + |01\rangle - |10\rangle - |11\rangle) \\ &= \frac{1}{2} (2|00\rangle + 2|01\rangle) \\ &= |00\rangle + |01\rangle \end{aligned}$$

Where the terms $|10\rangle$ and $|11\rangle$ cancel out due to destructive interference.

- The second term $|f(01)\rangle$:

$$\begin{aligned} H^{\otimes 2} (|01\rangle + |11\rangle) &= \frac{1}{2} (|00\rangle - |01\rangle + |10\rangle - |11\rangle) + \\ &\quad \frac{1}{2} (|00\rangle - |01\rangle - |10\rangle + |11\rangle) \\ &= \frac{1}{2} (2|00\rangle - 2|01\rangle) \\ &= |00\rangle - |01\rangle \end{aligned}$$

Where the terms $|10\rangle$ and $|11\rangle$ cancel out due to destructive interference.

Now we can substitute these results back into the expression:

$$\begin{aligned} |\psi_3\rangle &= \frac{1}{2} [(|00\rangle + |01\rangle) |f(00)\rangle + (|00\rangle - |01\rangle) |f(01)\rangle] \\ &= \frac{1}{2} [|00\rangle |f(00)\rangle + |01\rangle |f(00)\rangle + |00\rangle |f(01)\rangle - |01\rangle |f(01)\rangle] \\ &= \frac{1}{2} [|00\rangle (|f(00)\rangle + |f(01)\rangle) + |01\rangle (|f(00)\rangle - |f(01)\rangle)] \end{aligned}$$

Step 5. Measurement. We measure the first register. First of all, we can see that it contains only $|00\rangle$ and $|01\rangle$, so immediately:

$$P(10) = P(11) = 0$$

Thus, the only possible outcomes are:

$$y = 00, 01$$

Now the problem is that the “amplitude” of $|00\rangle$ is not just a scalar, but a superposition of the second register:

$$|00\rangle (|f(00)\rangle + |f(01)\rangle)$$

And similarly for $|01\rangle$:

$$|01\rangle (|f(00)\rangle - |f(01)\rangle)$$

However, we try to take the squared norm of that second-register vector:

$$P(00) = \left\| \frac{1}{2} (|f(00)\rangle + |f(01)\rangle) \right\|^2$$

Since the Simon’s promise tells us:

$$f(00) = f(10), \quad f(01) = f(11)$$

Then: $f(00) \neq f(01)$, so they are distinct computational-basis states. Using the orthonormality of the computational basis, we have:

$$\begin{aligned} P(00) &= \left\| \frac{1}{2} (|f(00)\rangle + |f(01)\rangle) \right\|^2 \\ &= \frac{1}{4} \| |f(00)\rangle + |f(01)\rangle \|^2 \\ &= \frac{1}{4} (\langle f(00)|f(00)\rangle + \langle f(00)|f(01)\rangle + \\ &\quad \langle f(01)|f(00)\rangle + \langle f(01)|f(01)\rangle) \\ &= \frac{1}{4} (1 + 0 + 0 + 1) \\ &= \frac{1}{2} \end{aligned}$$

Similarly:

$$\begin{aligned} P(01) &= \frac{1}{4} (\langle f(00)|f(00)\rangle - \langle f(00)|f(01)\rangle - \\ &\quad \langle f(01)|f(00)\rangle + \langle f(01)|f(01)\rangle) \\ &= \frac{1}{4} (1 - 0 - 0 + 1) \\ &= \frac{1}{2} \end{aligned}$$

Therefore, the measurement of the first register gives:

$$P(00) = P(01) = \frac{1}{2}, \quad P(10) = P(11) = 0$$

So the only possible outcomes are:

$$y = 00, 01$$

Let's try both of them:

- If $y = 00$, then the constraint is:

$$a \cdot y = 0 \quad \Rightarrow \quad a_1 \cdot 0 + a_2 \cdot 0 = 0$$

Which is always satisfied, so it gives no information about a .

- If $y = 01$, then the constraint is:

$$a \cdot y = 0 \quad \Rightarrow \quad a_1 \cdot 0 + a_2 \cdot 1 = 0$$

Which implies $a_2 = 0$, giving us information about a .

So the possible values of the hidden period a are:

$$a = 00, 10$$

But since Simon's promise tells us that $a \neq 00$, the a_1 term must be 1, so we conclude:

$$a = 10$$

We have therefore recovered the hidden period $a = 10$ using Simon's algorithm. For $n = 2$, only $n - 1 = 1$ linearly independent equation is required to determine the nonzero hidden period. In this execution, the measurement $y = 01$ provides exactly this equation. In general, however, more than $n - 1$ circuit executions may be necessary because some measured vectors may be zero or linearly dependent.

❓ **Exercise 2 (Qubit States and Operations).**

- Describe the state space of a single qubit in terms of the two basis states $|0\rangle$ and $|1\rangle$. Explain what the Bloch Sphere is, how a qubit can be represented on it, and the meaning of the global phase.
- Describe the mathematical formalism used to represent a qubit gate and the properties it must have. Show that gates σ_X and σ_Y satisfy those properties.
- Provide a brief description of the no-cloning and no-delete principles.

✔ **Solution 2 (Single-Qubit state space).** A single qubit is a normalized vector in the two-dimensional complex Hilbert space:

$$\mathcal{H} = \mathbb{C}^2$$

Using the computational basis:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

The most general pure state of a qubit is:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad \alpha, \beta \in \mathbb{C}$$

And normalization condition:

$$|\alpha|^2 + |\beta|^2 = 1$$

If the qubit is measured in the computational basis:

$$P(0) = |\alpha|^2, \quad P(1) = |\beta|^2$$

Global phase. At first, α and β appear to contain four real parameters because each is complex. However, not all of these parameters correspond to physically distinct states. In particular:

$$|\psi\rangle, \quad \text{and} \quad |\psi'\rangle = e^{i\gamma} |\psi\rangle$$

Represent the **same physical quantum state**, where $e^{i\gamma}$ is called a **global phase**. For example, if:

$$|\psi\rangle = \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

And:

$$|\psi'\rangle = i \frac{1}{\sqrt{2}} (|0\rangle + |1\rangle)$$

Then $|\psi\rangle$ and $|\psi'\rangle$ are physically indistinguishable, even though they have different coefficients. The global phase does not affect measurement probabilities. Because measurement probabilities are unchanged:

$$|e^{i\gamma}\alpha|^2 = |e^{i\gamma}|^2 |\alpha|^2 = |\alpha|^2$$

Since:

$$|e^{i\gamma}| = 1$$

And thus:

$$|\psi\rangle \sim e^{i\gamma} |\psi\rangle$$

💬 **Remark: Why is $|e^{i\gamma}| = 1$?**

Using **Euler's formula**:

$$e^{i\gamma} = \cos \gamma + i \sin \gamma$$

The modulus of a complex number $a + ib$ is:

$$|a + ib| = \sqrt{a^2 + b^2}$$

Therefore:

$$|e^{i\gamma}| = \sqrt{\cos^2 \gamma + \sin^2 \gamma} = \sqrt{1} = \boxed{1}$$

Geometrically, $e^{i\gamma}$ is a **point on the unit circle** in the complex plane. Changing γ only rotates the point; its distance from the origin always remains 1.

This is why multiplying a quantum state by a global phase $e^{i\gamma}$ does **not change the magnitude of any amplitude**, and hence does not change measurement probabilities.

Bloch Sphere. After imposing normalization and removing the physically irrelevant global phase, every pure single-qubit state can be written as:

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle$$

With:

$$0 \leq \theta \leq \pi, \quad 0 \leq \phi < 2\pi$$

This gives the **Bloch sphere representation**, every pure state of a single qubit corresponds to a point on the surface of a unit sphere.

The corresponding Cartesian coordinates are:

$$x = \sin \theta \cos \phi, \quad y = \sin \theta \sin \phi, \quad z = \cos \theta$$

And important states are therefore located at:

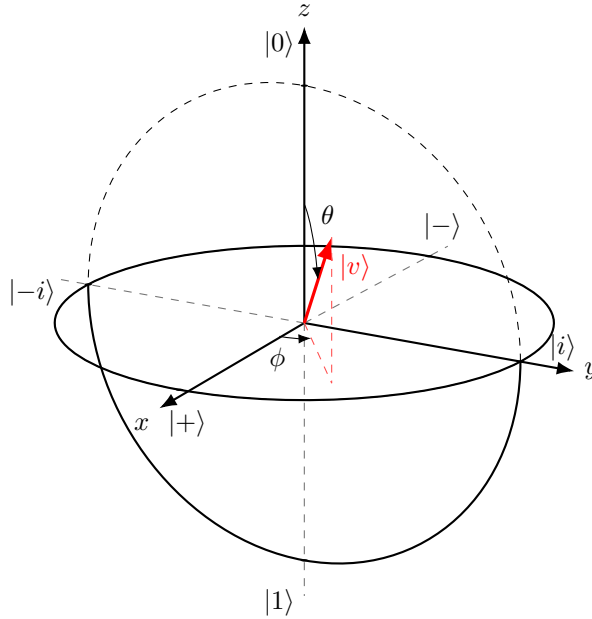
- North pole. The state $|0\rangle$ is located at the north pole of the Bloch sphere, where $z = 1$:

$$|0\rangle = \cos \frac{0}{2} |0\rangle + e^{i\phi} \sin \frac{0}{2} |1\rangle = 1 |0\rangle + 0 |1\rangle$$

💬 **Remark**

Since $|0\rangle$ is the state:

$$|0\rangle = 1 |0\rangle + 0 |1\rangle$$



Reference to the Bloch sphere: page 40

The corresponding Bloch sphere coordinates must satisfy:

$$\cos \frac{\theta}{2} = 1, \quad e^{i\phi} \sin \frac{\theta}{2} = 0$$

The cos term is 1 only if $\theta = 0$, and the sin term is 0 when $\theta = 0$. Thus, the coordinates of $|0\rangle$ are:

$$x = 0, \quad y = 0, \quad z = 1$$

- South pole. The state $|1\rangle$ is located at the south pole of the Bloch sphere, where $z = -1$:

$$|1\rangle = \cos \frac{\pi}{2} |0\rangle + e^{i\phi} \sin \frac{\pi}{2} |1\rangle = 0 |0\rangle + e^{i\phi} 1 |1\rangle$$

💬 Remark

Since $|1\rangle$ is the state:

$$|1\rangle = 0 |0\rangle + 1 |1\rangle$$

The corresponding Bloch sphere coordinates must satisfy:

$$\cos \frac{\theta}{2} = 0, \quad e^{i\phi} \sin \frac{\theta}{2} = 1$$

The cos term is 0 only if $\theta = \pi$, and the sin term is 1 when $\theta = \pi$. Thus, the coordinates of $|1\rangle$ are:

$$x = 0, \quad y = 0, \quad z = -1$$

- Equatorial plane. The states $|+\rangle$, $|-\rangle$, $|i\rangle$, and $|-i\rangle$ are all located on the equatorial plane of the Bloch sphere, where $z = 0$.

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \Rightarrow x = 1, y = 0, z = 0$$

$$|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \Rightarrow x = -1, y = 0, z = 0$$

$$|i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle) \Rightarrow x = 0, y = 1, z = 0$$

$$|-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle) \Rightarrow x = 0, y = -1, z = 0$$

📌 Key Takeaways 1

A useful pattern to remember is:

State	(θ, ϕ)	Bloch direction
$ 0\rangle$	$(0, -)$	$+z$
$ 1\rangle$	$(\pi, -)$	$-z$
$ +\rangle$	$(\pi/2, 0)$	$+x$
$ -\rangle$	$(\pi/2, \pi)$	$-x$
$ i\rangle$	$(\pi/2, \pi/2)$	$+y$
$ -i\rangle$	$(\pi/2, -\pi/2)$	$-y$

There is also a very useful connection with the Pauli matrices:

$$Z : |0\rangle (+1), \quad |1\rangle (-1),$$

$$X : |+\rangle (+1), \quad |-\rangle (-1),$$

$$Y : |i\rangle (+1), \quad |-i\rangle (-1)$$

So another excellent way to remember the Bloch sphere is:

$$Z \leftrightarrow \{|0\rangle, |1\rangle\}, \quad X \leftrightarrow \{|+\rangle, |-\rangle\}, \quad Y \leftrightarrow \{|i\rangle, |-i\rangle\}$$

The $+1$ eigenstate points along the positive axis and the -1 eigenstate along the negative axis.

✔ **Solution 2 (Mathematical formalism of quantum gates).** Since a qubit is represented by a vector in $\mathbb{C}^2$, a single-qubit quantum gate is represented by a 2×2 complex matrix U . It transforms $|\psi\rangle$ into:

$$|\psi'\rangle = U|\psi\rangle$$

But an arbitrary 2×2 matrix cannot represent a quantum gate. The matrix must be **unitary**:

$$U^\dagger U = U U^\dagger = I$$

Or equivalently:

$$U^{-1} = U^\dagger$$

Unitary is a necessary condition for a quantum gate because it **preserves orthonormality** and **total probability**, gives **deterministic reversible evolution**, and therefore does **not lose information**.

☺ **Remark: Why must a quantum gate be unitary?**

First: probabilities must remain probabilities. Suppose:

$$|\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

Its norm is:

$$\langle\psi|\psi\rangle = |\alpha|^2 + |\beta|^2 = 1$$

This is fundamental because these are the measurement probabilities:

$$P(0) = |\alpha|^2, \quad P(1) = |\beta|^2, \quad \Rightarrow \quad P(0) + P(1) = 1$$

Now apply a quantum gate U to $|\psi\rangle$:

$$|\psi'\rangle = U |\psi\rangle$$

We need the new state to remain normalized:

$$\langle\psi'|\psi'\rangle = 1$$

But:

$$\langle\psi'|\psi'\rangle = (\langle U|\psi\rangle)^\dagger \langle U|\psi\rangle = \langle\psi|U^\dagger U|\psi\rangle$$

Therefore, if:

$$U^\dagger U = I$$

Then:

$$\langle\psi'|\psi'\rangle = \langle\psi|\psi\rangle = 1$$

And that's exactly the definition of a **unitary** matrix:

$$U^\dagger U = U U^\dagger = I$$

So unitary guarantees that **total probability remains 1**.

Unitary also preserves inner products. Take two states: $|\psi\rangle$ and $|\phi\rangle$. After applying U :

$$|\psi'\rangle = U |\psi\rangle, \quad |\phi'\rangle = U |\phi\rangle$$

Their new inner product is:

$$\langle\psi'|\phi'\rangle = \langle\psi|U^\dagger U|\phi\rangle = \langle\psi|\phi\rangle$$

So:

$$\langle U\psi|U\phi\rangle = \langle\psi|\phi\rangle$$

This means that angles and orthogonality between quantum states are preserved. For example, initially

$$\langle 0|1\rangle = 0$$

After any unitary U :

$$\langle U0|U1\rangle = 0$$

So two orthogonal states cannot suddenly become non-orthogonal under a quantum gate.

💬 Remark: How the dagger behaves when we take the adjoint of a matrix-vector product

We start with:

$$|\psi'\rangle = U |\psi\rangle$$

We want the norm of the new state:

$$\langle\psi'|\psi'\rangle$$

From ket to a bra. If:

$$|\psi'\rangle = U |\psi\rangle$$

Then its bra is obtained by taking the Hermitian conjugate:

$$\langle\psi'| = (|\psi'\rangle)^\dagger = (U |\psi\rangle)^\dagger$$

Now recall the fundamental rule:

$$(AB)^\dagger = B^\dagger A^\dagger$$

So taking the dagger reverses the order. Therefore:

$$(U |\psi\rangle)^\dagger = |\psi\rangle^\dagger U^\dagger = \langle\psi| U^\dagger$$

Hence:

$$(U |\psi\rangle)^\dagger = \langle\psi| U^\dagger, \quad \text{and} \quad \langle\psi'| = \langle\psi| U^\dagger$$

Compute the inner product. We have:

$$\langle\psi'|\psi'\rangle$$

Substitute:

$$|\psi'\rangle = U |\psi\rangle$$

And:

$$\langle\psi'| = \langle\psi| U^\dagger \Rightarrow \langle\psi'|\psi'\rangle = (\langle\psi| U^\dagger) U |\psi\rangle = \langle\psi| U^\dagger U |\psi\rangle$$

Then because U is unitary:

$$\langle\psi'|\psi'\rangle = \langle\psi| U^\dagger U |\psi\rangle = \langle\psi| I |\psi\rangle = \langle\psi|\psi\rangle$$

Thus, the norm of the new state is equal to the norm of the original state, and total probability is preserved.

Unitary evolution is reversible. From:

$$U^\dagger U = I$$

We immediately get:

$$U^{-1} = U^\dagger$$

So if:

$$|\psi'\rangle = U |\psi\rangle$$

We can always recover the original state:

$$U^\dagger |\psi'\rangle = U^\dagger U |\psi\rangle = |\psi\rangle$$

For example, the X gate does:

$$|0\rangle \xrightarrow{X} |1\rangle$$

But:

$$X^{-1} = X$$

So:

$$|1\rangle \xrightarrow{X} |0\rangle$$

No information about the input is destroyed.

Why “deterministic” evolution? This wording can initially be confusing because **quantum measurement is probabilistic**. But a quantum **gate itself** is not probabilistic. If we know: $|\psi\rangle$ and apply a known gate U , the resulting quantum state is exactly:

$$|\psi'\rangle = U |\psi\rangle$$

There’s no randomness in that transformation. The randomness appears when we **measure** $|\psi'\rangle$.

Why “does not lose information”? Because a unitary transformation is one-to-one and invertible. Imagine a hypothetical operation:

$$|0\rangle \rightarrow |0\rangle, \quad |1\rangle \rightarrow |0\rangle$$

If afterward someone gives us $|0\rangle$, can we determine whether the original state was $|0\rangle$ or $|1\rangle$? No! The information has been lost:

$$|0\rangle, |1\rangle \rightarrow |0\rangle$$

Such an operation cannot be unitary. **A unitary gate instead cannot collapse two distinct orthogonal inputs into the same output, because it preserves their inner product.**

Show that σ_X is a valid quantum gate. The Pauli-X matrix is:

$$\sigma_X = X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

Its conjugate transpose is:

$$\sigma_X^\dagger = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = X$$

Then:

$$X^\dagger X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

Therefore, σ_X is unitary and is a valid quantum gate.

Show that σ_Y is a valid quantum gate. The Pauli-Y matrix is:

$$\sigma_Y = Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

Here we need to pay attention because the entries are complex. First of all, the conjugate is:

$$Y^* = \begin{pmatrix} 0 & i \\ -i & 0 \end{pmatrix}$$

Then the conjugate transpose is:

$$Y^\dagger = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = Y$$

Thus Y is Hermitian (i.e., $Y^\dagger = Y$). Now we compute:

$$Y^\dagger Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} = \begin{pmatrix} (-i)(i) & 0 \\ 0 & (i)(-i) \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

Therefore, σ_Y is unitary and is a valid quantum gate.

✔ **Solution 2 (No-cloning and no-deleting).** These two principles express fundamental restrictions on quantum information and are closely connected to the linear/unitary nature of quantum evolution.

- **No-cloning theorem.** The no-cloning theorem states that **it is impossible to perfectly copy an arbitrary unknown quantum state.** In other words, there is no universal unitary U such that:

$$U(|\psi\rangle|0\rangle) = |\psi\rangle|\psi\rangle$$

For **every** possible input state $|\psi\rangle$.

Proof. Suppose a cloning machine existed and could clone both $|\psi\rangle$ and $|\phi\rangle$:

$$U|\psi, 0\rangle = |\psi, \psi\rangle, \quad U|\phi, 0\rangle = |\phi, \phi\rangle$$

Unitary transformations preserve inner products, so:

$$\langle \psi, 0 | \phi, 0 \rangle = \langle \psi, \psi | \phi, \phi \rangle$$

The left-hand side is:

$$\langle \psi, 0 | \phi, 0 \rangle = \langle \psi | \phi \rangle \langle 0 | 0 \rangle = \langle \psi | \phi \rangle$$

The right-hand side is:

$$\langle \psi, \psi | \phi, \phi \rangle = \langle \psi | \phi \rangle \langle \psi | \phi \rangle = \langle \psi | \phi \rangle^2$$

Therefore:

$$\langle \psi | \phi \rangle = \langle \psi | \phi \rangle^2$$

This equation is satisfied only if:

$$\langle \psi | \phi \rangle = 0 \quad \text{or} \quad \langle \psi | \phi \rangle = 1$$

Which means that $|\psi\rangle$ and $|\phi\rangle$ must be either orthogonal or identical. But this is a **contradiction** because we assumed that $|\psi\rangle$ and $|\phi\rangle$ were **arbitrary unknown states**. Therefore, a universal cloning machine cannot exist. QED

Note: this does not mean that $|0\rangle$ and $|1\rangle$ cannot be copied. A CNOT gate can copy computational-basis states. What is impossible is a universal operation that clones an **arbitrary unknown superposition**

- **No-deleting principle.** The no-deleting principle is essentially the complementary restriction: **it is impossible to delete one copy of an arbitrary unknown quantum state using a quantum gate.** That is, there cannot be a universal unitary operation satisfying

$$U(|\psi\rangle |\psi\rangle) = |\psi\rangle |0\rangle$$

For every arbitrary input state $|\psi\rangle$.

Proof. Assume, by contradiction, that there exists a universal unitary deleting operator U such that, for every quantum state $|\psi\rangle$:

$$U(|\psi\rangle |\psi\rangle) = |\psi\rangle |0\rangle$$

Where $|0\rangle$ is a fixed blank state. Since the machine is universal, it must work for any two states $|\psi\rangle$ and $|\phi\rangle$:

$$U|\psi\rangle |\psi\rangle = |\psi\rangle |0\rangle$$

$$U|\phi\rangle |\phi\rangle = |\phi\rangle |0\rangle$$

Because U is unitary, it preserves inner products. Therefore:

$$\langle \psi | \phi \rangle^2 = \langle \psi | \phi \rangle \langle 0 | 0 \rangle$$

Since:

$$\langle 0 | 0 \rangle = 1$$

We obtain:

$$\langle\psi|\phi\rangle^2 = \langle\psi|\phi\rangle$$

But this equation is satisfied only if:

$$\langle\psi|\phi\rangle = 0 \quad \text{or} \quad \langle\psi|\phi\rangle = 1$$

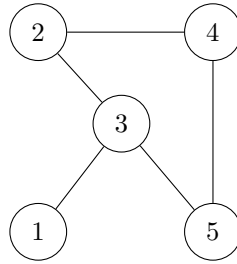
Which means that $|\psi\rangle$ and $|\phi\rangle$ must be either orthogonal or identical. But this is a contradiction because we assumed that $|\psi\rangle$ and $|\phi\rangle$ were arbitrary unknown states. Therefore, a universal deleting machine cannot exist. QED

❓ Exercise 3 (QUBO).

- Compute the QUBO formulation of the following SAT problem. Show its coefficient matrix in binary variables and spin variables, compute the Problem Hamiltonian and use it to identify which is the best solution.

$$\bar{x}_1 \vee x_2$$

- Compute the QUBO formulation of the Max-Cut problem related to this graph. Show its coefficient matrix in binary variables and spin variables. Draw the circuit required to solve this problem with QAOA using 1 layer.



Motivate your answer by showing all stages of the computation.

✔ Solution 3 (QUBO formulation of SAT problem).

📖 Deepening: QUBO formulation of SAT problem

QUBO stands for Quadratic Unconstrained Binary Optimization. It is a mathematical formulation used to express optimization problems in terms of binary variables:

$$\arg \min_x E(x), \quad x_i \in \{0, 1\}$$

Where $E(x)$ must be at most quadratic (i.e., it can contain terms like x_i , $x_i x_j$, but not higher-order terms like $x_i x_j x_k$). For Boolean problems, the strategy is simple: assign the **lowest energy to the Boolean assignments we want**, and a higher energy to the assignments we do not want. Then, minimizing the energy will give us the desired solution.

Step 1: From SAT to a minimization problem. We define the energy function as (page 465):

$$E(x_1, x_2) = x_1 - x_1 x_2$$

And then the QUBO problem is:

$$\min_{x_1, x_2} E(x_1, x_2) = \min_{x_1, x_2} (x_1 - x_1 x_2)$$

The three satisfying assignments (0, 0), (0, 1), and (1, 1) have energy 0, while the violating assignment has energy 1. Therefore, minimizing $E(x_1, x_2)$ will give us the satisfying assignments.

Deepening: From SAT to a minimization problem

Our original problem is not yet an optimization problem, it is simply the SAT clause:

$$\bar{x}_1 \vee x_2$$

SAT^a is a decision problem (page 465), which asks: “*which assignments of x_1 and x_2 make this logical expression **true**?*”. So before QUBO, we first understand the Boolean function itself. For OR:

$$A \vee B$$

It is false only when **both operands are false**. In our case the operands are:

$$A = \bar{x}_1, \quad B = x_2$$

Therefore the clause is false only when:

$$\bar{x}_1 = 0, \quad x_2 = 0$$

But:

$$\bar{x}_1 = 0 \iff x_1 = 1$$

So the only assignment that makes the clause false is:

$$x_1 = 1, \quad x_2 = 0$$

All other assignments make the clause true:

$$\text{SAT solutions} = \{(0, 0), (0, 1), (1, 1)\}$$

This is the first thing we want QUBO to reproduce.

Since QUBO does **not** directly understand logical expressions such as:

$$\bar{x}_1 \vee x_2$$

We need to translate “*make the clause true*” into “*minimize some numerical function*”. As we said, the strategy is to assign the lowest energy to the Boolean assignments we want, and a higher energy to the assignments we do not want. So for our clause we want to manufacture a polynomial with this table:

x_1	x_2	clause	desired $E(x)$
0	0	true	$E(0, 0) = 0$
0	1	true	$E(0, 1) = 0$
1	0	false	$E(1, 0) = 1$
1	1	true	$E(1, 1) = 0$

In other words, we want a mathematical function that behaves like an **indicator for the bad state** (1, 0).

^aDuring the exams, the students are allowed to bring a copy of the most common QUBO problems (page 462), including the SAT problem.

Step 2: Binary QUBO coefficient matrix. We can rewrite our desired objective without changing its meaning:

$$E(x) = x_1^2 - x_1x_2$$

We can immediately read off the coefficients:

$$1x_1^2 - 1x_1x_2 + 0x_2^2$$

$$Q_{11} = 1, \quad Q_{12} + Q_{21} = -1, \quad Q_{22} = 0$$

Hence, the **QUBO coefficient matrix** is:

$$Q = \begin{pmatrix} 1 & -1 \\ 0 & 0 \end{pmatrix}$$

Deepening: QUBO coefficient matrix

Because a QUBO objective is represented in matrix form as:

$$E(x) = x^T Q x$$

Where here:

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad E(x) = x_1 - x_1x_2$$

So our task is simply to find a matrix Q such that multiplying $x^T Q x$ reproduces exactly $x_1 - x_1x_2$.

Naturally, the matrix multiplication produces:

$$\begin{aligned} x^T Q x &= (x_1 \quad x_2) \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \\ &Q_{11}x_1^2 + (Q_{12} + Q_{21})x_1x_2 + Q_{22}x_2^2 \end{aligned}$$

And we can notice something important: **the diagonal entries of Q naturally multiply squared variables.** Thus Q_{11} multiplies x_1^2 , and Q_{22} multiplies x_2^2 . But our objective contains x_1 and x_2 only to the first power.

There's one little trick. Since x_1 and x_2 are binary variables, we can use the property:

$$x_i^2 = x_i, \quad \text{for } x_i \in \{0, 1\}$$

So we can freely replace x_1^2 with x_1 , and x_2^2 with x_2 .

Deepening: (optional) Step 2b, check the binary energy

Before continuing to spin variables, we can evaluate $E(x)$ for **all possible binary assignments** and verify that the minimum-energy assignments are exactly the solution of the original SAT problem:

- $x_1 = 0, x_2 = 0$: $E(0, 0) = 0 - 0 \cdot 0 = 0$
- $x_1 = 0, x_2 = 1$: $E(0, 1) = 0 - 0 \cdot 1 = 0$
- $x_1 = 1, x_2 = 0$: $E(1, 0) = 1 - 1 \cdot 0 = 1$
- $x_1 = 1, x_2 = 1$: $E(1, 1) = 1 - 1 \cdot 1 = 0$

The minimum energy is 0, and the corresponding assignments are:

$$(x_1, x_2) \in \{(0, 0), (0, 1), (1, 1)\}$$

Which are exactly the satisfying assignments of the original SAT clause.

Step 3: Convert binary variables to spin variables. The Ising/spin energy function is:

$$\begin{aligned}
 E(s_1, s_2) &= \frac{1 - s_1}{2} - \frac{1 - s_1}{2} \cdot \frac{1 - s_2}{2} \\
 &= \frac{1 - s_1}{2} - \frac{(1 - s_1)(1 - s_2)}{4} \\
 &= \frac{1 - s_1}{2} - \frac{1 - s_1 - s_2 + s_1 s_2}{4} \\
 &= \frac{2 - 2s_1 - 1 + s_1 + s_2 - s_1 s_2}{4} \\
 &= \frac{1 - s_1 + s_2 - s_1 s_2}{4} \\
 &= \boxed{\frac{1}{4} - \frac{s_1}{4} + \frac{s_2}{4} - \frac{s_1 s_2}{4}}
 \end{aligned}$$

Deepening: From binary to spin variables

Our QUBO currently uses binary variables:

$$x_i \in \{0, 1\}$$

But the Ising formulation uses spins with values:

$$s_i \in \{-1, +1\}$$

And this is particularly useful for quantum computing because the Pauli- Z operator has eigenvalues ± 1 :

$$Z|0\rangle = +1|0\rangle, \quad Z|1\rangle = -1|1\rangle$$

So we have a natural chain:

$$x_i \in \{0, 1\} \quad \rightarrow \quad s_i \in \{-1, +1\} \quad \rightarrow \quad Z_i$$

How do we map x_i to s_i ? We use the formula:

$$s_i = 1 - 2x_i \quad \Rightarrow \quad x_i = \frac{1 - s_i}{2}$$

Now we apply it to our energy function. We have:

$$E(x_1, x_2) = x_1 - x_1 x_2$$

Substitute:

$$x_1 = \frac{1 - s_1}{2}, \quad x_2 = \frac{1 - s_2}{2}$$

And the remaining steps are just algebraic simplifications, which we show in the main text.

Step 4: Spin QUBO coefficient matrix. We have reached the spin energy function, and now the exam asks for the **coefficient matrix in spin variables**. The idea is essentially the same as what we did for the binary QUBO matrix Q : we want to take the coefficients appearing in the polynomial and organize them into a matrix. We can do this in four steps:

1. **Separate the constant term.** Our expression contains:

$$E(s_1, s_2) = \underbrace{\frac{1}{4}}_{\text{constant}} - \frac{s_1}{4} + \frac{s_2}{4} - \frac{s_1 s_2}{4}$$

It does **not** go into the spin coefficient matrix, because it is not multiplied by any spin variable. It **shifts every possible energy by exactly the same amount, so it cannot change which assignment minimizes the problem**. Thus, for constructing the spin coefficient matrix, we can ignore it:

$$-\frac{s_1}{4} + \frac{s_2}{4} - \frac{s_1 s_2}{4}$$

2. **Identify the coefficients.** The remaining terms are:

$$\begin{aligned} s_1 \text{ with coefficient } -\frac{1}{4}, \quad s_2 \text{ with coefficient } +\frac{1}{4}, \\ s_1 s_2 \text{ with coefficient } -\frac{1}{4} \end{aligned}$$

3. **Construct the matrix.** The spin coefficient matrix is defined as:

- Linear coefficient of s_1 goes into the $(1, 1)$ entry.
- Linear coefficient of s_2 goes into the $(2, 2)$ entry.
- Coefficient of $s_1 s_2$ goes into the $(1, 2)$ entry. The $(2, 1)$ entry is usually set to 0, because the matrix is upper-triangular.

Therefore:

$$S = \begin{pmatrix} -\frac{1}{4} & -\frac{1}{4} \\ 0 & \frac{1}{4} \end{pmatrix} = \begin{pmatrix} -0.25 & -0.25 \\ 0 & 0.25 \end{pmatrix}$$

Step 5: Construct the Problem Hamiltonian. The Problem Hamiltonian is simply:

$$H_{\text{problem}} = -\frac{1}{4}Z_1 + \frac{1}{4}Z_2 - \frac{1}{4}Z_1 Z_2$$

Deepening: From spin energy function to Problem Hamiltonian

We currently have the spin energy function, and now the goal is to build the **Problem Hamiltonian** used by QAOA. The idea is that each classical spin variable:

$$s_i \in \{-1, +1\}$$

Is replaced by a Pauli-Z operator acting on qubit i :

$$s_i \rightarrow Z_i$$

It makes sense because the Pauli-Z matrix has eigenvalues ± 1 :

$$Z|0\rangle = +1|0\rangle, \quad Z|1\rangle = -1|1\rangle$$

Thus, the constant term in the spin energy function is ignored, and the remaining terms are replaced by Pauli-Z operators:

$$H_{\text{problem}} = \frac{1}{4}I - \frac{1}{4}Z_1 + \frac{1}{4}Z_2 - \frac{1}{4}Z_1Z_2$$

Where I is the identity operator. However, the constant term does not affect the optimization, because it shifts all energies by the same amount. Therefore, we can drop it.

Step 6: Compute the Hamiltonian explicitly. Let's calculate the diagonal entries of H_{problem} by evaluating each term on the computational-basis states:

- For $|00\rangle$, the first qubit is 0, so Z_1 contributes eigenvalue $+1$. Same for the second qubit, so Z_2 contributes eigenvalue $+1$. Therefore the energy of $|00\rangle$ is:

$$E_{00} = -\frac{1}{4} \underbrace{(+1)}_{Z_1} + \frac{1}{4} \underbrace{(+1)}_{Z_2} - \frac{1}{4} \underbrace{(+1)(+1)}_{Z_1Z_2} = -\frac{1}{4} + \frac{1}{4} - \frac{1}{4} = -\frac{1}{4}$$

- For $|01\rangle$, the first qubit is 0, so Z_1 contributes eigenvalue $+1$. The second qubit is 1, so Z_2 contributes eigenvalue -1 . Therefore the energy of $|01\rangle$ is:

$$E_{01} = -\frac{1}{4} \underbrace{(+1)}_{Z_1} + \frac{1}{4} \underbrace{(-1)}_{Z_2} - \frac{1}{4} \underbrace{(+1)(-1)}_{Z_1Z_2} = -\frac{1}{4} - \frac{1}{4} + \frac{1}{4} = -\frac{1}{4}$$

- For $|10\rangle$, the first qubit is 1, so Z_1 contributes eigenvalue -1 . The second qubit is 0, so Z_2 contributes eigenvalue $+1$. Therefore the energy of $|10\rangle$ is:

$$E_{10} = -\frac{1}{4} \underbrace{(-1)}_{Z_1} + \frac{1}{4} \underbrace{(+1)}_{Z_2} - \frac{1}{4} \underbrace{(-1)(+1)}_{Z_1Z_2} = \frac{1}{4} + \frac{1}{4} - \frac{1}{4} = \frac{1}{4}$$

- For $|11\rangle$, the first qubit is 1, so Z_1 contributes eigenvalue -1 . Same for the second qubit, so Z_2 contributes eigenvalue -1 . Therefore the energy

of $|11\rangle$ is:

$$E_{11} = -\frac{1}{4} \underbrace{(-1)}_{Z_1} + \frac{1}{4} \underbrace{(-1)}_{Z_2} - \frac{1}{4} \underbrace{(-1)(-1)}_{Z_1 Z_2} = \frac{1}{4} - \frac{1}{4} - \frac{1}{4} = -\frac{1}{4}$$

We have:

$$\text{diag}(H_{\text{problem}}) = \left(-\frac{1}{4}, -\frac{1}{4}, \frac{3}{4}, -\frac{1}{4} \right)$$

Therefore, the eigenvalues of H_{problem} are:

$$\begin{aligned} E(0,0) = |00\rangle &= -\frac{1}{4}, & E(0,1) = |01\rangle &= -\frac{1}{4}, \\ E(1,0) = |10\rangle &= \frac{3}{4}, & E(1,1) = |11\rangle &= -\frac{1}{4} \end{aligned}$$

The problem was defined as a minimization, so the diagonal entry with the lowest energy corresponds to the best solution.

Deepening: From Problem Hamiltonian to best solution

Imagine that we had $x_1, \dots, x_{100}$, there would be 2^{100} possible bitstrings. We don't want our quantum algorithm to be given a truth table containing all 2^{100} answers (because that would be exponential in size). Instead, we need to give the quantum computer a mathematical object that **encodes how good every possible solution is**. That object will be the **Problem Hamiltonian**.

A quantum computer works with **quantum states and operators**. Our candidate classical solution:

$$x_1 x_2 = 10$$

Can be represented by the quantum computational-basis state $|10\rangle$, likewise:

$$00 \leftrightarrow |00\rangle, \quad 01 \leftrightarrow |01\rangle, \quad 11 \leftrightarrow |11\rangle$$

So representing the **candidate solutions** quantum mechanically is easy. But we still have another problem: *how do we represent the classical cost function $E(x)$ quantum mechanically?* This is where the Problem Hamiltonian comes in.

We construct H_{problem} so that candidate bitstrings become its eigenstates and their eigenvalues encode their costs. In other words, we want:

$$H_{\text{problem}} |x_1 x_2\rangle = E(x_1, x_2) |x_1 x_2\rangle$$

So:

- ✓ If candidate $x_1 x_2$ is **good**, its corresponding quantum state should have **low energy**.
- ✗ If candidate $x_1 x_2$ is **bad**, its corresponding state should have **higher energy**.

Therefore, best classical solutions correspond to the lowest-energy (ground) quantum state. The lowest-energy eigenstate of a Hamiltonian is called its **ground state**. So we have transformed find the minimum of $E(x)$ into find the ground state of H_{problem} .

❓ Why did we go through spins? Because our classical variables are:

$$x_i \in \{0, 1\}$$

Whereas the Pauli-Z has eigenvalues ± 1 . So spins are a convenient intermediate step:

$$x_i \in \{0, 1\} \rightarrow s_i \in \{-1, +1\} \rightarrow Z_i$$

Once we have the spin energy function, we can simply replace s_i with Z_i to get the Problem Hamiltonian.

Now, once we have the Problem Hamiltonian, we want to see **what energy this Hamiltonian assigns to each candidate state**. Because H_{problem} contains only Z -type operators, it is diagonal in the computational basis:

$$H_{\text{problem}} = \begin{pmatrix} E(0,0) & 0 & 0 & 0 \\ 0 & E(0,1) & 0 & 0 \\ 0 & 0 & E(1,0) & 0 \\ 0 & 0 & 0 & E(1,1) \end{pmatrix}$$

So we can compute the energies of all candidate states by simply evaluating H_{problem} on each computational-basis state. *Why?* Because the Hamiltonian itself is a **sum**:

$$H = A + B + C$$

If all three matrices are diagonal:

$$A = \begin{pmatrix} a_1 & 0 \\ 0 & a_2 \end{pmatrix}, \quad B = \begin{pmatrix} b_1 & 0 \\ 0 & b_2 \end{pmatrix}, \quad C = \begin{pmatrix} c_1 & 0 \\ 0 & c_2 \end{pmatrix}$$

The ordinary matrix addition gives:

$$H = A + B + C = \begin{pmatrix} a_1 + b_1 + c_1 & 0 \\ 0 & a_2 + b_2 + c_2 \end{pmatrix}$$

Therefore:

$$\text{diag}(H) = (a_1 + b_1 + c_1, a_2 + b_2 + c_2)$$

Step 7: Identify the best solution. The minimum eigenvalue of H_{problem} is:

$$\min \left\{ -\frac{1}{4}, -\frac{1}{4}, \frac{3}{4}, -\frac{1}{4} \right\} = -\frac{1}{4}$$

And the corresponding eigenstates are:

$$|00\rangle, \quad |01\rangle, \quad |11\rangle$$

Thus there are **three optimal bitstrings** that minimize the energy, which correspond to the three satisfying assignments of the original SAT clause:

$$(x_1, x_2) \in \{(0, 0), (0, 1), (1, 1)\}$$

The only non-optimal assignment is $(1, 0)$, which has energy $E(1, 0) = \frac{3}{4}$.

✔ **Solution 3 (QUBO formulation of Max-Cut problem).** The graph has 5 vertices:

$$V = \{1, 2, 3, 4, 5\}$$

And 5 edges:

$$E = \{(2, 4), (2, 3), (3, 1), (3, 5), (4, 5)\}$$

Deepening: What does Max-Cut mean?

We want to partition the vertices into two sets so that the **largest possible number of edges have their endpoints in different sets**. In other words, we want to maximize the number of edges that are “cut” by the partition.

We associate one binary variable to each vertex:

$$x_i = \begin{cases} 0, & \text{if vertex } i \text{ belongs to one partition} \\ 1, & \text{if vertex } i \text{ belongs to the other partition} \end{cases}$$

For an edge (i, j) , we want to know whether i and j are on different sides.

The formula to use is (page 463):

$$x_i + x_j - 2x_i x_j$$

Because:

x_i	x_j	$x_i + x_j - 2x_i x_j$	Edge cut?
0	0	0	No
0	1	1	Yes
1	0	1	Yes
1	1	0	No

So the formula is exactly an **indicator for whether and edge is cut**.

Step 1: From Max-Cut to a minimization problem. The number of cut edges is:

$$C(x) = \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j)$$

But QUBO is expressed as a minimization problem. Therefore (page 463):

$$\min_x E(x) = \min_x -C(x) = \min_x - \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j)$$

💡 **Tip:** This expression is **always the same** for any graph.

Step 2: Apply the formula to our graph. We have 5 edges:

$$E = \{(2, 4), (2, 3), (3, 1), (3, 5), (4, 5)\}$$

So for each edge we apply the formula $x_i + x_j - 2x_i x_j$, where (i, j) are the endpoints of the edge. The energy function is:

$$\begin{aligned} E(x) &= -[(x_2 + x_4 - 2x_2 x_4) + \\ &\quad (x_2 + x_3 - 2x_2 x_3) + \\ &\quad (x_3 + x_1 - 2x_3 x_1) + \\ &\quad (x_3 + x_5 - 2x_3 x_5) + \\ &\quad (x_4 + x_5 - 2x_4 x_5)] \\ &= -x_2 - x_4 + 2x_2 x_4 \\ &\quad - x_2 - x_3 + 2x_2 x_3 \\ &\quad - x_3 - x_1 + 2x_3 x_1 \\ &\quad - x_3 - x_5 + 2x_3 x_5 \\ &\quad - x_4 - x_5 + 2x_4 x_5 \end{aligned}$$

Now we can collect the linear terms. Since the equation is huge, we will do it with a trick. First, we count how many times each variable appears in the linear terms:

$$x_1 : 1, \quad x_2 : 2, \quad x_3 : 3, \quad x_4 : 2, \quad x_5 : 2$$

Thus, the linear part is:

$$-x_1 - 2x_2 - 3x_3 - 2x_4 - 2x_5$$

The minus sign is because all linear terms are negative (see original expression). Second, we collect the quadratic terms:

$$2x_1 x_3 + 2x_2 x_3 + 2x_2 x_4 + 2x_3 x_5 + 2x_4 x_5$$

So the complete **QUBO energy function** (QUBO formulation) is:

$$\boxed{\begin{aligned} \min_x E(x) &= -x_1 - 2x_2 - 3x_3 - 2x_4 - 2x_5 + 2x_1 x_3 + \\ &\quad 2x_2 x_3 + 2x_2 x_4 + 2x_3 x_5 + 2x_4 x_5 \end{aligned}}$$

Step 3: Binary QUBO coefficient matrix. We want:

$$E(x) = x^T Q x$$

Thus, the diagonal contains the linear coefficients:

$$Q_{11} = -1, \quad Q_{22} = -2, \quad Q_{33} = -3, \quad Q_{44} = -2, \quad Q_{55} = -2$$

And the upper-triangular part contains the quadratic coefficients:

$$Q_{13} = 2, \quad Q_{23} = 2, \quad Q_{24} = 2, \quad Q_{35} = 2, \quad Q_{45} = 2$$

Therefore the **QUBO coefficient matrix in binary variables** is:

$$Q = \begin{pmatrix} -1 & 0 & 2 & 0 & 0 \\ 0 & -2 & 2 & 2 & 0 \\ 0 & 0 & -3 & 0 & 2 \\ 0 & 0 & 0 & -2 & 2 \\ 0 & 0 & 0 & 0 & -2 \end{pmatrix}$$

Deepening: QUBO coefficient matrix

Since we want to write the energy function in matrix form:

$$E(x) = x^T Q x$$

With:

$$x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix}$$

We need to find the 5×5 matrix Q such that multiplying $x^T Q x$ reproduces exactly the energy function we derived in Step 2. The diagonal entries of Q naturally multiply squared variables, so they correspond to the **linear terms**. So for example $-3x_3$ corresponds to $Q_{33} = -3$.

For **quadratic terms**, we use the rule:

$$cx_i x_j \Rightarrow Q_{ij} = c, \quad i < j$$

For example, $2x_1 x_3$ corresponds to $Q_{13} = 2$. The (j, i) entry is usually set to 0, because the matrix is upper-triangular.

In general, the memory rule is:

$$E(x) = \sum_i a_i x_i + \sum_{i < j} b_{ij} x_i x_j$$

Corresponds to the QUBO coefficient matrix:

$$Q_{ii} = a_i, \quad Q_{ij} = b_{ij}, \quad i < j$$

 For Max-Cut there is an even **faster shortcut**. For an unweighted graph:

$$Q_{ii} = -d_i$$

Where d_i is the degree of vertex i , while:

$$Q_{ij} = 2, \quad \text{if } (i, j) \in E \quad \text{otherwise } 0$$

Step 4: Convert to spin variables. We use the same formula as before:

$$s_i = 1 - 2x_i \Rightarrow x_i = \frac{1 - s_i}{2}$$

With $s_i \in \{-1, +1\}$. For one Max-Cut edge (i, j) , the cut indicator is (we saw this in Step 2):

$$x_i + x_j - 2x_i x_j$$

Substitute:

$$x_i = \frac{1 - s_i}{2}, \quad x_j = \frac{1 - s_j}{2}$$

Then:

$$x_i + x_j = \frac{1 - s_i}{2} + \frac{1 - s_j}{2} = 1 - \frac{s_i + s_j}{2}$$

And:

$$\begin{aligned} 2x_i x_j &= 2 \cdot \frac{1 - s_i}{2} \cdot \frac{1 - s_j}{2} \\ &= \frac{(1 - s_i)(1 - s_j)}{2} \\ &= \frac{1 - s_i - s_j + s_i s_j}{2} \end{aligned}$$

Now we can combine the two parts:

$$\begin{aligned} x_i + x_j - 2x_i x_j &= 1 - \frac{s_i + s_j}{2} - \frac{1 - s_i - s_j + s_i s_j}{2} \\ &= 1 - \frac{s_i + s_j}{2} - \frac{1}{2} + \frac{s_i + s_j}{2} - \frac{s_i s_j}{2} \\ &= \frac{1}{2} - \frac{s_i s_j}{2} \end{aligned}$$

So the cut indicator for one edge (i, j) in spin variables is:

$$x_i + x_j - 2x_i x_j = \frac{1}{2} - \frac{s_i s_j}{2}$$

Since our QUBO minimizes the **negative** of the cut indicator, we have:

$$\boxed{-(x_i + x_j - 2x_i x_j) = -\frac{1}{2} + \frac{s_i s_j}{2}}$$

💡 This part is **always the same** for any graph, because it is derived from the cut indicator formula, which is universal for Max-Cut.

Step 5: Apply cut indicator to all edges. The graph has:

$$E = \{(1, 3), (2, 3), (2, 4), (3, 5), (4, 5)\}$$

Each edge contributes:

$$-\frac{1}{2} + \frac{1}{2}s_i s_j$$

Since there are five edges:

$$5 \left(-\frac{1}{2} \right) = -\frac{5}{2}$$

Therefore:

$$E(s) = -\frac{5}{2} + \frac{1}{2} (s_1 s_3 + s_2 s_3 + s_2 s_4 + s_3 s_5 + s_4 s_5)$$

💡 Notice that there are **no linear spin terms**. That is not an accident: in Max-Cut, the s_i and s_j linear pieces cancel inside every edge contribution.

🔗 Deepening: What is a cut indicator?

A **cut** is simply a division of the graph's vertices into two groups. With binary variables, we encode the two groups as:

$$x_i = 0 \quad \text{or} \quad x_i = 1$$

For example, if $x_1 = 0$ and $x_3 = 1$, then vertices 1 and 3 are in different groups. If there is an edge $(1, 3)$, that edge is therefore **cut**.

The expression:

$$x_i + x_j - 2x_i x_j$$

Is called a **cut indicator** because it tells us whether the edge (i, j) crosses the cut:

x_i	x_j	$x_i + x_j - 2x_i x_j$
0	0	0
0	1	1
1	0	1
1	1	0

So it returns 1 if the edge is cut, and 0 if it is not cut. That is why we found this expression in the first place: Max-Cut wants to count how many edges connect vertices belonging to different groups. If the graph has edges E , then:

$$\sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j)$$

Literally counts the number of cut edges. But QUBO is written as a **minimization** problem, whereas Max-Cut wants to maximize that count. So we minimize its negative:

$$E(x) = - \sum_{(i,j) \in E} (x_i + x_j - 2x_i x_j)$$

Therefore a cut edge contributes -1 to the energy, while a non-cut edge contributes 0. So the more edges we cut, the lower the energy.

Now, we want to convert the cut indicator into spin variables. Thus:

$$x_i + x_j - 2x_i x_j = \frac{1}{2} - \frac{1}{2} s_i s_j$$

This means that **every edge contributes two pieces** to the spin energy function:

- A constant piece $-\frac{1}{2}$, which does not depend on the spin variables.
- A quadratic piece $\frac{1}{2}s_i s_j$, which does depend on the spin variables.

So the spin energy function is a sum of these two pieces over all edges:

$$E(s) = \sum_{(i,j) \in E} \left(-\frac{1}{2} + \frac{1}{2}s_i s_j \right)$$

In practice, we can calculate the constant piece separately:

$$\sum_{(i,j) \in E} -\frac{1}{2} = -\frac{|E|}{2} = -\frac{5}{2}$$

And the quadratic piece separately:

$$\sum_{(i,j) \in E} \frac{1}{2}s_i s_j = \frac{1}{2}(s_1 s_3 + s_2 s_3 + s_2 s_4 + s_3 s_5 + s_4 s_5)$$

Where each term corresponds to one edge in the graph, for example $s_1 s_3$ corresponds to edge $(1, 3)$. Finally, we combine the two pieces to get the complete spin energy function:

$$E(s) = -\frac{5}{2} + \frac{1}{2}(s_1 s_3 + s_2 s_3 + s_2 s_4 + s_3 s_5 + s_4 s_5)$$

There is also a very nice physical interpretation. Since $s_i, s_j \in \{-1, +1\}$:

- If the vertices are on the **same side** $s_i = s_j$, then:

$$s_i s_j = +1$$

And looking at the cut indicator of one edge:

$$-\frac{1}{2} + \frac{1}{2}s_i s_j = -\frac{1}{2} + \frac{1}{2}(+1) = 0$$

So it is **not cut**, and contributes 0 to the energy.

- If they are on **different sides** $s_i = -s_j$, then:

$$s_i s_j = -1$$

And looking at the cut indicator of one edge:

$$-\frac{1}{2} + \frac{1}{2}s_i s_j = -\frac{1}{2} + \frac{1}{2}(-1) = -1$$

So it is **cut**, and contributes -1 to the energy.

Therefore the spin expression is doing exactly the same thing as the original binary cut indicator:

edge status	energy contribution
not cut	0
cut	-1

And that's why, if all 5 edges are cut:

$$E = -1 - 1 - 1 - 1 - 1 = -5$$

The $-\frac{5}{2}$ alone is **not** the answer of the cut size. It is just the constant half of the transformed expression. The remaining:

$$\frac{1}{2} \sum_{(i,j) \in E} s_i s_j$$

Moves the energy up or down depending on whether each edge is cut or not.

Step 6: Spin coefficient matrix. The spin energy function is:

$$E(s) = -\frac{5}{2} + \frac{1}{2} (s_1 s_3 + s_2 s_3 + s_2 s_4 + s_3 s_5 + s_4 s_5)$$

Thus, the spin coefficient matrix is:

$$S = \begin{pmatrix} 0 & 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{2} \\ 0 & 0 & 0 & 0 & \frac{1}{2} \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Deepening: How to find the spin coefficient matrix

In general, the spin energy function is:

$$E(s) = c + \sum_i S_i i s_i + \sum_{i < j} S_{ij} s_i s_j$$

But there is a very nice shortcut that we can call “**read the polynomial**” **trick**.

Once we have converted the binary objective to spins:

$$E(s) = -\frac{5}{2} + \frac{1}{2} (s_1 s_3 + s_2 s_3 + s_2 s_4 + s_3 s_5 + s_4 s_5)$$

First, ignore the constant:

$$\cancel{\frac{5}{2}} + \frac{1}{2} (s_1 s_3 + s_2 s_3 + s_2 s_4 + s_3 s_5 + s_4 s_5)$$

Then remember a simple rule:

$$\begin{aligned}s_i &\rightarrow \text{diagonal entry } S_{ii} \\ s_i s_j &\rightarrow \text{position } (i, j)\end{aligned}$$

That's it!

So, for our exercise, there are no linear terms:

$$S_{11} = S_{22} = S_{33} = S_{44} = S_{55} = 0$$

And the quadratic terms are:

$$\begin{aligned}s_1 s_3 &\rightarrow S_{13} = \frac{1}{2} \\ s_2 s_3 &\rightarrow S_{23} = \frac{1}{2} \\ s_2 s_4 &\rightarrow S_{24} = \frac{1}{2} \\ s_3 s_5 &\rightarrow S_{35} = \frac{1}{2} \\ s_4 s_5 &\rightarrow S_{45} = \frac{1}{2}\end{aligned}$$

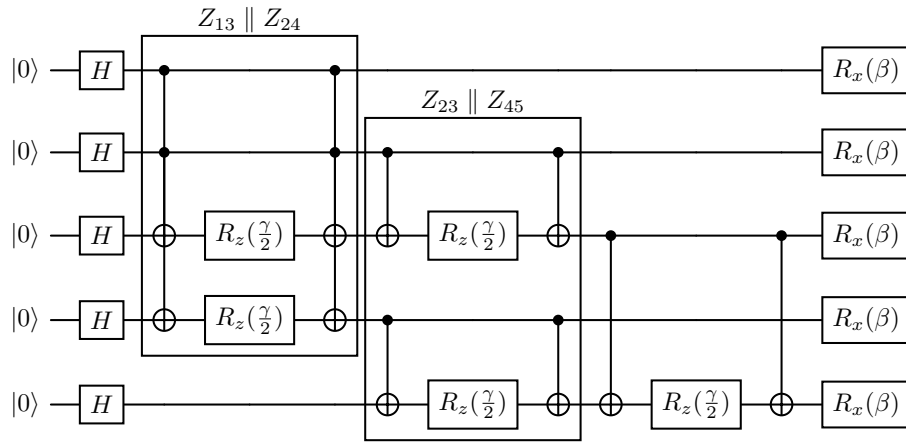
Step 7: From spin coefficient matrix to Cost Hamiltonian. To create the QAOA circuit, we need the Cost Hamiltonian. We simply replace each spin variable s_i with the corresponding Pauli-Z operator Z_i . Thus, the Cost Hamiltonian is:

$$H_{\text{problem}} = -\frac{5}{2}I + \frac{1}{2}(Z_1 Z_3 + Z_2 Z_3 + Z_2 Z_4 + Z_3 Z_5 + Z_4 Z_5)$$

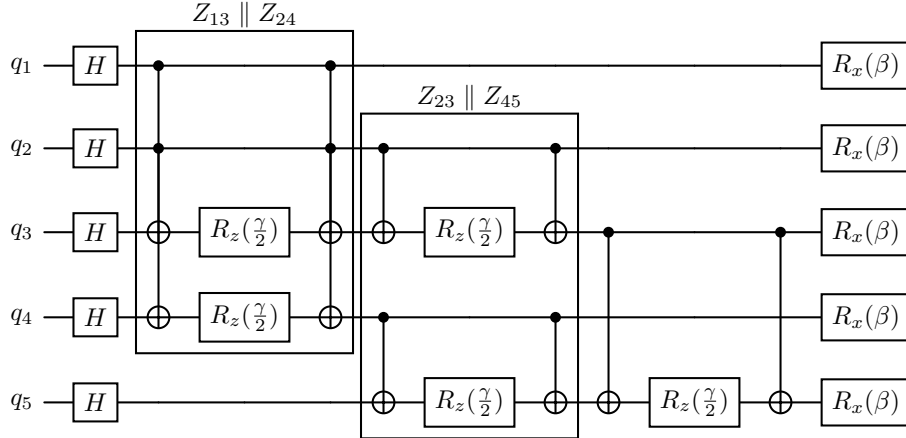
The constant term $-\frac{5}{2}I$ only shifts all energies by the same amount, so it can be omitted when constructing the QAOA cost unitary. Thus we use:

$$H_{\text{problem}} = \frac{1}{2}(Z_1 Z_3 + Z_2 Z_3 + Z_2 Z_4 + Z_3 Z_5 + Z_4 Z_5)$$

Step 8: Circuit implementation to solve Max-Cut problem. The QAOA circuit is built from the Cost Hamiltonian. The QAOA circuit $p = 1$ to solve our Max-Cut problem is:



Or **equivalently**, we can take a shortcut and write directly the Hadamard layer without the initial state preparation:



Deepening: How to implement the QAOA circuit for Max-Cut

Once we have reached the **spin energy function**, building the $p = 1$ QAOA circuit can become almost mechanical. For our exercise, the spin energy function is:

$$E(s) = -\frac{5}{2} + \frac{1}{2}(s_1 s_3 + s_2 s_3 + s_2 s_4 + s_3 s_5 + s_4 s_5)$$

The corresponding Cost Hamiltonian is:

$$H_{\text{problem}} = -\frac{5}{2}I + \frac{1}{2}(Z_1Z_3 + Z_2Z_3 + Z_2Z_4 + Z_3Z_5 + Z_4Z_5)$$

The constant term $-\frac{5}{2}I$ can be omitted, because it only shifts all energies by the same amount. So we can use:

$$H_{\text{problem}} = \frac{1}{2}(Z_1Z_3 + Z_2Z_3 + Z_2Z_4 + Z_3Z_5 + Z_4Z_5)$$

1. First, superposition layer. We begin with the initial state $|0\rangle^{\otimes n}$. In our case, $n = 5$ qubits. Then, since QAOA wants to consider all 2^n candidate binary solutions, we apply a Hadamard gate to each qubit to create an equal superposition of all $2^5 = 32$ states. Since:

$$H|0\rangle = |+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

We obtain:

$$|+\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle$$

For our exercise, this is:

$$|+\rangle^{\otimes 5} = \frac{1}{\sqrt{32}} \sum_{x \in \{0,1\}^5} |x\rangle = \frac{1}{\sqrt{32}}(|00000\rangle + |00001\rangle + \dots + |11111\rangle)$$

So the **first rule** is trivial: **one H on every qubit.**

2. Cost layer: encode the objective function. Now comes:

$$U_C(\gamma) = e^{-i\gamma H_{\text{problem}}}$$

The purpose of this layer is to make the quantum sensitive to the **cost/energy of each candidate solution**. Since H_{problem} is diagonal in the computational basis, it essentially gives different phases to solutions according to their cost. Obviously, we don't need to calculate the full exponential. Instead, look at the terms of H_{problem} :

- **Linear terms.** If:

$$H_{\text{problem}} \supset h_i Z_i$$

(in words, if there is a linear term in the spin energy function), then this produces a single-qubit Z -rotation on qubit i . Schematically:

$$h_i Z_i \Rightarrow R_z(h_i \gamma)$$

So, for example, if H_{problem} had a term $-2Z_1$, then we would apply a $R_z(-2\gamma)$ on qubit 1.

- **Quadratic terms.** If:

$$H_{\text{problem}} \supset J_{ij} Z_i Z_j$$

(in words, if there is a quadratic term in the spin energy function), then the cost unitary contains:

$$e^{-i\gamma J_{ij} Z_i Z_j}$$

As we saw in the theory section, we can implement this with a CNOT “sandwich” around a R_z rotation. Schematically:

$$\text{CNOT}_{ij} \rightarrow R_z(J_{ij}\gamma) \rightarrow \text{CNOT}_{ij}$$

For example, if H_{problem} had a term $\frac{1}{2} Z_1 Z_3$, then we would apply a CNOT with control qubit 1 and target qubit 3, then a $R_z(\frac{\gamma}{2})$ on qubit 3, and finally another CNOT with control qubit 1 and target qubit 3.

💡 To speedup this process, we can read the Cost Hamiltonian:

$$H_{\text{problem}} = \frac{1}{2} (Z_1 Z_3 + Z_2 Z_3 + Z_2 Z_4 + Z_3 Z_5 + Z_4 Z_5)$$

From this, we can see immediately:

- No linear terms, so no single-qubit R_z rotations.
- Five quadratic terms, so five CNOT- R_z -CNOT sandwiches.
- Each sandwich has a $R_z(\frac{\gamma}{2})$ rotation, because the coefficient of all quadratic terms is $\frac{1}{2}$.
- The control-target pairs are:

$$Z_1 Z_3 \rightarrow \text{CNOT}_{1,3} \rightarrow R_z\left(\frac{\gamma}{2}\right) \rightarrow \text{CNOT}_{1,3}$$

$$Z_2 Z_3 \rightarrow \text{CNOT}_{2,3} \rightarrow R_z\left(\frac{\gamma}{2}\right) \rightarrow \text{CNOT}_{2,3}$$

$$Z_2 Z_4 \rightarrow \text{CNOT}_{2,4} \rightarrow R_z\left(\frac{\gamma}{2}\right) \rightarrow \text{CNOT}_{2,4}$$

$$Z_3 Z_5 \rightarrow \text{CNOT}_{3,5} \rightarrow R_z\left(\frac{\gamma}{2}\right) \rightarrow \text{CNOT}_{3,5}$$

$$Z_4 Z_5 \rightarrow \text{CNOT}_{4,5} \rightarrow R_z\left(\frac{\gamma}{2}\right) \rightarrow \text{CNOT}_{4,5}$$

So the Hamiltonian already tells us the circuit.

3. Mixing layer: R_x rotations. After the cost layer comes:

$$U_M(\beta) = e^{-i\beta H_M}$$

Where the mixing Hamiltonian is:

$$H_M = \sum_{i=1}^n X_i$$

Because the X_i 's act on different qubits, this decomposes into independent X -rotations: $R_x(\beta)$ on **every qubit**. So the **third rule** is trivial: **one $R_x(\beta)$ on every qubit**. In our case, $n = 5$ qubits, therefore:

$$U_M(\beta) = R_x(\beta)^{\otimes 5}$$

The mixer is important because the cost layer by itself only changes **phases**. It doesn't directly make good solutions more probable. The mixer allows those phase to interfere and thereby changes the measurement probabilities.

❓ Can we parallelize the operations? Yes, in general QAOA cost gates can be often be parallelized, but not arbitrarily. However, actual gates cannot generally execute simultaneously if they require the same qubit. For example:

$$Z_1 Z_3 \quad \text{and} \quad Z_2 Z_3$$

Both require qubit 3, so they cannot be executed simultaneously. But:

$$Z_1 Z_3 \quad \text{and} \quad Z_2 Z_4$$

Can be executed simultaneously, because they require different qubits. So the circuit can be parallelized.

References

- [1] Bryan Eastin and Emanuel Knill. Restrictions on transversal encoded quantum gate sets. *Physical review letters*, 102(11):110502, 2009.
- [2] Albert Einstein, Boris Podolsky, and Nathan Rosen. Can quantum-mechanical description of physical reality be considered complete? *Physical review*, 47(10):777, 1935.
- [3] Cremonesi Paolo. Quantum computing. Slides from the HPC-E master's degree course on Politecnico di Milano, 2024.

Index

A

Adiabatic Theorem	453
Amplitude Damping	374
Ancilla Qubit	231, 388, 389
Ansatz	472
Ansätze	472
As Late As Possible (ALAP) Scheduling	398
As Soon As Possible (ASAP) Scheduling	398
Average Fidelity	380
Azimuthal Angle	39

B

Balanced Function	226
Barren Plateau	496
Basis	19
Bell States	118
Bernstein-Vazirani's Algorithm	248
Binary Expansion	417
Birthday Problem	286
Bit-Flip Error	375
Black Box	225
Bloch Sphere Representation of a Qubit	39
Blow-Up Problem	390
Born rule	83
Bra	12
Bra-Ket notation	78
Brownian Motion	385

C

Classical Correlation	347
Classical Optimizer	480
Clifford Gate	154
Coherent Quantum State	368
Common Factor	181
Complete Orthonormal Basis	138
Completeness Relation	138
Computation Qubit	231
Constant Function	225
Constructive Interference	371
Controlled Hadamard Gate	223
Controlled NOT (CNOT) Gate	103
Controlled Phase Gate	220
Controlled Unitary Operator	203
Controlled-Controlled-NOT (CCNOT) Gate	112
Cost Evolution Operator	518
Cost Function	477
Cost Hamiltonian	518, 529
Cost Operator	518

Cost Unitary	518
D	
Data Dependency	397
Data Qubit	230
Decoherence	365
Decoherence Time	382
Dephasing	368
Destructive Interference	371
Deutsch's Algorithm	226, 243
E	
Eastin-Knill theorem	171
Eigenspectrum	437
Eigenstate	53
Eigenstates of the Pauli-X operator	54
Einstein's Relativity Principle	342
Einstein-Podolsky-Rosen (EPR) Paradox	345
Energy Relaxation	374
Entangled	117
Entanglement	91
Entangling Gates	120
Error Operator	376
Excitation	374
Excited States	437
Expectation Value	142
F	
Fidelity	378, 383
Fourier transform over $\mathbb{Z}_2^n$	325
G	
Galois Field	316
Gate Error Probability	382
Gate Infidelity	365, 367
Gate Time	382
Generic Controlled Gate (CU)	106
Global Phase	181
Ground State	410
Ground-State Energy	437
H	
Hadamard basis	65
Hadamard basis states	54
Hamiltonian	432
Hardware Efficient Ansatz (HEA)	483
Heisenberg Representation	151
Hidden Variables	347
I	
Inner Product of Kets	78

Interference	371
Ising Energy Function	446
Ising Model	446
Ising model	459
K	
Ket	12
Kronecker delta	139
L	
Logical Gate	391
Logical Gate sets	173
Logical Qubit	168, 387
Loss of Phase	368
M	
Manhattan distance	395
Marginal Probability	180
Measurement	15
Measurement Operator	136
Minus State	54
Mixer Hamiltonian	511, 521
Mixer Operator	522
N	
NISQ (Noisy Intermediate-Scale Quantum)	467
NISQ (Noisy Intermediate-Scale Quantum) device	467
No-Cloning Theorem	34, 47, 333
No-Deleting Theorem	47, 338
No-Signaling Principle	341
Normalization Factor	181
Normalized quantum state	19
O	
Observable	126
Optimizer	474
Oracle	225, 230
Orthonormal Basis	19
Outer Product of Kets	78
P	
Pauli Gates	50
Pauli matrices	51
Penalty Term	420
Phase Gate	220
Phase Kickback	206
Phase-Flip Error	372
Photon	15
Physical Gate sets	173
Physical Qubit	168, 387
Placement	395

Plus State	53
Polarization	15
Polarizer	17
Probabilities from the Expectation Value	148
Problem-Agnostic Ansatz	478
Problem-Specific Ansatz	478
Projection Operator	80
Projector	80
Q	
Quadratic Unconstrained Binary Optimization (QUBO)	462
Quadratic Unconstrained Binary Optimization (QUBO) model	406
Quality Ratio	383
Quantum Algorithm	393
Quantum Annealing	453, 460
Quantum Approximate Optimization Algorithm (QAOA)	514
Quantum Approximate Optimization Algorithm (QAOA)	509
Quantum Circuit	76
Quantum Compilation	393
Quantum Correlation	347, 351
Quantum Error Correction Code (QECC)	390
Quantum Execution	394
Quantum Gates	46
Quantum Oracle	230
Quantum Processor	393
Quantum Programming Language	393
Quantum System	14
Qubit	18
Query	228
Query Complexity	228
Query Complexity Problem	225
R	
Rayleigh-Ritz Variational Principle	503
Relative Phase	38, 182
Relaxation	374
Routing	405
S	
Schrödinger Representation	151
Simon's Promise	281
Single Qubit Gates: Hadamard Gate (S)	65
Single Qubit Gates: Identity Gate	49
Single Qubit Gates: Initialization	45
Single Qubit Gates: Logic Gates	42
Single Qubit Gates: Measurement	43
Single Qubit Gates: Pauli-X Gate	51
Single Qubit Gates: Pauli-Y Gate	60
Single Qubit Gates: Pauli-Z Gate	56

Single Qubit Gates: Phase Gate (S)	63
Single Qubit Gates: T Gate	158
Single-Qubit Gates	46
Spectral Gap	458
Spectral Theorem	132
Spin	446
Stabilizer	161
Stabilizer Group	163
Stabilizer State	161
State Space	36
State Teleportation	352
Superposition	27
SWAP gate	109
Syndrome	388
Syndrome Measurement	388
T	
Target Qubit	231
Tensor Product	88
Thermal Excitation	374
Threshold Theorem	384
Time-Independent Schrödinger Equation	500
Toffoli Gate	112
Transpiling	393
Transversal Gate	169
Transverse Field	454
Transverse-Field Ising Model (TFIM)	453
U	
Unitary Gate	43
Unitary Matrix	42
Universal	164
Universal Set of Quantum Gates	164
V	
Variational Principle	503
Variational Quantum Algorithm (VQA)	470
Variational Quantum Algorithms (VQAs)	470
VQE Objective Function	505
W	
Walsh-Hadamard transform	325
When is a state entangled?	94
When is a state separable?	94
X	
X-basis states	54